

The Pencil Code:

A High-Order MPI code for MHD Turbulence

User's and Reference Manual



July 23, 2026

<https://pencil-code.org>

<https://github.com/pencil-code/pencil-code>

The PENCIL CODE: multi-purpose and multi-user maintained

<http://www.nordita.org/~brandenb/talks/misc/PencilCode04.htm>

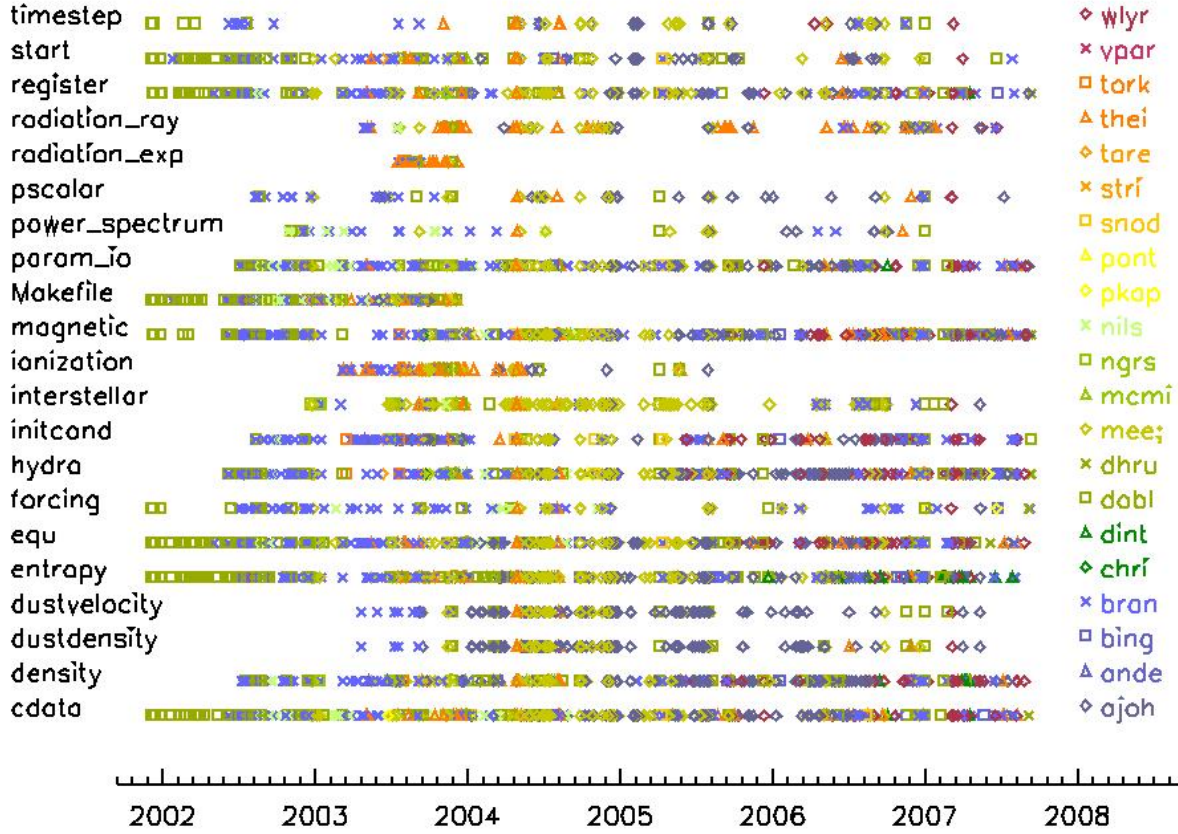


Figure 1: Check-in patterns as a function of time for different subroutines. The different users are marked by different symbols and different colors.

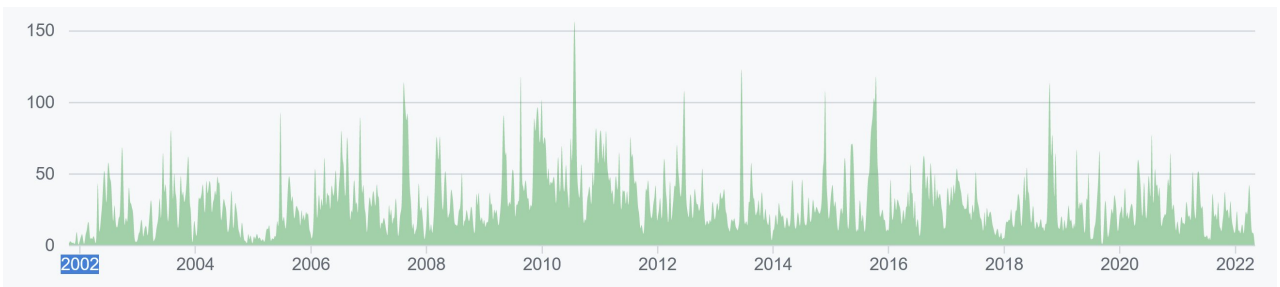


Figure 2: GitHub check-in pattern since 2002; see <https://github.com/pencil-code/pencil-code/graphs/contributors>

Contributors to the code

(in inverse alphabetical order according to their user name)

An up to date list of Pencil Code contributors can be found at [GitHub](https://github.com/pencil-code).

xiang-yu	Xiang-Yu Li	Pacific Northwest National Laboratory
wladimir.lyra	Wladimir Lyra	New Mexico State Univ.
weezy	S. Louise Wilkin	University of Newcastle
wdobler	Wolfgang Dobler	Bruker, Potsdam
vpariev	Vladimir Pariev	University of Rochester
torkel	Ulf Torkelsson	Chalmers University
tavo-buk	Gustavo Guerrero	Univ. Minas Gerais
tgastine	Thomas Gastine	MPI for Solar System Research
tobson, theine	Tobias (Tobi) Heinemann	NBIA, Copenhagen
tarek	Tarek A. Yousef	University of Trondheim
sven.bingert	Sven Bingert	Ges. f. wiss. Datenverarb.
steveb	Steve Berukoff	UCLA
asnodin	Andrew Snodin	University of Newcastle
rplasson	Raphael Plasson	Avignon Université
qiancg	Chengeng Qian	Beijing Inst. of Technology
pkapyla	Petri Käpylä	University of Göttingen
onlymee	Antony (tOnY) Mee	Bank of Am. Merrill Lynch, London
nishkpph	Nishant K. Singh	IUCAA
nils.e.haugen	Nils Erland L. Haugen	SINTEF, Trondheim
NBabkovskaia	Natalia Babkovskaia	University of Helsinki
mrheinhardt	Matthias Rheinhardt	Aalto University, Espoo
mppyali	Piyali Chatterjee	Bangalore
mkorpi	Maarit J. Korpi-Lagg (née Korpi, Mantere, Käpylä)	Aalto University
miikkavaisala	Miikka Väisälä	Academia Sinica, Inst. Astron. & Astro
michiellambrechts	Michiel Lambrechts	Lund Observatory
Mewek	Ewa Karchniwy	Silesian University of Techn.
mcmillan	David McMillan	York University, Toronto
mattias	Mattias Christensson	formerly at Nordita
luizfelippesr	Luiz Felipe S. Rodrigues	Radboud University
koenkemel	Koen Kemel	Nordita, Stockholm
karlsson	Torgny Karlsson	Nordita
jwarne	Jörn Warnecke	MPS, Göttingen
jpekkila	Johannes Pekkila	Aalto University
jnskrueger	Jonas Krueger	Trondheim
jaarnes	Jørgen Aarnes	Trondheim
joishi	Jeff S. Oishi	Bates College
JenSchober	Jennifer Schober	EPFL, Lausanne
Illarl	Illa R. Losada	McDonald Observatory, USA
Iomsn1	Simon Candelaresi	University of Augsburg
grsarson	Graeme R. Sarson	University of Newcastle
fredgent	Frederick Gent	Aalto University, Espoo
fadiesis	Fabio Del Sordo	Nordita, Stockholm
dorch	Bertil Dorch	University of Copenhagen
bdintrans	Boris Dintrans	Observatoire Midi-Pyrénées, Toulouse
dhrubaditya	Dhrubaditya Mitra	Nordita, Stockholm
colinmcnally	Colin McNally	NBIA, Copenhagen
ChristerSandin	Christer Sandin	Nordita
chaochinyang	Chao-Chin Yang	The University of Alabama
Bourdin.KIS	Philippe Bourdin	Space Res. Inst., Graz
AxelBrandenburg	Axel Brandenburg	Nordita, Stockholm
apichat	Apichat Neamvonk	University of Newcastle
amjed	Amjed Mohammed	University of Oldenburg
alihyder727	Ali Hyder	New Mexico State Univ.
alexanderhubbard	Alex Hubbard	Am. Museum Nat. History
andreas-schreiber	Andreas Schreiber	MPI Heidelberg
ajohan	Anders Johansen	GLOBE Institute, Copenhagen Univers

Copyright © 2001–2025 Wolfgang Dobler & Axel Brandenburg

Permission is granted to make and distribute verbatim copies of this manual provided the copyright notice and this permission notice are preserved on all copies.

Permission is granted to copy and distribute modified versions of this manual under the conditions for verbatim copying, provided that the entire resulting derived work is distributed under the terms of a permission notice identical to this one.

License agreement and giving credit

The content of all files under `:pserver:$USER@svn.nordita.org:/var/cvs/brandenb` are under the GNU General Public License (<http://www.gnu.org/licenses/gpl.html>).

We, the PENCIL CODE community, ask that in publications and presentations the use of the code (or parts of it) be acknowledged with reference to [42] “Pencil Code Collaboration, *J. Open Source Software*, **6**, 2807 (2021) The Pencil Code, a modular MPI code for partial differential equations and particles: multipurpose and multiuser-maintained.” This automatically gives a reference to the web sites <http://www.nordita.org/software/pencil-code/> and <https://github.com/pencil-code/pencil-code>. As a courtesy to the people involved in developing particularly important parts of the program (use `svn annotate src/*.f90` to find out who did what!) we suggest to give appropriate reference to one or several of the following (or other appropriate) papers (listed here in temporal order):

- Dobler, W., Haugen, N. E. L., Yousef, T. A., & Brandenburg, A.: 2003, “Bottleneck effect in three-dimensional turbulence simulations,” *Phys. Rev. E* **68**, 026304, 1-8 (astro-ph/0303324)
- Haugen, N. E. L., Brandenburg, A., & Dobler, W.: 2003, “Is nonhelical hydromagnetic turbulence peaked at small scales?” *Astrophys. J. Lett.* **597**, L141-L144 (astro-ph/0303372)
- Brandenburg, A., Käpylä, P., & Mohammed, A.: 2004, “Non-Fickian diffusion and tau-approximation from numerical turbulence,” *Phys. Fluids* **16**, 1020-1027 (astro-ph/0306521)
- Johansen, A., Andersen, A. C., & Brandenburg, A.: 2004, “Simulations of dust-trapping vortices in protoplanetary discs,” *Astron. Astrophys.* **417**, 361-371 (astro-ph/0310059)
- Haugen, N. E. L., Brandenburg, A., & Mee, A. J.: 2004, “Mach number dependence of the onset of dynamo action,” *Monthly Notices Roy. Astron. Soc.* **353**, 947-952 (astro-ph/0405453)
- Brandenburg, A., & Multamäki, T.: 2004, “How long can left and right handed life forms coexist?” *Int. J. Astrobiol.* **3**, 209-219 (q-bio/0407008)
- McMillan, D. G., & Sarson, G. R.: 2005, “Dynamo simulations in a spherical shell of ideal gas using a high-order Cartesian magnetohydrodynamics code,” *Phys. Earth Planet. Int.* **153**, 124-135
- Heinemann, T., Dobler, W., Nordlund, Å., & Brandenburg, A.: 2006, “Radiative transfer in decomposed domains,” *Astron. Astrophys.* **448**, 731-737 (astro-ph/0503510)
- Dobler, W., Stix, M., & Brandenburg, A.: 2006, “Convection and magnetic field generation in fully convective spheres,” *Astrophys. J.* **638**, 336-347 (astro-ph/0410645)
- Snodin, A. P., Brandenburg, A., Mee, A. J., & Shukurov, A.: 2006, “Simulating field-aligned diffusion of a cosmic ray gas,” *Monthly Notices Roy. Astron. Soc.* **373**, 643-652 (astro-ph/0507176)
- Johansen, A., Klahr, H., & Henning, T.: 2006, “Dust sedimentation and self-sustained Kelvin-Helmholtz turbulence in protoplanetary disc mid-planes,” *Astrophys. J.* **636**, 1121-1134 (astro-ph/0512272)
- de Val-Borro, M. and 22 coauthors (incl. Lyra, W.): 2006, “A comparative study of disc-planet interaction,” *Monthly Notices Roy. Astron. Soc.* **370**, 529-558 (astro-ph/0605237)
- Johansen, A., Oishi, J. S., Mac Low, M. M., Klahr, H., Henning, T., & Youdin, A.: 2007, “Rapid planetesimal formation in turbulent circumstellar disks,” *Nature* **448**,

1022–1025 (arXiv/0708.3890)

- Lyra, W., Johansen, A., Klahr, H., & Piskunov, N.: 2008, “Global magnetohydrodynamical models of turbulence in protoplanetary disks I. A cylindrical potential on a Cartesian grid and transport of solids,” *Astron. Astrophys.* **479**, 883-901 (arXiv/0705.4090)
- Brandenburg, A., Rädler, K.-H., Rheinhardt, M., & Käpylä, P. J.: 2008, “Magnetic diffusivity tensor and dynamo effects in rotating and shearing turbulence,” *Astrophys. J.* **676**, 740-751 (arXiv/0710.4059)
- Lyra, W., Johansen, A., Klahr, H., & Piskunov, N.: 2008, “Embryos grown in the dead zone. Assembling the first protoplanetary cores in low-mass selfgravitating circumstellar disks of gas and solids,” *Astron. Astrophys.* **491**, L41-L44
- Lyra, W., Johansen, A., Klahr, H., & Piskunov, N.: 2009, “Standing on the shoulders of giants. Trojan Earths and vortex trapping in low-mass selfgravitating protoplanetary disks of gas and solids,” *Astron. Astrophys.* **493**, 1125-1139
- Lyra, W., Johansen, A., Zsom, A., Klahr, H., & Piskunov, N.: 2009, “Planet formation bursts at the borders of the dead zone in 2D numerical simulations of circumstellar disks,” *Astron. Astrophys.* **497**, 869-888 (arXiv/0901.1638)
- Mitra, D., Tavakol, R., Brandenburg, A., & Moss, D.: 2009, “Turbulent dynamos in spherical shell segments of varying geometrical extent,” *Astrophys. J.* **697**, 923-933 (arXiv/0812.3106)
- Haugen, N. E. L., & Kragset, S.: 2010, “Particle impaction on a cylinder in a crossflow as function of Stokes and Reynolds numbers,” *J. Fluid Mech.* **661**, 239-261
- Rheinhardt, M., & Brandenburg, A.: 2010, “Test-field method for mean-field coefficients with MHD background,” *Astron. Astrophys.* **520**, A28 (arXiv/1004.0689)
- Babkovskaia, N., Haugen, N. E. L., Brandenburg, A.: 2011, “A high-order public domain code for direct numerical simulations of turbulent combustion,” *J. Comp. Phys.* **230**, 1-12 (arXiv/1005.5301)
- Johansen, A., Klahr, H., & Henning, Th.: 2011, “High-resolution simulations of planetesimal formation in turbulent protoplanetary discs,” *Astron. Astrophys.* **529**, A62
- Johansen, A., Youdin, A. N., & Lithwick, Y.: 2012, “Adding particle collisions to the formation of asteroids and Kuiper belt objects via streaming instabilities,” *Astron. Astrophys.* **537**, A125
- Lyra, W. & Kuchner, W. : 2013, “Formation of sharp eccentric rings in debris disks with gas but without planets,” *Nature* **499**, 184–187
- Yang, C.-C., & Johansen, A.: 2016, “Integration of Particle-Gas Systems with Stiff Mutual Drag Interaction,” *Astrophys. J. Suppl. Series* **224**, 39
- Roper Pol, A., Brandenburg, A. Kahniashvili, T., Kosowsky, A., & Mandal, S.: 2020, “The timestep constraint in solving the gravitational wave equations sourced by hydro-magnetic turbulence,” *Geophys. Astrophys. Fluid Dyn.* **114**, 130-161

This list is not always up-to-date. We therefore ask the developers to check in new relevant papers, avoiding however redundancies.

We are aware of the fact that certain extensions to the code may still be under intense development and no paper can be quoted yet. Again, if your work directly profits from such code, as a courtesy to those developers, we suggest to contact them, if possible, and ask whether there is anything else that can be quoted instead.

It is also sometimes nice to see that the PENCIL CODE is being acknowledged for having *inspired* certain other developments, so for example in the GALPROP program [43].

Foreword

This code was originally developed at the Turbulence Summer School of the Helmholtz Institute in Potsdam (2001). While some SPH and PPM codes for hydrodynamics and magnetohydrodynamics were publicly available, this did not seem to be generally the case for higher order finite-difference or spectral codes. This has changed since 2001; examples are the SpECTRE code, which is a discontinuous Galerkin code, and there are also the Snoopy and Dedalus codes, which are spectral. Having been approached by people interested in using our code, we decided to make it as flexible as possible and as user-friendly as seems reasonable, and to put it onto a public CVS repository. Since 21 September 2008 it is distributed via <https://github.com/pencil-code/pencil-code>. The code can certainly not be treated as a black box (no code can), and in order to solve a new problem in an optimal way, users will need to find their own optimal set of parameters. In particular, you need to be careful in choosing the right values of viscosity, magnetic diffusivity, and radiative conductivity.

The PENCIL CODE is primarily designed to deal with weakly compressible turbulent flows, which is why we use high-order first and second derivatives. To achieve good parallelization, we use explicit (as opposed to compact) finite differences. Typical scientific targets include driven MHD turbulence in a periodic box, convection in a slab with non-periodic upper and lower boundaries, a convective star embedded in a fully nonperiodic box, accretion disc turbulence in the shearing sheet approximation, etc. Furthermore, nonlocal radiation transport, inertial particles, dust coagulation, self-gravity, chemical reaction networks, and several other physical components are installed, but this number increases steadily. In addition to Cartesian coordinates, the code can also deal with spherical and cylindrical polar coordinates.

Magnetic fields are implemented in terms of the magnetic vector potential to ensure that the field remains solenoidal (divergence-free). At the same time, having the magnetic vector potential readily available is a big advantage if one wants to monitor the magnetic helicity, for example. The code is therefore particularly well suited for all kinds of dynamo problems.

The code is normally non-conservative; thus, conserved quantities should only be conserved up to the discretization error of the scheme (not to machine accuracy). There is no guarantee that a conservative code is more accurate with respect to quantities that are not explicitly conserved, such as entropy. Another important quantity that is (to our knowledge) not strictly conserved by ordinary flux conserving schemes is *magnetic helicity*.

There are currently no plans to implement adaptive mesh refinement into the code, which would cause major technical complications. Given that turbulence is generically space-filling, local refinement to smaller scales would often not be very useful anyway. On the other hand, in some geometries turbulence may well be confined to certain regions in space, so one could indeed gain by solving the outer regions with fewer points.

In order to be cache-efficient, we solve the equations along *pencils* in the x direction. One very convenient side-effect is that auxiliary and derived variables use very little memory, as they are only ever defined on one pencil. The domain can be tiled in the y and z directions. On multiprocessor computers, the code can use *MPI* (Message Passing Interface) calls to communicate between processors. An easy switching mechanism allows the user to run the code on a machine without MPI libraries (e.g., a notebook computer). Ghost zones are used to implement boundary conditions on physical and

processor boundaries.

A high level of flexibility is achieved by encapsulating individual physical processes and variables in individual *modules*, which can be switched on or off in the file ‘Makefile.local’ in the local ‘src’ directory. This approach avoids the use of difficult-to-read preprocessor directives, at the price of requiring one dummy module for each physics module. For nonmagnetic hydrodynamics, for example, one will use the module ‘nomagnetic.f90’ and specifies

```
MAGNETIC = nomagnetic
```

in ‘Makefile.local’, while for MHD simulations, ‘magnetic.f90’ will be used:

```
MAGNETIC = magnetic
```

Note that the term *module* as used here is only loosely related to Fortran modules: both ‘magnetic.f90’ and ‘nomagnetic.f90’ define an F90 module named *Magnetic* — this is the basis of the switching mechanism we are using.

Input parameters (which are set in the files ‘start.in’, ‘run.in’) can be changed without recompilation. Furthermore, one can change the list of variables for monitoring (diagnostic) output on the fly, and there are mechanisms for making the code reload new parameters or exit gracefully at runtime. You may want to check for correctness of these files with the command `pc_configtest`.

The requirements for using the Pencil-MPI code are modest: you can use it on any Linux or Unix system with a *F95* and *C* compiler suite, like *GNU gcc* and *gfortran*, together with the shell *CSH*, and the *Perl* interpreter are mandatory requirements.

Although the PENCIL CODE is mainly designed to run on supercomputers, more than 50% of the users run their code also on Macs, and the other half uses either directly Linux on their laptops or they use VirtualBox on their Windows machine on which they install Ubuntu Linux. If you have *IDL* as well, you will be able to visualize the results (a number of sample procedures are provided), but other tools such as *Python*, *DX* (OpenDX, data explorer) can also be used and some relevant tools and routines come with the PENCIL CODE.

If you want to make creative use of the code, this manual will contain far too little information. Its major aim is to give you an idea of the way the code is organized, so you can more efficiently *read the source code*, which contains a reasonable amount of comments. You might want to read through the various sample directories that are checked in. Choose one that is closest to your application and start modifying. For further enhancements that you may want to add to the code, you can take as an example the lines in the code that deal with related variables, functions, diagnostics, equations etc., which have already been implemented. Just remember: `grep` is one of your best friends when you want to understand how certain variables or functions are used in the code.

We will be happy to include user-supplied changes and updates to the code in future releases and welcome any feedback.

wdobler@gmail.com
AxelBrandenburg@gmail.com

Potsdam
Stockholm

Acknowledgments

Many people have contributed in different ways to the development of this code. We thank first of all Åke Nordlund (Copenhagen Observatory) and Bob Stein (University of Michigan) who introduced us to the idea of using high-order schemes in compressible flows and who taught us a lot about simulations in general.

The calculation of the power spectra, structure functions, the remeshing procedures, routines for changing the number of processors, as well as the shearing sheet approximation and the flux-limited diffusion approximation for radiative transfer were implemented by Nils Erland L. Haugen (University of Trondheim). Tobi Heinemann added the long characteristics method for radiative transfer as well as hydrogen ionization. He also added and/or improved shock diffusion for other variables and improved the resulting timestep control. Anders Johansen, Wladimir (Wlad) Lyra, and Jeff Oishi contributed to the implementation of the dust equations (which now comprises an array of different components). Antony (Tony) Mee (University of Newcastle) implemented shock viscosity and added the interstellar module together with Graeme R. Sarson (also University of Newcastle), who also implemented the geodynamo set-up together with David McMillan (currently also at the University of Newcastle). Tony also included a method for outputting auxiliary variables and enhanced the overall functionality of the code and related `idl` and `dx` procedures. He also added, together with Andrew Snodin, the evolution equations for the cosmic ray energy density. Vladimir Pariev (University of Rochester) contributed to the development and testing of the potential field boundary condition at an early stage. The implementation of spherical and cylindrical coordinates is due to Dhruvaditya (Dhruva) Mitra and Wladimir Lyra. Wlad also implemented the global set-up for protoplanetary disks (as opposed to the local shearing sheet formalism). He also added a N -body code (based on the particle module coded by Anders Johansen and Tony), and implemented the coupled evolution equations of neutrals and ions for two-fluid models of ambipolar diffusion. Boris Dintrans is in charge of implementing the anelastic and Boussinesq modules. Philippe-A. Bourdin implemented HDF5 support and wrote the optional IO-modules for high-performance computing featuring various communication strategies. He also contributed to the solar-corona module and worked on the IDL GUI, including the IDL routines for reading and working with large amounts of data. Again, this list contains other recent items that are not yet fully documented and acknowledged.

Use of the PPARC supported supercomputers in St Andrews (Mhd) and Leicester (Ukaff) is acknowledged. We also acknowledge the Danish Center for Scientific Computing for granting time on Horseshoe, which is a 512+140 processor Beowulf cluster in Odense (Horseshoe).

Contents

I	Using the PENCIL CODE	1
1	System requirements	1
2	Obtaining the code	2
2.1	Obtaining the code via git or svn	2
2.2	Updating via svn or git	2
2.3	Getting the last validated version	3
2.4	Getting older versions	3
3	Getting started	4
3.1	Setup	4
3.1.1	Environment settings	4
3.1.2	Linking scripts and source files	5
3.1.3	Adapting 'Makefile.src'	6
3.1.4	Running make	6
3.1.5	Choosing a data directory	6
3.1.6	Running the code	7
3.2	Further tests	9
4	Code structure	10
4.1	Directory tree	10
4.2	Basic concepts	11
4.2.1	Data access in pencils	11
4.2.2	Modularity	12
4.3	Files in the run directories	12
4.3.1	'start.in', 'run.in', 'print.in'	12
4.3.2	'datadir.in'	13
4.3.3	'sn_series.in'	13
4.3.4	'reference.out'	13
4.3.5	'start.csh', 'run.csh', 'getconf.csh' [obsolete; see Sect. 5.1]	13
4.3.6	'src/ '	13
4.3.7	'data/ '	13
5	Using the code	16
5.1	Configuring the code to compile and run on your computer	16
5.1.1	Locating the configuration file	16
5.1.2	Structure of configuration files	17
5.1.3	Compiling the code	19
5.1.4	Running the code	19
5.1.5	Testing the code	19
5.2	Adapting 'Makefile.src' [obsolete; see Sect. 5.1]	20
5.3	Changing the resolution	21
5.4	GPUs	21
5.4.1	Runtime compilation	23
5.4.2	Performance tips	23
5.5	Using a non-equidistant grid	23
5.6	Diagnostic output	26

5.7	Data files	26
5.7.1	Snapshot files	26
5.8	Video files and slices	27
5.9	Averages	30
5.9.1	One-dimensional output averaged in two dimensions	30
5.9.2	Two-dimensional output averaged in one dimension	30
5.9.3	Azimuthal averages	30
5.9.4	Time averages	31
5.10	Helper scripts	32
5.11	RELOAD, STOP and SAVE files	34
5.12	RERUN and NEWDIR files	35
5.13	Start and run parameters	35
5.14	Physical units	37
5.15	Minimum amount of viscosity	38
5.16	The time step	38
5.16.1	The usual RK-2N time step	38
5.16.2	The Runge-Kutta-Fehlberg time step	39
5.17	Boundary conditions	40
5.17.1	Where to specify boundary conditions	40
5.17.2	How to specify boundary conditions	40
5.18	Restarting a simulation	41
5.19	One- and two-dimensional runs	41
5.20	Visualization	42
5.20.1	Gnuplot	42
5.20.2	Data explorer	42
5.20.3	GDL	42
5.20.4	IDL	44
5.20.5	Python	47
5.21	Running on multi-processor computers	50
5.21.1	How to run a sample problem in parallel	50
5.21.2	Hierarchical networks (e.g., on Beowulf clusters)	51
5.21.3	Extra workload caused by the ghost zones	51
5.22	Running in double-precision	52
5.23	Power spectrum	53
5.24	Other power spectra	54
5.25	Structure functions	55
5.26	Particles	56
5.26.1	Particles in parallel	57
5.26.2	Large number of particles	59
5.26.3	Random number generator	60
5.27	Non-cartesian coordinate systems	60
6	The equations	62
6.1	Continuity equation	62
6.2	Equation of motion	62
6.3	Induction equation	63
6.4	Entropy equation	63
6.4.1	Viscous heating	64
6.4.2	Alternative description	64
6.5	Transport equation for a passive scalar	65

6.6	Bulk viscosity	65
6.6.1	Shock viscosity	65
6.7	Equation of state	66
6.8	Ionization	66
6.8.1	Ambipolar diffusion	68
6.9	Combustion	68
6.10	Radiative transfer	69
6.11	Self-gravity	69
6.12	Incompressible and anelastic equations	70
6.13	Dust equations	70
6.14	Cosmic ray pressure in diffusion approximation	71
6.15	Chiral MHD	72
6.16	Electromagnetism with displacement current	73
6.17	Particles	73
6.17.1	Tracer particles	73
6.17.2	Dust particles	74
6.18	<i>N</i> -body solver	75
6.19	Cosmological expansion and scale factor	76
6.20	Scale factor at the end of reheating	77
6.21	Test-field equations	77
6.22	Gravitational wave equations	78
7	Troubleshooting / Frequently Asked Questions	82
7.1	Download and setup	82
7.1.1	Download forbidden	82
7.1.2	Shell gives error message when sourcing ‘sourceme.X’	82
7.2	Compilation	82
7.2.1	Error: ‘relocation truncated to fit’	82
7.2.2	Problems compiling syscalls	83
7.2.3	Unable to open include file: chemistry.h	83
7.2.4	Compiling with <i>ifc</i> under Linux	83
7.2.5	Segmentation fault with <i>ifort</i> 8.0 under Linux	84
7.2.6	The underscore problem: linking with <i>MPI</i>	84
7.2.7	Compilation stops with the cryptic error message:	85
7.2.8	The code doesn’t compile,	85
7.2.9	Some samples don’t even compile,	85
7.2.10	Internal compiler error with Compaq/Dec F90	86
7.2.11	Assertion failure under SunOS	86
7.2.12	After some dirty tricks I got pencil code to compile with <i>MPI</i> ,	87
7.2.13	Error: Symbol ‘mpi_comm_world’ at (1) has no IMPLICIT type	87
7.2.14	Error: Can’t open included file ‘mpif.h’	87
7.2.15	Compilation fails on MacOS Sonoma or Monterey	88
7.2.16	Compilation fails on Tanmay’s MacOS	88
7.2.17	Missing ld.classic on MacOS	88
7.2.18	Further MacOS tips	88
7.3	Pencil check	88
7.3.1	The pencil check complains for no reason.	88
7.3.2	The pencil check reports MISSING PENCILS and quits	89
7.3.3	The pencil check reports unnecessary pencils	89
7.3.4	The pencil check reports that most or all pencils are missing	89

7.3.5	Running the pencil check triggers mathematical errors in the code	89
7.3.6	The pencil check still complains	89
7.3.7	The pencil check is annoying so I turned it off	89
7.4	Running	89
7.4.1	Periodic boundary conditions in ‘start.x’	89
7.4.2	csh problem?	90
7.4.3	‘run.csh’ doesn’t work:	90
7.4.4	Code crashes after restarting	90
7.4.5	auto-test gone mad...?	90
7.4.6	Can I restart with a different number of cpus?	91
7.4.7	Can I restart with a different number of cpus?	91
7.4.8	fft_xyz_parallel_3D: nygrid needs to be an integer multiple...	92
7.4.9	Unit-agnostic calculations?	92
7.5	Visualization	93
7.5.1	‘start.pro’ doesn’t work:	93
7.5.2	‘start.pro’ doesn’t work:	93
7.5.3	Something about tag name undefined:	93
7.5.4	Something INC in start.pro	94
7.5.5	nl2idl problem when reading param2.nml	94
7.5.6	Spurious dots in the time series file	94
7.5.7	Problems with pc_varcontent.pro	94
7.6	Programming new slices	95
7.7	General questions	95
7.7.1	“Installation” procedure	95
7.7.2	Small numbers in the code	96
7.7.3	Why do we need a /lphysics/ namelist in the first place?	96
7.7.4	Can I run the code on a Mac?	97
7.7.5	Wrong user-id in commit emails	97
7.7.6	Pencil Code discussion forum	98
7.7.7	The manual	98

II Programming the PENCIL CODE 99

8	Understanding the code	103
8.1	Example: how is the continuity equation being solved?	103
9	Adapting the code	105
9.1	The PENCIL CODE coding standard	105
9.2	Adding new output diagnostics	106
9.3	Output at one point in space	108
9.4	The f-array	108
9.5	The df-array	109
9.6	The fp-array	109
9.7	The pencil case	110
9.7.1	Pencil check	111
9.7.2	Adding new pencils	111
9.8	Adding new physics: the Special module	111
9.9	Adding switchable modules	113
9.10	Adding your initial conditions: the InitialCondition module	113

10 Testing the code	114
10.1 How to set up periodic tests (auto-tests)	114
10.2 Auto-tests with systemd	115
10.3 Testing the postprocessing modules	116
11 Useful internals	117
11.1 Global variables	117
11.2 Subroutines and functions	117
III Appendix	119
A Timings	119
A.1 Test case	125
A.2 Running the code	127
A.3 Triolith	128
A.4 Lindgren	128
B Coding standard	132
B.1 File naming conventions	132
B.2 Fortran Code	132
B.2.1 Indenting and whitespace	132
B.2.2 Comments	133
B.2.3 Module names	134
B.2.4 Variable names	134
B.2.5 Emacs settings	135
B.3 Other best practices	136
B.4 General changes to the code	136
C Some specific initial conditions	137
C.1 Random velocity or magnetic fields	137
C.2 Turbulent initial with given spectrum	137
C.3 Beltrami fields	138
C.4 Magnetic flux rings: initaa='fluxrings'	138
C.5 Vertical stratification	139
C.5.1 Isothermal atmosphere	139
C.5.2 Polytopic atmosphere	140
C.5.3 Changing the stratification	141
C.5.4 The Rayleigh number	141
C.5.5 Entropy boundary condition	142
C.5.6 Temperature boundary condition at the top	142
C.6 Potential-field boundary condition	143
C.7 Planet solution in the shearing box	144
D Some specific boundary conditions	145
D.1 Perfect-conductor boundary condition	145
D.2 Stress-free boundary condition	145
D.3 Normal-field-radial boundary condition	146
E High-frequency filters	147
E.1 Conservative hyperdissipation	147

E.2	Hyperviscosity	149
E.2.1	Conservative case	149
E.2.2	Non-conservative cases	150
E.2.3	Choosing the coefficient	150
E.2.4	Turbulence with hyperviscosity	151
E.3	Anisotropic hyperdissipation	151
E.4	Hyperviscosity in Burgers shock	152
E.5	Time-dependent viscosity and magnetic diffusivity	153
F	Special techniques	154
F.1	After changing <i>REAL PRECISION</i>	154
F.2	Remeshing (regridding)	154
F.2.1	Remeshing hdf5-formatted data	154
F.2.2	Remeshing unformatted fortran binary data using Python	154
F.2.3	Remeshing unformatted fortran binary - original method	156
F.3	Restarting with different I/O strategy	156
F.4	Restarting from a run with less physics	157
F.5	Restarting with particles from a run without them	158
G	Runs and reference data	160
G.1	Shock tests	160
G.1.1	Sod shock tube problem	160
G.1.2	Temperature jump	160
G.2	Random forcing function	160
G.3	Three-layered convection model	161
G.4	Magnetic helicity in the shearing sheet	162
H	Numerical methods	166
H.1	Sixth-order spatial derivatives	166
H.2	Upwind derivatives to avoid ‘wiggles’	167
H.3	The bidiagonal scheme for cross-derivatives	169
H.4	The 2N-scheme for time-stepping	170
H.5	Diffusive error from the time-stepping	171
H.6	Ionization	172
H.7	Radiative transfer	173
H.7.1	Solving the radiative transfer equation	173
H.7.2	Angular integration	174
I	Curvilinear coordinates	176
I.1	Covariant derivatives	176
I.2	Differential operators	176
I.2.1	Gradient	176
I.2.2	Divergence	177
I.2.3	Curl	177
I.2.4	Advection operator	178
I.2.5	Mixed advection operator	178
I.2.6	Shear term	178
I.2.7	Another mixed advection operator	179
I.2.8	Strain Matrix	179
I.2.9	Lambda effect	179
I.2.10	Laplacian of a scalar	180

I.2.11 Hessian of a scalar	180
I.2.12 Double curl	181
I.2.13 Gradient of a divergence	182
J Switchable modules	183
K Startup and run-time parameters	185
K.1 Autogenerated list of start and run parameters	185
K.2 Startup parameters for ‘start.in’	195
K.3 Runtime parameters for ‘run.in’	202
K.4 Parameters for ‘print.in’	210
K.5 Parameters for ‘video.in’	267
K.6 Parameters for ‘phiaver.in’	268
K.7 Parameters for ‘xyaver.in’	271
K.8 Parameters for ‘xzaver.in’	284
K.9 Parameters for ‘yzaver.in’	285
K.10 Parameters for ‘yaver.in’	288
K.11 Parameters for ‘zaver.in’	291
K.12 Boundary conditions	295
K.12.1 Boundary condition <i>bcx</i>	295
K.12.2 Boundary condition <i>bcy</i>	298
K.12.3 Boundary condition <i>bcz</i>	300
K.13 Initial condition parameter dependence	304
L bin scripts	307
IV Indexes	311

Part I

Using the PENCIL CODE

1 System requirements

To use the code, you will need the following:

1. Absolutely needed:
 - *F95* compiler
 - *C* compiler
2. Used heavily (if you don't have one of these, you will need to adjust many things manually):
 - a *Unix/Linux*-type system with *make* and *csh*
 - *Perl* (remember: if it doesn't run *Perl*, it's not a computer)
3. The following are dispensable, but enhance functionality in one way or the other:
 - an *MPI* implementation (for parallelization on multiprocessor systems)
 - *DX* alias *OpenDX* or *data explorer* (for 3-D visualization of results)
 - *IDL* (for visualization of results; the 7-minute demo license will do for many applications)

2 Obtaining the code

The code is now distributed via <https://github.com/pencil-code/pencil-code>, where you can either download a tarball, or, preferably, download it via *svn* or *git*. In Iran and some other countries, GitHub is not currently available. To alleviate this problem, we have made a recent copy available on <http://www.nordita.org/software/pencil-code/>. If you want us to update this tarball, please contact us.

To ensure at least some level of stability of the *svn/git* versions, a set of test problems (listed in '\$PENCIL_HOME/bin/auto-test') are routinely tested. This includes all problems in '\$PENCIL_HOME/samples'. See Sect. 10 for details.

2.1 Obtaining the code via git or svn

1. Many machines have *svn* installed (try `svn -v` or `which svn`). On Ubuntu, for example, *svn* comes under the package name *subversion*.
2. The code is now saved under Github, *git* can be obtained in Linux by typing `sudo apt-get install git`
3. Unless you are a privileged users with write access, you can download the code with the command

```
git clone https://github.com/pencil-code/pencil-code.git
```

or

```
svn checkout https://github.com/pencil-code/pencil-code/trunk/ ...\\
pencil-code --username MY_GITHUB_USERNAME
```

In order to push your changes to the repository, you have to ask the maintainer of pencil code for push access (to become a contributor), or put a pull request to the maintainer of the code.

Be sure to run *auto-test* before you check anything back in again. It can be very annoying for someone else to figure out what's wrong, especially if you are just up to something else. At the very least, you should do

```
pc_auto-test --level=0 --no-pencil-check -C
```

This allows you to run just 2 of the most essential tests starting with all the no-modules and then most-modules.

2.2 Updating via svn or git

Independent of how you installed the code in the first place (from tarball or via *svn/git*), you can update your version using *svn/git*. If you have done nontrivial alterations to your version of the code, you ought to be careful about upgrading: although *svn/git* is an excellent tool for distributed programming, conflicts are quite possible, since many of us are going to touch many parts of the code while we develop it further. Thus, despite the fact that the code is under *svn/git*, you should probably back up your important changes before upgrading.

Here is the upgrading procedure for *git*:

1. Perform a git update of the tree:

```
unix> git pull
```

2. Fix any conflicts you encounter and make sure the examples in the directory 'samples/' are still working.

Here is the upgrading procedure for svn:

1. Perform a svn update of the tree:

```
unix> pc_svnup
```

2. Fix any conflicts you encounter and make sure the examples in the directory 'samples/' are still working.

If you have made useful changes, please contact one of the (currently) 10 “Contributors” (listed under <https://github.com/pencil-code/pencil-code>) who can give you push or check-in permission. Be sure to have sufficient comments in the code and please follow our standard coding conventions explained in Section 9.1. There is also a script to check and fix the most common style breaks, `pc_codingstyle`.

2.3 Getting the last validated version

The script `pc_svnup` accepts arguments `-val` or `-validated`, which means that the current changes on a user's machine will be merged into the last working version. This way every user can be sure that any problems with the code must be due to the current changes done by this user since the last check-in.

Examples:

```
unix> pc_svnup -src -s -validated
```

brings all files in '`$PENCIL_HOME/src`' to the last validated status, and merges all your changes into this version. This allows you to work with this, but in order to check in your changes you have to update everything to the most recent status first, i.e.

```
unix> pc_svnup -src
```

Your own changes will be merged into this latest version as before.

NOTE: The functionality of the head of the trunk should be preserved at all times. However, accidents do happen. For the benefit of all other developers, any errors should be corrected within 1-2 hours. This is the reason why the code comes with a file '`pencil-code/license/developers.txt`', which should contain contact details of all developers. The `pc_svnup -val` option allows all other people to stay away from any trouble.

2.4 Getting older versions

You may find that the latest `svn` version of the code produces errors. If you have reasons to believe that this is due to changes introduced on 27 November 2008 (to give an example), you can check out the version prior to this by specifying a revision number with `svn update -r #####`. One reason why one cannot always reproduce exactly the same situation too far back in time is connected with the fact that processor architecture and the compiler were different, resulting, e.g., in different rounding errors.

3 Getting started

To get yourself started, you should run one or several examples which are provided in one of the ‘samples/’ subdirectories. Note that you will only be able to fully assess the numerical solutions if you visualize them with *IDL*, *DX* or other tools (see Sect. 5.20).

3.1 Setup

3.1.1 Environment settings

The functionality of helper scripts and IDL routines relies on a few environment variables being set correctly. The simplest way to achieve this is to go to the top directory of the code and source one of the two scripts ‘sourceme.csh’ or ‘sourceme.sh’ (depending on the type of shell you are using):

```
csh> cd pencil-code
csh> source ./sourceme.csh
```

for *tcsh* or *csh* users; or

```
sh> cd pencil-code
sh> . ./sourceme.sh
```

for users of *bash*, *Bourne shell*, or similar shells. You should get output similar to

```
PENCIL_HOME = </home/dobler/f90/pencil-code>
Adding /home/dobler/f90/pencil-code/bin to PATH
```

Apart from the PATH variable, the environment variable IDL_PATH is set to something like `./idl:../idl:+$PENCIL_HOME/idl:./data:<IDL_DEFAULT>`.

Note 1 The <IDL_DEFAULT> mechanism does not work for IDL versions 5.2 or older. In this case, you will have to edit the path manually, or adapt the ‘sourceme’ scripts.

Note 2 If you don’t want to rely on the ‘sourceme’ scripts’ (quite heuristic) ability to correctly identify the code’s main directory, you can set the environment variable PENCIL_HOME explicitly before you run the source command.

Note 3 Do not just source the ‘sourceme’ script from your shell startup file (‘~/ .cshrc’ or ‘~/ .bashrc’, because it outputs a few lines of diagnostics for each sub-shell, which will break many applications. To suppress all output, follow the instructions given in the header documentation of ‘sourceme.csh’ and ‘sourceme.sh’. Likewise, output from other files invoked by source should also be suppressed.

Note 4 The second time you source ‘sourceme’, it will not add anything to your PATH variable. This is on purpose to avoid cluttering of your environment: you can source the file as often as you like (in your shell startup script, then manually and in addition in some script you have written), without thinking twice. If, however, at the first sourcing, the setting of PENCIL_HOME was wrong, this mechanism would keep you from ever adding the right directory to your PATH. In this case, you need to first undefine the environment variable PENCIL_HOME:

```
csh> unsetenv PENCIL_HOME
csh> source ./sourceme.csh
```

or

```
sh> unset PENCIL_HOME
sh> . ./sourceme.sh
```

Note 5 If you want to be able to easily handle multiple versions/branches of Pencil, you can use the ‘modulefile’ mechanism that is used on most clusters to load libraries and programs. Create a file at, say, ‘\$HOME/.modulefiles/pencil-local’ with the following contents:

```
#!/Module4.6#####

proc ModulesHelp {} {
    global version prefix

    puts stderr "\tmodules - loads the modules software"
    puts stderr "& application environment"
    puts stderr "\n\tThis adds $prefix/* to several of the"
    puts stderr "\tenvironment variables."
    puts stderr "\n\tVersion $version\n"
}

module-whatis    "Environment setup for the Pencil code"

#change the following line according to the location of your local copy of Pencil
setenv          PENCIL_HOME      $env(HOME)/.software/pencil-code

setenv          _sourceme_quiet 1
source-sh       bash              $env(PENCIL_HOME)/sourceme.sh
unsetenv        _sourceme_quiet
```

To your ‘ ~/.bashrc’, add

```
MODULEPATH=$HOME/.modulefiles:$MODULEPATH
```

If you now open a new shell and run `module avail`, you will find the `pencil-local` module created above listed as an option. This requires version 4.6 of the `modules` program.

3.1.2 Linking scripts and source files

With your environment set up correctly, you can now go to the directory you want to work in and set up subdirectories and links. This is accomplished by the script ‘`pc_setupsrc`’, which is located in ‘`$PENCIL_HOME/bin`’ and is thus now in your executable path.

For concreteness, let us assume you want to use ‘`samples/conv-slab`’ as your *run directory*, i.e. you want to run a three-layer slab model of solar convection. You then do the following:

```
unix> cd samples/conv-slab
unix> pc_setupsrc
src already exists
2 files already exist in src
```

The script has linked a number of scripts from ‘\$PENCIL_HOME/bin’, generated a directory ‘src’ for the source code and linked the Fortran source files (plus a few more files) from ‘\$PENCIL_HOME/src’ to that directory:

```
unix> ls -F
reference.out  src/
start.csh@    run.csh@    getconf.csh@
start.in      run.in      print.in
```

3.1.3 Adapting ‘Makefile.src’

This step requires some input from you, but you only have to do this once for each machine you want to run the code on. See Sect. 5.2 for a description of the steps you need to take here.

Note: If you are lucky and use compilers similar to the ones we have, you may be able to skip this step; but blame yourself if things don’t compile, then. If not, you can run make with explicit flags, see Sect. 5.2 and in particular Table 1.

3.1.4 Running make

Next, you run make in the ‘src’ subdirectory of your run directory. Since you are using one of the predefined test problems, the settings in ‘src/Makefile.local’ and ‘src/cparam.local’ are all reasonable, and you just do

```
unix> make
```

If you have set up the compiler flags correctly, compilation should complete successfully.

3.1.5 Choosing a data directory

The code will by default write data like snapshot files to the subdirectory ‘data’ of the run directory. Since this will involve a large volume of IO-operations (at least for large grid sizes), one will normally try to avoid writing the data via NFS. The recommended way to set up a ‘data’ data directory is to generate a corresponding directory on the local disc of the computer you are running on and (soft-)link it to ‘./data’. Even if the link is part of an NFS directory, all the IO operations will be local. For example, if you have a local disc ‘/scratch’, you can do the following:

```
unix> mkdir -p /scratch/$USER/pencil-data/samples/conv-slab
unix> ln -s /scratch/$USER/pencil-data/samples/conv-slab ./data
```

This is done automatically by the `pc_mkdatadir` command which, in turn, is invoked when making a new run directory with the `pc_newrun` command, for example.

Even if you don’t have an NFS-mounted directory (say, on your notebook computer), it is probably still a good idea to have code and data well separated by a scheme like the one described above.

An alternative to symbolic links, is to provide a file called ‘datadir.in’ in the root of the run directory. This file should contain one line of text specifying the absolute or relative data directory path to use. This facility is useful if one wishes to switch one run directory between different data directories. It is suggested that in such cases symbolic links are again made in the run directory to the various locations, then the ‘datadir.in’ need contain only a short relative path.

3.1.6 Running the code

You are now ready to start the code:

```

unix> start.csh
Linux cincinnatus 2.4.18-4GB #1 Wed Mar 27 13:57:05 UTC 2002 i686 unknown
Non-MPI version
datadir = data
Fri Aug 8 21:36:43 CEST 2003
    src/start.x
CVS: io_dist.f90          v. 1.61          (brandenb ) 2003/08/03 09:26:55
[...]
CVS: start.in             v. 1.4           (dobler   ) 2002/10/02 20:11:14
nxgrid,nygrid,nzgrid=      32             32             32
thermodynamics: assume cp=1

uu: up-down
piecewise polytropic vertical stratification (lnrho)
init_lnrho: cs2bot,cs2top=  1.450000      0.3333330
e.g., for ionization runs: cs2bot,cs2top not yet set
piecewise polytropic vertical stratification (ss)

start.x has completed successfully

0.070u 0.020s 0:00.14 64.2%      0+0k 0+0io 180pf+0w

Fri Aug 8 21:36:43 CEST 2003

```

This runs ‘src/start.x’ to construct an initial condition based on the parameters set in ‘start.in’. This initial condition is stored in ‘data/proc0/var.dat’ (and in ‘data/proc1/var.dat’, etc. if you run the multiprocessor version). It is fair to say that this is now a rather primitive routine; see ‘pencil-code/idl/read’ for various reading routines. You can then visualize the data using standard idl language.

If you visualize the profiles using *IDL* (see below), the result should bear some resemblance to Fig. 3, but with different values in the ghost zones (the correct values are set at run-time only) and a simpler velocity profile.

Now we run the code:

```

unix> run.csh

```

This executes ‘src/run.x’ and carries out *nt* time steps, where *nt* and other run-time parameters are specified in ‘run.in’. On a decent PC (1.7 GHz), 50 time steps take about 10 seconds.

The relevant part of the code’s output looks like

```

--it----t-----dt-----urms----umax----rhom-----ssm-----dtc----dtu----dtnu----dtchi-
  0   0.34  6.792E-03  0.0060  0.0452  14.4708  -0.4478  0.978  0.013  0.207  0.346
 10   0.41  6.787E-03  0.0062  0.0440  14.4707  -0.4480  0.978  0.013  0.207  0.345
 20   0.48  6.781E-03  0.0064  0.0429  14.4705  -0.4481  0.977  0.012  0.207  0.345
 30   0.54  6.777E-03  0.0067  0.0408  14.4703  -0.4482  0.977  0.012  0.207  0.345
 40   0.61  6.776E-03  0.0069  0.0381  14.4702  -0.4482  0.977  0.011  0.207  0.346

```

and lists

1. the number *it* of the current time step;
2. the time, *t*;
3. the time step, *dt*;
4. the rms velocity, $urms = \sqrt{\langle u^2 \rangle}$;
5. the maximum velocity, $umax = \max |u|$;
6. the mean density, $rhom = \langle \rho \rangle$;
7. the mean entropy, $ssm = \langle s \rangle / c_p$;
8. the time step in units of the acoustic Courant step, $dtc = \delta t c_{s0} / \delta x_{\min}$;
9. the time step in units of the advective time step, $dtu = \delta t / (c_{\delta t, v} \delta x / \max |u|)$;
10. the time step in units of viscous time step, $dtvu = \delta t / (c_{\delta t, v} \delta x^2 / \nu_{\max})$;
11. the time step in units of the conductive time step, $dtchi = \delta t / (c_{\delta t, v} \delta x^2 / \chi_{\max})$.

The entries in this list can be added, removed or reformatted in the file ‘print.in’, see Sects 5.6 and K.4. The output is also saved in ‘data/time_series.dat’ and should be identical to the content of ‘reference.out’.

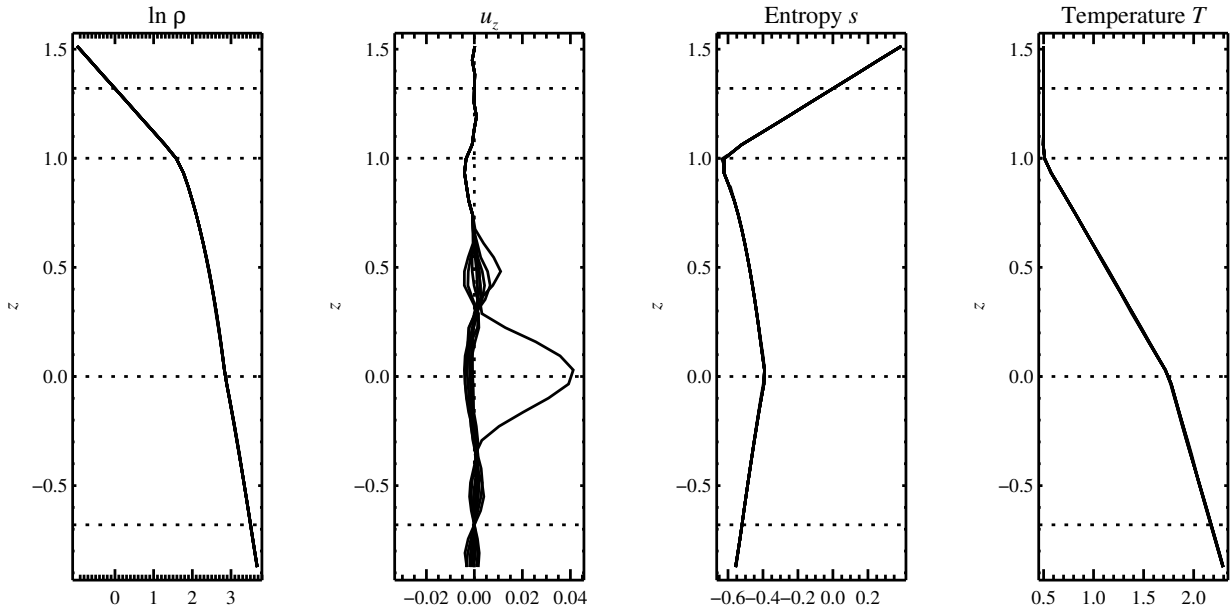


Figure 3: Stratification of the three-layer convection model in ‘samples/conv-slab’ after 50 timesteps ($t = 0.428$). Shown are (from left to right) density ρ , vertical velocity u_z , entropy s/c_p and temperature T as functions of the vertical coordinate z for about ten different vertical lines in the computational box. The dashed lines denote domain boundaries: $z < -0.68$ is the lower ghost zone (points have no physical significance); $-0.68 < z < 0$ is a stably stratified layer ($ds/dz > 0$); $0 < z < 1$ is the unstable layer ($ds/dz < 0$); $1 < z < 1.32$ is the isothermal top layer; $z > 1.32$ is the upper ghost zone (points have no physical significance).

If you have *IDL*, you can visualize the stratification with (see Sect. 5.20.4 for details)

```
unix > idl
IDL > pc_read_var,obj=var,/trimall
IDL > tvscl,var,uu(*,*,0,0)
```


which shows u_x in the xy plane through the first meshpoint in the z direction. There have been some now outdates specific routines that produce results like that shown in Fig. 3.

The same can be achieved using *Python* (see Sect. 5.20.5 for details) with

```
unix > ipython3 # (or 'ipython', or just 'python')
python > import pencil as pc
python > from matplotlib import pylab as plt
python > var = pc.read.var(trimall=True)
python > plt.imshow(var.uu[0, 0, :, :].T, origin='lower')
```

where we also make sure that the axis are shown in a natural way.

Note: If you want to run the code with *MPI*, you will probably need to adapt ‘getconf.csh’, which defines the commands and flags used to run *MPI* jobs (and which is sourced by the scripts ‘start.csh’ and ‘run.csh’). Try

```
csh -v getconf.csh
or
csh -x getconf.csh
```

to see how ‘getconf.csh’ makes its decisions. You would add a section for the host name of your machine with the particular settings. Since ‘getconf.csh’ is linked from the central directory ‘pencil-code/bin’, your changes will be useful for all your other runs too.

3.2 Further tests

There is a number of other tests in the ‘samples/’ directory. You can use the script ‘bin/auto-test’ to automatically run these tests and have the output compared to reference results.

4 Code structure

4.1 Directory tree

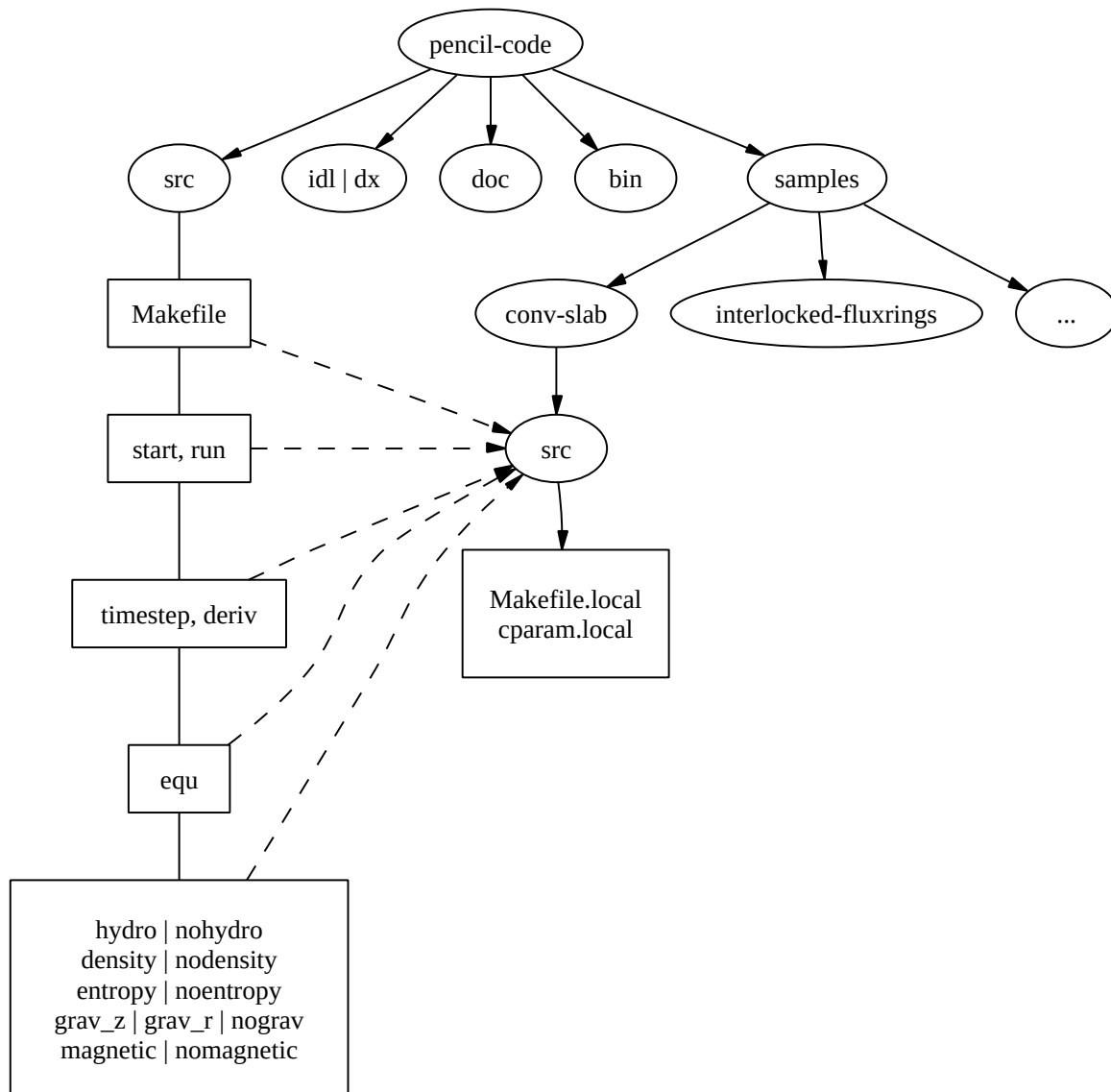


Figure 4: The basic structure of the code

The overall directory structure of the code is shown in Fig. 4. Under ‘pencil-code’, there are currently the following files and directories:

```

bin/  config/  doc/  idl/  license/  perl/  samples/  sourceme.sh  utils/
bugs/ dx/  lib/  misc/  README  sourceme.csh  src/  www/

```

Almost all of the source code is contained in the directory ‘src/’, but in order to encapsulate individual applications, the code is compiled separately for each run in a local directory ‘src’ below the individual run directory, like e. g. ‘samples/conv-slab/src’.

It may be a good idea to keep your own runs also under *svn* or *cvs* (which is older than but similar to *svn*), but this would normally be a different repository. On the machine where you are running the code, you may want to check them out into a subdirectory of ‘pencil-code/’. For example, we have our own runs in a repository called ‘pencil-runs’, so we do

```
unix> cd $PENCIL_HOME
unix> svn co runs pencil-runs
```

In this case, ‘runs’ contains individual run directories, grouped in classes (like ‘spher’ for spherical calculations, or ‘kinematic’ for kinematic dynamo simulations). The current list of classes in our own ‘pencil-runs’ repository is

```
1d-tests/  disc/          kinematic/  rings/
2d-tests/  discont/        Misc/       slab_conv/
3d-tests/  discussion/     OLD/       test/
buoy_tube/ forced/       pass_only/
convstar/  interstellar/  radiation/
```

The directory ‘forced/’ contains some forced turbulence runs (both magnetic and non-magnetic); ‘gravz/’ contains runs with vertical gravity; ‘rings/’ contains decaying MHD problems (interlocked flux rings as initial condition, for example); and ‘kinematic/’ contains kinematic dynamo problems where the hydrodynamics is turned off entirely. The file ‘samples/README’ should contain an up-to-date list and short description of the individual classes.¹

The subdirectory ‘src’ of each run directory contains a few local configuration files (currently these are ‘Makefile.local’ and ‘cparam.local’) and possibly ‘ctimeavg.local’. To compile the samples, links the files ‘.f90’, ‘.c’ and ‘Makefile.src’ need to be linked from the top file[src]/src directory to the local directory ‘./src’. These links are set up by the script pc_setupsrc) when used in the root of a run directory.

General-purpose visualization routines for *IDL* or *DX* are in the directories ‘idl’ and ‘dx’, respectively. There are additional and more specialized *IDL* directories in the different branches under ‘pencil-runs’.

The directory ‘doc’ contains this manual; ‘bin’ contains a number of utility scripts (mostly written in *csh* and *Perl*), and in particular the ‘start.csh’, ‘run.csh’, and ‘getconf.csh’ scripts. The ‘.svn’ directory is used (you guessed it) by *.svn*, and is not normally directly accessed by the user; ‘bugs’, finally is used by us for internal purposes.

The files ‘sourceme.csh’ and ‘sourceme.sh’ will set up some environment variables — in particular *PATH* — and aliases/shell functions for your convenience. If you do not want to source one of these files, you need to make sure your *IDL* path is set appropriately (provided you want to use *IDL*) and you will need to address the scripts from ‘bin’ with their explicit path name, or adjust your *PATH* manually.

4.2 Basic concepts

4.2.1 Data access in pencils

Unlike the *CRAY* computers that dominated supercomputing in the 80s and early 90s, all modern computers have a cache that constitutes a significant bottleneck for many codes. This is the case if large three-dimensional arrays are constantly used within each time step, which has the obvious advantage of working on long arrays and allows vectorization of elementary machine operations. This approach also implies conceptual simplicity of the code and allows extensive use of the intuitive *F90* array syntax. However,

¹Our ‘pencil-runs’ directory also contains runs that were done some time ago. Occasionally, we try to update these, especially if we have changed names or other input conventions.

a more cache-efficient way of coding is to calculate an entire time step (or substep of a multi-stage time-stepping scheme) only along a one-dimensional pencil of data within the numerical grid. This technique is more efficient for modern RISC processors: on Linux PCs and SGI workstations, for example, we have found a speed-up by about 60% in some cases. An additional advantage is a drastic reduction in temporary storage for auxiliary variables within each time step.

4.2.2 Modularity

Each run directory has a file ‘src/Makefile.local’ in which you choose certain *modules*², which tell the code whether or not entropy, magnetic fields, hydrodynamics, forcing, etc. should be invoked. For example, the settings for forced turbulent MHD simulations are

```
HYDRO      =  hydro
DENSITY    =  density
ENTROPY     =  noentropy
MAGNETIC    =  magnetic
GRAVITY     =  nogravity
FORCING     =  forcing

MPICOMM    =  nompicomm
GLOBAL      =  noglobal
IO          =  io_dist
FOURIER     =  nofourier
```

This file will be processed by *make* and the settings are thus assignments of *make* variables. Apart from the physics modules (equation of motion: yes, density [pressure]: yes, entropy equation: no, magnetic fields: yes, gravity: no, forcing: yes), a few technical modules can also be used or deactivated; in the example above, these are *MPI* (switched off), additional global variables (none), input/output (distributed), and *FFT* (not used). The table in Sect. J in the Appendix lists all currently available modules.

Note that most of these *make* variables *must* be set, but they will normally obtain reasonable default values in ‘Makefile’ (so you only need to set the non-standard ones in ‘Makefile.local’). It is by using this switching mechanism through *make* that we achieve high flexibility without resorting to excessive amounts of cryptic preprocessor directives or other switches within the code.

Many possible combinations of modules have already been tested and examples are part of the distribution, but you may be interested in a combination which was never tried before and which may not work yet, since the modules are not fully orthogonal. In such cases, we depend on user feedback for fixing problems and documenting the changes for others.

4.3 Files in the run directories

4.3.1 ‘start.in’, ‘run.in’, ‘print.in’

These files specify the startup and runtime parameters (see Sects. 5.13 and K.3), and the list of diagnostic variables to print (see 5.6). They specify the setup of a given simulation

²We stress once more that we are not talking about F90 modules here, although there is some connection, as most of our modules define F90 modules: For example each of the modules *gravity-simple*, *grav-r* and *nogravity* defines a Fortran module *Gravity*.

and are kept under *svn* in the individual ‘samples’ directories.

You may want to check for the correctness of these configuration files by issuing the command `pc_configtest`.

4.3.2 ‘datadir.in’

If this file exists, it must contain the name of an existing directory, which will be used as *data directory*, i. e. the directory where all results are written. If ‘datadir.in’ does not exist, the data directory is ‘data/’.

4.3.3 ‘sn_series.in’

Formatted file containing the times and locations at which future supernova events will occur, using same format as ‘sn_series.dat’ when `lsn_list`. (Only needed by the *interstellar* module.)

4.3.4 ‘reference.out’

If present, ‘reference.out’ contains the output you should obtain in the given run directory, provided you have not changed any parameters. To see whether the results of your run are OK, compare ‘time_series.dat’ to ‘reference.out’:

```
unix> diff data/time_series.dat reference.out
```

4.3.5 ‘start.csh’, ‘run.csh’, ‘getconf.csh’ [obsolete; see Sect. 5.1]

These are links to ‘\$PENCIL_HOME/bin’. You will be constantly using the scripts ‘start.csh’ and ‘run.csh’ to initialize the code. Things that are needed by both (like the name of the `mpirun` executable, *MPI* options, or the number of processors) are located in ‘getconf.csh’, which is never directly invoked.

4.3.6 ‘src/’

The ‘src’ directory contains two local files, ‘src/Makefile.local’ and ‘src/cparam.local’, which allow the user to choose individual modules (see 4.2.2) and to set parameters like the grid size and the number of processors for each direction. These two files are part of the setup of a given simulation and are kept under *svn* in the individual ‘samples’ directories.

The file ‘src/cparam.inc’ is automatically generated by the script ‘mkcparam’ and contains the number of fundamental variables for a given setup.

All other files in ‘src/’ are either links to source files (and ‘Makefile.src’) in the ‘\$PENCIL_HOME/src’ directory, or object and module files generated by the compiler.

4.3.7 ‘data/’

This directory (the name of which will actually be overwritten by the first line of ‘datadir.in’, if that file is present; see §4.3.2) contains the output from the code:

‘data/dim.dat’ The global array dimensions.

`'data/legend.dat'` The header line specifying the names of the diagnostic variables in `'time_series.dat'`.

`'data/time_series.dat'` Time series of diagnostic variables (also printed to stdout). You can use this file directly for plotting with *Gnuplot*, *IDL*, *Xmgrace* or similar tools (see also §5.20).

`'data/tsnap.dat'`, `'data/tvid.dat'` Time when the next snapshot `'VAR $N$ '` or animation slice should be taken.

`'data/params.log'` Keeps a log of all your parameters: `'start.x'` writes the startup parameters to this file, `'run.x'` appends the runtime parameters and appends them anew, each time you use the `'RELOAD'` mechanism (see §5.11).

`'data/param.nml'` Complete set of startup parameters, printed as Fortran namelist. This file is read in by `'run.x'` (this is how values of startup parameters are propagated to `'run.x'`) and by *IDL* (if you use it).

`'data/param2.nml'` Complete set of runtime parameters, printed as Fortran namelist. This file is read by *IDL* (if you use it).

`'data/index.pro'` Can be used as include file in *IDL* and contains the column in which certain variables appear in the diagnostics file (`'time_series.dat'`). It also contains the positions of variables in the `'VAR $N$ '` files. These positions depend on whether *entropy* or *noentropy*, etc, are invoked. This is a temporary solution and the file may disappear in future releases.

`'data/sn_series.dat'` Time series of SN explosions locations and diagnostics. Can be plotted using same machinery as for `'time_series.dat'` and stored as `'sn_series.in'` to replicate series in subsequent experiments. (Only needed by the *interstellar* module.)

`'data/proc0'`, `'data/proc1'`, ... These are the directories containing data from the individual processors. So after running an *MPI* job on two processors, you will have the two directories `'data/proc0'` and `'data/proc1'`. Each of the directories can contain the following files:

`'var.dat'` binary file containing the latest snapshot;

`'VAR $N$ '` binary file containing individual snapshot number N ;

`'dim.dat'` ASCII file containing the array dimensions as seen by the given processor;

`'time.dat'` ASCII file containing the time corresponding to `'var.dat'` (not actually *used* by the code, unless you use the *io_mpiodist.f90* module);

`'grid.dat'` binary file containing the part of the grid seen by the given processor;

`'seed.dat'` the random seed for the next time step (saved for reasons of reproducibility). For multi-processor runs with velocity forcing, the files `'proc $N$ /seed.dat'` must all contain the same numbers, because globally coherent waves of given wavenumber are used;

'*X.xy*', '*X.xz*', '*X.yz*' two-dimensional sections of variable *X*, where *X* stands for the corresponding variable. The current list includes

```
bx.xy  bx.xz  by.xy  by.xz  bz.xy  bz.xz  divu.xy  lnrho.xz
ss.xz  ux.xy  ux.xz  uz.xy  uz.xz
```

Each processor writes its own slice, so these need to be reassembled if one wants to plot a full slice.

5 Using the code

5.1 Configuring the code to compile and run on your computer

Note: We recommend to use the procedure described here, rather than the old method described in Sects. 5.2 and 4.3.5.

Quick instructions: You may compile with a default compiler-specific configuration:

1. Single-processor using the GNU compiler collection:

```
unix> pc_build -f GNU-GCC
```

2. Multi-processor using GNU with MPI support:

```
unix> pc_build -f GNU-GCC_MPI
```

Many compilers are supported already, please refer to the available config files in '\$PENCIL_HOME/config/compilers/*.conf', e.g., 'Intel.conf' and 'Intel_MPI.conf'.

If you have to set up some compiler options specific to a certain host system you work on, or if you like to create a host-specific configuration file so that you can simply execute `pc_build` without any options, you can clone an existing host-file, just include an existing compiler configuration, and simply only add the options you need. A good example of a host-file is '\$PENCIL_HOME/config/hosts/IWF/host-andromeda-GNU_Linux-Linux.conf'. You may save a clone under '\$PENCIL_HOME/config/hosts/<ID>.conf', where '<ID>' is to be replaced by the output of `pc_build -i`. This will be the new default for `pc_build`. Another way to specify the default is setting the environment variable `PENCIL_CONFIG_FILES` appropriately.

If you don't know what this was all about, read on.

In essence, configuration, compiling and running the code work like this:

1. Create a configuration file for your computer's *host ID*.
2. Compile the code using `pc_build`.
3. Run the code using `pc_run`

In the following, we will discuss the essentials of this scheme. Exhaustive documentation is available with the commands `perldoc Pencil::ConfigFinder` and `perldoc PENCIL::ConfigParser`.

5.1.1 Locating the configuration file

Commands like `pc_build` and `pc_run` use the Perl module 'Pencil::ConfigFinder' to locate an appropriate configuration file and 'Pencil::ConfigParser' to read and interpret it. When you use 'ConfigFinder' on a given computer, it constructs a *host ID* for the system it is running on, and looks for a file 'host_ID.conf' in any subdirectory of '\$PENCIL_HOME/config/hosts'.

E.g., if the host ID is "workhorse.pencil.org", 'ConfigFinder' would consider the file '\$PENCIL_HOME/config/hosts/pencil.org/workhorse.pencil.org.conf'.

Note 1: The location in the tree under 'hosts/' is irrelevant, which allows you to group related hosts by institution, owner, hardware, etc.

Note 2: ‘ConfigFinder’ actually uses the following search path:

1. ‘./config’
2. ‘\$PENCIL_HOME/config-local’
3. ‘\$HOME/.pencil/config’
4. ‘\$PENCIL_HOME/config’

This allows you to override part of the ‘config/’ tree globally on the given file system, or locally for a particular run directory, or for one given copy of the PENCIL CODE. This search path is used both, for locating the configuration file for your host ID, and for locating included files (see below).

The host ID is constructed based on information that is easily available for your system. The algorithm is as follows:

1. Most commands using ‘ConfigFinder’ have a ‘--host-id’ (sometimes abbreviated as ‘-H’) option to explicitly set the host ID.
2. If the environment variable *PENCIL_HOST_ID* is set, its value is used.
3. If any of the files ‘./host-ID’, ‘\$PENCIL_HOME/host-ID’, ‘\$HOME/.pencil/host-ID’, exists, its first line is used.
4. If ‘ConfigFinder’ can get hold of a *fully qualified host name*, that is used as host ID.
5. Else, a combination of host name, operating system name and possibly some other information characterizing the system is used.
6. If no config file for the host ID is found, the current operating system name is tried as fallback host ID.

To see which host IDs are tried (up to the first one for which a configuration file is found), run

```
unix> pc_build --debug-config
```

This command will tell you the *host-ID* of the system that you are using:

```
unix> pc_build -i
```

5.1.2 Structure of configuration files

It is strongly recommended to include in a users configuration file one of the preset compiler suite configuration files. Then, only minor options need to be set by a user, e.g., the optimization flags. One of those user configuration files looks rather simple:

```
# Simple config file. Most files don't need more.
#include compilers/GNU-GCC
```

or if you prefer a different compiler:

```
# Simple Intel compiler suite config file.
#include compilers/Intel
```

A more complex file (using MPI with additional options) would look like this:

```
# More complex config file.
#include compilers/GNU-GCC_MPI

%section Makefile
    MAKE_VAR1 = -j4    # joined compilation with four threads
    FFLAGS += -O3 -Wall -fbacktrace    # don't redefine, but append with '+= '
%endsection Makefile

%section runtime
    mpiexec = mpirun    # some MPI backends need a redefinition of mpiexec
%endsection runtime

%section environment
    SCRATCH_DIR=/var/tmp/$USER
%endsection environment
```

Adding ”_MPI” to a compiler suite’s name is usually sufficient to use MPI.

Note 3: We strongly advise *not* to mix Fortran- and C-compilers from different manufacturers or versions by manually including multiple separate compiler configurations.

Note 4: We strongly advise to use *at maximum* the optimization levels ’-O2’ for the Intel compiler and ’-O3’ for all other compilers. Higher optimization levels implicate an inadequate loss of precision.

The ‘.conf’ files consist of the following elements:

Comments: A # sign and any text following it on the same line are ignored.

Sections: There are three sections:

Makefile for setting make parameters

runtime for adding compiler flags used by pc_run

environment shell environment variables for compilation and running

Include statements: An %include ... statement is recursively replaced by the contents of the files it points to.³

The included path gets a .conf suffix appended. Included paths are typically “absolute”, e.g.,

```
%include os/Unix
```

will include the file ‘os/Unix.conf’ in the search path listed above (typically from ‘\$PENCIL_HOME/config’). However, if the included path starts with a dot, it is a relative path, so

```
%include ./Unix
```

will only search in the directory where the including file is located.

Assignments: Statements like FFLAGS += -O3 or mpiexec=mpirun are assignments and will set parameters that are used by pc_build/make or pc_run.

³However, if the include statement is inside a section, only the file’s contents inside that section are inserted.

Lines ending with a backslash ‘\’ are continuation lines.

If possible, one should always use *incremental assignments*, indicated by using a += sign instead of =, instead of redefining certain flags.

Thus,

```
CFLAGS += -O3
CFLAGS += -I../include -Wall
```

is the same as

```
CFLAGS = $(CFLAGS) -O3 -I../include -Wall
```

5.1.3 Compiling the code

Use the `pc_build` command to compile the code:

```
unix> pc_build                # use default compiler suite
unix> pc_build -f Intel_MPI    # specify a compiler suite
unix> pc_build -f os/GNU_Linux,mpi/open-mpi # explicitly specify config files
unix> pc_build -l              # use same config files as in last cal
unix> pc_build VAR=something   # set variables for the makefile
unix> pc_build --cleanall      # remove generated files
```

The third example circumvents the whole host ID mechanism by explicitly instructing `pc_build` which configuration files to use. In the fourth example, `pc_build` will apply the same configuration files as in its last invocation. They are stored in ‘`src/.config-files`’, which is automatically written, but can also be manually modified. The fifth example shows how to define extra variables (`VAR=something`) for the execution of the Makefile.

See `pc_build --help` for a complete list of options.

5.1.4 Running the code

Use the `pc_run` command to run the code:

```
unix> pc_run                  # start if necessary, then run
unix> pc_run start
unix> pc_run run

unix> pc_run start run^3      # start, then run 3 times
unix> pc_run start run run run # start, then run 3 times
unix> pc_run ^3               # start if necessary, then run 3 times
```

See `pc_run --help` for a complete list of options.

5.1.5 Testing the code

The `pc_auto-test` command uses `pc_build` for compiling and `pc_run` for running the tests. Here are a few useful options:

```
unix> pc_auto-test
unix> pc_auto-test --no-pencil-check # suppress pencil consistency check
unix> pc_auto-test --max-level=1    # run only tests in level 0 and 1
unix> pc_auto-test --time-limit=2m  # kill each test after 2 minutes
```

See `pc_auto-test --help` for a complete list of options.

The `pencil-test` script will use `pc_auto-test` if given the `--use-pc_auto-test` or `-b` option:

```
unix> pencil-test --use-pc_auto-test
unix> pencil-test -b                # ditto
unix> pencil-test -b                -Wa,--max-level=1,--no-pencil-check # quick pencil
```

See `pencil-test --help` for a complete list of options, and section 10 for more details.

5.2 Adapting ‘Makefile.src’ [obsolete; see Sect. 5.1]

By default, one should use the above described configuration mechanism for compilation. If for whatever reason one needs to work with a modified ‘Makefile’, there is a mechanism for picking the right set of compiler flags based on the hostname.

To give you an idea of how to add your own machines, let us assume you have several Linux boxes running a compiler `f95` that needs the options `-O2 -u`, while one of them, *Janus*, runs a compiler `f90` which needs the flags `-O3` and requires the additional options `-lmpi -llam` for *MPI*.

The ‘Makefile.src’ you need will have the following section:

```
### Begin machine dependent

## IRIX64:
[...] (leave as it is in the original Makefile)
## OSF1:
[...] (leave as it is in the original Makefile)

## Linux:
[...] (leave everything from original Makefile and add:)
#FC=f95
#FFLAGS=-O2 -u
#FC=f90          #(Janus)
#FFLAGS=-O3      #(Janus)
#LDMPI=-lmpi -llam #(Janus)

## SunOS:
[...] (leave as it is in the original Makefile)
## UNICOS/mk:
[...] (leave as it is in the original Makefile)
## HI-UX/MPP:
[...] (leave as it is in the original Makefile)
## AIX:
[...] (leave as it is in the original Makefile)

### End machine dependent
```

Note 1 There is a script for adapting the Makefile: ‘`adapt-mkfile`’. In the example above, `$(Janus)` is *not* a comment, but marks this line to be activated (uncommented) by `adapt-mkfile` if your hostname (`uname -n`) matches ‘`Janus`’ or ‘`janus`’ (capitalization

Table 1: Compiler flags for common compilers. Note that some combinations of OS and compiler require much more elaborate settings; also, if you use MPI, you will have to set LDMPI.

Compiler	FC	FFLAGS	CC	CFLAGS
<i>Unix/POSIX:</i>				
GNU	gfortran	-O3	gcc	-O3 -DFUNDERSO=1
Intel	ifort	-O2	icc	-O3 -DFUNDERSO=1
PGI	pgf95	-O3	pgcc	-O3 -DFUNDERSO=1
G95	g95	-O3 -fno-second-underscore	gcc	-O3 -DFUNDERSO=1
Absoft	f95	-O3 -N15	gcc	-O3 -DFUNDERSO=1
IBM XL	xlf95	-qsuffix=f=f90 -O3	xlc	-O3 -DFUNDERSO=1
<i>outdated:</i>				
IRIX Mips	f90	-64 -O3 -mips4	cc	-O3 -64 -DFUNDERSO=1
Compaq	f90	-fast -O3	cc	-O3 -DFUNDERSO=1

is irrelevant). You can combine machine names with a vertical bar: a line containing `$(onsager|Janus)` will be activated on both, *Janus* and *Onsager*.

Note 2 If you want to experiment with compiler flags, or if you want to get things running without setting up the machine-dependent section of the ‘Makefile’, you can set *make* variables at the command line in the usual manner:

```
src> make FC=f90 FFLAGS='-fast -u'
```

will use the compiler `f90` and the flags `-fast -u` for both compilation and linking. Table 1 summarizes flags we use for common compilers.

5.3 Changing the resolution

It is advisable to produce a new run directory each time you run a new case. (This does not include restarts from an old run, of course.) If you have a 32³ run in some directory ‘runA_32a’, then go to its parent directory, i.e.

```
runA_32a> cd ..
forced> pc_newrun runA_32a runA_64a
forced> cd runA_64a/src
forced> vi cparam.local
```

and edit the ‘cparam.local’ for the new resolution.

If you have ever wondered why we don’t do dynamic allocation of the main variable (`f`) array, the main reason is that with static allocation the compiler can check whether we are out of bounds.

5.4 GPUs

Pencil Code can also be run on GPUs via automatic code translation to the Astaroth GPU library. This requires you to have the following libraries:

```
cmake version >= 3.21,bison,flex,CUDA or HIP,python
```

The Astaroth library and the code translator for it come together with Pencil Code as git submodules. To get them and/or update them to the latest versions run:

```
git submodule update --init --remote
```

In 'src/Makefile.local' add the following options:

```
GPU=gpu_astaroth
```

Compilation and running are done in the same manner as for normal CPU setups. By default the code assumes communication is done with CUDA-aware MPI, which means the GPUs speak directly to each other. If you have to ask is your MPI CUDA-aware enabled then it most likely is not.⁴ Thus trying to run the code in CUDA-aware mode will most likely cause a segmentation fault and the code to crash.

To turn off CUDA-aware MPI add the following to your build config

```
CUDA_AWARE_MPI=OFF
```

The GPU-enabled version uses threads to compute the diagnostics alongside the time integration happening on the GPUs. Because of this you should have at least 2 threads executing per process. An example using 7 threads per process would be:

```
export OMP_NUM_THREADS=7
export OMP_MAX_ACTIVE_LEVELS=2
export OMP_PROC_BIND=close,spread
export OMP_WAIT_POLICY=PASSIVE
```

,where the other options are helpful to get maximal performance out of the multithreading. If you are using SLURM you would put these exports into your sbatch file. Additionally if you are using CUDA-aware MPI with MPICH (for example on the LUMI supercomputer) put the following export before calling run.x but after calling start.x

```
export MPICH_GPU_SUPPORT_ENABLED=1
```

The normal usage of GPUs will imply that compilation will happen during the run of the simulation. Because of this, it is imperative for module environment that the same modules are used for the run as for the initial compilation. To ensure this, add the following to your sbatch script:

```
source src/.moduleinfo
```

Note that not all of Pencil Code can run on GPUs. This is particularly obvious in view of the particle modules either in the Particle-in-Cell (PIC) or the N -body solvers of Pencil Code, neither of which are GPU accelerated. Additionally, more complicated computations outside of the RHSs are not (yet) included like the Navier-Stokes Characteristic Boundary Condition (NSCBC) and granulation calculations in the corona modules. To see which of the committed samples are 100 percent known to run on GPUs, check the list `gpu_test` in 'bin/pc_autotest'. As a rule of thumb, if the sample only requires the following features it can be run on GPUs:

- Integration of PDEs and ODEs.
- Boundary conditions that can be formulated as local operations. (Boundary conditions outside of those already covered require explicit use of code tools to generate the GPU code.)

⁴If Openmpi has been built with CUDA support, you will see `cuda` in the output of `mpi_info | grep "MPI extensions"`.

- Calculations of averages or other reductions before time integration. (Cases outside of those already covered require explicit use of code tools to generate the GPU code.)
- Fourier transforms (requires handwritten modifications to add new physics).

5.4.1 Runtime compilation

As stated normal usage of GPUs requires compilation during the running of the setup. This is because the code as is with its large number of conditionals would not be performant on the GPUs because for performance Astaroth needs to know at compile-time which stencils are used. Compiling after the parameters have been chosen fixes the problem.

However it is inconvenient if you want to run a spectrum of runs while varying some numerical parameters if each choice requires separate runtime compilation. Because of this it is possible to tell via code comments does a parameter need recompilation or not. The relevant subroutines are called `pushpars2c`. The following has examples for boolean and real variables where the former would force recompilation and the latter not.

```
call copy_addr(lklein_gordon,p_par(1)) ! bool
call copy_addr(wfreeze,p_par(2))
call copy_addr(lklein_gordon,p_par(1)) ! bool dconst
call copy_addr(wfreeze,p_par(2)) ! real dconst
```

If control-flow (if statements/other conditionals) depends on a variable it should trigger recompilation so Astaroth can optimise the code. If a variable does not affect control-flow it is often preferable to have the variables not trigger recompilation and it would be helpful to add these cases to the code. So, whenever you are annoyed by the constant need of recompilation because of a certain variable, please do change the variable to not need the recompilation if you have made sure it does not affect control-flow (and is not likely to do so in the future).

5.4.2 Performance tips

To hide the cost of communication as much as possible, it is preferable to have as large subdomains as possible. Commonly used subdomain (= per-process) sizes for accelerated setups are of the order of 256^3 or bigger. Another aspect worth considering is the cadence of the diagnostic outputs. Since the diagnostics are performed on the CPUs each diagnostic output needs to copy the mesh from the device, which is on most hardware is quite slow, and has to compute the diagnostics themselves on the slower hardware of CPUs and/or perform I/O, which is even slower. Thus an overly small cadence of diagnostic output will prevent any speedup. With a high enough cadence the diagnostics are computed/stored alongside the time integration and the time required for them is completely hidden. This is especially doing for power spectra, which requires a lot of communication and thus cannot be totally hidden since it competes with the time integration for access to the network.

5.5 Using a non-equidistant grid

We introduce a non-equidistant grid z_i (by now, this is also implemented for the other directions) as a function $z(\zeta)$ of an equidistant grid ζ_i with grid spacing $\Delta\zeta = 1$.

The way the parameters are handled, the box size and position are *not* changed when you switch to a non-equidistant grid, i.e., they are still determined by `xyz0` and `Lxyz`.

The first and second derivatives can be calculated by

$$\frac{df}{dz} = \frac{df}{d\zeta} \frac{d\zeta}{dz} = \frac{1}{z'} f'(\zeta), \quad \frac{d^2f}{dz^2} = \frac{1}{z'^2} f''(\zeta) - \frac{z''}{z'^3} f'(\zeta), \quad (1)$$

which can be written somewhat more compactly using the inverse function $\zeta(z)$:

$$\frac{df}{dz} = \zeta'(z) f'(\zeta), \quad \frac{d^2f}{dz^2} = \zeta'^2(z) f''(\zeta) + \zeta''(z) f'(\zeta). \quad (2)$$

Internally, the code uses the quantities $dz_1 \equiv 1/z' = \zeta'(z)$ and $dz_tilde \equiv -z''/z'^2 = \zeta''/\zeta'$, and stores them in ‘data/procN/grid.dat’.

The parameters *lequidist* (a 3-element logical array), *grid_func*, *coeff_grid* (a ≥ 2 -element real array) are used to choose a non-equidistant grid and define the function $z(\zeta)$. So far, one can choose between `grid_function='sinh'`, `grid_function='linear'` (which produces an equidistant grid for testing purposes), and `grid_function='step-linear'`.

The sinh profile: For `grid_function='sinh'`, the function $z(\zeta)$ is given by

$$z(\zeta) = z_0 + L_z \frac{\sinh a(\zeta - \zeta_*) + \sinh a(\zeta_* - \zeta_1)}{\sinh a(\zeta_2 - \zeta_*) + \sinh a(\zeta_* - \zeta_1)}, \quad (3)$$

where z_0 and $z_0 + L_z$ are the lowest and uppermost levels, ζ_1, ζ_2 are the ζ values representing those levels (normally $\zeta_1 = 0, \zeta_2 = N_z - 1$ for a grid of N_z vertical layers [excluding ghost layers]), and ζ_* is the ζ value of the inflection point of the sinh function. The z coordinate and ζ value of the inflection point are related via

$$z_* = z_0 + L_z \frac{\sinh a(\zeta_* - \zeta_1)}{\sinh a(\zeta_2 - \zeta_*) + \sinh a(\zeta_* - \zeta_1)}, \quad (4)$$

which can be inverted (“after some algebra”) to

$$\zeta_* = \frac{\zeta_1 + \zeta_2}{2} + \frac{1}{a} \operatorname{artanh} \left[\left(2 \frac{z_* - z_0}{L_z} - 1 \right) \tanh a \frac{\zeta_2 - \zeta_1}{2} \right]. \quad (5)$$

General profile: For a general monotonic function $\psi()$ instead of sinh we get,

$$z(\zeta) = z_0 + L_z \frac{\psi[a(\zeta - \zeta_*)] + \psi[a(\zeta_* - \zeta_1)]}{\psi[a(\zeta_2 - \zeta_*)] + \psi[a(\zeta_* - \zeta_1)]}, \quad (6)$$

and the reference point ζ_* is found by inverting

$$z_* = z_0 + L_z \frac{\psi[a(\zeta_* - \zeta_1)]}{\psi[a(\zeta_2 - \zeta_*)] + \psi[a(\zeta_* - \zeta_1)]}, \quad (7)$$

numerically.

Duct flow: The profile function `grid_function='duct'` generates a grid profile for turbulent flow in ducts. For a duct flow, most gradients are steepest close to the walls, and hence very fine resolution is required near the walls. Here we follow the method of [29] and use a Chebyshev-type grid with a cosine distribution of the grid points such that in the y direction they are located at

$$y_j = h \cos \theta_j, \quad (8)$$

where

$$\theta_j = \frac{(N_y - j) \pi}{N_y - 1}, \quad j = 1, 2, \dots, N_y \quad (9)$$

and $h = L_y/2$ is the duct half-width.

Currently this method is only adapted for ducts where x is the stream-wise direction, z is in the span-wise direction and the walls are at $y = y_0$ and $y = y_0 + L_y$.

In order to have fine enough resolution, the first grid point should be a distance $\delta = 0.05 l_w$ from the wall, where

$$l_w = \frac{\nu}{u_\tau}, \quad u_\tau = \sqrt{\frac{\tau_w}{\rho}}, \quad (10)$$

and τ_w is the shear wall stress. This is accomplished by using at least

$$N_y \geq N_y^* = \frac{\pi}{\arccos\left(1 - \frac{\delta}{h}\right)} + 1 \quad (11)$$

$$= \pi \sqrt{\frac{h}{2\delta}} + 1 - \frac{\pi}{24} \sqrt{\frac{2\delta}{h}} + O\left[\left(\frac{\delta}{h}\right)^{3/2}\right] \quad (12)$$

grid points in the y -direction. After rounding up to the next integer value, we find that the truncated condition

$$N_y \geq \left\lceil \pi \sqrt{\frac{h}{2\delta}} \right\rceil + 1 \quad (13)$$

(where $\lceil x \rceil$ denotes the *ceiling function*, i.e. the smallest integer equal to, or larger than, x) gives practically identical results.

Example: To apply the sinh profile, you can set the following in ‘`start.in`’ (this example is from ‘`samples/sound-spherical-noequi`’):

```
&init_pars
[...]
```

<code>xyz0 = -2., -2., -2.</code>	<code>! first corner of box</code>
<code>Lxyz = 4., 4., 4.</code>	<code>! box size</code>
<code>lperi = F, F, F</code>	<code>! periodic direction?</code>
<code>lequidist = F, F, T</code>	<code>! non-equidistant grid in z</code>
<code>xyz_star = , , -2.</code>	<code>! position of inflection point</code>
<code>grid_func = , , 'sinh'</code>	<code>! sinh function: linear for small, but</code>
	<code>! exponential for large z</code>

```
coeff_grid = , , 0.5
/
```

The parameter array *coeff_grid* represents z_* and a ; the bottom height z_0 and the total box height L_z are taken from *xyz0* and *Lxyz* as in the equidistant case. The inflection point of the sinh profile (the part where it is linear) is not in the middle of the box, because we have set *xyz_star(3)* (i. e. z_*) to -2 .

5.6 Diagnostic output

Every *it1* time steps (*it1* is a runtime parameter, see Sect. K.3), the code writes monitoring output to *stdout* and, parallel to this, to the file ‘data/time_series.dat’. The variables that appear in this listing and the output format are defined in the file ‘print.in’ and can be changed without touching the code (even while the code is running). A simple example of ‘print.in’ may look like this:

```
t(F10.3)
urms(E13.4)
rhom(F10.5)
oum
```

which means that the output table will contain time t in the first column formatted as (F10.3), followed by the mean squared velocity, *urms* (i.e. $\langle u^2 \rangle^{1/2}$) in the second column with format (E13.4), the average density *rhom* ($\langle \rho \rangle$, which allows to monitor mass conservation) formatted (F10.5) and the kinetic helicity *oum* (that is $\langle \omega \cdot u \rangle$) in the last column with the default format (E10.2).⁵ The corresponding diagnostic output will look like

```
----t-----urms-----rhom-----oum-----
 0.000   4.9643E-03   14.42457  -8.62E-06
 0.032   3.9423E-03   14.42446  -5.25E-06
 0.063   6.8399E-03   14.42449  -3.50E-06
 0.095   1.1437E-02   14.42455  -2.58E-06
 0.126   1.6980E-02   14.42457  -1.93E-06
```

5.7 Data files

5.7.1 Snapshot files

Snapshot files contain the values of all evolving variables and are sufficient to restart a run. In the case of an MHD simulation with entropy equation, for example, the snapshot files will contain the values of velocity, logarithmic density, entropy and the magnetic vector potential.

There are two kinds of snapshot files: the current snapshot and permanent snapshots, both of which reside in the directory ‘data/proc*N*/’. The parameter *isav* determines the frequency at which the *current snapshot* ‘data/proc*N*/var.dat’ is written. If you keep this frequency too high, the code will spend a lot of time on I/O, in particular for large jobs; too low a frequency makes it difficult to follow the evolution interactively during test runs. There is also the *ialive* parameter. Setting this to 1 or 10 gives an updated timestep in the files ‘data/proc*/alive.info’. You can put *ialive*=0 to turn this off to limit the I/O on your machine.

⁵The format specifiers are like in Fortran, apart from the fact that the E format will use standard scientific format, corresponding to the Fortran 1pE syntax. Seasoned Fortran IV programmers may use formats like (0pE13.4) to enjoy nostalgic feelings, or (1pF10.5) if they depend on getting wrong numbers.

The *permanent snapshots* ‘data/proc*/VAR*N*’ are written every *dsnap* time units. These files are numbered consecutively from $N = 1$ upward and for long runs they can occupy quite some disk space. On the other hand, if after a run you realize that some additional quantity q would have been important to print out, these files are the only way to reconstruct the time evolution of q without re-running the code.

File structure Snapshot files consist of the following *Fortran records*⁶:

1. variable vector f [$m_x \times m_y \times m_z \times nvar$]
2. time t [1], coordinate vectors x [m_x], y [m_y], z [m_z], grid spacings δx [1], δy [1], δz [1], shearing-box shift Δy [1]

All numbers (apart from the record markers) are single precision (4-byte) floating point numbers, unless you use double precision (see §5.22, in which case all numbers are 8-byte floating point numbers, while the record markers remain 4-byte integers).

The script `pc_tsnap` allows you to determine the time t of a snapshot file:

```
unix> pc_tsnap data/proc0/var.dat
data/proc0/var.dat:          t = 8.32456
unix> pc_tsnap data/proc0/VAR2
data/proc0/VAR2:            t = 2.00603
```

5.8 Video files and slices

We use the terms *video files* and *slice files* interchangeably. These files contain a time series of values of one variable in a given plane. The output frequency of these video snapshots is set by the parameter *dvid* (in code time units).

When output to video files is activated by some settings in ‘run.in’ (see example below) and the existence of ‘video.in’, slices are written for four planes:

1. x - z plane (y index iy ; file suffix `.xz`)
2. y - z plane (y index ix ; suffix `.yz`)
3. x - y plane (y index iz ; suffix `.xy`)
4. another slice parallel to the x - y plane (y index $iz2$; suffix `.xy2`)

You can specify the position of the slice planes by setting the parameters ix , iy , iz and $iz2$ in the namelist *run_pars* in ‘run.in’. Alternatively, you can set the input parameter *slice_position* to one of ‘p’ (periphery of box) or ‘m’ (middle of box). Or you can also specify the z -position in terms of z using the tags *zbot_slice* and *ztop_slice*. In this case, the *zbot_slice* slice will have the suffix `.xy` and the *ztop_slice* the suffix `.xy2`

In the file ‘video.in’ of your run directory, you can choose for which variables you want to get video files; valid choices are listed in § K.5.

The *slice files* are written in each processor directory ‘data/proc*/’ and have a file name indicating the individual variable (e.g. ‘slice_uu1.yz’ for a slice of u_x in the y - z plane). Before visualizing slices one normally wants to combine the sub-slices written by each processor into one global slice (for each plane and variable). This is done by running

⁶A *Fortran record* is marked by the 4-byte integer byte count of the data in the record at the beginning and the end, i.e. has the form $\langle N_{\text{bytes}}, \text{raw_data}, N_{\text{bytes}} \rangle$

'src/read_videofiles.x', which will prompt for the variable name, read the individual sub-slices and write global slices to 'data/'. Once all global slices have been assembled you may want to remove the local slices 'data/proc*/slice*'.

To read all sub-slices demanded in 'video.in' at once use 'src/read_all_videofiles.x'. This program doesn't expect any user input and can thus be submitted in computing queues.

For visualization of slices, you can use the *IDL* routines 'rvid_box.pro', 'rvid_plane.pro', or 'rvid_line.pro' which allows the flag '/png' for writing *PNG* images that can then be combined into an *MPEG* movie using *mpeg.encode*. Based on 'rvid_box', you can write your own video routines in *IDL*.

An example Suppose you have set up a run using *entropy.f90* and *magnetic.f90* (most probably together with *hydro.f90* and other modules). In order to animate slices of entropy s and the z -component B_z of the magnetic field, in planes passing through the center of the box, do the following:

1. Write the following lines to 'video.in' in your run directory:

```
ss
bb
divu
```

2. Edit the namelist *run_pars* in the file 'run.in'. Request slices by setting *write_slices* and set *dvid* and *slice_position* to reasonable values, say

```
!lwrite_slices=T !(no longer works; write requested slices into video.in)
dvid=0.05
slice_position='m'
```

3. Run the PENCIL CODE:

```
unix> start.csh
unix> run.csh
```

4. Say 'make read_videofiles' to compile the routine and then run 'src/read_videofiles.x' to assemble global slice files from those scattered across 'data/proc*':

```
unix> src/read_videofiles.x
  enter name of variable (lnrho, uu1, ..., bb3):  ss
unix> src/read_videofiles.x
  enter name of variable (lnrho, uu1, ..., bb3):  bb3
```

5. Start *IDL* and run 'rvid_box':

```
unix> idl
IDL> rvid_box,'bb3'
IDL> rvid_box,'ss',min=-0.3,max=2.
```

etc.

Another example Suppose you have set up a run using *magnetic.f90* and some other modules. This run studies some process in a "surface" layer inside the box. This "surface"

can represent a sharp change in density or turbulence. So you defined your box setting the $z = 0$ point at the surface. Therefore, your ‘start.in’ file will look something similar to:

```
&init_pars
  lperi=T,T,F
  bcz = 's','s','a','hs','s','s','a'
  xyz0 = -3.14159, -3.14159, -3.14159
  Lxyz = 6.28319, 6.28319, 9.42478
```

A smarter way of specifying the box size in units of π is to write

```
&init_pars
  xyz_units = 'pi', 'pi', 'pi'
  xyz0 = -1., -1., -1.
  Lxyz = 2., 2., 2.
```

Now you can analyze quickly the surface of interest and some other xy slice setting *zbot_slice* and *ztop_slice* in the ‘run.in’ file:

```
&run_pars
  slice_position='c'
  zbot_slice=0.
  ztop_slice=0.2
```

In this case, the slices with the suffix .xy will be at the “surface” and the ones with the suffix .xy2 will be at the position $z = 0.2$ above the surface. And you can visualize this slices by:

1. Write the following lines to ‘video.in’ in your run directory:

```
bb
```

2. Edit the namelist *run_pars* in the file ‘run.in’ to include *zbot_slice* and *ztop_slice*.
3. Run the PENCIL CODE:

```
unix> start.csh
unix> run.csh
```

4. Run ‘src/read_videofiles.x’ to assemble global slice files from those scattered across ‘data/proc*/’:

```
unix> src/read_videofiles.x
      enter name of variable (lnrho, uu1, ..., bb3):  bb3
```

5. Start *IDL*, load the slices with ‘pc_read_video’ and plot them at some time:

```
unix> idl
IDL> pc_read_video, field='bb3',ob=bb3,nt=ntv
IDL> tvscl,bb3.xy(*,*,100)
IDL> tvscl,bb3.xy2(*,*,100)
```

etc.

File structure *Slice files* consist of one Fortran record (see footnote on page 27) for each slice, which contains the data of the variable (without ghost zones), the time t of the snapshot and the position of the sliced variable (e. g. the x position for a y - z slice):

1. $\text{data}_1 [nx \times ny \times nz]$, time $t_1 [1]$, position₁ [1]
2. $\text{data}_2 [nx \times ny \times nz]$, time $t_2 [1]$, position₂ [1]
3. $\text{data}_3 [nx \times ny \times nz]$, time $t_3 [1]$, position₃ [1]

etc.

5.9 Averages

5.9.1 One-dimensional output averaged in two dimensions

In the file ‘xyaver.in’, z -dependent (horizontal) averages are listed. They are written to the file ‘data/xyaverages.dat’. A new line of averages is written every $it1$ th time steps.

There is the possibility to output two-dimensional averages. The result then depends on the remaining dimension. The averages are listed in the files ‘xyaver.in’, ‘xzaver.in’, and ‘yzaver.in’ where the first letters indicate the averaging directions. The output is then stored to the files ‘data/xyaverages.dat’, ‘data/xzaverages.dat’, and ‘data/yzaverages.dat’. The output is written every $it1d$ th time steps.

The rms values of the so defined mean magnetic fields are referred to as bmz , bmy and bmz , respectively, and the rms values of the so defined mean velocity fields are referred to as umz , umy , and umx . (The last letter indicates the direction on which the averaged quantity still depends.)

See Sect. 9.2 on how to add new averages.

In id1 such xy -averages can be read using the procedure ‘pc_read_xyaver’.

5.9.2 Two-dimensional output averaged in one dimension

There is the possibility to output one-dimensional averages. The result then depends on the remaining two dimensions. The averages are listed in the files ‘yaver.in’, ‘zaver.in’, and ‘phiaver.in’ where the first letter indicates the averaging direction. The output is then stored to the files ‘data/yaverages.dat’, ‘data/zaverages.dat’, and ‘data/phiaverages.dat’.

See Sect. 9.2 on how to add new averages.

Disadvantage: The output files, e.g., ‘data/zaverages.dat’, can be rather big because each average is just appended to the file.

5.9.3 Azimuthal averages

Azimuthal averages are controlled by the file ‘phiaver.in’, which currently supports the quantities listed in Sect. K.6. In addition, one needs to set `lwrite_phiaverages`, `lwrite_yaverages`, or `lwrite_zaverages` to `.true.`. For example, if ‘phiaver.in’ contains the single line

```
b2mphi
```

then you will get azimuthal averages of the squared magnetic field B^2 .

Azimuthal averages are written every *d2davg* time units to the files ‘data/averages/PHIAVG*N*’. The file format of azimuthal-average files consists of the following *Fortran records*:

1. number of radial points $N_{r,\phi\text{-avg}}$ [1], number of vertical points $N_{z,\phi\text{-avg}}$ [1], number of variables $N_{\text{var},\phi\text{-avg}}$ [1], number of processors in z direction [1]
2. time t [1], positions of cylindrical radius r_{cyl} [$N_{r,\phi\text{-avg}}$] and z [$N_{z,\phi\text{-avg}}$] for the grid, radial spacing δr_{cyl} [1], vertical spacing δz [1]
3. averaged data [$N_{r,\phi\text{-avg}} \times N_{z,\phi\text{-avg}}$]
4. label length [1], labels of averaged variables [$N_{\text{var},\phi\text{-avg}}$]

All numbers are 4-byte numbers (floating-point numbers or integers), unless you use double precision (see §5.22).

To read and visualize azimuthal averages in *IDL*, use ‘\$PENCIL_HOME/idl/files/pc_read_phiavg.pro’

```
IDL> avg = pc_read_phiavg('data/averages/PHIAVG1')
IDL> contour, avg.b2mphi, avg.rcyl, avg.z, TITLE='!17B!U2!N!X'
```

or have a look at ‘\$PENCIL_HOME/idl/phiavg.pro’ for a more sophisticated example.

5.9.4 Time averages

Time averages need to be prepared in the file ‘src/ctimeavg.local’, since they use extra memory. They are controlled by the averaging time τ_{avg} (set by the parameter *tavg* in ‘run.in’), and by the indices *idx_tavg* of variables to average.

Currently, averaging is implemented as exponential (memory-less) average,⁷

$$\langle f \rangle_{t+\delta t} = \langle f \rangle_t + \frac{\delta t}{\tau_{\text{avg}}} [f(t+\delta t) - \langle f \rangle_t], \quad (16)$$

which is equivalent to

$$\langle f \rangle_t = \int_{t_0}^t e^{-(t-t')/\tau_{\text{avg}}} f(t') dt'. \quad (17)$$

Here t_0 is the time of the snapshot the calculation started with, i.e. the snapshot read by the last *run.x* command. Note that the implementation (16) will approximate Eq. (17) only to first-order accuracy in δt . In practice, however, δt is small enough to make this accuracy suffice.

In ‘src/ctimeavg.local’, you need to set the number of slots used for time averages. Each of these slots uses $m_x \times m_y \times m_z$ floating-point numbers, i.e. half as much memory as each fundamental variable.

⁷At some point we may also implement the more straight-forward average

$$\langle f \rangle_{t+\delta t} = \langle f \rangle_t + \frac{\delta t}{t-t_0+\delta t} [f(t+\delta t) - \langle f \rangle_t], \quad (14)$$

which is equivalent to

$$\langle f \rangle_t = \frac{1}{t-t_0} \int_{t_0}^t f(t') dt', \quad (15)$$

but we do not expect large differences.

For example, if you want to get time averages of all variables, set

```
integer, parameter :: mtavg=mvar
```

in 'src/ctimeavg.local', and don't set *idx_tavg* in 'run.in'.

If you are only interested in averages of variables 1–3 and 6–8 (say, the velocity vector and the magnetic vector potential in a run with 'hydro.f90', 'density.f90', 'entropy.f90' and 'magnetic.f90'), then set

```
integer, parameter :: mtavg=6
```

in 'src/ctimeavg.local', and set

```
idx_tavg = 1,2,3,6,7,8      ! time-average velocity and vector potential
```

in 'run.in'.

Permanent snapshots of time averages are written every *tavg* time units to the files 'data/proc*/TAVN'. The current time averages are saved periodically in 'data/proc*/timeavg.dat' whenever 'data/proc*/var.dat' is written. The file format for time averages is equivalent to that of the snapshots; see § 5.7.1 above.

5.10 Helper scripts

The 'bin' directory contains a collection of utility scripts, some of which are discussed elsewhere. Here is a list of the more important ones.

adapt-mkfile Activate the settings in a 'Makefile' that apply to the given computer, see § 5.2.

auto-test Verify that the code compiles and runs in a set of run directories and compare the results to the reference output. These tests are carried out routinely to ensure that the svn version of the code is in a usable state.

cleanf95 Can be used to clean up the output from the Intel x86 Fortran 95 compiler (ifc).

copy-proc-to-proc Used for restarting in a different directory. Example
copy-proc-to-proc seed.dat ../hydro256e.

copy-snapshots Copy snapshots from a processor-local directory to the global directory. To be started in the background before 'run.x' is invoked. Used by 'start.csh' and 'run.csh' on network connected processors.

pc_copyvar var1 var2 source dest Copies snapshot files from one directory (source) to another (dest). See documentation in file.

pc_copyvar v v dir Copies all 'var.dat' files from current directory to 'var.dat' in 'dir' run directory. Used for restarting in a different directory.

pc_copyvar N v Used to restart a run from a particular snapshot 'VARN'. Copies a specified snapshot 'VARN' to 'var.dat' where *N* and (optionally) the target run directory are given on the command line.

cvs-add-rundir Add the current run directory to the svn repository.

cvsci_run Similar to cvs-add-rundir, but it also checks in the '*.in' and 'src/*.local' files. It also checks in the files 'data/time_series.dat', 'data/dim.dat' and

`'data/index.pro'` for subsequent processing in *IDL* on another machine. This is particularly useful if collaborators want to check each others' runs.

dx_* These script perform several data collection or reformatting exercises required to read particular files into *DX*. They are called internally by some of the *DX* macros in the `'dx/macros/'` directory.

getconf.csh See § 4.3.5

gpgrowth Plot simple time evolution with Gnuplot's ASCII graphics for fast orientation via a slow modem line.

local Materialize a symbolic link

mkparam Based on `'Makefile'` and `'Makefile.local'`, generate `'src/cparam.inc'`, which specifies the number *mvar* of fundamental variables, and *maux* of auxiliary variables. Called by the `'Makefile'`.

pc_mkdatadir Creates a link to a data directory in a suitable workspace. By default this is on `'/var/tmp/'`, but different locations are specified for different machines.

mkdotin Generate minimal `'start.in'`, `'run.in'` files based on `'Makefile'` and `'Makefile.local'`.

mkinpars Wrapper around `'mkdotin'` — needs proper documentation.

mkproc-tree Generates a multi-processor(`'procN/'`), directory structure. Useful when copying data files in a processor tree, such as slice files.

mkwww Generates a template HTML file for describing a run of the code, showing input parameters and results.

move-slice Moves all the slice files from a processor tree structure, `'procN/'`, to a new target tree creating directories where necessary.

nl2idl Transform a Fortran *namelist* (normally the files `'param.nml'`, `'param2.nml'` written by the code) into an *IDL* structure. Generates an *IDL* file that can be sourced from `'start.pro'` or `'run.pro'`.

pacx-adapt-makefile Version of adapt-makefile for highly distributed runs using PACX MPI.

pc_newrun Generates a new run directory from an old one. The new one contains a copy of the old `'*.local'` files, runs `pc_setupsrsc`, and makes also a copy of the old `'*.in'` and `'k.dat'` files.

pc_newscan Generates a new scan directory from an old one. The new one contains a copy of the old, e.g., the one given under `'samples/parameter_scan'`. Look in the `'README'` file for details.

pc_inspectrun Check the execution of the current run: prints legend and the last few lines of the `'time_series.dat'` file. It also appends this result to a file called `'SPEED'`, which contains also the current wall clock time, so you can work out the speed of the code (without being affected by i/o time).

read_videofiles.csh The script for running `read_videofiles.x`.

remote-top Create a file ‘top.log’ in the relevant ‘proc*N*/’ directory containing the output of top for the appropriate processor. Used in batch scripts for multi-processor runs.

run.csh The script for producing restart files with the initial condition; see § 4.3.5

scpdatadir Make a tarball of data directory, ‘data/’ and use scp to secure copy to copy it to the specified destination.

pc_setupsrc Link ‘start.csh’, ‘run.csh’ and ‘getconf.csh’ from ‘\$PENCIL_HOME/bin’. Generate ‘src/’ if necessary and link the source code files from ‘\$PENCIL_HOME/src’ to that directory.

start.csh The script for initializing the code; see § 4.3.5

summarize-history Evaluate ‘params.log’ and print a history of changes.

timestr Generate a unique time string that can be appended to file names from shell scripts through the backtick mechanism.

pc_tsnap Extract time information from a snapshot file, ‘VAR*N*’.

There are several additional scripts on ‘pencil-code/utils’. Some are located in separate folders according to users. There could be redundancies, but it is often just as easy to write your own new script than figuring out how something else works.

5.11 RELOAD, STOP and SAVE files

The code periodically (every *it* time steps) checks for the existence of two files, ‘RELOAD’ and ‘STOP’, which can be used to trigger certain behavior.

Reloading run parameters In the directory where you started the code, create the file ‘RELOAD’ with

```
unix> touch RELOAD
```

to force the code to re-read the runtime parameters from ‘run.in’. This will happen the next time the code is writing monitoring output (the frequency of this happening is controlled by the input parameter *it*, see Sect. 5.13).

Each time the parameters are reloaded, the new set of parameters is appended (in the form of *namelists*) to the file ‘data/params.log’ together with the time *t*, so you have a full record of your changes. If ‘RELOAD’ contains any text, its first line will be written to ‘data/params.log’ as well, which allows you to annotate changes:

```
unix> echo "Reduced eta to get fields growing" > RELOAD
```

Use the command `summarize-history` to print a history of changes.

Stopping the code In the directory where you started the code, create the file ‘STOP’ with

```
unix> touch STOP
```

to stop the code in a controlled manner (it will write the latest snapshot). Again, the action will happen the next time the code is writing monitoring output.

Saving a snapshot In the directory where you started the code, create the file ‘SAVE’ with

```
unix> touch SAVE
```

to save the current state of the simulation in the file `var.dat`. See **Stopping the code** for when this action is taken.

5.12 RERUN and NEWDIR files

After the code finishes (e.g., when the final timestep number is reached or when a ‘STOP’ file is found), the ‘`run.csh`’ script checks whether there is a ‘RERUN’ file. If so, the code will simply run again, perhaps even after you have recompiled the code. This is useful in the development phase when you changed something in the code, so you don’t need to wait for a new slot in the queue!

Even more naughty, as Tony says, is the ‘NEWDIR’ file, where you can enter a new directory path (relative path is ok, e.g., `../conv-slab`). If nothing is written in this file (e.g., via `touch NEWDIR`) it stays in the same directory. On distributed machines, the ‘NEWDIR’ method will copy all the ‘VAR#’ and ‘`var.dat`’ files back to and from the sever. This can be useful if you want to run with new data files, but you better do it in a separate directory, because with ‘NEWDIR’ the latest data from the code are written back to the server before running again.

Oh, by the way, if you want to be sure that you haven’t messed up the content of the pair of ‘NEWDIR’ files, you may want to try out the `pc_jobtransfer` command. It writes the decisive ‘STOP’ file only after the script has checked that the content of the two ‘NEWDIR’ files points to existing run directory paths, so if the new run crashes, the code returns safely to the old run directory. I’m not sure what Tony would say now, but this is now obviously extremely naughty.

5.13 Start and run parameters

All input parameters in ‘`start.in`’ and ‘`run.in`’ are grouped in Fortran *namelists*. This allows arbitrary order of the parameters (*within* the given namelist; the namelists need no longer be in the correct order), as well as enhanced readability through inserted Fortran comments and whitespace. One namelist (*init_pars* / *run_pars*) contains general parameters for initialization/running and is always read in. All other namelists are specific to individual modules and will only be read if the corresponding module is used.

The syntax of a namelist (in an input file like ‘`start.in`’) is

```
&init_pars
  ip=5, Lxyz=2,4,2
/
```

— in this example, the name of the namelist is *init_pars*, and we read just two variables (all other variables in the namelist retain their previous value): *ip*, which is set to 5, and *Lxyz*, which is a vector of length three and is set to (2, 4, 2).

While all parameters from the namelists can be set, in most cases reasonable default values are preset. Thus, the typical file *start.in* will only contain a minimum set of variables or (if you are *very* minimalistic) none at all. If you want to run a particular problem, it is best to start by modifying an existing example that is close to your application.

Before starting a simulation run, you may want to execute the command *pc_configtest* in order to test the correctness of your changes to these configuration files.

As an example, we give here the start parameters for ‘samples/helical-MHDTurb’

```
&init_pars
  cvsid='$Id:$',                ! identify version of start.in
  xyz0 = -3.1416, -3.1416, -3.1416, ! first corner of box
  Lxyz = 6.2832, 6.2832, 6.2832, ! box size
  lperi = T      , T      , T      , ! periodic in x, y, z
  random_gen='nr_f90'
/
&hydro_init_pars
/
&density_init_pars
  gamma=1.
/
&magnetic_init_pars
  initaa='gaussian-noise', amplaa=1e-4
/
```

The three entries specifying the location, size and periodicity of the box are just given for demonstration purposes here — in fact a periodic box from $-\pi$ to π in all three directions is the default. In this run, for reproducibility, we use a random number generator from the Numerical Recipes [39], rather than the compiler’s built-in generator. The adiabatic index γ is set explicitly to 1 (the default would have been 5/3) to achieve an isothermal equation of state. The magnetic vector potential is initialized with uncorrelated, normally distributed random noise of amplitude 10^{-4} .

The run parameters for ‘samples/helical-MHDTurb’ are

```
&run_pars
  cvsid='$Id:$',                ! identify version of start.in
  nt=10, it1=2, cdt=0.4, cdtv=0.80, isave=10, itorder=3
  dsnap=50, dvid=0.5
  random_gen='nr_f90'
/
&hydro_run_pars
/
&density_run_pars
/
&forcing_run_pars
  iforce='helical', force=0.07, relhel=1.
/
```

```

&magnetic_run_pars
  eta=5e-3
/
&viscosity_run_pars
  nu=5e-3
/

```

Here we run for $nt = 10$ timesteps, every second step, we write a line of diagnostic output; we require the time step to keep the advective *Courant number* ≤ 0.4 and the diffusive *Courant number* ≤ 0.8 , save ‘var.dat’ every 20 time steps, and use the 3-step time-stepping scheme described in Appendix H.4 (the Euler scheme *itorder* = 1 is only useful for tests). We write permanent snapshot file ‘VAR.N’ every *dsnap* = 50 time units and 2d slices for animation every *dvid* = 0.5 time units. Again, we use a deterministic random number generator. Viscosity ν and magnetic diffusivity η are set to 5×10^{-3} (so the mesh Reynolds number *based on the rms velocity* is about $u_{\text{rms}} \delta x / \nu = 0.3 \times (2\pi/32) / 5 \times 10^{-3} \approx 12$, which is in fact rather a bit to high). The parameters in *forcing_run_pars* specify fully helical forcing of a certain amplitude.

A full list of input parameters is given in Appendix K.

5.14 Physical units

Many calculations are unit-agnostic, in the sense that all results remain the same independent of the unit system in which you interpret the numbers. E.g. if you simulate a simple hydrodynamical flow in a box of length $L = 1$. and get a maximum velocity of $u_{\text{max}} = 0.5$ after $t = 3$ time units, then you may interpret this as $L = 1$ m, $u_{\text{max}} = 0.5$ m/s, $t = 3$ s, or as $L = 1$ pc, $u_{\text{max}} = 0.5$ pc/Myr, $t = 3$ Myr, depending on the physical system you have in mind. The units you are using must of course be consistent, thus in the second example above, the units for diffusivities would be pc²/Myr, etc.

The units of magnetic field and temperature are determined by the values $\mu_0 = 1$ and $c_p = 1$ used internally by the code⁸. This means that if your units for density and velocity are $[\rho]$ and $[v]$, then magnetic fields will be in

$$[B] = \sqrt{\mu_0 [\rho] [v]^2}, \quad (18)$$

and temperatures are in

$$[T] = \frac{[v]^2}{c_p} = \frac{\gamma-1}{\gamma} \frac{[v]^2}{\mathcal{R}/\mu}. \quad (19)$$

For some choices of density and velocity units, Table 2 shows the resulting units of magnetic field and temperature.

On the other hand, as soon as material equations are used (e.g., one of the popular parameterizations for radiative losses, Kramers opacity, Spitzer conductivities or ionization, which implies well-defined ionization energies), the corresponding routines in the code need to know the units you are using. This information is specified in ‘start.in’ or ‘run.in’ through the parameters *unit_system*, *unit_length*, *unit_velocity*, *unit_density* and *unit_temperature*⁹ like e. g.

⁸Note that $c_p = 1$ is only assumed if you use the module *noionization.f90*. If you work with *ionization.f90*, temperature units are specified by *unit_temperature* as described below.

⁹Note: the parameter *unit_temperature* is currently only used in connection with *ionization.f90*. If you are working with *noionization.f90*, the temperature unit is completely determined by Eq. (19) above.

Table 2: Units of magnetic field and temperature for some choices of $[\rho]$ and $[v]$ according to Eqs. (18) and (19). Values are for a monatomic gas ($\gamma = 5/3$) of mean atomic weight $\bar{\mu}_g = \bar{\mu}/1$ g in grams.

$[\rho]$	$[v]$	$[B]$	$[T]$
1 kg/m ³	1 m/s	1.12 mT = 11.2 G	$\left(\frac{\bar{\mu}_g}{0.6}\right) \times 2.89 \times 10^{-5}$ K
1 g/cm ³	1 cm/s	3.54×10^{-4} T = 3.54 G	$\left(\frac{\bar{\mu}_g}{0.6}\right) \times 2.89$ nK
1 g/cm ³	1 km/s	35.4 T = 354 kG	$\left(\frac{\bar{\mu}_g}{0.6}\right) \times 28.9$ K
1 g/cm ³	10 km/s	354 T = 3.54 MG	$\left(\frac{\bar{\mu}_g}{0.6}\right) \times 2890$ K

```
unit_system='SI',
unit_length=3.09e16, unit_velocity=978. ! [l]=1pc, [v]=1pc/Myr
```

Note: The default unit system is `unit_system='cgs'` which is a synonym for `unit_system='Babylonian cubits'`.

5.15 Minimum amount of viscosity

We emphasize that, by default, the code works with constant diffusion coefficients (viscosity ν , thermal diffusivity χ , magnetic diffusivity η , or passive scalar diffusivity $\mathcal{D}$). If any of these numbers is too small, you would need to have more meshpoints to get consistent numerical solutions; otherwise the code develops wiggles (‘ringing’) and will eventually crash. A useful criterion is given by the mesh Reynolds number *based on the maximum velocity*,

$$\text{Re}_{\text{mesh}} = \max(|\mathbf{u}|) \max(\delta x, \delta y, \delta z) / \nu, \quad (20)$$

which should not exceed a certain value which can be problem-dependent. Often the largest possible value of Re_{mesh} is around 5. Similarly there exist mesh Péclet and mesh magnetic Reynolds numbers that should not be too large.

Note that in some cases, ‘wiggles’ in $\ln \rho$ will develop despite sufficiently large diffusion coefficients, essentially because the continuity equation contains no dissipative term. For convection runs (but not only for these), we have found that this can often be prevented by *upwinding*, see Sect. H.2.

If the Mach number of the code approaches unity, i.e. if the rms velocity becomes comparable with the speed of sound, shocks may form. In such a case the mesh Reynolds number should be smaller. In order to avoid excessive viscosity in the unshocked regions, one can use the so-called shock viscosity (Sect. 6.6.1) to concentrate the effects of a low mesh Reynolds number to only those areas where it is necessary.

5.16 The time step

5.16.1 The usual RK-2N time step

RK-2N refers to the third order Runge-Kutta scheme by Williamson (1980) with a memory consumption of two chunks. Therefore the 2N in the name.

The time step is normally specified as Courant time step through the coefficients `cdt`

$(c_{\delta t})$, $cdtv(c_{\delta t,v})$ and $cdts(c_{\delta t,s})$. The resulting Courant step is given by

$$\delta t = \min \left(c_{\delta t} \frac{\delta x_{\min}}{U_{\max}}, c_{\delta t,v} \frac{\delta x_{\min}^2}{D_{\max}}, c_{\delta t,s} \frac{1}{H_{\max}} \right), \quad (21)$$

where

$$\delta x_{\min} \equiv \min(\delta x, \delta y, \delta z); \quad (22)$$

$$U_{\max} \equiv \max \left(|\mathbf{u}| + \sqrt{c_s^2 + v_A^2} \right), \quad (23)$$

c_s and v_A denoting sound speed and Alfvén speed, respectively;

$$D_{\max} = \max(\nu, \gamma\chi, \eta, D), \quad (24)$$

where ν denotes kinematic viscosity, $\chi = K/(c_p\rho)$ thermal diffusivity and η the magnetic diffusivity; and

$$H_{\max} = \max \left(\frac{2\nu \mathbf{S}^2 + \zeta_{\text{shock}} (\nabla \cdot \mathbf{u})^2 + \dots}{c_v T} \right), \quad (25)$$

where dots indicate the presence of other terms on the rhs of the entropy equation.

To fix the time step δt to a value independent of velocities and diffusivities, explicitly set the run parameter dt , rather than cdt or $cdtv$ (see p. 203).

If the time step exceeds the viscous time step the simulation may actually run ok for quite some time. Inspection of images usually helps to recognize the problem. An example is shown in Fig. 5.

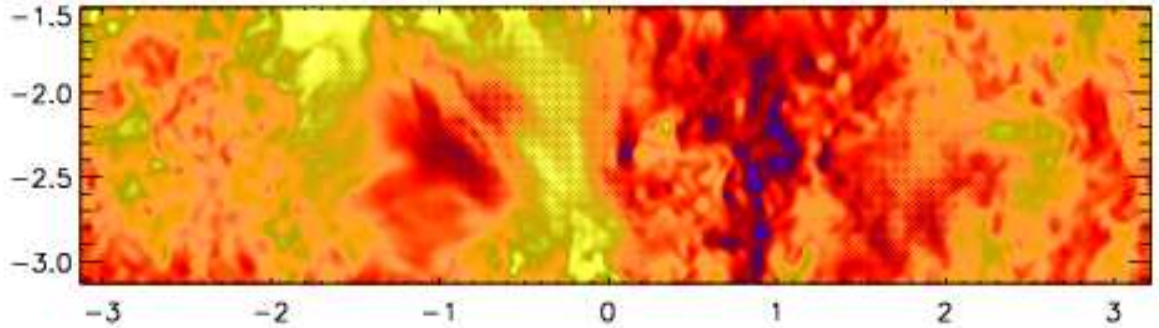


Figure 5: Example of a velocity slice from a run where the time step is too long. Note the spurious checkerboard modulation in places, for example near $x = -0.5$ and $-2.5 < y < -1.5$. This is an example of a hyperviscous turbulence simulations with 512^3 meshpoints and a third order hyperviscosity of $\nu_3 = 5 \times 10^{-12}$. Hyperviscosity is explained in the Appendix E.

Timestepping is accomplished using the Runge-Kutta 2N scheme. Regarding details of this scheme see Sect. H.4.

5.16.2 The Runge-Kutta-Fehlberg time step

A fifth order Runge-Kutta-Fehlberg time stepping procedure is available. It is used mostly for chemistry application, often together with the double precision option. In order to make this work, you need to compile with

```
TIMESTEP = timestep_rkf
```

in ‘src/Makefile.local’. In addition, you must put *itorder=5* in ‘run.in’. An example application is ‘samples/1d-tests/H2_flamespeed’. This procedure is still experimental.

5.17 Boundary conditions

5.17.1 Where to specify boundary conditions

In most tests that come with the PENCIL CODE, boundary conditions are set in `'run.in'`, which is a natural choice. However, this may lead to unexpected initial data written by `'start.x'`, since when you start the code (via `'start.csh'`), the boundary conditions are unknown and `'start.x'` will then fill the ghost zones assuming periodicity (the default boundary condition) in all three directions. These ghost data will never be used in a calculation, as `'run.x'` will apply the boundary conditions before using any ghost-zone values.

To avoid these periodic conditions in the initial snapshot, you can set the boundary conditions in `'start.in'` already. In this case, they will be inherited by `'run.x'`, unless you also explicitly set boundary conditions in `'run.in'`.

5.17.2 How to specify boundary conditions

Boundary conditions are implemented through three layers of *ghost points* on either boundary, which is quite a natural choice for an MPI code that uses ghost zones for representing values located on the neighboring processors anyway. The desired type of boundary condition is set through the parameters $bc\{x,y,z\}$ in `'run.in'`; the nomenclature used is as follows. Set $bc\{x,y,z\}$ to a sequence of letters like

```
bcx = 'p','p','p','p','p'
```

for periodic boundaries, or

```
bcz = 's','s','a','a2','c1:c2'
```

for non-periodic ones. Each element corresponds to one of the variables, which are those of the variables $u_x, u_y, u_z, \ln \rho, s/c_p, A_x, A_y, A_z, \ln c$ that are actually used *in this order*. The following conditions are available:

'p' periodic boundary condition

'a' antisymmetric condition w. r. t. the boundary, i. e. vanishing value

's' symmetric condition w. r. t. the boundary, i. e. vanishing first derivative

'a2' antisymmetry w. r. t. the arbitrary value on the boundary, i. e. vanishing second derivative

'c1' special boundary condition for $\ln \rho$ and s : constant heat flux through the boundary

'c2' special boundary condition for s : constant temperature at the boundary — requires boundary condition a2 for $\ln \rho$

'cT' special boundary condition for s or $\ln T$: constant temperature at the boundary (for arbitrarily set $\ln \rho$)

'ce' special boundary condition for s : set temperature in ghost points to value on boundary (for arbitrarily set $\ln \rho$)

'db' low-order one-sided derivatives ("no boundary condition") for density

'she' shearing-sheet boundary condition (default when the module *Shear* is used)

‘g’ force the value of the corresponding field on vertical boundaries (should be used in combination with the `force_lower_bound` and `force_upper_bound` flags set in the namelist *init_pars*)

‘hs’ special boundary condition for $\ln \rho$ and s which enforces hydrostatic equilibrium on vertical boundaries

The extended syntax *a:b* (e. g. ‘c1:c2’) means: use boundary condition *a* at the left/lower boundary, but *b* at the right/upper one.

If you build a new ‘run.in’ file from another one with a different number of variables (*noentropy* vs. *entropy*, for example), you need to remember to adjust the *length* of the arrays *bcx* to *bcz*. The advantage of the present approach is that it is very easy to exchange all boundary conditions by a new set of conditions in a particular direction (for example, make everything periodic, or switch off shearing sheet boundary conditions and have stress-free instead).

5.18 Restarting a simulation

When a run stops at the end of a simulation, you can just resubmit the job again, and it will start from the latest snapshot saved in ‘data/proc*/var.dat’. The value of the latest time is saved in a separate file, ‘data/proc*/time.dat’. On parallel machines it is possible that some (or just one) of the ‘var.dat’ are corrupt; for example after a system crash. Check for file size and date, and restart from a good ‘VAR’*N* file instead.

If you want to run on a different machine, you just need to copy the ‘data/proc*/var.dat’ (and, just to be sure) ‘data/proc*/time.dat’) files into a new directory tree. You may also need the ‘data/proc*/seed.dat’ files for the random number generator. The easiest way to get all these other files is to run `start.csh` again on the new machine (or in a new directory) and then to overwrite the ‘data/proc*/var.dat’ files with the correct ones.

For restarting from runs that didn’t have magnetic fields, passive scalar fields, or test fields, see Sect. F.4.

5.19 One- and two-dimensional runs

If you want to run two-dimensional problems, set the number of mesh points in one direction to unity, e.g., *nygrid*=1 or *nzgrid*=1 in ‘cparam.local’. Remember that the number of mesh points is still divisible by the number of processors. For 2D-runs, it is also possible to write only 2D-snapshots (i.e. VAR files written only in the considered (x, y) or (x, z) plane, with a size seven times smaller as we do not write the third unused direction). To do that, please add the logical flag `lwrite_2d=T` in the namelist *init_pars* in ‘start.in’.

Similarly, for one-dimensional problems, set, for example, *nygrid*=1 and *nzgrid*=1 in ‘cparam.local’. You can even do a zero-dimensional run, but then you better set *dt* (rather than *cdt*), because there is no Courant condition for the time step.

See *0d*, *1d*, *2d*, and *3d tests* with examples.

5.20 Visualization

5.20.1 Gnuplot

Simple visualization can easily be done using *Gnuplot* (<http://www.gnuplot.info>), an open-source plotting program suitable for two-dimensional plots.

For example, suppose you have the variables

```
---it-----t-----dt-----urms-----umax-----rho-----ssm-----dtc---
```

in ‘time_series.dat’ and want to plot $u_{\text{rms}}(t)$. Just start gnuplot and type

```
gnuplot> plot "data/time_series.dat" using 2:4 with lines
```

If you work over a slow line and want to see both $u_{\text{rms}}(t)$ and $u_{\text{max}}(t)$, use ASCII graphics:

```
gnuplot> set term dump
gnuplot> set logscale y
gnuplot> plot "data/time_series.dat" using 2:4 title "urms", \
gnuplot>          "data/time_series.dat" using 2:5 title "umax"
```

5.20.2 Data explorer

DX (*data explorer*; <http://www.opendx.org>) is an open-source tool for visualization of three-dimensional data.

The PENCIL CODE provides a few networks for *DX*. It is quite easy to read in a snapshot file from *DX* (the only tricky thing is the four extra bytes at the beginning of the file, representing a Fortran record marker), and whenever you run ‘start.x’, the code writes a file ‘var.general’ that tells *DX* all it needs to know about the data structure.

As a starting point for developing your own *DX* programs or *networks*, you can use a few generic *DX* scripts provided in the directory ‘dx/basic/’. From the run directory, start *DX* with

```
unix> dx -edit $PENCIL_HOME/dx/basic/lnrho
```

to load the file ‘dx/basic/lnrho.net’, and execute it with **[Ct]l-o** or **Execute** → **Execute Once**. You will see a set of iso-surfaces of logarithmic density. If the viewport does not fit to your data, you can reset it with **[Ct]l-f**. To rotate the object, drag the mouse over the Image window with the left or right mouse button pressed. Similar networks are provided for entropy (‘ss.net’), velocity (‘uu.net’) and magnetic field (‘bb.net’).

When you expand these simple networks to much more elaborate ones, it is probably a good idea to separate the different tasks (like Importing and Selecting, visualizing velocity, visualizing entropy, and Rendering) onto separate pages through **Edit** → **Page**.

Note Currently, *DX* can only read in data files written by one single processor, so from a multi-processor run, you can only visualize one subregion at a time.

5.20.3 GDL

GDL, also known as *Gnu Data Language* is a free visualization package that can be found at <http://gnudatalanguage.sourceforge.net/>. It aims at replacing the very expensive *IDL* package (see S. 5.20.4). For the way we use *IDL* for the Pencil Code, compatibility is currently not completely sufficient, but you can use *GDL* for many of the

visualization tasks. If you get spurious “Error opening file” messages, you can normally simply ignore them.

This section tells you how to get started with using GDL for visualization.

Setup As of GDL 0.9 – at least the version packed with Ubuntu Jaunty (9.10) – you will need to add GDL’s ‘examples/pro/’ directory to your *!PATH* variable. So the first call after starting *GDL* should be

```
GDL> .run setup_gdl
```

Starting visualization There are mainly two possibilities for visualization: using a simple GUI or loading the data with *pc_read* and work with it interactively. Please note that the GUI was written and tested only with IDL, see § 5.20.4.

Here, the *pc_read* family of IDL routines to read the data is described. Try

```
GDL> pc_read
```

to get an overview.

To plot a time series, use

```
GDL> pc_read_ts, OBJECT=ts
GDL> help, ts, /STRUCT ;; (to see which slots are available)
GDL> plot, ts.t, ts.umax
GDL> oplot, ts.t, ts.urms
```

Alternatively, you could simply use the ‘ts.pro’ script:

```
GDL> .run ts
```

To work with data from ‘var.dat’ and similar snapshot files, you can e.g., use the following routines:

```
GDL> pc_read_dim, OBJECT=dim
GDL> $$PENCIL_HOME/bin/nl2idl -d ./data/param.nml> ./param.pro
GDL> pc_read_param, OBJECT=par
GDL> pc_read_grid, OBJECT=grid
GDL> pc_read_var, OBJECT=var
```

Having thus read the data structures, we can have a look at them to see what information is available:

```
GDL> help, dim, /STRUCT
GDL> help, par, /STRUCT
GDL> help, grid, /STRUCT
GDL> help, var, /STRUCT
```

To visualize data, we can e.g., do¹⁰

```
GDL> plot, grid.x, var.ss[*, dim.ny/2, dim.nz/2]
GDL> contourfill, var.ss[*, *, dim.nz/2], grid.x, grid.y

GDL> ux_slice = var.uu[*, *, dim.nz/2, 0]
```

¹⁰If *contourfill* produces just contour lines instead of a color-coded plot, your version of GDL is too old. E.g. the version shipped with Ubuntu 9.10 is based on GDL 0.9rc1 and has this problem.

```
GDL> uy_slice = var.uu[:, :, dim.nz/2, 1]
GDL> wdvelovect, ux_slice, uy_slice, grid.x, grid.y

GDL> surface, var.lnrho[:, :, dim.nz/2, 0]
```

See also Sect. 5.20.4.

5.20.4 IDL

IDL is a commercial visualization program for two-dimensional and simple three-dimensional graphics. It allows to access and manipulate numerical data in a fashion quite similar to how Fortran handles them.

In ‘\$PENCIL_HOME/idl’, we provide a number of general-purpose *IDL* scripts that we are using all the time in connection with the PENCIL CODE. While *IDL* is quite an expensive software package, it is quite useful for visualizing results from numerical simulations. In fact, for many applications, the 7-minute demo version of *IDL* is sufficient.

Visualization in IDL The Pencil Code GUI is a data post-processing tool for the usage on a day-to-day basis. It allows fast inspection of many physical quantities, as well as advanced features like horizontal averages, streamline tracing, freely orientable 2D-slices, and extraction of cut images and movies. To use the Pencil Code GUI, it is sufficient to run:

```
unix> idl
IDL> .r pc_gui
```

If you like to load only some subvolume of the data, like any 2D-slices from the given data snapshots, or 3D-subvolumes, it is possible to choose the corresponding options in the file selector dialog. The Pencil Code GUI offers also some options to be set on the command-line, please refer to their description in the source code.

There are also other small GUIs available, e.g., the file ‘time-series.dat’ can easily be analyzed with the command:

```
unix> idl
IDL> pc_show_ts
```

The easiest way to derive physical quantities at the command-line is to use one of the many `pc_read_var`-variants (`pc_read_var_raw` is recommended for large datasets because of its high efficiency regarding computation and memory usage) for reading the data. With that, one can make use of `pc_get_quantity` to derive any implemented physical quantity. Packed in a script, this is the recommended way to get reproducible results, without any chance for accidental errors on the interactive IDL command-line.

Alternatively, by using the command-line to see the time evolution of e.g., velocity and magnetic field (if they are present in you run), start *IDL* ¹¹ and run ‘ts.pro’:

¹¹If you run IDL from the command line, you will highly appreciate the following tip: IDL’s command line editing is broken beyond hope. But you can fix it, by running IDL under `rlwrap`, a wrapper for the excellent GNU *readline* library.

Download and install `rlwrap` from <http://utopia.knoware.nl/~hlub/uck/rlwrap/> (on some systems you just need to run ‘`emerge rlwrap`’, or ‘`apt-get install rlwrap`’), and alias your `idl` command:

```
csh> alias idl 'rlwrap -a -c idl'
bash> alias idl='rlwrap -a -c idl'
```

From now on, you can

```
unix> idl
IDL> .run ts
```

The *IDL* script ‘ts.pro’ reads the time series data from ‘data/time_series.dat’ and sorts the column into the structure *ts*, with the slot names corresponding to the name of the variables (taken from the header line of ‘data/time_series.dat’). Thus, you can refer to time as *ts.t*, to the rms velocity as *ts.urms*, and in order to plot the mean density as a function of time, you would simply type

```
IDL> plot, ts.t, ts.rhom
```

The basic command sequence for working with a snapshot is:

```
unix> idl
IDL> .run start
IDL> .run r
IDL> [specific commands]
```

You run ‘start.pro’ once to initialize (or reinitialize, if the mesh size has changed, for example) the fields and read in the startup parameters from the code. To read in a new snapshot, run ‘r.pro’ (or ‘rall.pro’, see below).

If you are running in parallel on several processors, the data are scattered over different directories. To reassemble everything in *IDL*, use

```
IDL> .r rall
```

instead of *.r r* (here, *.r* is a shorthand for *.run*). The procedure ‘rall.pro’ reads (and assembles) the data from all processors and correspondingly requires large amounts of memory for very large runs. If you want to look at just the data from one processor, use ‘r.pro’ instead.

If you need the magnetic field or the current density, you can calculate them in *IDL* by¹²

```
IDL> bb=curl(aa)
IDL> jj=curl2(aa)
```

By default one is reading always the current snapshot ‘data/proc*N*/var.dat’; if you want to read one of the permanent snapshots, use (for example)

```
IDL> varfile='VAR2'
IDL> .r r (or .r rall)
```

See Sect. 5.7.1 for details on permanent snapshots.

With ‘r.pro’, you can switch the part of the domain by changing the variable *datadir*:

- use long command lines that correctly wrap around;
- type the first letters of a command and then **[PageUp]** to recall commands starting with these letters;
- capitalize, uppercase or lowercase a word with **[Esc]-C**, **[Esc]-U**, **[Esc]-L**;
- use command line history across *IDL* sessions (you might need to create ‘~/idl_history’ for this);
- complete file names with **[Tab]** (works to some extent);
- ... use all the other *readline* features that you are using in *bash*, *octave*, *bc*, *gnuplot*, *ftp*, etc.

¹²Keep in mind that *jj=curl(bb)* would use iterated first derivatives instead of the second derivatives and thus be numerically less accurate than *jj=curl2(aa)*, particularly at small scales.

```
IDL> datadir='data/proc3'
IDL> .r r
```

will read the data written by processor 3.

Reading data into IDL arrays or structures As an alternative to the method described above, there is also the possibility to read the data into structures. This makes some more operations possible, e.g., reading data from an IDL program where the command `.r` is not allowed.

An efficient and still scriptable way would look like the following:

```
IDL> pc_read_var_raw, obj=var, tags=tags
IDL> bb = pc_get_quantity ('B', var, tags)
IDL> jj = pc_get_quantity ('j', var, tags)
```

This reads the data into an array 'var', as well as the array indices of the contained physical variables into a separate structure 'tags'. To use a caching mechanism within `pc_get_quantity`, please refer to the documentation and the examples contained in 'pencil-code/idl/pc_get_quantity.pro', where you can also start adding not yet implemented physical quantities.

To read a snapshot 'VAR10' into the IDL structure `ff`, type the following command

```
IDL> pc_read_var, obj=ff, varfile='VAR10', /trimall
```

The option `/trimall` removes ghost zones from the data. A number of other options are documented in the source code of `pc_read_var`. You can see what data the structure contains by using the command `tag_names`

```
IDL> print, tag_names(ff)
T X Y Z DX DY DZ UU LNRHO AA
```

One can access the individual variables by typing `ff.varname`, e.g.,

```
IDL> help, ff.aa
<Expression>      FLOAT      = Array[32, 32, 32, 3]
```

There are a number of files that read different data into structures. They are placed in the directory `\$PENCIL_HOME/idl/files`. Here is a list of files (including suggested options to call them with)

- `pc_read_var_raw, obj=var, tags=tags`
Efficiently read a snapshot into an array.
- `pc_read_var, obj=ff, /trimall`
Read a snapshot into a structure.
- `pc_read_ts, obj=ts`
Read the time series into a structure.
- `pc_read_xyaver, obj=xya`
 `pc_read_xzaver, obj=xza`
 `pc_read_yzaver, obj=yza`
Read 1-D time series into a structure.
- `pc_read_const, obj=cst`
Read code constants into a structure.

- `pc_read_pvar, obj=fp`
Read particle data into a structure.
- `pc_read_param, obj=par`
Read startup parameters into a structure.
- `pc_read_param, obj=par2, /param2`
Read runtime parameters into a structure.

Other options are documented in the source code of the files.

For some examples on how to use these routines, see Sect. 5.20.3.

5.20.5 Python

The Pencil Code supports reading, processing and the visualization of data using *python*. A number of scripts are placed in the subfolder '\$PENCIL_HOME/python'. A README file placed under that subfolder contains the information needed to read Pencil output data into python.

Installation For modern operating systems, Python is generally installed together with the system. If not, it can be installed via your preferred package manager or downloaded from the website <https://www.python.org/>. For convenience, it is strongly recommend to also install IPython, which is a more convenient console for Python. You will also need the Python packages NumPy, matplotlib, h5py, Tk and often Dill.

Perhaps the easiest way to obtain all the required software mentioned above is to install either Continuum's *Anaconda* or Enthought's *Canopy*. These Python distributions also provide (or indeed are) integrated graphical development environments.

Another way of installing libraries, particularly on a cluster without root privileges, is to use pip or pip3: `pip install h5py` or `pip3 install h5py`.

In order for Python to find the Pencil Code commands, you will have to set the environment variable *PYTHONPATH*:

```
export PYTHONPATH=${PENCIL_HOME}/python
# or
export PYTHONPATH=${PENCIL_HOME}/python:${PYTHONPATH}
```

Normally, you will add one of these lines to your `.bashrc` file.

In addition, you have to import the pencil module each time you start a Python shell:

```
import pencil as pc
```

The next two paragraphs show how you can include this and some other imports into your Python interpreter setup, so they are automatically run when you start IPython or the Python REPL.

ipythonrc When using IPython, some Pencil Code users add the following line to their `.ipython/ipythonrc` (create the file if it doesn't exist):

```
import_all pencil
```

and in addition add to their `.ipython/profile_default/startup/init.py` the lines

```
import pencil as pc

import numpy as np
import pylab as plt
import matplotlib
from matplotlib import rc

plt.ion()
matplotlib.rcParams['savefig.directory'] = ''
```

.pythonrc If you don't have IPython and cannot install it (e.g., on some cluster), you can instead edit your `.pythonrc`:

```
#!/usr/bin/python
import numpy as np
import pylab as plt
import pencil as pc
import atexit
#import readline
import rlcompleter

# enables search with Ctrl+r in the history
try:
    import readline
except ImportError:
    print "Module readline not available."
else:
    import rlcompleter
    readline.parse_and_bind("tab: complete")
# enables command history
historyPath = os.path.expanduser("~/pyhistory")
def save_history(historyPath=historyPath):
    import readline
    readline.write_history_file(historyPath)
if os.path.exists(historyPath):
    readline.read_history_file(historyPath)
atexit.register(save_history)
del os, atexit, readline, rlcompleter, save_history, historyPath

plt.ion()
```

and create the file `.pythonhistory` and add to your `.bashrc`:

```
export PYTHONSTARTUP=~/.pythonrc
```

However, none of this is required to use the Pencil Code Python modules.

Pencil Code Commands in General For a list of all Pencil Code commands start IPython and type `pc. <TAB>` (as with auto completion). To access the help of any command just type the command followed by a `??` (no spaces), e.g.,

```
In [1]: pc.math.dot?
```


Signature: `pc.math.dot(a, b)`
 Docstring:
 Take dot product of two pencil-code vectors a and b.

call signature:

`dot(a, b):`

Keyword arguments:

`*a*, *b*:`
 Pencil-code vectors with shape `[3, mz, my, mx]`.
 File: `~/pencil-code/python/pencil/math/vector_multiplication.py`
 Type: `function`

You can also use `help(pc.dot)` for a more complete documentation of the command.

There are various reading routines for the Pencil Code data. All of them return an object with the data. To store the data into a user defined variable type, e.g.,

```
In [1]: ts = pc.read.ts()
```

Most commands take some arguments. For most of them there is a default value, e.g.,

```
In [1]: pc.read.ts(file_name='time_series.dat', datadir='data')
```

Reading and Plotting Time Series Reading the time series file is very easy. Simply type

```
In [1]: ts = pc.read.ts()
```

and Python stores the data in the variable `ts`. The physical quantities are members of the object `ts` and can be accessed accordingly, e.g., `ts.t`, `ts.emag`. To check which other variables are stored simply do the tab auto completion `ts. <TAB>`.

Plot the data with the matplotlib commands:

```
In [1]: plt.plot(ts.t, ts.emag)
```

The standard plots are not perfect and need a little polishing. See the online wiki for a few examples on how to make pretty plots (<https://github.com/pencil-code/pencil-code/wiki/PythonForPencil>). You can save the plot into a file using the GUI or with

```
In [1]: plt.savefig('plot.eps')
```

Reading and Plotting VAR files and slice files Read var files:

```
In [1]: var = pc.read.var()
```

Read slice files:

```
In [1]: slices = pc.read.slices(field='bb1', extension='xy')
```

This returns the object `slices` with indices `slices[nTimes, my, mx]` and the time array `t`.

If you want to plot, e.g., the x-component of the magnetic field at the central plane simply type:

```
In [1]: plt.imshow(var.bb[0, 128, :, :].T, origin='lower', extent=[-4, 4, -4, 4])
```

For a complete list of arguments of `plt.imshow` refer to its documentation.

For a more interactive plot use:

```
In [1]: pc.visu.animate_interactive(slices.xy.bb1, t)
```

Be aware: arrays from the reading routines are ordered `f[nvar, mz, my, mx]`, i.e. reversed to IDL. This affects reading var files and slice files.

5.21 Running on multi-processor computers

The code is parallelized using *MPI* (*message passing interface*) for a simple domain decomposition (data-parallelism), which is a straight-forward and very efficient way of parallelizing finite-difference codes. The current version has a few restrictions, which need to be kept in mind when using the MPI features.

The global number of grid points (but excluding the ghost zones) in a given direction must be an exact multiple of the number of processors you use in that direction. For example if you have `nprocy=8` processors for the *y* direction, you can run a job with `nygrid=64` points in that direction, but if you try to run a problem with `nygrid=65` or `nygrid=94`, the code will complain about an inconsistency and stop. (So far, this has not been a serious restriction for us.)

5.21.1 How to run a sample problem in parallel

To run the sample problem in the directory ‘samples/conv-slab’ on 16 CPUs, you need to do the following (in that directory):

1. Edit ‘src/Makefile.local’ and replace

```
MPICOMM    = nompicomm
```

by

```
MPICOMM    = mpicomm
```

2. Edit ‘src/cparam.local’ and replace

```
integer, parameter :: ncpus=1, nprocy=1, nprocz=ncpus/nprocy, nprocx=1
integer, parameter :: nxgrid=32, nygrid=nxgrid, nzgrid=nxgrid
```

by

```
integer, parameter :: ncpus=16, nprocy=4, nprocz=ncpus/nprocy, nprocx=1
integer, parameter :: nxgrid=128, nygrid=nxgrid, nzgrid=nxgrid
```

The first line specifies a 4×4 layout of the data in the *y* and *z* direction. The second line increases the resolution of the run because running a problem as small as 32^3 on 16 CPUs would be wasteful. Even 128^3 may still be quite small in that respect. For performance timings, one should try and keep the size of the problem per CPU the same, so for example 256^3 on 16 CPUs should be compared with 128^3 on 2 CPUs.

3. Recompile the code

```
unix> (cd src; make)
```

4. Run it

```
unix> start.csh
unix> run.csh
```

Make sure that all CPUs see the same ‘data/’ directory; otherwise things will go wrong. Remember that in order to visualize the full domain with IDL (rather than just the domain processed and written by one processor), you need to use ‘rall.pro’ instead of ‘r.pro’.

5.21.2 Hierarchical networks (e.g., on Beowulf clusters)

On big Beowulf clusters, a group of nodes is often connected with a switch of modest speed, and all these groups are connected via a n times faster uplink switch. When bandwidth-limited, it is important to make sure that consecutive processors are mapped onto the mesh such that the load on the uplink is $\lesssim n$ times larger than the load on the slower switch within each group. On a 512 node cluster, where groups of 24 processors are linked via fast ethernet switches, which in turn are connected via a Gigabit uplink (~ 10 times faster), we found that $nprocx=4$ is optimal. For 128 processors, for example we find that $nprocx \times nprocy = 4 \times 32$ is the optimal layout, while. For comparison, 8×16 is 3 times slower, and 16×8 is 17 (!) times slower. These results can be understood from the structure of the network, but the basic message is to watch out for such effects and to try varying $nprocy$ and $nprocz$.

In cases where $nygrid > nzgrid$ it may be advantageous to swap the ordering of processor numbers. This can be done by setting `lprocz_slowest=F`.

5.21.3 Extra workload caused by the ghost zones

Normally, the workload caused by the ghost zones is negligible. However, if one increases the number of processors, a significant fraction of work is done in the ghost zones. In other words, the effective mesh size becomes much larger than the actual mesh size.

Consider a mesh of size $N_w = N_x \times N_y \times N_z$, and distribute the task over $P_w = P_x \times P_y \times P_z$ processors. If no communication were required, the number of points per processor would be

$$\frac{N_w}{P_w} = \frac{N_x \times N_y \times N_z}{P_x \times P_y \times P_z}. \quad (26)$$

However, for finite difference codes some communication is required, and the amount of communication depends on spatial order of the scheme, Q . The PENCIL CODE works by default with sixth order finite differences, so $Q = 6$, i.e. one needs 6 ghost zones, 3 on each end of the mesh. With $Q \neq 0$ the number of points per processor is

$$\frac{N_w^{(\text{eff})}}{P_w} = \left(\frac{N_x}{P_x} + Q \right) \times \left(\frac{N_y}{P_y} + Q \right) \times \left(\frac{N_z}{P_z} + Q \right). \quad (27)$$

There is efficient scaling only when

$$\min \left(\frac{N_x}{P_x}, \frac{N_y}{P_y}, \frac{N_z}{P_z} \right) \gg Q. \quad (28)$$

In the special case were $N_x = N_y = N_z \equiv N = N_w^{1/3}$, with $P_x = 1$ and $P_y = P_z \equiv P = P_w^{1/2}$, we have

$$\frac{N_w^{(\text{eff})}}{P_w} = (N + Q) \times \left(\frac{N}{P} + Q \right)^2. \quad (29)$$

For $N = 128$ and $Q = 6$ the effective mesh size exceeds the actual mesh size by a factor

$$\frac{N_w^{(\text{eff})}}{N_w} = (N + Q) \times \left(\frac{N}{P} + Q \right)^2 \times \frac{P_w}{N_w}. \quad (30)$$

These factors are listed in Table 3.

Table 3: $N_w^{(\text{eff})}/N_w$ versus N and P .

$P \backslash N$	128	256	512	1024	2048
1	1.15	1.07	1.04	1.02	1.01
2	1.19	1.09	1.05	1.02	1.01
4	<u>1.25</u>	1.12	1.06	1.03	1.01
8	1.34	1.16	1.08	1.04	1.02
16	1.48	<u>1.22</u>	1.11	1.05	1.03
32	1.68	1.31	1.15	1.07	1.04
64	1.98	1.44	<u>1.21</u>	1.10	1.05
128	2.45	1.64	1.30	1.14	1.07
256	3.21	1.93	1.43	<u>1.20</u>	1.10
512	4.45	2.40	1.62	1.29	1.14

Ideally, one wants to keep the work in the ghost zones at a minimum. If one accepts that 20–25% of work are done in the ghost zones, one should use 4 processors for 128^3 mesh points, 16 processors for 256^3 mesh points, 64 processors for 512^3 mesh points, 256 processors for 1024^3 mesh points, and 512 processors for 1536^3 mesh points.

5.22 Running in double-precision

With many compilers, you can easily switch to double precision (8-byte floating point numbers) as follows.

Add the lines

```
# Use double precision
REAL_PRECISION = double
```

to ‘src/Makefile.local’ and (re-)run `pc_setupsrc`.

If `REAL_PRECISION` is set to ‘double’, the flag `FFLAGS_DOUBLE` is appended to the Fortran compile flags. The default for `FFLAGS_DOUBLE` is `-r8`, which works for `g95` or `ifort`; for `gfortran`, you need to make sure that `FFLAGS_DOUBLE` is set to `-fdefault-real-8`.

You can see the flags in ‘src/Makefile.inc’, which is the first place to check if you have problems compiling for double precision.

Using double precision might be important in turbulence runs where the resolution is 256^3 meshpoints and above (although such runs often seem to work fine at single

precision). To continue working in double precision, you just say `lread_from_other_prec=T` in `run_pars`; see Sect. F.1.

5.23 Power spectrum

Given a real variable u , its Fourier transform $\tilde{u}$ is given by

$$\begin{aligned} \tilde{u}(k_x, k_y, k_z) = \mathcal{F}(u(x, y, z)) &= \frac{1}{N_x N_y N_z} \sum_{p=0}^{N_x-1} \sum_{q=0}^{N_y-1} \sum_{r=0}^{N_z-1} u(x_p, y_q, z_r) \\ &\times \exp(-ik_x x_p) \exp(-ik_y y_q) \exp(-ik_z z_r), \end{aligned} \quad (31)$$

where

$$|k_x| < \frac{\pi N_x}{L_x}, \quad |k_y| < \frac{\pi N_y}{L_y}, \quad |k_z| < \frac{\pi N_z}{L_z},$$

when L is the size of the simulation box. The three-dimensional power spectrum $P(k)$ is defined as

$$P(k) = \frac{1}{2} \tilde{u} \tilde{u}^*, \quad (32)$$

where

$$k = \sqrt{k_x^2 + k_y^2 + k_z^2}. \quad (33)$$

Note that we can only reasonably calculate $P(k)$ for $k < \pi N_x / L_x$.

To get power spectra from the code, edit ‘`run.in`’ and add for example the following lines

```
dspec=5., ou_spec=T, ab_spec=T !(for energy spectra)
oned=T
```

under `run_pars`. The kinetic (`vel_spec`) and magnetic (`mag_spec`) power spectra will now be calculated every 5.0 (`dspec`) time units. The Fourier spectra is calculated using *fftpack*. In addition to calculating the three-dimensional power spectra also the one-dimensional power spectra will be calculated (`oned`).

In addition one must edit ‘`src/Makefile.local`’ and add the lines

```
FOURIER = fourier_fftpack
POWER = power_spectrum
```

Running the code will now create the files ‘`powerhel_mag.dat`’ and ‘`power_kin.dat`’ containing the three-dimensional magnetic and kinetic power spectra respectively. In addition to these three-dimensional files we will also find the one-dimensional files ‘`powerbx_x.dat`’, ‘`powerby_x.dat`’, ‘`powerbz_x.dat`’, ‘`powerux_x.dat`’, ‘`poweruy_x.dat`’ and ‘`poweruz_x.dat`’. In these files the data are stored such that the first line contains the time of the snapshot, the following $nxgrid/2$ numbers represent the power at each wavenumber, from the smallest to the largest. If several snapshots have been saved, they are being stored immediately following the preceding snapshot.

You can read the results with the `idl` procure ‘`power`’, like this:

```
power, '_kin', '_mag', k=k, spec1=spec1, spec2=spec2, i=n, tt=t, /noplot
power, 'hel_kin', 'hel_mag', k=k, spec1=spec1h, spec2=spec2h, i=n, tt=t, /noplot
```

If `powerhel` is invoked, `krms` is written during the first computation. The relevant output file is ‘`power_krms.dat`’. This is needed for a correct calculation of k used in the realizability conditions.

A caveat of the implementation of Fourier transforms in the PENCIL CODE is that, due to the parallelization, the permitted resolution is limited to the case when one direction is an integer multiple of the other. So, it can be done for

$$N_x = n \cdot N_y$$

Unfortunately, for some applications one wants $N_x < N_y$. Wlad experimented with arbitrary resolution by interpolating x to the same resolution of y prior to transposing, then transform the interpolated array and then interpolating it back (check ‘fourier_transform_y’ in ‘fourier_fftpack.f90’).

A feature of our current implementation with x parallelization is that `fft_xyz_parallel_3D` requires `nygrid` to be an integer multiple of `nprocx*nprocz`. Examples of good mesh layouts are listed in Table 4.

Table 4: Examples of mesh layouts for which Fourier transforms with x parallelization is possible.

ny	nprocx	nprocy	nprocz	ncpus
256	1	16	16	256
256	2	16	16	512
256	4	16	16	1024
256	8	16	16	2048
288	2	16	18	576
512	2	16	32	1024
512	4	16	16	1024
512	4	16	32	2048
576	4	18	32	2304
576	8	18	32	4608
576	16	18	32	9216
1024	4	32	32	4096
1024	4	16	64	4096
1024	8	16	32	4096
1152	4	36	32	4608
1152	4	32	36	4608
2304	2	32	72	4608
2304	4	36	64	9216
2304	4	32	72	9216

To visualize with *IDL* just type `power` and you get the last snapshot of the three-dimensional power spectrum. See head of ‘\$PENCIL_HOME/idl/power.pro’ for options to `power`.

By default, the spectra for the initial times time is not outputted, but it can sometimes be useful to have it. In that case, one can put `lspec_start=T`.

5.24 Other power spectra

Over the years, many more spectra have been outputted and not everything is immediately documented. Here some examples:

```
data/powerhel_mag.dat
data/powerhel_kin.dat
```

```

data/power_saffman_ub.dat
data/power_saffman_mag.dat
data/power_mag.dat
data/power_krms.dat
data/power_kin.dat

```

Here, the first two are helicity spectra that come “for free” (in addition to those with `_kin` and `_mag`) when one invokes `ou_spec=T` and `ab_spec=T`. The ones with `_saffman` refer to the spectra related to Saffman-like invariants such as the Hosking integral (which refers to `power_mag.dat`) and the cross-helicity Saffman-like invariant (`power_saffman_ub.dat`). Their slopes for $k \rightarrow 0$ are the invariants divided by $2\pi^2$; see the papers by Hosking & Schekochihin (2021) as well as Zhou et al. (2022).

5.25 Structure functions

We define the p -th order longitudinal structure function of u as

$$S_{\text{long}}^p(l) = \langle |u_x(x+l, y, z) - u_x(x, y, z)|^p \rangle, \quad (34)$$

while the transverse is

$$S_{\text{trans}}^p(l) = \langle |u_y(x+l, y, z) - u_y(x, y, z)|^p \rangle + \langle |u_z(x+l, y, z) - u_z(x, y, z)|^p \rangle. \quad (35)$$

Edit ‘`run.in`’ and add for example the following lines

```

dspec=2.3,
lsfu=T, lsfb=T, lsfbz1=T, lsfbz2=T

```

under `run_pars`. The velocity (`lsfu`), magnetic (`lsfb`) and Elsasser (`lsfbz1` and `lsfbz2`) structure functions will now be calculated every 2.3 (`dspec`) time unit.

In addition one must edit ‘`src/Makefile.local`’ and add the line

```
STRUCT_FUNC = struct_func
```

In ‘`src/cparam.local`’, define `lb_nxgrid` and make sure that

```
nxgrid = nygrid = nzgrid = 2*lb_nxgrid
```

E.g.

```

integer, parameter :: lb_nxgrid=5
integer, parameter :: nxgrid=2*lb_nxgrid, nygrid=nxgrid, nzgrid=nxgrid

```

Running the code will now create the files:

```

'sfu-1.dat', 'sfu-2.dat', 'sfu-3.dat' (velocity),
'sfb-1.dat', 'sfb-2.dat', 'sfb-3.dat' (magnetic field),
'sfbz1-1.dat', 'sfz1-2.dat', 'sfz1-3.dat' (first Elsasser variable),
'sfbz2-1.dat', 'sfz2-2.dat', 'sfz2-3.dat' (second Elsasser variable),

```

which contains the data of interest. The first line in each file contains the time t and the number q_{max} , such that the largest moment calculated is $q_{\text{max}} - 1$. The next i_{max} numbers represent the first moment structure function for the first snapshot, here

$$i_{\text{max}} = 2^{\frac{\ln(nxgrid)}{\ln 2}} - 2. \quad (36)$$

The next *imax* numbers contain the second moment structure function, and so on until *qmax* − 1. The following *imax* numbers then contain the data of the *signed* third order structure function i.e. $S_{\text{long}}^3(l) = \langle [u_x(x+l, y, z) - u_x(x, y, z)]^3 \rangle$.

The following *imax* × *qmax* × 2 numbers are zero if *nr_directions* = 1 (default), otherwise they are the same data as above but for the structure functions calculated in the y and z directions.

If the code has been run long enough as to calculate several snapshots, these snapshots will now follow, being stored in the same way as the first snapshot.

To visualize with *IDL* just type *structure* and you get the time-average of the first order longitudinal structure function (be sure that ‘pencil-runs/forced/idl/’ is in *IDL_PATH*). See head of ‘pencil-runs/forced/idl/structure.pro’ for options to *structure*.

5.26 Particles

The PENCIL CODE has modules for tracer particles and for dust particles (see Sect. 6.17). The particle modules are chosen by setting the value of the variable *PARTICLES* in *Makefile.local* to either *particles_dust* or *particles_tracers*. For the former case each particle has six degrees of freedom, three positions and three velocities. For the latter it suffices to have only three position variables as the velocity of the particles are equal to the instantaneous fluid velocity at that point. In addition one can choose to have several additional internal degrees of freedoms for the particles. For example one can temporally evolve the particles radius by setting *PARTICLES_RADIUS* to *particles_radius* in *Makefile.local*.

All particle infrastructure is controlled and organized by the *Particles_main* module. This module is automatically selected by *Makefile.src* if *PARTICLES* is different from *noparticles*. Particle modules are compiled as a separate library. This way the main part of the Pencil Code only needs to know about the *particles_main.a* library, but not of the individual particle modules.

For a simulation with particles one must in addition define a few parameters in *cparam.local*. Here is a sample of *cparam.local* for a parallel run with 2,000,000 particles:

```
integer, parameter :: ncpus=16, nprocy=4, nprocz=4, nprocx=1
integer, parameter :: nxgrid=128, nygrid=256, nzgrid=128
integer, parameter :: npar=2000000, mpar_loc=400000, npar_mig=1000
integer, parameter :: npar_species=2
```

The parameter *npar* is the number of particles in the simulation, *mpar_loc* is the number of particles that is allowed on each processor and *npar_mig* is the number of particles that are allowed to migrate from one processor to another in any time-step. For a non-parallel run it is enough to specify *npar*. The number of particle species is set through *npar_species* (assumed to be one if not set). The particle input parameters are given in *start.in* and *run.in*. Here is a sample of the particle part of *start.in* for dust particles:

```
/
&particles_init_pars
  initxvp='gaussian-z', initvvp='random'
  zp0=0.02, delta_vp0=0.01, eps_dtog=0.01, tausp=0.1
  lparticlemesh_tsc=T
```


/

The initial positions and velocities of the dust particles are set in `initxxp` and `initvvp`. The next four input parameters are further specifications of the initial condition. Interaction between the particles and the mesh, e.g., through drag force or self-gravity, require a mapping of the particles on the mesh. The PENCIL CODE currently supports NGP (Nearest Grid Point, default), CIC (Cloud in Cell, set `lparticlemesh_cic=T`) and TSC (Triangular Shaped Cloud, set `lparticlemesh_tsc=T`). See Youdin & Johansen (2007) for details.

Here is a sample of the particle part of `run.in` (also for dust particles):

```
/
&particles_run_pars
  ldragforce_gas_par=T
  cdtp=0.2
/
```

The logical `ldragforce_gas_par` determines whether the dust particles influence the gas with a drag force. `cdtp` tells the code how many friction times should be resolved in a minimum time-step.

The sample run ‘`samples/sedimentation/`’ contains the latest setup for dust particles.

5.26.1 Particles in parallel

The particle variables (e.g., x_i and v_i) are kept in the arrays `fp` and `dfp`. For parallel runs, particles must be able to move from processor to processor as they pass out of the (x, y, z) -interval of the local processor. Since not all particles are present at the same processor at the same time (hopefully), there is some memory optimization in making `fp` not big enough to contain all the particles at once. This is achieved by setting the code variable `mpar_loc` less than `npar` in `cparam.local` for parallel runs. When running with millions of particles, this trick is necessary to keep the memory need of the code down.

The communication of migrating particles between the processors happens as follows (see the subroutine `redist_particles_procs` in `particles_sub.f90`):

1. In the beginning of each time-step all processors check if any of their particles have crossed the local (x, y, z) -interval. These particles are called migrating particles. A run can have a maximum of `npar_mig` migrating particles in each time-step. The value of `npar_mig` must be set in `cparam.local`. The number should (of course) be slightly larger than the maximum number of migrating particles at any time-step during the run. The diagnostic variable `nmigmax` can be used to output the maximum number of migrating particles at a given time-step. One can set `lmigration_redo=T` in `&particles_run_pars` to force the code to redo the migration step if more than `npar_mig` want to migrate. This does slow the code down somewhat, but has the benefit that the code does not stop when more than `npar_mig` particles want to migrate.
2. The index number of the receiving processor is then calculated. This requires some assumption about the grid on other processors and will currently not work for nonequidistant grids. Particles do not always pass to neighboring processors as the global boundary conditions may send them to the other side of the global domains (periodic or shear periodic boundary conditions).

3. The migrating particle information is copied to the end of `fp`, and the empty spot left behind is filled up with the particle of the highest index number currently present at the processor.
4. Once the number of migrating particles is known, this information is shared with neighboring processors (including neighbors over periodic boundaries) so that they all know how many particles they have to receive and from which processors.
5. The communication happens as directed MPI communication. That means that processors 0 and 1 can share migrating particles at the same time as processors 2 and 3 do it. The communication happens from a chunk at the end of `fp` (migrating particles) to a chunk that is present just after the particle of the highest index number that is currently at the receiving processor. Thus the particles are put directly at their final destination, and the migrating particle information at the source processor is simply overwritten by other migrating particles at the next time-step.
6. Each processor keeps track of the number of particles that it is responsible for. This number is stored in the variable `npar_loc`. It must never be larger than `mpar_loc` (see above). When a particle leaves a processor, `npar_loc` is reduced by one, and then increased by one at the processor that receives that particle. The maximum number of particles at any processor is stored in the diagnostic variable `nparmax`. If this value is not close to `npar/ncpus`, the particles have piled up in such a way that computations are not evenly shared between the processors. One can then try to change the parallelization architecture (`nprocy` and `nprocz`) to avoid this problem.

In simulations with many particles (comparable to or more than the number of grid cells), it is crucial that particles are shared relatively evenly among the processors. One can as a first approach attempt to not parallelize directions with strong particle density variations. However, this is often not enough, especially if particles clump locally.

Alternatively one can use Particle Block Domain Decomposition (PBDD, see Johansen et al. 2011). The steps in Particle Block Domain Decomposition scheme are as follows:

1. The fixed mesh points are domain-decomposed in the usual way (with $\text{ncpus} = \text{nprocx} \times \text{nprocy} \times \text{nprocz}$).
2. Particles on each processor are counted in *bricks* of size $\text{nbx} \times \text{nby} \times \text{nbz}$ (typically $\text{nbx} = \text{nby} = \text{nbz} = 4$).
3. Bricks are distributed among the processors so that each processor has approximately the same number of particles
4. Adopted bricks are referred to as *blocks*.
5. The Pencil Code uses a third order Runge-Kutta time-stepping scheme. In the beginning of each sub-time-step particles are counted in blocks and the block counts communicated to the bricks on the parent processors. The particle density assigned to ghost cells is folded across the grid, and the final particle density (defined on the bricks) is communicated back to the adopted blocks. This step is necessary because the drag force time-step depends on the particle density, and each particle assigns density not just to the nearest grid point, but also to the neighboring grid points.
6. In the beginning of each sub-time-step the gas density and gas velocity field is communicated from the main grid to the adopted particle blocks.

7. Drag forces are added to particles and back to the gas grid points in the adopted blocks. This partition aims at load balancing the calculation of drag forces.
8. At the end of each sub-time-step the drag force contribution to the gas velocity field is communicated from the adopted blocks back to the main grid.

Particle Block Domain Decomposition is activated by setting `PARTICLES = particles_dust_blocks` and `PARTICLES_MAP = particles_map_blocks` in `Makefile.local`. A sample of `cparam.local` for Particle Block Domain Decomposition can be found in `samples/sedimentation/blocks`:

```
integer, parameter :: ncpus=4, nprocx=2, nprocy=2, nprocz=1
integer, parameter :: nxgrid=32, nygrid=32, nzgrid=32
integer, parameter :: npar=10000, mpar_loc=5000, npar_mig=100
integer, parameter :: npar_species=4
integer, parameter :: nbrickx=4, nbricky=4, nbrickz=4, nblockmax=32
```

The last line defines the number of bricks in the total domain – here we divide the grid into $4 \times 4 \times 4$ bricks each of size $8 \times 8 \times 8$ grid points. The parameter `nblockmax` tells the code the maximum number of blocks any processor may adopt. This should not be so low that there is not room for all the bricks with particles, nor so high that the code runs out of memory.

5.26.2 Large number of particles

When dealing with large number of particles, one needs to make sure that the number of particles `npar` is less than the maximum integer that the compiler can handle with. The maximum integer can be checked by the Fortran intrinsic function `huge`,

```
program huge_integers
  print *, huge(0_4) ! for default Fortran integer (32 Bit)
  print *, huge(0_8) ! for 64 Bit integer in Fortran
end program huge_integers
```

If the number of particles `npar` is larger than default maximum integer, one can promote the maximum integer to 64 Bit by setting

```
integer(kind=8), parameter :: npar=4294967296
```

in the `cparam.local` file. This works because the data type of `npar` is only set here. It is worth noting that one *should not* use the flag

```
FFLAGS += -integer-size 64
```

to promote all the integers to 64 Bit. This will break the Fortran-C interface. One should also make sure that `npar_mig` $\leq$ `npar/ncpus`. It is also beneficial to set `mpar_loc` $= 2 \times$ `npar/ncpus`. Sometimes one may encounter the following error, “additional relocation overflows omitted from the output” due to the 2G memory limit (caused by large static array). It is `mpar_loc` that determines the usage of memory instead of `npar`. There are two ways to resolve this issue:

- Use a specific memory model to generate code and store data by setting the following for Intel compiler in your configuration file,

```
FFLAGS += -shared-intel -mcmodel=medium
```

This will, however, affect the performance of the code [44] This method can handle at least the following setup,

```
integer, parameter :: ncpus=256,nprocx=4,nprocy=8,nprocz=ncpus/(nprocx*nprocy)
integer, parameter :: nxgrid=1024,nygrid=nxgrid,nzgrid=nxgrid
integer(kind=ikind8), parameter :: npar=124999680
integer, parameter :: npar_stalk=100000, npar_mig=npar/10
integer, parameter :: mpar_loc=npar/5
```

- Increase ncpu and decrease mpar_loc. For npar=124999680, ncpu=1024 is needed.

```
integer, parameter :: ncpus=1024,nprocx=8,nprocy=8,nprocz=ncpus/(nprocx*nprocy)
integer, parameter :: nxgrid=1024,nygrid=nxgrid,nzgrid=nxgrid
integer(kind=ikind8), parameter :: npar=124999680
integer, parameter :: mpar_loc=5*npar/ncpus
integer, parameter :: npar_stalk=100000, npar_mig=npar/ncpus
```

It is worth noting that even without particles, a simulation with 1024^3 resolution requires at least 512 CPUs to be compiled.

5.26.3 Random number generator

There are several methods to generate random number in the code. It is worth noting that when simulating coagulation with the super-particle approach, one should use the intrinsic random number generator of FORTRAN instead of the one implemented in the code. When invoking `random_number_wrapper`, there will be back-reaction to the gas flow. This unexpected back-reaction can be tracked by inspecting the power spectra, which exhibits the oscillation at the tail. To avoid this, one should set `luser_random_number_wrapper=F` under the module `particles_coag_run_pars` in `run.in`.

5.27 Non-cartesian coordinate systems

Spherical coordinates are invoked by adding the following line in the file ‘`start.in`’

```
&init_pars
  coord_system='spherical_coords'
```

One can also invoke cylindrical coordinates by saying `cylindrical_coords` instead. In practice, the names (x, y, z) are still used, but they refer then to (r, θ, ϕ) or (r, ϕ, z) instead.

When working with curvilinear coordinates it is convenient to use differential operators in the non-coordinate basis, so the derivatives are taken with respect to length, and not in a mixed fashion with respect to length for $\partial/\partial r$ and with respect to angle for $\partial/\partial \theta$ and $\partial/\partial \phi$. The components in the non-coordinate basis are denoted by hats, see, e.g., [34], p. 213; see also Appendix B of [35]. For spherical polar coordinates the only nonvanishing Christoffel symbols (or connection coefficients) are

$$\Gamma^{\hat{\theta}}_{\hat{r}\hat{\theta}} = \Gamma^{\hat{\phi}}_{\hat{r}\hat{\phi}} = -\Gamma^{\hat{r}}_{\hat{\theta}\hat{\theta}} = -\Gamma^{\hat{r}}_{\hat{\phi}\hat{\phi}} = 1/r, \quad (37)$$

$$\Gamma^{\hat{\phi}}_{\hat{\theta}\hat{\phi}} = -\Gamma^{\hat{\theta}}_{\hat{\phi}\hat{\phi}} = \cot \theta / r. \quad (38)$$

The Christoffel symbols enter as correction terms for the various differential operators in addition to a term calculated straightforwardly in the non-coordinate basis. The derivatives of some relevant Christoffel symbols are

$$\Gamma^{\hat{\theta}}_{\hat{r}\hat{\theta},\hat{\theta}} = \Gamma^{\hat{\phi}}_{\hat{r}\hat{\phi},\hat{\phi}} = \Gamma^{\hat{\phi}}_{\hat{\theta}\hat{\phi},\hat{\phi}} = 0 \quad (39)$$

$$\Gamma^{\hat{\theta}}_{\hat{r}\hat{\theta},\hat{r}} = \Gamma^{\hat{\phi}}_{\hat{r}\hat{\phi},\hat{r}} = -r^{-2} \quad (40)$$

$$\Gamma^{\hat{\phi}}_{\hat{\theta}\hat{\phi},\hat{\theta}} = -r^{-2} \sin^{-2}\theta \quad (41)$$

Further details are given in Appendix I.

6 The equations

The equations solved by the PENCIL CODE are basically the standard compressible MHD equations. However, the modular structure allows some variations of the MHD equations, as well as switching off some of the equations or individual terms of the equation (nomagnetic, noentropy, etc.).

In this section the equations are presented in their most complete form. It may be expected that the code can evolve most subsets or simplifications of these equations.

6.1 Continuity equation

In the code the continuity equation, $\partial\rho/\partial t + \nabla \cdot \rho \mathbf{u} = 0$, is written in terms of $\ln \rho$,

$$\frac{D \ln \rho}{Dt} = -\nabla \cdot \mathbf{u} . \quad (42)$$

Here ρ denotes density, $\mathbf{u}$ the fluid velocity, t is time and $D/Dt \equiv \partial/\partial t + \mathbf{u} \cdot \nabla$ is the advective derivative.

6.2 Equation of motion

In the equation of motion, using a perfect gas, the pressure term, can be expressed as $-\rho^{-1} \nabla p = -c_s^2 (\nabla s/c_p + \nabla \ln \rho)$, where the squared sound speed is given by

$$c_s^2 = \gamma \frac{p}{\rho} = c_{s0}^2 \exp \left[\gamma s/c_p + (\gamma-1) \ln \frac{\rho}{\rho_0} \right] , \quad (43)$$

and $\gamma = c_p/c_v$ is the ratio of specific heats, or *adiabatic index*. Note that c_s^2 is proportional to the temperature with $c_s^2 = (\gamma - 1)c_p T$.

The equation of motion is accordingly

$$\begin{aligned} \frac{D\mathbf{u}}{Dt} = & -c_s^2 \nabla \left(\frac{s}{c_p} + \ln \rho \right) - \nabla \Phi_{\text{grav}} + \frac{\mathbf{j} \times \mathbf{B}}{\rho} \\ & + \nu \left(\nabla^2 \mathbf{u} + \frac{1}{3} \nabla \nabla \cdot \mathbf{u} + 2\mathbf{S} \cdot \nabla \ln \rho \right) + \zeta (\nabla \nabla \cdot \mathbf{u}) ; \end{aligned} \quad (44)$$

Here Φ_{grav} is the gravity potential, $\mathbf{j}$ the electric current density, $\mathbf{B}$ the magnetic flux density, ν is kinematic viscosity, ζ describes a bulk viscosity, and, in Cartesian coordinates

$$S_{ij} = \frac{1}{2} \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} - \frac{2}{3} \delta_{ij} \nabla \cdot \mathbf{u} \right) \quad (45)$$

is the rate-of-shear tensor that is traceless, because it can be written as the generic rate-of-strain tensor minus its trace. In curvilinear coordinates, we have to replace partial differentiation by covariant differentiation (indicated by semicolons), so we write $S_{ij} = \frac{1}{2}(u_{i;j} + u_{j;i}) - \frac{1}{3}\delta_{ij} \nabla \cdot \mathbf{u}$.

The interpretation of the two viscosity terms varies greatly depending upon the Viscosity module used, and indeed on the parameters given to the module. See §6.6.

For isothermal hydrodynamics, see §6.4 below.

6.3 Induction equation

We start from the general induction equation for the vector potential,

$$\frac{\partial \mathbf{A}}{\partial t} = \mathbf{u} \times \mathbf{B} - \eta \mu_0 \mathbf{j} - \nabla \Phi, \quad (46)$$

where $\mathbf{A}$ is the magnetic vector potential, $\mathbf{B} = \nabla \times \mathbf{A}$ the magnetic flux density, Φ is the scalar potential, $\eta = 1/(\mu_0 \sigma)$ is the magnetic diffusivity (σ denoting the electrical conductivity), and μ_0 the magnetic vacuum permeability.

In the Weyl gauge, $\Phi = 0$, and the induction equation reduces to

$$\frac{\partial \mathbf{A}}{\partial t} = \mathbf{u} \times \mathbf{B} - \eta \mu_0 \mathbf{j}. \quad (47)$$

However, we do not normally use the Weyl gauge, but rather the resistive gauge,

$$\frac{\partial \mathbf{A}}{\partial t} = \mathbf{u} \times \mathbf{B} + \eta \nabla^2 \mathbf{A}. \quad (48)$$

In the *resistive gauge*, we have $\Phi = \eta \nabla \cdot \mathbf{A}$. It has the advantage that the terms $A_{x,xx}$, $A_{y,yy}$, and $A_{z,zz}$ are present and prevent wiggles the vector potential. Although the terms $A_{x,x}$, $A_{y,y}$, and $A_{z,z}$ are absent in $\mathbf{B}$, they can affect the result if the terms are large compared with $\mathbf{B}$.

6.4 Entropy equation

The current thermodynamics module *entropy* formulates the thermal part of the physics in terms of *entropy* s , rather than thermal energy e , which you may be more familiar with. Thus the two fundamental thermodynamical variables are $\ln \rho$ and s . The reason for this choice of variables is that entropy is the natural physical variable for (at least) convection processes: the sign of the entropy gradient determines convective (in)stability, the *Rayleigh number* is proportional to the entropy gradient of the associated hydrostatic reference solution, etc. The equation solved is

$$\rho T \frac{Ds}{Dt} = \mathcal{H} - \mathcal{C} + \nabla \cdot (K \nabla T) + \eta \mu_0 \mathbf{j}^2 + 2\rho \nu \mathbf{S} \otimes \mathbf{S} + \zeta \rho (\nabla \cdot \mathbf{u})^2. \quad (49)$$

Here, T is temperature, c_p the specific heat at constant pressure, $\mathcal{H}$ and $\mathcal{C}$ are explicit heating and cooling terms, K is the radiative (thermal) conductivity, ζ describes a bulk viscosity, and $\mathbf{S}$ is the rate-of-shear tensor that is traceless.

In the entropy module we solve for the specific entropy, s . The heat conduction term on the right hand side can be written in the form

$$\frac{\nabla \cdot (K \nabla T)}{\rho T} \quad (50)$$

$$= c_p \chi \left[\nabla^2 \ln T + \nabla \ln T \cdot \nabla (\ln T + \ln \chi + \ln \rho) \right] \quad (51)$$

$$= c_p \chi \left[\gamma \nabla^2 s / c_p + (\gamma - 1) \nabla^2 \ln \rho \right] + c_p \chi \left[\gamma \nabla s / c_p + (\gamma - 1) \nabla \ln \rho \right] \cdot \left[\gamma (\nabla s / c_p + \nabla \ln \rho) + \nabla \ln \chi \right], \quad (52)$$

where $\chi = K/(\rho c_p)$ is the thermal diffusivity. The latter equation shows that the diffusivity for s is $\gamma \chi$, which is what we have used in Eq. (24).

In an alternative formulation for a constant K , the heat conduction term on the right hand side can also be written in the form

$$\frac{\nabla \cdot (K \nabla T)}{\rho T} = \frac{K}{\rho} \left[\nabla^2 \ln T + (\nabla \ln T)^2 \right] \quad (53)$$

which is the form used when constant K is chosen.

Note that by setting $\gamma = 1$ and initially $s = 0$, one obtains an isothermal equation of state (albeit at some unnecessary expense of memory). Similarly, by switching off the evolution terms of entropy, one immediately gets polytropic behavior (if s was initially constant) or generalized polytropic behavior (where s is not uniform, but $\partial s / \partial t = 0$).

A better way to achieve isothermality is to use the *noentropy* module.

6.4.1 Viscous heating

We can write the viscosity as the divergence of a tensor $\tau_{ij,j}$,

$$\rho \frac{\partial u_i}{\partial t} = \dots + \tau_{ij,j}, \quad (54)$$

where $\tau_{ij} = 2\nu\rho S_{ij}$ is the stress tensor. The viscous power density P is

$$P = u_i \tau_{ij,j} \quad (55)$$

$$= \frac{\partial}{\partial x_j} (u_i \tau_{ij}) - u_{i,j} \tau_{ij} \quad (56)$$

The term under the divergence is the viscous energy flux and the other term is the kinetic energy loss due to heating. The heating term $+u_{i,j} \tau_{ij}$ is positive definite, because τ_{ij} is a symmetric tensor and the term only gives a contribution from the symmetric part of $u_{i,j}$, which is $\frac{1}{2}(u_{i,j} + u_{j,i})$, so

$$u_{i,j} \tau_{ij} = \frac{1}{2} \nu \rho (u_{i,j} + u_{j,i}) (2S_{ij}). \quad (57)$$

But, because S_{ij} is traceless, we can add anything proportional to δ_{ij} and, in particular, we can write

$$u_{i,j} \tau_{ij} = \frac{1}{2} (u_{i,j} + u_{j,i}) (2\nu\rho S_{ij}) \quad (58)$$

$$= \frac{1}{2} (u_{i,j} + u_{j,i} - \frac{1}{3} \delta_{ij} \nabla \cdot \mathbf{u}) (2\nu\rho S_{ij}) \quad (59)$$

$$= 2\nu\rho \mathbf{S}^2, \quad (60)$$

which is positive definite.

6.4.2 Alternative description

By setting `pretend_lnTT=T` in `init_pars` or `run_pars` (i.e. the general part of the name list) the logarithmic temperature is used instead of the entropy. This has computational advantages when heat conduction (proportional to $K \nabla T$) is important. Another alternative is to use another module, i.e. set `ENTROPY=temperature_idealgas` in `'Makefile.local'`.

When `pretend_lnTT=T` is set, the entropy equation

$$\frac{\partial s}{\partial t} = -\mathbf{u} \cdot \nabla s + \frac{1}{\rho T} \text{RHS} \quad (61)$$

is replaced by

$$\frac{\partial \ln T}{\partial t} = -\mathbf{u} \cdot \nabla \ln T + \frac{1}{\rho c_v T} \text{RHS} - (\gamma - 1) \nabla \cdot \mathbf{u}, \quad (62)$$

where RHS is the right hand side of equation (49).

6.5 Transport equation for a passive scalar

In conservative form, the equation for a passive scalar is

$$\frac{\partial}{\partial t}(\rho c) + \nabla \cdot [\rho c \mathbf{u} - \rho \mathcal{D} \nabla c] = 0. \quad (63)$$

Here c denotes the concentration (per unit mass) of the passive scalar and $\mathcal{D}$ its diffusion constant (assumed constant). In the code this equation is solved in terms of $\ln c$,

$$\frac{\mathcal{D} \ln c}{\mathcal{D} t} = \mathcal{D} [\nabla^2 \ln c + (\nabla \ln \rho + \nabla \ln c) \cdot \nabla \ln c]. \quad (64)$$

Using $\ln c$ instead of c has the advantage that it enforces $c > 0$ for all times. However, the disadvantage is that one cannot have $c = 0$. For this reason we ended up using the non-logarithmic version by invoking `PSCALAR=pscalar_nolog`.

6.6 Bulk viscosity

For a monatomic gas it can be shown that the bulk viscosity vanishes. We therefore don't use it in most of our runs. However, for supersonic flows, or even otherwise, one might want to add a shock viscosity which, in its simplest formulation, take the form of a bulk viscosity.

6.6.1 Shock viscosity

Shock viscosity, as it is used here and also in the Stagger Code of Åke Nordlund, is proportional to the positive flow convergence, maximum over five zones, and smoothed to second order,

$$\zeta_{\text{shock}} = c_{\text{shock}} \left\langle \max_5 [(-\nabla \cdot \mathbf{u})_+] \right\rangle (\min(\delta x, \delta y, \delta z))^2, \quad (65)$$

where c_{shock} is a constant defining the strength of the shock viscosity. In the code this dimensionless coefficient is called `nu_shock`, and it is usually chosen to be around unity. Assume that the shock viscosity only enters as a bulk viscosity, so the whole stress tensor is then

$$\tau_{ij} = 2\rho\nu S_{ij} + \rho\zeta_{\text{shock}}\delta_{ij}\nabla \cdot \mathbf{u}. \quad (66)$$

Assume $\nu = \text{const}$, but $\zeta \neq \text{const}$, so

$$\rho^{-1} \mathbf{F}_{\text{visc}} = \nu \left(\nabla^2 \mathbf{u} + \frac{1}{3} \nabla \nabla \cdot \mathbf{u} + 2\mathbf{S} \cdot \nabla \ln \rho \right) + \zeta_{\text{shock}} [\nabla \nabla \cdot \mathbf{u} + (\nabla \ln \rho + \nabla \ln \zeta_{\text{shock}}) \nabla \cdot \mathbf{u}]. \quad (67)$$

and

$$\rho^{-1} \Gamma_{\text{visc}} = 2\nu \mathbf{S}^2 + \zeta_{\text{shock}} (\nabla \cdot \mathbf{u})^2. \quad (68)$$

In the special case with periodic boundary conditions, we have $2\langle \mathbf{S}^2 \rangle = \langle \omega^2 \rangle + \frac{4}{3} \langle (\nabla \cdot \mathbf{u})^2 \rangle$.¹³

6.7 Equation of state

In its present configuration only hydrogen ionization is explicitly included. Other constituents (currently He and H₂) can have fixed values. The pressure is proportional to the total number of particles, i.e.

$$p = (n_{\text{HI}} + n_{\text{HII}} + n_{\text{H}_2} + n_e + n_{\text{He}} + \dots) k_B T. \quad (69)$$

It is convenient to normalize to the total number of H both in atomic and in molecular hydrogen, $n_{\text{Htot}} \equiv n_{\text{H}} + 2n_{\text{H}_2}$, where $n_{\text{HI}} + n_{\text{HII}} = n_{\text{H}}$, and define $x_e \equiv n_e/n_{\text{Htot}}$, $x_{\text{He}} \equiv n_{\text{He}}/n_{\text{Htot}}$, and $x_{\text{H}_2} \equiv n_{\text{H}_2}/n_{\text{Htot}}$. Substituting $n_{\text{H}} = n_{\text{Htot}} - 2n_{\text{H}_2}$, we have

$$p = (1 - x_{\text{H}_2} + x_e + x_{\text{He}} + \dots) n_{\text{Htot}} k_B T. \quad (70)$$

This can be written in the more familiar form

$$p = \frac{\mathcal{R}}{\mu} \rho T, \quad (71)$$

where $\mathcal{R} = k_B/m_u$ and m_u is the atomic mass unit (which is for all practical purposes the same as m_{Htot}) and

$$\mu = \frac{n_{\text{H}} + 2n_{\text{H}_2} + n_e + 4n_{\text{He}}}{n_{\text{H}} + n_{\text{H}_2} + n_e + n_{\text{He}}} = \frac{1 + 4x_{\text{He}}}{1 - x_{\text{H}_2} + x_e + x_{\text{He}}} \quad (72)$$

is the mean molecular weight (which is here dimensionless; see Kippenhahn & Weigert 1990, p. 102). The factor 4 is really to be substituted for 3.97153. Some of the familiar relations take still the usual form, in particular $e = c_v T$ and $h = c_p T$ with $c_v = \frac{3}{2} \mathcal{R}/\mu$ and $c_p = \frac{5}{2} \mathcal{R}/\mu$.

The number ratio, x_{He} , is more commonly expressed as the mass ratio, $Y = m_{\text{He}} n_{\text{He}} / (m_{\text{H}} n_{\text{Htot}} + m_{\text{He}} n_{\text{He}})$, or $Y = 4x_{\text{He}} / (1 + 4x_{\text{He}})$, or $4x_{\text{He}} = (1/Y - 1)^{-1}$. For example, $Y = 0.27$ corresponds to $x_{\text{He}} = 0.092$ and $Y = 0.25$ to $x_{\text{He}} = 0.083$. Note also that for 100% H₂ abundance, $x_{\text{H}_2} = 1/2$.

In the following, the ionization fraction is given as $y = n_e/n_{\text{H}}$, which can be different from x_e if there is H₂. Substituting for n_{H} in terms of n_{Htot} yields $y = x_e / (1 - 2x_{\text{H}_2})$.

6.8 Ionization

This part of the code can be invoked by setting EOS=eos_ionization (or EOS=eos_temperature_ionization) in the 'Makefile.local' file. The equation of state described below works for variable ionization, and the entropy offset is different from that used in Eq. (43), which is now no longer applicable. As a replacement, one can use EOS=eos_fixed_ionization, where the degree of ionization can be given by hand. Here the normalization of the entropy is the same as for EOS=eos_ionization. This case is described in more detail below.¹⁴

We treat the gas as being composed of partially ionized hydrogen and neutral helium. These are four different particle species, each of which regarded as a perfect gas.

¹³More generally, we have $2\mathbf{S}^2 = -\nabla \cdot \mathcal{F}^\omega - \omega^2 + \frac{4}{3}(\nabla \cdot \mathbf{u})^2$, where $\mathcal{F}^\omega = \omega \times \mathbf{u} - \frac{4}{3} \mathbf{u} \nabla \cdot \mathbf{u}$ is proportional to a viscous flux term. The 4/3 factor in front of the $\langle (\nabla \cdot \mathbf{u})^2 \rangle$ contribution to the heating was incorrectly given as 2/3 in Ref. [15].

¹⁴We omit here the contribution of H₂.

The ionization fraction y , which gives the ratio of ionized hydrogen to the total amount of hydrogen n_{H} , is obtained from the Saha equation which, in this case, may be written as

$$\frac{y^2}{1-y} = \frac{1}{n_{\text{H}}} \left(\frac{m_e k_{\text{B}} T}{2\pi \hbar^2} \right)^{3/2} \exp \left(-\frac{\chi_{\text{H}}}{k_{\text{B}} T} \right). \quad (73)$$

The temperature T cannot be obtained directly from the PENCIL CODE's independent variables $\ln \rho$ and s , but is itself dependent on y . Hence, the calculation of y essentially becomes a root finding problem.

The entropy of a perfect gas consisting of particles of type i is known from the Sackur-Tetrode equation

$$S_i = k_{\text{B}} N_i \left(\ln \left[\frac{1}{n_{\text{tot}}} \left(\frac{m_i k_{\text{B}} T}{2\pi \hbar^2} \right)^{3/2} \right] + \frac{5}{2} \right). \quad (74)$$

Here N_i is the number of particles of a single species and n_{tot} is the total number density of all particle species.

In addition to the individual entropies we also have to take the entropy of mixing, $S_{\text{mix}} = -N_{\text{tot}} k_{\text{B}} \sum_i p_i \ln p_i$, into account. Summing up everything, we can get a closed expression for the specific entropy s in terms of y , $\ln \rho$ and T , which may be solved for T .

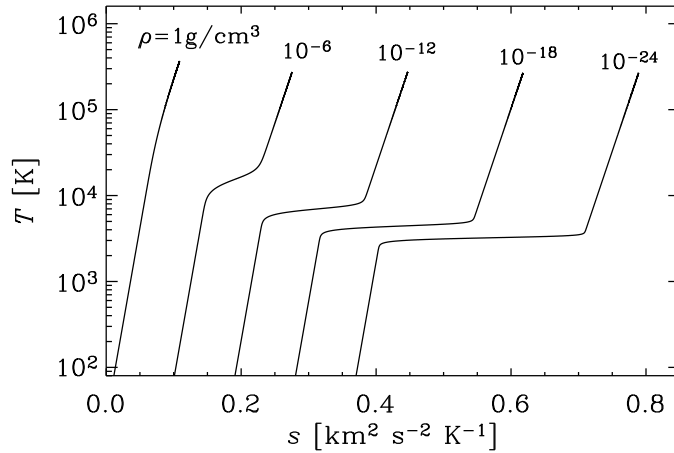


Figure 6: Dependence of temperature on entropy for different values of the density.

For given $\ln \rho$ and s we are then able to calculate the ionization fraction y by finding the root of

$$f(y) = \ln \left[\frac{1-y}{y^2} \frac{1}{n_{\text{H}}} \left(\frac{m_e k_{\text{B}} T(y)}{2\pi \hbar^2} \right)^{3/2} \right] - \frac{\chi_{\text{H}}}{k_{\text{B}} T(y)}. \quad (75)$$

In the ionized case, several thermodynamic quantities of the gas become dependent on the ionization fraction y such as its pressure, $P = (1 + y + x_{\text{He}}) n_{\text{H}} k_{\text{B}} T$, and its internal energy, $E = \frac{3}{2} (1 + y + x_{\text{He}}) n_{\text{H}} k_{\text{B}} T + y \chi_{\text{H}}$, where x_{He} gives the ratio of neutral helium to the total amount of hydrogen. The dependence of temperature on entropy is shown in Fig. 6 for different values of the density.

Note that for an ideal gas, $\ln T = s/c_v + (\gamma - 1) \ln \rho$, and that c_v is larger when $y = 1$, and smaller when $y = 0$. But this is a large effect when ρ is small.

For further details regarding the procedure of solving for the entropy see Sect. H.6 in the appendix. The full set of ionization equations is presented in [5].

6.8.1 Ambipolar diffusion

Another way of dealing with ionization in the PENCIL CODE is through use of the *neutrals* module. That module solves the coupled equations of neutral and ionized gas, in a two-fluid model

$$\frac{\partial \rho_i}{\partial t} = -\nabla \cdot (\rho_i \mathbf{u}_i) + \mathcal{G} \quad (76)$$

$$\frac{\partial \rho_n}{\partial t} = -\nabla \cdot (\rho_n \mathbf{u}_n) - \mathcal{G} \quad (77)$$

$$\frac{\partial (\rho_i \mathbf{u}_i)}{\partial t} = -\nabla \cdot (\rho_i \mathbf{u}_i : \mathbf{u}_i) - \nabla \left(p_i + p_e + \frac{B^2}{2\mu_0} \right) + \mathcal{F} \quad (78)$$

$$\frac{\partial (\rho_n \mathbf{u}_n)}{\partial t} = -\nabla \cdot (\rho_n \mathbf{u}_n : \mathbf{u}_n) - \nabla p_n - \mathcal{F} \quad (79)$$

$$\frac{\partial \mathbf{A}}{\partial t} = \mathbf{u}_i \times \mathbf{B} \quad (80)$$

where the subscripts n and i are for neutral and ionized, respectively. The terms $\mathcal{G}$ and $\mathcal{F}$, through which the two fluids exchange mass and momentum, are given by

$$\mathcal{G} = \zeta \rho_n - \alpha \rho_i^2 \quad (81)$$

$$\mathcal{F} = \zeta \rho_n \mathbf{u}_n - \alpha \rho_i^2 \mathbf{u}_i + \gamma \rho_i \rho_n (\mathbf{u}_n - \mathbf{u}_i). \quad (82)$$

In the above equations, ζ is the ionization coefficient, α is the recombination coefficient, and γ the collisional drag strength. By the time of writing (spring 2009), these three quantities are supposed constant. The electron pressure p_e is also assumed equal to the ion pressure. Only isothermal neutrals are supported so far.

In the code, Eq. (76) and Eq. (78) are solved in ‘density.f90’ and ‘hydro.f90’ respectively. Equation 77 is solved in ‘neutrals.f90’ and Eq. (79) in ‘neutralvelocity.f90’. The sample ‘1d-test/ambipolar-diffusion’ has the current setup for a two-fluid simulation with ions and neutrals.

6.9 Combustion

The easiest entry into the subject of simulating combustion is through samples/0d-tests/chemistry_H2_ignition_rkf or samples/1d-tests/H2_flamespeed. The former case is studying H2 ignition delay, while the second one is focusing on H2 flame-speed. If you want to study turbulent premixed flames, the recommended cases are samples/2d-tests/2d_methane_flame and samples/turbulent_flame. Here, the first of these examples is not really realistic since its only 2D, but this does of course make it faster to try out. Also, this case is easier to set up. The most realistic test case is samples/turbulent_flame, which studies a full 3D hydrogen flame. This test case requires that a set of smaller pre-runs are finalized before the main simulation can be run (see the associated README file for more details).

The `chemistry_H2_ignition_rkf` directory, for example, has the file `'tran.dat'`, which contains the parameters characterizing the transport properties, and `'chem.inp'`, which contains the NASA polynomials; see Eq. (18) of Ref. [2].

To set the number of chemical species required add the following to your `'src/cparam.local'` where the first number is the number of species and the second one is the number of degrees of freedom per species:

```
! NCHEMSPEC CONTRIBUTION 16*1
```

Importantly the code always assumes a single degree of freedom per chemical species and thus changing the second parameter won't have any effect.

6.10 Radiative transfer

Here we only state the basic equations. A full description about the implementation is given in Sect. H.7 and in the original paper by Heinemann et al. (2006).

The basic equation for radiative transfer is

$$\frac{dI}{d\tau} = -I + S, \quad (83)$$

where

$$\tau \equiv \int_0^s \chi(s') ds' \quad (84)$$

is the optical depth (s is the geometrical coordinate along the ray).

Note that radiative transfer is called also in `'start.csh'`, and again each time a snapshot is being written, provided the output of auxiliary variables is being requested `lwrite_aux=T`. (Also, of course, the pencil check runs radiative transfer 7 times, unless you put `pencil_check_small=F`.)

6.11 Self-gravity

The PENCIL CODE can consider the self-gravity of the fluid in the simulation box by adding the term

$$\frac{\partial \mathbf{u}}{\partial t} = \dots - \nabla \phi_{\text{self}} \quad (85)$$

to the equation of motion. The self-potential ϕ_{self} (or just ϕ for simplicity) satisfies Poisson's equation

$$\nabla^2 \phi = 4\pi G \rho. \quad (86)$$

The solution for a single Fourier component at scale $\mathbf{k}$ is

$$\phi_{\mathbf{k}} = -\frac{4\pi G \rho_{\mathbf{k}}}{k^2}. \quad (87)$$

Here we have assumed periodic boundary conditions. The potential is obtained by Fourier-transforming the density, then finding the corresponding potential at that scale, and finally Fourier-transforming back to real space.

The x -direction in the shearing sheet is not strictly periodic, but is rather shear periodic with two connected points at the inner and outer boundary separated by the distance $\Delta y(t) = \text{mod}[(3/2)\Omega_0 L_x t, L_y]$ in the y -direction. We follow here the method from [23]

to allow for shear-periodic boundaries in the Fourier method for self-gravity. First we take the Fourier transform along the periodic y -direction. We then shift the entire y -direction by the amount $\delta y(x) = \Delta y(t)x/L_x$ to make the x -direction periodic. Then we proceed with Fourier transforms along x and then z . After solving the Poisson equation in Fourier space, we transform back to real space in the opposite order. We differ here from the method by [23] in that we shift in Fourier space rather than in real space¹⁵. The Fourier interpolation formula has the advantage over polynomial interpolation in that it is continuous and smooth in all its derivatives.

6.12 Incompressible and anelastic equations

This part has not yet been documented and is still under development.

6.13 Dust equations

The code treats gas and dust as two separate fluids¹⁶. The dust and the gas interact through a drag force. This force can most generally be written as an additional term to the equation of motion as

$$\frac{D\mathbf{u}_d}{Dt} = \dots - \frac{1}{\tau_s} (\mathbf{u}_d - \mathbf{u}) . \quad (88)$$

Here τ_s is the so-called stopping time of the considered dust species. This measures the coupling strength between dust and gas. In the Epstein drag regime

$$\tau_s = \frac{a_d \rho_s}{c_s \rho} , \quad (89)$$

where a_d is the radius of the dust grain and ρ_s is the solid density of the dust grain.

Two other important effects work on the dust. The first is coagulation controlled by the discrete coagulation equation

$$\frac{dn_k}{dt} = \frac{1}{2} \sum_{i+j=k} A_{ij} n_i n_j - n_k \sum_{i=1}^{\infty} A_{ik} n_i . \quad (90)$$

In the code N discrete dust species are considered. Also the bins are logarithmically spaced in order to give better mass resolution. It is also possible to keep track of both number density and mass density of each bin, corresponding to having a variable grain mass in each bin.

Dust condensation is controlled by the equation

$$\frac{dN}{dt} = \frac{1}{\tau_{\text{cond}}} N^{\frac{d-1}{d}} . \quad (91)$$

Here N is the number of monomers in the dust grain (such as water molecules) and d is the physical dimension of the dust grain. The condensation time τ_{cond} is calculated from

$$\frac{1}{\tau_{\text{cond}}} = A_1 v_{\text{th}} \alpha n_{\text{mon}} \left\{ 1 - \frac{1}{S_{\text{mon}}} \right\} , \quad (92)$$

¹⁵We were kindly made aware of the possibility of interpolating in Fourier space by C. McNally on his website.

¹⁶See master's thesis of A. Johansen (can be downloaded from http://www.mpia.de/homes/johansen/research_en.php)

where A_1 is the surface area of a monomer, α is the condensation efficiency, n_{mon} is the number density of monomers in the gas and S_{mon} is the saturation level of the monomer given by

$$S_{\text{mon}} = \frac{P_{\text{mon}}}{P_{\text{sat}}}. \quad (93)$$

Here P_{sat} is the saturated vapor pressure of the monomer. Currently only water ice has been implemented in the code.

All dust species fulfill the continuity equation

$$\frac{\partial \rho_d}{\partial t} + \nabla \cdot (\rho_d \mathbf{u}_d) = 0. \quad (94)$$

To set the number of dust species and their degrees of freedom add the following to your ‘src/cparam.local’ where the first number is the number of species and the second one is the number of degrees of freedom per species:

```
! NDUSTSPEC CONTRIBUTION 3*4
```

6.14 Cosmic ray pressure in diffusion approximation

Cosmic rays are treated in the diffusion approximation. The equation of state is $p_c = (\gamma_c)e_c$ where the value of γ_c is usually somewhere between 14/9 and 4/3. In the momentum equation (44) the cosmic ray pressure force, $-\rho^{-1}\nabla p_c$ is added on the right hand side, and e_c satisfies the evolution equation

$$\frac{\partial e_c}{\partial t} + \nabla \cdot (e_c \mathbf{u}) + p_c \nabla \cdot \mathbf{u} = \partial_i (K_{ij} \partial_j e_c) + Q_c, \quad (95)$$

where Q_c is a source term and

$$K_{ij} = K_{\perp} \delta_{ij} + (K_{\parallel} - K_{\perp}) \hat{B}_i \hat{B}_j \quad (96)$$

is an anisotropic diffusivity tensor.

In the non-conservative formulation of this code it is advantageous to expand the diffusion term using the product rule, i.e.

$$\partial_i (K_{ij} \partial_j e_c) = -U_c \cdot \nabla e_c + K_{ij} \partial_i \partial_j e_c. \quad (97)$$

where $U_{ci} = -\partial K_{ij} / \partial x_j$ acts like an extra velocity trying to straighten magnetic field lines. We can write this term also as $U_c = -(K_{\parallel} - K_{\perp}) \nabla \cdot (\hat{B} \hat{B})$, where the last term is a divergence of the dyadic product of unit vectors.¹⁷ However, near magnetic nulls, this term can become infinite. In order to avoid this problem we are forced to limit $\nabla \cdot (\hat{B} \hat{B})$, and hence $|U_c|$, to the maximum possible value that can be resolved at a given resolution.

A physically appealing way of limiting the maximum propagation speed is to restore an explicit time dependence in the equation for the cosmic ray flux, and to replace the diffusion term in Eq. (95) by a divergence of a flux that in turn obeys the equation

$$\frac{\partial \mathcal{F}_{ci}}{\partial t} = -\tilde{K}_{ij} \nabla_j e_c - \frac{\mathcal{F}_{ci}}{\tau} \quad (\text{non-Fickian diffusion}), \quad (98)$$

where $K_{ij} = \tau \tilde{K}_{ij}$ would be the original diffusion tensor of Eq. (96), if the time derivative were negligible. Further details are described in Snodin et al. (2006).

¹⁷In practice, we calculate $\partial_j (\hat{B}_i \hat{B}_j) = (\delta_{ij} - 2\hat{B}_i \hat{B}_j) \partial_j \hat{B}_k B_{k,j} / |B|$, where derivatives of B are calculated as $B_{i,j} = \epsilon_{ikl} A_{l,jk}$.

6.15 Chiral MHD

At high energies, $k_B T \gtrsim 10$ MeV, a quantum effect called the *chiral magnetic effect* (CME) can modify the MHD equations. The CME occurs in magnetized relativistic plasmas, in which the number density of left-handed fermions, n_L , differs from the one of right-handed fermions, n_R (see e.g., Kharzeev et al. (2013) for a review). This asymmetry is described by the chiral chemical potential $\mu_5 \equiv 6(n_L - n_R)(\hbar c)^3/(k_B T)^2$, where T is the temperature, k_B is the Boltzmann constant, c is the speed of light, and $\hbar$ is the reduced Planck constant. In the presence of a magnetic field, μ_5 leads to the occurrence of the current

$$\mathbf{J}_{\text{CME}} = \frac{\alpha_{\text{em}}}{\pi \hbar} \mu_5 \mathbf{B}, \quad (99)$$

where $\alpha_{\text{em}} \approx 1/137$ is the fine structure constant.

The chiral current (99) adds to the classical Ohmic current, leading to a modification of the Maxwell equations. As a result, the induction equation is extended by one additional term:

$$\frac{\partial \mathbf{A}}{\partial t} = \mathbf{U} \times \mathbf{B} - \eta (\nabla \times \mathbf{B} - \mu \mathbf{B}), \quad (100)$$

where the chiral chemical potential μ_5 is normalized such that $\mu = (4\alpha_{\text{em}}/\hbar c)\mu_5$. The latter is determined by the evolution equation

$$\frac{D\mu}{Dt} = D_5 \Delta \mu + \lambda \eta [\mathbf{B} \cdot (\nabla \times \mathbf{B}) - \mu \mathbf{B}^2] - \Gamma_f \mu, \quad (101)$$

where D_5 is a chiral diffusion coefficient, λ the chiral feedback parameter, and Γ_f the rate of chiral flipping reactions. All remaining evolution equations are the same as in classical MHD. Details on the derivation of the chiral MHD equations can be found in Boyarsky et al. (2015) and Rogachevskii et al. (2017).

In the Pencil Code, the chiral MHD equations can be solved by adding the line

```
SPECIAL      =      special/chiral_mhd
```

to `src/Makefile.local`. Further, for running the chiral MHD module, one needs to add

```
&special_init_pars
initspecial='const', mu5_const=10.
```

to `start.in` and

```
&special_run_pars
diffmu5=1e-4, lambda5=1e3, cdtchiral=1.0
```

to `run.in`, where we have chosen exemplary values for the chiral parameters.

Caution should be taken when solving the chiral MHD equations numerically, since the evolution of the plasma can be strongly affected by the CME. In particular, the initial value of μ is related to a small-scale dynamo instability. In order to resolve this instability in a domain of size $(2\pi)^3$, the minimum number of grid points is given as:

$$N_{\text{grid}} \gtrsim \left(\frac{\mu_0}{\lambda}\right)^{1/2} \frac{2\pi}{\nu \text{Re}_{\text{mesh,crit}}}. \quad (102)$$

Also the value of λ should not be chosen too small, since it scales inversely with the saturation magnetic helicity produced by a chiral dynamo. Hence, for a very small λ

parameter, the Alfvén time step becomes extremely small in the non-linear stage which can lead to a code crash. More details on the numerical modeling of chiral MHD can be found in Schober et al. (2018).

6.16 Electromagnetism with displacement current

In electromagnetism, the displacement current is not neglected, and so we solve the equation

$$\frac{1}{c^2} \frac{\partial \mathbf{E}}{\partial t} = \nabla \times \mathbf{B} - \mu_0 \mathbf{J}, \quad (103)$$

in addition to the induction equation $\partial \mathbf{B} / \partial t = -\nabla \times \mathbf{E}$, or its uncurled version $\partial \mathbf{A} / \partial t = -\mathbf{E}$. Solving this equation is invoked by putting `SPECIAL=special/displacement`.

In the code, there is the line `df(l1:l2,m,n,iex:iez)=df(l1:l2,m,n,iex:iez)+c_light2*(p%curlb-mu0*p%jj_ohm)` where `c_light2` is the speed of light squared as input parameter, so it is usually not computed self-consistently from the actual speed of light, which would be known once we use physical dimensions. Furthermore, `p%curlb` is the curl of $\mathbf{B}$ (which is now *not* the electric current, while `p%jj_ohm` is the Ohmic current, obtained by solving $\mathbf{J} = \sigma(\mathbf{E} + \mathbf{u} \times \mathbf{B})$, and σ is the conductivity (which can be time-dependent). The vacuum permeability is `mu0`.

The electric field obtained in this way, which we now call the pseudo-electric field, $\tilde{\mathbf{E}}$, does in general not obey the equation for the charge density, ρ_e ,

$$\nabla \cdot \mathbf{E} = \rho_e / \epsilon_0. \quad (104)$$

The actual electric field $\mathbf{E}$ can be obtained from $\tilde{\mathbf{E}}$ as

$$\mathbf{E} = \tilde{\mathbf{E}} - \nabla \phi, \quad (105)$$

where ϕ is obtained by solving a Poisson-like equation,

$$\nabla^2 \phi = \nabla \cdot \tilde{\mathbf{E}} - \rho_e / \epsilon_0, \quad (106)$$

where ρ_e is obtained through time-integration of the charge continuity equation,

$$\frac{\partial \rho_e}{\partial t} = -\nabla \cdot \mathbf{J}, \quad (107)$$

which, in turn, follows from Eq. (103) by taking its divergence.

6.17 Particles

Particles are entities that each have a space coordinate and a velocity vector, where a fluid only has a velocity vector field (the continuity equation of a fluid in some way corresponds to the space coordinate of particles). In the code particles are present either as tracer particles or as dust particles

6.17.1 Tracer particles

Tracer particles always have the local velocity of the gas. The dynamical equations are thus

$$\frac{\partial \mathbf{x}_i}{\partial t} = \mathbf{u}, \quad (108)$$

where the index i runs over all particles. Here $\mathbf{u}$ is the gas velocity at the position of the particle. One can choose between a first order (default) and a second order spline interpolation scheme (set `lquadratic_interpolation=T` in `&particles_init_pars`) to calculate the gas velocity at the position of a tracer particle.

The sample run ‘`samples/dust-vortex`’ contains the latest setup for tracer particles.

6.17.2 Dust particles

Dust particles are allowed to have a velocity that is not similar to the gas,

$$\frac{d\mathbf{x}_i}{dt} = \mathbf{v}_i. \quad (109)$$

The particle velocity follows an equation of motion similar to a fluid, only there is no advection term. Dust particles also experience a drag force from the gas (proportional to the velocity difference between a particle and the gas).

$$\frac{d\mathbf{v}_i}{dt} = \dots - \frac{1}{\tau_s}(\mathbf{v}_i - \mathbf{u}). \quad (110)$$

Here τ_s is the stopping time of the dust particle. The interpolation of the gas velocity to the position of a particle is done using one of three possible particle-mesh schemes,

- NGP (Nearest Grid Point, default)
The gas velocity at the nearest grid point is used.
- CIC (Cloud in Cell, set `lparticlemesh_cic=T`)
A first order interpolation is used to obtain the gas velocity field at the position of a particle. Affects 8 grid points.
- TSC (Triangular Shaped Cloud, set `lparticlemesh_tsc=T`)
A second order spline interpolation is used to obtain the gas velocity field at the position of a particle. Affects 27 grid points.

The particle description is the proper description of dust grains, since they do not feel any pressure forces (too low number density). Thus there is no guarantee that the grains present within a given volume will be equilibrated with each other, although drag force may work for small grains to achieve that. Larger grains (meter-sized in protoplanetary discs) must be treated as individual particles.

To conserve momentum the dust particles must affect the gas with a friction force as well. The strength of this force depends on the dust-to-gas ratio ϵ_d , and it can be safely ignored when there is much more gas than there is dust, e.g., when $\epsilon_d = 0.01$. The friction force on the gas appears in the equation of motion as

$$\frac{\partial \mathbf{u}}{\partial t} = \dots - \frac{\rho_p^{(i)}}{\rho} \left(\frac{\partial \mathbf{v}^{(i)}}{\partial t} \right)_{\text{drag}} \quad (111)$$

Here $\rho_p^{(i)}$ is the dust density that particle i represents. This can be set through the parameter `eps_todt` in `&particle_init_pars`. The drag force is assigned from the particles onto the mesh using either NGP, CIC or TSC assignment. The same scheme is used both for interpolation and for assignment to avoid any risk of a particle accelerating itself (see Hockney & Eastwood 1981).

6.18 *N*-body solver

The *N*-body code takes advantage of the existing Particles module, which was coded with the initial intent of treating solid particles whose radius $a_\bullet$ is comparable to the mean free path λ of the gas, for which a fluid description is not valid. A *N*-body implementation based on that module only needed to include mass as extra state for the particles, solve for the N^2 gravitational pair interactions and distinguish between the *N*-body and the small bodies that are mapped into the grid as a ρ_p density field.

The particles of the *N*-body ensemble evolve due to their mutual gravity and by interacting with the gas and the swarm of small bodies. The equation of motion for particle i is

$$\frac{d\mathbf{v}_{p_i}}{dt} = \mathbf{F}_{g_i} - \sum_{j \neq i}^N \frac{GM_j}{\mathcal{R}_{ij}^2} \hat{\mathcal{R}}_{ij} \quad (112)$$

where $\mathcal{R}_{ij} = |\mathbf{r}_{p_i} - \mathbf{r}_{p_j}|$ is the distance between particles i and j , and $\hat{\mathcal{R}}_{ij}$ is the unit vector pointing from particle j to particle i . The first term of the R.H.S. is the combined gravity of the gas and of the dust particles onto the particle i , solved via

$$\mathbf{F}_{g_i} = -G \int_V \frac{[\rho_g(\mathbf{r}) + \rho_p(\mathbf{r})] \mathcal{R}_i}{(\mathcal{R}_i^2 + b_i^2)^{3/2}} dV, \quad (113)$$

where the integration is carried out over the whole disk. The smoothing distance b_i is taken to be as small as possible (a few grid cells). For few particles (<10), calculating the integral for every particle is practical. For larger ensembles one would prefer to solve the Poisson equation to calculate their combined gravitational potential.

The evolution of the particles is done with the same third-order Runge-Kutta time-stepping routine used for the gas. The particles define the timestep also by the Courant condition that they should not move more than one cell at a time. For pure particle runs, where the grid is absent, one can adopt a fixed time-step $t_p \ll 2\pi\Omega_{fp}^{-1}$ where Ω_{fp} is the angular frequency of the fastest particle.

By now (spring 2009), no inertial accelerations are included in the *N*-body module, so only the inertial frame - with origin at the barycenter of the *N*-body ensemble - is available. For a simulation of the circular restricted three-body problem with mass ratio $q=10^{-3}$, the Jacobi constant of a test particle initially placed at position $(x, y)=(2, 0)$ was found to be conserved up to one part in 10^5 within the time span of 100 orbits.

We stress that the level of conservation is poor when compared to integrators designed to specifically deal with long-term *N*-body problems. These integrators are usually symplectic, unlike the Runge-Kutta scheme of the PENCIL CODE. As such, PENCIL should not be used to deal with evolution over millions of years. But for the time-span typical of astrophysical hydrodynamical simulations, this degree of conservation of the Jacobi constant can be deemed acceptable.

As an extension of the particle's module, the *N*-body is fully compatible with the parallel optimization of PENCIL, which further speeds up the calculations. Parallelization, however, is not yet possible for pure particle runs, since it relies on splitting the grid between the processors. At the time of writing (spring 2009), the *N*-body code does not allow the particles to have a time-evolving mass.

The module 'pointmasses.f90' is also an *N*-body solver.

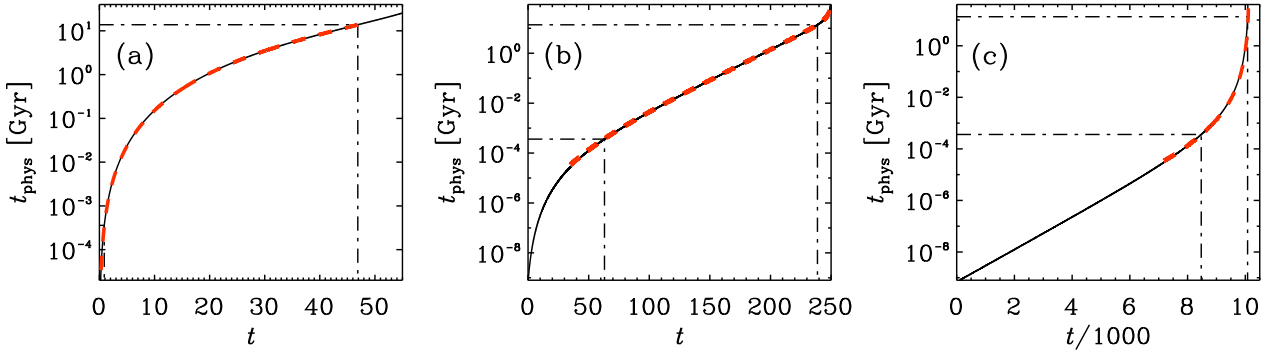


Figure 7: $\Omega_{\text{rad}} = 10^{-4}$, $\Omega_{\Lambda} = 0.73$, with $\Omega_{\text{rad}} = 10^{-4}$, $\Omega_{\text{mat}} = 0.31$, and $H_0 = 0.0692 \text{ Gyr}^{-1} \approx 0.71 \text{ km s}^{-1} \text{ kpc}^{-1}$. In the last panel, the dashed-dotted lines denote $z = 1100$ at $t_{\text{phys}} = 370,000 \text{ yr}$ and $z = 0$ at $t_{\text{phys}} = 13.8 \text{ Gyr}$.

6.19 Cosmological expansion and scale factor

Superconformal coordinates are defined through $dt = dt_{\text{phys}}/a^n$, where $n = 3/2$ [3] or $n = 2$ [33] and t_{phys} is the physical (or cosmic) time. To obtain $a(t)$, we assume a standard Λ CDM universe and integrate $d \ln a / dt_{\text{phys}} = H(a)$, where

$$H(a) = H_0 \sqrt{\Omega_{\text{rad}}/a^4 + \Omega_{\text{mat}}/a^3 + \Omega_{\Lambda}} \quad (114)$$

is the prescribed dependence of the Hubble parameter on $a(t)$. We work with conformal time t , use $d/dt_{\text{phys}} = a^{-n} d/dt$, and integrate to obtain $t_{\text{phys}}(t)$ and $a(t)$, i.e.,

$$\frac{dt_{\text{phys}}}{dt} = a^n \quad \text{and} \quad \frac{d \ln a}{dt} = a^n H(a). \quad (115)$$

To obtain the initial conditions for early times, we consider the limit $a \rightarrow 0$, so Eq. (114) becomes $H(a) = H_0 \Omega_{\text{rad}}^{1/2}/a^2$. We then integrate $t_{\text{phys}} = \int da/(aH)$, which yields $t_{\text{phys}} = a^2/(2H_0 \Omega_{\text{rad}}^{1/2})$, i.e., $a = (2H_0 \Omega_{\text{rad}}^{1/2} t_{\text{phys}})^{1/2}$. This relation is independent of n , but assumes that we start at a redshift that is well in the radiation-dominated era. Here we consider the initial redshifts $z_* = 4500$ and $z_* = 10^6$. Thus, we solve Eq. (115) with the initial conditions

$$a = a_* \equiv 1/(1+z_*), \quad t_{\text{phys}} = t_{\text{phys}}^* \equiv a_*^2/(2H_0 \Omega_{\text{rad}}^{1/2}). \quad (116)$$

The sample samples/2d-tests/dark-matter uses the special module SPECIAL=special/Lambda_CDM with input

```
&special_init_pars
  ascale_type='general'
  redshift0=4500.
  nconformal=2.
/
```

In Fig. 7, we compare the dependence of $t_{\text{phys}}(t)$ for $n = 2$, $3/2$, and 1 using $z_* = 4500$ and 10^6 and a constant time step. The range $10^3 \geq a(t) \geq 1$, corresponding to the time interval from recombination to the present time, is marked by dashed-dotted lines. We see that the dependence $t_{\text{phys}}(t)$ is concave for $n = 2$ and convex for $n = 1$, but approximately linear for $n = 3/2$. This indicates that the exponent $n = 3/2$ distributes the local change in $t_{\text{phys}}(t)$ approximately uniformly over the interval from recombination to the present time.

We recall that in all cases, our initial conformal time is always zero. However, when comparing $t_{\text{phys}}(t)$ for different initial redshifts, we can make the curves overlap by adding a suitable offset t_0 to $t \rightarrow t + t_0$ for the runs with the smaller initial redshift. We see that the curves for $z_* = 4500$ and 10^6 overlap well, although there is a very small difference at the very beginning of the runs with $z_* = 4500$ relative to those with $z_* = 10^6$.

6.20 Scale factor at the end of reheating

At the end of cosmological inflation, during what is called reheating, the scale factor changes from a quadratic to a linear growth profile. A smooth function $a(t)$ is obtained by solving the Friedmann equations for a piecewise constant equation of state, $w(a)$, which relates the pressure with the density through $p = w\rho$. Under the assumption of zero curvature, i.e., the Universe is conformally flat, but expanding, the Friedmann equations can be written as a single equation which, in physical time, takes the form

$$a^{-1} \frac{d^2 a}{dt_{\text{phys}}^2} = -\frac{1}{2} H^2 [1 + 3w(a)], \quad (117)$$

where $H = a^{-1} da/dt_{\text{phys}}$ is the standard Hubble parameter. Here, $w(a) = 1/3$ during the radiation-dominated era and $w(a) = 0$ during reheating when there were no photons, which is therefore equivalent to the matter-dominated era that also occurs later after recombination and before the Universe began to accelerate again. The accelerated exponential expansion of the Universe during inflation, and also the late acceleration of the present Universe, correspond to $w = -1$, but this will not be considered in the present paper.

It is convenient to solve the Friedmann equation with zero curvature in conformal time. It then takes the form $a''/a = \frac{1}{2} \mathcal{H}^2 (1 - 3w)$, where $\mathcal{H} = a'/a$ is the conformal Hubble parameter. It is related to the usual one, H , through $\mathcal{H} = da/\partial t_{\text{phys}} = aH$. Note the opposite sign of the terms on the right-hand side and the opposite sign in front of $3w(a)$ compared to the formulation in terms of physical time. The equation for a'' is easily solved by splitting it into two first-order equations and introducing a new variable $b(t)$ and solving for

$$a' = b, \quad b' = (b^2/2a)(1 - 3w); \quad (118)$$

see Refs. [26, 19] for similar work in another context.

Fig. 8 shows the solution for $a(t)$ and the ratios a'/a and a''/a compensated by t and t^2 , respectively, which allows us to see more clearly how a'/a changes from $2/t$ to $1/t$ and a''/a changes from $2/t^2$ to zero as we go from the reheating era to the radiation-dominated Universe after reheating.

The sample `samples/1d-tests/inflationary-B-genesis` reads in a file `a_vs_eta.dat` with input

```
&run_pars
  lread_scl_factor_file_new=T
/
```

which uses the subroutine `calc_scl_factor` in `sub.f90`.

6.21 Test-field equations

The test-field method is used to calculate turbulent transport coefficients for magneto-hydrodynamics. This is a rapidly evolving field and we refer the interested reader to

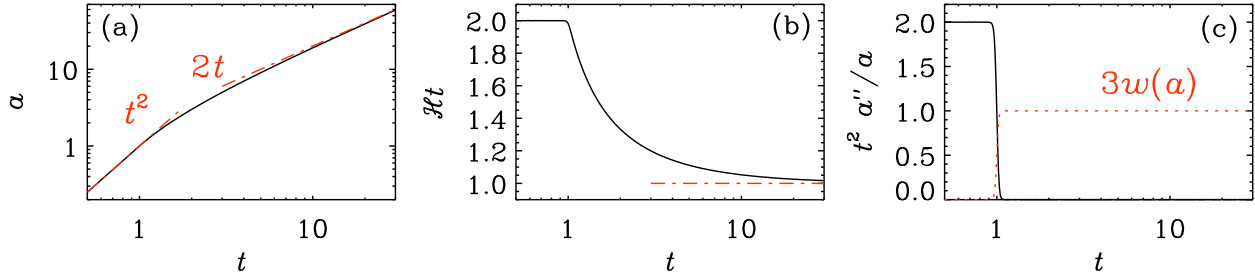


Figure 8: t dependence of (a) the scale factor $a(t)$, (b) the compensated Hubble coefficient $\mathcal{H}t = t a'/a$, and (c) the compensated left-hand side of the Friedmann equation, $t^2 a''/a$. In (a), the asymptotic dependences $a = t^2$ and $2t$ for $t \ll 1$ and $\gg 1$ are overplotted as dashed-dotted orange lines. In (c), the function $3w(a(t))$ is overplotted as a dotted red line.

Table 5: Scale factor and conformal Hubble parameter for different values of n .

n	a	$\mathcal{H}$	H
0	1	0	0
1/2	$\eta/2$	$1/\eta$	$1/\eta$
2/3	$\eta^2/3$	$2/\eta$	$6/\eta^2$

recent papers in this field, e.g., by Sur et al. (2008) or Brandenburg et al. (2008). For technical details; see also Sect. F.4.

6.22 Gravitational wave equations

The expansion of the universe with time is described by the scale factor $a(\tau)$, where τ is the physical time. Using conformal time, $t(\tau) = \int_0^\tau d\tau'/a(\tau')$, and dependent variables that are appropriately scaled with powers of a , the hydromagnetic equations can be expressed completely without scale factor [14, 22]. This is not true, however, for the gravitational wave (GW) equations, where a dependence on a remains [22]. The time dependence of a can be modeled as a power law, $a \propto \tau^n$, where $n = 1/2$ applies to the radiation-dominated era; see Table 5 the general relationship. To compare with cases where the expansion is ignored, we put $n = 0$.

In the transverse traceless (TT) gauge, the six components of the spatial part of the symmetric tensor characterizing the linearized evolution of the metric perturbations h_{ij} , reduce to two components which, in the linear polarization basis, are the $+$ and $\times$ polarizations. However, the projection onto that basis is computationally intensive, because it requires nonlocal operations involving Fourier transformations. It is therefore advantageous to evolve instead the perturbation of the metric tensor, h_{ij} , in an arbitrary gauge, compute then h_{ij}^{TT} in the TT gauge, and perform then the decomposition into the linear polarization basis whenever we compute diagnostic quantities such as averages or spectra. Thus, we solve the linearized GW equation in the form [22]

$$\frac{\partial^2 h_{ij}}{\partial t^2} = -2\mathcal{H} \frac{\partial h_{ij}}{\partial t} + c^2 \nabla^2 h_{ij} + \frac{16\pi G}{a^2 c^2} T_{ij} \quad (119)$$

for the six components $1 \leq i \leq j \leq 3$, where t is comoving time, a is the scale factor, $\mathcal{H} = \dot{a}/a$ is the comoving Hubble parameter, T_{ij} is the source term, c is the speed of light, and G is Newton's constant. For $n = 0$, when the cosmic expansion is ignored, we have

$a = 1$ and $\mathcal{H} = 0$. In practice, we solve the GW equation for the scaled variable $\tilde{h}_{ij} = ah_{ij}$,

$$\frac{\partial^2 \tilde{h}_{ij}}{\partial t^2} = c^2 \nabla^2 \tilde{h}_{ij} + \frac{16\pi G}{ac^2} T_{ij}. \quad (120)$$

For the numerical treatment of Eq. (119) or Eq. (120) and equations (122)–(124).

The source term is chosen to be the traceless part of the stress tensor,

$$T_{ij}(\mathbf{x}, t) = \rho u_i u_j - B_i B_j - \frac{1}{3} \delta_{ij} (\rho \mathbf{u}^2 - \mathbf{B}^2). \quad (121)$$

The removal of the trace is in principle not necessary, but it helps preventing a continuous build-up of a large trace, which would be numerically disadvantageous. We have ignored here the viscous stress, which is usually small.

We compute T_{ij} by solving the energy, momentum, and induction equations for an ultra-relativistic gas in the form [14, 16]

$$\frac{\partial \ln \rho}{\partial t} = -\frac{4}{3} (\nabla \cdot \mathbf{u} + \mathbf{u} \cdot \nabla \ln \rho) + \frac{1}{\rho} [\mathbf{u} \cdot (\mathbf{J} \times \mathbf{B}) + \eta \mathbf{J}^2], \quad (122)$$

$$\begin{aligned} \frac{D\mathbf{u}}{Dt} = & \frac{\mathbf{u}}{3} (\nabla \cdot \mathbf{u} + \mathbf{u} \cdot \nabla \ln \rho) - \frac{\mathbf{u}}{\rho} [\mathbf{u} \cdot (\mathbf{J} \times \mathbf{B}) + \eta \mathbf{J}^2] \\ & - \frac{1}{4} \nabla \ln \rho + \frac{3}{4\rho} \mathbf{J} \times \mathbf{B} + \frac{2}{\rho} \nabla \cdot (\rho \nu \mathbf{S}) + \mathbf{f}, \end{aligned} \quad (123)$$

$$\frac{\partial \mathbf{B}}{\partial t} = \nabla \times (\mathbf{u} \times \mathbf{B} - \eta \mathbf{J}), \quad (124)$$

where $\mathbf{B} = \nabla \times \mathbf{A}$ is the magnetic field expressed in terms of the magnetic vector potential to ensure that $\nabla \cdot \mathbf{B} = 0$, $\mathbf{J} = \nabla \times \mathbf{B}$ is the current density, $D/Dt = \partial/\partial t + \mathbf{u} \cdot \nabla$ is the advective derivative, $S_{ij} = \frac{1}{2}(u_{i,j} + u_{j,i}) - \frac{1}{3}\delta_{ij}u_{k,k}$ is the trace-free rate of strain tensor, and $p = \rho c_s^2$ is the pressure, where $c_s = c/\sqrt{3}$ is the sound speed for an ultra-relativistic gas. Lorentz-Heaviside units for the magnetic field are used.

We are interested in the rms value of the metric tensor perturbations and the GW energy density in the linear polarization basis. To compute h_{ij}^{TT} from h_{ij} , we Fourier transform the six components of h_{ij} and $\dot{h}_{ij}$,

$$\tilde{h}_{ij}(\mathbf{k}, t) = \int h_{ij}(\mathbf{x}, t) e^{-i\mathbf{k} \cdot \mathbf{x}} d^3x \quad \text{for } 1 \leq i \leq j \leq 3 \quad (125)$$

and compute the components in the TT gauge as

$$\tilde{h}_{ij}^{\text{TT}}(\mathbf{k}, t) = (P_{il}P_{jm} - \frac{1}{2}P_{ij}P_{lm}) \tilde{h}_{lm}(\mathbf{k}, t), \quad (126)$$

where $P_{ij} = \delta_{ij} - \hat{k}_i \hat{k}_j$ is the projection operator, and $\hat{\mathbf{k}} = \mathbf{k}/k$ is the unit vector of $\mathbf{k}$, with $k = |\mathbf{k}|$ being the modulus. Next, we compute the linear polarization bases

$$e_{ij}^+ = e_i^1 e_j^1 - e_i^2 e_j^2, \quad e_{ij}^\times = e_i^1 e_j^2 + e_i^2 e_j^1, \quad (127)$$

where e^1 and e^2 are unit vectors perpendicular to $\mathbf{k}$. Thus

$$\tilde{h}_+(\mathbf{k}, t) = \frac{1}{2} e_{ij}^+(\mathbf{k}) \tilde{h}_{ij}(\mathbf{k}, t), \quad (128)$$

$$\tilde{h}_\times(\mathbf{k}, t) = \frac{1}{2} e_{ij}^\times(\mathbf{k}) \tilde{h}_{ij}(\mathbf{k}, t). \quad (129)$$

We then return into real space and compute

$$h_{+/\times}(\mathbf{x}, t) = \int \tilde{h}_{+/\times}(\mathbf{k}, t) e^{i\mathbf{k} \cdot \mathbf{x}} d^3k / (2\pi)^3. \quad (130)$$

Analogous calculations are performed for $\dot{h}_{+/\times}(\mathbf{x}, t)$, which are used to compute the GW energy via

$$\mathcal{E}_{\text{GW}}(t) = \frac{c^2}{32\pi G} \left(\langle \dot{h}_+^2 \rangle + \langle \dot{h}_\times^2 \rangle \right), \quad (131)$$

where angle brackets denote volume averages.

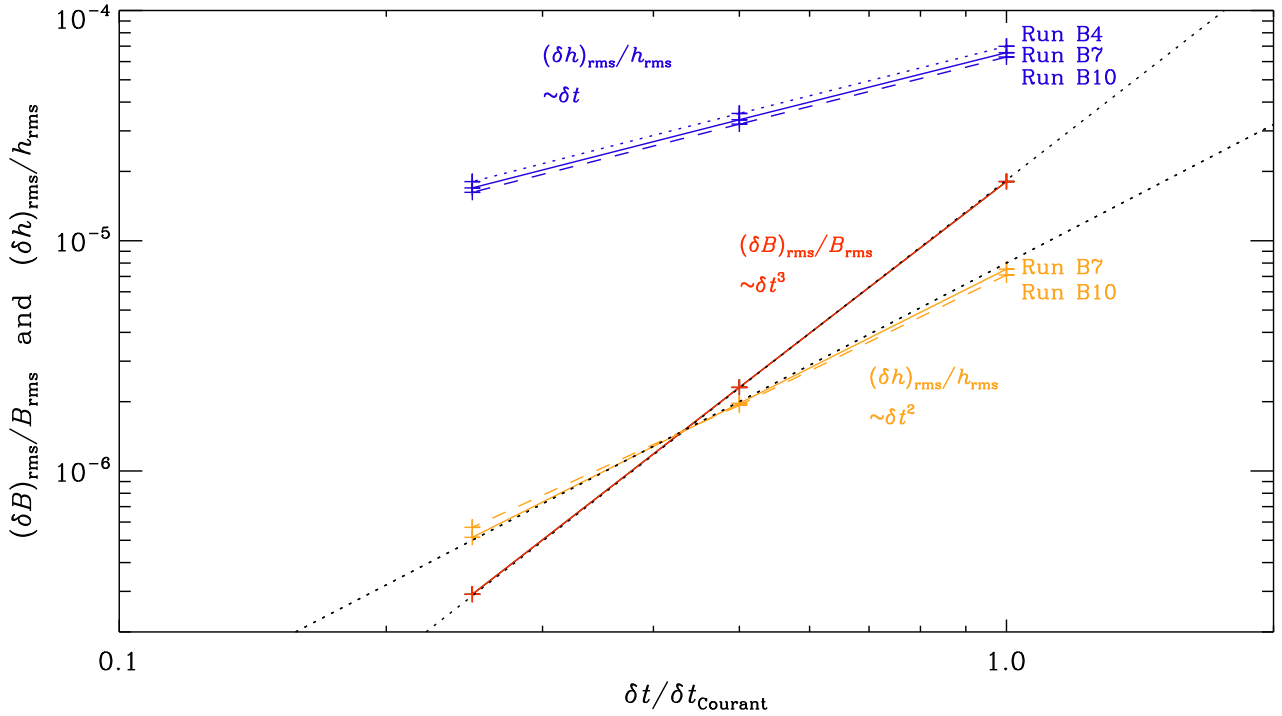


Figure 9: Scalings of the relative error in the magnetic field, $(\delta B)_{\text{rms}}/B_{\text{rms}}$, and the gravitational strain $(\delta h)_{\text{rms}}/h_{\text{rms}}$ for GWs generated by the chiral magnetic effect, which leads to an exponentially increasing magnetic field. Low resolution (32^3) versions of the Runs B4, B7, and B10 of [11].

Analogously to kinetic and magnetic energy and helicity spectra, it is convenient to compute the GW energy and polarization spectra integrated over concentric shells of surface $\int_{4\pi} k^2 d\Omega_{\mathbf{k}}$ in $\mathbf{k}$ space, defined by

$$S_h(k) = \int_{4\pi} \left(|\dot{h}_+|^2 + |\dot{h}_\times|^2 \right) k^2 d\Omega_{\mathbf{k}}, \quad (132)$$

$$A_h(k) = \int_{4\pi} 2 \text{Im} \left(\dot{h}_+ \dot{h}_\times^* \right) k^2 d\Omega_{\mathbf{k}}, \quad (133)$$

and normalized such that $\int_0^\infty S_h(k) dk = \langle \dot{h}_+^2 \rangle + \langle \dot{h}_\times^2 \rangle$ is proportional to the energy density and $\int_0^\infty A_h(k) dk$ is proportional to the polarized energy density. The $A_h(k)$ spectra are not to be confused with the magnetic vector potential $A(\mathbf{x}, t)$. The corresponding GW energy spectra are noted by

$$E_{\text{GW}}(k) = (c^2/32\pi G) S_h(k), \quad (134)$$

$$H_{\text{GW}}(k) = (c^2/32\pi G) A_h(k). \quad (135)$$

We also define spectra for the metric tensor perturbation,

$$S_h(k) = \int_{4\pi} \left(|\tilde{h}_+|^2 + |\tilde{h}_\times|^2 \right) k^2 d\Omega_{\mathbf{k}}, \quad (136)$$

$$A_h(k) = \int_{4\pi} 2 \operatorname{Im} \left(\tilde{h}_+ \tilde{h}_\times^* \right) k^2 d\Omega_{\mathbf{k}}, \quad (137)$$

which are normalized such that $\int_0^\infty S_h(k) dk = h_{\text{rms}}^2$ is the mean squared metric tensor perturbation.

By assuming the GW source to be constant between two time steps, one arrives at an accuracy of the GW solution that scales linearly with the time step, δt , while the magnetic field, related to the magnetic stress, scales cubically with δt . Since $\tilde{T}_{+/\times}$ are being stored in the f -array, it is easy to extract for each wavevector the increment of the stress, $\delta\tilde{T}_{+/\times}$, and to use it in an improved update of the GW field through

$$\tilde{h}_{+/\times} = \dots + \delta\tilde{T}_{+/\times} (1 - \sin \omega \delta t) / \omega^2 \quad (138)$$

$$\dot{\tilde{h}}_{+/\times} = \dots + \delta\tilde{T}_{+/\times} (1 - \cos \omega \delta t) / \omega^2 \delta t \quad (139)$$

This more accurate solver is invoked by setting `itorder_GW=2`, which now results in a quadratic scaling of the error of the GW field; see Fig. 9.

7 Troubleshooting / Frequently Asked Questions

7.1 Download and setup

7.1.1 *Download forbidden*

A: Both GitHub and SourceForge are banned from countries on the United States Office of Foreign Assets Control sanction list, including Cuba, Iran, Libya, North Korea, Sudan and Syria; see <http://de.wikipedia.org/wiki/GitHub> and <http://en.wikipedia.org/wiki/SourceForge>. As a remedy, you might download a tarball from <http://pencil-code.nordita.org/>; see also Section 2.

7.1.2 *When sourcing the ‘sourceme.sh’/‘sourceme.csh’ file or running pc_setupsrc, I get error messages from the shell, like ‘if: Expression Syntax.’ or ‘set: Variable name must begin with a letter.’*

A: This sounds like a buggy shell setup, either by yourself or your system administrator — or a shell that is even more idiosyncratic than the ones we have been working with.

To better diagnose the problem, collect the following information before filing a bug report to us:

1. `uname -a`
2. `/bin/csh -v`
3. `echo $version`
4. `echo $SHELL`
5. `ps -p $$`
6. If you have problems while sourcing the ‘sourceme’ script,
 - (a) unset the PENCIL_HOME variable:

for csh and similar: `unsetenv PENCIL_HOME`

for bash and similar: `unexport PENCIL_HOME; unset PENCIL_HOME`
 - (b) switch your shell in verbose mode,

for csh and similar: `set verbose; set echo`

for bash and similar: `set -v; set -x`

then source again.
7. If you have problems with pc_setupsrc, run it with csh in verbose mode:


```
/bin/csh -v -x $PENCIL_HOME/bin/pc_setupsrc
```

7.2 Compilation

7.2.1 *Error: ‘relocation truncated to fit’*

If you get errors while compiling and linking that are similar to:

```
density.f90:(.text+0x5e0): relocation truncated to fit: R_X86_64_PC32
against symbol 'cdata_mp_m_' defined in COMMON section in cdata.o
density.f90:(.text+0x644): additional relocation overflows omitted from the
output
make[2]: *** [start.x] Error 1
```

A: Your setup is probably too large to fit a ‘normal’ memory model. Please choose a ‘medium’ or ‘large’ memory model by adding one of these compiler options to your configuration: ‘-mmodel=medium’ or ‘-mmodel=large’. See Sect. 5.1 for configuration details. Alternatively, if you use *pc_build*, you may simply add the respective extension:

```
pc_build -f GNU-GCC_MPI,GNU-GCC_medium
```

or for the *Intel* compiler and a ‘large’ memory model you would use:

```
pc_build -f Intel_MPI,Intel_large
```

7.2.2 Linker can't find the syscalls functions:

```
ld: 0711-317 ERROR: Undefined symbol: .is_nan_c
ld: 0711-317 ERROR: Undefined symbol: .sizeof_real_c
ld: 0711-317 ERROR: Undefined symbol: .system_c
ld: 0711-317 ERROR: Undefined symbol: .get_env_var_c
ld: 0711-317 ERROR: Undefined symbol: .get_pid_c
ld: 0711-317 ERROR: Undefined symbol: .file_size_c
```

A: The Pencil Code needs a working combination of a Fortran- and a C-compiler. If this is not correctly set up, usually the linker won't find the functions inside the *syscalls* module. If that happens, either the combination of C- and Fortran-compiler is inappropriate (e.g., *ifort* needs *icc*), or the compiler needs additional flags, like *g95* might need the option ‘-fno-second-underscore’ and *xlf* might need the option ‘-qextname’. Please refer to Sect. 5.2, Table 1.

7.2.3 Make gives the following error now:

```
PGF90-S-0017-Unable to open include file: chemistry.h (nochemistry.f90: 43)
  0 inform,  0 warnings,  1 severes, 0 fatal for chemistry
```

Line 43 of the nochemistry routine, only has ‘contains’.

A: This is because somebody added a new module (together with a corresponding *nomodule.f90* and a *module.h* file (chemistry in this case). These files didn't exist before, so you need to say:

```
pc_setupsr
```

If this does not help, say first `make clean` and then `pc_setupsr`.

7.2.4 How do I compile the PENCIL CODE with the Intel (ifc) compiler under Linux?

A: The PENCIL CODE should compile successfully with *ifc* 6.x, *ifc* 7.0, sufficiently recent versions of *ifc* 7.1 (you should get the latest version; if yours is too old, you will typically get an ‘internal compiler error’ during compilation of ‘src/hydro.f90’), as well as with recent versions of *ifort* 8.1 (8.0 may also work).

You can find the *ifort* compiler at <ftp://download.intel.com/software/products/compilers/download>

On many current (as of November 2003) Linux systems, there is a mismatch between the *glibc* versions used by the compiler and the linker. To work around this, use the following flag for compiling

```
FC=ifc -i_dynamic
```

and set the environment variable

```
LD_ASSUME_KERNEL=2.4.1; export LD_ASSUME_KERNEL
```

or

```
setenv LD_ASSUME_KERNEL 2.4.1
```

This has solved the problems e.g., on a system with *glibc-2.3.2* and kernel 2.4.22.

Thanks to Leonardo J. Milano (<http://udel.edu/~lmilano/>) for part of this info.

7.2.5 I keep getting segmentation faults with 'start.x' when compiling with ifort 8.0

A: There was/is a number of issues with *ifort* 8.0. Make sure you have the latest patches applied to the compiler. A number of things to consider or try are:

1. Compile with the the '-static -nothreads' flags.
2. Set your stacksize to a large value (but a far too large value may be problematic, too), e.g.

```
limit stacksize 256m
ulimit -s 256000
```

3. Set the environment variable KMP_STACKSIZE to a large value (like 100M)

See also <http://softwareforums.intel.com/ids/board/message?board.id=11&message.id=1375>

7.2.6 When compiling with MPI on a Linux system, the linker complains:

```
mpicomm.o: In function 'mpicomm_mpicomm_init_':
mpicomm.o(.text+0x36): undefined reference to 'mpi_init_'
mpicomm.o(.text+0x55): undefined reference to 'mpi_comm_size_'
mpicomm.o(.text+0x6f): undefined reference to 'mpi_comm_rank_'
[...]
```

A: This is the infamous *underscore problem*. Your *MPI* libraries have been compiled with *G77* without the option '-fno-second-underscore', which makes the *MPI* symbol names incompatible with other Fortran compilers.

As a workaround, use

```
MPICOMM = mpicomm_
```

in 'Makefile.local'. Or, even better, you can set this globally (for the given computer) by inserting that line into the file '~/.adapt-mkfile.inc' (see `perldoc adapt-mkfile` for more details).

7.2.7 *Compilation stops with the cryptic error message:*

```
f95 -O3 -u -c .f90.f90
Error : Could not open sourcefile .f90.f90
compilation aborted for .f90.f90 (code 1)
make[1]: *** [.f90.o] Error 1
```

What is the problem?

A: There are two possibilities:

1. One of the variables for *make* has not been set, so *make* expands it to the empty string. Most probably you forgot to specify a module in 'src/Makefile.local'. One possibility is that you have upgraded from an older version of the code that did not have some of the modules the new version has.

Compare your 'src/Makefile.local' to one of the examples that work.

2. One of the variables for *make* has a space appended to it, e.g., if you use the line

```
MPICOMM = mpicomm_
```

(see § 7.2.6) with a trailing blank, you will encounter this error message. Remove the blank. This problem can also occur if you added a new module (and have an empty space after the module name in 'src/Makefile.src', i.e. *CHIRAL=nochiral_*), in which case the compiler will talk about "circular dependence" for the file 'nochiral'.

7.2.8 *The code doesn't compile,*

... there is a problem with *mvar*:

```
make start.x run.x
f95 -O3 -u -c cdata.f90
Error: cdata.f90, line 71: Implicit type for MVAR
      detected at MVAR@)
[f95 terminated - errors found by pass 1]
make[1]: *** [cdata.o] Error 2
```

A: Check and make sure that 'mkcparam' (directory '\$PENCIL_HOME/bin') is in your path. If this doesn't help, there may be an *empty* 'cparam.inc' file in your 'src' directory. Remove 'cparam.inc' and try again (Note that 'cparam.inc' is automatically generated from the 'Makefile').

7.2.9 *Some samples don't even compile,*

as you can see on the web, <http://www.nordita.org/software/pencil-code/tests.html>.

samples/helical-MHDTurb:

```
Compiling..          not ok:
make start.x run.x read_videofiles.x
make[1]: Entering directory '/home/dobler/f90/pencil-code/samples/helical-MHDTurb/src'
/usr/lib/lam/bin/mpif95 -O3 -c initcond.f90
/usr/lib/lam/bin/mpif95 -O3 -c density.f90
      use Gravity, only: gravz, nu_epicycle
      ^
```

```

Error 208 at (467:density.f90) : No such entity in the module
Error 355 : In procedure INIT_LNRHO variable NU_EPICYCLE has not been given a type
Error 355 : In procedure POLYTROPIC_LNRHO_DISC variable NU_EPICYCLE has not been given a
3 Errors
compilation aborted for density.f90 (code 1)
make[1]: *** [density.o] Error 1
make[1]: Leaving directory '/home/dobler/f90/pencil-code/samples/helical-MHdturb/src'
make: *** [code] Error 2

```

A: Somebody may have checked in something without having run auto-test beforehand. The problem here is that something has been added in one module, but not in the corresponding no-module. You can of course check with *svn* who it was...

7.2.10 Internal compiler error with Compaq/Dec F90

The Dec Fortran optimizer has occasional problems with 'nompicomm.f90':

```

make start.x run.x read_videofiles.x
f90 -fast -O3 -tune ev6 -arch ev6 -c cparam.f90
[...]
f90 -fast -O3 -tune ev6 -arch ev6 -c nompicomm.f90
otal vm 2755568      ota1 vm 2765296      ota1 vm 2775024
otal vm 2784752      ota1...
Assertion failure: Compiler internal error - please submit problem r...
  GEM ASSERTION, Compiler internal error - please submit problem report
Fatal error in: /usr/lib/cmplrs/fort90_540/decfort90 Terminated
*** Exit 3
Stop.
*** Exit 1
Stop.

```

A: The occurrence of this problem depends upon the grid size; and the problem never seems to occur with 'mpicomm.f90', except when *ncpus*=1. The problem can be avoided by switching off the loop transformation optimization (part of the '-O3' optimization), via:

```
#OPTFLAGS=-fast -O3 -notransform_loops
```

This is currently the default compiler setting in 'Makefile', although it has a measurable performance impact (some 8% slowdown).

7.2.11 Assertion failure under SunOS

Under SunOS, I get an error message like

```

user@sun> f90 -c param_io.f90
Assertion failed: at_handle_table[at_idx].tag == VAR_TAG,
                file ../srcfw/FWcvrt.c, line 4018
f90: Fatal error in f90comp: Abort

```

A: This is a compiler bug that we find at least with Sun's WorkShop Compiler version '5.0 00/05/17 FORTRAN 90 2.0 Patch 107356-05'. Upgrade the compiler version (and possibly also the operating system): we find that the code compiles and works with version 'Sun WorkShop 6 update 2 Fortran 95 6.2 Patch 111690-05 2002/01/17' under SunOS version '5.8 Generic_108528-11'.

7.2.12 *After some dirty tricks I got pencil code to compile with MPI, ...*

> Before that i installed lam-7.1.4 from source.

Goodness gracious me, you shouldn't have to compile your own MPI library.

A: Then don't use the old LAM-MPI. It is long superseded by open-mpi now. Open-mpi doesn't need a daemon to be running. I am using the version that ships with Ubuntu (e.g., 9.04):

```
frenesi:~> aptitude -w 210 search openmpi | grep '^i'
```

```
i  libopenmpi-dev - high performance message passing library -- header files
i A libopenmpi1   - high performance message passing library -- shared library
i  openmpi-bin    - high performance message passing library -- binaries
i A openmpi-common - high performance message passing library -- common files
i  openmpi-doc    - high performance message passing library -- man pages
```

Install that and keep your configuration (Makefile.src and getconf.csh) close to that for 'frenesi' or 'norlx50'. That should work.

7.2.13 *Error: Symbol 'mpi_comm_world' at (1) has no IMPLICIT type*

I installed the pencil code on Ubuntu system and tested "run.csh" in .../samples/conv-slab. Here the code worked pretty well. Nevertheless, running (auto-test), I found there are some errors.

The messages are,

```
Error: Symbol 'mpi_comm_world' at (1) has no IMPLICIT type
Fatal Error: Error count reached limit of 25.
make[2]: *** [mpicomm_double.o] Error 1
make[2]: Leaving directory
'/home/pkiwan/Desktop/pencil-code/samples/2d-tests/selfgravitating-shearwave/src'
make[1]: *** [code] Error 2
make[1]: Leaving directory
'/home/pkiwan/Desktop/pencil-code/samples/2d-tests/selfgravitating-shearwave/src'
make: *** [default] Error 2
```

Finally, ### auto-test failed ###

Will it be OK? Or, how can I fix this?

A: Thanks for letting me know about the status, and congratulations on your progress! Those tests that fail are those that use MPI. If your machine is a dual or multi core machine, you could run faster by running under MPI. But this is probably not crucial for you at this point. (I just noticed that there is a ToDo listed in the auto-test command to implement the option not to run the MPI tests, but this hasn't been done yet. So I guess you can start with the science next.

7.2.14 *Error: Can't open included file 'mpif.h'*

It always worked, but now, after some systems upgrade, I get

```
gfortran -O3 -o mpicomm.o -c mpicomm.f90
Error: Can't open included file 'mpif.h'
```

When I say `locate mpif.h` I only get things like

```
/scratch/ntest/1.2.7p1-intel/include/mpif.h
```

But since I use `FC=mpif90` I thought I don't need to worry.

A: Since you use `FC=mpif90` there must definitely be something wrong with their setup. Try `mpif90 -showme` or `mpif90 -show`; the '-I' option should say where it looks for 'mpif.h'. If those directories don't exist, it's no wonder that it doesn't work, and it is time to complain.

7.2.15 *Compilation fails on MacOS Sonoma or Monterey*

I am getting an error which hasn't been resolved yet. It has something to do with my Mac setup and no one has been able to figure out how to fix it. Maybe you've seen this before (at the `pc_build` stage):

```
'make -j FFLAGS_DOUBLE=-fdefault-real-8 -fdefault-double-8
CFLAGS_DOUBLE=-DDOUBLE_PRECISION LD_MPI= CFLAGS_FFTW3= FFLAGS_FFTW3=
LD_FFTW3= CFLAGS_FFTW2= FFLAGS_FFTW2= LD_FFTW2= FC=gfortran F77=$(FC)
FFLAGS=-O LDFLAGS_HELPER=-dynamic OMPFFLAGS=-fopenmp OMPLFLAGS=-lgomp
PPFLAGS=-cpp FSTD_95=-std=f95 FSTD_2003=-std=f2003 CC=gcc
CFLAGS=-DFUNDERSC=1 default_to_be' failed: <Inappropriate ioctl for device>
```

A: You have to use `gcc-14`. Default (plain `gcc`) does not work; it is wrongly set up.

7.2.16 *Compilation fails on Tanmay's MacOS*

In my case the `gcc` compiler was renamed `gcc-14`. Once this was fixed, all was good. Could it be that when Mac updates its OS it changes the name of the compiler? I don't know the backend of the Mac so well.

7.2.17 *Missing `ld_classic` on MacOS*

While running `pc_build`, I get an error message saying that `-ld_classic` is not found.

A: Note that `ld_classic` is *not* specifying a library called `libd_classic`, but is rather an option passed to Apple's `ld` binary. type `ld` should say `/usr/bin/ld`. This error has been noticed on systems where people tried to install `gfortran` using `anaconda`, which pulls in a wrongly configured `ld` binary. Run `conda remove ld64` and use another method, like `homebrew`, to install `gfortran`.

7.2.18 *Further MacOS tips*

For further MacOS tips, see also the "Instructions for MacOS installation" on <http://pencil-code.nordita.org/doc.php>.

7.3 Pencil check

7.3.1 *The pencil check complains for no reason.*

A: The pencil check only complains for a reason.

7.3.2 The pencil check reports MISSING PENCILS and quits

A: This could point to a serious problem in the code. Check where the missing pencil is used in the code. Request the right pencils, likely based on input parameters, by adapting one or more of the `pencil_criteria_MODULE` subroutines.

7.3.3 The pencil check reports unnecessary pencils

The pencil check reports possible overcalculation... pencil rho (43) is requested, but does not appear to be required!

A: Such warnings show that your simulation is possibly running too slowly because it is calculating pencils that are not actually needed. Check in the code where the unnecessary pencils are used and adapt one or more of the `pencil_criteria_MODULE` subroutines to request pencils only when they are actually needed.

7.3.4 The pencil check reports that most or all pencils are missing

A: This is typically a thing that can happen when testing new code development for the first time. It is usually an indication that the reference `df` changes every time you call `pde`. Check whether any newly implemented subroutines or functionality has a “memory”, i.e. if calling the subroutine twice with the same `f` gives different output `df`.

7.3.5 Running the pencil check triggers mathematical errors in the code

A: The pencil check puts random numbers in `f` before checking the dependence of `df` on the chosen set of pencils. Sometimes these random numbers are inconsistent with the physics and cause errors. In that case you can set `lrandom_f_pencil_check=F` in `&run_pars` in ‘`run.in`’. The initial condition may contain many idealized states (zeros or ones) which then do not trigger pencil check errors when `lrandom_f_pencil_check=F`, even if pencils are missing. But it does prevent mathematical inconsistencies.

7.3.6 The pencil check still complains

A: Then you need to look into the how the code and the pencil check operate. Reduce the problem in size and dimensions to find the smallest problem that makes the pencil check fail (e.g., `1x1x8` grid points). At the line of ‘`pencil_check.f90`’ when a difference is found between `df_ref` and `df`, add some debug lines telling you which variable is inconsistent and in what place. Often you will be surprised that the pencil check has correctly found a problem in the simulation.

7.3.7 The pencil check is annoying so I turned it off

A: Then you are taking a major risk. If one or more pencils are not calculated properly, then the results will be wrong.

7.4 Running

7.4.1 Why does ‘`start.x`’ / ‘`start.csh`’ write data with periodic boundary conditions?

A: Because you are setting the boundary conditions in ‘`run.in`’, not in ‘`start.in`’; see Sect. 5.17.1. There is nothing wrong with the initial data — the ghost-zone values will

be re-calculated during the very first time step.

7.4.2 *csch problem?*

Q: On some rare occasions we have problems with `csch` not being supported on other machines. (We hope to fix this by contacting the responsible person, but may not be that trivial today!) Oliver says this is a well known bug of some years ago, etc. But maybe in the long run it would be good to avoid `csch`.

A: These occasions will become increasingly frequent, and eventually for some architectures, there may not even be a `csch` variant that can be installed.

We never pushed people to use `pc_run` and friends (and to report corresponding bugs and get them fixed), but if we don't spend a bit of effort (or annoy users) now, we create a future emergency, where someone needs to run on some machine, but there is no `csch` and he or she just gets stuck.

We don't have that many `csch` files, and for years now it should be possible to compile run without `csch` (using `bin/pc_run`) — except that people still fall back on the old way of doing things. This is both cause and consequence of the 'new' way not being tested that much, at least for the corner cases like 'RERUN', 'NEWDIR', 'SCRATCH_DIR'.

7.4.3 '*run.csch*' doesn't work:

```
Invalid character ''' in NAMELIST input
Program terminated by fatal I/O error
Abort
```

A: The string array for the boundary condition, e.g., `bcx` or `bcz` is too long. Make sure it has exactly as many elements as `nvar` is big.

7.4.4 *Code crashes after restarting*

```
> > removing mu_r from the namelist just 'like that' makes the code
> > backwards incompatible.
>
> That means that we can never get rid of a parameter in start.in once we
> have introduced it, right?
```

A: In the current implementation, without a corresponding cleaning procedure, unfortunately yes.

Of course, this does not affect users' private changes outside the central svn tree.

7.4.5 *auto-test gone mad...?*

Q: Have you ever seen this before:

```
giga01:/home/pg/n7026413/cvs-src/pencil-code/samples/conv-slab> auto-test
.

/home/pg/n7026413/cvs-src/pencil-code/samples/conv-slab:
  Compiling..          ok
    No data directory; generating data -> /var/tmp/pencil-tmp-25318
  Starting..          ok
```

```
Running..                ok
Validating results..Malformed UTF-8 character (unexpected continuation
byte 0x80, with no preceding start byte) in split at
/home/pg/n7026413/cvs-src/pencil-code/bin/auto-test line 263.
Malformed UTF-8 character (unexpected continuation byte 0x80, with no
preceding start byte) in split at
/home/pg/n7026413/cvs-src/pencil-code/bin/auto-test line 263.
```

A: You are running on a RedHat 8 or 9 system, right?

Set `LANG=POSIX` in your shell's startup script and life will be much better.

7.4.6 Can I restart with a different number of cpus?

Q: I am running a simulation of nonhelical turbulence on the cluster using MPI. Suppose if I am running a 128^3 simulation on 32 cpus/cores i.e.

```
integer, parameter :: ncpus=32,nprocy=2,nprocz=ncpus/nprocy,nprocx=1
integer, parameter :: nxgrid=128,nygrid=nxgrid,nzgrid=nxgrid
```

And I stop the run after a bit. Is there a way to resume this run with different number of cpus like this :

```
integer, parameter :: ncpus=16,nprocy=2,nprocz=ncpus/nprocy,nprocx=1
integer, parameter :: nxgrid=128,nygrid=nxgrid,nzgrid=nxgrid
```

I understand it has to be so in a new directory but making sure that the run starts from where I left it off in the previous directory.

A: The answer is no, if you use the standard distributed io. There is also parallel io, but I never used it. That would write the data in a single file, and then you could use the data for restart in another processor layout.

7.4.7 Can I restart with a different number of cpus?

Q: Is it right that once the simulation is resumed, pencil-code takes the last data from `var.dat` (which is the current snapshot of the fields)? If that is true, then, is it not possible to give that as the initial condition for the run in the second directory (with changed "ncpus")? Is there a mechanism already in place for that?

A: Yes, the code restarts from the last `var.dat`. It is written after a successful completion of the run, but it crashes or you hit a time-out, there will be a `var.dat` that is overwritten every `isave` timesteps. If the system stops during writing, some `var.dat` files may be corrupt or have the wrong time. In that case you could restart from a good VAR file, if you have one, using, e.g.,

```
restart-new-dir-VAR . 46
```

where 46 is the number of your VAR file, i.e., VAR46 in this case. To restart in another directory, you say, from the old run directory,

```
restart-new-dir ../another_directory
```

Hope this helps. Look into `pencil-code/bin/restart-new-dir` to see what it is doing.

7.4.8 *fft_xyz_parallel_3D: nygrid needs to be an integer multiple...*

Q: I just got an:

```
fft_xyz_parallel_3D: nygrid needs to be an integer multiple of nproc*nprocz
```

In my case, $nygrid=2048$, $nproc=32$, and $nprocz=128$, so $nproc*nprocz=4096$. In other words, 2048 needs to be a multiple of 4096. But isn't this the case then?

A: No, because $2048 = 0.5 * 4096$ and 0.5 is not an integer. Maybe try either setting $nprocz=64$ or $nproc=64$. You could compensate the change of $npcus$ with the x -direction. For 2048^3 simulations, $nproc=32$ and $nprocz=64$ would be good. A list of good meshes is given in Table 4.

7.4.9 *Unit-agnostic calculations?*

Q: The manual speaks about unit-agnostic calculations, stating that one may choose to interpret the results in any (consistent) units, depending on the problem that is solved at hand. So, for example, if I chose to run the '2d-tests/battery_term' simulation for an arbitrary number of time-steps and then choose to examine the diagnostics, am I correct in assuming the following:

- 1) [Brms] = Gauss (as output by unit_magnetic, before the run begins)
- 2) [t] = s (since the default unit system is left as CGS)
- 3) [urms] = cm/s (again, as output by unit_velocity, before the run begins)
- 4) and etc. for the units of the other diagnostics

A: Detailed correspondence on this item can be found on: <https://groups.google.com/forum/?fromgroups#!topic/pencil-code-discuss/zek-uYNbgXI>. There is also working material on unit systems under <http://www.nordita.org/~brandenb/teach/PencilCode/MixedTopics.html> with a link to http://www.nordita.org/~brandenb/teach/PencilCode/material/AlfvenWave_SIunits/. Below is a pedagogical response from Wlad Lyra:

In the sample battery-term, the sound speed $cs0=1$ sets the unit of velocity. Together with the unit of length, that sets your unit of time. The unit of magnetic field follows from the unit of velocity, density, and your choice of magnetic permittivity, according to the definition of the Alfven velocity.

If you are assuming cgs, you are saying that your sound speed $cs0=1$ actually means $[U]=1$ cm/s. Your unit of length is equivalently 1 cm, and therefore the unit of time is $[t] = [L]/[U]=1$ s. The unit of density is $[\rho] = 1$ g/cm³. Since in cgs $v_A=B/\sqrt{4\pi * \rho}$, your unit of magnetic field is $[B] = [U] * \sqrt{[\rho] * 4\pi} \sim 3.5 \sqrt{\text{g/cm}} / \text{s} = 3.5$ Gauss.

If instead you are assuming SI, you have $cs0=1$ assuming that means $[U]=1$ m/s and $\rho0=1$ assuming that to mean $[\rho]=1$ kg/m³. Using $[L]=1$ m, you have still $[t]=1$ s, but now what appears as $B=1$ in your output is actually $[B] = [U] * \sqrt{(\mu * [\rho])} = 1 \text{ m/s} * \sqrt{4\pi * 1e-7 \text{ N}\cdot\text{A}^{-2} * 1 \text{ kg/m}^3} \sim 0.0011210 \text{ kg}/(\text{s}^2\cdot\text{A}) \sim 11$ Gauss.

You can make it more interesting and use units relevant to the problem. Say you are at the photosphere of the Sun. You may want to

use dimensionless $cs_0=1$ meaning a sound speed of 10 km/s. Your appropriate length can be a megameter. Now your time unit is $[t]=[L]/[U] = 1e3 \text{ km} / 10 \text{ km/s} = 10^2 \text{ s}$, i.e., roughly 1.5 minute. For density, assume $\rho=2 \times 10^{-4} \text{ kg/m}^3$, typical of the solar photosphere. Your unit of magnetic field is therefore $[B] = [U] * \sqrt{[\rho] * 4\pi} = 1e6 \text{ cm/s} * \sqrt{4\pi * 2e-7 \text{ g/cm}^3} \sim 1585.33 \text{ Gauss}$.

Notice that for $\mu_0=1$ and $\rho_0=1$ you simply have $v_A=B$. Then you can conveniently set the field strength by your choice of plasma beta ($= 2*cs^2/v_A^2$). There's a reason why we like dimensionless quantities!

7.5 Visualization

7.5.1 '*start.pro*' doesn't work:

```
Reading grid.dat..
Reading param.nml..
\% Expression must be a structure in this context: PAR.
\% Execution halted at:  \ $MAIN$          104
/home/brandenb/pencil-code/runs/forced/hell1/../../idl/start.pro
```

A: You don't have the subdirectory 'data' in your IDL variable *!path*. Make sure you source 'sourceme.csh'/'sourceme.sh' or set a sufficient IDL path otherwise.

7.5.2 '*start.pro*' doesn't work:

Isn't there some clever (or even trivial) way that one can avoid the annoying error messages that one gets, when running e.g., ".r rall" after a new variable has been introduced in "idl/varcontent.pro"? Ever so often there's a new variable that can't be found in my param2.nml – this time it was IECR, IGG, and ILNTT that I had to circumvent...

A: The simplest solution is to invoke 'NOERASE', i.e. say

```
touch NOERASE
start.csh
```

or, alternatively, start_run.csh. What it does is that it reruns src/start.x with a new version of the code; this then produces all the necessary auxiliary files, but it doesn't overwrite or erase the 'var.dat' and other 'VAR' and 'slice' files.

7.5.3 Something about tag name undefined:

Q: In one of my older run directories I can't read the data with idl anymore. What should I do? It says something like

```
Reading param.nml..
% Tag name LEQUIDIST is undefined for structure <Anonymous>.
% Execution halted at:  $MAIN$          182
/people/disk2/brandenb/pencil-code/idl/start.pro
```

A: Go into 'data/param.nml' and add , LEQUIDIST=T anywhere in the file (but before the last slash).

7.5.4 *Something INC in start.pro*

Q: start doesn't even work:

```
% Compiled module: $MAIN$.
nname=      11
Reading grid.dat..
Reading param.nml..
Can't locate Namelist.pm in INC (INC contains: /etc/perl /usr/local/lib/perl/5.8.4 /usr/l
BEGIN failed--compilation aborted at /home/brandenb/pencil-code/bin/nl2idl line 49.
```

A: Go into '\$PENCIL_HOME' and say `svn up sourceme.csh` and/or `svn up sourceme.sh`. (They were just out of date.)

7.5.5 *nl2idl problem when reading param2.nml*

Q: Does anybody encounter a backward problem with nl2idl? The file `param*.nml` files are checked in under 'pencil-code/axel/couette/SStrat128a_mu0.20_g2' and the problem is below.

```
at /people/disk2/brandenb/pencil-code/bin/nl2idl line 120
HCOND0= 0.0,HCOND1= 1.000000,HCOND2= 1.000000,WIDTHSS= 1.192093E-06,MPOLY0=
^----- HERE
at /people/disk2/brandenb/pencil-code/bin/nl2idl line 120
```

A: The problem is the stupid ifc compiler writing the following into the namelist file:

```
COOLING_PROFILE='gaussian                                ',COOLTYPE='Temp
',COOL= 0.0,CS2COOL= 0.0,RCOOL= 1.000000,WCOOL= 0.1000000,FBOT= 0.0,CHI_T= 0.0
```

If you add a comma after the closing quote:

```
COOLING_PROFILE='gaussian                                ',COOLTYPE='Temp
',COOL= 0.0,CS2COOL= 0.0,RCOOL= 1.000000,WCOOL= 0.1000000,FBOT= 0.0,CHI_T= 0.0
```

things will work.

Note that ifc cannot even itself read what it is writing here, so if this happened to occur in `param.nml`, the code would require manual intervention after each `start.csh`.

7.5.6 *Spurious dots in the time series file*

Q: Wolfgang, you explained it to me once, but I forget. How can one remove spurious dots after the timestep number if the time format overflows?

A: I don't know whether it exists anywhere, but it's easy. In Perl you'd say

```
perl -pe 's/^(\\s*[-0-9]+)\\.([-0-9eEdD])/\\1 \\2/g'
```

and in sed (but that's harder to read)

```
sed 's/^( *[-0-9]\\+\\.\\.([-0-9eEdD]\\)/\\1 \\2/g'
```

7.5.7 *Problems with pc_varcontent.pro*

Q:

```
% Subscript range values of the form low:high must be >= 0, < size, with low
<= high: VARCONTENT.
% Error occurred at: PC_VARCONTENT      391
/home/brandenb/pencil-code/idl/read/pc_varcontent.pro
%          PC_READ_VAR      318
/home/brandenb/pencil-code/idl/read/pc_read_var.pro
%          $MAIN$
```

A: Make sure you don't have any unused items in your `src/cparam.local` such as

```
! MAUX CONTRIBUTION 3
! COMMUNICATED AUXILIARIES 3
```

They would leave gaps in the counting of entries in your `data/index.pro` file.

7.6 Programming new slices

Q: I'm creating a new special module with a few new variables `f(:, :, :, inew1)`, `f(:, :, :, inew2)` etc. I'm wondering how to add new output of slices (i.e. video files) of these new variables (say `p%inew1`) and their combinations (say `p%inew1**2+p%inew2**2`)? I tried to look into other modules to find an example but got confused. If someone could clarify the calling tree it would be great!

A: Best you look into `hydro.f90`. For slices, which contain simply f-array variables, you do something like

```
case ('uu'); call assign_slices_vec(slices,f,iuu)
```

in `get_slices_hydro`. For slices, which contain derived quantities like your `p%inew1**2+p%inew2**2` you have to declare slice buffers as for `divu` in `hydro` (`divu_xy` etc.), you have to allocate them like

```
if (ivid_divu/=0) call alloc_slice_buffers(divu_xy,divu_xz, &
    divu_yz,divu_xy2,divu_xy3,divu_xy4,divu_xz2,divu_r)
```

to fill them in `calc_diagnostics_special` like

```
if (ivid_divu/=0) call store_slices(p%divu,divu_xy,divu_xz, &
    divu_yz,divu_xy2,divu_xy3,divu_xy4,divu_xz2,divu_r)
```

and finally to store them in `get_slices_special` like

```
call assign_slices_scal(slices,divu_xy,divu_xz,divu_yz, &
    divu_xy2,divu_xy3,divu_xy4,divu_xz2,divu_r)
```

7.7 General questions

7.7.1 "Installation" procedure

Why don't you use GNU *autoconf*/*automake* for installation of the PENCIL CODE?

A: What do you mean by "installation"? Unlike the applications that normally use *autoconf*, the *Pencil Code* is neither a binary executable, nor a library that you compile once and then dump somewhere in the system tree. *Autoconf* is the right tool for these applications, but not for numerical codes, where the typical compilation and usage pattern is very different:

You have different directories with different ‘Makefile.local’ settings, recompile after introducing that shiny new term in your equations, etc. Moreover, you want to sometimes switch to a different compiler (but just for that run directory) or another *MPI* implementation. Our adapt-mkfile approach gives you this flexibility in a reasonably convenient way, while doing the same thing with *autoconf* would be using that system against most of its design principles.

Besides, it would really get on my (WD’s) nerves if I had to wait two minutes for *autoconf* to finish before I can start compiling (or maybe 5–10 minutes if I worked on a NEC machine...).

Finally, if you have ever tried to figure out what a ‘configure’ script does, you will appreciate a comprehensible configuration system.

7.7.2 *Small numbers in the code*

What is actually the difference between *epsi*, *tini* and *tiny*?

A:

F90 has two functions `epsilon()` and `tiny()`, with

```
epsilon(x) = 1.1920929e-07
tiny(x)    = 1.1754944e-38
(and then there is huge(x) = 3.4028235e+38)
for a single-precision number x.
```

`epsilon(x)` is the smallest number that satisfies

```
1+epsilon(1.) /= 1 ,
while tiny(x) is the smallest number that can be represented without
precision loss.
```

In the code we have variants hereof,

```
epsi=5*epsilon(1.0)
tini=5*tiny(1.0)
huge1=0.2*huge(1.0)
that have added safety margins, so we don’t have to think about doing
things like 1/tini.
```

So in `sub.f90`,

```
-      evr = evr / spread(r_mn+epsi,2,3)
did (minimally) affect the result for r_mn=0(1), while the correct version
+      evr = evr / spread(r_mn+tini,2,3)
only avoids overflow.
```

7.7.3 *Why do we need a /lphysics/namelist in the first place?*

Wolfgang answered on 29 July 2010: “`cdata.f90` has the explanation”

```
! Constant 'parameters' cannot occur in namelists, so in order to get the
! now constant module logicals into the lphysics name list...
! We have some proxies that are used to initialize private local variables
! called lhydro etc, in the lphysics namelist!
```


So the situation is this: we want to write parameters like `ldensity` to `param.nml` so IDL (and potentially octave, python, etc.) can know whether density was on or not. To avoid confusion, we want them to have exactly their original names. But we cannot assemble the original `ldensity` etc. constants in a namelist, so we have to define a local `ldensity` variable. And to provide it with the value of the original `cdata.ldensity`, we need to transfer the value via `ldensity_var`. That's pretty scary, although it seems to work fine. I can track the code back to the big `eos_merger` commit, so it may originate from that branch. One obvious problem is that you have to add code in a number of places (the `ldensity` → `ldensity_var` assignment and the local definition of `ldensity`) to really get what you need. And when adding a new boolean of that sort to `'cdata.f90'`, you may not even have a clue that you need all the other voodoo.

There may be a cleaner solution involving generated code. Maybe something like

```
logical :: ldensity ! INCLUDE_IN_LPHYSICS
```

could later generate code (in some `param_io_extra.inc` file) that looks like this:

```
write(unit, *) 'ldensity = ', ldensity
```

i.e. we can manually write in namelist format. But maybe there are even simpler solutions?

7.7.4 Can I run the code on a Mac?

A: Macs work well for Linux stuff, except that the file structure is slightly different. Problems when following Linux installs can usually be traced to the `PATH`. For general reference, if you need to set an environment variable for an entire OS-X login session, google `environment.plist`. That won't be needed here.

For a Mac install, the following should work:

- a) Install Dev Tools (an optional install on the MacOS install disks). Unfortunately, last time I checked the svn version that comes with DevTools is obsolete. So:
- b) Install MacPorts (download from web). Note that MacPorts installs to a non-standard location, and will need to be sourced. The installation normally drops an appropriate line in `.profile`. If it does so, make sure that that line gets sourced. Otherwise

```
export PATH=/opt/local/bin:/opt/local/sbin:$PATH
export MANPATH=/opt/local/share/man:$MANPATH
```

- c) Install g95 (download from web). Make sure it is linked in `/bin`.
- d) execute `macports svn install`
- e) download the pencil-code and enjoy.

Note: the above way to get svn works. It takes a while however, so there are certainly faster ways out there. If you already have a non-obsolete svn version, use that instead.

7.7.5 Wrong user-id in commit emails

Why does it say

```
From: 'Philippe Bourdin' via pencil-code-commits
<pencil-code-commits@googlegroups.com>
```

when the committing author is not Philippe?

A: The associated email addresses in `account.pencil-code.org` should be the same as what is registered in github as your primary email address.

7.7.6 Pencil Code discussion forum

Do I just need to send an email somewhere to subscribe or what?

A: The answer is yes; just go to:

`http://groups.google.com/group/pencil-code-discuss`

7.7.7 The manual

It would be a good idea to add this useful information in the manual, no?

A: When you have added new stuff to the code, don't forget to mention this in the 'pencil-code/doc/manual.tex' file.

Again, the answer is yes; just go to:

```
cd pencil-code/doc/  
vi manual.tex  
svn ci -m "explanations about a new module in the code"
```

Part II

Programming the PENCIL CODE

All developers are supposed to have an up-to-date entry in the file ‘pencil-code/license/developers.txt’ so that they can be contacted in case a code change breaks an auto-test or other code functionality.

Several PENCIL CODE committers have done several hundred check-ins, but many of the currently 92 registered people on the repository have hardly done anything. To put a number to this, one can define an h index, which gives the number of users, who have done at least as many as that number of check-ins. This h index is currently 37, i.e., 37 users have done at least 37 check-ins; see Figure 10 from 2017, when the h index was only 32.

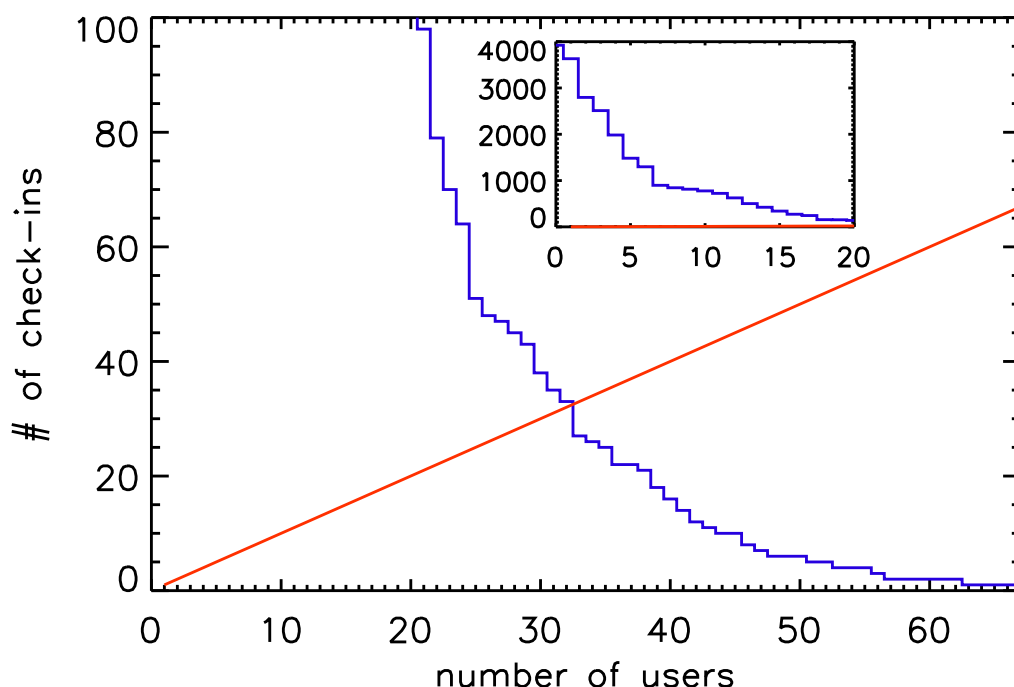


Figure 10: The h index of PENCIL CODE check-ins in 2017.

The PENCIL CODE has expanded approximately linearly in the number of lines of code and the number of subroutines (Fig. 11). The increase in the functionality of the code is documented by the rise in the number of sample problems (Fig. 12). It is important to monitor the performance of the code as well. Figure 13 shows that for most of the runs the run time has not changed much.

Before making changes to the code, it is important that you verify that you can run the `pc_auto-test` successfully. Don’t do this when you have already modified the code, because then you cannot be sure that any problems are caused by your changes, or because it wouldn’t have worked anyway. Also, keep in mind that the code is public, so your changes should make sense from a broader perspective and should not only be intended for yourself. Regarding more general aspects about coding standards see Sect. B.2.

In order to keep the development of the code going, it is important that the users are able to understand and modify (program!) the code. In this section we explain first how

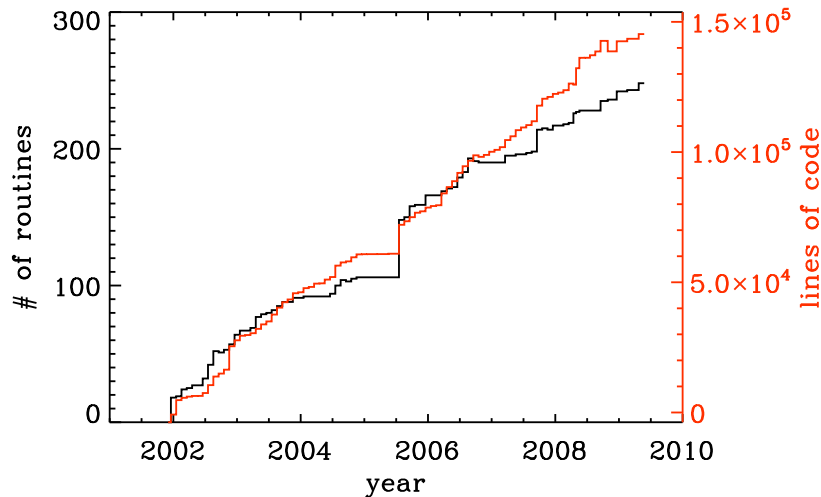


Figure 11: Number of lines of code and the number of subroutines since the end of 2001. The jump in the Summer of 2005 was the moment when the developments on the side branch (eos branch) were merged with the main trunk of the code. Note the approximately linear scaling with time.

to orient yourself in the code and to understand what is in it, and then to modify it according to your needs.

The Pencil Code check-ins occur regularly all the time. By the Pencil Code User Meeting 2010 we have arrived at a revision number of 15,000. In February 2017, the number of check-ins has risen to 26,804; see <https://github.com/pencil-code/pencil-code>. Major code changes are nowadays being discussed by the Pencil Code Steering Committee (<https://www.nordita.org/~brandenb/pencil-code/PCSC/>). The increase of the revision number with time is depicted in Figure 14. The number of Pencil Code developers increases too (Figure 15), but the really active ones are getting rare. This may indicate that new users can produce new science with the code as it is, but it may also indicate that it is getting harder to understand the code. How to understand the code will be discussed in the next section.

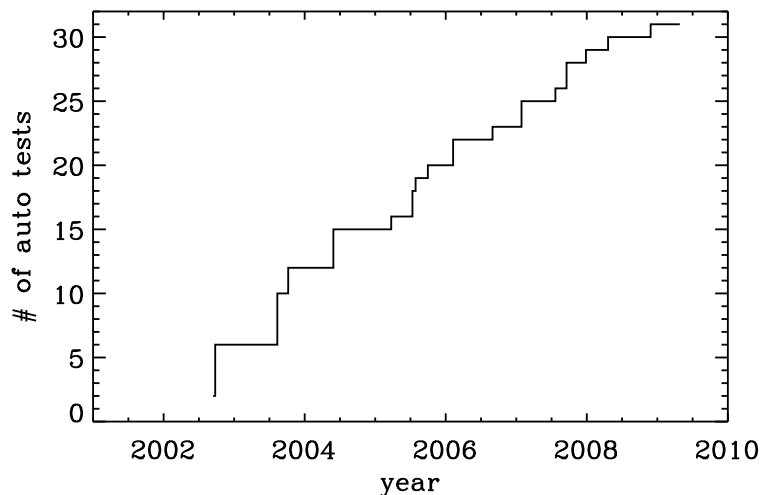


Figure 12: Number of tests in the sample directory that are used in the nightly auto tests. Note again the approximately linear scaling with time.

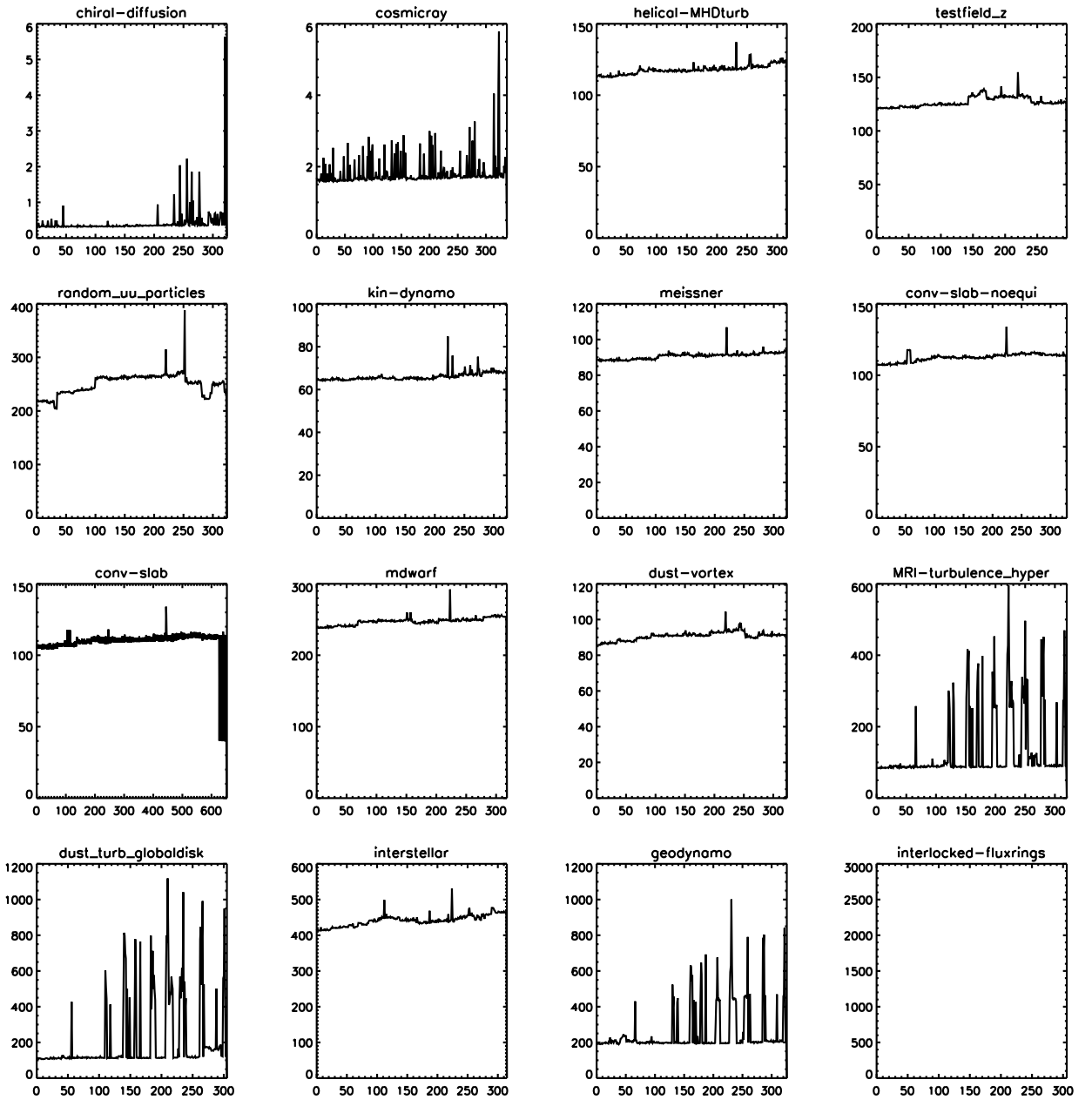


Figure 13: Run time of the daily auto-tests since August 17, 2008. For most of the runs the run time has not changed much. The occasional spikes are the results of additional load on the machine.

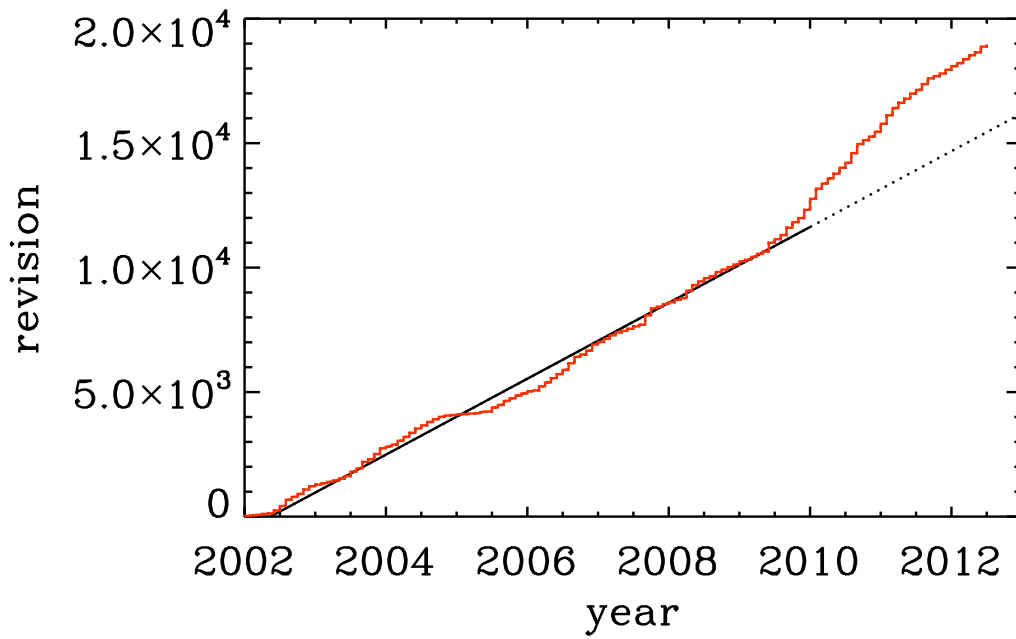


Figure 14: Number of check-ins since 2002. Note again the linear increase with time, although in the last part of the time series there is a notable speed-up.

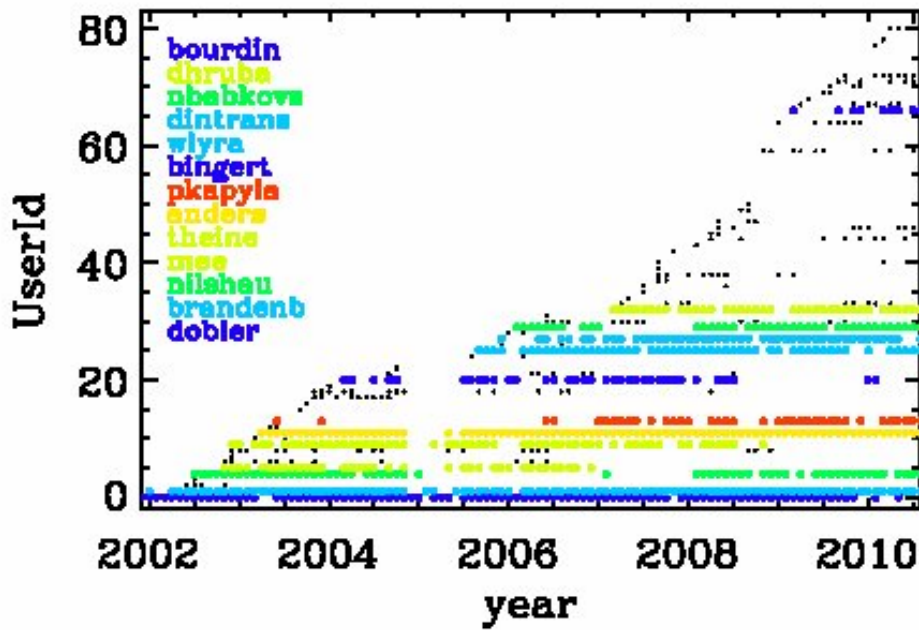


Figure 15: Check-ins since 2002 per user. Users with more than 100 check-ins are color coded.

8 Understanding the code

Understanding the code means looking through the code. This is not normally done by just printing out the entire code, but by searching your way through the code in order to address your questions. The general concept will be illustrated here with an example.

8.1 Example: how is the continuity equation being solved?

All the physics modules are solved in the routine `pde`, which is located in the file and module 'Equ'. Somewhere in the `pde` subroutine you find the line

```
call dlnrho_dt(f,df,p)
```

This means that here the part belonging to $\partial \ln \rho / \partial t$ is being assembled. Using the `grep` command you will find that this routine is located in the module `density`, so look in there and try to understand the pieces in this routine. We quickly arrive at the following crucial part of code,

```
!
!  Continuity equation.
!
      if (lcontinuity_gas) then
        if (ldensity_nolog) then
          df(11:12,m,n,irho) = df(11:12,m,n,irho) - p%ugrho - p%rho*p%divu
        else
          df(11:12,m,n,ilnrho) = df(11:12,m,n,ilnrho) - p%uglnrho - p%divu
        endif
      endif
endif
```

where, depending on some logicals that tell you whether the continuity equation should indeed be solved and whether we do want to solve for the logarithmic density and not the actual density, the correct right hand side is being assembled. Note that all these routines always only *add* to the existing `df(11:12,m,n,ilnrho)` array and never reset it. Resetting `df` is only done by the timestepping routine. Next, the pieces `p%uglnrho` and `p%divu` are being subtracted. These are *pencils* that are organized in the *structure* with the name `p`. The meaning of their names is obvious: `uglnrho` refers to $\mathbf{u} \cdot \nabla \ln \rho$ and `divu` refers to $\nabla \cdot \mathbf{u}$. In the subroutine `pencil_criteria_density` you find under which conditions these pencils are requested. Using `grep`, you also find where they are calculated. For example `p%uglnrho` is calculated in 'density.f90'; see

```
call u_dot_grad(f,ilnrho,p%glnrho,p%uu,p%uglnrho,UPWIND=lupw_lnrho)
```

So this is a call to a subroutine that calculates the $\mathbf{u} \cdot \nabla$ operator, where there is the possibility of *upwinding*, but this is *not* the default. The piece `divu` is calculated in 'hydro.f90' in the line

```
!
!  Calculate uij and divu, if requested.
!
      if (lpencil(i_uij)) call gij(f,iuu,p%uij,1)
      if (lpencil(i_divu)) call div_mn(p%uij,p%divu,p%uu)
```

Note that the divergence calculation uses the velocity gradient matrix as input, so no new derivatives are recalculated. Again, using `grep`, you will find that this calculation

and many other ones happen in the module and file 'sub.f90'. The various derivatives that enter here have been calculated using the `gij` routine, which calls the `der` routine, e.g., like so

```
k1=k-1
do i=1,3
  do j=1,3
    if (nder==1) then
      call der(f,k1+i,tmp,j)
```

For all further details you just have to follow the trail. So if you want to know how the derivatives are calculated, you have to look in `deriv.f90`, and only here is it where the indices of the `f` array are being addressed.

If you are interested in magnetic fields, you have to look in the file 'magnetic.f90'. The right hand side of the equation is assembled in the routine

```
!*****
  subroutine daa_dt(f,df,p)
!
!  Magnetic field evolution.
!
!  Calculate dA/dt=uxB+3/2 Omega_0 A_y x_dir -eta mu_0 J.
!  For mean field calculations one can also add dA/dt=...+alpha*bb+delta*WXJ.
!  Add jxb/rho to momentum equation.
!  Add eta mu_0 j2/rho to entropy equation.
!
```

where the header tells you already a little bit of what comes below. It is also here where ohmic heating effects and other possible effects on other equations are included, e.g.,

```
!
!  Add Ohmic heat to entropy or temperature equation.
!
  if (lentropy .and. lohmic_heat) then
    df(l1:l2,m,n,iss) = df(l1:l2,m,n,iss) &
      + etatotal*mu0*p%j2*p%rho1*p%TT1
  endif
```

We leave it at this and encourage the user to do similar inspection work on a number of other examples. If you think you find an error, file a ticket at <http://code.google.com/p/pencil-code/issues/list>. You can of course also repair it!

9 Adapting the code

9.1 The PENCIL CODE coding standard

As with any code longer than a few lines the appearance and layout of the source code is of the utmost importance. Well laid out code is more easy to read and understand and as such is less prone to errors.

A consistent coding style has evolved in the PENCIL CODE and we ask that those contributing try to be consistent for everybody's benefit. In particular, it would be appreciated if those committing changes of existing code via svn follow the given coding style.

There are not terribly many rules and using existing code as a template is usually the easiest way to proceed. In short the most important rules are:

- tab characters do not occur anywhere in the code (in fact the use of tab character is an extension to the Fortran standard).
- Code in any delimited block, e.g., if statements, do loops, subroutines etc., is indented by precisely 2 spaces. E.g.

```
if (lcylindrical) then
  call fatal_error('del2fjv','del2fjv not implemented')
endif
```

- continuation lines (i.e. the continuation part of a logical line that is split using the & sign) are indented by 4 spaces. E.g. (note the difference from the previous example)

```
if (lcylindrical) &
  call fatal_error('del2fjv','del2fjv not implemented')
[...]
```

- There is always one space separation between 'if' and the criterion following in parenthesis:

```
if (ldensity_nolog) then
  rho=f(l1:l2,m,n,irho)
endif
```

This is wrong:

```
if(ldensity_nolog) then    ! WRONG
  rho=f(l1:l2,m,n,irho)
endif
```

- In general, try to follow common practice used elsewhere in the code. For example, in the code fragment above there are no empty spaces within the mathematical expressions programmed in the code. A unique convention helps in finding certain expressions and patterns in the code. However, empty spaces are often used after commas and semicolons, for examples in name lists.
- Relational operators are written with symbols (==, / =, <, <=, >, >=), *not* with characters (.eq., .ne., .lt., .le., .gt., .ge.).
- In general all comments are placed on their own lines with the '!' appearing in the first column. It can be omitted in empty lines, but is yet recommended to be set in empty lines surrounding comments.

- All subroutine/functions begin with a standard comment block describing what they do, when and by whom they were created and when and by whom any non-trivial modifications were made.
- Lines longer than ~ 100 characters should be explicitly wrapped using the `&` character, unless there is a block of longer lines that can only be read easily when they are not wrapped. Always add one whitespace before the `&` character.

These and other issues are discussed in more depth and with examples in Appendix B, and in particular in Sect. B.2.

9.2 Adding new output diagnostics

With the implementation of new physics and the development of new procedures it will become necessary to monitor new diagnostic quantities that have not yet been implemented in the code. In the following, we describe the steps necessary to set up a new diagnostic variable.

This is nontrivial as, in order to keep latency effects low on multi-processor machines, the code minimizes the number of global reduction operations by assembling all quantities that need the maximum taken in *fmax*, and those that need to be summed up over all processors (mostly for calculating mean quantities) in *fsum* (see subroutine *diagnostic* in file `'src/equ.f90'`).

As a sample variable, let us consider *jbm* (the volume average $\langle j \cdot B \rangle$). Only the module *magnetic* will be affected, as you can see (the diagnostic quantity *jbm* is already implemented) with

```
unix> grep -i jbm src/*.f90
```

If we pretend for the sake of the exercise that no trace of *jbm* was in the code, and we were only now adding it, we would need to do the following

1. add the variable *idiag_jbm* to the *module variables* of *Magnetic* in both `'magnetic.f90'` and `'nomagnetic.f90'`:

```
integer :: idiag_jbm=0
```

The variable *idiag_jbm* is needed for matching the position of *jbm* with the list of diagnostic variables specified in `'print.in'`.

2. in the subroutine *daa_dt* in `'magnetic.f90'`, declare and calculate the quantity *jb* (the average of which will be *jbm*), and call *sum_mn_name*

```
real, dimension (nx) :: jb  !jj·BB
[...]
if (ldiagnos) then          ! only calculate if diagnostics is required
  if (idiag_jbm/=0) then     ! anybody asked for jbm?
    call dot_mn(jj,bb,jb)    ! assuming jj and bb are known
    call sum_mn_name(jb,i_jbm)
  endif
endif
endif
```

3. in the subroutine *rprint_magnetic* in both `'magnetic.f90'`, add the following:

```

!
!  reset everything in case of RELOAD
!  (this needs to be consistent with what is defined above!)
!
if (lreset) then  ! need to reset list of diagnostic variables?
  [...]
  idiag_jbm=0
  [...]
endif
!
!  check for those quantities that we want to evaluate online
!
do iname=1,nname
  [...]
  call parse_name(iname,cname(iname),cform(iname),'jbm',idiag_jbm)
  [...]
enddo
[...]
!
!  write column, i_XYZ, where our variable XYZ is stored
!
[...]
write(3,*) 'i_jbm=',idiag_jbm
[...]

```

4. in the subroutine `rprint_magnetic` in `'nomagnetic.f90'`, add the following (newer versions of the code may not require this any more):

```

!
!  write column, i_jbm, where our variable jbm is stored
!  idl needs this even if everything is zero
!
[...]
write(3,*) 'i_jbm=',idiag_jbm
[...]

```

5. and don't forget to add your new variable to `'print.in'`:

```
jbm(f10.5)
```

If, instead of a mean value, you want a new maximum quantity, you need to replace `sum_mn_name()` by `max_mn_name()`.

Sect. 5.9.1 describes how to output horizontal averages of the magnetic and velocity fields. New such averages can be added to the code by using the existing averaging procedures `calc_bmz()` or `calc_jmz()` as examples.

9.3 Output at one point in space

Various variables at one point can be printed on the command line. This is often important when you want to check for oscillations where the sign changes. You would not see it in the rms or max values. The extensions `pt` and `p2` refer to variables that are taken from two particular points in space.

Note: this would need to be reworked if one later makes the output positions processor-dependent. At the moment, those positions are in that part of the mesh that is on the root processor.

The file `'pt_positions.dat'` lists the coordinate positions where the data is taken from.

9.4 The f-array

The `'f'` array is the largest array in the PENCIL CODE and its primary role is to store the current state of the timestepped PDE variables. The `f`-array and its slightly smaller counter part (the `df`-array; see below) are the only full size 3D arrays in the code. The `f`-array is of type `real` but PDEs for a complex variable may be solved by using two slots in the `f`-array. The actual size of the `f`-array is $mx \times my \times mz \times mfarray$. Here, $mfarray = mvar + maux + mglobal + mscratch$ where `mvar` refers to the number of real PDE variables.

As an example, we describe here how to put the time-integrated velocity, `uut`, into the `f`-array (see `'hydro.f90'`). If this is to be invoked, there must be the following call somewhere in the code:

```
call farray_register_auxiliary('uut',iuut,vector=3)
```

Here, `iuut` is the index of the variable `uut` in the `f`-array. Of course, this requires that `maux` is increased by 3, but in order to do this for a particular run only one must write a corresponding entry in the `'cparam.local'` file,

```
!                               --f90--      (for Emacs)
!  cparam.local
!
!** AUTOMATIC CPARAM.INC GENERATION ****
! Declare (for generation of cparam.inc) the number of f array
! variables and auxiliary variables added by this module
!
! MAUX CONTRIBUTION 3
!
! ****
! Local settings concerning grid size and number of CPUs.
! This file is included by cparam.f90
!
integer, parameter :: ncpus=1,nprocy=1,nprocz=ncpus/nprocy,nprocx=1
integer, parameter :: nxgrid=16,nygrid=nxgrid,nzgrid=nxgrid
```

This way such a change does not affect the memory usage for other applications where this addition to `'cparam.local'` is not made. In order to output this part of the `f`-array, one must write `lwrite_aux=T` in the `init_pars` of `'start.in'`. (Technically, `[lwrite_aux]lwrite_aux=T` can also be invoked in `run_pars` of `'run.in'`, but this does not work at the moment.)

9.5 The df-array

The ‘df’ array is the second largest chunk of data in the PENCIL CODE. By using a 2N storage scheme (see H.4) after Williamson [41] the code only needs one more storage area for each timestepped variable on top of the current state stored in the f-array. As such, and in contrast to the f-array, the df-array is of size $m_x \times m_y \times m_z \times m_{var}$. Like the df-array it is of type real. In fact the ghost zones of df are not required or calculated but having f- and df-arrays of the same size make the coding more transparent. For m_x , m_y and m_z large the wasted storage becomes negligible.

9.6 The fp-array

Similar to the ‘f’ array the code also has a ‘fp’ array which contains current states of all the particles. Like the f-array the fp-array also has a time derivative part, the dfp-array. The dimension of the fp-array is $mpar_local \times mpvar$ where *mpar_local* is the number of particles in the local processor (for serial runs this is the total number of particles) and *mpvar* depends on the problem at hand. For example if we are solving for only tracer particles then $mpvar = 3$, for dust particles $mpvar = 6$. The sequence in which the slots in the fp-array are filled up depends on the sequence in which different particle modules are called from the particles_main.f90. The following are the relevant lines from particles_main.f90.

```
!*****
      subroutine particles_register_modules()
!
!   Register particle modules.
!
!   07-jan-05/anders: coded
!
      call register_particles          ()
      call register_particles_radius   ()
      call register_particles_spin     ()
      call register_particles_number   ()
      call register_particles_mass     ()
      call register_particles_selfgrav ()
      call register_particles_nbody    ()
      call register_particles_viscosity ()
      call register_pars_diagnos_state ()
!
      endsubroutine particles_register_modules
!*****
```

The subroutine register_particles can mean either the tracer particles or dust particles. For the former the first three slots of the fp-array are the three spatial coordinates. For the latter the first six slots of the fp-array are the three spatial coordinates followed by the three velocity components. The seventh slot (or the fourth if we are use tracer particles) is the radius of the particle which can also change as a function of time as particles collide and fuse together to form bigger particles.

9.7 The pencil case

Variables that are derived from the basic physical variables of the code are stored in one-dimensional *pencils* of length nx . All the pencils that are defined for a given set of physics modules are in turn bundled up in a Fortran structure called *p* (or, more illustrative, the *pencil case*). Access to individual pencils happens through the variable *p%name*, where *name* is the name of a pencil, e.g., *rho* that is a derived variable of the logarithmic density *lnrho*.

The pencils provided by a given physics module are declared in the header of the file, e.g., in the Density module:

```
! PENCILS PROVIDED lnrho; rho; rho1; glnrho(3); grho(3); uglnrho; ugrho
```

Notice that the pencil names are separated with a semi-colon and that vector pencils are declared with “(3)” after the name, and “(3,3)” for a 3×3 matrix. Before compiling the code, the script ‘mkcparam’ collects the names of all pencils that are provided by the chosen physics modules. It then defines the structure *p* with slots for every single of these pencils. The definition of the pencil case *p* is written in the include file ‘cparam_pencils.inc’. When the code is run, the actual pencils that are needed for the run are chosen based on the input parameters. This is done in the subroutines *pencil_criteria_modulename* that are present in each physics module. They are all called once before entering the time loop. In the *pencil_criteria* subroutines the logical arrays *lpenc_requested*, *lpenc_diagnos*, *lpenc_diagnos2d*, and *lpenc_video* are set according to the pencils that are needed for the given run. Some pencils depend on each other, e.g., *uglnrho* depends on *uu* and *glnrho*. Such interdependencies are sorted out in the subroutines *pencil_interdep_modulename* that are called after *pencil_criteria_modulename*.

In each time-step the values of the pencil logicals *lpenc_requested*, *lpenc_diagnos*, *lpenc_diagnos2d*, and *lpenc_video* are combined to one single pencil array *lpencil* which is different from time-step to time-step depending on, e.g., whether diagnostics or video output are done in that time-step. The pencils are then calculated in the subroutines *calc_pencils_modulename*. This is done before calculating the time evolution of the physical variables, as this depends very often on derived variables in pencils.

The centralized pencil calculation scheme is a guarantee that

- All pencils are only calculated once, and only once.
- Pencils are always calculated by the proper physics module.

Since the PENCIL CODE is a multipurpose code that has many different physics modules, it can lead to big problems if a module tries to calculate a derived variable that actually belongs to another module, because different input parameters can influence how the derived variables are calculated. One example is that the Density module can consider both logarithmic and non-logarithmic density, so if the Magnetic module calculates

```
rho = exp(f(l1:l2,m,n,ilnrho))
```

it is wrong if the Density module works with non-logarithmic density! The proper way for the Magnetic module to get to know the density is to request the pencil *rho* in *pencil_criteria_magnetic*.

9.7.1 Pencil check

To check that the correct pencils have been requested for a given run, one can run a *pencil consistency check* in the beginning of a run by setting the logical `lpencil_check` in `&run_pars`. The check is meant to see if

- All needed pencils have been requested
- All requested pencils are needed

The consistency check first calculates the value of `df` with all the requested pencils. Then the pencil requests are flipped one at a time – requested to not requested, not requested to requested. The following combination of events can occur:

- not requested → requested, `df` not changed
The pencil is not requested and is not needed.
- not requested → requested, `df` changed
The pencil is not requested, but is needed. The code stops.
- requested → not requested, `df` not changed
The pencil is requested, but is not needed. The code gives a warning.
- requested → not requested, `df` changed
The pencil is requested and is needed.

9.7.2 Adding new pencils

Adding a new pencil to the pencil case is trivial but requires a few steps.

- Declare the name of the pencil in the header of the proper physics module. Pencils names must appear come in a “;” separated list, with dimensions in parenthesis after the name [(3) for vector, (3,3) for matrix, etc.].
- Set interdependency of the new pencil (i.e. what other pencils does it depend on) in the subroutine `pencil_interdep_modulename`
- Make rule for calculating the pencil in `calc_pencils_modulename`
- Request the new pencil based on the input parameters in any relevant physics module

Remember that the centralized pencilation scheme is partially there to force the users of the code to think in general terms when implementing new physics. Any derived variable can be useful for a number of different physics problems, and it is important that a pencil is accessible in a transparent way to all modules.

9.8 Adding new physics: the Special module

If you want to add new physics to the code, you will in many cases want to add a new Special module. Doing so is relatively straightforward and there is even a special directory for such additions.

To create your own special module, copy ‘`nospecial.f90`’ from the `src/` directory to a new name in the `src/special/` directory. In many cases, users may want to put all new bits of physics, needed for the specific problem at hand, into a single special module. The name chosen for it should then relate to that problem. It is also possible to employ several (at present up to five) different special modules at a time in a single setup which allows to

let naming follow the specific physics being implemented (for technicalities in this case, see the end of this section).

The first thing to do in your new module is to change the `lspecial=.false.` header to say `lspecial=.true.`

The file is heavily commented though all such comments can be removed as you go. You may implement any of the subroutines/function that exist in *nospecial.f90* and those routines must have the names and parameters as in *nospecial.f90*. You do not however need to implement all routines, and you may either leave the dummy routines copied from *nospecial.f90* or delete them all together (provided the "include 'special-dummy.inc'" is kept intact at the end of the file. Beyond that, and data and subroutines can be added to a special module as required, though only for use within that module.

There are routines in the special interface to allow you to add new equations, modify the existing equation, add diagnostics, add slices, and many more things. If you feel there is something missing extra hooks can easily be added - please contact the PENCIL CODE team for assistance.

You are encouraged to submit/commit your special modules to the Pencil Code source. When you have added new stuff to the code, don't forget to mention this in the 'pencil-code/doc/manual.tex' file.

Using more than one special module at a time requires that the environment variables `$MODULE_PREFIX`, `$MODULE_SUFFIX` and `$MODULE_SUFFIX` are set properly at runtime. They can be derived from the *qualified names* of module functions which have in general the form `<prefix><module name><infix><function name><suffix>` with its details depending on the Fortran compiler used. These can be learned by employing the `nm` command, say, by

```
unix> nm src/general.o | more .
```

The environment variables are most conveniently set in the user's `.bashrc`, `.cshrc` or a proper configuration file of the PENCIL CODE (section environment). In the `Makefile.local` file, the requested special modules are simply specified as a list of names: `SPECIAL = special/<module 1> special/<module 2> ...`. In contrast to the case with only a single special module, where the namelists' names are `special_init_pars` and `special_run_pars`, these are individualized for multiple special modules, viz. `<module name>_init_pars` etc. As explicit linking at runtime is employed for multiple special modules, code errors, which normally would break the build, show possibly up only at runtime and are hence hard to debug. Therefore in case of unclear runtime failure, it is useful to perform tests with only one of the special modules at a time, thus guaranteeing full linking at build time.

For example, when

```
SPECIAL = special/gravitational_waves_hTXk special/chiral_mhd
```

is used, the namelist that is usually referenced as

```
&special_run_pars
/
```

needs to be replaced by:

```
&gravitational_waves_hTXk_run_pars
```



```
/
&chiral_mhd_run_pars
/
```

Internally, a number of *automatic* replacements occur in the code. Code that is automatically modified in this way is also automatically unmodified while checking in changes to the repository. But to facility comparison with the original code, one can do the unmodification also oneself using the `pencil-code/utis/axel/pc_mksspecial.sh` command.

9.9 Adding switchable modules

In some cases where a piece of physics is thought to be more fundamental, useful in many situations or simply more flexibility is required it may be necessary to add a new module *newphysics* together with the corresponding *nonewphysics* module. The special modules follow the same structure as the rest of the switchable modules and so using a special module to prototype new ideas can make writing a new switchable module much easier.

For an example of module involving a new variable (and PDE), the *pscalar* module is a good prototype. The `grep` command

```
unix> grep -i pscalar src/*
```

gives you a good overview of which files you need to edit or add.

9.10 Adding your initial conditions: the InitialCondition module

Although the code has many initial conditions implemented, we now *discourage* such practice. We aim to eventually removed most of them. The recommended course of action is to make use of the InitialCondition module.

InitialCondition works pretty much like the Special module. To implement your own custom initial conditions, copy the file `'noinitial_condition.f90'` from the `src/` to `src/initial_condition`, with a new, descriptive, name.

The first thing to do in your new module is to change the `linitialcondition=.false.` header to say `linitialcondition=.true.` Also, don't forget to add `../` in front of the file names in include statements.

This file has hooks to implement a custom initial condition to most variables. After implementing your initial condition, add the line `INITIAL_CONDITION=initial_condition/myinitialcondition` to your `'src/Makefile.local'` file. Here, `myinitialcondition` is the name you gave to your initial condition file. Add also `initial_condition_pars` to the `'start.in'` file, just below `init_pars`. This is a namelist, which you can use to add whichever quantity your initial condition needs defined, or passed. You must also un-comment the relevant lines in the subroutines for reading and writing the namelists. For compiling reasons, these subroutines in `'noinitial_condition.f90'` are dummies. The lines are easily identifiable in the code.

Check, e.g., the samples `'2d-tests/baroclinic'`, `'2d-tests/spherical_viscous_ring'`, or `'interlocked-fluxrings'`, for examples of how the module is used.

10 Testing the code

To maintain reproducibility despite sometimes quite rapid development, the PENCIL CODE is tested nightly on various architectures. The front end for testing are the scripts `pc_auto-test` and (possibly) `pencil-test`.

To see which samples would be tested, run

```
unix> pc_auto-test -l
```

, to actually run the tests, use

```
unix> pc_auto-test
```

or

```
unix> pc_auto-test --clean
```

. The latter compiles every test sample from scratch and currently (September 2009) takes about 2 hours on a mid-end Linux PC.

The `pencil-test` script is useful for cron jobs and allows the actual test to run on a remote computer. See Sect. 10.1 below.

For a complete list of options, run `pc_auto-test --help` and/or `pencil-test --help`.

10.1 How to set up periodic tests (auto-tests)

To set up a nightly test of the PENCIL CODE, carry out the following steps.

1. Identify a host for running the actual tests (the *work host*) and one to initiate the tests and collect the results (the *scheduling host*). On the scheduling host, you should be able to
 - (a) run cron jobs,
 - (b) ssh to the work host without password,
 - (c) publish HTML files (optional, but recommended),
 - (d) send e-mail (optional, but recommended).

Work host and scheduling host can be the same (in this case, use `pencil-test`'s `-l` option, see below), but often they will be two different computers.

2. [Recommended, but optional:] On the work host, check out a separate copy of the PENCIL CODE to reduce the risk that you start coding in the auto-test tree. In the following, we will assume that you checked out the code as `~/pencil-auto-test`.
3. On the work host, make sure that the code finds the correct configuration file for the tests you want to carry out. [Elaborate on that: `PENCIL_HOME/local_config` and `-f` option; give explicit example]

Remember that you can set up a custom host ID file for your auto-test tree under `${PENCIL_HOME}/config-local/hosts/`.

4. On the scheduling host, use `crontab -e` to set up a cron job similar to the following:

```
30 02 * * * $HOME/pencil-auto-test/bin/pencil-test \
-D $HOME/pencil-auto-test \
--use-pc_auto-test \
```

```
-N15 -Uc -rs \
-T $HOME/public_html/pencil-code/tests/timings.txt \
-t 15m
-m <email1@inter.net,email2@inter.net,...> \
<work-host.inter.net> \
-H > $HOME/public_html/pencil-code/tests/nightly-tests.html
```

Note 1: This has to be one long line. The backslash characters are written only for formatting purposes for this manual *you cannot use them in a crontab file*.

Note 2: You will have to adapt some parameters listed here and may want to modify a few more:

‘-D <dir>’: Sets the directory (on the work host) to run in.

‘-T <file>’: If this option is given, append a timing statistics line for each test to the given *file*.

‘--use-pc’: You want this option (and at some point, it will be the default).

‘-t 15m’: Limit the time for ‘start.x’ and ‘run.x’ to 15 minutes.

‘-N 15’: Run the tests at nice level 15 (may not have an effect for MPI tests).

‘-Uc’: Do svn update and pc_build --cleanall before compiling.

‘-m <email-list>’: If this option is given, send e-mails to everybody in the (comma-separated) list of e-mail addresses if any test fails. As soon as this option is set, the maintainers (as specified in the ‘README’ file) of failed tests will also receive an e-mail.

work-host.inter.net|-l: Replace this with the remote host that is to run the tests. If you want to run locally, write -l instead.

‘-H’: Output HTML.

> \$HOME/public_html/pencil-code/tests/nightly-tests.html: Write output to the given file.

If you want to run fewer or more tests, you can use the ‘-Wa,--max-level’ option:

```
-Wa,--max-level=3
```

will run all tests up to (and including) level 3. The default corresponds to ‘-Wa,--max-level=2’.

For a complete listing of pencil-test options, run

```
unix> pencil-test --help
```

10.2 Auto-tests with systemd

On modern Linux systems, you can use systemd (instead of cron) to run periodic auto-tests. You need to create a couple of files in ‘~/ .config/systemd/user/’:

‘pencil_test.service’¹⁸

¹⁸Options and filepaths may need to be modified; note that %h is used to denote the user’s home directory.

```
[Unit]
Description=Pencil-code test

[Service]
Type=simple
Environment="PENCIL_HOME=%h/.software/pencil-code-for-tests"
ExecStart=%h/.software/pencil-code-for-tests/bin/pencil-test \
  -N 15 --update --html --clean --local --use-pc_auto-test \
  --auto-test-options="--max-level=3 --script-tests=python --time-limit=5m"
StandardOutput=truncate:%h/public_html/pencil_tests/master_full.html
```

(the backslashes can be left as-is) and ‘pencil_test.timer’

```
[Unit]
Description=Run pencil test daily at 10pm

[Timer]
OnCalendar=*-*-* 22:00:00
Persistent=True

[Install]
WantedBy=timers.target
```

After creating these files, run

```
systemctl --user enable pencil_test.timer
systemctl --user start pencil_test.timer
```

10.3 Testing the postprocessing modules

Some of the samples contain additional scripts that test the Python and IDL postprocessing modules. They are not checked by `pc_auto-test` by default; to include these tests, use the `--script-tests` option, e.g.

```
pc_auto-test --max-level=3 --script-tests=python
```

The Python postprocessing modules contain an additional set of quick tests that can be invoked as described in ‘PENCIL_HOME/python/tests/README.md’.

11 Useful internals

11.1 Global variables

The following variables are defined in ‘*cdata.f90*’ and are available in any routine that uses the module *Cdata*.

<i>Variable</i>	<i>Meaning</i>
real	
<i>t</i>	simulated time <i>t</i> .
integer	
<i>n[xyz]grid</i>	global number of grid points (excluding ghost cells) in <i>x</i> , <i>y</i> and <i>z</i> direction.
<i>nx, ny, nz</i>	number of grid points (excluding ghost cells) as seen by the current processor, i. e. $ny = nygrid/nprocy$, etc.
<i>mx, my, mz</i>	number of grid points seen by the current processor, but <i>including ghost cells</i> . Thus, the total box for the <i>ivarth</i> variable (on the given processor) is given by $f(1:mx, 1:my, 1:mz, ivar)$.
<i>l1, l2</i>	smallest and largest <i>x</i> -index for the physical domain (i. e. excluding ghost cells) on the given processor.
<i>m1, m2</i>	smallest and largest <i>y</i> -index for physical domain.
<i>n1, n2</i>	smallest and largest <i>z</i> -index for physical domain, i. e. the physical part of the <i>ivarth</i> variable is given by $f(l1:l2, m1:m2, n1:n2, ivar)$
<i>m, n</i>	pencil indexing variables: During each time-substep the box is traversed in <i>x</i> -pencils of length <i>mx</i> such that the current pencil of the <i>ivarth</i> variable is $f(l1:l2, m, n, ivar)$.
logical	
<i>lroot</i>	true only for MPI root processor.
<i>lfirst</i>	true only during first time-substep of each time step.
<i>headt</i>	true only for very first full time step (comprising 3 substeps for the 3rd-order Runge–Kutta scheme) on root processor.
<i>headtt</i>	$= (lfirst \text{ .and. } lroot)$: true only during very first time-substep on root processor.
<i>lfirstpoint</i>	true only when the very first pencil for a given time-substep is processed, i. e. for the first set of (<i>m, n</i>), which is probably (3, 3) .
<i>lout</i>	true when diagnostic output is about to be written.

11.2 Subroutines and functions

`output(file,a,nv)` (module *IO*): Write (in each ‘*procN*’ directory) the content of the global array *a* to a file called *file*, where *a* has dimensions $mx \times my \times mz \times nv$, or $mx \times my \times mz$ if $nv=1$.

`output_pencil(file,a,nv)` (module *IO*): Same as `output()`, but for a pencil variable, i. e. an auxiliary variable that only ever exists on a pencil (e. g. the magnetic field strength *bb* in ‘magnetic.f90’, or the squared sound speed *cs2* in ‘entropy.f90’). The file has the same structure as those written by `output()`, because the values of *a* on the different pencils are accumulated in the file. This involves a quite non-trivial access pattern to the file and has thus been coded in C (‘src/debug_c.c’).

`cross(a,b,c)` (module *Sub*): Calculate the cross product of two vectors *a* and *b* and store in *c*. The vectors must either all be of size $mx \times my \times mz \times 3$ (global arrays), or of size $nx \times 3$ (pencil arrays).

`dot(a,b,c)` (module *Sub*): Calculate the dot product of two vectors *a* and *b* and store in *c*. The vectors must either be of size $mx \times my \times mz \times 3$ (*a* and *b*) and $mx \times my \times mz$ (*c*), or of size $nx \times 3$ (*a* and *b*) and *nx* (*c*).

`dot2(a,c)` (module *Sub*): Same as `dot(a,a,c)`.

Part III

Appendix

APPENDIX

Date, Revision

A Timings

In the following table we list the results of timings of the code on different machines. Shown is (among other quantities) the wall clock time per mesh point (excluding the ghost zones) and per full 3-stage time step, a quantity that is printed by the code at the end of a run.¹⁹

As these results were assembled during the development phase of the code (that hasn't really finished yet,...), you may not get the same numbers, but they should give some orientation of what to expect for your specific application on your specific hardware.

The code will output the timing (in microseconds per grid point per time-step) at the end of a run. You can also specify `walltime` in `print.in` to have the code continuously output the physical time it took to reach the time-steps where diagnostics is done. The time-dependent code speed can then be calculated by differentiating, e.g., in IDL with

```
IDL> pc_read_ts, obj=ts
```

```
IDL> plot, ts.it, 1/nw*deriv(ts.it,ts.walltime/1.0e-6), psym=2
```

where `nw=nx*ny*nz`.

proc	machine	$\frac{\mu s}{pt \ step}$	resol.	what	mem/proc	when	who
1	Nl3	19	64 ³	kinematic	10 MB	20-may-02	AB
1	Nl3	30	64 ³	magn/noentro	20 MB	20-may-02	AB
1	Nq1	10	64 ³	magn/noentro		30-may-02	AB
1	Ukaff	9.2	64 ³	magn/noentro		20-may-02	AB
1	Nl6	6.8	64 ³	magn/noentro		10-mar-03	AB
1	Nl6	36.3	64×128×64	nomag/entro/dust		19-sep-03	AB
1	Nl6	42.7	16 ² ×256	nomag/entro/rad6/ion		22-oct-03	AB
1	Nl6	37.6	16 ² ×256	nomag/entro/rad2/ion		22-oct-03	AB
1	Nl6	19.6	16 ² ×256	nomag/entro/ion		22-oct-03	AB
1	Nl6	8.7	16 ² ×256	nomag/entro		22-oct-03	AB
1	Nl6n	9.8	32 ³	magn/noentro/pscalar		17-mar-06	AB
1	Mhd	7.8	64 ³	magn/noentro		20-may-02	AB
1	Nq4	14.4	128 ³	magn/noentro		8-oct-02	AB
1	Nq5	6.7	128 ³	magn/noentro		8-oct-02	AB
1	fe1	5.1	128 ³	magn/noentro		9-oct-02	AB
1	Kabul	4.4	128 ³	magn/noentro	130 MB	20-jun-02	WD
1	Hwvsx5	3.4	256 ³	convstar	7.8 GB	29-jan-03	WD
1	Mac/g95	7.7	32 ³	magn/noentro		14-jan-07	BD
1	Mac/ife	4.5	32 ³	magn/noentro		14-jan-07	BD
2	Kabul	2.5	128 ³	magn/noentro	80 MB	20-jun-02	WD
2	Nq3+4	7.4	128 ³	magn/noentro		8-oct-02	AB
2	Nq4+4	8.9	128 ³	magn/noentro		8-oct-02	AB
2	Nq4+5	7.3	128 ³	magn/noentro		8-oct-02	AB
2	Nq5+5	3.7	128 ³	magn/noentro		8-oct-02	AB
2	fe1	3.45	128 ³	magn/noentro		9-oct-02	AB

¹⁹Note that when using 'nompicomm.f90', the timer currently used will overflow on some machines, so you should not blindly trust the timings given by the code.

2	Nq2	9.3	64 ³	magn/noentro		11-sep-02	AB
2	Nq1+2	8.3	64 ³	magn/noentro		11-sep-02	AB
2	Hwwsx5	1.8	256 ³	convstar	7.9 GB	29-jan-03	WD
4	Nq1+2	5.4	64 ³	magn/noentro		11-sep-02	AB
4	Nq1235	4.1	128 ³	magn/noentro		11-sep-02	AB
4	Nq0-3	6.8	256 ³	magn/noentro	294 MB	10-jun-02	AB
4	Mhd	2.76	64 ³	magn/noentro		30-may-02	AB
4	fe1	3.39	32 ³	magn/noentro		16-aug-02	AB
4	Rasm.	2.02	64 ³	magn/noentro	2x2	8-sep-02	AB
4	Mhd	8.2	64 ² ×16	nomag/entro		23-jul-02	AB
4	fe1	6.35	64×128×64	nomag/entro/dust		19-sep-03	AB
4	fe1	2.09	128 ³	magn/noentro		9-oct-02	AB
4	fe1	1.45	128 ³	magn/noentro	giga	9-oct-02	AB
4	fe1	7.55	16 ² ×512	nomag/entro/rad2/ion	4x1	1-nov-03	AB
4	fe1	5.48	16 ² ×512	nomag/entro/rad2/ion	1x4	1-nov-03	AB
4	Luci	1.77	64 ³	magn/noentro		27-feb-07	AB
4	Lenn	0.65	64 ³	nomag/noentro		13-jan-07	AB
4	Lenn	1.21	64 ³	magn/noentro		7-nov-06	AB
4	Kabul	1.5	128 ³	magn/noentro	47 MB	20-jun-02	WD
4	Hwwsx5	1.8	256 ³	convstar	8.2 GB	29-jan-03	WD
8	Nqall	3.0	128 ³	magn/noentro		8-oct-02	AB
8	fe1	3.15	64 ³	magn/noentro	1x8	8-sep-02	AB
8	fe1	2.36	64 ³	magn/noentro	2x4	8-sep-02	AB
8	Ukaff	1.24	64 ³	magn/noentro		20-may-02	AB
8	Kabul	1.25	64 ² ×128	nomag/entro		11-jul-02	WD
8	fe1	1.68	128 ³	magn/noentro	1x8	8-sep-02	AB
8	fe1	1.50	128 ³	magn/noentro	2x4	8-sep-02	AB
8	fe1	1.44	128 ³	magn/noentro	4x2	8-sep-02	AB
8	Kabul	0.83	128 ³	magn/noentro	28 MB	20-jun-02	WD
8	Gridur	1.46	128 ³	magn/noentro		19-aug-02	NE
8	Kabul	0.87	256 ³	magn/noentro	160 MB	20-jun-02	WD
8	fe1	0.99	256 ³	magn/noentro	2x4	8-sep-02	AB
8	fe1	0.98	256 ³	magn/noentro	4x2	8-sep-02	AB
8	cetus	0.58	64 ³	magn/noentro	4x2	19-aug-07	SS
8	cetus	0.73	256 ³	magn/noentro	4x2,156M	19-aug-07	SS
8	Neolith	0.82	64 ³	magn/noentro	4x2	5-dec-07	AB
8	Mhd	1.46	160 ² ×40	nomag/entro	46 MB	7-oct-02	AB
8	Hwwsx5	0.50	256 ³	convstar	8.6 GB	29-jan-03	WD
8	Neolith	0.444	128 ³	magn/noentro		6-dec-07	AB
8	Ferlin	0.450	64 ³	ltest/noentro		21-jun-09	AB
8	Ferlin	0.269	64 ³	magn/noentro		2-apr-10	AB
8	Ferlin	0.245	128 ³	magn/noentro		2-feb-11	AB
8	nor52	2.00	32 ³	magn/noentro		2-dec-09	AB
9	hydra(2)	0.317	72 ³	magn/noentro	1x3x3	8-may-16	AB
9	charybdis	0.169	72 ³	magn/noentro	1x3x3	8-may-16	AB
9	scylla	0.150	72 ³	magn/noentro	1x3x3	8-may-16	AB
12	scylla	0.151	72 ³	magn/noentro	1x4x3	8-may-16	AB
12	janus	6.02	72 ² × 22	coag/noentro		17-dec-15	AB
16	fe1	1.77	64 ³	convstar		9-feb-03	AB
16	copson	0.596	128 ³	geodynamo/ks95		21-nov-03	DM
16	fe1	0.94	128 ³	magn/noentro	4x4	8-sep-02	AB
16	fe1	0.75	128 ³	magn/noentro	4x4/ifc6	9-may-03	AB
16	workq	0.88	128 ³	magn/noentro	4x4/ifc6	21-aug-04	AB
16	giga	0.76	128 ³	magn/noentro	4x4/ifc6	21-aug-04	AB
16	giga2	0.39	128 ³	magn/noentro	4x4/ifc6	20-aug-04	AB
16	giga	0.47	128 ³	chiral	4x4/ifc6	29-may-04	AB
16	giga	0.43	128 ³	nomag/noentro	4x4/ifc6	28-apr-03	AB
16	Mhd	2.03	128 ³	magn/noentro		26-nov-02	AB
16	Mhd	0.64	256 ³	magn/noentro	60 MB	22-may-02	AB
16	fe1	0.56	256 ³	magn/noentro	4x4	16-aug-02	AB

16	fe1	6.30	128×256×128	nomag/entro/dust		19-sep-03	AB
16	fe1	1.31	128 ² ×512	nomag/entro/rad2/ion	4x4	1-nov-03	AB
16	Ukaff	0.61	128 ³	magn/noentro		22-may-02	AB
16	Ukaff	0.64	256 ³	magn/noentro		20-may-02	AB
16	Kabul	0.80	128 ³	magn/noentro	16 MB	20-jun-02	WD
16	Kabul	0.51	256 ³	magn/noentro	9 MB	20-jun-02	WD
16	Gridur	0.81	128 ³	magn/noentro		19-aug-02	NE
16	Gridur	0.66	256 ³	magn/noentro		19-aug-02	NE
16	Sander	0.53	256 ³	magn/noentro		8-sep-02	AB
16	Luci	0.375	128 ³	magn/noentro		28-oct-06	AB
16	Lenn	0.284	128 ³	magn/noentro		8-nov-06	AB
16	Neolith	0.180	256 ³	magn/noentro		6-dec-07	AB
16	Triolith	0.075	128 ³	magn/noentro	2x2x4	1-mar-14	AB
16	Triolith	0.065	128 ³	magn/noentro	1x4x4	1-mar-14	AB
16	Triolith	0.054	256 ³	magn/noentro	1x4x4	1-mar-14	AB
16	Coma	0.603	128 ³	GWo/magn/noentro	1x4x4	27-jul-17	SM
24	Gardar	0.44	128 ² × 48	magn/noentro		6-nov-13	AB
24	Summit	0.041	144 ³	magn/noentro		28-jul-17	AB
32	giga?	0.32	256 ³	magn/noentro		13-sep-03	AB
32	Ukaff	0.34	256 ³	magn/noentro		20-may-02	AB
32	Ukaff	0.32	512 ³	magn/noentro		20-may-02	AB
32	Hermit	0.200	256×512×256	spherical conv/magn	1x8x4	22-aug-13	PJK
32	fe1	0.168	512 ³	nomag/noentro		9-oct-02	AB
32	Dardel	0.038	128 ³	nomag/noentro		21-oct-21	AB
32	fe1	1.26	64 ² ×256	nomag/entro/rad/ion		7-sep-03	AB
32	DarCO0	0.120	32 ³	magn/noentro		22-oct-21	AB
32	Lenn	0.147	256 ³	nomag/entro/cool/fo	4x8	8-nov-06	AB
32	Steno	0.076	256 ³	nomag/entro/cool/fo	4x8	20-jun-06	AB
32	Steno	0.081	256 ³	nomag/entro/cool	4x8	20-jun-06	AB
32	Steno	0.085	256 ³	nomag/entro/cool/sh	4x8	20-jun-06	AB
32	Steno	0.235	512 ² ×256	mag/entro	4x8	9-jul-06	AB
32	Sanss	0.273	128×256 ²	nomag	4x8	3-jul-07	AB
32	Neolith	0.275	128 ³	testfield4		24-oct-08	AB
32	Ferlin	0.556	128 ³	testscalar		7-jan-09	AB
36	Kraken	0.177	192×384×64	magn/noentro	3x6x2	12-jan-12	WL
36	scylla	0.096	72 ³	magn/noentro	1x6x6	8-may-16	AB
48	janus	0.028	72 ² * 216	magn/noentro	4x12	28-mar-16	AB
64	Coma	0.573	128 ³	GWo/magn/noentro	1x8x8	7-aug-17	SM
64	fe1	0.24	256 ³	magn/noentro	8x8	2-sep-02	AB
64	giga	0.11	256 ³	nomag/noentro	4x16	29-apr-03	AB
64	giga	0.23	256 ³	nomag/noentro/hyp	4x16	8-dec-03	AB
64	fe1	0.164	512 ³	nomag/noentro/hyp	4x16	17-dec-03	AB
64	giga	0.091	512 ³	nomag/noentro/hyp	4x16	17-dec-03	AB
64	giga	0.150	256 ³	magn/noentro	4x16	1-jul-03	AB
64	giga	0.166	512 ³	magn/noentro	64*173MB	10-jul-03	AB
64	Gridur	0.25	256 ³	magn/noentro		19-aug-02	NE
64	Ukaff	0.17	512 ³	magn/noentro		21-may-02	AB
64	Steno	0.075	512 ³	magn/noentro	8x16	19-oct-06	AB
64	Neolith	0.0695	256 ³	magn/noentro		6-dec-07	AB
64	Ferlin	8.51	150×128 ²	Li mechanism	8x8	21-jun-09	AB
64	Ferlin	0.156	256 ³	magn/noentro	8x8	14-jun-09	AB
64	Akka	0.038	256 ² ×512	magn/noentro	8x8	27-dec-12	AB
64	Triolith	0.0146	256 ³	magn/noentro	1x8x8	1-mar-14	AB
64	Triolith	0.0164	256 ³	magn/noentro	2x4x8	1-mar-14	AB
64	Hermit	0.101	256×512×256	spherical conv/magn	1x8x8	22-aug-13	PJK
64	Sisu	0.00205	256×512×256	spherical conv/magn	1x8x8	22-aug-13	PJK
72	Kraken	0.093	192×384×64	magn/noentro	3x12x2	12-jan-12	WL
72	Kraken	0.151	96×192×16	magn/noentro	6x12	17-jan-12	WL
72	Kraken	0.091	192×384×32	magn/noentro	6x12	17-jan-12	WL
72	Kraken	0.071	384×768×64	magn/noentro	6x12	17-jan-12	WL

72	Summit	0.0128	576 ³	magn/noentro		7-aug-17	AB
128	fe1	0.44	256 ³	nomag/entro/rad8/ion	4x32	10-mar-04	TH
128	fe1	2.8	512 ³	magn/noentro	16x8	5-sep-02	AB
128	fe1	0.51	512 ³	magn/noentro	8x16	5-sep-02	AB
128	fe1	0.27	512 ³	magn/noentro	4x32	5-sep-02	AB
128	fe1	0.108	512 ³	magn/noentro	4x32/ifc6	5-jan-02	AB
64+64	giga2	0.0600	512 ³	magn/noentro	4x32/ifc6	21-aug-04	AB
128l	giga2	0.0605	512 ³	magn/noentro	4x32/ifc6	21-aug-04	AB
128	fe1	0.35	512 ³	magn/noentro	2x64	9-sep-02	AB
128	fe1	0.094	786 ³	magn/noentro	4x32/ifc6	9-sep-02	AB
128	DarCO0	0.019	128 ³	magn/noentro	4x4x8	22-oct-21	AB
128	Hermit	0.0532	256x512x256	spherical conv/magn	1x16x8	22-aug-13	PJK
128	Hermit	0.0493	256x512x256	spherical conv/magn	2x8x8	22-aug-13	PJK
128	Sisu	0.00108	256x512x256	spherical conv/magn	1x16x8	22-aug-13	PJK
144	Kraken	0.080	96x192x32	magn/noentro	6x12x2	13-jan-12	WL
144	Kraken	0.058	192x384x64	magn/noentro	6x12x2	17-jan-12	WL
144	Kraken	0.044	384x768x128	magn/noentro	6x12x2	18-jan-12	WL
144	Gardar	2.19	288x1x288	coag43	8x1x18	13-sep-15	AB
144	Summit	0.0064	576 ³	magn/noentro		7-aug-17	AB
192	Janus	0.0123	144x288x72	magn/noentro/sph	1x24x32	24-jul-16	AB
256	Hermit	0.0328	512x1024x512	spherical conv/magn	1x16x16	22-aug-13	PJK
256	Hermit	0.0285	256x512x256	spherical conv/magn	1x16x16	22-aug-13	PJK
256	giga2	0.028	1024 ³	magn/noentro	4x64/ifc6	20-aug-04	AB
256	Hermit	0.0262	256x512x256	spherical conv/magn	2x16x8	22-aug-13	PJK
256	Hermit	0.0254	512x1024x512	spherical conv/magn	2x16x8	22-aug-13	PJK
256	Hermit	0.0226	512x1024x512	spherical conv/magn	4x8x8	22-aug-13	PJK
256	Akka	0.0113	512 ³	magn/noentro	16x16	12-jun-11	AB
256	Beskow	0.0045	128 ³	magn/noentro	4x8x8	22-jul-20	AB
256	Sisu	0.00618	256x512x256	spherical conv/magn	1x16x16	22-aug-13	PJK
256	Sisu	0.00500	512x1024x512	spherical conv/magn	1x16x16	22-aug-13	PJK
256	Triolith	0.030	256 ² × 512	magn/rad	1x16x16	17-mar-14	AB
256	Triolith	0.0049	256 ³	magn/noentro	1x16x16	1-mar-14	AB
256	DarCO0th	0.00103	128 ³	magn/noentro	1x16x16	21-oct-21	AB
256	Beskow	3.36	1x1x1024	coag43	1x1x256	3-mar-15	AB
256	Beskow	7e-3	256 ³	coag43	1x16x16	9-jan-20	AB
288	Gardar	0.042	576 ² × 288	magn/rad	1x18x16	17-mar-14	AB
288	Kraken	0.0432	192x384x64	magn/noentro	6x12x4	12-jan-12	WL
288	Kraken	0.0447	96x192x64	magn/noentro	6x12x4	13-jan-12	WL
288	Kraken	0.0201	384x768x256	magn/noentro	6x12x4	18-jan-12	WL
288	Janus	0.0360	288 ³	magn/entro/rad	1x16x18	22-feb-16	AB
288	Summit	0.0033	576 ³	magn/noentro	1x16x18	7-aug-17	AB
512	Hermit	0.01717	512x1024x512	spherical conv/magn	1x32x16	22-aug-13	PJK
512	Hermit	0.0166	256x512x256	spherical conv/magn	1x32x16	22-aug-13	PJK
512	Hermit	0.0142	256x512x256	spherical conv/magn	2x16x16	22-aug-13	PJK
512	Hermit	0.01340	512x1024x512	spherical conv/magn	2x16x16	22-aug-13	PJK
512	Hermit	0.01189	512x1024x512	spherical conv/magn	8x8x8	22-aug-13	PJK
512	Hermit	0.01165	512x1024x512	spherical conv/magn	4x16x8	22-aug-13	PJK
512	Akka	0.0081	512 ³	magn/noentro	16x32	10-sep-11	AB
512	Neolith	0.0073	256 ³	magn/noentro		20-nov-09	AB
512	Gardar	0.0035	512 ³	magn/noentro		14-jan-13	AB
512	Lindgren	0.0040	512 ² × 1024	magn/noentro	16x32	8-jul-12	AB
512	Beskow	0.0019	512 ³	magn/noentro	1x8x16	24-jul-20	AB
512	Beskow	0.0015	512 ³	magn/noentro	1x8x16	21-oct-21	AB
512	DarCO0	0.00061	128 ³	magn/noentro	8x8x8	22-oct-21	AB
512	DarCO0	0.00042	256 ³	magn/noentro	8x8x8	22-oct-21	AB
512	DarCO1	0.00014	256 ³	magn/noentro	8x8x8	22-oct-21	AB
512	DarCO2	0.00012	256 ³	magn/noentro	8x8x8	22-oct-21	AB
512	Sisu	0.00446	256x512x256	spherical conv/magn	4x16x8	22-aug-13	PJK
512	Sisu	0.00435	1024x2048x1024	spherical conv/magn		22-aug-13	PJK
512	Sisu	0.00268	512x1024x512	spherical conv/magn	1x32x16	22-aug-13	PJK

576	Kraken	0.0257	192×384×64	magn/noentro	6x24x4	12-jan-12	WL
576	Kraken	0.0317	192 ² ×64	magn/noentro	12 ² x4	13-jan-12	WL
576	Kraken	0.0116	768 ² ×256	magn/noentro	12 ² x4	18-jan-12	WL
576	Summit	0.00183	576 ³	magn/noentro	1x24x48	29-jul-17	AB
576	Beskow	0.00174	576 ³	magn/noentro	1x24x48	23-may-16	AB
768	Lindgren	0.0049	256×1152 ²	magn/noentro/sph	1x24x32	17-oct-14	SJ
1024	Hermit	0.00943	512×1024×512	spherical conv/magn	1x32x32	22-aug-13	PJK
1024	Hermit	0.00707	512×1024×512	spherical conv/magn	2x32x16	22-aug-13	PJK
1024	Hermit	0.00698	1024×2048×1024	spherical conv/magn	4x16x16	22-aug-13	PJK
1024	Hermit	0.00630	512×1024×512	spherical conv/magn	4x16x16	22-aug-13	PJK
1024	Triolith	0.00236	256 ³	magn/noentro	4x16x16	1-mar-14	AB
1024	Triolith	0.00126	512 ³	magn/noentro	2x16x32	1-mar-14	AB
1024	Triolith	0.00129	512 ³	magn/noentro	4x16x16	1-mar-14	AB
1024	Sisu	0.00225	1024×2048×1024	spherical conv/magn		22-aug-13	PJK
1024	Sisu	0.00148	512×1024×512	spherical conv/magn	2x32x16	22-aug-13	PJK
1152	Kraken	0.0212	192×384×64	magn/noentro	12x24x4	13-jan-12	WL
1152	Kraken	0.00856	384×768×128	magn/noentro	12x24x4	17-jan-12	WL
1152	Kraken	0.00549	768×1536×256	magn/noentro	12x24x4	17-jan-12	WL
1152	Lindgren	0.016	512 ² ×512	magn/rad	1x36x32	17-mar-14	AB
1152	Lindgren	0.0066	1152 ³	magn/noentro	1x32x36	25-nov-14	AB
1152	Beskow	0.0055	1152 ³	GWo/magn/noentro	1x32x36	27-aug-17	AB
1152	Beskow	0.0024	1152 ³	magn/noentro	1x32x36	20-jan-15	AB
1152	Beskow	0.00098	1152 ³	magn/noentro	1x32x36	18-jan-16	AB-gnu
1152	Beskow	0.00090	1152 ³	magn/noentro	1x32x36	30-mar-17	AB
1152	Beskow	0.0060	1152 ³	GWo/magn/noentro	1x32x36	31-mar-18	AB
1152	Beskow	0.0063	576 ³	magn/entro/rad	1x32x36	17-feb-18	AB
1152	Beskow	0.0030	1152 ³	GWn/nomagn/noentro	1x32x36	30-jul-20	AB
1152	Beskow	0.0017	1152 ³	GWo/nomagn/noentro	1x32x36	30-jul-20	AB
1536	Lindgren	0.00171	512 ² ×384	magn/noentro	2x32x24	15-jul-13	AB
2048	Hermit	0.00451	1024×2048×1024	spherical conv/magn	2x32x32	22-aug-13	PJK
2048	Hermit	0.00380	512×1024×512	spherical conv/magn	8x16x16	22-aug-13	PJK
2048	Hermit	0.00355	512×1024×512	spherical conv/magn	4x32x16	22-aug-13	PJK
2048	Hermit	0.00350	1024×2048×1024	spherical conv/magn	4x32x16	22-aug-13	PJK
2048	Beskow	0.0022	1024 ³	GWn/nomagn/noentro	8x16x16	16-aug-20	AB
2048	Lindgren	0.00129	512 ² ×1024	magn/noentro	32x64	20-apr-13	AB
2048	Lindgren	0.00129	1024 ² ×2048	magn/noentro	32x64	31-jul-12	AB
2048	Triolith	9.3×10 ⁻⁴	512 ³	magn/noentro	4x16x32	1-mar-14	AB
2048	Sisu	0.00120	1024×2048×1024	spherical conv/magn		22-aug-13	PJK
2048	Sisu	9.2×10 ⁻⁴	512×1024×512	spherical conv/magn	4x32x16	22-aug-13	PJK
2304	Triolith	1.07×10 ⁻³	576 ³	magn/noentro	4x18x32	1-mar-14	AB
2304	Kraken	0.02267	192×384×64	magn/noentro	12x24x8	13-jan-12	WL
2304	Kraken	0.01233	192×768×64	magn/noentro	12x48x4	13-jan-12	WL
2304	Kraken	0.00300	768×3072×256	magn/noentro	12x48x4	18-jan-12	WL
4096	Hermit	0.00193	1024×2048×1024	spherical conv/magn	4x32x32	22-aug-13	PJK
4096	Triolith	3.6×10 ⁻⁴	1024 ³	magn/noentro	4x32x32	1-mar-14	AB
4096	Triolith	3.8×10 ⁻⁴	1024 ³	magn/noentro	8x16x32	1-mar-14	AB
4096	Triolith	4.2×10 ⁻⁴	1024 ³	magn/noentro	4x16x64	1-mar-14	AB
4096	Lindgren	4.6×10 ⁻⁴	2048 ³	magn/noentro	4x16x64	26-mar-13	AB
4096	Sisu	6.7×10 ⁻⁴	1024×2048×1024	spherical conv/magn		22-aug-13	PJK
4096	Dardel	1.06 ns	1024 ³	magn/noentro/CME	16x16x16	24-sep-22	AB
4608	Triolith	7.4×10 ⁻⁴	576 ³	magn/noentro	8x18x32	1-mar-14	AB
4608	Triolith	2.7×10 ⁻⁴	1152 ³	magn/noentro	4x32x36	1-mar-14	AB
4608	Triolith	3.0×10 ⁻⁴	1152 ³	magn/noentro	4x36x32	1-mar-14	AB
4608	Triolith	3.7×10 ⁻⁴	1152 ³	magn/noentro	4x18x64	1-mar-14	AB
4608	Triolith	2.36×10 ⁻⁴	2304 ³	magn/noentro	2x32x72	1-mar-14	AB
4608	Kraken	0.00764	192×768×128	magn/noentro	12x48x8	13-jan-12	WL
4608	Kraken	0.00144	768×3072×512	magn/noentro	12x48x8	18-jan-12	WL
6144	Lindgren	4.2×10 ⁻⁴	1024 ³ × 1536	magn/noentro	4x16x64	21-oct-13	AB
6144	Lindgren	8.9×10 ⁻⁴	256 ²	magn/noentro/sph	2x48x64	6-jan-15	SJ
8192	Hermit	0.00101	1024×2048×1024	spherical conv/magn	8x32x32	22-aug-13	PJK

8192	Sisu	4.1×10^{-4}	1024×2048×1024	spherical conv/magn		22-aug-13	PJK
8192	Triolith	1.48×10^{-4}	2048 ³	magn/noentro	4x32x64	1-mar-14	AB
9216	Kraken	0.00485	192×768×256	magn/noentro	24x48x8	13-jan-12	WL
9216	Kraken	0.00158	768×1536×256	magn/noentro	24x48x8	17-jan-12	WL
9216	Kraken	8.0×10^{-4}	1536×3072×512	magn/noentro	24x48x8	18-jan-12	WL
9216	Lindgren	2.36×10^{-4}	2304 ³	magn/noentro	4x48x48	15-feb-14	AB
9216	Triolith	1.04×10^{-3}	576 ³	magn/noentro	16x18x32	1-mar-14	AB
9216	Triolith	1.28×10^{-4}	2304 ³	magn/noentro	4x36x64	1-mar-14	AB
9216	Triolith	1.30×10^{-4}	2304 ³	magn/noentro	4x32x72	1-mar-14	AB
16384	Hermit	6.4×10^{-4}	1024×2048×1024	spherical conv/magn	16x32x32	22-aug-13	PJK
16384	Dardel	1.37×10^{-4}	16384×16384	2D-MHD-6th	128x128	6-feb-24	AB
16384	Dardel	2.05×10^{-4}	16384×16384	2D-MHD-10th	128x128	6-feb-24	AB
18432	Kraken	0.00316	384×768×256	magn/noentro	24x48x16	13-jan-12	WL
18432	Kraken	8.8×10^{-4}	768×1536×512	magn/noentro	24x48x16	17-jan-12	WL
18432	Kraken	4.0×10^{-4}	1536×3072×1024	magn/noentro	24x48x16	18-jan-12	WL
32768	Dardel	171 ps	1024 ³	magn/noentro/CME	32x32x32	24-sep-22	AB
36864	Kraken	0.0020	384×768×512	magn/noentro	48 ² x16	14-jan-12	WL
36864	Kraken	4.9×10^{-4}	1536 ² ×512	magn/noentro	48 ² x16	17-jan-12	WL
36864	Kraken	2.2×10^{-4}	1536×3072×2048	magn/noentro	24x48x32	18-jan-12	WL
73728	Kraken	0.00121	768 ² ×512	magn/noentro	48 ² x32	19-jan-12	WL
73728	Kraken	2.9×10^{-4}	1536 ² ×1024	magn/noentro	48 ² x32	26-jan-12	WL
73728	Kraken	1.2×10^{-4}	3072 ² ×2048	magn/noentro	48 ² x32	26-jan-12	WL

The machines we have used can be characterized as follows:

Nl3: 500 MHz Pentium III single CPU; RedHat Linux 6.2; 256 MB memory

Nq0: 931 MHz Pentium III single CPU; RedHat Linux 7.3; 0.5 GB memory

Nq[1-4]: 869 MHz Pentium III dual-CPU cluster; RedHat Linux 7.3; 0.77 GB memory per (dual) node

Nq[5-6]: 1.2 GHz Athlon dual-CPU cluster; RedHat Linux 7.3; 1 GB memory per (dual) node

Kabul: 1.9 GHz Athlon dual-CPU cluster; 1 GB memory per (dual) node; 256 kB cache per CPU; Gigabit ethernet; SuSE Linux 8.0; LAM-MPI

Cinnatus: 1.7 GHz Pentium 4 single CPU; 1 GB memory; 256 kB cache per CPU; SuSE Linux 7.3

Horseshoe (fe1, giga, and giga2): consists of different subclusters. The old one (queue name: workq, referred to as fe1) 2.0 GHz Pentium 512 single CPU; 25x 24-port fast ethernet switches with gigabit ethernet uplink; 1 30-port gigabit ethernet switch; 1 GB memory. The next generation has gigabit switches directly between nodes, and 2.6 GHz processors. The third generation (giga2) has 3.2 GHz processors (most of which have 1 GB, some 2 GB), is organized in 2 blocks interconnected with 2 Gb links, with 10 Gb uplinks within each block.

Ukaff: SGI Origin 3000; 400 MHz IP35 CPUs; IRIX 6.5; native MPI

Mhd: EV6 Compaq cluster with 4 CPUs per node; 4 GB memory per node (i. e. 1 GB per CPU) OSF1 4.0; native MPI

Sander and Rasmussen: Origin 3000

Steno 118 node IBM cluster with dual node AMD Opteron processors with 10 Gb infiniband network, compiled with `pgf90 -fastsse -tp k8-64e` (Copenhagen).

Gridur: Origin 3000

Luci: (full name Lucidor) is an HP Itanium cluster, each of the 90 nodes has two 900 MHz Itanium 2 "McKinley" processors and 6 GB of main memory. The interconnect is myrinet.

Lenn: (full name Lenngren) is a Dell Xeon cluster with 442 nodes. Each node has two 3.4GHz "Nocona" Xeon processors and 8GB of main memory. A high performance Infiniband network from Mellanox is used for MPI traffic.

Kraken: Cray Linux Environment (CLE) 3.1, with a peak performance of 1.17 PetaFLOP; the cluster has 112,896 cores, 147 TB of memory, in 9,408 nodes. Each node has two 2.6 GHz six-core AMD Opteron processors (Istanbul), 12 cores, and 16 GB of memory. Connection via Cray SeaStar2+ router.

Hermit: Cray XE6 with 7104 2.3 GHz AMD Interlagos 16 core processors (113,664 cores in total), nodes with either 1 or 2 GB of memory per core.

Sisu: Cray XC30 with 1472 2.6 GHz Intel (Xeon) Sandy Bridge 8 core (E5-2670) processors (11,776 cores in total), 2 GB of memory per core.

Beskow: Cray XC40 with 2.3 GHz Intel (Xeon) Haswell 16 core (E5-2698v3) processors (67,456 cores in total), 2 GB of memory per core. Theoretical peak performance 2.43 pflops.

Table 8 shows a similar list, but for a few well-defined sample problems. The `svn` check-in patterns are displayed graphically in Figs. 1 and 2.

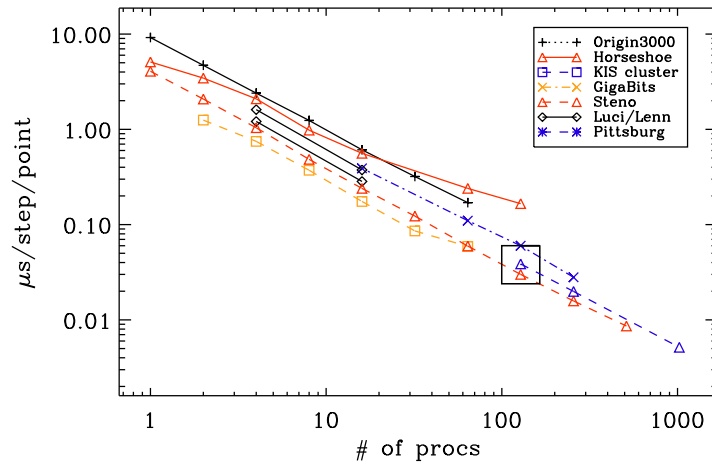


Figure 16: Scaling results on three different machines. The thin straight line denotes perfectly linear scaling.

A.1 Test case

In the following test samples, we run isothermal magnetohydrodynamics in a periodic domain²⁰. Power spectra are computed during the run, but our current parallelization of the Fourier transform requires that the meshpoint number is an integer multiple of

²⁰Run directories are available on <http://norlxs51.nordita.org/~brandenb/pencil-code/timings/bforced/>

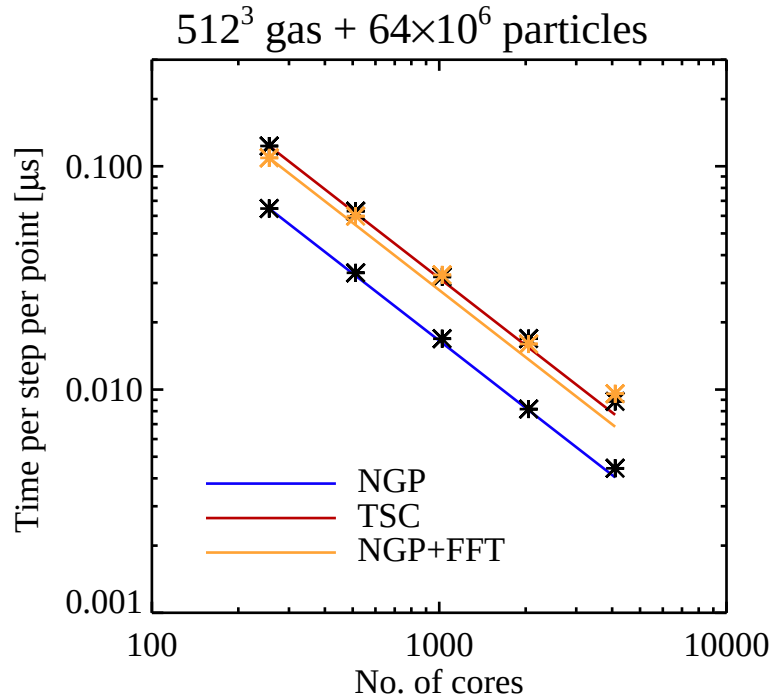


Figure 17: Scaling results of particle-mesh problem on Blue Gene/P on up to 4096 cores. The different lines denote different particle-mesh schemes (NGP=Nearest Grid Point, TSC=Triangular Shaped Cloud) and whether self-gravity is included (FFT).

the product of processor numbers in the y and z directions and the product of processor numbers in the x and y directions. In addition, the number of processors in one direction should not be so large that the number of mesh points per processor becomes comparable to or less than the number of ghost zones (which is 6).

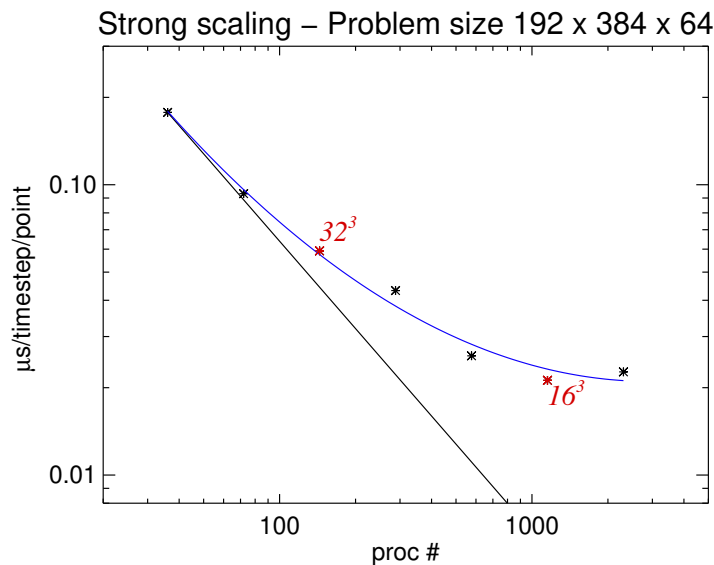


Figure 18: Scaling results on Kraken at fixed problem size, for a magnetized disk model in cylindrical coordinates. The black line shows ideal scaling from 32 cores. The blue line is the best second-order fit to the data points. A load of 16³ mesh points per processor marks the best strong scaling.

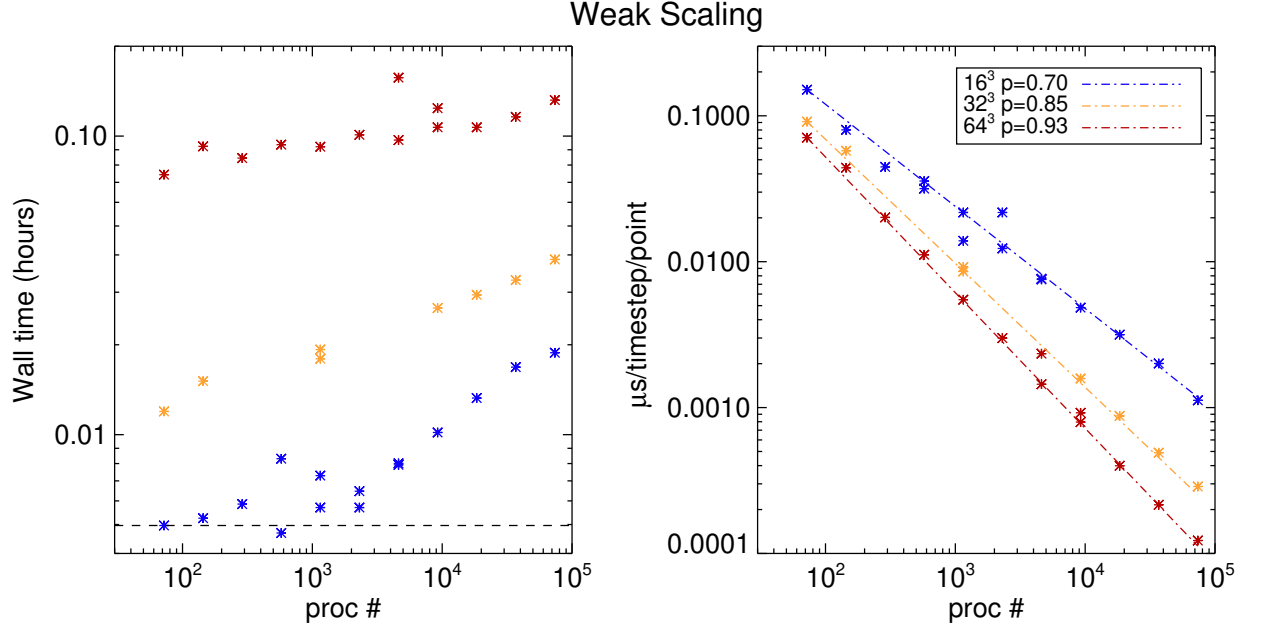


Figure 19: Scaling results on Kraken at fixed load per processor, for a magnetized disk model in cylindrical coordinates. The figure shows, after determining that 16^3 is the best load per processor for strong scaling, how far one can push with weak scaling. The scaling index is found to be 0.7 for 16^3 and 0.93 for 64^3 , up to 73 728 processors.

A.2 Running the code

To run the code, get one of the sample run directories, e.g., http://norlxs51.nordita.org/~brandenb/pencil-code/timings/bforced/512_4x16x32. The relevant file to be changed is `src/cparam.local`

```
ncpus=2048,nprocx=4,nprocy=16,nprocz=ncpus/(nprocx*nprocy)
nxgrid=512,nygrid=nxgrid,nzgrid=nxgrid
```

in particular the values of `ncpus`, `nprocx`, `nprocy`, and `nxgrid`. Once they are chosen, say `make`, and submit `start.run.csh`.

Table 8: Like previous table, but for the versions from the ‘samples’ directory.

proc(s)	machine	$\frac{\mu s}{pt \ step}$	resol.	mem./proc	when	who
<i>conv-slab</i>						
1	Mhd	6.45	32^3	4 MB	23-jul-02	wd
1	Cincinnatus	4.82	32^3	3 MB	23-jul-02	wd
1	Cincinnatus	11.6	64^3	14 MB	23-jul-02	wd
1	Cincinnatus	20.8	128^3	93 MB	23-jul-02	wd
1	Kabul	3.91	32^3		23-jul-02	wd
1	Kabul	3.88	64^3		23-jul-02	wd
1	Kabul	4.16	128^3	93 MB	23-jul-02	wd
<i>conv-slab-flat</i>						
1	Kabul	3.02	$128^2 \times 32$	29 MB	23-jul-02	wd
2	Kabul	1.81	$128^2 \times 32$	18 MB	23-jul-02	wd
4	Kabul	1.03	$128^2 \times 32$	11 MB	23-jul-02	wd
8	Kabul	0.87	$128^2 \times 32$	9 MB	23-jul-02	wd

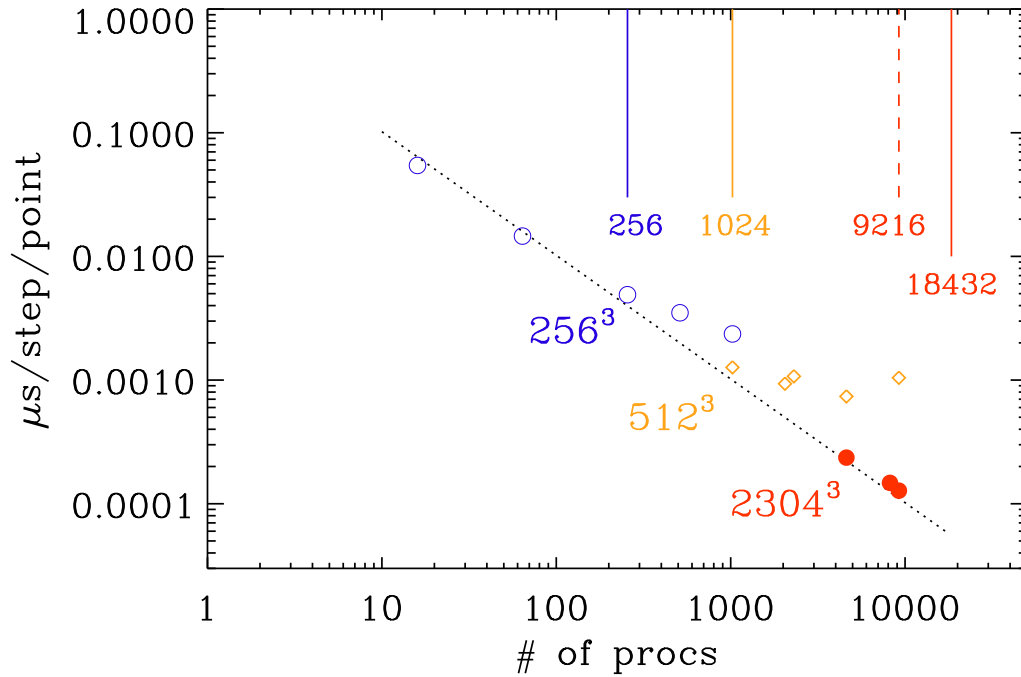


Figure 20: Strong scaling on Triolith (2014).

A.3 Triolith

On Triolith, strong scaling tests have been performed for three mesh sizes. The time per time step and mesh point is given for different processor numbers and layouts. Generally, it is advantageous to keep the number of processors in the x direction small.

Comments. Although on Triolith the number of processors per node is 16, resolutions with one or two powers of 3 (such as 576) still work well. Furthermore, the number of processors above which the scaling becomes poor increases quadratically with the number of mesh points. This implies that the RAM per processor increases linearly with the problem size per direction. However, this will not be a limitation, because even for 2304^3 meshes, the required RAM is still below 100 MB.

In Stockholm there is now a new machine called Dardel. Strong scaling tests have been performed for five mesh sizes; see Fig. 22. The time per time step and mesh point is given for different processor numbers and layouts. Generally, as Jörn Warnecke pointed out earlier, it is advantageous to minimize the processor surface area, and to keep the number of processors in the x direction small.

Performance-wise, Cray with O2 optimization is equivalent to gnu with O3. While gnu-O3 is able to handle memory or whatever compiler problems much better, it is otherwise not better than Cray-O2, and often some 10–20% slows, but this is within the measurement accuracy. More details can be found on <https://github.com/pencil-code/pencil-code/tree/master/doc/timings>.

A.4 Lindgren

On Lindgren, we have performed weak scaling tests and compare with weak scaling results for Triolith. Triolith is about twice as fast as Lindgren.

Table 9: Triolith timings

proc	$\frac{\mu\text{s}}{\text{pt step}}$	resol.	layout
16	0.075	128^3	2x2x4
16	0.065	128^3	1x4x4
16	0.0544	256^3	1x4x4
64	0.0146	256^3	1x8x8
64	0.0164	256^3	2x4x8
256	0.0049	256^3	1x16x16
512	0.0035	256^3	2x16x16
1024	0.00236	256^3	2x16x32
1024	0.00127	512^3	2x16x32
1024	0.00129	512^3	4x16x16
2048	9.34×10^{-4}	512^3	4x16x32
2304	0.00107	576^3	4x18x32
4096	3.6×10^{-4}	1024^3	4x32x32
4096	3.8×10^{-4}	1024^3	8x16x32
4096	4.2×10^{-4}	1024^3	4x16x64
4608	7.38×10^{-4}	576^3	8x18x32
4608	2.66×10^{-4}	1152^3	4x32x36
4608	3.03×10^{-4}	1152^3	4x36x32
4608	3.12×10^{-4}	1152^3	4x18x64
4608	2.36×10^{-4}	2304^3	2x32x72
8192	1.475×10^{-4}	2048^3	4x32x64
9216	0.00104	576^3	16x18x32
9216	1.276×10^{-4}	2304^3	4x36x64
9216	1.30×10^{-4}	2304^3	4x32x72

Table 10: Lindgren timings

proc	$\frac{\mu\text{s}}{\text{pt step}}$	resol.	layout
1536	0.00171	$512^2 \times 384$	2x32x24
2048	0.00129	$512^2 \times 1024$	1x32x64
2048	0.00129	$1024^2 \times 2048$	1x32x64
4096	4.6×10^{-4}	2048^3	4x16x64
9216	2.36×10^{-4}	2304^3	4x48x48

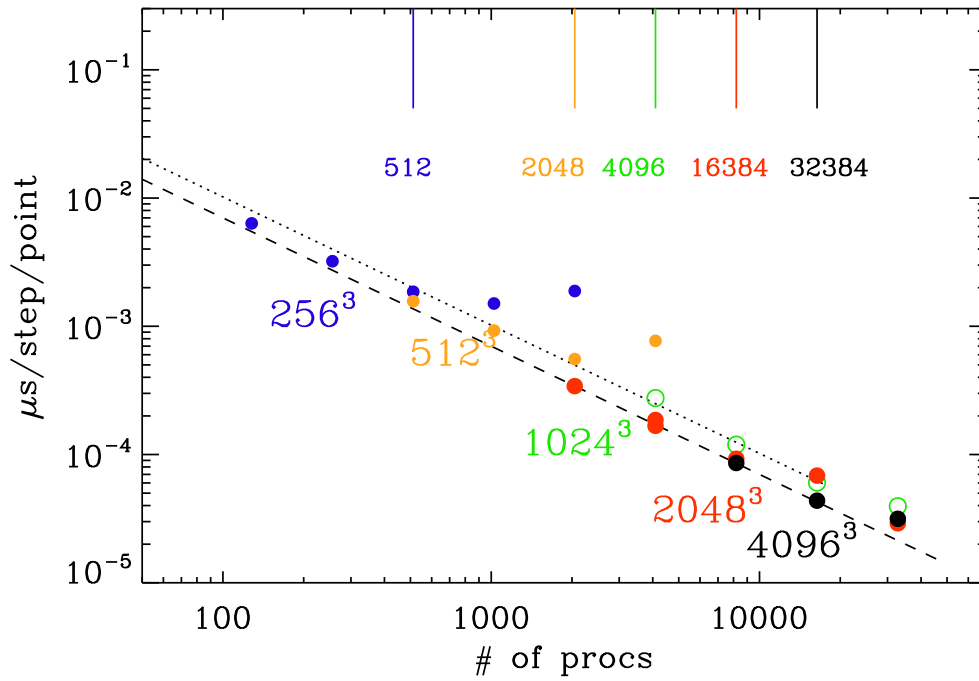


Figure 21: Strong scaling on Dardel. The dotted and dashed lines corresponds to $1.02 \mu\text{s}/\text{proc}/\text{step}/\text{point}$ and $0.70 \mu\text{s}/\text{proc}/\text{step}/\text{point}$, respectively.

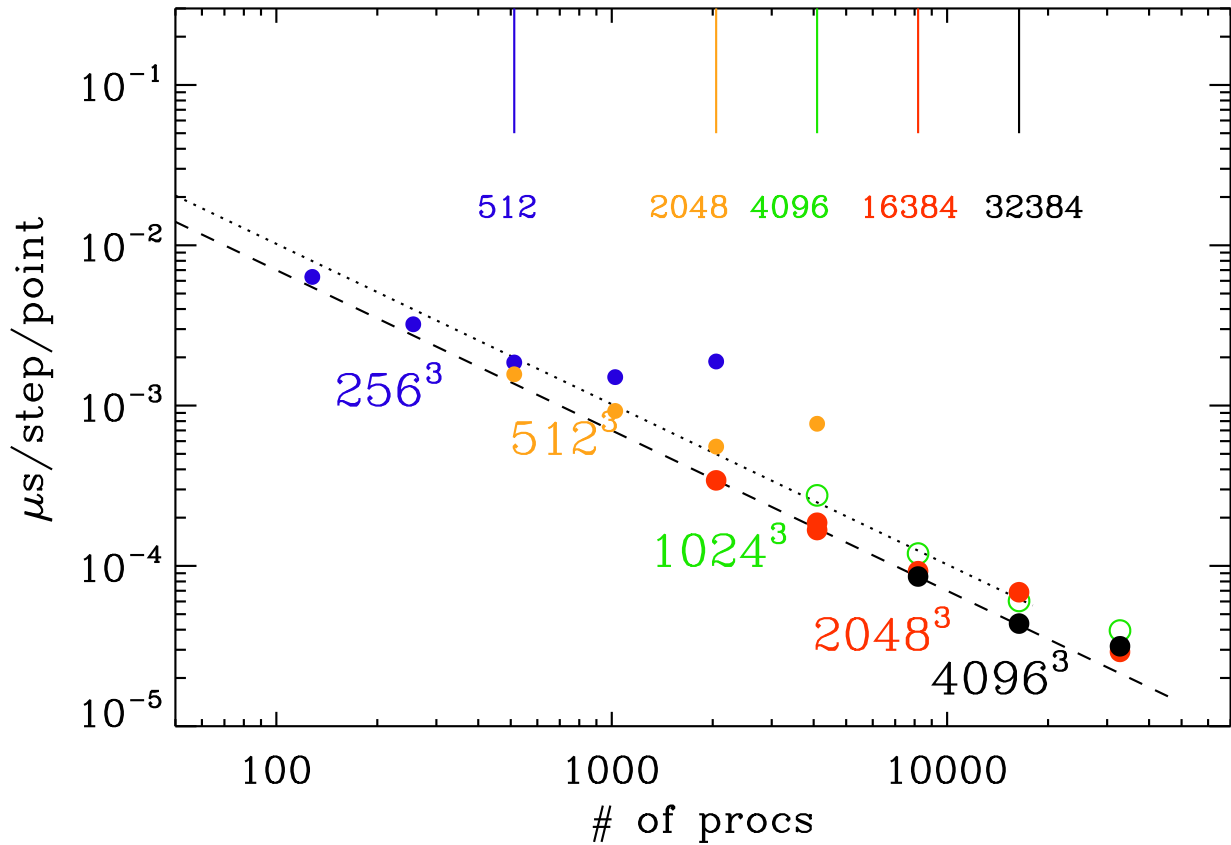


Figure 22: Strong scaling on Dardel. The dotted and dashed lines corresponds to $1.02 \mu\text{s}/\text{proc}/\text{step}/\text{point}$ and $0.70 \mu\text{s}/\text{proc}/\text{step}/\text{point}$, respectively.

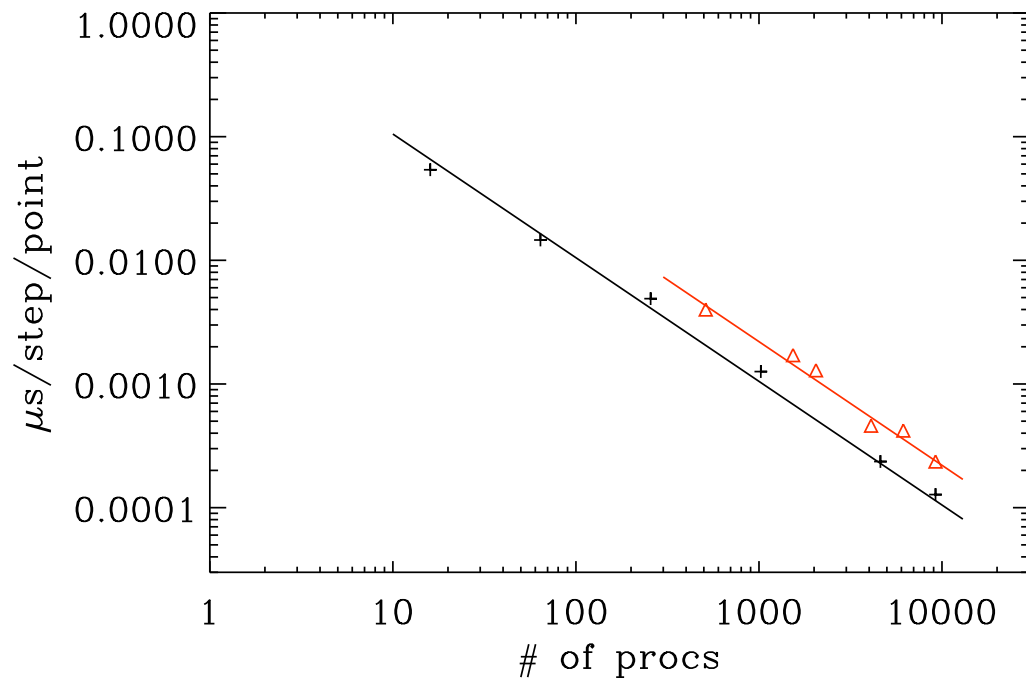


Figure 23: Comparison Triolith (black, plus signs) and Lindgren (red, triangles). Weak scaling (2014).

B Coding standard

The numerous elements that make up the PENCIL CODE are written in a consistent style that has evolved since it was first created. Many people have contributed their knowledge and experience with in this and the result is what we believe is and extremely readable and manageable code.

As well as improving the readability of the code, by having some naming conventions for example aids greatly in understanding what the code does.

There is a standard for all aspects of the code, be it Fortran source, shell scripts, Perl scripts, LaTeX source, Makefiles, or otherwise. Where nothing has been explicitly stated it is recommended that similar existing examples found in the code are used as a template.

B.1 File naming conventions

All files with the exception of the 'Makefile's are given lowercase filenames.

Fortran source files all have the '.f90' extension. Files that contain 'non-executable code' i.e. declarations that are included into other files are given the extension '.h' and those that are generated dynamically at compile time have an '.inc' extension.

Fortran source code defining a module is placed in files whose names begin with the Fortran module name in all lowercase. Where there exist multiple implementations of a specific module, the filenames are extended using an underscore and a brief name relating to what they do.

Text files containing parameters to be read by the code at run time are placed in files with the extension '.in'

B.2 Fortran Code

The code should remain fully compatible with the Fortran90 standard. This ensures that the code will run on all platforms. Indeed, an important aspect of PENCIL CODE philosophy is to be maximally flexible. This also means that useful non-standard extensions to the code should be hidden in and be made accessible through suitable non-default modules.

Fortran is not case-sensitive but in almost all instances we prescribe some form of capitalization for readability.

In general all Fortran code including keywords, variable names etc. are written in lowercase. Some of the coding standard has already been discussed in Sect. 9.1. Here we discuss and amplify some remaining matters.

B.2.1 Indenting and whitespace

Whitespace should be removed from the end of lines.

Blank lines are kept to a minimum, and when occurring in subroutines or functions are replaced by a single '!' in the first column.

Tab characters are not used anywhere in the code. Tab characters are not in fact allowed by the Fortran standard and compilers that accept them do so as an extension.

All lines are kept to be not more than 80 characters long. Where lines are longer they must be explicitly wrapped using the Fortran continuation character '&'. Longer lines (up to 132 characters) and additional spaces are allowed in cases where the readability of the code is enhanced, e.g., when one line is followed by a similar one with minor differences in some places.

Code in syntactic blocks such as `if–endif`, `do–enddo`, `subroutine–endsubroutine` etc. is always indented by precisely two spaces. The exception to this is that nested loops where only the innermost loop contains executable code should be written with the `do–enddo` pairs at the same level of indentation,

```
do n=n1,n2
do m=m1,m2
  [...]
enddo
enddo
```

Alternatively nested loops may be written on a single line, i.e.

```
do n=n1,n2; do m=m1,m2
  [...]
enddo; enddo
```

B.2.2 Comments

Descriptive comments are written on their own lines unless there is a strong reason to do otherwise. Comments are never indented and the '`!`' should appear in the first column followed by two spaces and then the text of the comment. Extremely short comments may follow at the end of a line of code, provided there is space.

Comments also must not exceed the 78 character line length and should be wrapped onto more lines as needed.

Typically comments should appear with a blank commented line above and below the wrapped text of the comment.

All subroutine/functions begin with a standard comment block describing what they do, when and by whom they were created and when and by whom any non-trivial modifications were made.

Comments should be written in sentences using the usual capitalization and punctuation of English, similar to how text is formatted in an e-mail or a journal article.

For example:

```
    some fortran code
    some more fortran code
!
!  A descriptive comment explaining what the following few lines
!  of code do.
!
    the fortran code being described
    the fortran code being described
    ...
!
```

```
! A final detail described here.
!
!     the final fortran code
!     the final fortran code
!     ...
```

Subroutines and functions are started with a comment block describing what they do, when and by whom they were created and when and by whom any non-trivial modifications were made. The layout of this comment block is a standard, for example:

```
!*****
!     subroutine initialize_density(f,lstarting)
!
! Perform any post-parameter-read initialization i.e. calculate derived
! parameters.
!
! For compatibility with other applications, we keep the possibility
! of giving diffrho units of dxmin*cs0, but cs0 is not well defined general.
!
! 24-nov-02/tony: coded
! 1-aug-03/axel: normally, diffrho should be given in absolute units
!
```

where dates are written in dd-mmm-yy format as shown and names appearing after the ‘/’ are either the users cvs login name or, where such exists amongst the PENCIL CODE community, the accepted short form (≈ 4 characters) of the authors name.

B.2.3 Module names

The names of modules are written with initial letter capitalization of each word and the multiple words written consecutively without any separator.

B.2.4 Variable names

Variable are given short but meaningful names and written in all lowercase. Single character names are avoided except for commonly used loop indices and the two code data structures of the PENCIL CODE: ‘f’ the main state array (see 9.4) and ‘p’ the pencil case structure (see 9.7).

Quantities commonly represented by a particular single character in mathematics are typically given names formed by repeating the character (usually in lowercase), e.g., the velocity u becomes ‘uu’, specific entropy s becomes ‘ss’ etc.

Temperature in variable names is denoted with a capital T so as not to be confused with time as represented by a lowercase t. Note however the since Fortran is not case sensitive the variables for example ‘TT’ and ‘tt’ are the same so distinct names must be used. For this reason time is usually represented by a single t contrary to the above guideline.

The natural log of a quantity is represented by using adding ‘ln’ to its name, for example log of temperature would be ‘lnTT’.

There are some standard prefixes used to help identify the type and nature of variables they are as follows:

- `i` – Denotes integer variables typically used as array indices.
- `i_` – Denotes pencil case array indices.
- `idiag_` – Denotes diagnostic indices.
- `l` – Denotes logical/boolean flags
- `cdt` – Denotes timestep constraint parameters.
- `unit_` – Denotes conversion code/physics unit conversion parameters.

B.2.5 Emacs settings

Here are some settings from wd's '~/.emacs' file:

```
;;; ~/.f90.emacs
;;; Set up indentation and similar things for coding the {\sc Pencil Code}.
;;; Most of this can probably be set through Emacs' Customize interface
;;; as well.
;;; To automatically load this file, put the lines
;;; (if (file-readable-p "~/.f90.emacs")
;;;     (load-file "~/.f90.emacs"))
;;; into your ~/.emacs file.

;; F90-mode indentation widths
(setq f90-beginning-ampersand nil) ; no 2nd ampersand at continuation line
(setq f90-do-indent 2)
(setq f90-if-indent 2)
(setq f90-type-indent 2)
(setq f90-continuation-indent 4)

;; Don't use any tabs for indentation (with TAB key).
;; This is actually already set for F90-mode.
(setq-default indent-tabs-mode nil)

;; Ensure Emacs uses F90-mode (and not Fortran-mode) for F90 files:
(setq auto-mode-alist
  (append
    '(
      ("\\. [fF]90$" . f90-mode)
      ("\\. inc$" . f90-mode)
    )
    auto-mode-alist))

;; Make M-Backspace behave in Xemacs as it does in GNU Emacs. The default
;; behavior is apparently a long-known bug the fix for which wasn't
;; propagated from fortran.el to f90.el.
;; (http://list-archive.xemacs.org/xemacs-patches/200109/msg00026.html):
(add-hook 'f90-mode-hook
  (function (lambda ()
    (define-key f90-mode-map [(meta backspace)] 'backward-kill-word)
  )))
```

B.3 Other best practices

When implementing IF or SELECT blocks always write code for all cases – including the default or else case. This should be done even when that code is only a call to raise an error that the case should not have been reached. If you see a missing case anywhere then do add it. These failsafes are essential in a large multi-purpose multi-user code like the PENCIL CODE.

If a case is supposed to do nothing and it may be unclear that the coder has recognized this fact then make it explicit by adding the default case with a comment like
! Do Nothing. The compiler will clean away any such empty blocks.

B.4 General changes to the code

It is sometimes necessary to do major changes to the code. Since this may affect many people and may even be controversial among the developers, such changes are restricted to the time of the next Pencil Code User Meeting. Such meetings are advertised on <http://www.nordita.org/software/pencil-code/> under the news section. Notes about previous such meetings can be found under <http://www.nordita.org/software/pencil-code/UserMeetings/>.

Major changes can affect those developers who have not checked in their latest changes for some time. Before doing such changes it is therefore useful to contact the people who have contributed to the latest developments on that module. If it is not functional or otherwise in bad shape, it should be moved to ‘experimental’, i.e. one says `svn mv file.f90 experimental/file.f90`. However, any such directory change constitutes a major change in itself and should be performed in agreement with those involved in the development. Otherwise any file that has been changed in the mean time will end up being outside revision control, which is to be avoided at all cost.

C Some specific initial conditions

C.1 Random velocity or magnetic fields

Obtained with `inituu='gaussian-noise'` (or `initaa='gaussian-noise'`). The vector u (or A) is set to normally distributed, uncorrelated random numbers in all meshpoints for all three components. The power spectrum of u (A) increases then quadratically with wavenumber k (without cutoff) and the power spectrum of ω (or B) increases like k^4 .

Note that a random initial condition contains significant power at the Nyquist wavenumber $k_{Ny} = \pi/\delta x$, where δx is the mesh spacing. In a decay calculation, because of the discretization error, such power decays slower than it ought to; see Fig. 24, where we show the evolution for a random initial velocity field for 64^3 meshpoints, $\nu = 5 \times 10^{-2}$ (fairly large!), and `nfilter=30`.

It is clearly a good idea to filter the initial condition to prevent excess power at k_{Ny} . On the other hand, such excess power is weak by comparison with the power at the energy carrying scale, so one does not see it in visualizations in real space. Furthermore, as seen from Fig. 24, for $k < k_{Ny}/2$ the power spectra for filtered and unfiltered initial conditions is almost the same.

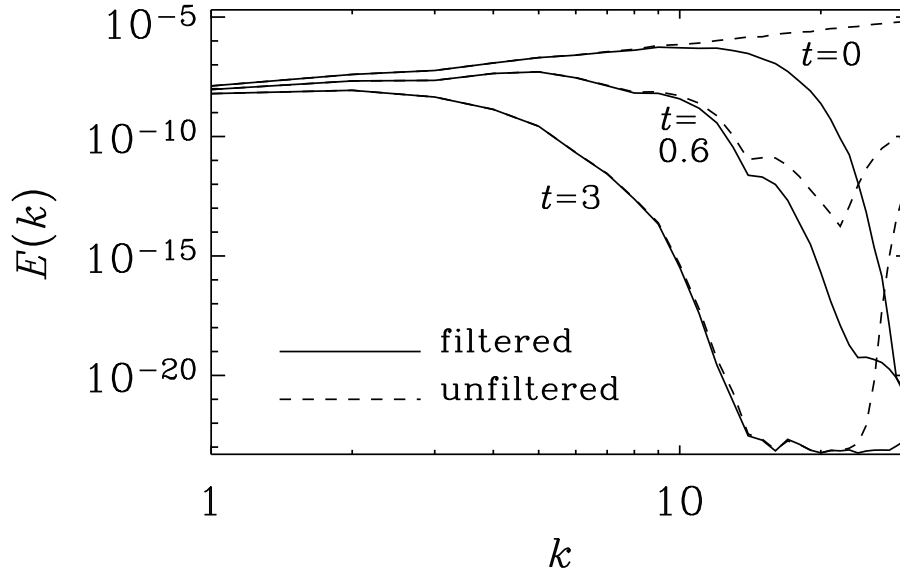


Figure 24: Velocity power spectra at three different times with and without filtering of the initial condition.

C.2 Turbulent initial with given spectrum

The most general procedure for producing an initial condition with a turbulent spectrum is `inituu='power_randomphase_hel'`, which allows one to set two different slopes, together with an exponential cutoff as well as a Gaussian peak within the spectrum. By default, the field is solenoidal unless one puts `lskip_projection=.true.` and can have fractional helicity by setting `relhel_uu` to a value between -1 and 1 . By default it is 0 , which means it is nonhelical.

The spectral indices `initpower` and `initpower2` refer to energy spectral indices. By default, `initpower2=-5/3`, corresponding to a Kolmogorov spectrum. For a delta-correlated

spectrum, we have to put *initpower*=2, corresponding to a k^2 energy spectrum for kinetic energy. This would be suitable for the subinertial range from $k = 1$ to $k = k_p$ (corresponding to the variable *kpeak*).

If *cutoff*=0, no cutoff will be imposed. Otherwise, the spectrum will be multiplied by an exponential function with $\exp(-k^{2n})$, where $n = \text{'ncutoff=1'}$ by default.

Example, for *ampluu*=1e-1, *initpower*=4., and *kpeak*=3., we get *urms*=3.981E-01 when *relhel_uu*=1 and *urms*=3.981E-01 when *relhel_uu*=0 and *urms*=5.560E-01 when *relhel_uu*=1. The *urms* values scale linearly with *ampluu* and for *initpower*=2 also approximately linearly with *kpeak*. For the magnetic field, we initialize the magnetic vector potential, so to get a k^4 spectrum, we have to put *initpower_aa*=2. Everything else is analogous; see, e.g.,

```
&hydro_init_pars
  inituu='power_randomphase_hel', ampluu=1e-1, initpower=4., kpeak=3.
  relhel_uu=0., cutoff=30.
/
&magnetic_init_pars
  initaa='power_randomphase_hel', amplaa=1e-1, initpower_aa=2., kpeak_aa=3.
  relhel_aa=0., cutoff_aa=30.
/
```

for which we get *urms*=3.981E-01 and *brms*=3.871E-01.

C.3 Beltrami fields

Obtained with *inituu*='Beltrami-z' or *initaa*='Beltrami-z'.

$$\mathbf{A} = (\cos z, \sin z, 0), \quad \text{or} \quad \mathbf{u} = (\cos z, \sin z, 0) \quad (140)$$

C.4 Magnetic flux rings: *initaa*='fluxrings'

This initial condition sets up two interlocked thin magnetic tori (i. e. thin, torus-shaped magnetic flux tubes). One torus of radius R lying in the plane $z = 0$ can be described in cylindrical coordinates, (r, φ, z) , by the vector potential

$$\mathbf{A} = \Phi_m \begin{pmatrix} 0 \\ 0 \\ -\theta(r-R)\delta(z) \end{pmatrix}, \quad (141)$$

resulting in a magnetic field

$$\mathbf{B} = \Phi_m \begin{pmatrix} 0 \\ \delta(r-R)\delta(z) \\ 0 \end{pmatrix}. \quad (142)$$

Here Φ_m is the magnetic flux through the tube, $\theta(x)$ denotes the Heaviside function, and

$$\delta(x) = \theta'(x) \quad (143)$$

is Dirac's delta function.

Any smoothed versions of $\theta(x)$ and $\delta(x)$ will do, as long as the consistency condition (143) is satisfied. E. g. the pairs

$$\delta_\varepsilon(x) = \frac{1}{\sqrt{2\pi\varepsilon^2}} e^{-\frac{x^2}{2\varepsilon^2}}, \quad \theta_\varepsilon(x) = \frac{1}{2} \left(1 + \operatorname{erf} \frac{x}{\sqrt{2}\varepsilon} \right) \quad (144)$$

or

$$\delta_\varepsilon(x) = \frac{1}{2\varepsilon} \frac{1}{\cosh^2 \frac{x}{\varepsilon}}, \quad \theta_\varepsilon(x) = \frac{1}{2} \left(1 + \tanh \frac{x}{\varepsilon} \right) \quad (145)$$

are quite popular. Another possibility is a constant or box-like profile with

$$\delta_\varepsilon(x) = \frac{1}{2\varepsilon} \theta(|x| - \varepsilon), \quad \theta_\varepsilon(x) = \frac{1}{2} \{1 + \max[-1, \min(x/\varepsilon, 1)]\} \quad (146)$$

Note, however, that the Gaussian profile (144) is the only one that yields a radially symmetric (with respect to the distance from the central line of the torus) magnetic field profile $B_\varphi = B_\phi(\sqrt{(r-R)^2 + z^2})$ if ε is sufficiently small.

In Cartesian coordinates, the vector potential (141) takes the form

$$\mathbf{A} = \Phi_m \begin{pmatrix} 0 \\ 0 \\ -\theta \left(\sqrt{x^2 + y^2} - R \right) \delta(z) \end{pmatrix}. \quad (147)$$

C.5 Vertical stratification

Gravity, $\mathbf{g} = -\nabla\Phi$, is specified in terms of a potential Φ . In slab geometry, $\Phi = \Phi(z)$, we have $\mathbf{g} = (0, 0, g_z)$ and $g_z = -d\Phi/dz$.

Use `gravz_profile='const'` together with `gravz = -1` to get

$$\Phi = (z - z_\infty)(-g_z), \quad (-g_z) > 0. \quad (148)$$

Use `gravz_profile='linear'` to get

$$\Phi = \frac{1}{2}(z^2 - z_\infty^2)\nu_g^2, \quad g_z = -\nu_g^2 z \quad (149)$$

where ν_g is the vertical epicyclic frequency. For a Keplerian accretion disc, $\nu_g = \Omega$. For galactic discs, $\nu_g = 0.5\Omega$ is representative of the solar neighborhood.

The value of z_∞ is determined such that $\rho = \rho_0$ and $c_s^2 = c_{s0}^2$ at $z = z_{\text{ref}}$. This depends on the values of γ and the polytropic index m (see below).

C.5.1 Isothermal atmosphere

Here we want $c_s = c_{s0} = \text{const}$. Using `initlnrho='isothermal'` means

$$\ln \frac{\rho}{\rho_0} = -\gamma \frac{\Phi}{c_{s0}^2}. \quad (150)$$

The entropy is then initialized to

$$\frac{s}{c_p} = (\gamma - 1) \frac{\Phi}{c_{s0}^2}. \quad (151)$$

In order that $\rho = \rho_0$ and $c_s^2 = c_{s0}^2$ at $z = z_{\text{ref}}$, we have to choose $z_\infty = z_{\text{ref}}$.

C.5.2 Polytropic atmosphere

For a polytropic equation of state, $p = K\rho^\Gamma$, where generally $\Gamma \neq \gamma$, we can write

$$-\nabla h + T\nabla s = -\frac{1}{\rho}\nabla p = -\nabla \left(\frac{\Gamma K}{\Gamma-1} \rho^{\Gamma-1} \right) \equiv -\nabla \tilde{h}, \quad (152)$$

where we have introduced a pseudo enthalpy $\tilde{h}$ as

$$\tilde{h} = \frac{\Gamma K}{\Gamma-1} \rho^{\Gamma-1} = \left[\left(1 - \frac{1}{\gamma}\right) / \left(1 - \frac{1}{\Gamma}\right) \right] h. \quad (153)$$

Obviously, for $\Gamma = \gamma$, the pseudo enthalpy $\tilde{h}$ is identical to h itself. Instead of specifying Γ , one usually defines the polytropic index $m = 1/(\Gamma-1)$. Thus, $\Gamma = 1 + 1/m$, and

$$\tilde{h} = (m+1) \left(1 - \frac{1}{\gamma}\right) h \quad (154)$$

This is consistent with a fixed entropy dependence, where s only depends on ρ like

$$\frac{s}{c_p} = \left(\frac{\Gamma}{\gamma} - 1 \right) \ln \frac{\rho}{\rho_0}, \quad (155)$$

and implies that

$$\ln \frac{c_s^2}{c_{s0}^2} = (\Gamma-1) \ln \frac{\rho}{\rho_0}. \quad (156)$$

For hydrostatic equilibrium we require $\tilde{h} + \Phi = \tilde{h}_0 = \text{const}$. For gravity potentials that vanish at infinity, we can have $\tilde{h}_0 \neq 0$, i.e. a finite pseudo enthalpy at infinity. For $g_z = -1$ or $g_z = -z$, this is not the case, so we put $\tilde{h}_0 = 0$, and therefore $\tilde{h} = -\Phi$. Using $c_s^2 = (\gamma-1)h$ together with (154) we find

$$c_s^2 = -\frac{\gamma}{m+1} \Phi. \quad (157)$$

In order that $\rho = \rho_0$ and $c_s^2 = c_{s0}^2$ at $z = z_{\text{ref}}$, we have to choose (remember that g_z is normally negative!)

$$z_\infty = z_{\text{ref}} + (m+1) \frac{c_{s0}^2}{\gamma(-g_z)} \quad \text{for gravz_profile='const'}, \quad (158)$$

and

$$z_\infty^2 = z_{\text{ref}}^2 + (m+1) \frac{c_{s0}^2}{\frac{1}{2}\gamma\nu_g^2} \quad \text{for gravz_profile='linear'}. \quad (159)$$

Thus, when using `initlnrho='polytropic_simple'` we calculate

$$\ln \frac{c_s^2}{c_{s0}^2} = \ln \left[-\frac{\gamma\Phi}{(m+1)c_{s0}^2} \right] \quad (160)$$

and so the stratification is given by

$$\ln \frac{\rho}{\rho_0} = m \ln \frac{c_s^2}{c_{s0}^2}, \quad \frac{s}{c_p} = \left(\frac{\Gamma}{\gamma} - 1 \right) m \ln \frac{c_s^2}{c_{s0}^2}. \quad (161)$$

C.5.3 Changing the stratification

Natural: measure length in units of c_{s0}^2/g_z . Can increase stratification by moving z_{top} close to z_∞ or, better still, keeping $z_{\text{top}} = 0$ and moving $z_{\text{bot}} \rightarrow -\infty$. Disadvantage: in the limit of weak stratification, the box size will be very small (in nondimensional units).

Box units: measure length in units of d . Can increase stratification by increasing g_z to g_{max} , which can be obtained by putting $z_{\text{top}} = z_\infty$ in (158), so

$$g_{\text{max}} = \frac{m+1}{\gamma} \frac{c_{s0}^2}{z_{\text{top}} - z_{\text{ref}}}. \quad (162)$$

For $m = 1$, $\gamma = 5/3$, $z_{\text{top}} = 1$, and $z_{\text{ref}} = 0$, for example, we have $g_{\text{max}} = 6/5 = 1.2$.

Gravitational box units: measure speed in units of $\sqrt{g_z d}$. The limit of vanishing stratification corresponds to $c_{s0} \rightarrow \infty$. This seems optimal if we want to approach the Boussinesq case.

In Hurlburt et al. (1984), z increased downward and the atmosphere always terminated at $z = 0$. In order to reproduce their case most directly, we put $z_\infty = 0$ and consider only negative values of z . To reproduce their case with a density stratification of 1:1.5, we place the top of the model at $z = -2$ and the bottom at $z = -3$. In addition, the reference height, z_{ref} , is chosen to be at the top of the box, i.e. $z_{\text{ref}} = -2$. From Eq. (158) we have $c_{s0}^2 = \gamma(-g_z)(-z_{\text{ref}})/(m+1)$. Using $(-g_z) = 1$ and $m = 1$ we find $c_{s0}^2 = \gamma$, so $c_{s0} = 1.291$ (for $\gamma = 5/3$). Values for other combinations are listed in Table 11.

Table 11: Correspondence between density contrast, top and bottom values of z , and c_{s0} for $(-g_z) = 1$, $m = 1$, and $\gamma = 5/3$.

$\rho_{\text{bot}}/\rho_{\text{top}}$	z_{bot}	z_{top}	c_{s0}
1.5	3	2	1.291
3	1.5	0.5	0.645
6	1.2	0.2	0.408
11	1.1	0.1	0.289
21	1.05	0.05	0.204

C.5.4 The Rayleigh number

In Ref. [15] the Rayleigh number is defined as

$$\text{Ra} = \frac{gd^4}{\bar{\nu} \bar{\chi}} \left(-\frac{ds/c_p}{dz} \right)_{\text{hydrostat}}, \quad (163)$$

where the (negative) entropy gradient was evaluated in the middle of the box for the associated hydrostatic reference solution, and $\bar{\chi} = K/(\bar{\rho}c_p)$ and either $\bar{\nu} = \nu$ (if ν was assumed constant) or $\bar{\nu} = \mu/\bar{\rho}$ (if μ was assumed constant). Note that $\bar{\rho}$ is the average mass in the box per volume, which is conserved. For a polytrope we have

$$\left(-\frac{ds/c_p}{dz} \right)_{\text{hydrostat}} = \left[1 - (m+1) \left(1 - \frac{1}{\gamma} \right) \right] \frac{1}{z_\infty - z_m}, \quad (164)$$

where $z_m = (z_1 + z_2)/2$. This factor was also present in the definition of Hurlburt et al. [28], but their definition differs slightly from Eq. (163), because they normalized the

density not with respect to the average value (which is constant for all times), but with respect to the value at the top of the initial hydrostatic solution. Since the Rayleigh number is proportional to ρ^2 , their definition included the extra factor $[(z_\infty - z_m)/d]^2$. Therefore

$$\text{Ra}_{\text{HTM}} = \left(\frac{z_\infty - z_m}{d} \right)^{2m} \left(\frac{\rho_{\text{top}}}{\bar{\rho}} \right)^2 \text{Ra} \quad (165)$$

In the first model of Hurlburt et al. (1984), the Rayleigh number, Ra_{HTM} , was chosen to be 310 times supercritical, and the critical Rayleigh number was around 400, so $\text{Ra}_{\text{HTM}} = 1.25 \times 10^5$. In their model the density contrast was 1:1.5 and $m = 1$. This turns out to correspond to $\text{Ra} = 4.9 \times 10^4$, $F_{\text{bot}} = 0.0025$, and $K = 0.002$.

Another model that was considered by Hurlburt & Toomre (1988) had $\text{Ra}_{\text{HTM}} = 10^5$, a density contrast of 11, and had a vertical imposed magnetic field (Chandrasekhar number $Q = 72$). This corresponds to $\text{Ra} = 3.6 \times 10^8$, $K = 0.0011$, $F_{\text{bot}} = 0.0014$.

C.5.5 Entropy boundary condition

This discussion only applies to the case of convection in a slab. A commonly used lower boundary condition is to prescribe the radiative flux at the bottom, i.e. $F_{\text{bot}} = -K dT/dz$. Assuming that the density in the ghost zones has already been updated, we can calculate the entropy gradient from

$$F_{\text{bot}} = -\frac{K}{c_p} \frac{c_s^2}{\gamma - 1} \left((\gamma - 1) \frac{d \ln \rho}{dz} + \gamma \frac{ds/c_p}{dz} \right), \quad (166)$$

which gives

$$\frac{ds/c_p}{dz} = -\frac{\gamma - 1}{\gamma} \left(c_p \frac{F_{\text{bot}}}{K c_s^2} + \frac{d \ln \rho}{dz} \right) \quad (167)$$

for the derivative of the entropy at the bottom. This is implemented as the ‘c1’ boundary condition at the bottom.

C.5.6 Temperature boundary condition at the top

In earlier papers the temperature at the top was set in terms of the quantity ξ_0 , which is the ratio of the pressure scale height relative to the depth of the unstable layer. Expressed in terms of the sound speed at the top we have

$$c_{s,\text{top}}^2 = \gamma \xi_0 g d. \quad (168)$$

$$c_{s,\text{bot}}^2 = \left(\xi_0 + \frac{1}{m+1} \right) \gamma g d. \quad (169)$$

Table 12: Correspondence between ξ_0 and $c_{s,\text{bot}}^2$ in single layer polytropes.

ξ_0	$c_{s,\text{bot}}^2$
10.00	17.500
0.20	1.167
0.10	1.000
0.05	0.917
0.02	0.867

C.6 Potential-field boundary condition

The ‘pot’ [or currently rather the ‘pwd’] boundary condition for the magnetic vector potential implements a *potential-field boundary condition* in z for the case of an x - y -periodic box. In this section, we discuss the relevant formulas and their implementation in the PENCIL CODE.

If the top boundary is at $z = 0$, the relevant potential field for $z > 0$ is given by

$$\tilde{A}_x(k_x, k_y, z) = C_x(\mathbf{k}_{xy}) e^{-\kappa z}, \quad (170)$$

$$\tilde{A}_y(k_x, k_y, z) = C_y(\mathbf{k}_{xy}) e^{-\kappa z}, \quad (171)$$

$$\tilde{A}_z(k_x, k_y, z) = C_z(\mathbf{k}_{xy}) e^{-\kappa z}, \quad (172)$$

where

$$\tilde{A}_i(k_x, k_y, z) \equiv \int e^{-i\mathbf{k}_{xy} \cdot \mathbf{x}} A_i(x, y, z) dx dy \quad (173)$$

is the horizontal Fourier transform with $\mathbf{k}_{xy} \equiv (k_x, k_y, 0)$, and $k \equiv |\mathbf{k}_{xy}|$. Note that this implies a certain gauge and generally speaking the z dependence in Eq. (172) is completely arbitrary, but the form used here works well in terms of numerical stability.

At the very boundary, the potential field (170)–(172) implies

$$\frac{\partial \tilde{\mathbf{A}}}{\partial z} + \kappa \tilde{\mathbf{A}} = 0, \quad (174)$$

and, due to natural continuity requirements on the vector potential, these conditions also hold for the interior field at the boundary.

Robin boundary conditions and ghost points To implement a homogeneous Robin boundary condition, i. e. a condition of the form

$$\frac{df}{dz} + \kappa f = 0 \quad (175)$$

using ghost points, we first write it as

$$\frac{d}{dz} (f e^{\kappa z}) = 0 \quad (176)$$

and implement this as symmetry condition for the variable $\phi(z) \equiv f(z) e^{\kappa z}$:

$$\phi_{N-j} = \phi_{N+j}, \quad j = 1, 2, 3 \quad (177)$$

(where z_N is the position of the top boundary and $z_{N+1}, \dots$ are the boundary points). In terms of f , this becomes

$$f_{N+j} = f_{N-j} e^{-\kappa(z_{N+j} - z_{N-j})}. \quad (178)$$

Note that although the exponential term in Eq. (178) looks very much like the exterior potential field (170)–(172), our ghost-zone values do *not* represent the exterior field – they are rather made-up values that allow us to implement a local boundary condition at $z = 0$.

C.7 Planet solution in the shearing box

In order to test the setup for accretion discs and the sliding periodic shearing sheet boundary condition, a useful initial condition is the so-called planet solution of Goodman, Narayan, & Goldreich [24].

Assume $s = 0$ (isentropy), so the equations in 2-D are

$$u_x u_{x,x} + (u_y^{(0)} + u_y) u_{x,y} = 2\Omega u_y - h_{,x} \quad (179)$$

$$u_x u_{y,x} + (u_y^{(0)} + u_y) u_{y,y} = -(2 - q)\Omega u_x - h_{,y} \quad (180)$$

where $u_y^{(0)} = -q\Omega x$. Express $\mathbf{u}$ in terms of a stream function, so $\mathbf{u} = \nabla \times (\psi \hat{\mathbf{z}})$, or

$$u_x = \psi_{,y}, \quad u_y = -\psi_{,x}. \quad (181)$$

Ansatz for enthalpy

$$h = \frac{1}{2}\delta^2\Omega^2(R^2 - x^2 - \epsilon^2 y^2 - z^2/\delta^2) \quad (182)$$

$$\psi = -\frac{1}{2}\sigma\Omega(R^2 - x^2 - \epsilon^2 y^2) - \frac{1}{2}q\Omega x^2 \quad (183)$$

This implies

$$u_x = \sigma\Omega\epsilon^2 y, \quad u_y = (q - \sigma)\Omega x \quad (184)$$

and $u_{x,x} = u_{y,y} = 0$. Inserting into Eqs (179) and (180) yields

$$(-q + q - \sigma)\sigma\epsilon^2 = 2(q - \sigma) + \delta^2 \quad (185)$$

$$\sigma(q - \sigma) = -(2 - q)\sigma + \delta^2 \quad (186)$$

where we have already canceled common Ω^2 factors in both equations and common ϵ^2 factors in the last equation. Simplifying both equations yields

$$-\sigma^2\epsilon^2 = 2(q - \sigma) + \delta^2 \quad (187)$$

$$-\sigma^2 = -2\sigma + \delta^2 \quad (188)$$

The second equation yields

$$\delta^2 = (2 - \sigma)\sigma \quad (189)$$

and subtracting the two yields

$$\sigma^2 = 2q/(1 - \epsilon^2) \quad (190)$$

Table 13: Dependence of ϵ and δ on ϵ .

ϵ	σ	δ
0.1	1.74	0.67
0.2	1.77	0.64
0.3	1.82	0.58
0.4	1.89	0.46
0.48	1.97	0.22
0.5	2	0

D Some specific boundary conditions

In this section, we formulate and discuss the implementation of some common boundary conditions in spherical and cylindrical coordinates.

D.1 Perfect-conductor boundary condition

This is a popular boundary condition for the magnetic field; it implies that

$$B_n = 0 \quad (191)$$

and

$$\mathbf{E}_t = 0 \quad (192)$$

on the boundary, where the subscript n denotes the normal component, and $\mathbf{E}_t$ denotes the tangential components of the electric field.

In Cartesian geometry, and when the boundary is impenetrable, these conditions can be implemented by employing the 'a' condition for the two tangential components of the vector potential $\mathbf{A}$, which forces both their values and their second normal derivatives to be zero on the boundary. In addition the normal derivative of the normal $\mathbf{A}$ component must be zero, hence use 's' for it. It is easy to see that this also works in arbitrary curvilinear coordinates.

In particular, for spherical coordinates on a radial boundary we must have

$$r \sin \theta B_r = \partial_\theta (\sin \theta A_\phi) - \partial_\phi A_\theta = 0 . \quad (193)$$

This can be achieved by setting

$$A_\phi = A_\theta = 0 \quad (194)$$

everywhere on the boundary. Note that this does not impose any condition on the radial component of the vector potential.

Next, in spherical coordinates on a boundary with constant θ , we must have

$$B_\theta = \frac{1}{r \sin \theta} \partial_\phi A_r - \frac{1}{r} \partial_r (r A_\phi) = 0 . \quad (195)$$

Again this can be achieved by $A_r = A_\phi = 0$.

D.2 Stress-free boundary condition

On an impenetrable, stress-free boundary, we have

$$u_n = 0 , \quad (196)$$

and the shear stress components S_{nt} must vanish for any tangential direction t . At the radial boundary, the relevant components of the strain tensor (required to vanish at the boundary) are:

$$S_{r\theta} = \frac{1}{r} \partial_\theta u_r + r \partial_r \left(\frac{u_\theta}{r} \right) , \quad (197)$$

$$S_{r\phi} = \frac{1}{r \sin \theta} \partial_\phi u_r + r \partial_r \left(\frac{u_\phi}{r} \right) . \quad (198)$$

Both of them vanish if we require

$$u_r = 0, \quad \partial_r(u_\theta/r) = 0, \quad \partial_r(u_\phi/r) = 0. \quad (199)$$

We implement this by requiring u_r to be antisymmetric and u_θ/r and u_ϕ/r to be symmetric with respect to the boundary.

The more general condition

$$r^\alpha \partial_r(u_\theta/r^\alpha) = \partial_r u_\theta - \frac{\alpha}{r} u_\theta = 0 \quad (200)$$

(where α is a constant) can be implemented by requiring u_θ/r^α to be symmetric.

At a boundary $\theta = \text{const}$, the stress-free boundary condition will take the form

$$S_{r\theta} = \frac{1}{r} \partial_\theta u_r + r \partial_r \left(\frac{u_\theta}{r} \right) = 0, \quad (201)$$

$$S_{\theta\phi} = \frac{1}{r \sin \theta} \partial_\phi u_\theta + \sin \theta \partial_\theta \left(\frac{u_\phi}{r \sin \theta} \right) = 0. \quad (202)$$

With $u_\theta = 0$, the first condition gives $\partial_\theta u_r = 0$, i.e. we require u_r to be symmetric with respect to the boundary. The second condition requires

$$\frac{\sin \theta}{r} \partial_\theta \left(\frac{u_\phi}{\sin \theta} \right) = 0 \quad (203)$$

and is implemented by requiring $u_\phi/\sin \theta$ to be symmetric.

D.3 Normal-field-radial boundary condition

While unphysical, this boundary condition is often used as a cheap replacement for a potential-field condition for the magnetic field. It implies that the two tangential components of the magnetic field are zero at the boundary, while the normal component is left unconstrained.

At a radial boundary, this gives:

$$B_\theta = \frac{1}{r \sin \theta} \partial_\phi A_r - \frac{1}{r} \partial_r(r A_\phi) = 0, \quad (204)$$

$$B_\phi = \frac{1}{r} \partial_r(r A_\theta) - \frac{1}{r} \partial_\theta A_r = 0. \quad (205)$$

Which are satisfied by setting

$$A_r = 0, \quad \partial_r(r A_\theta) = 0, \quad \partial_r(r A_\phi) = 0, \quad (206)$$

and these are implemented by requiring A_r to be antisymmetric, and $r A_\theta$ and $r A_\phi$ to be symmetric.

On a boundary $\theta = \text{const}$, we have

$$r \sin \theta B_r = \partial_\theta(\sin \theta A_\phi) - \partial_\phi A_\theta = 0, \quad (207)$$

$$r B_\phi = \partial_r(r A_\theta) - \partial_\theta A_r = 0 \quad (208)$$

which can be achieved by setting

$$\partial_\theta A_r = 0, \quad A_\theta = 0, \quad \partial_\theta(\sin \theta A_\phi) = 0. \quad (209)$$

We thus require A_r and $\sin \theta A_\phi$ to be symmetric, and A_θ to be antisymmetric.

E High-frequency filters

Being high order, PENCIL CODE has much reduced numerical dissipation. In order to perform inviscid simulations, high-frequency filters can be used to provide extra dissipation for modes approaching the Nyquist frequency. Usual Laplacian viscosity $\nu \nabla^2 \mathbf{u}$ is equivalent to a multiplication by k^2 in Fourier space, where k is the wavenumber. Another tool is hyperviscosity, which replaces the k^2 dependency by a higher power-law, k^n , $n > 2$. The idea behind it is to provide large dissipation only where it is needed, at the grid scale (high k), while minimizing it at the largest scales of the box (small k). In principle, one can use as high n as desired, but in practice we are limited by the order of the code. A multiplication by k^n is equivalent to an operator ∇^n in real space. As PENCIL CODE is of sixth order, three ghost cells are available in each direction, thus the sixth-order derivative is the highest we can compute. The hyperdissipation we use is therefore ∇^6 , or k^6 in Fourier space. Figure 25 illustrates how such tool maximizes the inertial range of a simulation.

Simplified hyperdiffusivity has been implemented for many dynamical variables and can be found in the respective modules. A strict generalization of viscosity and resistivity to higher order is implemented in the modules ‘hypervisc_strict_2nd’ and ‘hyperresi_strict_2nd’.

Hyperdiffusivity is meant purely as a numerical tool to dissipate energy at small scales and comes with no guarantee that results are convergent with regular second order dissipation. See Haugen & Brandenburg (2004) for a discussion. In fact, large-scale dynamo action is known to be seriously altered in simulations of closed systems where magnetic helicity is conserved: this results in prolonged saturation times and enhanced saturation amplitudes (Brandenburg & Sarson 2002).

E.1 Conservative hyperdissipation

It is desirable to have this high-frequency filter obeying the conservation laws. So, for density we want a mass conserving term, for velocities we want a momentum conserving term, for magnetic fields we want a term conserving magnetic flux, and for entropy we want an energy conserving term. These enter as hyperdiffusion, hyperviscosity, hyperresistivity, and hyper heat conductivity terms in the evolution equations. To ensure conservation under transport, they must take the form of the divergence of the flux $\mathcal{J}$ of the quantity ψ , so that Gauss theorem applies and we have

$$\frac{\partial \psi}{\partial t} + \nabla \cdot \mathcal{J} = 0 \quad (210)$$

For density, the flow due to mass diffusion is usually taken as the phenomenological Fick’s Law

$$\mathcal{J} = -D \nabla \rho \quad (211)$$

i.e., proportional to the density gradient, in the opposite direction. This leads to the usual Laplacian diffusion

$$\frac{\partial \rho}{\partial t} = D \nabla^2 \rho \quad (212)$$

under the assumption that the diffusion coefficient D is isotropic. Higher order hyperdiffusion of order $2n$ involves a generalization of Eq. (211), to

$$\mathcal{J}^{(n)} = (-1)^n D^{(n)} \nabla^{2n-1} \rho. \quad (213)$$

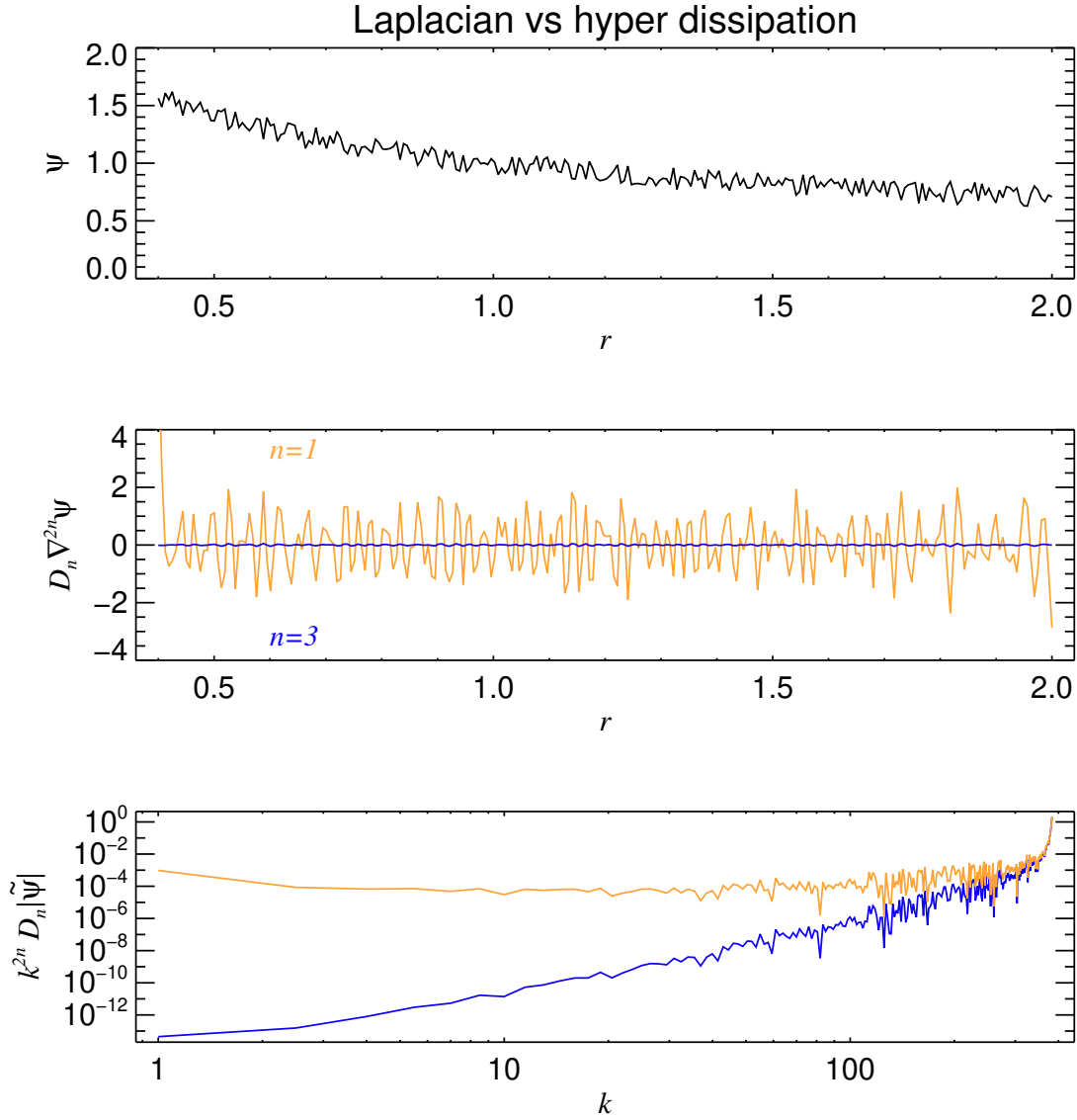


Figure 25: Dissipation acting on a scalar field ψ , for $n=1$ (Laplacian dissipation) and $n=3$ (third-order hyperdissipation). The field is initially seeded with noise (upper panel). For $n=3$ the large scale is not affected as much as in the $n=1$ case, which is seen by the larger wiggling of the latter in the middle panel. In Fourier space (lower panel) we see that near the grid scale both formulations give strong dissipation. It also illustrates that at the large scales ($k \simeq 1$), the effect of $n=3$ is indeed negligible.

In our case, we are interested in the case $n = 3$, so that the hyperdiffusion term is

$$\frac{\partial \rho}{\partial t} = D^{(3)} \nabla^6 \rho. \quad (214)$$

The hyperdiffusion coefficient $D^{(3)}$ can be calculated from D assuming that at the Nyquist frequency the two formulations (212) and (214) yield the same quenching. Considering a wave as a Fourier series in one dimension (x), one element of the series is expressed as

$$\psi_k = A e^{i(kx + \omega t)} \quad (215)$$

Plugging it into the second order diffusion equation (212) we have the dispersion condition $i\omega = -Dk^2$. The sixth order version (214) yields $i\omega = -D^{(3)}k^6$. Equating both we have $D^{(3)} = Dk^{-4}$. This condition should hold at the grid scale, where $k = \pi/\Delta x$,

therefore

$$D^{(3)} = D \left(\frac{\Delta x}{\pi} \right)^4 \quad (216)$$

For the magnetic potential, resistivity has the same formulation as mass diffusion

$$\frac{\partial \mathbf{A}}{\partial t} = -\eta \nabla \times \mathbf{B} = \eta \nabla^2 \mathbf{A}, \quad (217)$$

where we used the Coulomb gauge $\nabla \cdot \mathbf{A} = 0$. The algebra is the same as above, also yielding $\eta^{(3)} = \eta(\Delta x/\pi)^4$. For entropy, the heat conduction term is

$$\frac{\partial S}{\partial t} = \frac{1}{\rho T} \nabla \cdot (K \nabla T), \quad (218)$$

and requiring that K be constant, we substitute it by

$$\frac{\partial S}{\partial t} = \frac{K^{(3)}}{\rho T} \nabla^6 T. \quad (219)$$

also with $K^{(3)} = K(\Delta x/\pi)^4$.

E.2 Hyperviscosity

Viscosity has some caveats where subtleties apply. The difference is that the momentum flux due to viscosity is not proportional to the velocity gradient, but to the rate-of-strain tensor

$$S_{ij} = \frac{1}{2} \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} - \frac{2}{3} \delta_{ij} \nabla \cdot \mathbf{u} \right), \quad (220)$$

which only allows the viscous acceleration to be reduced to the simple formulation $\nu \nabla^2 \mathbf{u}$ under the condition of incompressibility and constant dynamical viscosity $\mu = \nu \rho$. Due to this, the general expression for conservative hyperviscosity involves more terms. In some cases, it is no great overhead, but for others, simpler formulations can be applied.

E.2.1 Conservative case

In the general case, the viscous acceleration is

$$f_{\text{visc}} = \rho^{-1} \nabla \cdot (2\rho\nu \mathbf{S}) \quad (221)$$

So, for the hyperviscous force, we must replace the rate-of-strain tensor by a high order version

$$f_{\text{visc}}^{(\text{hyper})} = \rho^{-1} \nabla \cdot (2\rho\nu_n \mathbf{S}^{(n)}) \quad (222)$$

where the n^{th} -order rate of strain tensor is

$$\mathbf{S}^{(n)} = (-\nabla^2)^{n-1} \mathbf{S}. \quad (223)$$

For the $n = 3$ case it is

$$S_{ij}^{(3)} = \frac{1}{2} \left(\frac{\partial^5 u_j}{\partial x_i^5} + \frac{\partial^4}{\partial x_i^4} \left(\frac{\partial u_i}{\partial x_j} \right) - \frac{1}{3} \frac{\partial^4}{\partial x_i^4} (\nabla \cdot \mathbf{u}) \right). \quad (224)$$

Plugging it into Eq. (222), and assuming $\mu_3 = \rho\nu_3 = \text{const}$

$$f_{\text{visc}}^{(\text{hyper})} = \nu_3 \left(\nabla^6 \mathbf{u} + \frac{1}{3} \nabla^4 (\nabla (\nabla \cdot \mathbf{u})) \right). \quad (225)$$

For $\nu_3 = \text{const}$, we have to take derivatives of density as well

$$f_{\text{visc}}^{(\text{hyper})} = \nu_3 \left(\nabla^6 \mathbf{u} + \frac{1}{3} \nabla^4 (\nabla (\nabla \cdot \mathbf{u})) + 2\mathbf{S}^{(3)} \cdot \nabla \ln \rho \right) \quad (226)$$

E.2.2 Non-conservative cases

Equations (225) and (226) explicitly conserve linear *and* angular momentum. Although desirable properties, such expressions are cumbersome and numerically expensive, due to the fourth order derivatives of $\nabla(\nabla \cdot \mathbf{u})$.

This term, however, is only important when high compressibility is present (since it depends on the divergence of $\mathbf{u}$). In practice we drop this term and use a simple hyperviscosity

$$f_{\text{visc}} = \begin{cases} \nu_3 \nabla^6 \mathbf{u} & \text{if } \mu = \text{const} \\ \nu_3 \left(\nabla^6 \mathbf{u} + 2\mathbf{S}^{(3)} \cdot \nabla \ln \rho \right) & \text{if } \nu = \text{const} \end{cases} \quad (227)$$

Notice that this can indeed be expressed as the divergence of a simple rate-of-strain tensor

$$S_{ij}^{(3)} = \frac{\partial^5 u_i}{\partial x_j^5}, \quad (228)$$

so it does conserve linear momentum. It does *not*, however, conserve *angular* momentum, since the symmetry of the rate-of-strain tensor was dropped. Thus, vorticity sinks and sources may be spuriously generated at the grid scale.

A symmetric tensor can be computed, that conserves angular momentum and can be easily implemented

$$S_{ij} = \frac{1}{2} \left(\frac{\partial^5 u_i}{\partial x_j^5} + \frac{\partial^5 u_j}{\partial x_i^5} \right) \quad (229)$$

This tensor, however, is not traceless, and therefore accurate only for weak compressibility. It should work well if the turbulence is subsonic. Major differences are not expected, since the spectral range in which hyperviscosity operates is very limited: as a numerical tool, only its performance as a high-frequency filter is needed. This also supports the usage of the highest order terms only, since these are the ones that provide quenching at high k . Momentum conservation is a cheap bonus. Angular momentum conservation is perhaps playing it too safe, at great computational expense.

E.2.3 Choosing the coefficient

When changing the resolution, one wants to keep the grid Reynolds number, here defined as

$$\text{Re}_{\text{grid}} = u_{\text{rms}} / (\nu_n k_{\text{Ny}}^{2n-1}) \quad (230)$$

approximately constant. Here, $k_{\text{Ny}} = \pi/\delta x$ is the Nyquist wavenumber and δx is the mesh spacing. Thus, when doubling the number of meshpoints, we can decrease the viscosity by a factor of about $2^5 = 32$ (Haugen & Brandenburg 2004). This shows that

hyperviscosity can allow a dramatic increase of the Reynolds number based on the scale of the box.

By choosing *idiff*='hyper3_mesh' in *density_run_pars* the hyperdiffusion for density is being set automatically in a mesh-independent way. A hyper-mesh Reynolds number of 30 corresponds to a coefficient *diffrho_hyper3_mesh*=2 if *maxadvec* is about 1, but in practice we need a bit more (5 is currently the default).

E.2.4 Turbulence with hyperviscosity

When comparing hyperviscous simulations with non-hyperviscous ones, it turns out that the Reynolds number at half the Nyquist frequency is usually in the range 5–7, i.e.

$$\text{Re}_{\text{half-grid}} = u_{\text{rms}} / [\nu_n (k_{\text{Ny}}/2)^{2n-1}] \approx 5-7 \quad (231)$$

The following table gives some typical values used in simulations with forcing wavenumber $k_f = 1.5$ and a forcing amplitude of $f_0 = 0.02$. If hyperdiffusion D_3 is used in the continuity equation, the corresponding values are about 30 times smaller than those of ν_3 ; see Table 14.

Table 14: Empirical values of viscosity and hyperviscosity, as well as hyperdiffusion for density, at different numerical resolution, for simulations with forcing wavenumber $k_f = 1.5$ and a forcing amplitude of $f_0 = 0.02$ in a 2π periodic domain. In all cases the half-mesh Reynolds number is about 5–7. For comparison, estimates of the numerical 4th order hyperdiffusion resulting from a third order time step are give for two values of the CFL parameter.

N	ν_1	ν_2	ν_3	D_3	$\kappa_2^{\text{CFL}=0.4}$	$\kappa_2^{\text{CFL}=0.9}$
16	1×10^{-2}	3×10^{-4}	2×10^{-5}	6×10^{-7}	7×10^{-4}	1×10^{-4}
32	5×10^{-3}	4×10^{-5}	6×10^{-7}	2×10^{-8}	1×10^{-6}	2×10^{-5}
64	2×10^{-3}	5×10^{-6}	2×10^{-8}	6×10^{-10}	2×10^{-7}	3×10^{-6}
128	1×10^{-3}	6×10^{-7}	6×10^{-10}	2×10^{-11}	3×10^{-8}	4×10^{-7}
256	5×10^{-4}	8×10^{-8}	2×10^{-11}	6×10^{-13}	4×10^{-9}	5×10^{-8}
512	2×10^{-4}	1×10^{-8}	6×10^{-13}	2×10^{-14}	5×10^{-10}	6×10^{-9}
1024	1×10^{-4}	1×10^{-9}	2×10^{-14}	6×10^{-16}	6×10^{-11}	8×10^{-10}
	1×10^{-5}	1×10^{-9}	6×10^{-19}	6×10^{-16}	6×10^{-11}	8×10^{-10}

For comparison, we give in Table 14 estimates of the numerical 4th order hyperdiffusion resulting from a third order time step, for which we have

$$\kappa_2^{\text{CFL}} = \frac{1}{24} u_{\text{rms}} (C_{\text{CFL}} \delta x)^3 \quad (232)$$

where C_{CFL} is the CFL parameter which is either 0.4 in the conservative case or 0.9 in the more progressive case.

Figure 26 shows the scaling of the diffusion velocity normalized by the rms velocity of the turbulence. We see that (i) the value of k/k_{Ny} increases with the number of mesh points and (ii) the diffusion speed decreases inversely linear with the wavenumber.

E.3 Anisotropic hyperdissipation

As we want quenching primarily at the Nyquist frequency, hyperdissipation depends intrinsically on the resolution, according to Eq. (216). Because of this, *isotropic* hyperdissipation only gives equal quenching in all spatial directions if $\Delta x = \Delta y = \Delta z$, i.e., if the

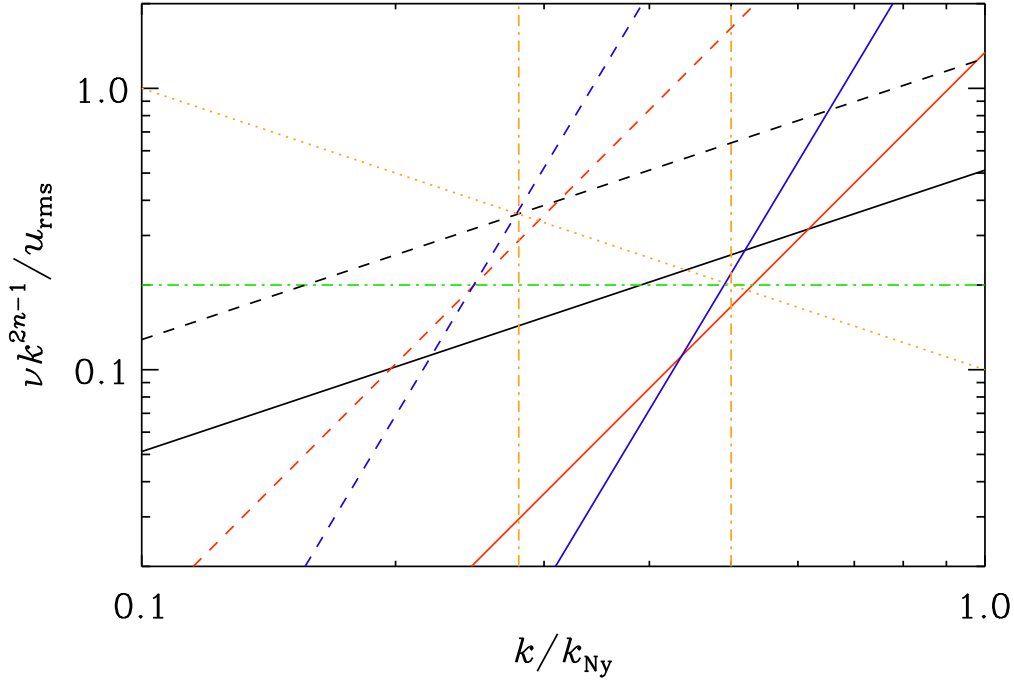


Figure 26: Scaling of the diffusion velocity normalized by the rms velocity of the turbulence.

cells are cubic. For non-cubic cells, anisotropic dissipation is required as different directions may be better/worse sampled, thus needing less/more numerical smoothing. Such generalization is straightforward. For that, we replace Eq. (213) by

$$\mathcal{J} = \left(D_x \frac{\partial^5 \rho}{\partial x^5}, D_y \frac{\partial^5 \rho}{\partial y^5}, D_z \frac{\partial^5 \rho}{\partial z^5} \right), \quad (233)$$

so that different diffusion operates in different directions. Since D_x , D_y and D_z are constants, the divergence of this vector is

$$\nabla \cdot \mathcal{J} = D_x \frac{\partial^6 \rho}{\partial x^6} + D_y \frac{\partial^6 \rho}{\partial y^6} + D_z \frac{\partial^6 \rho}{\partial z^6}. \quad (234)$$

The formulation for resistivity and heat conductivity are strictly the same. For viscosity it also assumes the same form if we consider the simple non-conservative rate-of-strain tensor (228).

Mathematically, these operations can be written compactly by noticing that the coefficients in Eq. (234) transform like diagonal tensors $\chi_{ij}^{(3)} = \chi_k^{(3)} \delta_{ijk}$, where δ_{ijk} is the unit diagonal third order tensor, $\chi^{(3)}$ is the vector containing the dissipative coefficients (diffusion, viscosity, resistivity, or heat conductivity) in x , y , and z , and summation over repeated indices applies.

Therefore, for a scalar quantity ψ (density, any of the three components of the velocity or magnetic potential), we can write

$$\frac{\partial \psi}{\partial t} = -\chi_{ij}^{(3)} \partial_i \partial_j^5 \psi = -\sum_q \chi_q^{(3)} \frac{\partial^6}{\partial x_q^6} \psi. \quad (235)$$

E.4 Hyperviscosity in Burgers shock

Hyperviscosity has the unfortunate property of introducing (numerically stable) wiggles, even if one just adds a little bit of hyperviscosity to a run with normal viscosity; see left hand side of Fig. 27. Running with just hyperviscosity give strong wiggles.

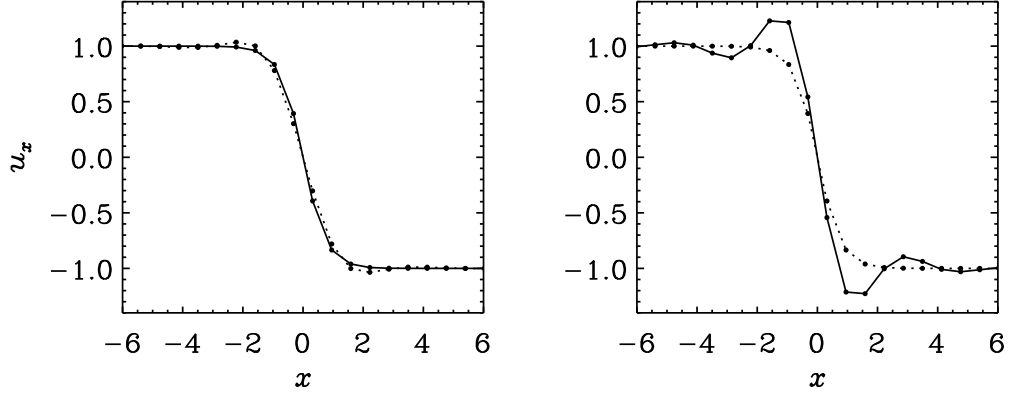


Figure 27: Left: Burgers shock from teach/PencilCode/material/BurgersShock (in the teaching material) with $-20 \leq x \leq 20$, $n_x = 64$ mesh points, $u_x = \mp 1$ on the two ends, $\nu = 0.4$ and either $\nu_3 = 0$ (solid line) or $\nu_3 = 0.05$ (dashed line). Right: similar to the left hand side, but with $\nu = 0$ and $\nu_3 = 0.05$ (dashed line), compared with the case $\nu = 0.4$ and $\nu_3 = 0$ (solid line).

E.5 Time-dependent viscosity and magnetic diffusivity

In connection with decaying hydrodynamic and MHD turbulence studies, [38] noted that the equations are invariant under rescaling of space and time coordinates, along with a corresponding rescaling of the other dependent variables.

$$\begin{aligned} t &= \tau t', & \mathbf{x} &= \tau^q \mathbf{x}', & \nu &= \tau^{2q-1} \nu', & \eta &= \tau^{2q-1} \eta', \\ \mathbf{u} &= \tau^{q-1} \mathbf{u}', & \mathbf{B} &= \tau^{q-1} \mathbf{B}'. \end{aligned} \quad (236)$$

Inserting these variables into equation (44) and (47), the resulting equation in the primed quantities has the same form as the equations in their original formulation. This requires that $\nu \propto \eta \propto t^r$ where $r = 2q - 1$. For $q < 1/2$, r is negative so t^r becomes singular for $t \rightarrow 0$. Therefore, we use in such cases

$$\nu(t) = \nu_0 [\max(1, t/t_0)]^r, \quad \eta(t) = \eta_0 [\max(1, t/t_0)]^r, \quad (237)$$

where t_0 is the time below which ν and η are assumed fixed; see Ref. [10] for details. This can be accomplished by putting

```
&viscosity_run_pars
  ivisc='nu-tdep', nu=1e-3, nu_tdep_t0=.1, nu_tdep_exponent=-.43
  lvisc_nu_tdep_t0_norm=T
/
```

and

```
&magnetic_run_pars
  iresistivity='eta-tdep', eta=1e-3, eta_tdep_t0=.1, eta_tdep_exponent=-.43
  lresi_eta_tdep_t0_norm=T
/
```

For backward compatibility, the default is `lvisc_nu_tdep_t0_norm=F` and likewise `lresi_eta_tdep_t0_norm=F`. This means that instead of Eq. (237), we use $\nu(t) = \nu_0 [\max(t, t_0)]^r$ and $\eta(t) = \eta_0 [\max(t, t_0)]^r$, which has the disadvantage that then ν_0 and η_0 have funny dimensions.

F Special techniques

F.1 After changing *REAL_PRECISION*

To continue working in double precision (*REAL_PRECISION*=double), you just say `lread_from_other_prec=T` in `run_pars`. Afterwards, remember to put `lread_from_other_prec=F`. If continuation is done in a new run directory, first execute `start.csh` there and then copy the files `var.dat` (and if present `global.dat`) from the old to the new directory, using `pc-copyvar`.

F.2 Remeshing (regridding)

Currently (29.07.19) two options are available for taking an existing run with a given resolution and processor layout and continuing with a changed resolution or different layout of processors. These apply for the original fortran binary data format used by the Pencil Code. The original version is described in section F.2.3, while a recent more versatile option employing Python to perform the task is described in section F.2.2.

The parallel hdf5 format for storing Pencil Code data has now been implemented to replace the less portable fortran unformatted data. As the hdf5 data is stored in a single file, the processor layout can be revised at any time during a run, without the need to change the data files at all. Changing resolution or grid dimensions is also more convenient, using appropriate interpolation tools. When scripts to automatically resize the grid and physical data for the full f-array become available, instructions on their use shall be included in section F.2.1.

F.2.1 Remeshing hdf5-formatted data

Instructions shall follow shortly ...

F.2.2 Remeshing unformatted fortran binary data using Python

Ensure that `$PENCIL_HOME/python` has been added to your `$PYTHON_PATH`. Let us assume you have an existing run and you would like to continue an experiment from a mature state, but with

- higher (or lower) resolution,
- changes to the size of the numerical domain,
- change to the processor layout to improve efficiency and/or speed of the calculation,
- added/reduced variables included in the model,

or any combination involving at least one of the first three options in the list. The following procedure can be used to handle the first three options. The remeshed model with the ‘`var.dat`’ files obtained from an existing run can then also be used to advance with additional physics, following section F.4. As part of the remeshing procedure reduced physics could be obtained by omitting unwanted variables (not yet implemented).

1. First set up the new run. One option is to navigate to the path of the existing run `$path-to-old-run` and apply the command

```
$path-to-old-run> pc_newrun $path-to-new-run
```

`$path-to-new-run` is the full path of the new run or its relative path assuming it will be prefixed by `../`. Then navigate to `$path-to-new-run`. Revise as required the files `'start.in'`, `'src/cparam.local'` and `'src/Makefile.local'` to match the parameters for the remeshed run. Note, any changes involving domain size will need to ensure continuity across any periodic boundaries. Then compile and start the new run

```
$path-to-new-run> pc_build
$path-to-new-run> pc_run start
```

or otherwise submit a batch script containing the call to `'start.csh'`.

2. Once the new run has been started successfully, create in the new run directory a python script called `'local_remesh.py'` or similar, as described in the notes at the end of `'$PENCIL_HOME/python/pencil/files/sim/remesh.py'`. Add import `pencil` as `pc` at the start of the file.
3. To read the old data and then interpolate this onto the new grid as a complete f-array object `fnew`, add to the `'local_remesh.py'` the lines

```
fnew = pc.interp_var(
    source_path=$path-to-old-run,
    target_path=$path-to-new-run,
    arrs=['uu', 'rho', 'lnrho', 'ss', 'aa',
         'shock', 'netheat', 'cooling'],
)
```

By default this will read in the data from the old run `'var.dat'` files. If another source file is required add the line `'oldvar=$VAR,'` between the brackets, where `$VAR` is one of the snapshots with prefix `'VAR'`. Replace the default list of variables `arrs` with the variables used in the old run listed in f-array index order. For other options inspect `'$PENCIL_HOME/python/pencil/files/remesh.py'`. Handling particles is not yet implemented, but should be possible with minor edits.

4. To write `fnew` to the new run `'var.dat'` files add to `'local_remesh.py'` the lines

```
pc.distribute_fort(
    fnew,
    $path-to-new-run,
)
```

5. To execute the script run

```
$path-to-new-run> python local_remesh.py
```

To preserve long term the remeshed data

```
$path-to-new-run> pc_copyvar v 0 -e
```

6. Once the remeshing is completed edit as required `'run.in'` and execute

```
$path-to-new-run> pc_run run
```

or submit batch script as appropriate.

F.2.3 Remeshing unformatted fortran binary - original method

[This should be written up in a better way and put somewhere else. But currently, remeshing is only available for the Pencil developers anyway.]

Suppose you have a directory `run_64` with a 64^3 run (running on $N_0 = n_y \times n_z = 2 \times 1$ CPUs) that you want to continue from 'VAR1' at 128^3 (on $n_y \times n_z = 4 \times 4$ CPUs).

1. The remeshing code is part of the PENCIL CODE repository.
2. Create another run directory with current 'VAR1' as 'var.dat' (remesh.csh so far only works with 'var.dat'):

```
unix> cd run_64
run_64> pc_newrun ../tmp_64 or new tmp_64
run_64> mkdir -p ../tmp_64/data or (cd ../tmp_64/; crtmp)
run_64> (cd ../tmp_64/data ; mkproc-tree  $N_0$ )
run_64> restart-new-dir-VAR ../tmp_64 1
```

3. Create the new run directory (linking the executables with -s):

```
run_64> cd ../tmp_64
tmp_64> pc_newrun -s ../run_128 or new run_128
tmp_64> vi ../run_128/src/cparam.local
# set nxgrid=128, ncpus=16, nprocy=4
tmp_64> (cd ../run_128; crtmp; pc_setupsrsc; make)
```

4. Setup and do remeshing

```
tmp_64> setup-remesh
tmp_64> vi remesh/common.local
# set muly=2, mulz=4, remesh_par=2
tmp_64> (cd remesh; make)
tmp_64> vi remesh.in
# Replace line by ../run_128
tmp_64> remesh.csh
# Answer 'yes'
```

F.3 Restarting with different I/O strategy

One might want to switch the I/O strategy for a run, ongoingly to be continued by restarts, typically from one of the more traditional schemes, reading and writing FORTRAN binary or formatted data, to using the HDF5 data format. For this, include a definition for `IO_IN` in `Makefile.local`, say `IO_IN=io_dist`, change the definition of `IO`, say into `IO=io.hdf5`, and recompile. The restarted run will write all data with the new I/O scheme and write an (empty) control file `IO_LOCK` which prevents the code from trying to read the data at the next restart still with the old I/O scheme, specified by `IO_IN`. A backswitch from HDF5 to binary format is also possible, but note that averages and slices will be continued to be written in HDF5 unless you recompile the code once more with `IO_IN` removed (to be improved).

F.4 Restarting from a run with less physics

First, prepare a new run directory with the new physics included. By new physics, we mean that the new run wants to read in more fields (e.g., magnetic fields, if the old run didn't have magnetic fields).

Example for test fields:

1. Prepare 'src/cparam.local'

Add the following 2 fragments into the 'cparam.local' file. The first piece comes in the beginning and the second in the end of the file.

```
! ** AUTOMATIC CPARAM.INC GENERATION *****
! Declare (for generation of cparam.inc) the number of f array
! variables and auxiliary variables added by this module
! Use MVAR to reserve the appropriate workspace for testfield_z.f90
! The MAUX number must be equally big and is used for uxb in the f-array.
! At the end of this file, njtest must be set such that 3*njtest=MVAR.
!
! MVAR CONTRIBUTION 12
! MAUX CONTRIBUTION 12
!
! *****
!
! note that MVAR=MAUX=3*njtest must be obeyed
!
integer, parameter :: njtest=4
!
```

2. Prepare 'src/Makefile.local'

Add the line `TESTFIELD=test_methods/testfield_z` to the file. Finally, compile the code.

3. Prepare restart data

Go into data directory of the new run and prepare the directory tree using, e.g., the command `pc_mkproctree 16`. [In principle this could be automatized, but it isn't yet.]

Next, go into old run directory and say `restart-new-dir ../32c`, if '`../32c`' is the name of the new run directory. This procedure copies all the files from the processor tree, plus files like 'param.nml', but this file may need some manual modification (or you could just use one from another runs with the new physics included, which is definitely the simplest!).

4. Prepare 'run.in'

Set `lread_oldsnap_notestfield=T` in *run_pars*. This means (as the name says) that one reads an old snapshot that did not have test fields in it.

Reset boundary conditions and add stuff for the newly added fields, e.g., `bcs='a:s','a','a:s','a2','a','a','s','a','a','s','a','a','s','a','a','s'` in *run_pars*. If you don't do this, you would effectively use periodic boundary

conditions for the response to the test field, which is hardly correct once you set non-periodic boundary conditions for the other variables.

Add something like the following text fragments in the right position (after *grav_run_pars* and *magnetic_run_pars*, but before *shear_run_pars* and *viscosity_run_pars*).

```
&testfield_run_pars
  !linit_aatest=T, daainit=100.
  itestfield='B11-B22'
  etatest=1e-4
  lsoca=F
/
```

Make sure that the data above are correct. You may want to change the values of *daainit* or *etatest*.

If you now run, and if you didn't fix the file 'data/param.nml' you might get something like the following error:

```
forrtl: severe (24): end-of-file during read, unit 1, file /wkspc/brandenb/pencil-
```

The reason for this is that it reads the old boundary data, but the corresponding array is too short. This includes stuff like *FBCX1* to *FBCX2.2*, but it is still not enough. Therefore it is easiest to use the 'data/param.nml' file from another run. You may well just use one from a single processor run with a different mesh. But remember to fix the 'start.in' file by correcting the boundary conditions and adding things like

```
&testfield_init_pars
  luxb_as_aux=T
/
```

5. Prepare 'print.in', 'xyaver.in', and other obvious files such as 'video.in'.
6. Once it works and is running, you must say *explicitly*

```
&run_pars
  ...
  lread_oldsnap_notestfield=F
/
```

because otherwise you won't read in your precious test field data next time you restart the code! (If you instead just remove this line, it will remember *lread_oldsnap_notestfield=T* from the previous run, which is of course wrong!)

Comments: For large magnetic Reynolds numbers the solutions to the test-field equations can show a linear instability, which can introduce large fluctuations. In that case it is best to reset the dependent test-field variable to zero in regular intervals. This is done by setting *linit_aatest=T*. Note that *daainit=100* sets the reset interval to 100.

F.5 Restarting with particles from a run without them

If you want to restart from a run without particles to a run with them, you need to

- (1) Compile a run with particles,

```
RunWithParticles$ pc_build
```

(2) Copy a VARN or var.dat into this new run directory. Say the old run is at the directory OldRun and you want to copy the var.dat of that run and restart with particles. You do

```
RunWithParticles$ pc_copyvar v v ../OldRun . -e
```

(3) In the start.in init_pars of the new run, add the lines

```
lread_oldsnap = T  
ireset_tstart = 0
```

(4) Remove all calls to initial conditions (make all initlnrho, inituu, initss, initaa, etc ‘nothing’), and

5) Run pc_start.

The variable `lread_oldsnap` makes the code on start time read from the f-array of the snapshot, instead of the default, which is to initialize it with zeros. The necessity in step (4) to remove all calls to initial conditions is because otherwise the code would rewrite the content of the f-array with these initial conditions, or add them on top of the existing values of the snapshot.

The variable `ireset_tstart` when set to zero makes it read the timestamp of the old snapshot and restart from that time. The default is 2, which sets the timestamp back to zero.

G Runs and reference data

For reference purposes we document here some results obtained with various samples of the code.

G.1 Shock tests

G.1.1 Sod shock tube problem

Table 15: Combinations of ρ , p , and s/c_p that are relevant for the Sod shock tube problem with constant temperature and different pressure ratios on the left and right hand sides of the shock.

ρ	p	s
1.0	1.0	0.3065
0.1	0.1	1.2275
0.01	0.01	2.1486

G.1.2 Temperature jump

Table 16: Combinations of c_s^2 , p , and s/c_p that are relevant for the temperature shock problem with constant density, $\rho = 1$, and different temperature ratios on the left and right hand sides of the shock.

c_s^2	s
1.0	0.0
0.1	-2.3
0.01	-4.6
10^{-4}	-9.2

G.2 Random forcing function

A solenoidal random forcing function f can be invoked by putting `iforce='helical'` in the `forcing_run_pars` namelist. This produces the forcing function f of the form

$$f(\mathbf{x}, t) = \text{Re}\{N \mathbf{f}_{\mathbf{k}(t)} \exp[i\mathbf{k}(t) \cdot \mathbf{x} + i\phi(t)]\}, \quad (238)$$

where $\mathbf{k}(t) = (k_x, k_y, k_z)$ is a random time dependent wave vector, $\mathbf{x} = (x, y, z)$ is position, and $\phi(t)$ with $|\phi| < \pi$ is a random phase. On dimensional grounds the normalization factor is chosen to be $N = f_0 c_s (k c_s / \delta t)^{1/2}$, where f_0 is a nondimensional factor, $k = |\mathbf{k}|$, and δt is the length of the timestep. The $\delta t^{-1/2}$ dependence ensures that the forcing, which is delta-correlated in time, is properly normalized such that the correlator of the forcing function is independent of the length of the time step, δt . We focus on the case where $|\mathbf{k}|$ is around 5, and select at each timestep randomly one of the 350 possible vectors in $4.5 < |\mathbf{k}| < 5.5$. We force the system with eigenfunctions of the curl operator,

$$\mathbf{f}_{\mathbf{k}} = \frac{i\sigma \mathbf{k} \times (\mathbf{k} \times \mathbf{e}) + |\mathbf{k}|(\mathbf{k} \times \mathbf{e})}{\sqrt{1 + \sigma^2} k^2 \sqrt{1 - (\mathbf{k} \cdot \mathbf{e})^2 / k^2}}, \quad (239)$$

where $\mathbf{e}$ is an arbitrary unit vector needed in order to generate a vector $\mathbf{k} \times \mathbf{e}$ that is perpendicular to $\mathbf{k}$. Note that $|\mathbf{f}_{\mathbf{k}}|^2 = 1$ and, for $\sigma = 1$, $i\mathbf{k} \times \mathbf{f}_{\mathbf{k}} = |\mathbf{k}| \mathbf{f}_{\mathbf{k}}$, so the helicity density of this forcing function satisfies

$$\mathbf{f} \cdot \nabla \times \mathbf{f} = |\mathbf{k}| f^2 > 0 \quad (\text{for } \sigma = 1) \quad (240)$$

at each point in space. We note that since the forcing function is like a delta-function in k -space, this means that all points of f are correlated at any instant in time, but are different at the next timestep. Thus, the forcing function is delta-correlated in time (but the velocity is not). This is the forcing function used in Brandenburg (2001), Brandenburg & Dobler (2001), and other papers in that series.

For $\sigma = 0$, the forcing function is completely nonhelical and reduces to the simpler form

$$f_k = (\mathbf{k} \times \mathbf{e}) / \sqrt{k^2 - (\mathbf{k} \cdot \mathbf{e})^2}. \quad (241)$$

For $0 < |\sigma| < 1$, the forcing function has fractional helicity, where $\sigma \approx \langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle / (k_f \langle u^2 \rangle)$; see Sect. 4.5 of Ref. [13]. In the code and the *forcing_run_pars* namelist, σ is called *relhel*.

In the code, the possible wavevectors are pre-calculated and stored in ‘k.dat’, which is being read in the beginning the code runs. To change the wavevectors (e.g., the typical value of k_f , you need to change the file. In the directory ‘\$PENCIL_HOME/samples/helical-MHdturb/K_VECTORS/’ there are several such files prepared:

```
k10.dat  k1.dat   k2.dat   k3.dat   k5.dat
k15.dat  k27.dat  k30.dat  k4.dat   k8.dat
```

and more can be prepared in IDL with the procedure ‘\$PENCIL_HOME/samples/helical-MHdturb/idl/generate_kvectors.pro’. There is also more help in the ‘README’ file in ‘helical-MHdturb’.

In *forcing_hel*: if *lcrosshel_forcing*=T and if *ltestfield_forcing*=T and *ltestflow_forcing*=T, *uu* and *aa* (*uu0* and *aa0* for *testfield**) are simultaneously forced, using the same file *k.dat*. Relative scaling by *force1_scl* for vector potentials and *force2_scl* for velocities (default: 1). Simplified “cross helicity forcing” is now activated by *lhydro_forcing* and *lmagnetic_forcing*=.true. or by *ltestfield_forcing* and *ltestflow_forcing*=.true.; *lcrosshel_forcing* is now obsolete.

G.3 Three-layered convection model

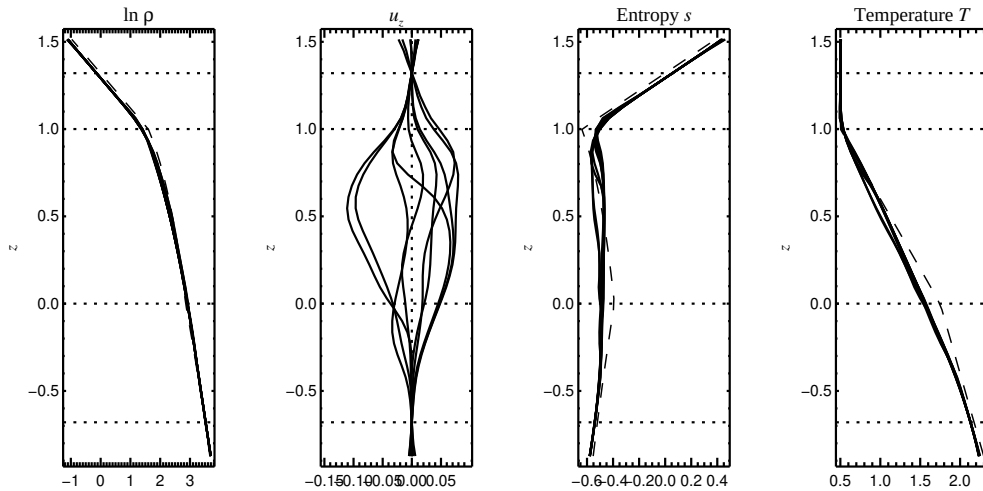


Figure 28: Like in Fig. 3, but at time $t = 50$.

In Sect. 3 we have shown the early stages of the convection model located in ‘samples/conv-slab’. To arrive at fully developed convection, you will need to run the

code for many more time steps. Figure 28 shows the vertical profiles of four basic quantities at time $t = 50$. Figure 29 shows the time evolution of rms and maximum velocity for the model for $0 < t < 50$.

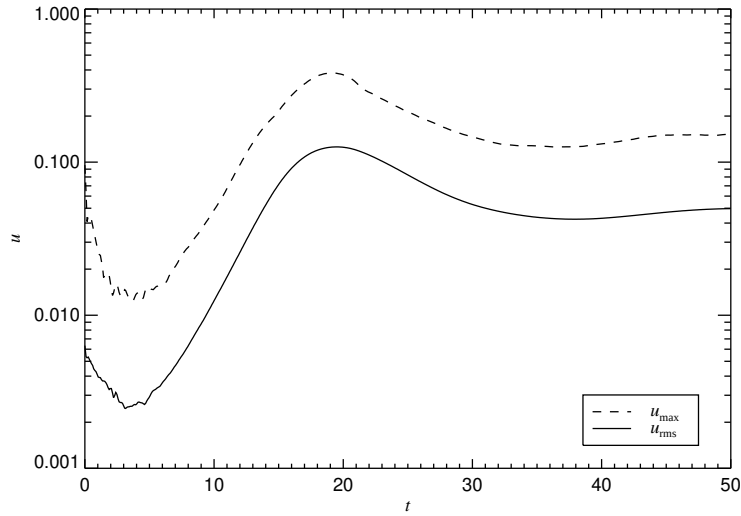


Figure 29: Time evolution of rms and maximum velocity for the model ‘samples/conv-slab’. Similar plots can be produced by running the IDL script ‘ts.pro’.

Figures 30 and 31 show vertical and horizontal sections for time $t = 50$.

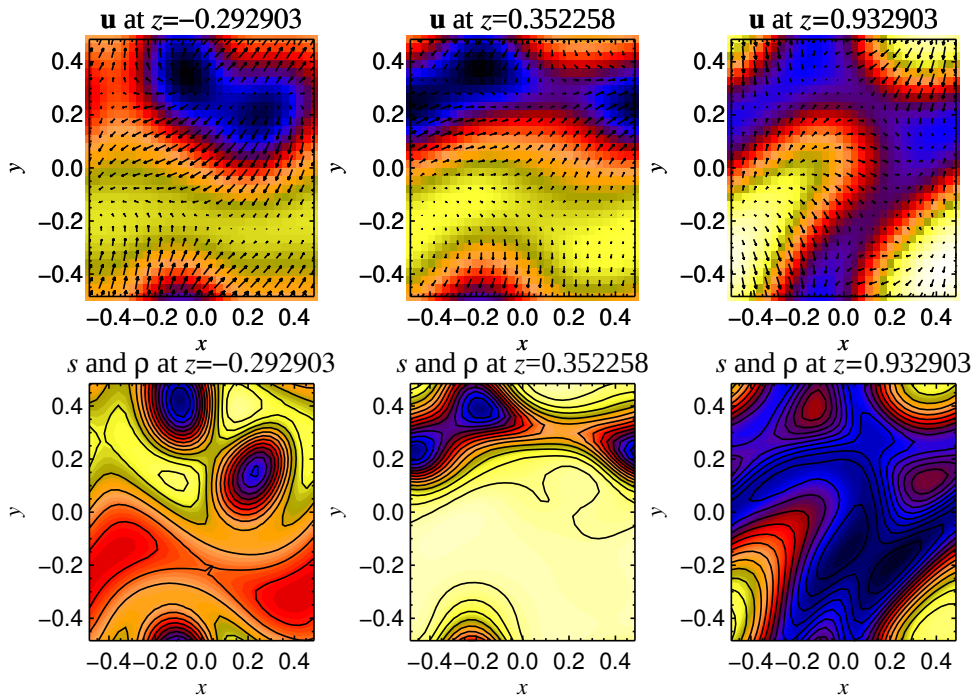


Figure 30: Horizontal sections for $t = 50$. Top: velocity field. Bottom: entropy (color coded) and density (isocontours). Plots of this type can be produced by running the IDL script ‘hsections.pro’.

G.4 Magnetic helicity in the shearing sheet

To test magnetic helicity evolution in the shearing shear, we can choose as initial condition `initaa='Beltrami-y'` with `amplaa=1.` in `magnetic_init_pars` together with `Sshear=-1.` in `shear_run_pars`.

Thus, in ‘src/Makefile.local’ we just use

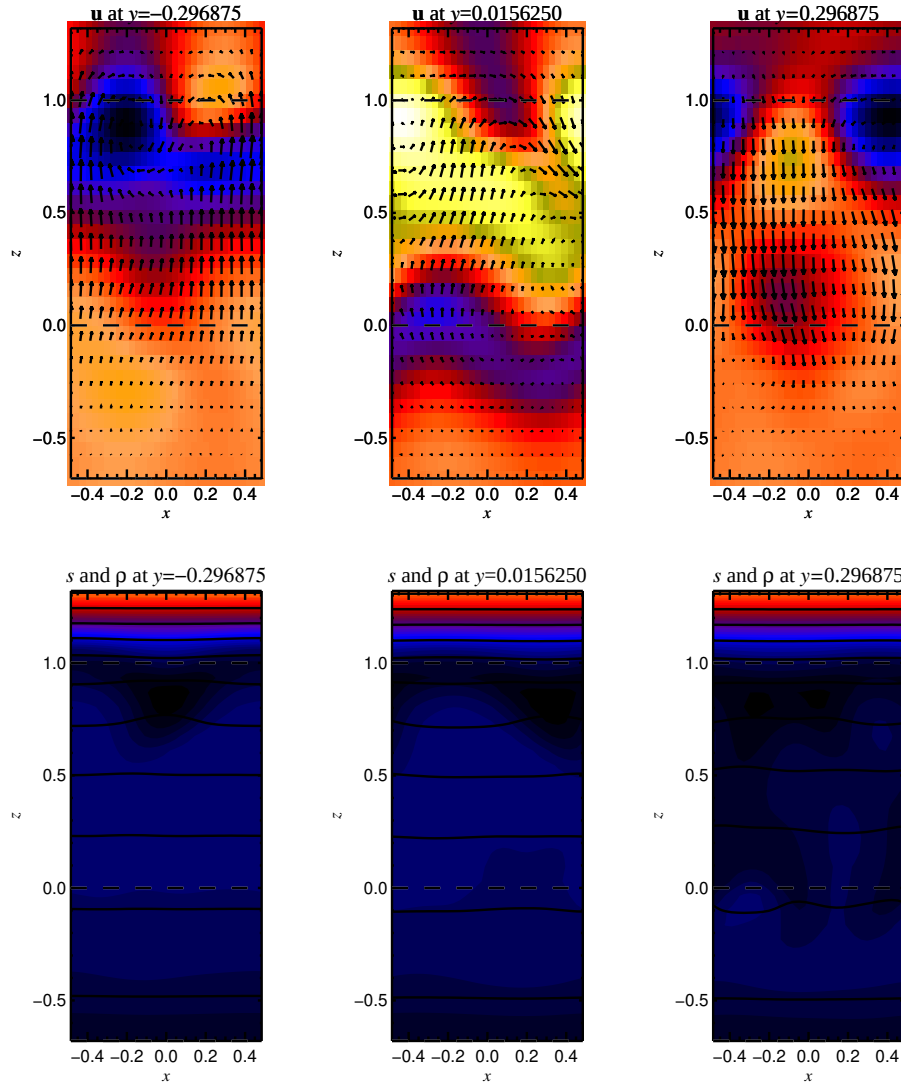


Figure 31: Vertical section $y = 0.516$ at $t = 50$. Top: velocity field. Bottom: entropy (color coded) and density (isocontours). Plots of this type can be produced by running the IDL scripts ‘vsections.pro’) or ‘vsections2.pro’).

```

MAGNETIC=magnetic
HYDRO=nohydro
EOS=noeos
DENSITY=nodensity
SHEAR=shear
VISCOSITY=noviscosity

```

and put

```

&init_pars
  cvsid='$Id$',
/
&magnetic_init_pars
  initaa='Beltrami-y', amplaa=1.
/
&shear_init_pars
/

```

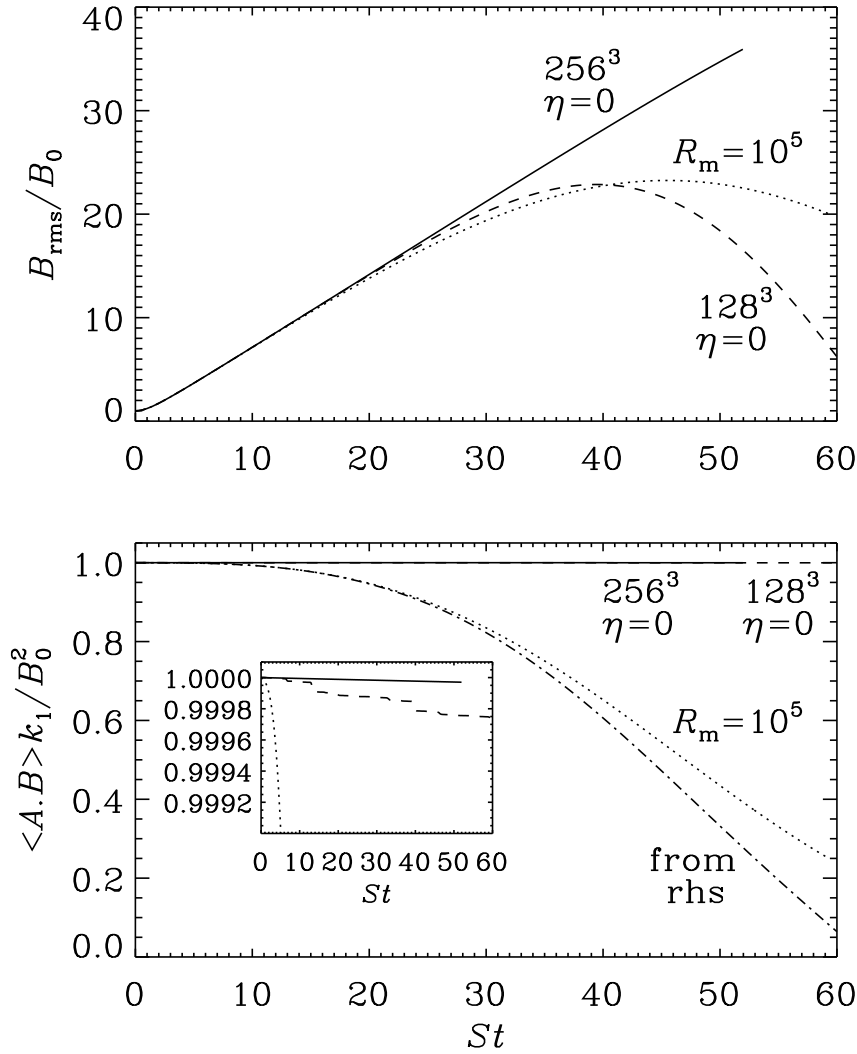


Figure 32: Wind-up of the magnetic field leads to a linear increase in the rms magnetic field strength until Ohmic diffusion begins to become important (top panel). During this time the magnetic helicity is conserved. With Ohmic diffusion, the decay of $\langle A \cdot B \rangle$ is well described by integrating $-2\eta \langle J \cdot B \rangle$ (indicated by “from rhs” in the second panel).

in ‘start.in’ and, for example,

```
&run_pars
  cvsid='$Id$'
  nt=150000, it1=10, cdt=0.9, isave=50, itorder=3
  dsnap=100. dvid=5., ialive=1
/
&magnetic_run_pars
  eta=0.
/
&shear_run_pars
  Sshear=-1.
/
```

in ‘run.in’. The output includes, among other things

```
arms(f10.7)
brms(f12.7)
```

```
jrms(f14.7)
abm(f14.11)
jbm(f14.7)
```

The result is shown in Figure 32, where we show the wind-up of the magnetic field, which leads to a linear increase in the rms magnetic field strength until Ohmic diffusion begins to become important (top panel). During this time the magnetic helicity is conserved. With Ohmic diffusion, the decay of $\langle \mathbf{A} \cdot \mathbf{B} \rangle$ is well described by integrating $-2\eta\langle \mathbf{J} \cdot \mathbf{B} \rangle$ (indicated by “from rhs” in the second panel).

H Numerical methods

H.1 Sixth-order spatial derivatives

Spectral methods are commonly used in almost all studies of ordinary (usually incompressible) turbulence. The use of this method is justified mainly by the high numerical accuracy of spectral schemes. Alternatively, one may use high order finite differences that are faster to compute and that can possess almost spectral accuracy. Nordlund & Stein [37] and Brandenburg et al. [18] use high order finite difference methods, for example fourth and sixth order compact schemes [32].²¹

The sixth order first and second derivative schemes are given by

$$f'_i = (-f_{i-3} + 9f_{i-2} - 45f_{i-1} + 45f_{i+1} - 9f_{i+2} + f_{i+3})/(60\delta x), \quad (242)$$

$$f''_i = (2f_{i-3} - 27f_{i-2} + 270f_{i-1} - 490f_i + 270f_{i+1} - 27f_{i+2} + 2f_{i+3})/(180\delta x^2), \quad (243)$$

In Fig. 33 we plot effective wavenumbers for different schemes. Apart from the different *explicit* finite difference schemes given above, we also consider a *compact* scheme of 6th order, which can be written in the form

$$\frac{1}{3}f'_{i-1} + f'_i + \frac{1}{3}f'_{i+1} = (f_{i-2} - 28f_{i-1} + 28f_{i+1} - f_{i+2})/(36\delta x), \quad (244)$$

for the first derivative, and

$$\frac{2}{11}f''_{i-1} + f''_i + \frac{2}{11}f''_{i+1} = (3f_{i-2} + 48f_{i-1} - 102f_i + 48f_{i+1} + 3f_{i+2})/(44\delta x^2). \quad (245)$$

for the second derivative. As we have already mentioned in the introduction, this scheme involves obviously solving tridiagonal matrix equations and is therefore effectively non-local.

In the second panel of Fig. 33 we have plotted effective wavenumbers for second derivatives, which were calculated as

$$(\cos kx)''_{\text{num}} = -k_{\text{eff}}^2 \cos kx. \quad (246)$$

Of particular interest is the behavior of the second derivative at the Nyquist frequency, because that is relevant for damping zig-zag modes. For a second-order finite difference scheme k_{eff}^2 is only 4, which is less than half the theoretical value of $\pi^2 = 9.87$. For fourth, sixth, and tenth order schemes this value is respectively 5.33, 6.04, 6.83. The last value is almost the same as for the 6th order compact scheme, which is 6.86. Significantly stronger damping at the Nyquist frequency can be obtained by using hyperviscosity, which Nordlund & Galsgaard (1995) treat as a quenching factor that diminishes the value of the second derivative for wavenumbers that are small compared with the Nyquist frequency. Accurate high order second derivatives (with no quenching factors) are important when calculating the current $\mathbf{J}$ in the Lorentz force $\mathbf{J} \times \mathbf{B}$ from a vector potential $\mathbf{A}$ using $-\mu_0 \mathbf{J} = \nabla^2 \mathbf{A} - \nabla \nabla \cdot \mathbf{A}$. This will be important in the MHD calculations presented below.

²¹The fourth order compact scheme is really identical to calculating derivatives from a cubic spline, as was done in Ref. [37]. In the book by Collatz [20] the compact methods are also referred to as *Hermitian methods* or as *Mehrstellen-Verfahren*, because the derivative in one point is calculated using the derivatives in neighboring points.

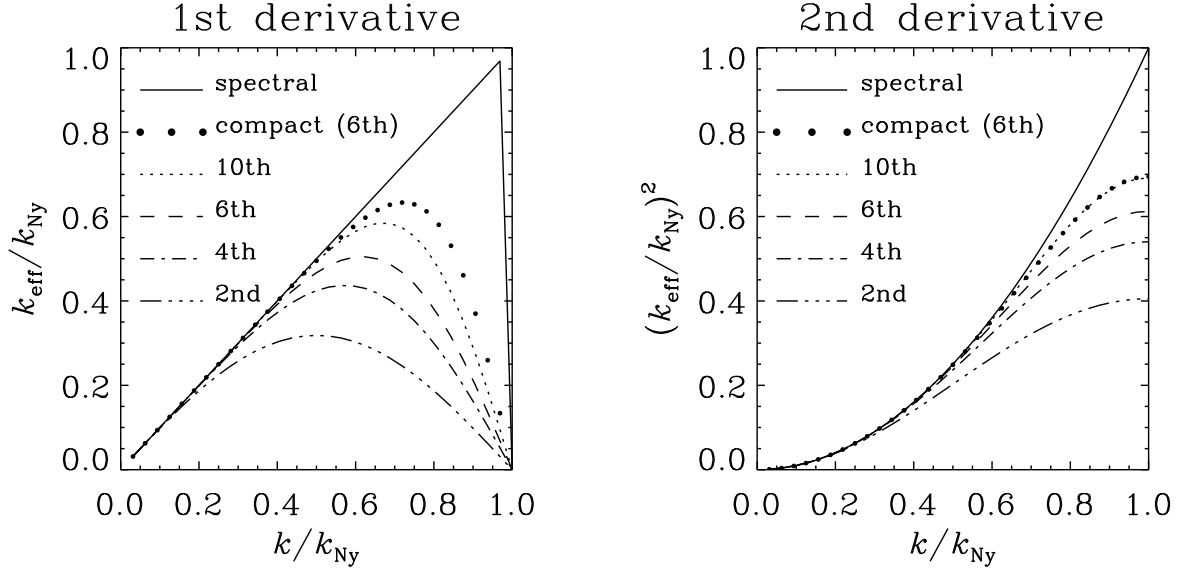


Figure 33: Effective wave numbers for first and second derivatives using different schemes. Note that for the second derivatives the sixth order compact scheme is almost equivalent to the tenth order explicit scheme. For the first derivative the sixth order compact scheme is still superior to the tenth order explicit scheme.

H.2 Upwind derivatives to avoid ‘wiggles’

High-order centered-difference convection simulations often show “wiggles” (Nyquist zigzag pattern) in $\ln \rho$, which are apparently caused by a velocity profile where the velocity approaches zero on the boundary or inside the box.²² This causes the density profile to be squeezed into a boundary layer where eventually numerical resolution is insufficient and, for centered differences, a spurious Nyquist signal is generated that almost instantaneously propagates into much of the whole box.

Even if the stagnation point is on the boundary (and enforced by the boundary conditions), this behavior is hardly influenced by the boundary conditions on $\ln \rho$ at all. A solution, however, is to apply upwinded derivative operators. The simplest upwind derivative is a finite-difference derivative operator where the point furthest downwind is excluded from the stencil. For $u > 0$, that means that instead of

$$f'_0 = \frac{-f_{-3} + 9f_{-2} - 45f_{-1} + 45f_1 - 9f_2 + f_3}{60 \delta x} - \frac{\delta x^6 f^{(7)}}{140} = D^{(\text{cent},6)} + O(\delta x^6) , \quad (247)$$

one takes

$$f'_0 = \frac{-2f_{-3} + 15f_{-2} - 60f_{-1} + 20f_0 + 30f_1 - 3f_2}{60 \delta x} + \frac{\delta x^5 f^{(6)}}{60} = D^{(\text{up},5)} + O(\delta x^5) . \quad (248)$$

A fourth-order upwind scheme (excluding two downwind points) would be

$$f'_0 = \frac{-f_{-3} + 6f_{-2} - 18f_{-1} + 10f_0 + 3f_1}{12 \delta x} - \frac{\delta x^4 f^{(5)}}{20} = D^{(\text{up},4)} + O(\delta x^4) . \quad (249)$$

²²A simple one-dimensional test profile would be $u(x) = 1 - x^2$ on $x \in [-1, 1]$, which will accumulate more and more mass near the right boundary $x = 1$.

In two- or three-dimensional settings, the presence of stagnation points of X-type leads to the same configuration, this time with the possibility of a steady state (i.e. without accumulation of mass). Such stagnation points occur, e.g., at the top of an upwelling, or at the bottom of a downdraft in convection simulations, where locally $u_z \propto z_X - z$.

The effect of upwinding is mostly to stop the Nyquist perturbations from spreading away from the boundary or stagnation point. With the fourth-order formula they actually hardly ever develop.

The difference between central and fifth-order upwind derivative is

$$[D^{(\text{up},5)} - D^{(\text{cent},6)}]f_0 = \frac{-f_{-3} + 6f_{-2} - 15f_{-1} + 20f_0 - 15f_1 + 6f_2 - f_3}{60\delta x} = -\frac{\delta x^5}{60}f_0^{(6)}. \quad (250)$$

In other words, 5th-order upwinding can be represented for any sign of u as hyperdiffusion (Dobler et al. 2006):

$$-uf'_{(\text{up},5\text{th})} = -uf'_{(\text{centr},6\text{th})} + \frac{|u|\delta x^5}{60}f^{(6)}. \quad (251)$$

The advantage over adopting constant hyperdiffusion is that in the upwinding scheme hyperdiffusion is only applied where it is needed (i.e. where advection is happening, hence the factor $|u|$).

The form (251) also suggests an easy way to get ‘stronger’ upwinding: Rather than excluding more points in the downwind direction, we can simply treat the weight of the hyperdiffusion term as a free parameter α :

$$-uf'_{(\text{up},5\text{th},\alpha)} = -uf'_{(\text{centr},6\text{th})} + \alpha|u|\delta x^5f^{(6)}. \quad (252)$$

If α is large, this may affect the time step, but for $\alpha = 1/60$, the stability requirement for the hyperdiffusive term should always be weaker than the advective Courant criterion.

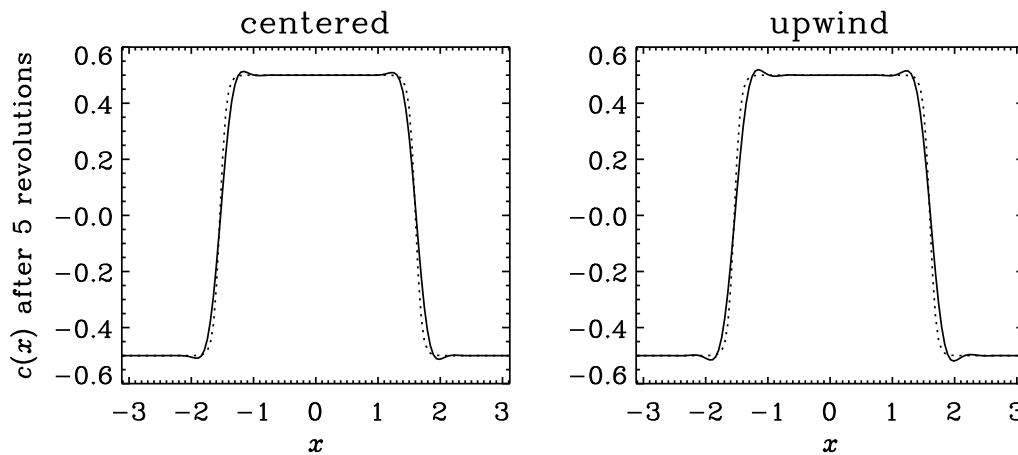


Figure 34: Advection with centered (left) and upwinding (right) schemes with diffusivity $\kappa = 10^{-4}$, $n_x = 128$ mesh points, and an advection velocity of unity.

A standard advection experiment is shown in Fig. 34 for $n_x = 128$ mesh points over a 2π domain with advection velocity $U_x = 1$ and diffusivity $\kappa = 10^{-4}$, so $\kappa/U_x\delta x = 0.002$. We see that upwinding causes additional wiggles.

In Fig. 35, we compare turbulence simulations with 512^3 mesh points using $\nu = 10^{-4}$, a forcing amplitude $f_0 = 0.02$, and a forcing wavenumber of 1.5. The resulting Mach number is then 0.13 and the Reynolds number is $u_{\text{rms}}/\nu k_f = 900$. We also show a comparison with a run without regular diffusion, using just slope-limited diffusion (SLD) instead. Below an excerpt from the ‘run.in’ file for one of the runs.

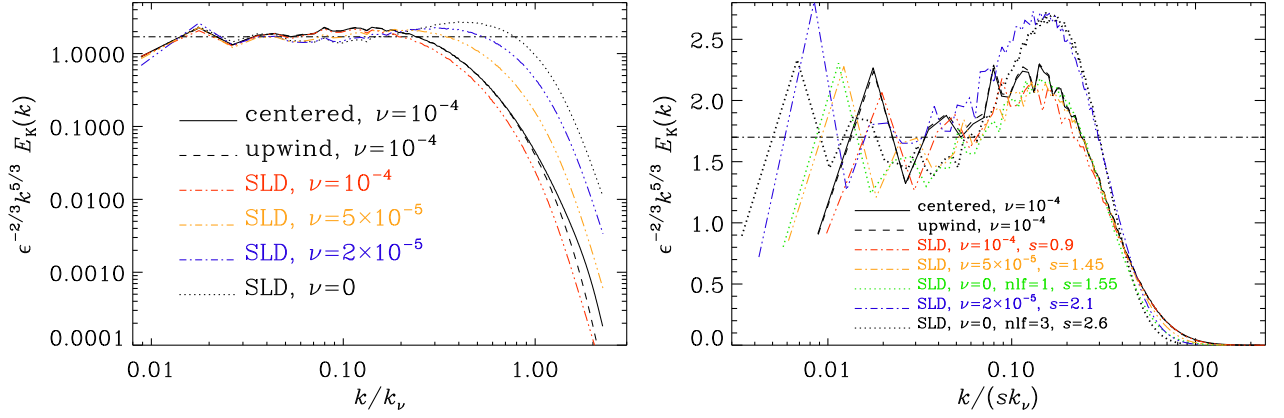


Figure 35: Comparison of centered (solid lines) and upwinding (dashed lines) advection in a 512^3 isothermal turbulence simulation forced with $f_0 = 0.02$. The computational domain is L^3 with $L = 2\pi$ and the forcing wavenumber is $k_f = 1.5$. The viscosity is $\nu = 10^{-4}$. The dotted line gives a run without regular diffusion, but with slope-limited diffusion (SLD) instead. In the right panel, the curves are plotted versus $k/(s k_\nu)$, where $k_\nu = (\epsilon_K/\nu^3)^{1/2}$ is the nominal viscous cutoff wavenumber, and s is an empirical gain factor that has been introduced to collapse the spectra in the inertial range. The maximum gain factor is 2.6, but at the price of increasing the bottleneck.

```
&viscosity_run_pars
  ivisc='nu-slope-limited', 'nu-const'
  h_sld_visc=1.0
  nlf_sld_visc=3.0
  nu=1e-4
```

H.3 The bidiagonal scheme for cross-derivatives

The *old* default scheme used for cross-derivatives of type $\partial^2/(\partial x \partial y)$ used to read as follows:

```
df=fac*( &
    270.*( f(l1+1:l2+1,m+1,n,k)-f(l1-1:l2-1,m+1,n,k) &
           +f(l1-1:l2-1,m-1,n,k)-f(l1+1:l2+1,m-1,n,k)) &
    - 27.*( f(l1+2:l2+2,m+2,n,k)-f(l1-2:l2-2,m+2,n,k) &
           +f(l1-2:l2-2,m-2,n,k)-f(l1+2:l2+2,m-2,n,k)) &
    + 2.*( f(l1+3:l2+3,m+3,n,k)-f(l1-3:l2-3,m+3,n,k) &
           +f(l1-3:l2-3,m-3,n,k)-f(l1+3:l2+3,m-3,n,k)) &
    )
```

and is “visualized” in the left part of Fig. 36. It is way more efficient than the straightforward approach of first taking the x and the y derivative consecutively. (shown in the right part of Fig. 36).

-2	0	0	0	0	0	+2
0	+27	0	0	0	-27	0
0	0	-270	0	+270	0	0
0	0	0	0	0	0	0
0	0	+270	0	-270	0	0
0	-27	0	0	0	+27	0
+2	0	0	0	0	0	-2

9	-27	135	0	-135	27	-9
-27	81	-405	0	405	-81	27
135	-405	2025	0	-2025	405	-135
0	0	0	0	0	0	0
-135	405	-2025	0	2025	-405	135
27	-81	405	0	-405	81	-27
-9	27	-135	0	135	-27	9

Figure 36: Weights of bidiagonal scheme (left) and consecutive scheme (right) for mixed derivatives $\partial^2/\partial x \partial y$. The numbers shown need to be divided by $720 \delta x \delta y$ for the bidiagonal and by $3600 \delta x \delta y$ for the consecutive scheme.

Off-diagonal terms enter not only the diffusion terms through $\nabla \nabla \cdot \mathbf{u}$ and $\nabla \nabla \cdot \mathbf{A}$ terms, but also through the $\mathbf{J} = \nabla \times \nabla \times \mathbf{A}$ operator. The general formula is $J_i = A_{j,ij} - A_{i,jj}$, so in 2-D in the xy -plane we have

$$J_x = A_{x,xx} + A_{y,xy} - A_{x,xx} - A_{x,yy} = A_{y,xy} - A_{x,yy} , \quad (253)$$

$$J_y = A_{x,yx} + A_{y,yy} - A_{y,xx} - A_{y,yy} = A_{x,yx} - A_{y,xx} \quad (254)$$

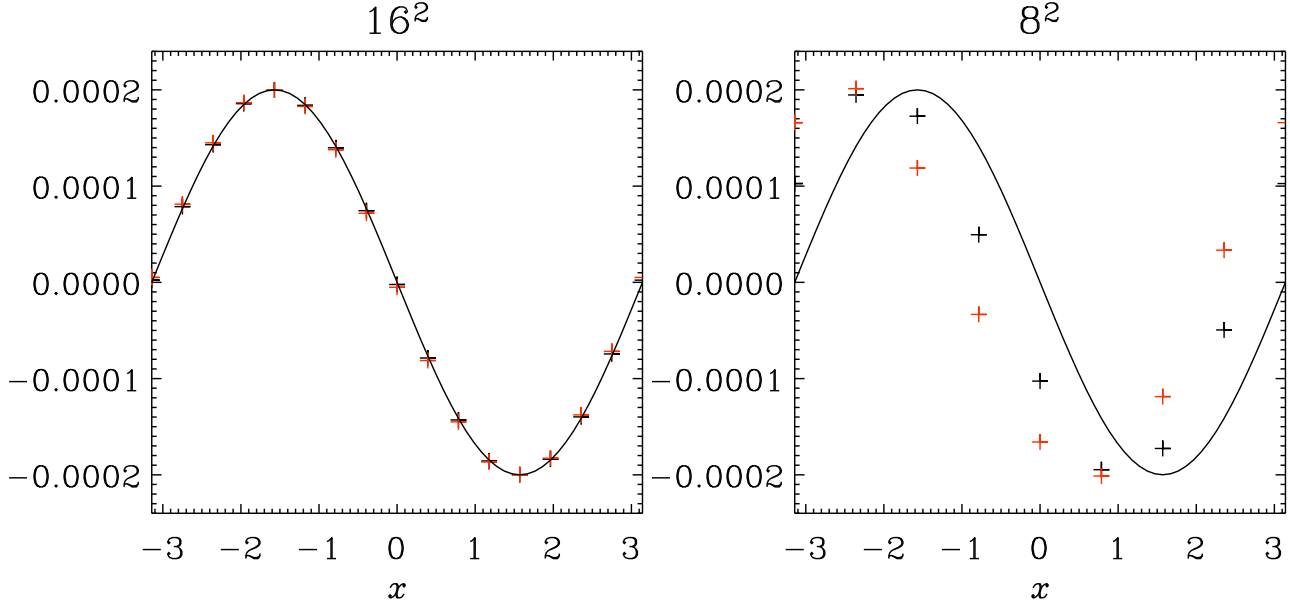


Figure 37: Alfvén wave for $\mathbf{B}_0 = (1, 2, 0)$ and $\mathbf{k} = (1, 2, 0)$ after $t = 2\pi$. The wave travels in the direction of $\mathbf{k}$. Red symbols are for the bidiagonal scheme, black symbols show results obtained using the consecutive scheme. Already for 16^2 mesh points there are no clear differences. For 8^2 mesh points both schemes are equally imprecise regarding the phase error, but the amplitude error is still quite small (but this is mainly a property of the time stepping scheme).

Figure 37 shows how the two schemes perform for the propagation of Alfvén waves,

$$\dot{u}_z = J_x B_{0y} - J_y B_{0x} , \quad (255)$$

$$\dot{A}_x = -u_z B_{0y} , \quad (256)$$

$$\dot{A}_y = +u_z B_{0x} . \quad (257)$$

The initial condition (as implemented in subroutine `alfven_xy`) is

$$u_z \sim \cos(k_x x + k_y y - \omega t) , \quad (258)$$

$$A_x \sim +B_{0y} \sin(k_x x + k_y y - \omega t)/\omega , \quad (259)$$

$$A_y \sim -B_{0x} \sin(k_x x + k_y y - \omega t)/\omega , \quad (260)$$

where $\omega = \mathbf{k} \cdot \mathbf{B}_0$. The figure shows that there is no clear advantage of either scheme, so the code uses the more efficient bidiagonal one.

H.4 The 2N-scheme for time-stepping

For time stepping, higher-order schemes are necessary in order to reduce the amplitude and phase errors of the scheme and, to some extent, to allow longer time steps. Usually

such schemes require large amounts of memory. However, we here use the memory-effective $2N$ -schemes that require only two sets of variables to be held in memory. Such schemes work for arbitrarily high order, although not all Runge-Kutta schemes can be written as $2N$ -schemes [41, 40]. Consider the ordinary differential equation (ODE)

$$\dot{u} = F(u, t) , \quad (261)$$

which can also be used as a prototype for a system of ODEs to be solved, like the ones obtained by spatial discretization of PDEs. The $2N$ -schemes construct an approximation to the solution

$$u^{(n)} \equiv u(t_n) \quad (262)$$

according to the iterative procedure

$$w_i = \alpha_i w_{i-1} + \delta t F(u_{i-1}, t_{i-1}) , \quad (263)$$

$$u_i = u_{i-1} + \beta_i w_i . \quad (264)$$

For a three-step (RK3-2N) scheme we have $i = 1, \dots, 3$. In order to advance the variable u from $u^{(n)}$ at time $t^{(n)}$ to $u^{(n+1)}$ at time $t^{(n+1)} = t^{(n)} + \delta t$ we set in Eq. (264)

$$u_0 = u^{(n)} \quad \text{and, after the last step,} \quad u^{(n+1)} = u_3, \quad (265)$$

with u_1 and u_2 being intermediate steps. In order to be able to calculate the first step, $i = 1$, for which no $w_{i-1} \equiv w_0$ exists, we have to require $\alpha_1 = 0$. Thus, we are left with 5 unknowns, $\alpha_2, \alpha_3, \beta_1, \beta_2$, and β_3 . Three conditions follow from the fact that the scheme be third order for linear equations, so we have to have two more conditions. One possibility is to choose the fractional times at which the right hand side is evaluated, for example $(0, 1/3, 2/3)$ or even $(0, 1/2, 1)$. Yet another possibility is to require that inhomogeneous equations of the form $\dot{u} = t^n$ with $n = 1$ and 2 are solved exactly. The corresponding coefficients are listed in Table 17 and compared with those given by Williamson [41]. In practice all of them are about equally good when it comes to real applications, although we found the first one in Table 17 ('symmetric') marginally better in some simple test problems where an analytic solution was known. In Ref. [7] the accuracy of some non-linear equations is tested.

Table 17: Coefficients for different $2N$ -type third-order Runge-Kutta schemes. The coefficients c_i (which are determined by the α_i, β_i) give the time for each substep, $t_i = t_0 + c_i \delta t$

scheme	c_1	c_2	c_3	α_2	α_3	β_1	β_2	β_3
symmetric	0	1/3	2/3	-2/3	-1	1/3	1	1/2
[predictor/corrector]	0	1/2	1	-1/4	-4/3	1/2	2/3	1/2
inhomogeneous	0	15/32	4/9	-17/32	-32/27	1/4	8/9	3/4
Williamson (1980)	0	4/9	15/32	-5/9	-153/128	1/3	15/16	8/15

H.5 Diffusive error from the time-stepping

In many cases we use centered first derivatives for the advection operator, so the resulting discretization errors are only of dispersive nature (proportional to odd derivatives). A diffusive error can be obtained from the discretization error of the time-stepping scheme. For the RK3-2N scheme we have

$$\left(\frac{df}{dt} \right)_{nth \text{ order}} = \left(\frac{df}{dt} \right)_{\text{exact}} + a_n \delta t^n \left(\frac{d^{n+1}f}{dt^{n+1}} \right) + \dots, \quad (266)$$

where $a_n = 1/(n+1)! = 0.5$. In particular, for $n = 1$ we have $a_1 = 1/2 = 0.2$ and for $n = 3$ we have $a_3 = 1/24 \approx 0.04$. The advection operator leads to a diffusive error equal to $a_1 \delta t (\mathbf{u} \cdot \nabla)^2$ for $n = 1$ and a hyperdiffusive error equal to $a_3 \delta t^3 (\mathbf{u} \cdot \nabla)^4$ for $n = 3$. Substituting $\delta t = c_{\text{CFL}} \delta x / |\mathbf{u}|$, where c_{CFL} is Courant-Friedrich-Lewy constant, we have a diffusive error $\nu \nabla^2$ with negative $\nu = -a_1 c_{\text{CFL}} |\mathbf{u}| \delta x$ for $n = 1$, and a hyperdiffusive error $-\nu_{\text{hyp}} \nabla^4$ with positive $\nu_{\text{hyp}} = a_3 c_{\text{CFL}}^3 |\mathbf{u}| \delta x^3$ for $n = 3$. The fact that the hyperdiffusive error has a positive effective hyperdiffusivity is an important factor in the choice of this scheme.

To decide whether the effective hyperdiffusivity from the diffusive error is significant, we can compare with the error that *would* occur had we used a third-order upwinding scheme (Sect. H.2). In that case we would have an effective hyperdiffusive coefficient $|\mathbf{u}| \delta x / 12$ that is $1/(12a_3 c_{\text{CFL}}^3) \approx 5.8$ times larger than that from the time stepping scheme. In this sense, the hyperdiffusive error can be regarded as small.

Since the hyperdiffusive error is proportional to $-\nabla^4$, we cannot directly compare with the physical diffusion which is proportional to ∇^2 . Therefore we define an effective viscosity as $\nu_{\text{eff}} = \nu_{\text{hyp}} k_{\text{Ny}}^2$ with $k_{\text{Ny}} = \pi/\delta x$ being the Nyquist wavenumber of the mesh of the domain covered by N mesh points. We define Reynolds number based on the Nyquist wavenumber as $\text{Re}_{\text{Ny}} = |\mathbf{u}|/\nu_{\text{eff}} k_{\text{Ny}}$, and find $\text{Re} = -24/(\pi c_{\text{CFL}})^3 \approx 2.3$ for our favorite choice $c_{\text{CFL}} = 0.7$. Thus, at the scale of the mesh, the effective Reynolds number is comparable to the value often obtained in simulations. However, in turbulence simulations the viscous cutoff wavenumber is usually 5–10 times smaller than k_{Ny} , so the relevant Reynolds number at the viscous scale is then another 2–3 orders of magnitude larger and does therefore not impose a constraint in view of the physical viscosity that is applied in such calculations.

H.6 Ionization

The specific entropy of each particle species (neutral hydrogen, electrons, protons and neutral helium) may be written as

$$\frac{s_i}{s_0} = x_i \left(\ln \left[\frac{1}{x_{\text{tot}}} \frac{\rho_i}{\rho} \left(\frac{T}{T_0} \right)^{3/2} \right] + \frac{5}{2} \right), \quad (267)$$

where

$$x_{\text{H}} = 1 - y, \quad x_{\text{e}} = x_{\text{p}} = y, \quad x_{\text{tot}} = 1 + y + x_{\text{He}} \quad (268)$$

$$s_0 = \frac{k_{\text{B}}}{\mu m_{\text{H}}}, \quad T_0 = \frac{\chi_{\text{H}}}{k_{\text{B}}}, \quad (269)$$

and

$$\rho_i = \mu m_{\text{H}} \left(\frac{m_i \chi_{\text{H}}}{2\pi \hbar^2} \right)^{3/2}. \quad (270)$$

The specific entropy of mixing is

$$\frac{s_{\text{mix}}}{s_0} = - \sum_i x_i \ln \frac{x_i}{x_{\text{tot}}}. \quad (271)$$

Summing up everything, we get the total specific entropy

$$\frac{s}{s_0} = \sum_i \frac{s_i}{s_0} + \frac{s_{\text{mix}}}{s_0} = \sum_i x_i \left(\ln \left[\frac{1}{x_i} \frac{\rho_i}{\rho} \left(\frac{T}{T_0} \right)^{3/2} \right] + \frac{5}{2} \right) \quad (272)$$

$$= \sum_i x_i \ln \frac{\rho_i}{x_i} + x_{\text{tot}} \left(\ln \left[\frac{1}{\rho} \left(\frac{T}{T_0} \right)^{3/2} \right] + \frac{5}{2} \right). \quad (273)$$

Solving for T gives

$$\frac{3}{2} \ln \frac{T}{T_0} = \frac{s/s_0 + \sum_i x_i \ln x_i / \rho_i}{x_{\text{tot}}} + \ln \rho - \frac{5}{2}. \quad (274)$$

Using this expression and the constants defined above, we may obtain the ionization fraction y for given $\ln \rho$ and s by finding the root of

$$F = \ln \left[\frac{\rho_e}{\rho} \left(\frac{T}{T_0} \right)^{3/2} \frac{1-y}{y^2} \right] - \frac{T_0}{T}. \quad (275)$$

The derivative with respect to y for Newton-Raphson is

$$\frac{\partial F}{\partial y} = \left(\frac{3}{2} + \frac{T_0}{T} \right) \frac{\partial \ln T/T_0}{\partial y} - \frac{1}{1-y} - \frac{2}{y}, \quad (276)$$

where

$$\frac{\partial \ln T/T_0}{\partial y} = \frac{\frac{2}{3} (\ln \rho_H / \rho_P - F - T_0/T) - 1}{1 + y + x_{\text{He}}}. \quad (277)$$

In order to compute the pressure gradient in the momentum equation, the derivative of y with respect to $\ln \rho$ and s needs to be known:

$$\frac{\partial \ln P}{\partial \ln \rho} = \frac{1}{1 + y + x_{\text{He}}} \frac{\partial y}{\partial \ln \rho} + \frac{\partial \ln T}{\partial \ln \rho} + \frac{\partial \ln T}{\partial y} \frac{\partial y}{\partial \ln \rho} + 1, \quad (278)$$

$$\frac{\partial \ln P}{\partial s} = \frac{1}{1 + y + x_{\text{He}}} \frac{\partial y}{\partial s} + \frac{\partial \ln T}{\partial s} + \frac{\partial \ln T}{\partial y} \frac{\partial y}{\partial s}. \quad (279)$$

Since $F = 0$ for all desired solutions $(y, \ln \rho, s)$ we also have

$$dF = \frac{\partial F}{\partial \ln \rho} d \ln \rho + \frac{\partial F}{\partial s} ds + \frac{\partial F}{\partial y} dy = 0, \quad (280)$$

and thus

$$\frac{\partial y}{\partial \ln \rho} = \left(\frac{dy}{d \ln \rho} \right)_{ds=0} = - \frac{\partial F / \partial \ln \rho}{\partial F / \partial y} \quad (281)$$

and

$$\frac{\partial y}{\partial s} = \left(\frac{dy}{ds} \right)_{d \ln \rho=0} = - \frac{\partial F / \partial s}{\partial F / \partial y}. \quad (282)$$

H.7 Radiative transfer

H.7.1 Solving the radiative transfer equation

A formal solution of Eq. (83) is given by

$$I(\tau) = I(\tau_0) e^{-(\tau-\tau_0)} + \int_{\tau_0}^{\tau} e^{-(\tau-\tau')} S(\tau') d\tau'. \quad (283)$$

Using a generalization of the trapezoidal rule,

$$\int_{\tau_0}^{\tau} e^{-(\tau-\tau')} f(\tau') d\tau' \approx \int_{\tau_0}^{\tau} e^{-(\tau-\tau')} \left[f(\tau_0) + \frac{f(\tau) - f(\tau_0)}{\tau - \tau_0} (\tau' - \tau_0) \right] d\tau' \quad (284)$$

$$= [1 - e^{-(\tau-\tau_0)}] f(\tau) - \frac{1 - e^{-(\tau-\tau_0)} (1 + \tau - \tau_0)}{\tau - \tau_0} [f(\tau) - f(\tau_0)], \quad (285)$$

which is exact for linear functions $S(\tau)$, we discretize this as

$$I_{k+1} = I_k e^{-\delta\tau} + (1 - e^{-\delta\tau}) S_{k+1} - \frac{1 - e^{-\delta\tau}(1 + \delta\tau)}{\delta\tau} (S_{k+1} - S_k), \quad (286)$$

$$= I_k e^{-\delta\tau} + (1 - e^{-\delta\tau}) S_k + \frac{e^{-\delta\tau} - 1 + \delta\tau}{\delta\tau} (S_{k+1} - S_k). \quad (287)$$

Here the simplest way to calculate $\delta\tau$ is

$$\delta\tau = \frac{\chi_k + \chi_{k+1}}{2} \delta x; \quad (288)$$

more accurate alternatives are

$$\delta\tau = \sqrt{\chi_k \chi_{k+1}} \delta x \quad (289)$$

or

$$\delta\tau = \frac{\chi_{k+1} - \chi_k}{\ln \frac{\chi_{k+1}}{\chi_k}} \delta x. \quad (290)$$

H.7.2 Angular integration

Table 18: Sums $\sqrt{4\pi} Y_l^m(\theta_i, \phi_i)$ for special sets of directions. For all degrees and orders up to $l = 8$ not mentioned in this table, the sums are 0. The label ‘Non-h. f-d.’ stands for ‘non-horizontal face-diagonals’, i.e. the eight face diagonals that are not in the horizontal plane.

Directions	Y_0^0	Y_2^0	Y_4^0	$Y_4^{\pm 4}$	Y_6^0	$Y_6^{\pm 4}$	Y_8^0	$Y_8^{\pm 4}$	$Y_8^{\pm 8}$
Coord.	6	0	$\frac{21}{2}$	$\frac{3}{4}\sqrt{70}$	$\frac{3}{4}\sqrt{13}$	$-\frac{3}{8}\sqrt{182}$	$\frac{99}{32}\sqrt{17}$	$\frac{3}{32}\sqrt{2618}$	$\frac{3}{64}\sqrt{24310}$
Face diag.	12	0	$-\frac{21}{4}$	$-\frac{3}{8}\sqrt{70}$	$-\frac{39}{16}\sqrt{13}$	$\frac{39}{32}\sqrt{182}$	$\frac{891}{256}\sqrt{17}$	$\frac{27}{256}\sqrt{2618}$	$\frac{27}{512}\sqrt{24310}$
Space diag.	8	0	$-\frac{28}{3}$	$-\frac{2}{3}\sqrt{70}$	$\frac{16}{9}\sqrt{13}$	$-\frac{8}{9}\sqrt{182}$	$\frac{11}{9}\sqrt{17}$	$\frac{1}{27}\sqrt{2618}$	$\frac{1}{54}\sqrt{24310}$
Non-h. f-d.	8	$2\sqrt{5}$	$-\frac{39}{4}$	$\frac{3}{8}\sqrt{70}$	$-\frac{19}{16}\sqrt{13}$	$\frac{27}{32}\sqrt{182}$	$\frac{611}{256}\sqrt{17}$	$\frac{51}{256}\sqrt{2618}$	$\frac{3}{512}\sqrt{24310}$
Coord. x, y	4	$-2\sqrt{5}$	$\frac{9}{2}$	$\frac{4}{3}\sqrt{70}$	$-\frac{5}{4}\sqrt{13}$	$-\frac{3}{8}\sqrt{182}$	$\frac{35}{32}\sqrt{17}$	$\frac{3}{32}\sqrt{2618}$	$\frac{3}{64}\sqrt{24310}$
Coord. z	2	$2\sqrt{5}$	6	0	$2\sqrt{13}$	0	$2\sqrt{17}$	0	0

For angular integration over the full solid angle, we make the ansatz

$$\int_{4\pi} f(\theta, \phi) \frac{d\omega}{4\pi} = \sum_{i=1}^N w_i f(\theta_i, \phi_i) + R_N. \quad (291)$$

Table 18 shows the sums $\sqrt{4\pi} Y_l^m(\theta_i, \phi_i)$ over special sets of directions (θ_i, ϕ_i) . Using these numbers and requiring that angular integration is exact for $l \leq l_{\max}$, we find the following weights w_i for different sets of directions (see also [1], §25.4.65).²³

Cooling times have been determined numerically in [4]. Comparing with analytic expressions obtained in the Eddington approximation, the proposed suitable switches in 1-D and 2-D problems.

²³

1. Axes

Coordinate axes: 1/6

$$l_{\max} = 3$$

2. *Face diagonals*

Face diagonals: $1/12$

$$l_{\max} = 3$$

3. *Space diagonals*

Space diagonals: $1/8$

$$l_{\max} = 3$$

4. *Axes + face diagonals*

Coordinate axes: $1/30$

Face diagonals: $1/15$

$$l_{\max} = 5$$

5. *Axes + space diagonals*

Coordinate axes: $1/15$

Space diagonals: $3/40$

$$l_{\max} = 5$$

6. *Face + space diagonals*

Face diagonals: $2/15$

Space diagonals: $-3/40$

$$l_{\max} = 5$$

7. *Axes, face + space diagonals*

Coordinate axes: $1/21$

Face diagonals: $4/105$

Space diagonals: $9/280$

$$l_{\max} = 7$$

8. *Axes, non-horizontal face diagonals*

Coordinate axes x, y : $1/10$

Coordinate axes z : $1/30$

Non-hor. face diagonals: $1/15$

$$l_{\max} = 3$$

9. *Axes, non-horizontal face diagonals, space diagonals*

Coordinate axes x, y : $12/215$

Coordinate axes z : $10/129$

Non-hor. face diagonals: $-14/645$

Space diagonals: $171/1720$

$$l_{\max} = 5$$

I Curvilinear coordinates

The use and implementation of non-cartesian coordinate systems is briefly explained in Section 5.27. All differential operators look like their cartesian counterparts, except that all derivatives are now replaced by covariant derivatives. The relevant reference for the PENCIL CODE is [35]; see their Appendix B. Here some details.

I.1 Covariant derivatives

$$A_{\hat{\alpha};\hat{\beta}} = A_{\hat{\alpha},\hat{\beta}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\beta}} A_{\hat{\sigma}}, \quad (292)$$

$$A_{\hat{\alpha}\hat{\beta};\hat{\gamma}} = A_{\hat{\alpha}\hat{\beta},\hat{\gamma}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\gamma}} A_{\hat{\sigma}\hat{\beta}} - \Gamma^{\hat{\sigma}}_{\hat{\beta}\hat{\gamma}} A_{\hat{\alpha}\hat{\sigma}}. \quad (293)$$

Second derivative tensor

$$\begin{aligned} A_{\hat{\alpha};\hat{\beta}\hat{\gamma}} &= A_{\hat{\alpha},\hat{\beta},\hat{\gamma}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\gamma}} A_{\hat{\sigma};\hat{\beta}} - \Gamma^{\hat{\sigma}}_{\hat{\beta}\hat{\gamma}} A_{\hat{\alpha};\hat{\sigma}} \\ &= A_{\hat{\alpha},\hat{\beta},\hat{\gamma}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\beta}} A_{\hat{\sigma},\hat{\gamma}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\gamma}} A_{\hat{\sigma},\hat{\beta}} - \Gamma^{\hat{\sigma}}_{\hat{\beta}\hat{\gamma}} A_{\hat{\alpha},\hat{\sigma}} + \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\gamma}} \Gamma^{\hat{\nu}}_{\hat{\sigma}\hat{\beta}} A_{\hat{\nu}} - \Gamma^{\hat{\sigma}}_{\hat{\beta}\hat{\gamma}} \Gamma^{\hat{\nu}}_{\hat{\sigma}\hat{\alpha}} A_{\hat{\nu}}. \end{aligned}$$

Elements of the first derivative tensor

$$A_{\hat{r};\hat{r}} = A_{\hat{r},\hat{r}}, \quad A_{\hat{\theta};\hat{\theta}} = A_{\hat{\theta},\hat{\theta}} + r^{-1} A_{\hat{r}}, \quad A_{\hat{\phi};\hat{\phi}} = A_{\hat{\phi},\hat{\phi}} + r^{-1} A_{\hat{r}} + r^{-1} \cot\theta A_{\hat{\theta}}. \quad (294)$$

$$\begin{aligned} A_{\hat{\phi};\hat{\theta}} &= A_{\hat{\phi},\hat{\theta}} & A_{\hat{\theta};\hat{\phi}} &= A_{\hat{\theta},\hat{\phi}} - r^{-1} \cot\theta A_{\hat{\phi}} \\ A_{\hat{r};\hat{\phi}} &= A_{\hat{r},\hat{\phi}} - r^{-1} A_{\hat{\phi}} & A_{\hat{\phi};\hat{r}} &= A_{\hat{\phi},\hat{r}} \\ A_{\hat{\theta};\hat{r}} &= A_{\hat{\theta},\hat{r}} & A_{\hat{r};\hat{\theta}} &= A_{\hat{r},\hat{\theta}} - r^{-1} A_{\hat{\theta}} \end{aligned} \quad (295)$$

I.2 Differential operators

All differential operators look like their cartesian counterparts, except that all derivatives are now replaced by covariant derivatives.

I.2.1 Gradient

For the gradient operator the covariant and partial derivatives are the same, i.e.

$$\nabla\Psi = \Psi_{;\hat{\alpha}} = \Psi_{,\hat{\alpha}} = \begin{pmatrix} \partial_{\hat{r}}\Psi \\ \partial_{\hat{\theta}}\Psi \\ \partial_{\hat{\phi}}\Psi \end{pmatrix} \quad (296)$$

where

$$\partial_{\hat{r}} = \partial_r \quad (297)$$

$$\partial_{\hat{\theta}} = r^{-1} \partial_{\theta} \quad (298)$$

$$\partial_{\hat{\phi}} = \varpi^{-1} \partial_{\phi} \quad (299)$$

and $\varpi = r \sin\theta$ is the cylindrical radius. Thus,

$$\nabla\Psi = (\nabla\Psi)^{(0)}, \quad (300)$$

where the superscript (0) indicated the straightforward calculation in the non-coordinate basis. The coordinate and non-coordinate bases are related to each other via

$$(\nabla\Psi)^0 \equiv \begin{pmatrix} \Psi_{,\hat{r}} \\ \Psi_{,\hat{\theta}} \\ \Psi_{,\hat{\phi}} \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & r^{-1} & 0 \\ 0 & 0 & \varpi^{-1} \end{pmatrix} (\nabla\Psi)^{(\text{coord})}. \quad (301)$$

Here, the result in the coordinate basis is just what one would get by computing as if one had cartesian coordinates. In the PENCIL CODE the output of the subroutine `der` is now in this non-coordinate basis.

I.2.2 Divergence

For the divergence operator we have a ‘correction term’, i.e.

$$\nabla \cdot \mathbf{A} = A^{\hat{\alpha}}_{;\hat{\alpha}} \quad (302)$$

$$= A^{\hat{\alpha}}_{,\hat{\alpha}} + \Gamma^{\hat{\alpha}}_{\hat{\beta}\hat{\alpha}} A^{\hat{\beta}}, \quad (303)$$

where the only non-vanishing contributions to the last term are

$$\Gamma^{\hat{\alpha}}_{\hat{\beta}\hat{\alpha}} A^{\hat{\beta}} = \Gamma^{\hat{\theta}}_{\hat{r}\hat{\theta}} A^{\hat{r}} + \Gamma^{\hat{\phi}}_{\hat{r}\hat{\phi}} A^{\hat{r}} + \Gamma^{\hat{\phi}}_{\hat{\theta}\hat{\phi}} A^{\hat{\theta}} \quad (304)$$

$$= 2r^{-1} A^{\hat{r}} + r^{-1} \cot \theta A^{\hat{\theta}}. \quad (305)$$

Thus,

$$\nabla \cdot \mathbf{A} = (\nabla \cdot \mathbf{A})^{(0)} + M_{\hat{\alpha}}^{(\text{div})} A_{\hat{\alpha}}, \quad (306)$$

where

$$M_{\hat{\alpha}}^{(\text{div})} = \begin{pmatrix} 2r^{-1} \\ r^{-1} \cot \theta \\ 0 \end{pmatrix} \quad (307)$$

represents the correction term.

Alternatively:

$$A_{\hat{\alpha};\hat{\alpha}} = A_{\hat{\alpha},\hat{\alpha}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\alpha}} A_{\hat{\sigma}} \quad (308)$$

$$= A_{\hat{\alpha},\hat{\alpha}} + 2r^{-1} A_{\hat{r}} + r^{-1} \cot \theta A_{\hat{\theta}}. \quad (309)$$

I.2.3 Curl

The curl operator is given by

$$(\nabla \times \mathbf{B})^{\hat{\alpha}} = \epsilon^{\hat{\alpha}\hat{\beta}\hat{\gamma}} B_{\hat{\gamma},\hat{\beta}}. \quad (310)$$

So

$$\nabla \times \mathbf{A} = \begin{pmatrix} A_{\hat{\phi},\hat{\theta}} - A_{\hat{\theta},\hat{\phi}} \\ A_{\hat{r},\hat{\phi}} - A_{\hat{\phi},\hat{r}} \\ A_{\hat{\theta},\hat{r}} - A_{\hat{r},\hat{\theta}} \end{pmatrix} \quad (311)$$

$$\nabla \times \mathbf{A} = \begin{pmatrix} A_{\hat{\phi},\hat{\theta}} - A_{\hat{\theta},\hat{\phi}} + \Gamma^{\hat{\phi}}_{\hat{\theta}\hat{\phi}} A^{\hat{\phi}} \\ A_{\hat{r},\hat{\phi}} - A_{\hat{\phi},\hat{r}} - \Gamma^{\hat{\phi}}_{\hat{r}\hat{\phi}} A^{\hat{\phi}} \\ A_{\hat{\theta},\hat{r}} - A_{\hat{r},\hat{\theta}} + \Gamma^{\hat{\theta}}_{\hat{r}\hat{\theta}} A^{\hat{\theta}} \end{pmatrix} \quad (312)$$

Thus,

$$\nabla \times \mathbf{A} = (\nabla \times \mathbf{A})^{(0)} + M_{\hat{\alpha}\hat{\beta}}^{(\text{curl})} A_{\hat{\beta}}, \quad (313)$$

where

$$M_{\hat{\alpha}\hat{\beta}}^{(\text{curl})} = \begin{pmatrix} 0 & 0 & r^{-1} \cot \theta \\ 0 & 0 & -r^{-1} \\ 0 & r^{-1} & 0 \end{pmatrix} \quad (314)$$

is the correction term. In the PENCIL CODE we use the subroutine `curl_mn(aij,bb,aa)`, which uses `aij` = $A_{\hat{\alpha},\hat{\beta}}$ and `aa` = $A_{\hat{\alpha}}$ as input pencils and produces `bb` = $B_{\hat{\alpha}}$ as output.

I.2.4 Advection operator

[The usage of roman indices here is insignificant.]

$$\mathbf{u} \cdot \nabla \mathbf{u} = \boldsymbol{\omega} \times \mathbf{u} + \frac{1}{2} \nabla \mathbf{u}^2 = -\mathbf{u} \times \boldsymbol{\omega} + \frac{1}{2} \nabla \mathbf{u}^2 \quad (315)$$

$$\begin{aligned} (\mathbf{u} \cdot \nabla \mathbf{u})_i &= -\epsilon_{ijk} \epsilon_{klm} u_j u_{m;l} + u_j u_{j,i} \\ &= (-\delta_{il} \delta_{jm} + \delta_{im} \delta_{jl}) u_j u_{m;l} + u_j u_{j,i} \\ &= -u_j u_{j;i} + u_j u_{i;j} + u_j u_{j,i} \\ &= u_j u_{i;j} + \Gamma_{ji}^k u_j u_k \\ &= u_j u_{i,j} + (-\Gamma_{ij}^k + \Gamma_{ji}^k) u_j u_k \end{aligned} \quad (316)$$

Note that the terms with $\Gamma_{\hat{\theta}\hat{\theta}}^{\hat{r}}$, $\Gamma_{\hat{\phi}\hat{\phi}}^{\hat{r}}$, and $\Gamma_{\hat{\phi}\hat{\phi}}^{\hat{\theta}}$ drop out. Thus, we have

$$\begin{aligned} (\mathbf{u} \cdot \nabla \mathbf{u})_{\hat{r}} &= u_j u_{\hat{r},j} + (-\Gamma_{\hat{r}j}^k + \Gamma_{j\hat{r}}^k) u_j u_k \\ &= u_j u_{\hat{r},j} - \Gamma_{\hat{r}\hat{\theta}}^{\hat{\theta}} u_{\hat{\theta}}^2 - \Gamma_{\hat{r}\hat{\phi}}^{\hat{\phi}} u_{\hat{\phi}}^2 \\ &= u_j u_{\hat{r},j} - r^{-1} (u_{\hat{\theta}}^2 + u_{\hat{\phi}}^2) \end{aligned} \quad (317)$$

$$\begin{aligned} (\mathbf{u} \cdot \nabla \mathbf{u})_{\hat{\theta}} &= u_j u_{\hat{\theta},j} + (-\Gamma_{\hat{\theta}j}^k + \Gamma_{j\hat{\theta}}^k) u_j u_k \\ &= u_j u_{\hat{\theta},j} - \Gamma_{\hat{\theta}\hat{\phi}}^{\hat{\phi}} u_{\hat{\phi}}^2 + \Gamma_{\hat{\theta}\hat{r}}^{\hat{r}} u_{\hat{r}} u_{\hat{\theta}} \\ &= u_j u_{\hat{\theta},j} - r^{-1} \cot \theta u_{\hat{\phi}}^2 + r^{-1} u_{\hat{r}} u_{\hat{\theta}} \end{aligned} \quad (318)$$

$$\begin{aligned} (\mathbf{u} \cdot \nabla \mathbf{u})_{\hat{\phi}} &= u_j u_{\hat{\phi},j} + (-\Gamma_{\hat{\phi}j}^k + \Gamma_{j\hat{\phi}}^k) u_j u_k \\ &= u_j u_{\hat{\phi},j} + \Gamma_{\hat{\phi}\hat{r}}^{\hat{r}} u_{\hat{r}} u_{\hat{\phi}} + \Gamma_{\hat{\phi}\hat{\theta}}^{\hat{\theta}} u_{\hat{\theta}} u_{\hat{\phi}} \\ &= u_j u_{\hat{\phi},j} + r^{-1} u_{\hat{r}} u_{\hat{\phi}} + r^{-1} \cot \theta u_{\hat{\theta}} u_{\hat{\phi}} \end{aligned} \quad (319)$$

Note that the formulation above is slightly misleading, because

$$\Gamma_{j\hat{\theta}}^k u_j u_k = \Gamma_{\hat{\theta}\hat{\theta}}^{\hat{r}} u_{\hat{\theta}} u_{\hat{r}} + \Gamma_{\hat{r}\hat{\theta}}^{\hat{\theta}} u_{\hat{r}} u_{\hat{\theta}} = 0 \quad (320)$$

$$\Gamma_{j\hat{\phi}}^k u_j u_k = \Gamma_{\hat{\phi}\hat{\phi}}^{\hat{r}} u_{\hat{\phi}} u_{\hat{r}} + \Gamma_{\hat{r}\hat{\phi}}^{\hat{\phi}} u_{\hat{r}} u_{\hat{\phi}} + \Gamma_{\hat{\phi}\hat{\theta}}^{\hat{\theta}} u_{\hat{\phi}} u_{\hat{\theta}} + \Gamma_{\hat{\theta}\hat{\phi}}^{\hat{\phi}} u_{\hat{\theta}} u_{\hat{\phi}} = 0 \quad (321)$$

I.2.5 Mixed advection operator

$$\mathbf{u} \times \mathbf{B} = \mathbf{u} \times \nabla \mathbf{A} = u_j \epsilon_{ijk} \epsilon_{klm} A_{m;l} = u_j (\delta_{il} \delta_{jm} - \delta_{im} \delta_{jl}) A_{m;l} = u_j A_{j;i} - u_j A_{i;j} \quad (322)$$

I.2.6 Shear term

$$u_j A_{j;i} = (u_j A_j)_{;i} - u_{j;i} A_j = (u_j A_j)_{;i} - u_{j;i} A_j \quad (323)$$

$$u_{j;i} A_j = u_{j,i} A_j - \Gamma_{ji}^k u_k A_j \quad (324)$$

So

$$u_{\hat{\phi};\hat{r}} A_{\hat{\phi}} = u_{\hat{\phi},\hat{r}} A_{\hat{\phi}} - \Gamma_{\hat{\phi}\hat{r}}^k u_k A_{\hat{\phi}} = u_{\hat{\phi},\hat{r}} A_{\hat{\phi}} \quad (325)$$

$$u_{\hat{r};\hat{\phi}} A_{\hat{r}} = u_{\hat{r},\hat{\phi}} A_{\hat{r}} - \Gamma_{\hat{r}\hat{\phi}}^{\hat{\phi}} u_{\hat{\phi}} A_{\hat{r}} \quad (326)$$

I.2.7 Another mixed advection operator

$$u_{\hat{\beta}} A_{\hat{\alpha};\hat{\beta}} = u_{\hat{\beta}} A_{\hat{\alpha},\hat{\beta}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\beta}} u_{\hat{\beta}} A_{\hat{\sigma}}. \quad (327)$$

Write out

$$\begin{aligned} (\mathbf{u} \cdot \nabla \mathbf{A})_{\hat{r}} &= u_{\hat{r}} A_{\hat{r};\hat{r}} + u_{\hat{\theta}} A_{\hat{r};\hat{\theta}} + u_{\hat{\phi}} A_{\hat{r};\hat{\phi}} = u_{\hat{r}} A_{\hat{r},\hat{r}} + u_{\hat{\theta}} A_{\hat{r},\hat{\theta}} - r^{-1} u_{\hat{\theta}} A_{\hat{\theta}} + u_{\hat{\phi}} A_{\hat{r},\hat{\phi}} - r^{-1} u_{\hat{\phi}} A_{\hat{\phi}} \\ (\mathbf{u} \cdot \nabla \mathbf{A})_{\hat{\theta}} &= u_{\hat{r}} A_{\hat{\theta};\hat{r}} + u_{\hat{\theta}} A_{\hat{\theta};\hat{\theta}} + u_{\hat{\phi}} A_{\hat{\theta};\hat{\phi}} = u_{\hat{r}} A_{\hat{\theta},\hat{r}} + u_{\hat{\theta}} A_{\hat{\theta},\hat{\theta}} + r^{-1} u_{\hat{\theta}} A_{\hat{r}} + u_{\hat{\phi}} A_{\hat{\theta},\hat{\phi}} - r^{-1} \cot\theta u_{\hat{\phi}} A_{\hat{\phi}} \\ (\mathbf{u} \cdot \nabla \mathbf{A})_{\hat{\phi}} &= u_{\hat{r}} A_{\hat{\phi};\hat{r}} + u_{\hat{\theta}} A_{\hat{\phi};\hat{\theta}} + u_{\hat{\phi}} A_{\hat{\phi};\hat{\phi}} = u_{\hat{r}} A_{\hat{\phi},\hat{r}} + u_{\hat{\theta}} A_{\hat{\phi},\hat{\theta}} + u_{\hat{\phi}} A_{\hat{\phi},\hat{\phi}} + r^{-1} u_{\hat{\phi}} A_{\hat{r}} + r^{-1} \cot\theta u_{\hat{\phi}} A_{\hat{\theta}}. \end{aligned}$$

Reorder

$$\begin{aligned} (\mathbf{u} \cdot \nabla \mathbf{A})_{\hat{r}} &= u_{\hat{r}} A_{\hat{r},\hat{r}} + u_{\hat{\theta}} A_{\hat{r},\hat{\theta}} + u_{\hat{\phi}} A_{\hat{r},\hat{\phi}} - r^{-1} u_{\hat{\theta}} A_{\hat{\theta}} - r^{-1} u_{\hat{\phi}} A_{\hat{\phi}} \\ (\mathbf{u} \cdot \nabla \mathbf{A})_{\hat{\theta}} &= u_{\hat{r}} A_{\hat{\theta},\hat{r}} + u_{\hat{\theta}} A_{\hat{\theta},\hat{\theta}} + u_{\hat{\phi}} A_{\hat{\theta},\hat{\phi}} + r^{-1} u_{\hat{\theta}} A_{\hat{r}} - r^{-1} \cot\theta u_{\hat{\phi}} A_{\hat{\phi}} \\ (\mathbf{u} \cdot \nabla \mathbf{A})_{\hat{\phi}} &= u_{\hat{r}} A_{\hat{\phi},\hat{r}} + u_{\hat{\theta}} A_{\hat{\phi},\hat{\theta}} + u_{\hat{\phi}} A_{\hat{\phi},\hat{\phi}} + r^{-1} u_{\hat{\phi}} A_{\hat{r}} + r^{-1} \cot\theta u_{\hat{\phi}} A_{\hat{\theta}}. \end{aligned} \quad (328)$$

I.2.8 Strain Matrix

The strain matrix takes the form $2s_{\hat{\alpha}\hat{\beta}} = u_{\hat{\alpha};\hat{\beta}} + u_{\hat{\beta};\hat{\alpha}}$. The components of the covariant derivative tensor were given in Eqs. (294) and (295), so the final result reads

$$2s \equiv \begin{pmatrix} 2u_{\hat{r},\hat{r}} & u_{\hat{r},\hat{\theta}} + u_{\hat{\theta},\hat{r}} - u_{\hat{\theta}}/r & u_{\hat{r},\hat{\phi}} + u_{\hat{\phi},\hat{r}} - u_{\hat{\phi}}/r \\ u_{\hat{r},\hat{\theta}} + u_{\hat{\theta},\hat{r}} - u_{\hat{\theta}}/r & 2u_{\hat{\theta},\hat{\theta}} + 2u_{\hat{r}}/r & u_{\hat{\theta},\hat{\phi}} + u_{\hat{\phi},\hat{\theta}} - u_{\hat{\phi}} \frac{\cot\theta}{r} \\ u_{\hat{r},\hat{\phi}} + u_{\hat{\phi},\hat{r}} - u_{\hat{\phi}}/r & u_{\hat{\theta},\hat{\phi}} + u_{\hat{\phi},\hat{\theta}} - u_{\hat{\phi}} \frac{\cot\theta}{r} & 2u_{\hat{\phi},\hat{\phi}} + 2u_{\hat{r}}/r + 2u_{\hat{\theta}} \frac{\cot\theta}{r} \end{pmatrix} \quad (329)$$

The trace-less rate of strain matrix is given by

$$S_{ij} = s_{ij} - \frac{1}{3} \delta_{ij} \nabla \cdot \mathbf{u}. \quad (330)$$

I.2.9 Lambda effect

$$Q_{ij} = \begin{pmatrix} 0 & 0 & \Lambda_V \Omega \\ 0 & 0 & \Lambda_H \Omega \\ \Lambda_V \Omega & \Lambda_H \Omega & 0 \end{pmatrix}, \quad (331)$$

where $\Omega = u_{\hat{\phi}}/r \sin\theta$. Next, compute $Q_{ij;j}$.

$$Q_{\hat{\alpha}\hat{\beta};\hat{\gamma}} = Q_{\hat{\alpha}\hat{\beta},\hat{\gamma}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\gamma}} Q_{\hat{\sigma}\hat{\beta}} - \Gamma^{\hat{\sigma}}_{\hat{\beta}\hat{\gamma}} Q_{\hat{\alpha}\hat{\sigma}} \quad (332)$$

$$\begin{aligned} Q_{\hat{\theta}\hat{r};\hat{r}} &= Q_{\hat{\theta}\hat{r},\hat{r}} - \Gamma^{\hat{\sigma}}_{\hat{\theta}\hat{r}} Q_{\hat{\sigma}\hat{r}} - \Gamma^{\hat{\sigma}}_{\hat{r}\hat{r}} Q_{\hat{\theta}\hat{\sigma}} \\ &= 0. \end{aligned} \quad (333)$$

$$\begin{aligned} Q_{\hat{\theta}\hat{\theta};\hat{\theta}} &= Q_{\hat{\theta}\hat{\theta},\hat{\theta}} - \Gamma^{\hat{\sigma}}_{\hat{\theta}\hat{\theta}} Q_{\hat{\sigma}\hat{\theta}} - \Gamma^{\hat{\sigma}}_{\hat{\theta}\hat{\theta}} Q_{\hat{\theta}\hat{\sigma}} \\ &= -\Gamma^{\hat{\phi}}_{\hat{\theta}\hat{\theta}} Q_{\hat{\phi}\hat{\theta}} - \Gamma^{\hat{\phi}}_{\hat{\theta}\hat{\theta}} Q_{\hat{\theta}\hat{\phi}} \\ &= 0. \end{aligned} \quad (334)$$

$$\begin{aligned} Q_{\hat{\theta}\hat{\phi};\hat{\phi}} &= Q_{\hat{\theta}\hat{\phi},\hat{\phi}} - \Gamma^{\hat{\sigma}}_{\hat{\theta}\hat{\phi}} Q_{\hat{\sigma}\hat{\phi}} - \Gamma^{\hat{\sigma}}_{\hat{\phi}\hat{\phi}} Q_{\hat{\theta}\hat{\sigma}} \\ &= Q_{\hat{\theta}\hat{\phi},\hat{\phi}} - \Gamma^{\hat{\phi}}_{\hat{\theta}\hat{\phi}} Q_{\hat{\phi}\hat{\phi}} - \Gamma^{\hat{r}}_{\hat{\phi}\hat{\phi}} Q_{\hat{\theta}\hat{r}} - \Gamma^{\hat{\theta}}_{\hat{\phi}\hat{\phi}} Q_{\hat{\theta}\hat{\theta}} \\ &= 0. \end{aligned} \quad (335)$$

ϕ terms

$$\begin{aligned} Q_{\hat{\phi}\hat{r};\hat{r}} &= Q_{\hat{\phi}\hat{r},\hat{r}} - \Gamma^{\hat{\sigma}}_{\hat{\phi}\hat{r}} Q_{\hat{\sigma}\hat{r}} - \Gamma^{\hat{\sigma}}_{\hat{r}\hat{r}} Q_{\hat{\phi}\hat{\sigma}} \\ &= Q_{\hat{\phi}\hat{r},\hat{r}}. \end{aligned} \quad (336)$$

$$\begin{aligned} Q_{\hat{\phi}\hat{\theta};\hat{\theta}} &= Q_{\hat{\phi}\hat{\theta},\hat{\theta}} - \Gamma^{\hat{\sigma}}_{\hat{\phi}\hat{\theta}} Q_{\hat{\sigma}\hat{\theta}} - \Gamma^{\hat{\sigma}}_{\hat{\theta}\hat{\theta}} Q_{\hat{\phi}\hat{\sigma}} \\ &= Q_{\hat{\phi}\hat{\theta},\hat{\theta}} - \Gamma^{\hat{r}}_{\hat{\theta}\hat{\theta}} Q_{\hat{\phi}\hat{r}} \\ &= Q_{\hat{\phi}\hat{\theta},\hat{\theta}} + r^{-1} Q_{\hat{\phi}\hat{r}}. \end{aligned} \quad (337)$$

$$\begin{aligned} Q_{\hat{\phi}\hat{\phi};\hat{\phi}} &= Q_{\hat{\phi}\hat{\phi},\hat{\phi}} - \Gamma^{\hat{\sigma}}_{\hat{\phi}\hat{\phi}} Q_{\hat{\sigma}\hat{\phi}} - \Gamma^{\hat{\sigma}}_{\hat{\phi}\hat{\phi}} Q_{\hat{\phi}\hat{\sigma}} \\ &= -\Gamma^{\hat{\sigma}}_{\hat{\phi}\hat{\phi}} Q_{\hat{\sigma}\hat{\phi}} - \Gamma^{\hat{\sigma}}_{\hat{\phi}\hat{\phi}} Q_{\hat{\phi}\hat{\sigma}} \\ &= -\Gamma^{\hat{r}}_{\hat{\phi}\hat{\phi}} Q_{\hat{r}\hat{\phi}} - \Gamma^{\hat{\theta}}_{\hat{\phi}\hat{\phi}} Q_{\hat{\theta}\hat{\phi}} - \Gamma^{\hat{r}}_{\hat{\phi}\hat{\phi}} Q_{\hat{\phi}\hat{r}} - \Gamma^{\hat{\theta}}_{\hat{\phi}\hat{\phi}} Q_{\hat{\phi}\hat{\theta}} \\ &= +r^{-1} Q_{\hat{r}\hat{\phi}} + \cot \theta r^{-1} Q_{\hat{\theta}\hat{\phi}} + r^{-1} Q_{\hat{\phi}\hat{r}} + \cot \theta r^{-1} Q_{\hat{\phi}\hat{\theta}} \\ &= +2r^{-1} \Lambda_V \Omega + 2 \cot \theta r^{-1} \Lambda_H \Omega. \end{aligned} \quad (338)$$

So, the ϕ component of the divergence of the Lambda tensor is then

$$Q_{\hat{\phi}\hat{\sigma};\hat{\sigma}} = Q_{\hat{\phi}\hat{r},\hat{r}} + Q_{\hat{\phi}\hat{\theta},\hat{\theta}} + 3r^{-1} \Lambda_V \Omega + 2 \cot \theta r^{-1} \Lambda_H \Omega. \quad (339)$$

1.2.10 Laplacian of a scalar

Let $E_{\hat{\alpha}} \equiv (\nabla \Psi)_{\hat{\alpha}} = \partial_{\hat{\alpha}} \Psi$ Then

$$\Delta \Psi = E_{\hat{\beta};\hat{\beta}} = (\partial_{\hat{\beta}} \Psi)_{,\hat{\beta}} + \frac{2}{r} \Psi_{,\hat{r}} + \frac{\cot \theta}{r} \Psi_{,\hat{\theta}}. \quad (340)$$

1.2.11 Hessian of a scalar

$$s_{;\hat{\alpha}} = s_{,\hat{\alpha}} \quad (341)$$

$$s_{;\hat{r}\hat{r}} = s_{,\hat{r}\hat{r}} - \Gamma^{\hat{\sigma}}_{\hat{r}\hat{r}} s_{,\hat{\sigma}} = s_{,\hat{r}\hat{r}} \quad (342)$$

$$s_{;\hat{\theta}\hat{\theta}} = s_{,\hat{\theta}\hat{\theta}} - \Gamma^{\hat{r}}_{\hat{\theta}\hat{\theta}} s_{,\hat{r}} = s_{,\hat{\theta}\hat{\theta}} + r^{-1} s_{,\hat{r}}, \quad (343)$$

$$s_{;\hat{\phi}\hat{\phi}} = s_{,\hat{\phi}\hat{\phi}} - \Gamma^{\hat{\sigma}}_{\hat{\phi}\hat{\phi}} s_{,\hat{\sigma}} = s_{,\hat{\phi}\hat{\phi}} - \Gamma^{\hat{r}}_{\hat{\phi}\hat{\phi}} s_{,\hat{r}} - \Gamma^{\hat{\theta}}_{\hat{\phi}\hat{\phi}} s_{,\hat{\theta}} = s_{,\hat{\phi}\hat{\phi}} + r^{-1} s_{,\hat{r}} + r^{-1} \cot \theta s_{,\hat{\theta}}, \quad (344)$$

$$s_{;\hat{r}\hat{\theta}} = s_{,\hat{r}\hat{\theta}} - \Gamma^{\hat{\theta}}_{\hat{r}\hat{\theta}} s_{,\hat{\theta}} = s_{,\hat{r}\hat{\theta}} - r^{-1} s_{,\hat{\theta}}, \quad (345)$$

$$s_{;\hat{r}\hat{\phi}} = s_{,\hat{r}\hat{\phi}} - \Gamma^{\hat{\phi}}_{\hat{r}\hat{\phi}} s_{,\hat{\phi}} = s_{,\hat{r}\hat{\phi}} - r^{-1} s_{,\hat{\phi}} \quad (346)$$

$$s_{;\hat{\theta}\hat{r}} = s_{,\hat{\theta}\hat{r}} - \Gamma^{\hat{\sigma}}_{\hat{\theta}\hat{r}} s_{,\hat{\sigma}} = s_{,\hat{\theta}\hat{r}} \quad (347)$$

$$s_{;\hat{\phi}\hat{r}} = s_{,\hat{\phi}\hat{r}} - \Gamma^{\hat{\sigma}}_{\hat{\phi}\hat{r}} s_{,\hat{\sigma}} = s_{,\hat{\phi}\hat{r}} \quad (348)$$

$$s_{;\hat{\theta}\hat{\phi}} = s_{,\hat{\theta}\hat{\phi}} - \Gamma^{\hat{\phi}}_{\hat{\theta}\hat{\phi}} s_{,\hat{\phi}} = s_{,\hat{\theta}\hat{\phi}} - r^{-1} \cot \theta s_{,\hat{\phi}} \quad (349)$$

$$s_{;\hat{\phi}\hat{\theta}} = s_{,\hat{\phi}\hat{\theta}} - \Gamma^{\hat{\sigma}}_{\hat{\phi}\hat{\theta}} s_{,\hat{\sigma}} = s_{,\hat{\phi}\hat{\theta}} \quad (350)$$

So,

$$s_{;\hat{\alpha}\hat{\beta}} = s_{,\hat{\alpha}\hat{\beta}} + \begin{pmatrix} 0 & -r^{-1} s_{,\hat{\theta}} & -r^{-1} s_{,\hat{\phi}} \\ 0 & +r^{-1} s_{,\hat{r}} & -r^{-1} \cot \theta s_{,\hat{\phi}} \\ 0 & 0 & -\Gamma^{\hat{r}}_{\hat{\phi}\hat{\phi}} s_{,\hat{r}} - \Gamma^{\hat{\theta}}_{\hat{\phi}\hat{\phi}} s_{,\hat{\theta}} \end{pmatrix} \quad (351)$$

I.2.12 Double curl

For the calculation of $\mathbf{J} = \nabla \times \mathbf{B}$ we use the same curl routine, but with different arguments, $\text{bij} = B_{\hat{\alpha},\hat{\beta}}$ and $\text{bb} = B_{\hat{\alpha}}$ as input pencils and $\text{jj} = J_{\hat{\alpha}}$ as output, i.e. `curl_mn(bij,jj,bb)`, so that

$$\mathbf{J} = \begin{pmatrix} B_{\hat{\phi},\hat{\theta}} - B_{\hat{\theta},\hat{\phi}} + \Gamma_{\hat{\theta}\hat{\phi}}^{\hat{\phi}} B^{\hat{\phi}} \\ B_{\hat{r},\hat{\phi}} - B_{\hat{\phi},\hat{r}} - \Gamma_{\hat{r}\hat{\phi}}^{\hat{\phi}} B^{\hat{\phi}} \\ B_{\hat{\theta},\hat{r}} - B_{\hat{r},\hat{\theta}} + \Gamma_{\hat{r}\hat{\theta}}^{\hat{\theta}} B^{\hat{\theta}} \end{pmatrix} = (\nabla \times \mathbf{B})^{(0)} + M_{\hat{\alpha}\hat{\beta}}^{(\text{curl})} B_{\hat{\beta}}, \quad (352)$$

where

$$(\nabla \times \mathbf{B})^{(0)\hat{\alpha}} = \epsilon^{\hat{\alpha}\hat{\beta}\hat{\gamma}} B_{\hat{\gamma},\hat{\beta}}. \quad (353)$$

Expressing the components of $(\nabla \times \mathbf{B})^{(0)}$ in terms of $A_{\hat{\alpha}}$ we have

$$\begin{aligned} B_{\hat{\phi},\hat{\theta}} &= (A_{\hat{\theta},\hat{r}} - A_{\hat{r},\hat{\theta}} + \Gamma_{\hat{r}\hat{\theta}}^{\hat{\theta}} A^{\hat{\theta}})_{,\hat{\theta}} & B_{\hat{\theta},\hat{\phi}} &= (A_{\hat{r},\hat{\phi}} - A_{\hat{\phi},\hat{r}} - \Gamma_{\hat{r}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}})_{,\hat{\phi}} \\ B_{\hat{r},\hat{\phi}} &= (A_{\hat{\phi},\hat{\theta}} - A_{\hat{\theta},\hat{\phi}} + \Gamma_{\hat{\theta}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}})_{,\hat{\phi}} & B_{\hat{\phi},\hat{r}} &= (A_{\hat{\theta},\hat{r}} - A_{\hat{r},\hat{\theta}} + \Gamma_{\hat{r}\hat{\theta}}^{\hat{\theta}} A^{\hat{\theta}})_{,\hat{r}} \\ B_{\hat{\theta},\hat{r}} &= (A_{\hat{r},\hat{\phi}} - A_{\hat{\phi},\hat{r}} - \Gamma_{\hat{r}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}})_{,\hat{r}} & B_{\hat{r},\hat{\theta}} &= (A_{\hat{\phi},\hat{\theta}} - A_{\hat{\theta},\hat{\phi}} + \Gamma_{\hat{\theta}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}})_{,\hat{\theta}} \end{aligned} \quad (354)$$

or

$$\begin{aligned} B_{\hat{\phi},\hat{\theta}} &= A_{\hat{\theta},\hat{r}\hat{\theta}} - A_{\hat{r},\hat{\theta}\hat{\theta}} + (\Gamma_{\hat{r}\hat{\theta}}^{\hat{\theta}} A^{\hat{\theta}})_{,\hat{\theta}} & B_{\hat{\theta},\hat{\phi}} &= A_{\hat{r},\hat{\phi}\hat{\phi}} - A_{\hat{\phi},\hat{r}\hat{\phi}} - (\Gamma_{\hat{r}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}})_{,\hat{\phi}} \\ B_{\hat{r},\hat{\phi}} &= A_{\hat{\phi},\hat{\theta}\hat{\phi}} - A_{\hat{\theta},\hat{\phi}\hat{\phi}} + (\Gamma_{\hat{\theta}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}})_{,\hat{\phi}} & B_{\hat{\phi},\hat{r}} &= A_{\hat{\theta},\hat{r}\hat{r}} - A_{\hat{r},\hat{\theta}\hat{r}} + (\Gamma_{\hat{r}\hat{\theta}}^{\hat{\theta}} A^{\hat{\theta}})_{,\hat{r}} \\ B_{\hat{\theta},\hat{r}} &= A_{\hat{r},\hat{\phi}\hat{r}} - A_{\hat{\phi},\hat{r}\hat{r}} - (\Gamma_{\hat{r}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}})_{,\hat{r}} & B_{\hat{r},\hat{\theta}} &= A_{\hat{\phi},\hat{\theta}\hat{\theta}} - A_{\hat{\theta},\hat{\phi}\hat{\theta}} + (\Gamma_{\hat{\theta}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}})_{,\hat{\theta}}. \end{aligned} \quad (355)$$

Thus,

$$B_{\hat{\alpha},\hat{\beta}} = (B_{\hat{\alpha},\hat{\beta}})^{(0)} + M_{\hat{\alpha}\hat{\beta}\hat{\gamma}\hat{\delta}}^{(2\text{curl}2)} A_{\hat{\gamma},\hat{\delta}} + M_{\hat{\alpha}\hat{\beta}\hat{\gamma}}^{(1\text{curl}2)} A_{\hat{\gamma}}, \quad (356)$$

where

$$(B_{\hat{\alpha},\hat{\delta}})^{(0)} = \epsilon_{\hat{\alpha}\hat{\beta}\hat{\gamma}} A_{\hat{\gamma},\hat{\beta}\hat{\delta}} \quad (357)$$

with

$$\Psi_{,\hat{r}\hat{r}} = \Psi_{,rr} \quad (358)$$

$$\Psi_{,\hat{\theta}\hat{\theta}} = \Psi_{,\theta\theta} r^{-2} \quad (359)$$

$$\Psi_{,\hat{\phi}\hat{\phi}} = \Psi_{,\phi\phi} r^{-2} \sin^{-2}\theta \quad (360)$$

$$\Psi_{,\hat{r}\hat{\theta}} = \Psi_{,r\theta} r^{-1} \quad (361)$$

$$\Psi_{,\hat{\theta}\hat{r}} = \Psi_{,\theta r} r^{-1} - \Psi_{,\hat{\theta}} r^{-1} \quad (362)$$

$$\Psi_{,\hat{r}\hat{\phi}} = \Psi_{,r\phi} r^{-1} \sin^{-1}\theta \quad (363)$$

$$\Psi_{,\hat{\phi}\hat{r}} = \Psi_{,\phi r} r^{-1} \sin^{-1}\theta - \Psi_{,\hat{\phi}} r^{-1} \quad (364)$$

$$\Psi_{,\hat{\theta}\hat{\phi}} = \Psi_{,\theta\phi} r^{-2} \sin^{-1}\theta \quad (365)$$

and

$$\begin{aligned} B_{\hat{\phi},\hat{\theta}} &= \dots + \Gamma_{\hat{r}\hat{\theta}}^{\hat{\theta}} A^{\hat{\theta}}_{,\hat{\theta}} + \Gamma_{\hat{r}\hat{\theta}}^{\hat{\theta}} A^{\hat{\theta}} \\ B_{\hat{r},\hat{\phi}} &= \dots + \Gamma_{\hat{\theta}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}}_{,\hat{\phi}} + \Gamma_{\hat{\theta}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}} \\ B_{\hat{\theta},\hat{r}} &= \dots - \Gamma_{\hat{r}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}}_{,\hat{r}} - \Gamma_{\hat{r}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}} \end{aligned} \quad \begin{aligned} B_{\hat{\theta},\hat{\phi}} &= \dots - \Gamma_{\hat{r}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}}_{,\hat{\phi}} - \Gamma_{\hat{r}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}} \\ B_{\hat{\phi},\hat{r}} &= \dots + \Gamma_{\hat{r}\hat{\theta}}^{\hat{\theta}} A^{\hat{\theta}}_{,\hat{r}} + \Gamma_{\hat{r}\hat{\theta}}^{\hat{\theta}} A^{\hat{\theta}} \\ B_{\hat{r},\hat{\theta}} &= \dots + \Gamma_{\hat{\theta}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}}_{,\hat{\theta}} + \Gamma_{\hat{\theta}\hat{\phi}}^{\hat{\phi}} A^{\hat{\phi}} \end{aligned} \quad (366)$$

Note that some derivatives of Christoffel symbols vanish, so we are left with

$$\begin{aligned} B_{\hat{\phi},\hat{\theta}} &= \dots + r^{-1} A^{\hat{\theta}}_{,\hat{\theta}} & B_{\hat{\theta},\hat{\phi}} &= \dots - r^{-1} A^{\hat{\phi}}_{,\hat{\phi}} \\ B_{\hat{r},\hat{\phi}} &= \dots + r^{-1} \cot\theta A^{\hat{\phi}}_{,\hat{\phi}} & B_{\hat{\phi},\hat{r}} &= \dots + r^{-1} A^{\hat{\theta}}_{,\hat{r}} - r^{-2} A^{\hat{\theta}} \\ B_{\hat{\theta},\hat{r}} &= \dots - r^{-1} A^{\hat{\phi}}_{,\hat{r}} + r^{-2} A^{\hat{\phi}} & B_{\hat{r},\hat{\theta}} &= \dots + r^{-1} \cot\theta A^{\hat{\phi}}_{,\hat{\theta}} - r^{-2} \sin^{-2}\theta A^{\hat{\phi}} \end{aligned} \quad (367)$$

1.2.13 Gradient of a divergence

$$A_{\hat{\alpha};\hat{\alpha}\hat{\gamma}} = A_{\hat{\alpha},\hat{\alpha}\hat{\gamma}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\alpha}} A_{\hat{\sigma},\hat{\gamma}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\alpha},\hat{\gamma}} A_{\hat{\sigma}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\gamma}} A_{\hat{\sigma},\hat{\alpha}} + \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\gamma}} \Gamma^{\hat{\nu}}_{\hat{\sigma}\hat{\alpha}} A_{\hat{\nu}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\gamma}} A_{\hat{\alpha},\hat{\sigma}} + \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\gamma}} \Gamma^{\hat{\nu}}_{\hat{\alpha}\hat{\sigma}} A_{\hat{\nu}}.$$

$\hat{r}$ component:

$$\begin{aligned} A_{\hat{\alpha};\hat{\alpha}\hat{r}} &= A_{\hat{\alpha},\hat{\alpha}\hat{r}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\alpha}} A_{\hat{\sigma},\hat{r}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\alpha},\hat{r}} A_{\hat{\sigma}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{r}} A_{\hat{\sigma},\hat{\alpha}} + \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{r}} \Gamma^{\hat{\nu}}_{\hat{\sigma}\hat{\alpha}} A_{\hat{\nu}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{r}} A_{\hat{\alpha},\hat{\sigma}} + \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{r}} \Gamma^{\hat{\nu}}_{\hat{\alpha}\hat{\sigma}} A_{\hat{\nu}} \\ &= A_{\hat{\alpha},\hat{\alpha}\hat{r}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\alpha}} A_{\hat{\sigma},\hat{r}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\alpha},\hat{r}} A_{\hat{\sigma}} \\ &= A_{\hat{\alpha},\hat{\alpha}\hat{r}} + 2r^{-1} A_{\hat{r},\hat{r}} + r^{-1} \cot\theta A_{\hat{\theta},\hat{r}} - 2r^{-2} A_{\hat{r}} - r^{-2} \cot\theta A_{\hat{\theta}} \end{aligned} \quad (368)$$

$\hat{\theta}$ component:

$$\begin{aligned} A_{\hat{\alpha};\hat{\alpha}\hat{\theta}} &= A_{\hat{\alpha},\hat{\alpha}\hat{\theta}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\alpha}} A_{\hat{\sigma},\hat{\theta}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\alpha},\hat{\theta}} A_{\hat{\sigma}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\theta}} A_{\hat{\sigma},\hat{\alpha}} + \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\theta}} \Gamma^{\hat{\nu}}_{\hat{\sigma}\hat{\alpha}} A_{\hat{\nu}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\theta}} A_{\hat{\alpha},\hat{\sigma}} + \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\theta}} \Gamma^{\hat{\nu}}_{\hat{\alpha}\hat{\sigma}} A_{\hat{\nu}} \\ &= A_{\hat{\alpha},\hat{\alpha}\hat{\theta}} + 2r^{-1} A_{\hat{r},\hat{\theta}} + r^{-1} \cot\theta A_{\hat{\theta},\hat{\theta}} - r^{-2} \sin^{-2}\theta A_{\hat{\theta}} \end{aligned} \quad (369)$$

Note that the last four terms in the above expression canceled, because

$$\begin{aligned} & -\Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\theta}} A_{\hat{\sigma},\hat{\alpha}} + \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\theta}} \Gamma^{\hat{\nu}}_{\hat{\sigma}\hat{\alpha}} A_{\hat{\nu}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\theta}} A_{\hat{\alpha},\hat{\sigma}} + \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\theta}} \Gamma^{\hat{\nu}}_{\hat{\alpha}\hat{\sigma}} A_{\hat{\nu}} \\ &= -r^{-1} A_{\hat{\theta},\hat{r}} + r^{-1} A_{\hat{r},\hat{\theta}} - r^{-2} A_{\hat{\theta}} - r^{-1} A_{\hat{r},\hat{\theta}} + r^{-1} A_{\hat{\theta},\hat{r}} + r^{-2} A_{\hat{\theta}} = 0 \end{aligned} \quad (370)$$

$\hat{\phi}$ component:

$$\begin{aligned} A_{\hat{\alpha};\hat{\alpha}\hat{\phi}} &= A_{\hat{\alpha},\hat{\alpha}\hat{\phi}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\alpha}} A_{\hat{\sigma},\hat{\phi}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\alpha},\hat{\phi}} A_{\hat{\sigma}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\phi}} A_{\hat{\sigma},\hat{\alpha}} + \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\phi}} \Gamma^{\hat{\nu}}_{\hat{\sigma}\hat{\alpha}} A_{\hat{\nu}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\phi}} A_{\hat{\alpha},\hat{\sigma}} + \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\phi}} \Gamma^{\hat{\nu}}_{\hat{\alpha}\hat{\sigma}} A_{\hat{\nu}} \\ &= A_{\hat{\alpha},\hat{\alpha}\hat{\phi}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\alpha}} A_{\hat{\sigma},\hat{\phi}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\alpha},\hat{\phi}} A_{\hat{\sigma}} \\ &= A_{\hat{\alpha},\hat{\alpha}\hat{\phi}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\alpha}} A_{\hat{\sigma},\hat{\phi}} \\ &= A_{\hat{\alpha},\hat{\alpha}\hat{\phi}} - \Gamma^{\hat{r}}_{\hat{\theta}\hat{\theta}} A_{\hat{r},\hat{\phi}} - \Gamma^{\hat{r}}_{\hat{\phi}\hat{\phi}} A_{\hat{r},\hat{\phi}} - \Gamma^{\hat{\theta}}_{\hat{\phi}\hat{\phi}} A_{\hat{\theta},\hat{\phi}} \\ &= A_{\hat{\alpha},\hat{\alpha}\hat{\phi}} + r^{-1} A_{\hat{r},\hat{\phi}} + r^{-1} A_{\hat{r},\hat{\phi}} + r^{-1} \cot\theta A_{\hat{\theta},\hat{\phi}} \\ &= A_{\hat{\alpha},\hat{\alpha}\hat{\phi}} + 2r^{-1} A_{\hat{r},\hat{\phi}} + r^{-1} \cot\theta A_{\hat{\theta},\hat{\phi}}. \end{aligned} \quad (371)$$

In the first line of the expression above, the last four terms vanish²⁴ and the ϕ derivative of any Christoffel symbol (term before that) also vanishes.

²⁴The following four terms vanish because

$$\begin{aligned} & -\Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\phi}} A_{\hat{\sigma},\hat{\alpha}} + \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\phi}} \Gamma^{\hat{\nu}}_{\hat{\sigma}\hat{\alpha}} A_{\hat{\nu}} - \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\phi}} A_{\hat{\alpha},\hat{\sigma}} + \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\phi}} \Gamma^{\hat{\nu}}_{\hat{\alpha}\hat{\sigma}} A_{\hat{\nu}} \\ &= \Gamma^{\hat{\sigma}}_{\hat{\alpha}\hat{\phi}} (-A_{\hat{\sigma},\hat{\alpha}} + \Gamma^{\hat{\nu}}_{\hat{\sigma}\hat{\alpha}} A_{\hat{\nu}} - A_{\hat{\alpha},\hat{\sigma}} + \Gamma^{\hat{\nu}}_{\hat{\alpha}\hat{\sigma}} A_{\hat{\nu}}) \\ &= \Gamma^{\hat{\phi}}_{\hat{r}\hat{\phi}} (-A_{\hat{\phi},\hat{r}} + \Gamma^{\hat{\nu}}_{\hat{\phi}\hat{r}} A_{\hat{\nu}} - A_{\hat{r},\hat{\phi}} + \Gamma^{\hat{\nu}}_{\hat{r}\hat{\phi}} A_{\hat{\nu}}) \\ &+ \Gamma^{\hat{r}}_{\hat{\phi}\hat{\phi}} (-A_{\hat{r},\hat{\phi}} + \Gamma^{\hat{\nu}}_{\hat{r}\hat{\phi}} A_{\hat{\nu}} - A_{\hat{\phi},\hat{r}} + \Gamma^{\hat{\nu}}_{\hat{\phi}\hat{r}} A_{\hat{\nu}}) \\ &+ \Gamma^{\hat{\theta}}_{\hat{\phi}\hat{\phi}} (-A_{\hat{\phi},\hat{\theta}} + \Gamma^{\hat{\nu}}_{\hat{\theta}\hat{\phi}} A_{\hat{\nu}} - A_{\hat{\theta},\hat{\phi}} + \Gamma^{\hat{\nu}}_{\hat{\theta}\hat{\phi}} A_{\hat{\nu}}) \\ &+ \Gamma^{\hat{\theta}}_{\hat{\phi}\hat{\phi}} (-A_{\hat{\theta},\hat{\phi}} + \Gamma^{\hat{\nu}}_{\hat{\theta}\hat{\phi}} A_{\hat{\nu}} - A_{\hat{\phi},\hat{\theta}} + \Gamma^{\hat{\nu}}_{\hat{\theta}\hat{\phi}} A_{\hat{\nu}}) \\ &= \Gamma^{\hat{\phi}}_{\hat{r}\hat{\phi}} (-A_{\hat{\phi},\hat{r}} - A_{\hat{r},\hat{\phi}} + \Gamma^{\hat{\nu}}_{\hat{r}\hat{\phi}} A_{\hat{\nu}}) \\ &+ \Gamma^{\hat{r}}_{\hat{\phi}\hat{\phi}} (-A_{\hat{r},\hat{\phi}} + \Gamma^{\hat{\nu}}_{\hat{r}\hat{\phi}} A_{\hat{\nu}} - A_{\hat{\phi},\hat{r}}) \\ &+ \Gamma^{\hat{\theta}}_{\hat{\phi}\hat{\phi}} (-A_{\hat{\phi},\hat{\theta}} - A_{\hat{\theta},\hat{\phi}} + \Gamma^{\hat{\nu}}_{\hat{\theta}\hat{\phi}} A_{\hat{\nu}}) \\ &+ \Gamma^{\hat{\theta}}_{\hat{\phi}\hat{\phi}} (-A_{\hat{\theta},\hat{\phi}} + \Gamma^{\hat{\nu}}_{\hat{\theta}\hat{\phi}} A_{\hat{\nu}} - A_{\hat{\phi},\hat{\theta}}) \\ &= r^{-1} (-A_{\hat{\phi},\hat{r}} - A_{\hat{r},\hat{\phi}} + \Gamma^{\hat{\nu}}_{\hat{r}\hat{\phi}} A_{\hat{\nu}}) \\ &- r^{-1} (-A_{\hat{r},\hat{\phi}} - A_{\hat{\phi},\hat{r}} + \Gamma^{\hat{\nu}}_{\hat{r}\hat{\phi}} A_{\hat{\nu}}) \\ &+ r^{-1} \cot\theta (-A_{\hat{\phi},\hat{\theta}} - A_{\hat{\theta},\hat{\phi}} + \Gamma^{\hat{\nu}}_{\hat{\theta}\hat{\phi}} A_{\hat{\nu}}) \\ &- r^{-1} \cot\theta (-A_{\hat{\theta},\hat{\phi}} - A_{\hat{\phi},\hat{\theta}} + \Gamma^{\hat{\nu}}_{\hat{\theta}\hat{\phi}} A_{\hat{\nu}}) = 0 \end{aligned} \quad (372)$$

J Switchable modules

The material in this section is being assembled from the file ‘inlinedoc-modules.tex’, which is automatically assembled. However, it is currently incomplete and contains only a small number of the available modules.

<i>Module</i>	<i>Description</i>
<i>hydro.f90</i>	This module takes care of most of the things related to velocity. Pressure, for example, is added in the energy (entropy) module.
<i>chemistry.f90</i>	This modules adds chemical species and reactions. The units used in the chem.in files are cm3,mole,sec,kcal and K
<i>geometrical_types.f90</i>	Collection of geometrical object types. (Presently only rectangular toroid)
<i>gpu_astaroth.f90</i>	This module contains GPU related types and functions to be used with the ASTAROTH nucleus.
<i>hydro_potential.f90</i>	This module takes care of most of the things related to velocity. Pressure, for example, is added in the energy (entropy) module.
<i>noentropy.f90</i>	Calculates pressure gradient term for polytropic equation of state $p = \text{const} \rho^\Gamma$.
<i>nogpu.f90</i>	This module contains GPU related dummy types and functions.
<i>nohydro.f90</i>	no variable <i>u</i> : useful for kinematic dynamo runs.
<i>nopower_spectrum.f90</i>	reads in full snapshot and calculates power spetrum of u
<i>noyinyang.f90</i>	This module contains Yin-Yang related dummy types and functions.
<i>noyinyang_mpi.f90</i>	This module contains Yin-Yang related dummy types and functions.
<i>particles_adsorbed.f90</i>	This module takes care of the evolution of adsorbed species on the particle surface for reactive particles
<i>particles_chemistry.f90</i>	This module implements reactive particles.
<i>particles_surfspec.f90</i>	immediate vicinity of reactive particles.
<i>power_spectrum.f90</i>	reads in full snapshot and calculates power spetrum of u
<i>test_chemistry.f90</i>	This modules adds chemical species and reactions. The units used in the chem.in files are cm3,mole,sec,kcal and K
<i>timestep.f90</i>	Runge-Kutta time advance, accurate to order itorder.

	At the moment, itorder can be 1, 2, or 3.
<i>timestep_strang.f90</i>	Runge-Kutta time advance, accurate to order itorder. At the moment, itorder can be 1, 2, or 3. Split one dt into two dt/2 steps with RK method. Please add documentation on why this is beneficial...
<i>timestep_subcycle.f90</i>	This is a highly specified timestep module currently only working together with the special module coronae.f90.
<i>yinyang.f90</i>	This module contains Yin-Yang related types and functions which are incompatible with FORTRAN 95.
<i>yinyang_mpi.f90</i>	This module contains Yin-Yang related types and functions which are incompatible with FORTRAN 95.

K Startup and run-time parameters

K.1 Autogenerated list of start and run parameters

The parameters are grouped by the module in which the corresponding namelist appears (note that the global parameters which are specified in *init_pars* and *run_pars*) are shown under ‘param_io.f90’). In due course of time, the table below will replace those in sections K.2 and K.3. In the square brackets that follow the description of a parameter, ‘init’ means that the parameter can be used ‘start.in’, while ‘run’ means that it can be used in ‘run.in’. The square brackets also contain the default value if known.²⁵

<i>Variable</i>	<i>Meaning</i>
Module ‘param_io.f90’	
<i>cdtsrc</i>	empirical time step factor for source terms (1/time) [run; default = 1.0]
<i>cvsid</i>	the svn identification string, which allows you to keep track of the version of ‘start.in’ or ‘run.in’. [init,run; default = ‘no cvsid is given in start.in or run.in!’]
<i>d1davg</i>	save 1D (xy, yz, ...) averages every d1davg time units (overrides it1d) [run]
<i>d2davg</i>	save 2D (x, y, ...) averages every d2davg time units [run; default = 100.]
<i>dsnap</i>	save a snapshot (VAR file) every dsnap time units [run; default = 100.]
<i>dsnap_down</i>	save a downsampled snapshot every dsnap_down time units [run; default = 0.]
<i>dspec</i>	save power spectra every dspec time units [run]
<i>dvid</i>	save slice (video) files every dvid time units. [init,run; default = 0.]
<i>ip</i>	(anti-)verbosity level: ip=1 produces lots of diagnostic output, ip=14 virtually none. [init,run; default = 14]
<i>it1_ldt_report</i>	interval for time step constraint reports onto command line [run]
<i>lappend_pc_constants</i>	always append when true to preserve history [init,run; default = .false.]
<i>lbaryons</i>	interpret rho as the baryon velocity; it affects vA2 pencil. [init; default = .true.]
<i>Inowrite</i>	if set T, all initialization files are written, including the param.nml file, except ‘var.dat’. This option allows you to use old ‘var.dat’ files, but updates all other initialization files. This could be useful after having changed the code and, in particular, when the ‘var.dat’ files will be overwritten by ‘remesh.csh’. [init,run; default = .false.]

²⁵The variables that do not have a default value listed are most likely initialized separately from their declaration. The default values of such variables need to be manually documented.

<i>lperi</i>	vector of the form (x,y,z) ; determines whether the corresponding direction is periodic [init,run; default = .true.]
<i>lprocz_slowest</i>	if set to F, the ordering of processor numbers is changed, so the z processors are now in the inner loop. Since <i>nproc</i> =4 is optimal (see Sect. 5.21.2), you may want to put <i>lprocz_slowest</i> =T when <i>nygrid</i> > <i>nzgrid</i> . [init; default = .true.]
<i>lread_aux</i>	This controls whether auxiliary variables are read from snapshots. This is only required for configurations in which the auxiliary variables are actually not calculated at each timestep (e.g., when you set <i>kinematic_flow</i> ='from-snap' in <i>hydro_run_pars</i>). [init,run; default = .true.]
<i>lread_oldsnap</i>	if set T, the old snapshot will be read in by start.csh before producing (overwriting) initial conditions. For example, if you just want to add a perturbation to the magnetic field, you'd give no initial condition for density and velocity (so you keep the data from a hopefully relaxed run), and just add whatever you need for the magnetic field. In this connection you may want to touch NOERASE, so as not to erase the previous data. Also, in filestart.in, put: <i>ireset_tstart</i> =0, <i>lread_oldsnap</i> =T, <i>lwrite_var_anyway</i> =T [init; default = .false.]
<i>lread_oldsnap_nomag</i>	if set T, the old snapshot from a non-magnetic run will be read in before producing (overwriting) initial conditions. This allows one to let a hydrodynamic run relax before adding a magnetic field. However, for this to work one has to modify <i>manually</i> 'data/param.nml' by adding an entry for <i>MAGNETIC_INIT_PARS</i> or <i>PSCALAR_INIT_PARS</i> . In addition, for <i>idl</i> to read correctly after the first restarted run, you must adjust the value of <i>mvar</i> in 'data/dim.dat' [init,run; default = .false.]
<i>lread_oldsnap_nopscalar</i>	if set T, the old snapshot from a run without passive scalar will be read in before producing (overwriting) initial conditions. This allows one to let a hydrodynamic run relax before adding a passive scalar. [init,run; default = .false.]
<i>lshift_origin</i>	if set T for any or some of the three directions, the mesh is shifted by 1/2 meshpoint in that or those directions so that the mesh goes through the origin. [init,run; default = .false.]
<i>lshift_origin_lower</i>	same as above, but downward. [init,run; default = .false.]
<i>lsnap_tcrit</i>	allow setting critical time for switching to log interval for dsnap. [run; default = .false.]
<i>lspec_tcrit</i>	allow setting critical time for switching to log interval for dspec. [run; default = .false.]

<i>lvid_tcrit</i>	allow setting critical time for switching to log interval for dvid. [run; default = .false.]
<i>lwrite_2d</i>	if set T, only 2D-snapshots are written into VAR files in the case of 2D-runs with <i>nygrid</i> = 1 or <i>nzgrid</i> = 1. [init,run; default = .false.]
<i>lwrite_aux</i>	if set T, write auxiliary variables (those calculated at each step, but not evolved mathematically) to 'var.dat' after the evolved quantities. [init,run; default = .false.]
<i>lwrite_ic</i>	if set T, the initial data are written into the file 'VAR0'. This is generally useful, but doing this all the time uses up plenty of disk space. [init; default = .true.]
<i>Lxyz</i>	vector of the form (<i>x</i> , <i>y</i> , <i>z</i>); determines the box size [init,run]
<i>offset_min_calc</i>	offset value for computing negative minima, when some values are masked out with zeros. [init,run; default = 0.]
<i>pretend_lnTT</i>	selects $\ln T$ as fundamental thermodynamic variable in the entropy module [init,run; default = .false.]
<i>random_gen</i>	choose random number generator; currently valid choices are 'system' (your compiler's generator), 'min_std' (the 'minimal standard' generator ran0() from 'Numerical Recipes'), 'nr_f90' (the Parker-Miller-Marsaglia generator ran() from 'Numerical Recipes for F90'). [init,run; default = 'min_std']
<i>trigger_snap</i>	trigger quantity for snapshot output [run; default = 'code_time']
<i>trigger_vid</i>	trigger quantity for video output [run; default = 'code_time']
<i>tsnap_crit_log_interval</i>	critical time above which we set dsnap to a negative value. [run]
<i>tspec_crit_log_interval</i>	critical time above which we set dspec to a negative value. [run]
<i>tvid_crit_log_interval</i>	critical time above which we set dvid to a negative value. [run]
<i>unit_density</i>	Example: if you want your unit density to be 10^{-24} g/cm ³ you say unit_density='1e-24'. Default: 1. [init,run]
<i>unit_length</i>	allows you to set the unit length. Suppose you want the unit length to be 1kpc, then you would say unit_length='3e21'. (Of course, politically correct would be to say unit_system='SI' in which case you say unit_length='3e19'.) Default: 1. [init,run]

<i>unit_system</i>	you can set this character string to 'SI', which means that you can give physical dimensions in SI units. The default is cgs units. [init,run; default = 'cgs']
<i>unit_temperature</i>	Example: unit_temperature='1e6' if you want mega-Kelvin. Default: 1. [init,run]
<i>unit_velocity</i>	Example: if you want km/s you say unit_length='1e5'. Default: 1. [init,run]
<i>xyz0</i>	vector of the form (x, y, z) ; determines the left (lower) corner of the box [init,run; default = -pi]
Module 'hydro.f90'	
<i>ampluu</i>	amplitude for some types of initial velocities. [init,run; default = 0.0]
<i>inituu</i>	initialization of velocity. Some valid choices are 'zero' ($u = 0$), 'gaussian-noise' (random, normally-distributed u_x, u_z), 'gaussian-noise-x' (random, normally-distributed u_x), 'sound-wave' (sound wave in x direction), 'shock-tube' (polytropic standing shock), 'bullets' (blob-like velocity perturbations), 'Alfven-circ-x' (circularly polarized Alfven wave in x direction), 'const-ux' (constant x -velocity), 'const-uy' (constant y -velocity), 'tang-discont-z' (tangential discontinuity: velocity is directed along x , jump is at $z = 0$), 'Fourier-trunc' (truncated Fourier series), 'up-down' (flow upward in one spot, downward in another; not solenoidal). [init,run; default = 'nothing']
<i>lampluu_adjust_ascale</i>	automatically adjust initial u -amplitude [init; default = .false.]
<i>lforcing_cont_uu_diff</i>	forcing with respect the velocity difference [run; default = .false.]
<i>urand</i>	additional random perturbation of u . If urand>0, the perturbation is additive, $u_i \mapsto u_i + u_{\text{rand}}\mathcal{U}_{[0.5,0.5]}$; if urand<0, it is multiplicative, $u_i \mapsto u_i \times u_{\text{rand}}\mathcal{U}_{[0.5,0.5]}$; in both cases, $\mathcal{U}_{[0.5,0.5]}$ is a uniformly distributed random variable on the interval $[-0.5, 0.5]$. [init; default = 0.]
<i>uu_left</i>	needed for inituu='shock-tube' [init; default = 0.]
<i>uu_right</i>	needed for inituu='shock-tube' [init; default = 0.]
<i>widthuu</i>	width for some types of initial velocities. [init; default = .1]

Module 'density.f90'

<i>ampllnrho</i>	amplitude for some types of initial densities [init; default = 0.0]
<i>initlnrho</i>	initialization of density. Currently valid choices are 'zero' ($\ln \rho = 0$), 'isothermal' (isothermal stratification), 'polytropic_simple' (polytropic stratification), 'hydrostatic-z-2' (hydrostatic vertical stratification for isentropic atmosphere), 'xjump' (density jump in x of width <i>widthlnrho</i>), 'rho-jump-z' (density jump in z of width <i>widthlnrho</i>), 'piecew-poly' (piecewise polytropic vertical stratification for solar convection), 'polytropic' (polytropic vertical stratification), 'sound-wave' (sound wave), 'shock-tube' (polytropic standing shock), 'gaussian-noise' (Gaussian-distributed, uncorrelated noise), 'gaussian-noise' (Gaussian-distributed, uncorrelated noise in x , but uniform in y and z), 'hydrostatic-r' (hydrostatic radial density stratification for isentropic or isothermal sphere), 'sin-xy' (sine profile in x and y), 'sin-xy-rho' (sine profile in x and y , but in ρ , not $\ln \rho$), 'linear' (linear profile in $k \cdot x$), 'planet' (planet solution; see §C.7).
<i>lpress_equil_uu</i>	[init,run; default = 'nothing'] adjust lnrho so that $.5*u^2 + cs^2*lnrho = \text{const.}$
<i>rho_left</i>	[init; default = .false.] needed for <i>initlnrho</i> ='shock-tube'
<i>rho_right</i>	[init; default = 1.0] needed for <i>initlnrho</i> ='shock-tube'
<i>Sc</i>	[init; default = 1.0] given value to compute <i>kap_tdep</i> (<i>diffrho</i>) based on <i>nu_tdep</i>
<i>widthlnrho</i>	[run; default = 0.0] width for some types of initial densities [init; default = 0.1]

Module 'entropy.f90'

<i>isothtop</i>	flag for isothermal top layer for <i>initss</i> = <i>piecew-poly</i> . [init]
-----------------	--

Module 'magnetic.f90'

<i>amplaa</i>	amplitude for some types of initial magnetic fields. [init,run; default = 0.0]
<i>amplaa2</i>	amplitude for some types of magnetic field perturbation [init; default = 0.0]
<i>epsilonaa</i>	used by some initial fields [init,run; default = 0.01]

<i>lpress_equil</i>	flag for pressure equilibrium (can be used in connection with all initial fields) [init; default = .false.]
<i>luse_bgb_as_jxb</i>	replace JxB by B.gradB in auxiliary array, for test purposes. [run; default = .false.]
<i>radius</i>	used by some initial fields [init;run; default = 0.1]
<i>widthaa</i>	used by some initial fields [init; default = 0.5]
<i>z0aa</i>	used by some initial fields [init; default = 0.0]
Module 'backreact_infl.f90'	
<i>a4rhophim_crit</i>	critical value of a4rhophim below lsolve_phi=F is set. [run; default = 0.]
<i>aphimax</i>	maximum a value above which the phi potential is quenched. [run; default = 0., aphimax2=0.]
<i>dlnascale_reheating</i>	width in lna over which reheating should occur [run; default = 0.]
<i>Gamma_phi0</i>	damping factor for phi above aphimax [init;run; default = impossible, Gamma_phi]
<i>Gamma_phi_exp</i>	scale factor exponent on Gamma_phi [run; default = 3.]
<i>it1_reset_value</i>	new it1 value after lit1_reset [run]
<i>la2rhop_wrong_factor</i>	should be false to have correct results; kept for backward compatibility [init; default = .false.]
<i>lapply_BD_kNy_factor</i>	apply $1/N^{(D/2)}$ factor in the Bunch-Davies initial condition. [init; default = .false.]
<i>lapply_BD_k1D_factor</i>	apply k_1^D factor in the Bunch-Davies initial condition (NOTE typo in name!) [init; default = .false.]
<i>lBD_scaling_wHubble</i>	Bunch-Davies scaling with Hubble (true for backward compatibility, but should be false to be correct) [init; default = .true.]
<i>ldefine_a2rhophi_with_Vpotential</i>	define a2rhophi with Vpotential [run; default = .true.]
<i>ldefine_a2rhopm_without_Vpotential</i>	should be false to have correct results [init; default = .false.]
<i>ldt_Gamma_phi</i>	time step constraint from Gam_phi [run; default = .false.]
<i>lg_Gamma_phi_fraction_firststep</i>	lg of Gamma phi fraction at firststep [run; default = 1e-3]
<i>lheating_keep_on</i>	should be true to prevent further checks on heating [run; default = .true.]

<i>linclude_rho_EB_in_wstate</i>	include rhoE and rhoB, but not rhokin in wstate [run; default = .false.]
<i>linclude_rho_EBK_in_wstate</i>	include rhoE, rhoB, and rhokin in wstate [run; default = .false.]
<i>linclude_rhokin_in_a2rho</i>	include rhokin in a2rho [run; default = .false.]
<i>linv_BD</i>	apply forward transform in the Bunch-Davies initial condition. [init; default = .true.]
<i>lit1_reset_if_lsolve_for_phi</i>	allow to check for <code>lit1_reset_if_lsolve_for_phi</code> [run; default = .false.]
<i>lna_switch_toMHD</i>	critical value when to switch to MHD (if <code>lswitch_toMHD_at_lna=T</code>) [run; default = 4.17]
<i>lold_ldt_phi</i>	for backward compatibility. [run; default = .true.]
<i>lold_lrho_chi_dtconstraint</i>	old <code>lrho_chi dt</code> constraint (use false for new and correct version) [run; default = .true.]
<i>lrho_chi_corrected</i>	for backward compatibility; when false, we use the wrong scale factor in the <code>rho_chi</code> equation [run; default = .true.]
<i>lrho_chi_inhom</i>	inhomogeneous heating [init,run; default = .false.]
<i>lrho_rad</i>	radiation from inflaton decay [init,run; default = .false.]
<i>lrho_rad_apply</i>	radiation from inflaton decay, and also applied to <code>df(l1:l2,m,n,iinfl_dphi)</code> [run; default = .true.]
<i>lrho_rad_apply2</i>	radiation from inflaton decay, and also applied to <code>df(l1:l2,m,n,iinfl_dphi)</code> [run; default = .true.]
<i>lsigE_const</i>	put <code>sigE</code> to a constant if true. [run; default = .false.]
<i>lsigE_const_if_lsolve_for_phi</i>	allow to check for <code>lsigE_const_if_lsolve_for_phi</code> [run; default = .false.]
<i>lsigE_const_ifnot_lsolve_for_phi</i>	allow to check for <code>lsigE_const_if_lsolve_for_phi</code> [run; default = .false.]
<i>lsmooth_Gamma_phi</i>	smooth increase of <code>Gamma_phi</code> [run; default = .false.]
<i>lsolve_for_phi_always</i>	misnomer: is now used for switching off the phi solver: is now used for switching off the phi solver [init,run; default = .true.]

<i>lsolve_for_phi_switch</i>	switch must be on for automatically switching off the phi solver. [init,run; default = .true.]
<i>lswitch_toMHD_at_lna</i>	option to use lna as criterion for switching to MHD [run; default = .false.]
<i>lswitch_toMHD_when_heating</i>	switch to MHD at the start of reheating [run; default = .false.]
<i>lswitch_toMHD_when_nophi</i>	switch to MHD when phi evolution is turned off (phi is not included in current MHD solver). [run; default = .true.]
<i>lwstate_crit</i>	lwstate_crit switch (would put phi=0, is false by default) [init,run; default = .true.]
<i>lwstate_crit_old</i>	lwstate_crit_old (to restore the old wstate criterion used in the 0d-tests/reheating autotest) [init,run; default = .false.]
<i>wstate_crit</i>	critical w (EoS) value (1/3) [init,run; default = 0.333333333]
<i>wstate_tolerance</i>	tolerance w (EoS) value [init,run; default = 0.]

Module 'disp_current.f90'

<i>cdt_sigE</i>	time step constraint from 1/sigE [run; default = 1.]
<i>etaSchw_max</i>	constant value if set [run]
<i>ladvance_ee</i>	can turn off time advance of E. [run; default = .true.]
<i>lcorrect_sign_adphiB_term</i>	correct sign adphiB term [run; default = .false.]
<i>ldt_disp_current</i>	invoke timestep constraint from sigE [run; default = .true.]
<i>lignore_adphiB_term_in_MHD_current</i>	correct sign adphiB term [run; default = .true.]
<i>linclude_dphiB_in_MHD</i>	include dphi*B in MHD approximation [run; default = .false.]
<i>linclude_gphixE_in_MHD</i>	include gphixE in MHD approximation [run; default = .false.]
<i>llate_reset_el_pencil</i>	late reset of el pencil, should probably be true in future. [run; default = .false.]
<i>lohmic_heating_ee</i>	positive definite part of heating with J^2/sigE . [run; default = .true.]
<i>lohmic_heating_justee</i>	positive definite heating with just $E^2 * \text{sigE}$. [run; default = .false.]
<i>lresistive_gauge_disp</i>	resistive gauge when displacement current is solved for. [run; default = .false.]
<i>lresistive_gauge_ee</i>	possibility of resistive gauge when ladvance_ee=F.

<i>sigE_ceiling</i>	[run; default = .false.] ceiling [run]
<i>sigE_const_value</i>	constant value if set [run]
<i>sigE_max</i>	time step constraint from 1/sigE [run]
Module 'eos_idealgas.f90'	
<i>cs0</i>	can be used to set the dimension of velocity; larger values can be used to decrease stratification. Default = 1. [init,run]
<i>gamma</i>	adiabatic index $\gamma = c_p/c_v$. [init,run; default = 5.0/3.0]
<i>rho0</i>	reference values of sound speed and density, i.e. values at height <i>zref</i> . [init,run; default = 1.]
Module 'gravity_simple.f90'	
<i>grav_amp</i>	specific to the tilted gravity case (amplitude). [init,run; default = 0.0]
<i>grav_tilt</i>	specific to the tilted gravity case (angle wrt the vertical direction). [init,run; default = 0.0]
<i>gravz</i>	vertical gravity component g_z . [init,run; default = 0.0]
<i>gravz_profile</i>	constant gravity $g_z = \text{gravz}$ (<i>gravz_profile</i> ='const') gravity or linear profile $g_z = \text{gravz} \cdot z$ (<i>gravz_profile</i> ='linear', for accretion discs and similar). [init,run; default = 'zero']
<i>nu_epicycle</i>	vertical epicyclic frequency; for accretion discs it should be equal to Omega, but not for galactic discs; see Eq. (149) in Sect. C.5. [init,run; default = 1.0]
<i>z1</i>	specific to the solar convection case <i>initlnrho</i> ='piecew-poly'. The stable layer is $z_0 < z < z_1$, the unstable layer $z_1 < z < z_2$, and the top (isothermal) layer is $z_2 < z < z_{\text{top}}$. [init; default = 0.0]
<i>z2</i>	specific to the solar convection case <i>initlnrho</i> ='piecew-poly'. The stable layer is $z_0 < z < z_1$, the unstable layer $z_1 < z < z_2$, and the top (isothermal) layer is $z_2 < z < z_{\text{top}}$. [init; default = 1.0]
<i>zref</i>	reference height where in the initial stratification $c_s^2 = c_{s0}^2$ and $\ln \rho = \ln \rho_0$. Default = 0. [init,run]
Module 'meanfield.f90'	
<i>alpha_tdep_t1</i>	start and end time of temporal modulation of alpha effect [run; default = 0., alpha_tdep_t2=0.]

<i>alpha_tdep_type</i>	type of temporal modulation of alpha effect [run]
<i>etat_tdep_t1</i>	start and end time of temporal modulation of turbulent diffusivity [run; default = 0., etat_tdep_t2=0.]
<i>etat_tdep_type</i>	type of temporal modulation of turbulent diffusivity [run]
<i>lalpha_tdep</i>	true if temporal modulation of alpha effect is invoked [run; default = .false.]
<i>letat_tdep</i>	true if temporal modulation of turbulent diffusivity is invoked [run; default = .false.]
Module 'particles_breakup.f90'	
<i>nu_liq</i>	kinematic viscosity of the liquid (in cgs units) [init,run; default = 1e-2]
Module 'particles_stalker.f90'	
<i>dstalk</i>	times between printout of stalker data [init,run; default = 0.1]
<i>lstalk_ap</i>	particles diameter [init; default = .true.]
<i>lstalk_bb</i>	magnetic field at particles position [init; default = .true.]
<i>lstalk_grho</i>	gas density gradient at particles position [init; default = .false.]
<i>lstalk_guu</i>	gas velocity gradient at particles position [init; default = .false.]
<i>lstalk_relvel</i>	particles relative velocity to gas [init; default = .false.]
<i>lstalk_rho</i>	gas density at particles position [init; default = .true.]
<i>lstalk_uu</i>	gas velocity at particles position [init; default = .true.]
<i>lstalk_vv</i>	particles velocity [init; default = .true.]
<i>lstalk_xx</i>	particles position [init; default = .true.]
Module 'power_spectrum.f90'	
<i>ckxrange</i>	Define the k_x range for power_xy. [run; default = '']
<i>lcorrect_integer_kcalc</i>	use $k=i-N/2$ instead of $i-(N+1)/2$. [run; default = .false.]
<i>trigger_spec</i>	trigger quantity for spectral output [run; default = 'code_time']
Module 'schur_triple_decomp.f90'	
<i>leigvec</i>	compute eigenvectors [run; default = .false.]
Module 'selfgravity.f90'	

<i>lBryan_etal14</i>	Bryan_etal14 convention, where velocity is not comoving. [init; default = .false.]
<i>luse_G_Newton</i>	Use physical Newton constant (in the correct units) [init; default = .false.]
Module ‘viscosity.f90’	
<i>ascale_visc</i>	value of ascale below which nu for recombination is constant [run; default = 1.]
<i>lvisc_const_below_ascale</i>	visc=const for ascale below ascale_visc [run; default = .false.]
<i>PrM</i>	given value to compute nu based on sigEm_all [run; default = 0.0]

K.2 List of startup parameters for ‘start.in’

The following table lists all (at the time of writing, September 2002) namelists used in ‘start.in’, with the corresponding parameters and their default values (in square brackets). Any variable referred to as a *flag* can be set to any nonzero value to switch the corresponding feature on. Not all parameters are used for a given scenario. This list is not necessarily up to date; also, in many cases it can only give an idea of the corresponding initial state; to get more insight and the latest set of parameters, you need to look at the code.

The value ε corresponds to 5 times the smallest number larger than zero. For single precision, this is typically about $\varepsilon \approx 5 \times 1.2 \times 10^{-7} = 6 \times 10^{-7}$; for double precision, $\varepsilon \approx 10^{-15}$.

Variable [default value]	Meaning
Namelist <i>init_pars</i>	
<i>cvsid</i> [' ']	the <i>svn</i> identification string, which allows you to keep track of the version of ‘start.in’.
<i>ip</i> [14]	(anti-)verbosity level: <i>ip</i> =1 produces lots of diagnostic output, <i>ip</i> =14 virtually none.
<i>xyz0</i> [$(-\pi, -\pi, -\pi)$], <i>Lxyz</i> [$(2\pi, 2\pi, 2\pi)$], <i>lperi</i> [(T,T,T)]	determine the geometry of the box. All three are vectors of the form (<i>x</i> -comp., <i>y</i> -comp., <i>z</i> -comp.); <i>xyz0</i> describes the left (lower) corner of the box, <i>Lxyz</i> the box size. <i>lperi</i> specifies whether a direction is considered periodic (in which case the last point is omitted) or not. In all cases, three ghost zones will be added.
<i>lprocz_slowest</i> [T]	if set to F, the ordering of processor numbers is changed, so the <i>z</i> processors are now in the inner loop. Since <i>nprocy</i> =4 is optimal (see Sect. 5.21.2), you may want to put <i>lprocz_slowest</i> =T when <i>nygrid</i> > <i>nzgrid</i> .
<i>lwrite_ic</i> [F]	if set T, the initial data are written into the file ‘VAR0’. This is generally useful, but doing this all the time uses up plenty of disk space.

<i>lnowrite</i> [F]	if set T, all initialization files are written, including the param.nml file, except 'var.dat'. This option allows you to use old filevar.dat files, but updates all other initialization files. This could be useful after having changed the code and, in particular, when the 'var.dat' files will be overwritten by 'remesh.csh'.
<i>lwrite_aux</i> [F]	if set T, auxiliary variables (those calculated at each step, but not evolved mathematically) to 'var.dat' after the evolved quantities.
<i>lwrite_2d</i> [F]	if set T, only 2D-snapshots are written into VAR files in the case of 2D-runs with <i>nygrid</i> = 1 or <i>nzgrid</i> = 1.
<i>lread_aux</i> [F]	This controls whether auxiliary variables are read from snapshots. This is only required for configurations in which the auxiliary variables are actually not calculated at each timestep (e.g., when you set <i>kinematic_flow</i> ='from-snap' in <i>hydro.run.pars</i>).
<i>lread_oldsnap</i> [F]	if set T, the old snapshot will be read in by start.csh before producing (overwriting) initial conditions. For example, if you just want to add a perturbation to the magnetic field, you'd give no initial condition for density and velocity (so you keep the data from a hopefully relaxed run), and just add whatever you need for the magnetic field. In this connection you may want to touch NOERASE, so as not to erase the previous data. Also, in filestart.in, put: <i>ireset_tstart</i> =0, <i>lread_oldsnap</i> =T, <i>lwrite_var_anyway</i> =T
<i>lread_oldsnap_nomag</i> [F]	if set T, the old snapshot from a non-magnetic run will be read in before producing (overwriting) initial conditions. This allows one to let a hydrodynamic run relax before adding a magnetic field. However, for this to work one has to modify <i>manually</i> 'data/param.nml' by adding an entry for <i>MAGNETIC_INIT_PARS</i> or <i>PSCALAR_INIT_PARS</i> . In addition, for <i>idl</i> to read correctly after the first restarted run, you must adjust the value of <i>mvar</i> in 'data/dim.dat'
<i>lread_oldsnap_nopsalar</i> [F]	if set T, the old snapshot from a run without passive scalar will be read in before producing (overwriting) initial conditions. This allows one to let a hydrodynamic run relax before adding a passive scalar.
<i>lshift_origin</i> [F,F,F]	if set T for any or some of the three directions, the mesh is shifted by 1/2 meshpoint in that or those directions so that the mesh goes through the origin.
<i>lshift_origin_lower</i> [F,F,F]	if set T for any or some of the three directions, the mesh is shifted by 1/2 meshpoint <i>downward</i> in that or those directions so that, again, the mesh goes through the origin.
<i>unit_system</i> ['cgs']	you can set this character string to 'SI', which means that you can give physical dimensions in SI units. The default is cgs units.

<i>unit_length</i> [1]	allows you to set the unit length. Suppose you want the unit length to be 1 kpc, then you would say <code>unit_length='3e21'</code> . (Of course, politically correct would be to say <code>unit_system='SI'</code> in which case you say <code>unit_length='3e19'</code> .)
<i>unit_velocity</i> [1]	Example: if you want km/s you say <code>unit_length='1e5'</code> .
<i>unit_density</i> [1]	Example: if you want your unit density to be 10^{-24} g/cm ³ you say <code>unit_density='1e-24'</code> .
<i>unit_temperature</i> [1]	Example: <code>unit_temperature='1e6'</code> if you want mega-Kelvin.
<i>random_gen</i> [min_std]	choose random number generator; currently valid choices are 'system' (your compiler's generator), 'min_std' (the 'minimal standard' generator <code>ran0()</code> from 'Numerical Recipes'), 'nr_f90' (the Parker-Miller-Marsaglia generator <code>ran()</code> from 'Numerical Recipes for F90').
<i>bcx</i> [('p', 'p', ...)], <i>bcy</i> [('p', 'p', ...)], <i>bcz</i> [('p', 'p', ...)]	boundary conditions. See Sect. 5.17 for a discussion of where and how to set these.
<i>pretend_lnTT</i> [F]	selects $\ln T$ as fundamental thermodynamic variable in the entropy module

Namelist *hydro_init_pars*

<i>inituu</i> ['zero']	initialization of velocity. Currently valid choices are 'zero' ($\mathbf{u} = 0$), 'gaussian-noise' (random, normally-distributed u_x, u_z), 'gaussian-noise-x' (random, normally-distributed u_x), 'sound-wave' (sound wave in x direction), 'shock-tube' (polytropic standing shock), 'bullets' (blob-like velocity perturbations), 'Alfven-circ-x' (circularly polarized Alfven wave in x direction), 'const-ux' (constant x -velocity), 'const-uy' (constant y -velocity), 'tang-discont-z' (tangential discontinuity: velocity is directed along x , jump is at $z = 0$), 'Fourier-trunc' (truncated Fourier series), 'up-down' (flow upward in one spot, downward in another; not solenoidal).
<i>ampluu</i> [0.]	amplitude for some types of initial velocities.
<i>widthuu</i> [0.1]	width for some types of initial velocities.

<i>urand</i> [0.]	additional random perturbation of u . If $urand > 0$, the perturbation is additive, $u_i \mapsto u_i + u_{rand} \mathcal{U}_{[0.5, 0.5]}$; if $urand < 0$, it is multiplicative, $u_i \mapsto u_i \times u_{rand} \mathcal{U}_{[0.5, 0.5]}$; in both cases, $\mathcal{U}_{[0.5, 0.5]}$ is a uniformly distributed random variable on the interval $[-0.5, 0.5]$.
<i>uu_left</i> [0.], <i>uu_right</i> [0.]	needed for <code>inituu='shock-tube'</code> .
Namelist <i>density_init_pars</i>	
<i>initlnrho</i> ['zero']	initialization of density. Currently valid choices are 'zero' ($\ln \rho = 0$), 'isothermal' (isothermal stratification), 'polytropic_simple' (polytropic stratification), 'hydrostatic-z-2' (hydrostatic vertical stratification for isentropic atmosphere), 'xjump' (density jump in x of width <i>widthlnrho</i>), 'rho-jump-z' (density jump in z of width <i>widthlnrho</i>), 'piecew-poly' (piecewise polytropic vertical stratification for solar convection), 'polytropic' (polytropic vertical stratification), 'sound-wave' (sound wave), 'shock-tube' (polytropic standing shock), 'gaussian-noise' (Gaussian-distributed, uncorrelated noise), 'gaussian-noise' (Gaussian-distributed, uncorrelated noise in x , but uniform in y and z), 'hydrostatic-r' (hydrostatic radial density stratification for isentropic or isothermal sphere), 'sin-xy' (sine profile in x and y), 'sin-xy-rho' (sine profile in x and y , but in ρ , not $\ln \rho$), 'linear' (linear profile in $k \cdot x$), 'planet' (planet solution; see §C.7).
<i>gamma</i> [5./3]	adiabatic index $\gamma = c_p/c_v$.
<i>cs0</i> [1.]	can be used to set the dimension of velocity; larger values can be used to decrease stratification
<i>rho0</i> [1.]	reference values of sound speed and density, i. e. values at height <i>zref</i> .
<i>ampllnrho</i> [0.], <i>widthlnrho</i> [0.1]	amplitude and width for some types of initial densities.
<i>rho_left</i> [1.], <i>rho_right</i> [1.]	needed for <code>initlnrho='shock-tube'</code> .
<i>cs2bot</i> [1.], <i>cs2top</i> [1.]	sound speed at bottom and top. Needed for some types of stratification.
Namelist <i>grav_init_pars</i>	

<i>zref</i> [0.]	reference height where in the initial stratification $c_s^2 = c_{s0}^2$ and $\ln \rho = \ln \rho_0$.
<i>gravz</i> [-1.]	vertical gravity component g_z .
<i>gravz_profile</i> [‘const’]	constant gravity $g_z = \text{gravz}$ (<i>gravz_profile</i> =‘const’) gravity or linear profile $g_z = \text{gravz} \cdot z$ (<i>gravz_profile</i> =‘linear’, for accretion discs and similar).
<i>z1</i> [0.], <i>z2</i> [1.]	specific to the solar convection case <i>initlnrho</i> =‘piecew-poly’. The stable layer is $z_0 < z < z_1$, the unstable layer $z_1 < z < z_2$, and the top (isothermal) layer is $z_2 < z < z_{\text{top}}$.
<i>nu_epicycle</i> [1.]	vertical epicyclic frequency; for accretion discs it should be equal to Ω , but not for galactic discs; see Eq. (149) in Sect. C.5.
<i>grav_amp</i> [0.], <i>grav_tilt</i> [0.]	specific to the tilted gravity case (amplitude and angle wrt the vertical direction).
Namelist <i>entropy_init_pars</i>	
<i>initss</i> [‘nothing’]	initialization of entropy. Currently valid choices are ‘nothing’ (leaves the initialization done in the density module unchanged), ‘zero’ (put $s = 0$ explicitly; this may overwrite the initialization done in the density module), ‘isothermal’ (isothermal stratification, $T = \text{const}$), ‘isobaric’ (isobaric, $p = \text{const}$), ‘isentropic’ (isentropic with superimposed hot [or cool] bubble), ‘linprof’ (linear entropy profile in z), ‘piecew-poly’ (piecewise polytropic stratification for convection), ‘polytropic’ (polytropic stratification, polytropic exponent is <i>mpoly0</i>), ‘blob’ (puts a gaussian blob in entropy for buoyancy experiments; see Ref. [9] for details) ‘xjump’ (jump in x direction), ‘hor-tube’ (horizontal flux tube in entropy, oriented in the y -direction).
<i>pertss</i> [‘zero’]	additional perturbation to entropy. Currently valid choices are ‘zero’ (no perturbation) ‘hexagonal’ (hexagonal perturbation for convection).
<i>ampl_ss</i> [0.], <i>widthss</i> [2ϵ]	amplitude and width for some types of initial entropy.
<i>grads0</i> [0.]	initial entropy gradient for <i>initss</i> =linprof.
<i>radius_ss</i> [0.1]	radius of bubble for <i>initss</i> =isentropic.

mpoly0 [1.5],
mpoly1 [1.5],
mpoly2 [1.5]

specific to the solar convection case
 initss=piecew-poly: polytropic indices of unstable
 (*mpoly0*), stable (*mpoly1*) and top layer (*mpoly2*). If
 the flag *isothtop* is set, the top layer is initialized to
 be isothermal, otherwise thermal (plus hydrostatic)
 equilibrium is assumed for all three layers, which
 results in a piecewise polytropic stratification.

isothtop [0]
khorrss [1.]

flag for isothermal top layer for initss=piecew-poly.
 horizontal wave number for pertss=hexagonal

Namelist *magnetic_init_pars*

initaa ['zero']

initialization of magnetic field (vector potential).
 Currently valid choices are

'Alfven-x' (Alfvén wave traveling in the x -
 direction; this also sets the velocity),

'Alfven-z' (Alfvén wave traveling in the z -
 direction; this also sets the velocity),

'Alfvenz-rot' (same as 'Alfven-z', but with rota-
 tion),

'Alfven-circ-x' (circularly polarized Alfven wave
 in x direction),

'Beltrami-x' (x -dependent Beltrami wave),

'Beltrami-y' (y -dependent Beltrami wave),

'Beltrami-z' (z -dependent Beltrami wave),

'Bz(x)' ($B_z \propto \cos(kx)$),

'crazy' (for testing purposes).

'diffrot' ([needs to be documented]),

'fluxrings' (two interlocked magnetic fluxrings;
 see § C.4),

'gaussian-noise' (white noise),

'halfcos-Bx' ([needs to be documented]),

'hor-tube' (horizontal flux tube in B , oriented in
 the y -direction).

'hor-fluxlayer' (horizontal flux layer),

'mag-support' ([needs to be documented]),

'mode' ([needs to be documented]),

'modeb' ([needs to be documented]),

'propto-ux' ([needs to be documented]),

'propto-uy' ([needs to be documented]),

'propto-uz' ([needs to be documented]),

'sinxsinz' ($\sin x \sin z$),

'uniform-Bx' (uniform field in x direction),

'uniform-By' (uniform field in y direction),

'uniform-Bz' (uniform field in z direction),

'zero' (zero field),

<i>initaa2</i> [‘zero’]	additional perturbation of magnetic field. Currently valid choices are ‘zero’ (zero perturbation), ‘Beltrami-x’ (x -dependent Beltrami wave), ‘Beltrami-y’ (y -dependent Beltrami wave), ‘Beltrami-z’ (z -dependent Beltrami wave).
<i>amplaa</i> [0.] <i>amplaa2</i> [0.]	amplitude for some types of initial magnetic fields. amplitude for some types of magnetic field perturbation.
<i>fring</i> {1,2} [0.], <i>Iring</i> {1,2} [0.], <i>Rring</i> {1,2} [1.], <i>wr</i> {1,2} [0.3]	flux, current, outer and inner radius of flux ring 1/2; see Sect. C.4.
<i>radius</i> [0.1]	used by some initial fields.
<i>epsilonaa</i> [10^{-2}]	used by some initial fields.
<i>widthaa</i> [0.5]	used by some initial fields.
<i>z0aa</i> [0.]	used by some initial fields.
<i>kx_aa</i> [1.], <i>ky_aa</i> [1.], <i>kz_aa</i> [1.]	wavenumbers used by some initial fields.
<i>lpress.equil</i> [F]	flag for pressure equilibrium (can be used in connection with all initial fields)

Namelist *pscalar_init_pars*

<i>initlncc</i> [‘zero’]	initialization of passive scalar (concentration per unit mass, c). Currently valid choices (for $\ln c$) are ‘zero’ ($\ln c = 0$.), ‘gaussian-noise’ (white noise), ‘wave-x’ (wave in x direction), ‘wave-y’ (wave in y direction), ‘wave-z’ (wave in z direction), ‘tang-discont-z’ (Kelvin-Helmholtz instability), ‘hor-tube’ (horizontal tube in concentration; used as a marker for magnetic flux tubes).
<i>initlncc2</i> [‘zero’]	additional perturbation of passive scalar concentration c . Currently valid choices are ‘zero’ ($\delta \ln c = 0$.), ‘wave-x’ (add x -directed wave to $\ln c$).
<i>ampllncc</i> [0.] <i>ampllncc2</i> [0.]	amplitude for some types of initial concentration. amplitude for some types of concentration perturbation.
<i>kx_lncc</i> [1.], <i>ky_lncc</i> [1.], <i>kz_lncc</i> [1.]	wave numbers for some types of initial concentration.

Namelist *shear_init_pars*

<i>qshear</i> [0.]	degree of shear for shearing-box simulations (the shearing-periodic boundaries are the x -boundaries and are sheared in the y -direction). The shear velocity is $\mathbf{U} = -q\Omega x \hat{\mathbf{y}}$.
--------------------	---

Namelist *particles_ads_init_pars*

<i>init_ads_mol_frac</i> [0.]	initial adsorbed mole fraction
-------------------------------	--------------------------------

Namelist *particles_surf_init_pars*

<i>init_surf_mol_frac</i> [0.]	initial surface mole fraction
--------------------------------	-------------------------------

Namelist *particles_chem_init_pars*

<i>total_carbon_sites</i> [$1.08e - 8$]	carbon sites per surface area [mol/cm] ²
---	---

Namelist *particles_stalker_init_pars*

<i>dstalk</i> [0.1]	times between printout of stalker data
<i>lstalk_xx</i> [F]	particles position
<i>lstalk_vv</i> [F]	particles velocity
<i>lstalk_uu</i> [F]	gas velocity at particles position
<i>lstalk_guu</i> [F]	gas velocity gradient at particles position
<i>lstalk_rho</i> [F]	gas density at particles position
<i>lstalk_grho</i> [F]	gas density gradient at particles position
<i>lstalk_ap</i> [F]	particles diameter
<i>lstalk_bb</i> [T]	magnetic field at particles position
<i>lstalk_relvel</i> [F]	particles relative velocity to gas

K.3 List of runtime parameters for ‘run.in’

The following table lists all (at the time of writing, September 2002) namelists used in file ‘run.in’, with the corresponding parameters and their default values (in square brackets). Default values marked as [start] are taken from ‘start.in’. Any variable referred to as a *flag* can be set to any nonzero value to switch the corresponding feature on. Not all parameters are used for a given scenario. This list is not necessarily up to date; also, in many cases it can only give an idea of the corresponding setup; to get more insight and the latest set of parameters, you need to look at the code.

Once you have changed any of the ‘*.in’ files, you may want to first execute the command `pc_configtest` in order to test the correctness of these configuration files, before you apply them in an active simulation run.

<i>Variable [default value]</i>	<i>Meaning</i>
Namelist <i>run_pars</i>	
<i>cvsid</i> [‘ ’]	svn identification string, which allows you to keep track of the version of ‘run.in’.
<i>ip</i> [14]	(anti-)verbosity level: <code>ip=1</code> produces lots of additional diagnostic output, <code>ip=14</code> virtually none.
<i>nt</i> [0]	number of time steps to run. This number can be increased or decreased during the run by touch RELOAD.

<i>it1</i> [10]	write diagnostic output every <i>it1</i> time steps (see Sect. 5.6).
<i>it1d</i> [<i>it1</i>]	write averages every <i>it1d</i> time steps (see Sect. 5.9.1). <i>it1d</i> has to be greater than or equal to <i>it1</i> .
<i>cdt</i> [0.9]	Courant coefficient for advective time step; see §5.16.
<i>cdtv</i> [0.25]	Courant coefficient for diffusive time step; see §5.16.
<i>dt</i> [0.]	time step; if $\neq 0$., this overwrites the Courant time step. See §5.16 for a discussion of the latter.
<i>dtmin</i> [10^{-6}]	abort if time step $\delta t < \delta t_{\min}$.
<i>tmax</i> [10^{33}]	don't run time steps beyond this time. Useful if you want to run for a given amount of time, but don't know the necessary number of time steps.
<i>isave</i> [100]	update current snapshot ‘var.dat’ every <i>isave</i> time steps.
<i>itorder</i> [3]	order of time step (1 for Euler; 2 for 2nd-order, 3 for 3rd-order Runge–Kutta).
<i>dsnap</i> [100.]	save permanent snapshot every <i>dsnap</i> time units to files ‘VAR <i>N</i> ’, where <i>N</i> counts from <i>N</i> = 1 upward. (This information is stored in the file ‘data/tsnap.dat’; see the module <i>wsnaps.f90</i> , which in turn uses the subroutines <i>out1</i> and <i>out2</i>).
<i>dvid</i> [100.]	write two-dimensional sections for generation of videos every <i>dvid</i> time units (not timesteps; see the subroutines <i>out1</i> and <i>out2</i> in the code).
<i>iwig</i> [0]	if $\neq 0$, apply a Nyquist filter (a filter eliminating any signal at the Nyquist frequency, but affecting large scales as little as possible) every <i>iwig</i> time steps to logarithmic density (sometimes necessary with convection simulations).
<i>ix</i> [−1], <i>iy</i> [−1], <i>iz</i> [−1], <i>iz2</i> [−1]	position of slice planes for video files. Any negative value of some of these variables will be overwritten according to the value of <i>slice_position</i> . See § 5.8) for details.
<i>slice_position</i> [‘p’]	symbolic specification of slice position. Currently valid choices are ‘p’ (<i>periphery</i> of the box) ‘m’ (<i>middle</i> of the box) ‘e’ (<i>equator</i> for half-sphere calculations, i. e. <i>x</i> , <i>y</i> centered, <i>z</i> bottom) These settings are overridden by explicitly setting <i>ix</i> , <i>iy</i> , <i>iz</i> or <i>iz2</i> . See § 5.8) for details.
<i>zbot_slice</i> [value]	<i>z</i> position of slice xy-plane. The value can be any float number inside the <i>z</i> domain. These settings are overridden by explicitly setting <i>ix</i> , <i>iy</i> , <i>iz</i> or <i>iz2</i> . Saved as slice with the suffix <i>xy</i> . See § 5.8) for details.
<i>ztop_slice</i> [value]	<i>z</i> position of slice xy-plane. The value can be any float number inside the <i>z</i> domain. These settings are overridden by explicitly setting <i>ix</i> , <i>iy</i> , <i>iz</i> or <i>iz2</i> . Saved as slice with the suffix <i>xy2</i> . See § 5.8) for details.

<i>tavg</i> [0]	averaging time τ_{avg} for time averages (if $\neq 0$); at the same time, time interval for writing time averages. See § 5.9.4 for details.
<i>idx_tavg</i> [(0, 0, ..., 0)]	indices of variables to time-average. See § 5.9.4 for details.
<i>d2davg</i> [100.]	time interval for azimuthal and z -averages, i.e. the averages that produce 2d data. See § 5.9.3 for details.
<i>ialive</i> [0]	if $\neq 0$, each processor writes the current time step to ‘alive.info’ every <i>ialive</i> time steps. This provides the best test that the job is still alive. (This can be used to find out which node has crashed if there is a problem and the run is hanging.)
<i>bcx</i> [(‘p’, ‘p’, ...)], <i>bcy</i> [(‘p’, ‘p’, ...)], <i>bcz</i> [(‘p’, ‘p’, ...)]	boundary conditions. See Sect. 5.17 for a discussion of where and how to set these.
<i>random_gen</i> [start]	see start parameters, p. 197
<i>lwrite_aux</i> [start]	if set T, auxiliary variables (those calculated at each step, but not evolved mathematically) to ‘var.dat’ and ‘VAR’ files after the evolved quantities.
<i>dspec</i> [value]	time unit to compute the power spectra. See subsect. 5.23
<i>oned</i> [T]	if set T, the Fourier spectra is computed only in the x direction.
<i>onedall</i> [F]	if set T, the Fourier spectra is computed in all directions.
<i>vel_spec</i> [F]	if set T, computes the velocity power spectra.
<i>ou_spec</i> [F]	if set T, computes the power and helicity spectra.
<i>ab_spec</i> [F]	if set T, computes the magnetic power spectra.
<i>xy_spec</i> [string]	if set, computes extra power spectra files. For instance a value of ‘uz’ will generate the file ‘poweruz_xy.dat’, with information on wavenumbers and z positions.

Namelist *hydro_run_pars*

<i>Omega</i> [0.]	magnitude of angular velocity for <i>Coriolis force</i> (note: the centrifugal force is turned off by default, unless <i>lcentrifugal_force</i> =T is set).
<i>theta</i> [0.]	direction of angular velocity in degrees ($\theta = 0$ for z -direction, $\theta = 90$ for the negative x -direction, corresponding to a box located at the equator of a rotating sphere. Thus, e.g., $\theta = 60$ corresponds to 30° latitude. (Note: prior to April 29, 2007, there was a minus sign in the definition of θ .)
<i>ttransient</i> [0.]	initial time span for which to do something special (transient). Currently just used to smoothly switch on heating [Should be in <i>run_pars</i> , rather than here].
<i>dampu</i> [0.], <i>tdamp</i> [0.],	

<i>ldamp_fade</i> [F]	damp motions during the initial time interval $0 < t < t_{\text{damp}}$ with a damping term $-damp_u(u)$. If <i>ldamp_fade</i> is set, smoothly reduce damping to zero over the second half of the time interval <i>tdamp</i> . Initial velocity damping is useful for situations where initial conditions are far from equilibrium.
<i>dampuint</i> [0.],	weighting of damping external to spherical region (see <i>wdamp</i> , <i>damp_u</i> below).
<i>dampuext</i> [0.],	weighting of damping in internal spherical region (see <i>wdamp</i> , <i>damp_u</i> below).
<i>rdampint</i> [0.],	radius of internal damping region
<i>rdampext</i> [impossible],	radius of external damping region, used in place of former variable <i>rdamp</i>
<i>wdamp</i> [0.2],	permanently damp motions in $ x < r_{\text{dampint}}$ with damping term $-damp_{u\text{int}} u \chi(r - r_{\text{dampint}})$ or $ x > r_{\text{dampext}}$ with damping term $-damp_{u\text{ext}} u \chi(r - r_{\text{dampext}})$, where $\chi(\cdot)$ is a smooth profile of width <i>wdamp</i> .
<i>ampl_forc</i> [0.],	amplitude of the ux-forcing or uy-forcing on the vertical boundaries that is of the form $u_x(t) = \text{ampl_forc} * \sin(k_forc * x) * \cos(w_forc * t)$ [must be used in connection with <i>bcx</i> =‘g’ or <i>bcz</i> =‘g’ and <i>force_lower_bound</i> =‘vel.time’ or <i>force_upper_bound</i> =‘vel.time’]
<i>k_forc</i> [0.],	corresponding horizontal wavenumber
<i>w_forc</i> [0.]	corresponding frequency

Namelist *density_run_pars*

<i>cs0</i> [start],	see start parameters, p. 198 Coefficient for mass diffusion (diffusion term will be $c_{\text{diffrho}} \delta x c_{s0}$.
<i>rho0</i> [start],	
<i>gamma</i> [start]	
<i>cdiffrho</i> [0.]	
<i>cs2bot</i> [start],	squared sound speed at bottom and top for boundary condition ‘c2’.
<i>cs2top</i> [start]	
<i>lupw_lnrho</i> [.false.]	use 5th-order upwind derivative operator for the advection term $u \cdot \nabla \ln \rho$ to avoid spurious Nyquist signal (‘wiggles’); see §H.2.

Namelist *entropy_run_pars*

hcond0 [0.],
hcond1 [start],

<i>hcond2</i> [start]	specific to the solar convection case initss=piecew-poly: heat conductivities K in the individual layers. <i>hcond0</i> is the value K_{unst} in the unstable layer, <i>hcond1</i> is the <i>ratio</i> $K_{\text{stab}}/K_{\text{unst}}$ for the stable layer, and <i>hcond2</i> is the <i>ratio</i> $K_{\text{top}}/K_{\text{unst}}$ for the top layer. The function $K(z)$ is not discontinuous, as the transition between the different values is smoothed over the width <i>widthss</i> . If <i>hcond1</i> or <i>hcond2</i> are not set, they are calculated according to the polytropic indices of the initial profile, $K \propto m+1$.
<i>iheatcond</i> ['K-const']	select type of heat conduction. Currently valid choices are 'K-const' (constant heat conductivity), 'K-profile' (vertical or radial profile), 'chi-const' (constant thermal diffusivity), 'magnetic' (heat conduction by electrons in magnetic field – currently still experimental).
<i>lcalc_heatcond_constchi</i> [F]	flag for assuming thermal diffusivity $\chi = K/(c_p\rho) = \text{const}$, rather than $K = \text{const}$ (which is the default). <i>This is currently only correct with 'noionization.f90'</i> . Superseded by <i>iheatcond</i> .
<i>chi</i> [0.]	value of χ when <i>lcalc_heatcond_constchi</i> =T.
<i>widthss</i> [start]	width of transition region between layers. See start parameters, p. 200.
<i>isothtop</i> [start]	flag for isothermal top layer for solar convection case. See start parameters, p. 200.
<i>luminosity</i> [0.], <i>wheat</i> [0.1] <i>cooltype</i> ['Temp']	strength and width of heating region. type of cooling; <i>currently only implemented for spherical geometry</i> . Currently valid choices are 'Temp', 'cs2' (cool temperature toward $c_s^2 = \text{cs2cool}$) with a cooling term $-\mathcal{C} = -c_{\text{cool}} \frac{c_s^2 - c_{s\text{cool}}^2}{c_{s\text{cool}}^2}$) 'Temp-rho', 'cs2-rho' (cool temperature toward $c_s^2 = \text{cs2cool}$) with a cooling term $-\mathcal{C} = -c_{\text{cool}} \rho \frac{c_s^2 - c_{s\text{cool}}^2}{c_{s\text{cool}}^2}$ — this avoids numerical instabilities in low-density regions [currently, the cooling coefficient $c_{\text{cool}} \equiv \text{cool}$ is not taken into account when the time step is calculated]) 'entropy' (cool entropy toward 0.).
<i>cool</i> [0.],	

<i>wcool</i> [0.1]	strength c_{cool} and smoothing width of cooling region.
<i>rcool</i> [1.]	radius of cooling region: cool for $ x \geq r_{\text{cool}}$.
<i>Fbot</i> [start]	heat flux for bottom boundary condition ‘c1’. For polytropic atmospheres, if <i>Fbot</i> is not set, it will be calculated from the value of <i>hcond0</i> in ‘start.x’, provided the entropy boundary condition is set to ‘c1’.
<i>chi.t</i> [0.]	entropy diffusion coefficient for diffusive term $\partial s / \partial t = \dots + \chi_t \nabla^2 s$ in the entropy equation, that can represent some kind of turbulent (sub-grid) mixing. It is probably a bad idea to combine this with heat conduction $h_{\text{cond0}} \neq 0$.
<i>lupw_ss</i> [.false.]	use 5th-order upwind derivative operator for the advection term $u \cdot \nabla s$ to avoid spurious Nyquist signal (‘wiggles’); see §H.2.
<i>tauheat_buffer</i> [0.]	time scale for heating to target temperature (= <i>TTheat_buffer</i>); zero disables the buffer zone.
<i>zheat_buffer</i> [0.]	z coordinate of the thermal buffer zone. Buffering is active in $ z > T_{\text{Theat_buffer}}$.
<i>dheat_buffer1</i> [0.]	Inverse thickness of transition to buffered layer.
<i>TTheat_buffer</i> [0.]	target temperature in thermal buffer zone (z direction only).
<i>lhcond_global</i> [F]	flag for calculating the heat conductivity K (and also $\nabla \log K$) globally using the global arrays facility. Only valid when <i>iheatcond</i> =‘K-profile’.
<i>lread_hcond</i> [F]	flag for reading the conductivity profile and its derivative from the file “hcond_glhc.dat” or “hcond_glhc.ascii”.
Namelist <i>magnetic_run_pars</i>	
<i>B_ext</i> [(0., 0., 0.)]	uniform background magnetic field (for fully periodic boundary conditions, uniform fields need to be explicitly added, since otherwise the vector potential A has a linear x -dependence which is incompatible with periodicity).
<i>ignore_Bext_in_b2</i> [F] or <i>luse_Bext_in_b2</i> [T]	add uniform background magnetic field when computing b^2 pencils
<i>eta</i> [0.]	magnetic diffusivity $\eta = 1/(\mu_0 \sigma)$, where σ is the electric conductivity.
<i>height_eta</i> [0.], <i>eta_out</i> [0.]	used to add extra diffusivity in a halo region.
<i>eta_int</i> [0.]	used to add extra diffusivity inside sphere of radius r_{int} .
<i>eta_ext</i> [0.]	used to add extra diffusivity outside sphere of radius r_{ext} .
<i>kx</i> [1.], <i>ky</i> [1.], <i>kz</i> [1.]	wave numbers for <i>ABC</i> flow.
<i>ABC_A</i> [1.], <i>ABC_B</i> [1.], <i>ABC_C</i> [1.]	amplitudes A , B and C for <i>ABC</i> flow.

Namelist *pscalar_run_pars*

<i>pscalar_diff</i> [0.]	diffusion for passive scalar concentration c .
<i>tensor_pscalar_diff</i> [0.]	coefficient for non-isotropic diffusion of passive scalar.
<i>reinitialize_lnc</i> [F]	reinitialize the passive scalar to the value of <i>cc_const</i> in <i>start.in</i> at next run

Namelist *forcing_run_pars*

<i>iforce</i> [2]	select form of forcing in the equation of motion; currently valid choices are 'zero' (no forcing), 'irrotational' (irrotational forcing), 'helical' (helical forcing), 'fountain' (forcing of "fountain flow"; see Ref. [17]), 'horizontal-shear' (forcing localized horizontal sinusoidal shear). 'variable_gravz' (time-dependent vertical gravity for forcing internal waves),
<i>iforce2</i> [0]	select form of additional forcing in the equation of motion; valid choices are as for <i>iforce</i> .
<i>force</i> [0.]	amplitude of forcing.
<i>relhel</i> [1.]	helicity of forcing. The parameter <i>relhel</i> corresponds to σ introduced in Sect. G.2. ($\sigma = \pm 1$ corresponds to maximum helicity of either sign).
<i>height_ff</i> [0.]	multiply forcing by z -dependent profile of width <i>height_ff</i> (if $\neq 0$).
<i>r_ff</i> [0.]	if $\neq 0$, multiply forcing by spherical cutoff profile (of radius <i>r_ff</i>) and flip signs of helicity at equatorial plane.
<i>width_ff</i> [0.5]	width of vertical and radial profiles for modifying forcing.
<i>kfountain</i> [5]	horizontal wavenumber of the fountain flow.
<i>fountain</i> [1.]	amplitude of the fountain flow.
<i>omega_ff</i> [1.]	frequency of the cos or sin forcing [e.g., $\cos(\text{omega_ff} \cdot t)$].
<i>ampl_ff</i> [1.]	amplitude of forcing in front of cos or sin [e.g., $\text{ampl_ff} \cdot \cos(\text{omega_ff} \cdot t)$].

Namelist *grav_run_pars*

<i>zref</i> [start],	
<i>gravz</i> [start],	
<i>gravz_profile</i> [start]	see p. 199.
<i>nu_epicycle</i> [start]	see Eq. (149) in Sect. C.5.

Namelist *viscosity_run_pars*

<i>nu</i> [0.]	kinematic viscosity.
<i>nu_hyper2</i> [0.]	kinematic hyperviscosity (with $\nabla^4 u$).
<i>nu_hyper3</i> [0.]	kinematic hyperviscosity (with $\nabla^6 u$).
<i>zeta</i> [0.]	bulk viscosity.

<i>ivisc</i> [‘nu-const’]	select form of viscous term (see §6.2); currently valid choices are
‘nu-const’	– viscous force for $\nu = \text{const}$, $\mathbf{F}_{\text{visc}} = \nu(\nabla^2 \mathbf{u} + \frac{1}{3} \nabla \nabla \cdot \mathbf{u} + 2\mathbf{S} \cdot \nabla \ln \rho)$
‘rho_nu-const’	– viscous force for $\mu \equiv \rho\nu = \text{const}$, $\mathbf{F}_{\text{visc}} = (\mu/\rho)(\nabla^2 \mathbf{u} + \frac{1}{3} \nabla \nabla \cdot \mathbf{u})$. With this option, the input parameter <i>nu</i> actually sets the value of μ/ρ_0 (<i>rho0</i> = ρ_0 is another input parameter, see pp. 198 and 205)
‘simplified’	– simplified viscous force $\mathbf{F}_{\text{visc}} = \nu \nabla^2 \mathbf{u}$

Namelist *shear_run_pars*

<i>qshear</i> [start]	See p. 202.
-----------------------	-------------

Namelist *particles_run_pars*

<i>ldragforce_dust_par</i> [F]	dragforce on particles
<i>ldragforce_gas_par</i> [F]	particle-gas friction force
<i>ldraglaw_steadystate</i> [F]	particle forces only with $\frac{1}{\tau} \Delta v$
<i>lpscalar_sink</i> [F]	particles consume passive scalar
<i>pscalar_sink_rate</i> [0]	volumetric pscalar consumption rate
<i>lbubble</i> [F]	addition of the virtual mass term

Namelist *particles_ads_run_pars*

<i>placeholder</i> [start]	placeholder
----------------------------	-------------

Namelist *particles_surf_run_pars*

<i>lspecies_transfer</i> [T]	Species transfer from solid to fluid phase
------------------------------	--

Namelist *particles_chem_run_pars*

<i>lthiele</i> [T]	Modeling of particle porosity by application of Thiele modulus
--------------------	--

Namelist *power_spectrum_run_pars*

<i>inz</i> [0]	Compute the power_x at a given z.
<i>ckxrange</i> ["]	Define the k_x range for power_xy.
<i>ckyrange</i> ["]	Define the k_y range for power_xy.
<i>czrange</i> ["]	Define the z range for power_xy.
<i>lintegrate_z</i> [T]	Compute the z-integrated power spectra.
<i>lintegrate_shell</i> [T]	Compute the shell-integrated power spectra.
<i>lcomplex</i> [F]	outputs the complex Fourier transform.
<i>lcylindrical_spectra</i> [F]	Compute the cylindrical power spectra.
<i>n_segment_x</i> [1]	not yet operational! $\rightarrow$ n_segment_x always 1
<i>lhalf_factor_in_GW</i> [F]	if [F], the total(S)=gg2m; if [T], total(S)= $\frac{1}{2}$ gg2m.
<i>pdf_max</i> [30.]	Maximum value of the pdf.
<i>pdf_min</i> [-30.]	Minimum value of the pdf.
<i>pdf_min_logscale</i> [3.0]	Minimum value of the logarithmic pdf.
<i>pdf_max_logscale</i> [-3.0]	Maximum value of the logarithmic pdf.

<i>lread_gauss_quadrature</i> [F]	if [T], generates polar coordinates in gauss-legendre quadrature; need to provide file <code>gauss_legendre_-quadrature.dat</code>
<i>legendre_lmax</i> [1]	Maximum number of Legendre coefficients in the polar spectrum.

K.4 List of parameters for ‘print.in’

The following table lists all possible inputs to the file ‘print.in’ that are documented.

<i>Variable</i>	<i>Meaning</i>
Module ‘cdata.f90’	
<i>it</i>	number of time step (since beginning of job only)
<i>t</i>	time t (since start.csh)
<i>dt</i>	time step δt
<i>walltime</i>	wall clock time since start of run.x, in seconds
<i>dtv</i>	advective timestep as a fraction of the actual one
<i>dtsrc</i>	timestep from a source term, fraction of the actual one
<i>dtdiffus</i>	diffusive timestep as a fraction of the actual one
<i>dtdiffus2</i>	hyperdiffusive (hyper2) timestep as a fraction of the actual one
<i>dtdiffus3</i>	hyperdiffusive (hyper3) timestep as a fraction of the actual one
<i>Rmesh</i>	R_{mesh}
<i>Rmesh3</i>	$R_{\text{mesh}}^{(3)}$
<i>maxadvec</i>	maxadvec
<i>eps_rkf</i>	time step accuracy threshold
Module ‘hydro.f90’	
<i>u2tm</i>	$\left\langle \mathbf{u}(t) \cdot \int_0^t \mathbf{u}(t') dt' \right\rangle$
<i>uotm</i>	$\left\langle \mathbf{u}(t) \cdot \int_0^t \boldsymbol{\omega}(t') dt' \right\rangle$
<i>outm</i>	$\left\langle \boldsymbol{\omega}(t) \cdot \int_0^t \mathbf{u}(t') dt' \right\rangle$
<i>fkinzm</i>	$\left\langle \frac{1}{2} \rho \mathbf{u}^2 u_z \right\rangle$
<i>gamm</i>	$\langle \gamma \rangle$
<i>gamrms</i>	$\langle \gamma^2 \rangle^{1/2}$
<i>gammax</i>	$\max(\gamma)$
<i>u2m</i>	$\langle \mathbf{u}^2 \rangle$
<i>u2sphm</i>	$\int_{r=0}^{r=r^{\text{diag}}} \mathbf{u}^2 dV$, where $r = \sqrt{x^2 + y^2 + z^2}$
<i>uxpt</i>	$u_x(x_1, y_1, z_1, t)$
<i>uypt</i>	$u_y(x_1, y_1, z_1, t)$
<i>uzpt</i>	$u_z(x_1, y_1, z_1, t)$
<i>uxp2</i>	$u_x(x_2, y_2, z_2, t)$
<i>uyp2</i>	$u_y(x_2, y_2, z_2, t)$
<i>uzp2</i>	$u_z(x_2, y_2, z_2, t)$
<i>uxuypt</i>	$(u_x u_y)(x_1, y_1, z_1, t)$
<i>uyuzpt</i>	$(u_y u_z)(x_1, y_1, z_1, t)$
<i>uzuxpt</i>	$(u_z u_x)(x_1, y_1, z_1, t)$

<i>urms</i>	$\langle \mathbf{u}^2 \rangle^{1/2}$
<i>urmsx</i>	$\langle \mathbf{u}^2 \rangle^{1/2}$ for the hydro_xaver_range
<i>urmsz</i>	$\langle \mathbf{u}^2 \rangle^{1/2}$ for the hydro_zaver_range
<i>durms</i>	$\langle \delta \mathbf{u}^2 \rangle^{1/2}$
<i>umax</i>	$\max(\mathbf{u})$
<i>umin</i>	$\min(\mathbf{u})$
<i>uxrms</i>	$\langle u_x^2 \rangle^{1/2}$
<i>uyrms</i>	$\langle u_y^2 \rangle^{1/2}$
<i>uzrms</i>	$\langle u_z^2 \rangle^{1/2}$
<i>uxmin</i>	$\min(u_x)$
<i>uymin</i>	$\min(u_y)$
<i>uzmin</i>	$\min(u_z)$
<i>uxmax</i>	$\max(u_x)$
<i>uymax</i>	$\max(u_y)$
<i>uzmax</i>	$\max(u_z)$
<i>uxm</i>	$\langle u_x \rangle$
<i>uym</i>	$\langle u_y \rangle$
<i>uzm</i>	$\langle u_z \rangle$
<i>uzcx10m</i>	$\langle u_z \cos 10x \rangle$
<i>uzsx10m</i>	$\langle u_z \sin 10x \rangle$
<i>uduum</i>	$\langle \mathbf{u} \cdot \mathbf{u} \rangle$
<i>ux2m</i>	$\langle u_x^2 \rangle$
<i>uy2m</i>	$\langle u_y^2 \rangle$
<i>uz2m</i>	$\langle u_z^2 \rangle$
<i>ux3m</i>	$\langle u_x^3 \rangle$
<i>uy3m</i>	$\langle u_y^3 \rangle$
<i>uz3m</i>	$\langle u_z^3 \rangle$
<i>ux4m</i>	$\langle u_x^4 \rangle$
<i>uy4m</i>	$\langle u_y^4 \rangle$
<i>uz4m</i>	$\langle u_z^4 \rangle$
<i>u4m</i>	$\langle u^4 \rangle$
<i>u6m</i>	$\langle u^6 \rangle$
<i>u8m</i>	$\langle u^8 \rangle$
<i>uxuy2m</i>	$\langle u_x^2 u_y^2 \rangle$
<i>uyuz2m</i>	$\langle u_y^2 u_z^2 \rangle$
<i>uzux2m</i>	$\langle u_z^2 u_x^2 \rangle$
<i>velxx2m</i>	$\langle w \gamma^2 u_x^2 \rangle$
<i>velxy2m</i>	$\langle w \gamma^2 u_y^2 \rangle$
<i>velxz2m</i>	$\langle w \gamma^2 u_z^2 \rangle$
<i>velxrms</i>	$\langle \sqrt{w \gamma^2 u^2} \rangle^{1/2}$
<i>T00m</i>	$\langle T_{00} \rangle$
<i>Txxm</i>	$\langle T_{xx} \rangle$
<i>Tyym</i>	$\langle T_{yy} \rangle$
<i>Tzzm</i>	$\langle T_{zz} \rangle$
<i>Txym</i>	$\langle T_{xy} \rangle$
<i>Tyzm</i>	$\langle T_{yz} \rangle$
<i>Tzxm</i>	$\langle T_{zx} \rangle$
<i>T0x2m</i>	$\langle T_{0x}^2 \rangle$
<i>T0y2m</i>	$\langle T_{0y}^2 \rangle$

<i>T0z2m</i>	$\langle T_{0z}^2 \rangle$	
<i>T0irms</i>	$\langle T_{0i}^2 \rangle^{1/2}$	
<i>ux2ccm</i>	$\langle u_x^2 \cos^2 kz \rangle$	
<i>ux2ssm</i>	$\langle u_x^2 \sin^2 kz \rangle$	
<i>uy2ccm</i>	$\langle u_y^2 \cos^2 kz \rangle$	
<i>uy2ssm</i>	$\langle u_y^2 \sin^2 kz \rangle$	
<i>uxuyesm</i>	$\langle u_x u_y \cos kz \sin kz \rangle$	
<i>uxuym</i>	$\langle u_x u_y \rangle$	
<i>uxuzm</i>	$\langle u_x u_z \rangle$	
<i>uyuzm</i>	$\langle u_y u_z \rangle$	
<i>umx</i>	$\langle u_x \rangle$	
<i>umy</i>	$\langle u_y \rangle$	
<i>umz</i>	$\langle u_z \rangle$	
<i>omumz</i>	$\langle \langle \mathbf{W} \rangle_{xy} \cdot \langle \mathbf{U} \rangle_{xy} \rangle$	(<i>xy</i> -averaged mean cross helicity production)
<i>umamz</i>	$\langle \langle \mathbf{u} \rangle_{xy} \cdot \langle \mathbf{A} \rangle_{xy} \rangle$	
<i>umbmz</i>	$\langle \langle \mathbf{U} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle$	(<i>xy</i> -averaged mean cross helicity production)
<i>umxbmz</i>	$\langle \langle \mathbf{U} \rangle_{xy} \times \langle \mathbf{B} \rangle_{xy} \rangle_z$	(<i>xy</i> -averaged mean emf)
<i>rux2m</i>	$\langle \rho u_x^2 \rangle$	
<i>ruy2m</i>	$\langle \rho u_y^2 \rangle$	
<i>ruz2m</i>	$\langle \rho u_z^2 \rangle$	
<i>divum</i>	$\langle \text{div} \mathbf{u} \rangle$	
<i>rdivum</i>	$\langle \varrho \text{div} \mathbf{u} \rangle$	
<i>divu2m</i>	$\langle (\text{div} \mathbf{u})^2 \rangle$	
<i>gdivu2m</i>	$\langle (\text{grad div} \mathbf{u})^2 \rangle$	
<i>u3u21m</i>	$\langle u_3 u_{2,1} \rangle$	
<i>u1u32m</i>	$\langle u_1 u_{3,2} \rangle$	
<i>u2u13m</i>	$\langle u_2 u_{1,3} \rangle$	
<i>u2u31m</i>	$\langle u_2 u_{3,1} \rangle$	
<i>u3u12m</i>	$\langle u_3 u_{1,2} \rangle$	
<i>u1u23m</i>	$\langle u_1 u_{2,3} \rangle$	
<i>ruxm</i>	$\langle \varrho u_x \rangle$	(mean <i>x</i> -momentum density)
<i>ruym</i>	$\langle \varrho u_y \rangle$	(mean <i>y</i> -momentum density)
<i>ruzum</i>	$\langle \varrho u_z \rangle$	(mean <i>z</i> -momentum density)
<i>ruxtot</i>	$\langle \rho u \rangle$	(mean absolute <i>x</i> -momentum density)
<i>rumax</i>	$\max(\varrho \mathbf{u})$	(maximum modulus of momentum)
<i>ruxuym</i>	$\langle \varrho u_x u_y \rangle$	(mean Reynolds stress)
<i>ruxuzm</i>	$\langle \varrho u_x u_z \rangle$	(mean Reynolds stress)
<i>ruyuzm</i>	$\langle \varrho u_y u_z \rangle$	(mean Reynolds stress)
<i>divrhours</i>	$ \nabla \cdot (\varrho \mathbf{u}) _{\text{rms}}$	
<i>divrhousmax</i>	$ \nabla \cdot (\varrho \mathbf{u}) _{\text{max}}$	
<i>rlxm</i>	$\langle \rho y u_z - z u_y \rangle$	
<i>rlym</i>	$\langle \rho z u_x - x u_z \rangle$	
<i>rlzm</i>	$\langle \rho x u_y - y u_x \rangle$	
<i>rlx2m</i>	$\langle (\rho y u_z - z u_y)^2 \rangle$	
<i>rlz2m</i>	$\langle (\rho z u_x - x u_z)^2 \rangle$	
<i>rlz2m</i>	$\langle (\rho x u_y - y u_x)^2 \rangle$	

<i>tot_ang_mom</i>	Total angular momentum in spherical coordinates about the axis.
<i>dtu</i>	$\delta t / [c_{\delta t} \delta x / \max \mathbf{u}]$ (time step relative to advective time step; see § 5.16)
<i>oum</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle$
<i>oxum</i>	$\langle \boldsymbol{\omega} \times \mathbf{u} \rangle$
<i>ourms</i>	$\langle (\boldsymbol{\omega} \cdot \mathbf{u})^2 \rangle^{1/2}$
<i>oxurms</i>	$\langle (\boldsymbol{\omega} \times \mathbf{u})^2 \rangle^{1/2}$
<i>ou_int</i>	$\int_V \boldsymbol{\omega} \cdot \mathbf{u} dV$
<i>fum</i>	$\langle \mathbf{f} \cdot \mathbf{u} \rangle$ (continuous forcing only)
<i>odel2um</i>	$\langle \boldsymbol{\omega} \nabla^2 \mathbf{u} \rangle$
<i>o2m</i>	$\langle \boldsymbol{\omega}^2 \rangle \equiv \langle (\nabla \times \mathbf{u})^2 \rangle$
<i>o2u2m</i>	$\langle \mathbf{u}^2 \boldsymbol{\omega}^2 \rangle$
<i>o2sphm</i>	$\int_{r=0}^{r=r^{\text{diag}}} \boldsymbol{\omega}^2 dV$, where $r = \sqrt{x^2 + y^2 + z^2}$
<i>orms</i>	$\langle \boldsymbol{\omega}^2 \rangle^{1/2}$
<i>omax</i>	$\max(\boldsymbol{\omega})$
<i>ox2m</i>	$\langle \omega_x^2 \rangle$
<i>oy2m</i>	$\langle \omega_y^2 \rangle$
<i>oz2m</i>	$\langle \omega_z^2 \rangle$
<i>ox3m</i>	$\langle \omega_x^3 \rangle$
<i>oy3m</i>	$\langle \omega_y^3 \rangle$
<i>oz3m</i>	$\langle \omega_z^3 \rangle$
<i>ox4m</i>	$\langle \omega_x^4 \rangle$
<i>oy4m</i>	$\langle \omega_y^4 \rangle$
<i>oz4m</i>	$\langle \omega_z^4 \rangle$
<i>oxuzxm</i>	$\langle \omega_x u_{z,x} \rangle$
<i>oyuzym</i>	$\langle \omega_y u_{z,y} \rangle$
<i>oxoym</i>	$\langle \omega_x \omega_y \rangle$
<i>oxozm</i>	$\langle \omega_x \omega_z \rangle$
<i>oyozm</i>	$\langle \omega_y \omega_z \rangle$
<i>qfm</i>	$\langle \mathbf{q} \cdot \mathbf{f} \rangle$
<i>q2m</i>	$\langle \mathbf{q}^2 \rangle$
<i>qrms</i>	$\langle \mathbf{q}^2 \rangle^{1/2}$
<i>qmax</i>	$\max(\mathbf{q})$
<i>qom</i>	$\langle \mathbf{q} \cdot \boldsymbol{\omega} \rangle$
<i>quxom</i>	$\langle \mathbf{q} \cdot (\mathbf{u} \times \boldsymbol{\omega}) \rangle$
<i>qsglnrhom</i>	$\langle \mathbf{q} \cdot \mathbf{G} \rangle$
<i>oglnrxgdum</i>	$\langle \boldsymbol{\omega} \cdot (\ln \varrho) \times \nabla \nabla \cdot \mathbf{u} \rangle$
<i>qezxum</i>	$\langle (\mathbf{e}_z \times \mathbf{u}) \cdot \mathbf{q} \rangle$
<i>quysm</i>	$\langle \frac{1}{\tau} (u_y^S - u_y) \hat{\mathbf{y}} \cdot \mathbf{q} \rangle$
<i>jxbrqm</i>	$\langle (\mathbf{J} \times \mathbf{B} / \rho) \cdot \mathbf{q} \rangle$
<i>pvzm</i>	$\langle \omega_z + 2\Omega / \varrho \rangle$ (z component of potential vorticity)
<i>oumph</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_\varphi$
<i>ormph</i>	$\langle \omega_r \rangle_\varphi$
<i>opmph</i>	$\langle \omega_\phi \rangle_\varphi$
<i>ozmph</i>	$\langle \omega_z \rangle_\varphi$
<i>ugurmsx</i>	$\langle (\mathbf{u} \nabla \mathbf{u})^2 \rangle^{1/2}$ for the <code>hydro_xaver_range</code>
<i>gu2m</i>	$\langle (\nabla \mathbf{u})^2 \rangle$
<i>ugu2m</i>	$\langle \mathbf{u} \nabla \mathbf{u} \rangle^2$

<i>dudx</i>	$\langle \frac{\delta \mathbf{u}}{\delta x} \rangle$
<i>Marms</i>	$\langle \mathbf{u}^2 / c_s^2 \rangle$ (rms Mach number)
<i>Mamax</i>	$\max \mathbf{u} / c_s$ (maximum Mach number)
<i>EEK</i>	$\langle \varrho \mathbf{u}^2 \rangle / 2$
<i>EEK2</i>	$\langle (\varrho \mathbf{u}^2 / 2)^2 \rangle$
<i>EEK3</i>	$\langle (\varrho \mathbf{u}^2 / 2)^3 \rangle$
<i>EEK4</i>	$\langle (\varrho \mathbf{u}^2 / 2)^4 \rangle$
<i>ekin</i>	$\langle \frac{1}{2} \varrho \mathbf{u}^2 \rangle$
<i>ekintot</i>	$\int_V \frac{1}{2} \varrho \mathbf{u}^2 dV$
<i>uxglnrym</i>	$\langle u_x \partial_y \ln \varrho \rangle$
<i>uyglnrxm</i>	$\langle u_y \partial_x \ln \varrho \rangle$
<i>uzdivum</i>	$\langle u_z \nabla \cdot \mathbf{u} \rangle$
<i>uxuydivum</i>	$\langle u_x u_y \nabla \cdot \mathbf{u} \rangle$
<i>divuHrms</i>	$(\nabla_H \cdot \mathbf{u}_H)^{\text{rms}}$
<i>uxxrms</i>	$u_{x,x}^{\text{rms}}$
<i>uyyrms</i>	$u_{y,y}^{\text{rms}}$
<i>uzzrms</i>	$u_{z,z}^{\text{rms}}$
<i>uxzrms</i>	$u_{x,z}^{\text{rms}}$
<i>uyzrms</i>	$u_{y,z}^{\text{rms}}$
<i>uzyrms</i>	$u_{z,y}^{\text{rms}}$
<i>coriolis_number</i>	$\left\langle \frac{ 2\boldsymbol{\Omega} \times \mathbf{u} }{ \mathbf{u} \cdot \nabla \mathbf{u} } \right\rangle$
<i>dtF</i>	$\delta t / [c_{\delta t} \delta x / \max \mathbf{F}]$ (time step relative to max force time step; see § 5.16) $\int u_r(\theta, \phi) Y_\ell^m(\theta, \phi) \sin(\theta) d\theta d\phi$
<i>udpxxm</i>	components of symmetric tensor $\langle u_i \partial_j p + u_j \partial_i p \rangle$

Module ‘density.f90’

<i>rhom</i>	$\langle \varrho \rangle$ (mean density)
<i>rhomxmask</i>	$\langle \varrho \rangle$ for the density_xaver_range
<i>rhomzmask</i>	$\langle \varrho \rangle$ for the density_zaver_range
<i>rho2m</i>	$\langle \varrho^2 \rangle$
<i>rho4m</i>	$\langle \varrho^4 \rangle$
<i>rho6m</i>	$\langle \varrho^6 \rangle$
<i>rho8m</i>	$\langle \varrho^8 \rangle$
<i>rho12m</i>	$\langle \varrho^{12} \rangle$
<i>rhof2m</i>	$\langle \varrho'^2 \rangle$
<i>drho2m</i>	$\langle (\varrho - \varrho_0)^2 \rangle$
<i>drhom</i>	$\langle \varrho - \varrho_0 \rangle$
<i>rhomin</i>	$\min(\rho)$
<i>rhomax</i>	$\max(\rho)$
<i>rhominloc</i>	location of $\min(\rho)$
<i>rhomaxloc</i>	location of $\max(\rho)$
<i>lnrhomin</i>	$\min(\log \rho)$
<i>lnrhomax</i>	$\max(\log \rho)$
<i>rhorms</i>	$\sqrt{\langle \varrho^2 \rangle}$
<i>lnrhorms</i>	$\sqrt{\langle (\ln \varrho)^2 \rangle}$
<i>ugrhom</i>	$\langle \mathbf{u} \cdot \nabla \varrho \rangle$
<i>uglnrhom</i>	$\langle \mathbf{u} \cdot \nabla \ln \varrho \rangle$
<i>totmass</i>	$\int \varrho dV$
<i>mass</i>	$\int \varrho dV$
<i>sphmass</i>	$\int \varrho dV$ inside $r < r_{\text{diag}}$.

<i>inertiayy</i>	<i>yy</i> component of the inertia tensor (spherical coordinates)
<i>inertiaz</i>	<i>zz</i> component of the inertia tensor (spherical coordinates)
<i>inertiayy_car</i>	<i>yy</i> component of the inertia tensor (Cartesian coordinates)
<i>inertiaz_car</i>	<i>zz</i> component of the inertia tensor (Cartesian coordinates)
<i>vol</i>	$\int dV$ (volume)
<i>grhomax</i>	$\max(\nabla \rho)$
<i>kap_tdep</i>	time-dependent diffusivity

Module ‘entropy.f90’

<i>dte</i>	$\delta t / [c_{\delta t} \delta x / \max c_s]$ (time step relative to acoustic time step; see § 5.16)
<i>ethm</i>	$\langle \rho e \rangle$ (mean thermal [=internal] energy)
<i>ssruzm</i>	$\langle s \rho u_z / c_p \rangle$
<i>ssuzm</i>	$\langle s u_z / c_p \rangle$
<i>ssm</i>	$\langle s / c_p \rangle$ (mean entropy)
<i>ssbycpm</i>	$\langle s / c_p \rangle$ (mean entropy)
<i>ss2m</i>	$\langle (s / c_p)^2 \rangle$ (mean squared entropy)
<i>ss_run_averm</i>	(mean of the running average of entropy)
<i>eem</i>	$\langle e \rangle$
<i>ppm</i>	$\langle p \rangle$
<i>ppmax</i>	$\max(p)$
<i>ppmin</i>	$\min(p)$
<i>csm</i>	$\langle c_s \rangle$
<i>csmx</i>	$\max(c_s)$
<i>cgam</i>	$\langle c_\gamma \rangle$
<i>pdivum</i>	$\langle p \nabla \cdot \mathbf{u} \rangle$
<i>fradbot</i>	$\int F_{\text{bot}} \cdot d\mathbf{S}$
<i>fradtop</i>	$\int F_{\text{top}} \cdot d\mathbf{S}$
<i>TTtop</i>	$\int T_{\text{top}} d\mathbf{S}$
<i>ethtot</i>	$\int_V \rho e dV$ (total thermal [=internal] energy)
<i>dtchi</i>	$\delta t / [c_{\delta t, v} \delta x^2 / \chi_{\max}]$ (time step relative to time step based on heat conductivity; see § 5.16)
<i>Hmax</i>	$H_{\max}$ (net heat sources summed see § 5.16)
<i>tauhmin</i>	$\min(\tau_{\text{heat}})$
<i>dtH</i>	$\delta t / [c_{\delta t, s} c_v T / H_{\max}]$ (time step relative to time step based on heat sources; see § 5.16)
<i>yHm</i>	mean hydrogen ionization
<i>yHmax</i>	max of hydrogen ionization
<i>TTm</i>	$\langle T \rangle$
<i>TTmax</i>	$T_{\max}$
<i>TTmin</i>	$T_{\min}$
<i>TTmaxloc</i>	location of $T_{\max}$
<i>TTminloc</i>	location of $T_{\min}$
<i>gTmax</i>	$\max(\nabla T)$
<i>ssmax</i>	$s_{\max}$
<i>ssmin</i>	$s_{\min}$
<i>gTrms</i>	$(\nabla T)_{\text{rms}}$
<i>gsrms</i>	$(\nabla s)_{\text{rms}}$

<i>gTxgsrms</i>	$(\nabla T \times \nabla s)_{\text{rms}}$
<i>gTxgsom</i>	$\langle (\nabla T \times \nabla s) \cdot \boldsymbol{\omega} \rangle$
<i>fconvm</i>	$\langle c_p \varrho u_z T \rangle$
<i>ufpresm</i>	$\langle -u / \rho \nabla p \rangle$
<i>Kkramersm</i>	$\langle K_{\text{kramers}} \rangle$
<i>chikrammin</i>	$\min(\chi_{\text{kramers}})$
<i>chikrammax</i>	$\max(\chi_{\text{kramers}})$
<i>TT2m</i>	$\langle (T)^2 \rangle$ (mean squared temperature)

Module ‘magnetic.f90’

<i>eta_tdep</i>	t -dependent η
<i>ab_int</i>	$\int \mathbf{A} \cdot \mathbf{B} \, dV$
<i>jb_int</i>	$\int \mathbf{j} \cdot \mathbf{B} \, dV$
<i>b2tm</i>	$\left\langle \mathbf{b}(t) \cdot \int_0^t \mathbf{b}(t') dt' \right\rangle$
<i>bjtm</i>	$\left\langle \mathbf{b}(t) \cdot \int_0^t \mathbf{j}(t') dt' \right\rangle$
<i>jbtm</i>	$\left\langle \mathbf{j}(t) \cdot \int_0^t \mathbf{b}(t') dt' \right\rangle$
<i>ujtm</i>	$\left\langle \mathbf{u}(t) \cdot \int_0^t \mathbf{j}(t') dt' \right\rangle$
<i>jutm</i>	$\left\langle \mathbf{j}(t) \cdot \int_0^t \mathbf{u}(t') dt' \right\rangle$
<i>ubtm</i>	$\left\langle \mathbf{u}(t) \cdot \int_0^t \mathbf{b}(t') dt' \right\rangle$
<i>butm</i>	$\left\langle \mathbf{b}(t) \cdot \int_0^t \mathbf{u}(t') dt' \right\rangle$
<i>b2ruz</i>	$\langle \mathbf{B}^2 \rho u_z \rangle$
<i>b2uz</i>	$\langle \mathbf{B}^2 u_z \rangle$
<i>ubbz</i>	$\langle (\mathbf{u} \cdot \mathbf{B}) B_z \rangle$
<i>b1m</i>	$\langle \mathbf{B} \rangle$
<i>b2m</i>	$\langle \mathbf{B}^2 \rangle$
<i>EEM</i>	$\langle \mathbf{B}^2 \rangle / 2$
<i>EEM2</i>	$\langle (\mathbf{B}^2 / 2)^2 \rangle$
<i>EEM3</i>	$\langle (\mathbf{B}^2 / 2)^3 \rangle$
<i>EEM4</i>	$\langle (\mathbf{B}^2 / 2)^4 \rangle$
<i>b4m</i>	$\log_{10} \langle \mathbf{B}^4 \rangle$
<i>b6m</i>	$\log_{10} \langle \mathbf{B}^6 \rangle$
<i>b8m</i>	$\log_{10} \langle \mathbf{B}^8 \rangle$
<i>b12m</i>	$\log_{10} \langle \mathbf{B}^{12} \rangle$
<i>logbm</i>	$\langle \log B \rangle$
<i>bm2</i>	$\max(\mathbf{B}^2)$
<i>j2m</i>	$\langle \mathbf{j}^2 \rangle$
<i>jm2</i>	$\max(\mathbf{j}^2)$
<i>abm</i>	$\langle \mathbf{A} \cdot \mathbf{B} \rangle$
<i>gLamam</i>	$\langle \nabla \Lambda \cdot \mathbf{A} \rangle$
<i>gLambm</i>	$\langle \nabla \Lambda \cdot \mathbf{B} \rangle$
<i>abumx</i>	$\langle u_x \mathbf{A} \cdot \mathbf{B} \rangle$
<i>abumy</i>	$\langle u_y \mathbf{A} \cdot \mathbf{B} \rangle$
<i>abumz</i>	$\langle u_z \mathbf{A} \cdot \mathbf{B} \rangle$
<i>abmh</i>	$\langle \mathbf{A} \cdot \mathbf{B} \rangle$ (temp)
<i>abmn</i>	$\langle \mathbf{A} \cdot \mathbf{B} \rangle$ (north)
<i>abms</i>	$\langle \mathbf{A} \cdot \mathbf{B} \rangle$ (south)
<i>abrms</i>	$\langle (\mathbf{A} \cdot \mathbf{B})^2 \rangle^{1/2}$

<i>jbrms</i>	$\langle (\mathbf{j} \cdot \mathbf{B})^2 \rangle^{1/2}$
<i>jxbrms</i>	$\langle (\mathbf{j} \times \mathbf{B})^2 \rangle^{1/2}$
<i>ajm</i>	$\langle \mathbf{j} \cdot \mathbf{A} \rangle$
<i>jbm</i>	$\langle \mathbf{j} \cdot \mathbf{B} \rangle$
<i>a2b2m</i>	$\langle \mathbf{A}^2 \cdot \mathbf{B}^2 \rangle$
<i>j2b2m</i>	$\langle \mathbf{j}^2 \cdot \mathbf{B}^2 \rangle$
<i>jbmh</i>	$\langle \mathbf{J} \cdot \mathbf{B} \rangle$ (temp)
<i>jbmh</i>	$\langle \mathbf{J} \cdot \mathbf{B} \rangle$ (north)
<i>jbms</i>	$\langle \mathbf{J} \cdot \mathbf{B} \rangle$ (south)
<i>ubm</i>	$\langle \mathbf{u} \cdot \mathbf{B} \rangle$
<i>dubrms</i>	$\langle (\mathbf{u} - \mathbf{B})^2 \rangle^{1/2}$
<i>dobrms</i>	$\langle (\boldsymbol{\omega} - \mathbf{B})^2 \rangle^{1/2}$
<i>uxbxm</i>	$\langle u_x B_x \rangle$
<i>uybxm</i>	$\langle u_y B_x \rangle$
<i>uzbxm</i>	$\langle u_z B_x \rangle$
<i>uxbym</i>	$\langle u_x B_y \rangle$
<i>uybym</i>	$\langle u_y B_y \rangle$
<i>uzbym</i>	$\langle u_z B_y \rangle$
<i>uxbzm</i>	$\langle u_x B_z \rangle$
<i>uybzm</i>	$\langle u_y B_z \rangle$
<i>uzbzm</i>	$\langle u_z B_z \rangle$
<i>uxjxm</i>	$\langle u_x J_x \rangle$
<i>uxjym</i>	$\langle u_x J_y \rangle$
<i>uxjzm</i>	$\langle u_x J_z \rangle$
<i>uyjxm</i>	$\langle u_y J_x \rangle$
<i>uyjym</i>	$\langle u_y J_y \rangle$
<i>uyjzm</i>	$\langle u_y J_z \rangle$
<i>uzjxm</i>	$\langle u_z J_x \rangle$
<i>uzjym</i>	$\langle u_z J_y \rangle$
<i>uzjzm</i>	$\langle u_z J_z \rangle$
<i>cosubm</i>	$\langle \mathbf{U} \cdot \mathbf{B} / (\mathbf{U} \mathbf{B}) \rangle$
<i>jxbxm</i>	$\langle j_x B_x \rangle$
<i>jybxm</i>	$\langle j_y B_x \rangle$
<i>jzbxm</i>	$\langle j_z B_x \rangle$
<i>jxbym</i>	$\langle j_x B_y \rangle$
<i>jybym</i>	$\langle j_y B_y \rangle$
<i>jzbym</i>	$\langle j_z B_y \rangle$
<i>jxbzm</i>	$\langle j_x B_z \rangle$
<i>jybzm</i>	$\langle j_y B_z \rangle$
<i>jzbzm</i>	$\langle j_z B_z \rangle$
<i>uam</i>	$\langle \mathbf{u} \cdot \mathbf{A} \rangle$
<i>obm</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{B} \rangle$
<i>ujm</i>	$\langle \mathbf{u} \cdot \mathbf{J} \rangle$
<i>fbm</i>	$\langle \mathbf{f} \cdot \mathbf{B} \rangle$
<i>fxbxm</i>	$\langle f_x B_x \rangle$
<i>epsM</i>	$\langle \eta \mu_0 \mathbf{j}^2 \rangle$
<i>epsM2</i>	$\langle (\eta \mu_0 \mathbf{j}^2)^2 \rangle$
<i>epsM3</i>	$\langle (\eta \mu_0 \mathbf{j}^2)^3 \rangle$
<i>epsM4</i>	$\langle (\eta \mu_0 \mathbf{j}^2)^4 \rangle$
<i>epsAD</i>	$\langle \rho^{-1} t_{AD} (\mathbf{J} \times \mathbf{B})^2 \rangle$ (heating by ion-neutrals friction)

<i>bxpt</i>	$B_x(x_1, y_1, z_1, t)$
<i>bypt</i>	$B_y(x_1, y_1, z_1, t)$
<i>bzpt</i>	$B_z(x_1, y_1, z_1, t)$
<i>bxbypt</i>	$(B_x B_y)(x_1, y_1, z_1, t)$
<i>bybzip</i>	$(B_y B_z)(x_1, y_1, z_1, t)$
<i>bzbixpt</i>	$(B_z B_x)(x_1, y_1, z_1, t)$
<i>jxpt</i>	$J_x(x_1, y_1, z_1, t)$
<i>jypt</i>	$J_y(x_1, y_1, z_1, t)$
<i>jzpt</i>	$J_z(x_1, y_1, z_1, t)$
<i>Expt</i>	$\mathcal{E}_x(x_1, y_1, z_1, t)$
<i>Eypt</i>	$\mathcal{E}_y(x_1, y_1, z_1, t)$
<i>Ezpt</i>	$\mathcal{E}_z(x_1, y_1, z_1, t)$
<i>axpt</i>	$A_x(x_1, y_1, z_1, t)$
<i>aypt</i>	$A_y(x_1, y_1, z_1, t)$
<i>azpt</i>	$A_z(x_1, y_1, z_1, t)$
<i>bxp2</i>	$B_x(x_2, y_2, z_2, t)$
<i>byp2</i>	$B_y(x_2, y_2, z_2, t)$
<i>bzp2</i>	$B_z(x_2, y_2, z_2, t)$
<i>jxp2</i>	$J_x(x_2, y_2, z_2, t)$
<i>jyp2</i>	$J_y(x_2, y_2, z_2, t)$
<i>jzp2</i>	$J_z(x_2, y_2, z_2, t)$
<i>Exp2</i>	$\mathcal{E}_x(x_2, y_2, z_2, t)$
<i>Eyp2</i>	$\mathcal{E}_y(x_2, y_2, z_2, t)$
<i>Ezp2</i>	$\mathcal{E}_z(x_2, y_2, z_2, t)$
<i>axp2</i>	$A_x(x_2, y_2, z_2, t)$
<i>ayp2</i>	$A_y(x_2, y_2, z_2, t)$
<i>azp2</i>	$A_z(x_2, y_2, z_2, t)$
<i>exabot</i>	$\int \mathbf{E} \times \mathbf{A} dS _{\text{bot}}$
<i>exatop</i>	$\int \mathbf{E} \times \mathbf{A} dS _{\text{top}}$
<i>emag</i>	$\int_V \frac{1}{2\mu_0} \mathbf{B}^2 dV$
<i>km0EM</i>	$\int E_M(k) dk$
<i>km1EM</i>	$\int k^{-1} E_M(k) dk$
<i>brms</i>	$\langle \mathbf{B}^2 \rangle^{1/2}$
<i>bfrms</i>	$\langle \mathbf{B}'^2 \rangle^{1/2}$
<i>bf2m</i>	$\langle \mathbf{B}'^2 \rangle$
<i>bf4m</i>	$\langle \mathbf{B}'^4 \rangle$
<i>bmax</i>	$\max(\mathbf{B})$
<i>bxmin</i>	$\min(B_x)$
<i>bymin</i>	$\min(B_y)$
<i>bzmin</i>	$\min(B_z)$
<i>bxmax</i>	$\max(B_x)$
<i>bymax</i>	$\max(B_y)$
<i>bzmax</i>	$\max(B_z)$
<i>bbxmax</i>	$\max(B_x) \text{ excluding } Bv_{\text{ext}}$
<i>bbymax</i>	$\max(B_y) \text{ excluding } Bv_{\text{ext}}$
<i>bbzmax</i>	$\max(B_z) \text{ excluding } Bv_{\text{ext}}$
<i>jxmax</i>	$\max(jv_x)$
<i>jymax</i>	$\max(jv_y)$

<i>jzmax</i>	$\max(jv_z)$
<i>jrms</i>	$\langle j^2 \rangle^{1/2}$
<i>hjrms</i>	$\langle j^2 \rangle^{1/2}$
<i>jmax</i>	$\max(j)$
<i>vA23rms</i>	$\langle B^2/\varrho^{4/3} \rangle^{1/2}$
<i>vArms</i>	$\langle B^2/\varrho \rangle^{1/2}$
<i>vAmax</i>	$\max(B^2/\varrho)^{1/2}$
<i>dtb</i>	$\delta t/[c_{\delta t} \delta x/v_{A,\max}]$ (time step relative to Alfvén time step; see § 5.16)
<i>dteta</i>	$\delta t/[c_{\delta t,v} \delta x^2/\eta_{\max}]$ (time step relative to resistive time step; see § 5.16)
<i>dteta3</i>	$\delta t/[c_{\delta t,v3} \delta x^6/\eta_{\max}^{\text{hyper}}]$ (time step relative to hyper resistive time step; see § 5.16)
<i>a2m</i>	$\langle A^2 \rangle$
<i>arms</i>	$\langle A^2 \rangle^{1/2}$
<i>amax</i>	$\max(A)$
<i>divarms</i>	$\langle (\nabla \cdot A)^2 \rangle^{1/2}$
<i>beta1m</i>	$\langle B^2/(2\mu_0 p) \rangle$ (mean inverse plasma beta)
<i>beta1max</i>	$\max[B^2/(2\mu_0 p)]$ (maximum inverse plasma beta)
<i>betam</i>	$\langle \beta \rangle$
<i>betamax</i>	$\max \beta$
<i>betamin</i>	$\min \beta$
<i>Azmid_min</i>	$\min A_z^{\text{mid}}$
<i>Azmid_max</i>	$\max A_z^{\text{mid}}$
<i>bxm</i>	$\langle B_x \rangle$
<i>bym</i>	$\langle B_y \rangle$
<i>bzm</i>	$\langle B_z \rangle$
<i>jxm</i>	$\langle J_x \rangle$
<i>jym</i>	$\langle J_y \rangle$
<i>jzm</i>	$\langle J_z \rangle$
<i>bxbym</i>	$\langle B_x B_y \rangle$
<i>bxbzm</i>	$\langle B_x B_z \rangle$
<i>bybzm</i>	$\langle B_y B_z \rangle$
<i>bij_cov_diffmax</i>	difference between two implementations of covariant derivatives
<i>bmz</i>	$\langle \langle B \rangle_{yz}^2 \rangle^{1/2}$ (energy of <i>yz</i> -averaged mean field)
<i>bmy</i>	$\langle \langle B \rangle_{xz}^2 \rangle^{1/2}$ (energy of <i>xz</i> -averaged mean field)
<i>bmz</i>	$\langle \langle B \rangle_{xy}^2 \rangle^{1/2}$ (energy of <i>xy</i> -averaged mean field)
<i>bmzS2</i>	$\langle \langle B_S \rangle_{xy}^2 \rangle$
<i>bmzA2</i>	$\langle \langle B_A \rangle_{xy}^2 \rangle$
<i>jmx</i>	$\langle \langle J \rangle_{yz}^2 \rangle^{1/2}$ (energy of <i>yz</i> -averaged mean current density)
<i>jmy</i>	$\langle \langle J \rangle_{xz}^2 \rangle^{1/2}$ (energy of <i>xz</i> -averaged mean current density)
<i>jmx</i>	$\langle \langle J \rangle_{xy}^2 \rangle^{1/2}$ (energy of <i>xy</i> -averaged mean current density)
<i>bmzph</i>	Phase of a Beltrami field
<i>bmzphe</i>	Error of phase of a Beltrami field

<i>bsinphz</i>	sine of phase of a Beltrami field
<i>bcosphz</i>	cosine of phase of a Beltrami field
<i>emxamz3</i>	$\langle \langle \mathbf{E} \rangle_{xy} \times \langle \mathbf{A} \rangle_{xy} \rangle$ (<i>xy</i> -averaged mean field helicity flux)
<i>embmz</i>	$\langle \langle \mathbf{E} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle$ (<i>xy</i> -averaged mean field helicity production)
<i>ambmz</i>	$\langle \langle \mathbf{A} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle$ (magnetic helicity of <i>xy</i> -averaged mean field)
<i>ambmzh</i>	$\langle \langle \mathbf{A} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle$ (magnetic helicity of <i>xy</i> -averaged mean field, temp)
<i>ambmzn</i>	$\langle \langle \mathbf{A} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle$ (magnetic helicity of <i>xy</i> -averaged mean field, north)
<i>ambmzs</i>	$\langle \langle \mathbf{A} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle$ (magnetic helicity of <i>xy</i> -averaged mean field, south)
<i>jmbmz</i>	$\langle \langle \mathbf{J} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle$ (current helicity of <i>xy</i> -averaged mean field)
<i>Rmmz</i>	$\langle \frac{ \mathbf{u} \times \mathbf{B} }{ \eta \mathbf{J} } \rangle_{xy}$
<i>kx_aa</i>	k_x
<i>kmz</i>	$\langle \langle \mathbf{J} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle / \langle \langle \mathbf{B} \rangle_{xy}^2 \rangle$
<i>bx2m</i>	$\langle B_x^2 \rangle$
<i>by2m</i>	$\langle B_y^2 \rangle$
<i>bz2m</i>	$\langle B_z^2 \rangle$
<i>bx3m</i>	$\langle B_x^3 \rangle$
<i>by3m</i>	$\langle B_y^3 \rangle$
<i>bz3m</i>	$\langle B_z^3 \rangle$
<i>bx4m</i>	$\langle B_x^4 \rangle$
<i>by4m</i>	$\langle B_y^4 \rangle$
<i>bz4m</i>	$\langle B_z^4 \rangle$
<i>jx2m</i>	$\langle J_x^2 \rangle$
<i>jy2m</i>	$\langle J_y^2 \rangle$
<i>jz2m</i>	$\langle J_z^2 \rangle$
<i>jx4m</i>	$\langle J_x^4 \rangle$
<i>jy4m</i>	$\langle J_y^4 \rangle$
<i>jz4m</i>	$\langle J_z^4 \rangle$
<i>jh2m1</i>	$\langle J_\perp^2 \rangle^I$
<i>jx2m1</i>	$\langle J_x^2 \rangle^I$
<i>jy2m1</i>	$\langle J_y^2 \rangle^I$
<i>jx2m2</i>	$\langle J_x^2 \rangle^{II}$
<i>jy2m2</i>	$\langle J_y^2 \rangle^{II}$
<i>jx2m3</i>	$\langle J_x^2 \rangle^{III}$
<i>jy2m3</i>	$\langle J_y^2 \rangle^{III}$
<i>uxbm</i>	$\langle \mathbf{u} \times \mathbf{B} \rangle \cdot \mathbf{B}_0 / B_0^2$
<i>jxbm</i>	$\langle \mathbf{j} \times \mathbf{B} \rangle \cdot \mathbf{B}_0 / B_0^2$
<i>vmagfricmax</i>	$\max(1/\nu_{\text{mag}} \mathbf{j} \times \mathbf{B} / B^2)$
<i>vmagfricrms</i>	$\langle 1/\nu_{\text{mag}} \mathbf{j} \times \mathbf{B} / B^2 ^2 \rangle^{1/2}$
<i>b3b21m</i>	$\langle B_3 B_{2,1} \rangle$

<i>b3b12m</i>	$\langle B_3 B_{1,2} \rangle$
<i>b1b32m</i>	$\langle B_1 B_{3,2} \rangle$
<i>b1b23m</i>	$\langle B_1 B_{2,3} \rangle$
<i>b2b13m</i>	$\langle B_2 B_{1,3} \rangle$
<i>b2b31m</i>	$\langle B_2 B_{3,1} \rangle$
<i>uxbm_x</i>	$\langle (\mathbf{u} \times \mathbf{B})_x \rangle$
<i>uxbm_y</i>	$\langle (\mathbf{u} \times \mathbf{B})_y \rangle$
<i>uxbm_z</i>	$\langle (\mathbf{u} \times \mathbf{B})_z \rangle$
<i>jxbm_x</i>	$\langle (\mathbf{j} \times \mathbf{B})_x \rangle$
<i>jxbm_y</i>	$\langle (\mathbf{j} \times \mathbf{B})_y \rangle$
<i>jxbm_z</i>	$\langle (\mathbf{j} \times \mathbf{B})_z \rangle$
<i>exam_x</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_x$
<i>exam_y</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_y$
<i>exam_z</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_z$
<i>exatotalm_x</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_x$
<i>exatotalm_y</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_y$
<i>exatotalm_z</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_z$
<i>exjm_x</i>	$\langle \mathbf{E} \times \mathbf{J} \rangle_x$
<i>exjm_y</i>	$\langle \mathbf{E} \times \mathbf{J} \rangle_y$
<i>exjm_z</i>	$\langle \mathbf{E} \times \mathbf{J} \rangle_z$
<i>dexbm_x</i>	$\langle \nabla \times \mathbf{E} \times \mathbf{B} \rangle_x$
<i>dexbm_y</i>	$\langle \nabla \times \mathbf{E} \times \mathbf{B} \rangle_y$
<i>dexbm_z</i>	$\langle \nabla \times \mathbf{E} \times \mathbf{B} \rangle_z$
<i>phibm_x</i>	$\langle \phi \mathbf{B} \rangle_x$
<i>phibm_y</i>	$\langle \phi \mathbf{B} \rangle_y$
<i>phibm_z</i>	$\langle \phi \mathbf{B} \rangle_z$
<i>b2divum</i>	$\langle \mathbf{B}^2 \nabla \cdot \mathbf{u} \rangle$
<i>jdel2am</i>	$\langle \mathbf{J} \cdot \nabla^2 \mathbf{A} \rangle$
<i>jem</i>	$\langle \mathbf{j} \cdot \mathbf{E} \rangle$
<i>aem</i>	$\langle \mathbf{A} \cdot \mathbf{E} \rangle$
<i>ujxbm</i>	$\langle \mathbf{u} \cdot (\mathbf{J} \times \mathbf{B}) \rangle$
<i>WL2D</i>	$\langle J_i u_j A_{i,j} \rangle$
<i>WL3D</i>	$-\langle J_i u_j A_{j,i} \rangle$
<i>WL3D2</i>	$\langle J_i A_j u_{j,i} \rangle$
<i>gb2m</i>	$\langle \hat{B}_{i,j} ^2 \rangle$
<i>bij2m</i>	$\langle \hat{B}_{i,j} ^2 \rangle$
<i>sijbibjm</i>	$\langle S_{i,j} B_i B_j \rangle$
<i>ubgbpm</i>	$\langle \mathbf{u} \cdot (\mathbf{B} \cdot \nabla \mathbf{B}) \rangle$
<i>ugb22m</i>	$\langle \mathbf{u} \cdot \nabla \mathbf{B}^2 / 2 \rangle$
<i>jxbrmax</i>	$\max(\mathbf{J} \times \mathbf{B} / \rho)$
<i>jxbr2m</i>	$\langle (\mathbf{J} \times \mathbf{B} / \rho)^2 \rangle$
<i>jxbrqm</i>	$\langle (\mathbf{J} \times \mathbf{B} / \rho) \cdot \mathbf{q} \rangle$
<i>bmxy_rms</i>	$\sqrt{[\langle b_x \rangle_z(x, y)]^2 + [\langle b_y \rangle_z(x, y)]^2 + [\langle b_z \rangle_z(x, y)]^2}$
<i>etasmagm</i>	Mean of Smagorinsky resistivity
<i>etasmagmin</i>	Min of Smagorinsky resistivity
<i>etasmagmax</i>	Max of Smagorinsky resistivity
<i>etavamax</i>	Max of artificial resistivity $\eta \sim v_A$
<i>etajmax</i>	Max of artificial resistivity $\eta \sim J / \sqrt{\rho}$
<i>etaj2max</i>	Max of artificial resistivity $\eta \sim J^2 / \rho$

<i>etajrhomax</i>	Max of artificial resistivity $\eta \sim J/\rho$
<i>etaaniso</i>	η_1
<i>etaanisoBB</i>	η_{BB}
<i>cosjbm</i>	$\langle \mathbf{J} \cdot \mathbf{B} / (\mathbf{J} \mathbf{B}) \rangle$
<i>jparallelm</i>	Mean value of the component of $\mathbf{J}$ parallel to $\mathbf{B}$
<i>jperpm</i>	Mean value of the component of $\mathbf{J}$ perpendicular to $\mathbf{B}$
<i>hjparallelm</i>	Mean value of the component of J_{hyper} parallel to $\mathbf{B}$
<i>hperpm</i>	Mean value of the component of J_{hyper} perpendicular to $\mathbf{B}$
<i>b2sphm</i>	$\int_{r=0}^{r=r_{\text{diag}}} \mathbf{B}^2 dV$, where $r = \sqrt{x^2 + y^2 + z^2}$
<i>brmsx</i>	$\langle \mathbf{B}^2 \rangle^{1/2}$ for the magnetic_xaver_range
<i>brmsz</i>	$\langle \mathbf{B}^2 \rangle^{1/2}$ for the magnetic_zaver_range
<i>Exmxy</i>	$\langle \mathcal{E}_x \rangle_z$
<i>Eymxy</i>	$\langle \mathcal{E}_y \rangle_z$
<i>Ezmxy</i>	$\langle \mathcal{E}_z \rangle_z$

Module 'pscalar.f90'

<i>rhoccm</i>	$\langle \rho c \rangle$
<i>ccmax</i>	$\max(c)$
<i>ccglrm</i>	$\langle c \nabla_z \rho \rangle$

Module '1D_loop.f90'

<i>dtchi2</i>	heatconduction
<i>dtrad</i>	radiative loss from RTV
<i>dtspitzer</i>	Spitzer heat conduction time step
<i>qmax</i>	max of heat flux vector
<i>qrms</i>	rms of heat flux vector

Module '3level-atom.f90'

<i>muSm</i>	$\langle \mu_S \rangle$
<i>muSrms</i>	$\langle \mu_S^2 \rangle^{1/2}$
<i>muSmax</i>	$\max \mu$
<i>mu5m</i>	$\langle \mu_5 \rangle$
<i>mu51m</i>	$\langle \mu_5 \rangle$
<i>mu53m</i>	$\langle \mu_5^3 \rangle$
<i>mu54m</i>	$\langle \mu_5^4 \rangle$
<i>mu5rms</i>	$\langle \mu_5^2 \rangle^{1/2}$
<i>mu5min</i>	$\min \mu_5$
<i>mu5max</i>	$\max \mu_5$
<i>mu5abs</i>	$\max \mu_5 $
<i>srce5m</i>	$\langle S_5 \rangle$
<i>gamf5m</i>	$\langle \Gamma_5 \rangle$
<i>gmu5rms</i>	$\langle (\nabla \mu_5)^2 \rangle^{1/2}$
<i>gmuSrms</i>	$\langle (\nabla \mu_S)^2 \rangle^{1/2}$
<i>gmu5mx</i>	$\langle \nabla \mu_5 \rangle_x$
<i>gmu5my</i>	$\langle \nabla \mu_5 \rangle_y$
<i>gmu5mz</i>	$\langle \nabla \mu_5 \rangle_z$
<i>bgmu5rms</i>	$\langle (\mathbf{B} \cdot \nabla \mu_5)^2 \rangle^{1/2}$
<i>bgmuSrms</i>	$\langle (\mathbf{B} \cdot \nabla \mu_S)^2 \rangle^{1/2}$
<i>mu5bjm</i>	$\langle \mu_5 ((\nabla \times \mathbf{B}) \cdot \mathbf{B}) \rangle$

<i>mu5bjrms</i>	$\langle (\mu_5((\nabla \times \mathbf{B}) \cdot \mathbf{B}))^2 \rangle^{1/2}$
<i>dt_lambda5</i>	$\min(\mu_5/B^2)\delta x/(\lambda\eta)$
<i>dt_D5</i>	$(\lambda\eta\max(\mathbf{B}^2))^{-1}$
<i>dt_gammaf5</i>	$1/\Gamma_f$
<i>dt_CMW</i>	$\delta x/((C_\mu C_5)^{1/2}\max(\mathbf{B}))$
<i>dt_Dmu</i>	$(\lambda\eta\min(\mathbf{B}^2))^{-1}$
<i>dt_vmu</i>	$\delta x/(\eta\max(\mu_5))$
<i>dt_chiral</i>	total time-step contribution from chiral MHD
<i>mu5bxm</i>	$\langle \mu_5 B_x \rangle$
<i>mu5b2m</i>	$\langle \mu_5 B^2 \rangle$
<i>mu5jbm</i>	$\langle \mu_5 \mathbf{J} \cdot \mathbf{B} \rangle$
<i>jxm</i>	$\langle J_x \rangle$
<i>Dmu5_tdep</i>	$D(t)$

Module ‘Lambda_CDM.f90’

<i>redshift</i>	redshift z
<i>Hubble</i>	$H(a)$
<i>ascale</i>	a
<i>lna</i>	$\ln a$
<i>tph</i>	t_{phys}

Module ‘advective_gauge.f90’

<i>LamRAm</i>	$\langle \Lambda_{r \rightarrow a} \rangle$
<i>LamRApt</i>	$\Lambda_{r \rightarrow a}(x1, y1, z1)$
<i>LamRAp2</i>	$\Lambda_{r \rightarrow a}(x2, y2, z2)$
<i>LamRArms</i>	$\langle \Lambda_{r \rightarrow a}^2 \rangle^{1/2}$
<i>LamRAbzm</i>	$\langle \Lambda_{r \rightarrow a} B_z \rangle$
<i>LamRAbzmz</i>	$\langle \Lambda_{r \rightarrow a} B_z \rangle_{xy}$
<i>gLamRAbm</i>	$\langle \Lambda_{r \rightarrow a} \mathbf{B} \rangle$
<i>apbrms</i>	$\langle (\mathbf{A}' \mathbf{B})^2 \rangle^{1/2}$
<i>jxarms</i>	$\langle (\mathbf{J} \times \mathbf{A})^2 \rangle^{1/2}$
<i>jxaprms</i>	$\langle (\mathbf{J} \times \mathbf{A}')^2 \rangle^{1/2}$
<i>jxgLamRArms</i>	$\langle (\mathbf{J} \times \nabla \Lambda_{r \rightarrow a})^2 \rangle^{1/2}$
<i>gLamRArms</i>	$\langle (\nabla \Lambda_{r \rightarrow a})^2 \rangle^{1/2}$
<i>divabrms</i>	$\langle [(\nabla \cdot \mathbf{A}) \mathbf{B}]^2 \rangle^{1/2}$
<i>divapbrms</i>	$\langle [(\nabla \cdot \mathbf{A}') \mathbf{B}]^2 \rangle^{1/2}$
<i>d2LamRAbrms</i>	$\langle [(\nabla^2 \Lambda_{r \rightarrow a}) \mathbf{B}]^2 \rangle^{1/2}$
<i>d2LamRArms</i>	$\langle [\nabla^2 \Lambda_{r \rightarrow a}]^2 \rangle^{1/2}$
<i>aabm</i>	$\langle \mathbf{A}_{\text{Adv}} \cdot \mathbf{B} \rangle$

Module ‘anelastic.f90’

<i>rhom</i>	$\langle \varrho \rangle$ (mean density)
<i>ugrhom</i>	$\langle \mathbf{u} \cdot \nabla \varrho \rangle$
<i>mass</i>	$\int \varrho dV$
<i>divrhom</i>	$\langle \nabla \cdot (\varrho \mathbf{u}) \rangle$
<i>divrhorms</i>	$ \nabla \cdot (\varrho \mathbf{u}) _{\text{rms}}$
<i>divrhoumax</i>	$ \nabla \cdot (\varrho \mathbf{u}) _{\text{max}}$

Module ‘ascale_collapse.f90’

<i>redshift</i>	redshift z
<i>Hubble</i>	$H(a)$
<i>ascale</i>	a
<i>lna</i>	$\ln a$
<i>tph</i>	t_{phys}

Module 'axionSU2back.f90'

<i>a</i>	a
<i>phi</i>	ϕ
<i>phidot</i>	$\dot{\phi}$
<i>H</i>	$H_{\text{parameter}}$
<i>Q</i>	Q
<i>Qdot</i>	$\dot{Q}$
<i>Qddot</i>	$\ddot{Q}$
<i>chi</i>	χ
<i>chidot</i>	$\dot{\chi}$
<i>chiddot</i>	$\ddot{\chi}$
<i>psi</i>	ψ
<i>psiL</i>	ψ_L
<i>psidot</i>	$\dot{\psi}$
<i>psiddot</i>	$\ddot{\psi}$
<i>TR</i>	T_R
<i>TL</i>	T_L
<i>uR</i>	u_R
<i>uL</i>	u_L
<i>TRdot</i>	$\dot{T}_R$
<i>TRddot</i>	$\ddot{T}_R$
<i>imTR</i>	$\Im T_R$
<i>psi_anal</i>	ψ^{anal}
<i>TR_anal</i>	T_R^{anal}
<i>TReff2m</i>	$ T_R _{\text{eff}}^2$
<i>TReff2km</i>	$k T_R _{\text{eff}}^2$
<i>TRdoteff2m</i>	$ T_R \dot{T}_R _{\text{eff}}$
<i>TRdoteff2km</i>	$k T_R \dot{T}_R _{\text{eff}}$
<i>TRpsim</i>	$\langle T_R^* \psi \rangle$
<i>TRpsikm</i>	$\langle T_R^* \psi(k/a) \rangle$
<i>TRpsidotm</i>	$\langle T_R^* \dot{\psi} \rangle$
<i>TRdotpsim</i>	$\langle T_R^{*'} \psi \rangle$
<i>TLeff2m</i>	$ T_L _{\text{eff}}^2$
<i>TLeff2km</i>	$k T_L _{\text{eff}}^2$
<i>uReff2m</i>	$ u_R _{\text{eff}}^2$
<i>uReff2km</i>	$k u_R _{\text{eff}}^2$
<i>uLeff2m</i>	$ u_L _{\text{eff}}^2$
<i>uLeff2km</i>	$k u_L _{\text{eff}}^2$
<i>TLdoteff2m</i>	$ T_R \dot{T}_R _{\text{eff}}$
<i>TLdoteff2km</i>	$k T_R \dot{T}_R _{\text{eff}}$
<i>dgrant_up</i>	$\mathcal{T}^\chi$
<i>grand2</i>	$\mathcal{T}^Q$ (test)
<i>dgrant</i>	$\dot{\mathcal{T}}^\chi$
<i>rhoT</i>	ρ_T

<i>JJ_R</i>	J_R
<i>JJ_L</i>	J_L
<i>JJ</i>	J
<i>fact</i>	$\Theta(t)$
<i>k0</i>	k_0
<i>dk</i>	dk

Module ‘axionU1back.f90’

<i>a</i>	a
<i>phi</i>	ϕ
<i>phidot</i>	$\dot{\phi}$
<i>H</i>	$Hubble_parameter$
<i>AL</i>	$\dot{\psi}$
<i>ALdot</i>	$\ddot{\psi}$
<i>AR</i>	T_R
<i>ARdot</i>	T_L
<i>imAR</i>	$\Im T_R$
<i>imAL</i>	T_R^{anal}
<i>imARdot</i>	$\Im T_R$
<i>k0</i>	k_0
<i>dk</i>	dk
<i>rhoe</i>	ρ_e
<i>rhob</i>	ρ_B
<i>edotb</i>	$\dot{e}$

Module ‘backreact_infl.f90’

<i>phim</i>	$\langle \phi \rangle$
<i>phi2m</i>	$\langle \phi^2 \rangle$
<i>phirms</i>	$\langle \phi^2 \rangle^{1/2}$
<i>dphim</i>	$\langle \phi' \rangle$
<i>dphi2m</i>	$\langle (\phi')^2 \rangle$
<i>dphirms</i>	$\langle (\phi')^2 \rangle^{1/2}$
<i>dtphi</i>	$dt/cdtphi$
<i>Hscriptm</i>	$\langle -1 * \mathcal{H} \rangle$
<i>ascale</i>	a
<i>lnam</i>	$\langle \ln a \rangle$
<i>ddotam</i>	a''/a
<i>a2rhopm</i>	$a^2(\rho + p)$
<i>a2rhom</i>	$a^2 \rho$
<i>a2rhophim</i>	$a^2 \rho_\phi$
<i>a4rhophim</i>	$a^4 \rho_\phi$
<i>a2rhogphim</i>	$0.5 < grad \phi^2 >$
<i>rho_chi</i>	ρ_χ
<i>rho_rad</i>	ρ_{rad}
<i>sigEma</i>	ρ_χ
<i>sigBma</i>	ρ_χ
<i>count_eb0a</i>	f_{EB0}
<i>heating</i>	θ_{heat}
<i>wstate</i>	w_{state}
<i>wstate_aver</i>	$\langle w_{state} \rangle$

<i>Gamma_phi</i>	$\langle w_{\text{state}} \rangle$
<i>Gam_phi</i>	$\langle w_{\text{state}} \rangle$
<i>dtGphirho</i>	dtGphirho
<i>tph</i>	dtGphirho
Module ‘backreact_infl_before.f90’	
<i>phim</i>	$\langle \phi \rangle$
<i>phi2m</i>	$\langle \phi^2 \rangle$
<i>phirms</i>	$\langle \phi^2 \rangle^{1/2}$
<i>dphim</i>	$\langle \phi' \rangle$
<i>dphi2m</i>	$\langle (\phi')^2 \rangle$
<i>dphirms</i>	$\langle (\phi')^2 \rangle^{1/2}$
<i>Hscriptm</i>	$\langle \neg * \mathcal{H} \rangle$
<i>lnam</i>	$\langle \ln a \rangle$
<i>ddotam</i>	a''/a
<i>a2rhopm</i>	$a^2(\rho + p)$
<i>a2rhom</i>	$a^2\rho$
<i>a2rhophim</i>	$a^2\rho$
<i>a2rhogphim</i>	$0.5 < \text{grad}\phi^2 >$
<i>rho_chi</i>	ρ_χ
<i>sigEma</i>	ρ_χ
<i>sigBma</i>	ρ_χ
<i>count_eb0a</i>	f_{EB0}
Module ‘bfield.f90’	
<i>bmax</i>	$\max B$
<i>bmin</i>	$\min B$
<i>brms</i>	$\langle B^2 \rangle^{1/2}$
<i>bm</i>	$\langle B \rangle$
<i>b2m</i>	$\langle B^2 \rangle$
<i>bxmax</i>	$\max B_x $
<i>bymax</i>	$\max B_y $
<i>bzmax</i>	$\max B_z $
<i>bxm</i>	$\langle B_x \rangle$
<i>bym</i>	$\langle B_y \rangle$
<i>bzm</i>	$\langle B_z \rangle$
<i>bx2m</i>	$\langle B_x^2 \rangle$
<i>by2m</i>	$\langle B_y^2 \rangle$
<i>bz2m</i>	$\langle B_z^2 \rangle$
<i>bxbym</i>	$\langle B_x B_y \rangle$
<i>bxbzm</i>	$\langle B_x B_z \rangle$
<i>bybzm</i>	$\langle B_y B_z \rangle$
<i>dbxmax</i>	$\max B_x - B_{\text{ext},x} $
<i>dbymax</i>	$\max B_y - B_{\text{ext},y} $
<i>dbzmax</i>	$\max B_z - B_{\text{ext},z} $
<i>dbxm</i>	$\langle B_x - B_{\text{ext},x} \rangle$
<i>dbym</i>	$\langle B_y - B_{\text{ext},y} \rangle$
<i>dbzm</i>	$\langle B_z - B_{\text{ext},z} \rangle$
<i>dbx2m</i>	$\langle (B_x - B_{\text{ext},x})^2 \rangle$
<i>dbym</i>	$\langle (B_y - B_{\text{ext},y})^2 \rangle$

<i>dbz2m</i>	$\langle (B_z - B_{\text{ext},z})^2 \rangle$
<i>jmax</i>	$\max J$
<i>jmin</i>	$\min J$
<i>jrms</i>	$\langle J^2 \rangle^{1/2}$
<i>jm</i>	$\langle J \rangle$
<i>j2m</i>	$\langle J^2 \rangle$
<i>jxmax</i>	$\max J_x $
<i>jymax</i>	$\max J_y $
<i>jzmax</i>	$\max J_z $
<i>jxm</i>	$\langle J_x \rangle$
<i>jym</i>	$\langle J_y \rangle$
<i>jzm</i>	$\langle J_z \rangle$
<i>jx2m</i>	$\langle J_x^2 \rangle$
<i>jy2m</i>	$\langle J_y^2 \rangle$
<i>jz2m</i>	$\langle J_z^2 \rangle$
<i>divbmax</i>	$\max \nabla \cdot \mathbf{B} $
<i>divbrms</i>	$\langle (\nabla \cdot \mathbf{B})^2 \rangle^{1/2}$
<i>betamax</i>	$\max \beta$
<i>betamin</i>	$\min \beta$
<i>betam</i>	$\langle \beta \rangle$
<i>vAmax</i>	$\max v_A$
<i>vAmin</i>	$\min v_A$
<i>vAm</i>	$\langle v_A \rangle$

Module ‘chemistry.f90’

<i>dtchem</i>	dt_{chem}
<i>nuclrmin</i>	$\langle r_{\text{min}} \rangle$
<i>nuclrate</i>	$\langle J \rangle$

Module ‘chemistry_simple.f90’

<i>dtchem</i>	dt_{chem}
---------------	--------------------

Module ‘chiral_mhd.f90’

<i>muSm</i>	$\langle \mu_S \rangle$
<i>muSrms</i>	$\langle \mu_S^2 \rangle^{1/2}$
<i>muSmax</i>	$\max \mu$
<i>mu5m</i>	$\langle \mu_5 \rangle$
<i>mu51m</i>	$\langle \mu_5 \rangle$
<i>mu53m</i>	$\langle \mu_5^3 \rangle$
<i>mu54m</i>	$\langle \mu_5^4 \rangle$
<i>mu5rms</i>	$\langle \mu_5^2 \rangle^{1/2}$
<i>mu5min</i>	$\min \mu_5$
<i>mu5max</i>	$\max \mu_5$
<i>mu5abs</i>	$\max \mu_5 $
<i>srce5m</i>	$\langle S_5 \rangle$
<i>gamf5m</i>	$\langle \Gamma_5 \rangle$
<i>gmu5rms</i>	$\langle (\nabla \mu_5)^2 \rangle^{1/2}$
<i>gmuSrms</i>	$\langle (\nabla \mu_S)^2 \rangle^{1/2}$
<i>gmu5mx</i>	$\langle \nabla \mu_5 \rangle_x$
<i>gmu5my</i>	$\langle \nabla \mu_5 \rangle_y$

<i>gmu5mz</i>	$\langle \nabla \mu_5 \rangle_z$
<i>bgmu5rms</i>	$\langle (\mathbf{B} \cdot \nabla \mu_5)^2 \rangle^{1/2}$
<i>bgmuSrms</i>	$\langle (\mathbf{B} \cdot \nabla \mu_S)^2 \rangle^{1/2}$
<i>mu5bjm</i>	$\langle \mu_5 ((\nabla \times \mathbf{B}) \cdot \mathbf{B}) \rangle$
<i>mu5bjrms</i>	$\langle (\mu_5 ((\nabla \times \mathbf{B}) \cdot \mathbf{B}))^2 \rangle^{1/2}$
<i>dt_lambda5</i>	$\min(\mu_5 / \mathbf{B}^2) \delta x / (\lambda \eta)$
<i>dt_D5</i>	$(\lambda \eta \max(\mathbf{B}^2))^{-1}$
<i>dt_gammaf5</i>	$1/\Gamma_f$
<i>dt_CMW</i>	$\delta x / ((C_\mu C_5)^{1/2} \max(\mathbf{B}))$
<i>dt_Dmu</i>	$(\lambda \eta \min(\mathbf{B}^2))^{-1}$
<i>dt_vmu</i>	$\delta x / (\eta \max(\mu_5))$
<i>dt_chiral</i>	total time-step contribution from chiral MHD
<i>dt_Cflow1</i>	$\delta x \eta C_{\text{flow}} \mu \mathbf{U} / \mathbf{B} $
<i>dt_Cflow2</i>	$\lambda_5 \eta C_{\text{flow}} \mu \mathbf{U} \cdot \mathbf{B} / \mu_5$
<i>mu5bxm</i>	$\langle \mu_5 B_x \rangle$
<i>mu5b2m</i>	$\langle \mu_5 B^2 \rangle$
<i>mu5jbm</i>	$\langle \mu_5 \mathbf{J} \cdot \mathbf{B} \rangle$
<i>jxm</i>	$\langle J_x \rangle$
<i>Dmu5_tdep</i>	$D(t)$
Module ‘collapse.f90’	
<i>betm</i>	$\langle \beta \rangle$
<i>massm</i>	$\langle m \rangle$
Module ‘coronae.f90’	
<i>dtchi2</i>	$\delta t / [c_{\delta t, v} \delta x^2 / \chi_{\max}]$ (time step relative to time step based on heat conductivity; see § 5.16)
<i>dtspitzer</i>	Spitzer heat conduction time step
<i>dtrad</i>	radiative loss from RTV
Module ‘cosmicray_current.f90’	
<i>ekincr</i>	$\langle \frac{1}{2} \varrho \mathbf{u}_{\text{cr}}^2 \rangle$
<i>ethmcr</i>	$\langle \varrho_{\text{cr}} e_{\text{cr}} \rangle$
Module ‘density_stratified.f90’	
<i>mass</i>	$\int \rho d^3x$
<i>rhomin</i>	$\min \rho $
<i>rhomax</i>	$\max \rho $
<i>drhom</i>	$\langle \Delta \rho / \rho_0 \rangle$
<i>drho2m</i>	$\langle (\Delta \rho / \rho_0)^2 \rangle$
<i>drhorms</i>	$\langle \Delta \rho / \rho_0 \rangle_{rms}$
<i>drhomax</i>	$\max \Delta \rho / \rho_0 $
Module ‘detonate.f90’	
<i>detn</i>	Number of detonated sites (summed over time steps between adjacent outputs)
<i>dettot</i>	Total energy input (summed over time steps between adjacent outputs)
Module ‘disp_current.f90’	
<i>EEEM</i>	$\langle \mathbf{E}^2 + \mathbf{B}^2 \rangle / 2$

<i>erms</i>	$\langle \mathbf{E}^2 \rangle^{1/2}$
<i>epriimerms</i>	$\langle (E')^2 \rangle^{1/2}$
<i>bprimermms</i>	$\langle (B')^2 \rangle^{1/2}$
<i>jprimermms</i>	$\langle (J')^2 \rangle^{1/2}$
<i>gam_EBrms</i>	$\langle (\gamma')^2 \rangle^{1/2}$
<i>boostprms</i>	$\langle \text{boost}^2 \rangle^{1/2}$
<i>edotrms</i>	$\langle \dot{\mathbf{E}}^2 \rangle^{1/2}$
<i>emax</i>	$\max(\mathbf{E})$
<i>a0rms</i>	$\langle A_0^2 \rangle^{1/2}$
<i>grms</i>	$\langle C - \nabla \cdot \mathbf{A} \rangle^{1/2}$
<i>da0rms</i>	$\langle C - \nabla \cdot \mathbf{A} \rangle^{1/2}$
<i>BcurlEm</i>	$\langle \mathbf{B} \cdot \nabla \times \mathbf{E} \rangle$
<i>BcurlBm</i>	$\langle \mathbf{B} \cdot \nabla \times \mathbf{B} \rangle$
<i>divJrms</i>	$\langle \nabla \mathbf{J}^2 \rangle^{1/2}$
<i>divErms</i>	$\langle \nabla \mathbf{E}^2 \rangle^{1/2}$
<i>rhoerms</i>	$\langle \rho_e^2 \rangle^{1/2}$
<i>divJm</i>	$\langle \nabla \mathbf{J} \rangle$
<i>divEm</i>	$\langle \nabla \mathbf{E} \rangle$
<i>rhoem</i>	$\langle \rho_e \rangle$
<i>count_eb0</i>	f_{EB0}
<i>mfpf</i>	$-f'/f$
<i>fppf</i>	f''/f
<i>afact</i>	a (scale factor)
<i>constrainedeqn</i>	$< \text{deldot}E+ >$
<i>exm</i>	$\langle E_x \rangle$
<i>eym</i>	$\langle E_y \rangle$
<i>ezm</i>	$\langle E_z \rangle$
<i>etaSchw</i>	$1/\langle \sigma_E \rangle$
<i>dteta</i>	$dt/cdtv$
<i>dt sigE</i>	dt/cdt_sigE
<i>sigEm</i>	$\langle \sigma_E \rangle$
<i>sigBm</i>	$\langle \sigma_B \rangle$
<i>sigErms</i>	$\langle \sigma_E^2 \rangle^{1/2}$
<i>sigBrms</i>	$\langle \sigma_B^2 \rangle^{1/2}$
<i>sigEE2m</i>	$\langle \sigma_E \mathbf{E}^2 \rangle$
<i>sigBBEm</i>	$\langle \sigma_E \mathbf{B} \cdot \mathbf{E} \rangle$
<i>BdEdtm</i>	$\langle \mathbf{B} \cdot \partial \mathbf{E} / \partial t \rangle$
<i>adphiBm</i>	$\langle (\alpha/f) < \phi' \mathbf{B} \cdot \mathbf{E} \rangle$
<i>adphiB2m</i>	$\langle (\alpha/f) < \phi' \mathbf{B}^2 / \sigma_E + \nabla \phi' \times \mathbf{E} \rangle$
<i>adphiB21m</i>	$\langle (\alpha/f) < \phi' \mathbf{B}^2 / \sigma_E \rangle$
<i>adphiJBm</i>	$\langle (\alpha/f) < \phi' \mathbf{J} \cdot \mathbf{B} / \sigma_E \rangle$
<i>adphiBrms</i>	$\langle [(\alpha/f) < \phi' \mathbf{B}] \rangle^{1/2}$
<i>Johmrms</i>	$\langle \mathbf{J}^2 \rangle^{1/2}$
<i>J2sigEm</i>	$\langle \mathbf{J}^2 / \sigma_E \rangle$
<i>ujxb1m</i>	$\langle \mathbf{u} \cdot (\mathbf{J} \times \mathbf{B})_E \rangle$
<i>curlBrms</i>	$\langle (\text{curl} \mathbf{B})^2 \rangle^{1/2}$
<i>echarge</i>	$\langle e_{\text{eff}} \rangle$

<i>ebm</i>	$\langle \mathbf{E} \cdot \mathbf{B} \rangle$
Module ‘dustdensity.f90’	
<i>KKm</i>	$\sum \mathcal{T}_k^{\text{coag}}$
<i>KK2m</i>	$\sum \mathcal{T}_k^{\text{coag}}$
<i>MMxm</i>	$\sum \mathcal{M}_{k,\text{coag}}^x$
<i>MMym</i>	$\sum \mathcal{M}_{k,\text{coag}}^y$
<i>MMzm</i>	$\sum \mathcal{M}_{k,\text{coag}}^z$
Module ‘electroweaksu2.f90’	
<i>W1rms</i>	$\langle \mathbf{W}^{12} \rangle^{1/2}$
<i>W2rms</i>	$\langle \mathbf{W}^{22} \rangle^{1/2}$
<i>W3rms</i>	$\langle \mathbf{W}^{32} \rangle^{1/2}$
<i>W1max</i>	$\max(\mathbf{W}^1)$
<i>W2max</i>	$\max(\mathbf{W}^2)$
<i>W3max</i>	$\max(\mathbf{W}^3)$
<i>dW1rms</i>	$\langle \dot{\mathbf{W}}_1^2 \rangle^{1/2}$
<i>dW2rms</i>	$\langle \dot{\mathbf{W}}_2^2 \rangle^{1/2}$
<i>dW3rms</i>	$\langle \dot{\mathbf{W}}_3^2 \rangle^{1/2}$
<i>dW1max</i>	$\max(\dot{\mathbf{W}}_1)$
<i>dW2max</i>	$\max(\dot{\mathbf{W}}_2)$
<i>dW3max</i>	$\max(\dot{\mathbf{W}}_3)$
<i>W1ddotrms</i>	$\langle \ddot{\mathbf{W}}_1^2 \rangle^{1/2}$
<i>W2ddotrms</i>	$\langle \ddot{\mathbf{W}}_2^2 \rangle^{1/2}$
<i>W3ddotrms</i>	$\langle \ddot{\mathbf{W}}_3^2 \rangle^{1/2}$
<i>divW1rms</i>	$\langle \nabla \mathbf{W}_1^2 \rangle^{1/2}$
<i>divW2rms</i>	$\langle \nabla \mathbf{W}_2^2 \rangle^{1/2}$
<i>divW3rms</i>	$\langle \nabla \mathbf{W}_3^2 \rangle^{1/2}$
<i>divW1m</i>	$\langle \nabla \mathbf{W}_1 \rangle$
<i>divW2m</i>	$\langle \nabla \mathbf{W}_2 \rangle$
<i>divW3m</i>	$\langle \nabla \mathbf{W}_3 \rangle$
<i>divdotW1rms</i>	$\langle \nabla \dot{\mathbf{W}}_1^2 \rangle^{1/2}$
<i>divdotW2rms</i>	$\langle \nabla \dot{\mathbf{W}}_2^2 \rangle^{1/2}$
<i>divdotW3rms</i>	$\langle \nabla \dot{\mathbf{W}}_3^2 \rangle^{1/2}$
<i>rhoW1rms</i>	$\langle \rho \mathbf{W}_1^2 \rangle^{1/2}$
<i>rhoW2rms</i>	$\langle \rho \mathbf{W}_2^2 \rangle^{1/2}$
<i>rhoW3rms</i>	$\langle \rho \mathbf{W}_3^2 \rangle^{1/2}$
<i>divdotW1m</i>	$\langle \nabla \dot{\mathbf{W}}_1 \rangle$

<i>divdotW2m</i>	$\langle \nabla \dot{\mathbf{W}}_2 \rangle$
<i>divdotW3m</i>	$\langle \nabla \dot{\mathbf{W}}_3 \rangle$
<i>rhoW1m</i>	$\langle \rho_e \mathbf{W}_1 \rangle$
<i>rhoW2m</i>	$\langle \rho_e \mathbf{W}_2 \rangle$
<i>rhoW3m</i>	$\langle \rho_e \mathbf{W}_3 \rangle$
<i>constrainteqnW</i>	$< deldotW + >$
<i>W1xm</i>	$\langle W_x^1 \rangle$
<i>W1ym</i>	$\langle W_y^1 \rangle$
<i>W1zm</i>	$\langle W_z^1 \rangle$
<i>W2xm</i>	$\langle W_x^2 \rangle$
<i>W2ym</i>	$\langle W_y^2 \rangle$
<i>W2zm</i>	$\langle W_z^2 \rangle$
<i>W3xm</i>	$\langle W_x^3 \rangle$
<i>W3ym</i>	$\langle W_y^3 \rangle$
<i>W3zm</i>	$\langle W_z^3 \rangle$
<i>dW1xm</i>	$\langle \dot{W}_x^1 \rangle$
<i>dW1ym</i>	$\langle \dot{W}_y^1 \rangle$
<i>dW1zm</i>	$\langle \dot{W}_z^1 \rangle$
<i>dW2xm</i>	$\langle \dot{W}_x^2 \rangle$
<i>dW2ym</i>	$\langle \dot{W}_y^2 \rangle$
<i>dW2zm</i>	$\langle \dot{W}_z^2 \rangle$
<i>dW3xm</i>	$\langle \dot{W}_x^3 \rangle$
<i>dW3ym</i>	$\langle \dot{W}_y^3 \rangle$
<i>dW3zm</i>	$\langle \dot{W}_z^3 \rangle$
<i>W1dotW1m</i>	$\langle \dot{\mathbf{W}}_1 \cdot \mathbf{W}_1 \rangle$
<i>W2dotW2m</i>	$\langle \dot{\mathbf{W}}_2 \cdot \mathbf{W}_2 \rangle$
<i>W3dotW3m</i>	$\langle \dot{\mathbf{W}}_3 \cdot \mathbf{W}_3 \rangle$

Module ‘entropy_anelastic.f90’

<i>dtc</i>	$\delta t / [c_{\delta t} \delta x / \max c_s]$ (time step relative to acoustic time step; see § 5.16)
<i>ethm</i>	$\langle \varrho e \rangle$ (mean thermal [=internal] energy)
<i>ssm</i>	$\langle s/c_p \rangle$ (mean entropy)
<i>ss2m</i>	$\langle (s/c_p)^2 \rangle$ (mean squared entropy)
<i>eem</i>	$\langle e \rangle$
<i>ppm</i>	$\langle p \rangle$
<i>csm</i>	$\langle c_s \rangle$
<i>pdivum</i>	$\langle p \nabla \mathbf{u} \rangle$
<i>fradbot</i>	$\int F_{\text{bot}} \cdot d\mathbf{S}$
<i>fradtop</i>	$\int F_{\text{top}} \cdot d\mathbf{S}$
<i>TTtop</i>	$\int T_{\text{top}} d\mathbf{S}$
<i>ethtot</i>	$\int_V \varrho e dV$ (total thermal [=internal] energy)
<i>dtchi</i>	$\delta t / [c_{\delta t, v} \delta x^2 / \chi_{\max}]$ (time step relative to time step based on heat conductivity; see § 5.16)

<i>ssmxy</i>	$\langle s \rangle_z$
<i>ssmxz</i>	$\langle s \rangle_y$
Module ‘forcing.f90’	
<i>bfm</i>	$\langle \mathbf{B} \cdot \mathbf{f} \rangle$
<i>jfm</i>	$\langle \mathbf{J} \cdot \mathbf{f} \rangle$
<i>rufm</i>	$\langle \rho \mathbf{f} \cdot \mathbf{u} \rangle$
<i>ufm</i>	$\langle \mathbf{f} \cdot \mathbf{u} \rangle$
<i>ofm</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{f} \rangle$
Module ‘gravitational_waves.f90’	
<i>hhT2m</i>	$\langle h_T^2 \rangle$
<i>hhX2m</i>	$\langle h_X^2 \rangle$
<i>hhThhXm</i>	$\langle h_T h_X \rangle$
<i>ggTpt</i>	$g_T(x_1, y_1, z_1, t)$
<i>strTpt</i>	$S_T(x_1, y_1, z_1, t)$
<i>strXpt</i>	$S_X(x_1, y_1, z_1, t)$
Module ‘gravitational_waves_hTXk.f90’	
<i>STrept</i>	$ReS_T(k_1, k_1, k_1, t)$
<i>STimpt</i>	$ImS_T(k_1, k_1, k_1, t)$
<i>SXrept</i>	$ReS_X(k_1, k_1, k_1, t)$
<i>SXimpt</i>	$ImS_X(k_1, k_1, k_1, t)$
<i>hTrept</i>	$Reh_T(k_1, k_1, k_1, t)$
<i>hTimpt</i>	$Imh_T(k_1, k_1, k_1, t)$
<i>hXrept</i>	$Reh_X(k_1, k_1, k_1, t)$
<i>hXimpt</i>	$Imh_X(k_1, k_1, k_1, t)$
<i>gTrept</i>	$Reh_T(k_1, k_1, k_1, t)$
<i>gTimpt</i>	$Imh_T(k_1, k_1, k_1, t)$
<i>gXrept</i>	$Reh_X(k_1, k_1, k_1, t)$
<i>gXimpt</i>	$Imh_X(k_1, k_1, k_1, t)$
<i>STrep2</i>	$ReS_T(k_2, k_2, k_2, t)$
<i>STimp2</i>	$ImS_T(k_2, k_2, k_2, t)$
<i>SXrep2</i>	$ReS_X(k_2, k_2, k_2, t)$
<i>SXimp2</i>	$ImS_X(k_2, k_2, k_2, t)$
<i>hTrep2</i>	$Reh_T(k_2, k_2, k_2, t)$
<i>hTimp2</i>	$Imh_T(k_2, k_2, k_2, t)$
<i>hXrep2</i>	$Reh_X(k_2, k_2, k_2, t)$
<i>hXimp2</i>	$Imh_X(k_2, k_2, k_2, t)$
<i>gTrep2</i>	$Reg_T(k_2, k_2, k_2, t)$
<i>gTimp2</i>	$Img_T(k_2, k_2, k_2, t)$
<i>gXrep2</i>	$Reg_X(k_2, k_2, k_2, t)$
<i>gXimp2</i>	$Img_X(k_2, k_2, k_2, t)$
<i>g11pt</i>	$g_{11}(x_1, y_1, z_1, t)$
<i>g22pt</i>	$g_{22}(x_1, y_1, z_1, t)$
<i>g33pt</i>	$g_{33}(x_1, y_1, z_1, t)$
<i>g12pt</i>	$g_{12}(x_1, y_1, z_1, t)$
<i>g23pt</i>	$g_{23}(x_1, y_1, z_1, t)$
<i>g31pt</i>	$g_{31}(x_1, y_1, z_1, t)$
<i>hhTpt</i>	$h_T(x_1, y_1, z_1, t)$
<i>hhXpt</i>	$h_X(x_1, y_1, z_1, t)$

<i>ggTpt</i>	$\dot{h}_T(x_1, y_1, z_1, t)$
<i>ggXpt</i>	$\dot{h}_X(x_1, y_1, z_1, t)$
<i>hhTp2</i>	$h_T(x_1, y_1, z_1, t)$
<i>hhXp2</i>	$h_X(x_1, y_1, z_1, t)$
<i>ggTp2</i>	$\dot{h}_T(x_1, y_1, z_1, t)$
<i>ggXp2</i>	$\dot{h}_X(x_1, y_1, z_1, t)$
<i>hrms</i>	$\langle h_T^2 + h_X^2 \rangle^{1/2}$
<i>EEGW</i>	$\langle g_T^2 + g_X^2 \rangle c^2 / (32\pi G)$
<i>gg2m</i>	$\langle g_T^2 + g_X^2 \rangle$
<i>Stgm</i>	$\langle S_T g_T + S_X g_X \rangle$
<i>hhT2m</i>	$\langle h_T^2 \rangle$
<i>hhX2m</i>	$\langle h_X^2 \rangle$
<i>hhTXm</i>	$\langle h_T h_X \rangle$
<i>ggT2m</i>	$\langle g_T^2 \rangle$
<i>ggX2m</i>	$\langle g_X^2 \rangle$
<i>ggTXm</i>	$\langle g_T g_X \rangle$
<i>nlin0</i>	$\langle nlin0 \rangle$
<i>nlin1</i>	$\langle nlin1 \rangle$
<i>nlin2</i>	$\langle nlin2 \rangle$
<i>h11rms</i>	$\langle h_{11}^2 \rangle^{1/2}$
<i>h22rms</i>	$\langle h_{22}^2 \rangle^{1/2}$
<i>h33rms</i>	$\langle h_{33}^2 \rangle^{1/2}$
<i>h12rms</i>	$\langle h_{12}^2 \rangle^{1/2}$
<i>h23rms</i>	$\langle h_{23}^2 \rangle^{1/2}$
<i>h31rms</i>	$\langle h_{31}^2 \rangle^{1/2}$

Module ‘gravitational_waves_hTXk_no_xpara.f90’

<i>g11pt</i>	$g_{11}(x_1, y_1, z_1, t)$
<i>g22pt</i>	$g_{22}(x_1, y_1, z_1, t)$
<i>g33pt</i>	$g_{33}(x_1, y_1, z_1, t)$
<i>g12pt</i>	$g_{12}(x_1, y_1, z_1, t)$
<i>g23pt</i>	$g_{23}(x_1, y_1, z_1, t)$
<i>g31pt</i>	$g_{31}(x_1, y_1, z_1, t)$
<i>hhTpt</i>	$h_T(x_1, y_1, z_1, t)$
<i>hhXpt</i>	$h_X(x_1, y_1, z_1, t)$
<i>ggTpt</i>	$\dot{h}_T(x_1, y_1, z_1, t)$
<i>ggXpt</i>	$\dot{h}_X(x_1, y_1, z_1, t)$
<i>hhTp2</i>	$h_T(x_1, y_1, z_1, t)$
<i>hhXp2</i>	$h_X(x_1, y_1, z_1, t)$
<i>ggTp2</i>	$\dot{h}_T(x_1, y_1, z_1, t)$
<i>ggXp2</i>	$\dot{h}_X(x_1, y_1, z_1, t)$
<i>hrms</i>	$\langle h_T^2 + h_X^2 \rangle^{1/2}$
<i>EEGW</i>	$\langle g_T^2 + g_X^2 \rangle c^2 / (32\pi G)$
<i>gg2m</i>	$\langle g_T^2 + g_X^2 \rangle$
<i>hhT2m</i>	$\langle h_T^2 \rangle$
<i>hhX2m</i>	$\langle h_X^2 \rangle$
<i>hhTXm</i>	$\langle h_T h_X \rangle$
<i>ggT2m</i>	$\langle g_T^2 \rangle$
<i>ggX2m</i>	$\langle g_X^2 \rangle$

<i>ggTXm</i>	$\langle g_T g_X \rangle$
Module 'gravitational_waves_hij6.f90'	
<i>h22rms</i>	h_{22}^{rms}
<i>h33rms</i>	h_{33}^{rms}
<i>h23rms</i>	h_{23}^{rms}
<i>g11pt</i>	$g_{11}(x_1, y_1, z_1, t)$
<i>g22pt</i>	$g_{22}(x_1, y_1, z_1, t)$
<i>g33pt</i>	$g_{33}(x_1, y_1, z_1, t)$
<i>g12pt</i>	$g_{12}(x_1, y_1, z_1, t)$
<i>g23pt</i>	$g_{23}(x_1, y_1, z_1, t)$
<i>g31pt</i>	$g_{31}(x_1, y_1, z_1, t)$
<i>hhTpt</i>	$h_T(x_1, y_1, z_1, t)$
<i>hhXpt</i>	$h_X(x_1, y_1, z_1, t)$
<i>ggTpt</i>	$\dot{h}_T(x_1, y_1, z_1, t)$
<i>ggXpt</i>	$\dot{h}_X(x_1, y_1, z_1, t)$
<i>hhTp2</i>	$h_T(x_1, y_1, z_1, t)$
<i>hhXp2</i>	$h_X(x_1, y_1, z_1, t)$
<i>ggTp2</i>	$\dot{h}_T(x_1, y_1, z_1, t)$
<i>ggXp2</i>	$\dot{h}_X(x_1, y_1, z_1, t)$
<i>hrms</i>	$\langle h_T^2 + h_X^2 \rangle^{1/2}$
<i>EEGW</i>	$\langle g_T^2 + g_X^2 \rangle c^2 / (32\pi G)$
<i>gg2m</i>	$\langle g_T^2 + g_X^2 \rangle$
<i>hhT2m</i>	$\langle h_T^2 \rangle$
<i>hhX2m</i>	$\langle h_X^2 \rangle$
<i>hhTXm</i>	$\langle h_T h_X \rangle$
<i>ggT2m</i>	$\langle g_T^2 \rangle$
<i>ggX2m</i>	$\langle g_X^2 \rangle$
<i>ggTXm</i>	$\langle g_T g_X \rangle$
<i>ggTm</i>	$\langle g_T \rangle$
<i>ggXm</i>	$\langle g_X \rangle$
<i>hijij2m</i>	$\langle h_{ij,ij}^2 \rangle$
<i>gijij2m</i>	$\langle g_{ij,ij}^2 \rangle$
Module 'gravity_simple.f90'	
<i>epot</i>	$\langle \varrho \Phi_{\text{grav}} \rangle$ (mean potential energy)
<i>epottot</i>	$\int_V \varrho \Phi_{\text{grav}} dV$ (total potential energy)
<i>ugm</i>	$\langle \mathbf{u} \cdot \mathbf{g} \rangle$
<i>rugm</i>	$\langle \varrho \mathbf{u} \cdot \mathbf{g} \rangle$
<i>rgxm</i>	$\langle \varrho g_x \rangle$
<i>Wgrav</i>	$\int \varrho \mathbf{u} \cdot \mathbf{g} dV$
<i>Fgravx</i>	$\int \varrho g_x dV$
Module 'heatflux.f90'	
<i>dtspitzer</i>	Spitzer heat conduction time step
<i>dtq</i>	heatflux time step
<i>dtq2</i>	heatflux time step due to tau
<i>qmax</i>	$\max(\mathbf{q})$
<i>tauqmax</i>	$\max(\tau_{\text{Spitzer}})$
<i>qxmin</i>	$\min(q_x)$

<i>qymin</i>	$\min(q_y)$
<i>qzmin</i>	$\min(q_z)$
<i>qxmax</i>	$\max(q_x)$
<i>qymax</i>	$\max(q_y)$
<i>qzmax</i>	$\max(q_z)$
<i>qrms</i>	$\sqrt{ \mathbf{q} ^2}$
<i>qsatmin</i>	minimum of qsat/qabs
<i>qsatrms</i>	rms of qsat/abs
Module ‘hydro_kinematic.f90’	
<i>ourms</i>	$\langle (\boldsymbol{\omega} \cdot \mathbf{u})^2 \rangle^{1/2}$
<i>oxurms</i>	$\langle (\boldsymbol{\omega} \times \mathbf{u})^2 \rangle^{1/2}$
<i>EEK</i>	$\langle \varrho \mathbf{u}^2 \rangle / 2$
Module ‘hydro_potential.f90’	
<i>u2tm</i>	$\left\langle \mathbf{u}(t) \cdot \int_0^t \mathbf{u}(t') dt' \right\rangle$
<i>fkinzm</i>	$\left\langle \frac{1}{2} \varrho \mathbf{u}^2 u_z \right\rangle$
<i>u2m</i>	$\langle \mathbf{u}^2 \rangle$
<i>uxpt</i>	$u_x(x_1, y_1, z_1, t)$
<i>uypt</i>	$u_y(x_1, y_1, z_1, t)$
<i>uzpt</i>	$u_z(x_1, y_1, z_1, t)$
<i>uxp2</i>	$u_x(x_2, y_2, z_2, t)$
<i>uyp2</i>	$u_y(x_2, y_2, z_2, t)$
<i>uzp2</i>	$u_z(x_2, y_2, z_2, t)$
<i>urms</i>	$\langle \mathbf{u}^2 \rangle^{1/2}$
<i>urmsx</i>	$\langle \mathbf{u}^2 \rangle^{1/2}$ for the hydro_xaver_range
<i>urmsz</i>	$\langle \mathbf{u}^2 \rangle^{1/2}$ for the hydro_zaver_range
<i>durms</i>	$\langle \delta \mathbf{u}^2 \rangle^{1/2}$
<i>umax</i>	$\max(\mathbf{u})$
<i>umin</i>	$\min(\mathbf{u})$
<i>uxrms</i>	$\langle u_x^2 \rangle^{1/2}$
<i>uyrms</i>	$\langle u_y^2 \rangle^{1/2}$
<i>uzrms</i>	$\langle u_z^2 \rangle^{1/2}$
<i>uxmin</i>	$\min(u_x)$
<i>uymin</i>	$\min(u_y)$
<i>uzmin</i>	$\min(u_z)$
<i>uxmax</i>	$\max(u_x)$
<i>uymax</i>	$\max(u_y)$
<i>uzmax</i>	$\max(u_z)$
<i>uxm</i>	$\langle u_x \rangle$
<i>uym</i>	$\langle u_y \rangle$
<i>uzm</i>	$\langle u_z \rangle$
<i>ux2m</i>	$\langle u_x^2 \rangle$
<i>uy2m</i>	$\langle u_y^2 \rangle$
<i>uz2m</i>	$\langle u_z^2 \rangle$
<i>ux2ccm</i>	$\langle u_x^2 \cos^2 kz \rangle$
<i>ux2ssm</i>	$\langle u_x^2 \sin^2 kz \rangle$
<i>uy2ccm</i>	$\langle u_y^2 \cos^2 kz \rangle$

<i>uy2ssm</i>	$\langle u_y^2 \sin^2 kz \rangle$
<i>uxuyesm</i>	$\langle u_x u_y \cos kz \sin kz \rangle$
<i>uxuym</i>	$\langle u_x u_y \rangle$
<i>uxuzm</i>	$\langle u_x u_z \rangle$
<i>uyuzm</i>	$\langle u_y u_z \rangle$
<i>umx</i>	$\langle u_x \rangle$
<i>umy</i>	$\langle u_y \rangle$
<i>umz</i>	$\langle u_z \rangle$
<i>omumz</i>	$\langle \langle \mathbf{W} \rangle_{xy} \cdot \langle \mathbf{U} \rangle_{xy} \rangle$ (<i>xy</i> -averaged mean cross helicity production)
<i>umamz</i>	$\langle \langle \mathbf{u} \rangle_{xy} \cdot \langle \mathbf{A} \rangle_{xy} \rangle$
<i>umbmz</i>	$\langle \langle \mathbf{U} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle$ (<i>xy</i> -averaged mean cross helicity production)
<i>umxbmz</i>	$\langle \langle \mathbf{U} \rangle_{xy} \times \langle \mathbf{B} \rangle_{xy} \rangle_z$ (<i>xy</i> -averaged mean emf)
<i>rux2m</i>	$\langle \rho u_x^2 \rangle$
<i>ruy2m</i>	$\langle \rho u_y^2 \rangle$
<i>ruz2m</i>	$\langle \rho u_z^2 \rangle$
<i>divum</i>	$\langle \text{div} \mathbf{u} \rangle$
<i>rdivum</i>	$\langle \varrho \text{div} \mathbf{u} \rangle$
<i>divu2m</i>	$\langle (\text{div} \mathbf{u})^2 \rangle$
<i>gdivu2m</i>	$\langle (\text{grad div} \mathbf{u})^2 \rangle$
<i>u3u21m</i>	$\langle u_3 u_{2,1} \rangle$
<i>u1u32m</i>	$\langle u_1 u_{3,2} \rangle$
<i>u2u13m</i>	$\langle u_2 u_{1,3} \rangle$
<i>u2u31m</i>	$\langle u_2 u_{3,1} \rangle$
<i>u3u12m</i>	$\langle u_3 u_{1,2} \rangle$
<i>u1u23m</i>	$\langle u_1 u_{2,3} \rangle$
<i>ruxm</i>	$\langle \varrho u_x \rangle$ (mean <i>x</i> -momentum density)
<i>ruym</i>	$\langle \varrho u_y \rangle$ (mean <i>y</i> -momentum density)
<i>ruzm</i>	$\langle \varrho u_z \rangle$ (mean <i>z</i> -momentum density)
<i>ruxtot</i>	$\langle \rho u \rangle$ (mean absolute <i>x</i> -momentum density)
<i>rumax</i>	$\max(\varrho \mathbf{u})$ (maximum modulus of momentum)
<i>ruxuym</i>	$\langle \varrho u_x u_y \rangle$ (mean Reynolds stress)
<i>ruxuzm</i>	$\langle \varrho u_x u_z \rangle$ (mean Reynolds stress)
<i>ruyuzm</i>	$\langle \varrho u_y u_z \rangle$ (mean Reynolds stress)
<i>divrhourms</i>	$ \nabla \cdot (\varrho \mathbf{u}) _{\text{rms}}$
<i>divrhoumax</i>	$ \nabla \cdot (\varrho \mathbf{u}) _{\text{max}}$
<i>rlxm</i>	$\langle \rho y u_z - z u_y \rangle$
<i>rlym</i>	$\langle \rho z u_x - x u_z \rangle$
<i>rlzm</i>	$\langle \rho x u_y - y u_x \rangle$
<i>rlx2m</i>	$\langle (\rho y u_z - z u_y)^2 \rangle$
<i>rlz2m</i>	$\langle (\rho z u_x - x u_z)^2 \rangle$
<i>rlz2m</i>	$\langle (\rho x u_y - y u_x)^2 \rangle$
<i>tot_ang_mom</i>	Total angular momentum in spherical coordinates about the axis.
<i>dtu</i>	$\delta t / [c_{\delta t} \delta x / \max \mathbf{u}]$ (time step relative to advective time step; see § 5.16)
<i>oum</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle$

<i>ou_int</i>	$\int_V \boldsymbol{\omega} \cdot \mathbf{u} dV$
<i>fum</i>	$\langle \mathbf{f} \cdot \mathbf{u} \rangle$
<i>odel2um</i>	$\langle \boldsymbol{\omega} \nabla^2 \mathbf{u} \rangle$
<i>o2m</i>	$\langle \boldsymbol{\omega}^2 \rangle \equiv \langle (\nabla \times \mathbf{u})^2 \rangle$
<i>orms</i>	$\langle \boldsymbol{\omega}^2 \rangle^{1/2}$
<i>omax</i>	$\max(\boldsymbol{\omega})$
<i>ox2m</i>	$\langle \omega_x^2 \rangle$
<i>oy2m</i>	$\langle \omega_y^2 \rangle$
<i>oz2m</i>	$\langle \omega_z^2 \rangle$
<i>oxuzxm</i>	$\langle \omega_x u_{z,x} \rangle$
<i>oyuzym</i>	$\langle \omega_y u_{z,y} \rangle$
<i>oxoym</i>	$\langle \omega_x \omega_y \rangle$
<i>oxozm</i>	$\langle \omega_x \omega_z \rangle$
<i>oyozm</i>	$\langle \omega_y \omega_z \rangle$
<i>qfm</i>	$\langle \mathbf{q} \cdot \mathbf{f} \rangle$
<i>q2m</i>	$\langle \mathbf{q}^2 \rangle$
<i>qrms</i>	$\langle \mathbf{q}^2 \rangle^{1/2}$
<i>qmax</i>	$\max(\mathbf{q})$
<i>qom</i>	$\langle \mathbf{q} \cdot \boldsymbol{\omega} \rangle$
<i>quxom</i>	$\langle \mathbf{q} \cdot (\mathbf{u} \times \boldsymbol{\omega}) \rangle$
<i>oumph</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_\varphi$
<i>dudx</i>	$\langle \frac{\delta \mathbf{u}}{\delta x} \rangle$
<i>Marms</i>	$\langle \mathbf{u}^2 / c_s^2 \rangle$ (rms Mach number)
<i>Mamax</i>	$\max \mathbf{u} / c_s$ (maximum Mach number)
<i>EEK</i>	$\langle \varrho \mathbf{u}^2 \rangle / 2$
<i>ekin</i>	$\langle \frac{1}{2} \varrho \mathbf{u}^2 \rangle$
<i>ekintot</i>	$\int_V \frac{1}{2} \varrho \mathbf{u}^2 dV$
<i>uxglnr</i>	$\langle u_x \partial_y \ln \varrho \rangle$
<i>uyglnr</i>	$\langle u_y \partial_x \ln \varrho \rangle$
<i>uzdivum</i>	$\langle u_z \nabla \cdot \mathbf{u} \rangle$
<i>uxuydivum</i>	$\langle u_x u_y \nabla \cdot \mathbf{u} \rangle$
<i>divuHrms</i>	$(\nabla_H \cdot \mathbf{u}_H)^{\text{rms}}$
<i>uxxrms</i>	$u_{x,x}^{\text{rms}}$
<i>uyyrm</i>	$u_{y,y}^{\text{rms}}$
<i>uxzrm</i>	$u_{x,z}^{\text{rms}}$
<i>uyzrm</i>	$u_{y,z}^{\text{rms}}$
<i>uzyrm</i>	$u_{z,y}^{\text{rms}}$
<i>udpxxm</i>	components of symmetric tensor $\langle u_i \partial_j p + u_j \partial_i p \rangle$
Module ‘interstellar.f90’	
<i>taucmin</i>	$\min(\tau_{\text{cool}})$
<i>Hmax_ism</i>	$\max(\Gamma - \rho \Lambda)$
<i>Lamm</i>	$\langle \Lambda \rangle$
<i>nrhom</i>	TBC
<i>rhoLm</i>	$\langle \rho \Lambda \rangle$
<i>Gamm</i>	$\langle \Gamma \rangle$
Module ‘inverse_cascade.f90’	
<i>aa2m</i>	$\langle A^2 \rangle$
<i>ee2m</i>	$\langle E^2 \rangle$

<i>EEEM</i>	$\langle (E^2 + B^2)/2 \rangle$
<i>akxpt</i>	Akx^{pt}
<i>ekxpt</i>	$E kx^{pt}$
<i>sigma</i>	σ
<i>emag</i>	$\int_V \frac{1}{2\mu_0} \mathbf{B}^2 dV$
<i>bmax</i>	$\max(\mathbf{B})$
<i>brms</i>	$\langle \mathbf{B}^2 \rangle^{1/2}$
<i>arms</i>	$\langle \mathbf{A}^2 \rangle^{1/2}$
<i>erms</i>	$\langle \mathbf{E}^2 \rangle^{1/2}$
<i>bfrms</i>	$\langle \mathbf{B}^{\prime 2} \rangle^{1/2}$

Module 'klein_gordon.f90'

<i>phim</i>	$\langle \phi \rangle$
<i>phi2m</i>	$\langle \phi^2 \rangle$
<i>phirms</i>	$\langle \phi^2 \rangle^{1/2}$
<i>dphim</i>	$\langle \phi' \rangle$
<i>dphi2m</i>	$\langle (\phi')^2 \rangle$
<i>dphirms</i>	$\langle (\phi')^2 \rangle^{1/2}$
<i>psim</i>	$\langle \psi \rangle$
<i>psi2m</i>	$\langle \psi^2 \rangle$
<i>psirms</i>	$\langle \psi^2 \rangle^{1/2}$
<i>dpsim</i>	$\langle \psi' \rangle$
<i>dpsi2m</i>	$\langle (\psi')^2 \rangle$
<i>dpsirms</i>	$\langle (\psi')^2 \rangle^{1/2}$
<i>gphirms</i>	$\langle \partial phi \rangle$
<i>Hscriptm</i>	$\langle -1 * \mathcal{H} \rangle$
<i>lnam</i>	$\langle \ln a \rangle$
<i>Vprimem</i>	$\langle V_{,\phi} \rangle$
<i>Vprimepsim</i>	$\langle V_{,\psi} \rangle$
<i>rho_phi</i>	$\langle \rho phi \rangle$
<i>a</i>	$\langle a \rangle$!Sovan
<i>ddotam</i>	a''/a
<i>a2rhopm</i>	$a^2(rho + p)$
<i>a2rhom</i>	$a^2 rho$
<i>a2rhophim</i>	$a^2 rho$
<i>a2rhogphim</i>	$0.5 < grad phi^2 >$
<i>a2rhopsim</i>	$a^2 rho$
<i>a2rhogpsim</i>	$0.5 < grad psi^2 >$
<i>rho_chi</i>	ρ_χ
<i>sigEma</i>	ρ_χ
<i>sigBma</i>	ρ_χ
<i>count_eb0a</i>	f_{EB0}
<i>plasma_frictm</i>	$\langle \eta_\phi U_\nu \partial^\nu \phi \rangle$
<i>wall_vel</i>	v_w
<i>wall_pos</i>	r_w
<i>wall_lorentz</i>	$\frac{1}{\sqrt{1-v_w^2}}$

Module 'klein_gordon_philippe.f90'

<i>phim</i>	$\langle \phi \rangle$
<i>phi2m</i>	$\langle \phi^2 \rangle$
<i>phirms</i>	$\langle \phi^2 \rangle^{1/2}$
<i>dphim</i>	$\langle \phi' \rangle$
<i>dphi2m</i>	$\langle (\phi')^2 \rangle$
<i>dphirms</i>	$\langle (\phi')^2 \rangle^{1/2}$
<i>psim</i>	$\langle \psi \rangle$
<i>psi2m</i>	$\langle \psi^2 \rangle$
<i>psirms</i>	$\langle \psi^2 \rangle^{1/2}$
<i>dpsim</i>	$\langle \psi' \rangle$
<i>dpsi2m</i>	$\langle (\psi')^2 \rangle$
<i>dpsirms</i>	$\langle (\psi')^2 \rangle^{1/2}$
<i>Hscriptm</i>	$\langle \neg * \mathcal{H} \rangle$
<i>lnam</i>	$\langle \ln a \rangle$
<i>ddotam</i>	a''/a
<i>a2rhopm</i>	$a^2(\rho + p)$
<i>a2rhom</i>	$a^2\rho$
<i>a2rhophim</i>	$a^2\rho$
<i>a2rhogphim</i>	$0.5 < \text{grad}\phi^2 >$
<i>a2rhopsim</i>	$a^2\rho$
<i>a2rhogpsim</i>	$0.5 < \text{grad}\psi^2 >$
<i>rho_chi</i>	ρ_χ
<i>sigEma</i>	ρ_χ
<i>sigBma</i>	ρ_χ
<i>count_eb0a</i>	f_{EB0}

Module ‘klein_gordon_tmp.f90’

<i>phim</i>	$\langle \phi \rangle$
<i>phi2m</i>	$\langle \phi^2 \rangle$
<i>phirms</i>	$\langle \phi^2 \rangle^{1/2}$
<i>dphim</i>	$\langle \phi' \rangle$
<i>dphi2m</i>	$\langle (\phi')^2 \rangle$
<i>dphirms</i>	$\langle (\phi')^2 \rangle^{1/2}$
<i>psim</i>	$\langle \psi \rangle$
<i>psi2m</i>	$\langle \psi^2 \rangle$
<i>psirms</i>	$\langle \psi^2 \rangle^{1/2}$
<i>dpsim</i>	$\langle \psi' \rangle$
<i>dpsi2m</i>	$\langle (\psi')^2 \rangle$
<i>dpsirms</i>	$\langle (\psi')^2 \rangle^{1/2}$
<i>Hscriptm</i>	$\langle \neg * \mathcal{H} \rangle$
<i>lnam</i>	$\langle \ln a \rangle$
<i>ddotam</i>	a''/a
<i>a2rhopm</i>	$a^2(\rho + p)$
<i>a2rhom</i>	$a^2\rho$
<i>a2rhophim</i>	$a^2\rho$
<i>a2rhogphim</i>	$0.5 < \text{grad}\phi^2 >$
<i>a2rhopsim</i>	$a^2\rho$
<i>a2rhogpsim</i>	$0.5 < \text{grad}\psi^2 >$
<i>rho_chi</i>	ρ_χ

<i>sigEma</i>	ρ_χ
<i>sigBma</i>	ρ_χ
<i>count_eb0a</i>	f_{EB0}
Module ‘lorenz_gauge.f90’	
<i>phim</i>	$\langle \phi \rangle$
<i>phipt</i>	$\phi(x1, y1, z1)$
<i>phip2</i>	$\phi(x2, y2, z2)$
<i>phibzm</i>	$\langle \phi B_z \rangle$
<i>phibzmz</i>	$\langle \phi B_z \rangle_{xy}$
Module ‘lucky_droplet.f90’	
<i>rad</i>	r/r_*
<i>tauk</i>	τ_k
<i>tt1m</i>	$\langle T \rangle$
<i>qq1m</i>	$\langle \ln T \rangle$
<i>qq2m</i>	$\langle \ln T^2 \rangle$
<i>qq3m</i>	$\langle \ln T^3 \rangle$
<i>qq4m</i>	$\langle \ln T^4 \rangle$
Module ‘magnetic_shearboxJ.f90’	
<i>ab_int</i>	$\int \mathbf{A} \cdot \mathbf{B} \, dV$
<i>jb_int</i>	$\int \mathbf{j} \cdot \mathbf{B} \, dV$
<i>b2tm</i>	$\left\langle \mathbf{b}(t) \cdot \int_0^t \mathbf{b}(t') dt' \right\rangle$
<i>bjtm</i>	$\left\langle \mathbf{b}(t) \cdot \int_0^t \mathbf{j}(t') dt' \right\rangle$
<i>jbtm</i>	$\left\langle \mathbf{j}(t) \cdot \int_0^t \mathbf{b}(t') dt' \right\rangle$
<i>b2ruz</i>	$\langle \mathbf{B}^2 \rho u_z \rangle$
<i>b2uz</i>	$\langle \mathbf{B}^2 u_z \rangle$
<i>ubbz</i>	$\langle (\mathbf{u} \cdot \mathbf{B}) B_z \rangle$
<i>b1m</i>	$\langle \mathbf{B} \rangle$
<i>b2m</i>	$\langle \mathbf{B}^2 \rangle$
<i>bm2</i>	$\max(\mathbf{B}^2)$
<i>j2m</i>	$\langle \mathbf{j}^2 \rangle$
<i>jm2</i>	$\max(\mathbf{j}^2)$
<i>abm</i>	$\langle \mathbf{A} \cdot \mathbf{B} \rangle$
<i>abumx</i>	$\langle u_x \mathbf{A} \cdot \mathbf{B} \rangle$
<i>abumy</i>	$\langle u_y \mathbf{A} \cdot \mathbf{B} \rangle$
<i>abumz</i>	$\langle u_z \mathbf{A} \cdot \mathbf{B} \rangle$
<i>abmh</i>	$\langle \mathbf{A} \cdot \mathbf{B} \rangle$ (temp)
<i>abmn</i>	$\langle \mathbf{A} \cdot \mathbf{B} \rangle$ (north)
<i>abms</i>	$\langle \mathbf{A} \cdot \mathbf{B} \rangle$ (south)
<i>abrms</i>	$\langle (\mathbf{A} \cdot \mathbf{B})^2 \rangle^{1/2}$
<i>jbrms</i>	$\langle (\mathbf{j} \cdot \mathbf{B})^2 \rangle^{1/2}$
<i>ajm</i>	$\langle \mathbf{j} \cdot \mathbf{A} \rangle$
<i>jbm</i>	$\langle \mathbf{j} \cdot \mathbf{B} \rangle$
<i>jbmh</i>	$\langle \mathbf{J} \cdot \mathbf{B} \rangle$ (temp)
<i>jbmh</i>	$\langle \mathbf{J} \cdot \mathbf{B} \rangle$ (north)
<i>jbms</i>	$\langle \mathbf{J} \cdot \mathbf{B} \rangle$ (south)
<i>ubm</i>	$\langle \mathbf{u} \cdot \mathbf{B} \rangle$

<i>dubrms</i>	$\langle (\mathbf{u} - \mathbf{B})^2 \rangle^{1/2}$
<i>dobrms</i>	$\langle (\boldsymbol{\omega} - \mathbf{B})^2 \rangle^{1/2}$
<i>uxbxm</i>	$\langle u_x B_x \rangle$
<i>uybxm</i>	$\langle u_y B_x \rangle$
<i>uzbxm</i>	$\langle u_z B_x \rangle$
<i>uxbym</i>	$\langle u_x B_y \rangle$
<i>uybym</i>	$\langle u_y B_y \rangle$
<i>uzbym</i>	$\langle u_z B_y \rangle$
<i>uxbzm</i>	$\langle u_x B_z \rangle$
<i>uybzm</i>	$\langle u_y B_z \rangle$
<i>uzbzm</i>	$\langle u_z B_z \rangle$
<i>cosubm</i>	$\langle \mathbf{U} \cdot \mathbf{B} / (\mathbf{U} \mathbf{B}) \rangle$
<i>jxbxm</i>	$\langle j_x B_x \rangle$
<i>jybxm</i>	$\langle j_y B_x \rangle$
<i>jzbxm</i>	$\langle j_z B_x \rangle$
<i>jxbym</i>	$\langle j_x B_y \rangle$
<i>jybym</i>	$\langle j_y B_y \rangle$
<i>jzbym</i>	$\langle j_z B_y \rangle$
<i>jxbzm</i>	$\langle j_x B_z \rangle$
<i>jybz</i>	$\langle j_y B_z \rangle$
<i>jzbzm</i>	$\langle j_z B_z \rangle$
<i>uam</i>	$\langle \mathbf{u} \cdot \mathbf{A} \rangle$
<i>ujm</i>	$\langle \mathbf{u} \cdot \mathbf{J} \rangle$
<i>fbm</i>	$\langle \mathbf{f} \cdot \mathbf{B} \rangle$
<i>fxbxm</i>	$\langle f_x B_x \rangle$
<i>epsM</i>	$\langle \eta \mu_0 \mathbf{j}^2 \rangle$
<i>epsAD</i>	$\langle \rho^{-1} t_{AD} (\mathbf{J} \times \mathbf{B})^2 \rangle$ (heating by ion-neutrals friction)
<i>bxpt</i>	$B_x(x_1, y_1, z_1, t)$
<i>bypt</i>	$B_y(x_1, y_1, z_1, t)$
<i>bzpt</i>	$B_z(x_1, y_1, z_1, t)$
<i>jxpt</i>	$J_x(x_1, y_1, z_1, t)$
<i>jypt</i>	$J_y(x_1, y_1, z_1, t)$
<i>jzpt</i>	$J_z(x_1, y_1, z_1, t)$
<i>Expt</i>	$\mathcal{E}_x(x_1, y_1, z_1, t)$
<i>Eypt</i>	$\mathcal{E}_y(x_1, y_1, z_1, t)$
<i>Ezpt</i>	$\mathcal{E}_z(x_1, y_1, z_1, t)$
<i>axpt</i>	$A_x(x_1, y_1, z_1, t)$
<i>aypt</i>	$A_y(x_1, y_1, z_1, t)$
<i>azpt</i>	$A_z(x_1, y_1, z_1, t)$
<i>bxp2</i>	$B_x(x_2, y_2, z_2, t)$
<i>byp2</i>	$B_y(x_2, y_2, z_2, t)$
<i>bzp2</i>	$B_z(x_2, y_2, z_2, t)$
<i>jxp2</i>	$J_x(x_2, y_2, z_2, t)$
<i>jyp2</i>	$J_y(x_2, y_2, z_2, t)$
<i>jzp2</i>	$J_z(x_2, y_2, z_2, t)$
<i>Exp2</i>	$\mathcal{E}_x(x_2, y_2, z_2, t)$
<i>Eyp2</i>	$\mathcal{E}_y(x_2, y_2, z_2, t)$
<i>Ezp2</i>	$\mathcal{E}_z(x_2, y_2, z_2, t)$
<i>axp2</i>	$A_x(x_2, y_2, z_2, t)$
<i>ayp2</i>	$A_y(x_2, y_2, z_2, t)$

<i>azp2</i>	$A_z(x_2, y_2, z_2, t)$
<i>exabot</i>	$\int \mathbf{E} \times \mathbf{A} dS _{\text{bot}}$
<i>exatop</i>	$\int \mathbf{E} \times \mathbf{A} dS _{\text{top}}$
<i>emag</i>	$\int_V \frac{1}{2\mu_0} \mathbf{B}^2 dV$
<i>brms</i>	$\langle \mathbf{B}^2 \rangle^{1/2}$
<i>bfrms</i>	$\langle \mathbf{B}'^2 \rangle^{1/2}$
<i>bmax</i>	$\max(\mathbf{B})$
<i>bxmin</i>	$\min(B_x)$
<i>bymin</i>	$\min(B_y)$
<i>bzmin</i>	$\min(B_z)$
<i>bxmax</i>	$\max(B_x)$
<i>bymax</i>	$\max(B_y)$
<i>bzmax</i>	$\max(B_z)$
<i>bbxmax</i>	$\max(B_x) \text{ excluding } Bv_{ext}$
<i>bbymax</i>	$\max(B_y) \text{ excluding } Bv_{ext}$
<i>bbzmax</i>	$\max(B_z) \text{ excluding } Bv_{ext}$
<i>jxmax</i>	$\max(jv_x)$
<i>jymax</i>	$\max(jv_y)$
<i>jzmax</i>	$\max(jv_z)$
<i>jrms</i>	$\langle \mathbf{j}^2 \rangle^{1/2}$
<i>hjrms</i>	$\langle \mathbf{j}^2 \rangle^{1/2}$
<i>jmax</i>	$\max(\mathbf{j})$
<i>vArms</i>	$\langle \mathbf{B}^2 / \varrho \rangle^{1/2}$
<i>vAmax</i>	$\max(\mathbf{B}^2 / \varrho)^{1/2}$
<i>dtb</i>	$\delta t / [c_{\delta t} \delta x / v_{A, \max}]$ (time step relative to Alfvén time step; see § 5.16)
<i>dteta</i>	$\delta t / [c_{\delta t, v} \delta x^2 / \eta_{\max}]$ (time step relative to resistive time step; see § 5.16)
<i>a2m</i>	$\langle \mathbf{A}^2 \rangle$
<i>arms</i>	$\langle \mathbf{A}^2 \rangle^{1/2}$
<i>amax</i>	$\max(\mathbf{A})$
<i>divarms</i>	$\langle (\nabla \cdot \mathbf{A})^2 \rangle^{1/2}$
<i>beta1m</i>	$\langle \mathbf{B}^2 / (2\mu_0 p) \rangle$ (mean inverse plasma beta)
<i>beta1max</i>	$\max[\mathbf{B}^2 / (2\mu_0 p)]$ (maximum inverse plasma beta)
<i>betam</i>	$\langle \beta \rangle$
<i>betamax</i>	$\max \beta$
<i>betamin</i>	$\min \beta$
<i>bxm</i>	$\langle B_x \rangle$
<i>bym</i>	$\langle B_y \rangle$
<i>bzm</i>	$\langle B_z \rangle$
<i>bxbym</i>	$\langle B_x B_y \rangle$
<i>bmz</i>	$\langle \langle \mathbf{B}_{yz}^2 \rangle \rangle^{1/2}$ (energy of yz -averaged mean field)
<i>bmy</i>	$\langle \langle \mathbf{B}_{xz}^2 \rangle \rangle^{1/2}$ (energy of xz -averaged mean field)
<i>bmz</i>	$\langle \langle \mathbf{B}_{xy}^2 \rangle \rangle^{1/2}$ (energy of xy -averaged mean field)
<i>bmzS2</i>	$\langle \langle \mathbf{B}_S^2 \rangle_{xy} \rangle$
<i>bmzA2</i>	$\langle \langle \mathbf{B}_A^2 \rangle_{xy} \rangle$

<i>jmx</i>	$\langle \langle \mathbf{J} \rangle_{yz}^2 \rangle^{1/2}$ (energy of <i>yz</i> -averaged mean current density)
<i>jmy</i>	$\langle \langle \mathbf{J} \rangle_{xz}^2 \rangle^{1/2}$ (energy of <i>xz</i> -averaged mean current density)
<i>jnz</i>	$\langle \langle \mathbf{J} \rangle_{xy}^2 \rangle^{1/2}$ (energy of <i>xy</i> -averaged mean current density)
<i>bmzph</i>	Phase of a Beltrami field
<i>bmzphe</i>	Error of phase of a Beltrami field
<i>bsinphz</i>	sine of phase of a Beltrami field
<i>bcosphz</i>	cosine of phase of a Beltrami field
<i>emxamz3</i>	$\langle \langle \mathbf{E} \rangle_{xy} \times \langle \mathbf{A} \rangle_{xy} \rangle$ (<i>xy</i> -averaged mean field helicity flux)
<i>embmz</i>	$\langle \langle \mathbf{E} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle$ (<i>xy</i> -averaged mean field helicity production)
<i>ambmz</i>	$\langle \langle \mathbf{A} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle$ (magnetic helicity of <i>xy</i> -averaged mean field)
<i>ambmzh</i>	$\langle \langle \mathbf{A} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle$ (magnetic helicity of <i>xy</i> -averaged mean field, temp)
<i>ambmzn</i>	$\langle \langle \mathbf{A} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle$ (magnetic helicity of <i>xy</i> -averaged mean field, north)
<i>ambmzs</i>	$\langle \langle \mathbf{A} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle$ (magnetic helicity of <i>xy</i> -averaged mean field, south)
<i>jmbmz</i>	$\langle \langle \mathbf{J} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle$ (current helicity of <i>xy</i> -averaged mean field)
<i>kx_aa</i>	k_x
<i>kmz</i>	$\langle \langle \mathbf{J} \rangle_{xy} \cdot \langle \mathbf{B} \rangle_{xy} \rangle / \langle \langle \mathbf{B} \rangle_{xy}^2 \rangle$
<i>bx2m</i>	$\langle B_x^2 \rangle$
<i>by2m</i>	$\langle B_y^2 \rangle$
<i>bz2m</i>	$\langle B_z^2 \rangle$
<i>uxbm</i>	$\langle \mathbf{u} \times \mathbf{B} \rangle \cdot \mathbf{B}_0 / B_0^2$
<i>jxbm</i>	$\langle \mathbf{j} \times \mathbf{B} \rangle \cdot \mathbf{B}_0 / B_0^2$
<i>magfricmax</i>	Magneto-Frictional velocity $\langle \mathbf{j} \times \mathbf{B} \rangle \cdot \mathbf{B}^2$
<i>b3b21m</i>	$\langle B_3 B_{2,1} \rangle$
<i>b3b12m</i>	$\langle B_3 B_{1,2} \rangle$
<i>b1b32m</i>	$\langle B_1 B_{3,2} \rangle$
<i>b1b23m</i>	$\langle B_1 B_{2,3} \rangle$
<i>b2b13m</i>	$\langle B_2 B_{1,3} \rangle$
<i>b2b31m</i>	$\langle B_2 B_{3,1} \rangle$
<i>uxbmx</i>	$\langle (\mathbf{u} \times \mathbf{B})_x \rangle$
<i>uxbmy</i>	$\langle (\mathbf{u} \times \mathbf{B})_y \rangle$
<i>uxbmz</i>	$\langle (\mathbf{u} \times \mathbf{B})_z \rangle$
<i>jxbmx</i>	$\langle (\mathbf{j} \times \mathbf{B})_x \rangle$
<i>jxbmy</i>	$\langle (\mathbf{j} \times \mathbf{B})_y \rangle$
<i>jxbmz</i>	$\langle (\mathbf{j} \times \mathbf{B})_z \rangle$
<i>examx</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_x$
<i>examy</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_y$
<i>examz</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_z$
<i>exjmx</i>	$\langle \mathbf{E} \times \mathbf{J} \rangle_x$
<i>exjmy</i>	$\langle \mathbf{E} \times \mathbf{J} \rangle_y$
<i>exjnz</i>	$\langle \mathbf{E} \times \mathbf{J} \rangle_z$

<i>dexbm_x</i>	$\langle \nabla \times \mathbf{E} \times \mathbf{B} \rangle _x$
<i>dexbm_y</i>	$\langle \nabla \times \mathbf{E} \times \mathbf{B} \rangle _y$
<i>dexbm_z</i>	$\langle \nabla \times \mathbf{E} \times \mathbf{B} \rangle _z$
<i>phibm_x</i>	$\langle \phi \mathbf{B} \rangle _x$
<i>phibm_y</i>	$\langle \phi \mathbf{B} \rangle _y$
<i>phibm_z</i>	$\langle \phi \mathbf{B} \rangle _z$
<i>b2divum</i>	$\langle \mathbf{B}^2 \nabla \cdot \mathbf{u} \rangle$
<i>ujxbm</i>	$\langle \mathbf{u} \cdot (\mathbf{J} \times \mathbf{B}) \rangle$
<i>jxbrmax</i>	$\max(\mathbf{J} \times \mathbf{B}/\rho)$
<i>jxbr2m</i>	$\langle (\mathbf{J} \times \mathbf{B}/\rho)^2 \rangle$
<i>bmxy_rms</i>	$\sqrt{[\langle b_x \rangle_z(x, y)]^2 + [\langle b_y \rangle_z(x, y)]^2 + [\langle b_z \rangle_z(x, y)]^2}$
<i>etasmagm</i>	Mean of Smagorinsky resistivity
<i>etasmagmin</i>	Min of Smagorinsky resistivity
<i>etasmagmax</i>	Max of Smagorinsky resistivity
<i>etavamax</i>	Max of artificial resistivity $\eta \sim v_A$
<i>etajmax</i>	Max of artificial resistivity $\eta \sim J/\sqrt{\rho}$
<i>etaj2max</i>	Max of artificial resistivity $\eta \sim J^2/\rho$
<i>etajrhmax</i>	Max of artificial resistivity $\eta \sim J/\rho$
<i>cosjbm</i>	$\langle \mathbf{J} \cdot \mathbf{B}/(\mathbf{J} \mathbf{B}) \rangle$
<i>jparallelm</i>	Mean value of the component of $\mathbf{J}$ parallel to $\mathbf{B}$
<i>jperpm</i>	Mean value of the component of $\mathbf{J}$ perpendicular to $\mathbf{B}$
<i>hjparallelm</i>	Mean value of the component of J_{hyper} parallel to $\mathbf{B}$
<i>hjperpm</i>	Mean value of the component of J_{hyper} perpendicular to $\mathbf{B}$
<i>brmsx</i>	$\langle \mathbf{B}^2 \rangle^{1/2}$ for the magnetic_xaver_range
<i>brmsz</i>	$\langle \mathbf{B}^2 \rangle^{1/2}$ for the magnetic_zaver_range
<i>Exmxy</i>	$\langle \mathcal{E}_x \rangle_z$
<i>Eymxy</i>	$\langle \mathcal{E}_y \rangle_z$
<i>Ezmxy</i>	$\langle \mathcal{E}_z \rangle_z$

Module 'maxwell.f90'

<i>aa2m</i>	$\langle A^2 \rangle$
<i>ee2m</i>	$\langle E^2 \rangle$
<i>EEEM</i>	$\langle (E^2 + B^2)/2 \rangle$
<i>akxpt</i>	Akx^{pt}
<i>ekxpt</i>	Ekx^{pt}
<i>sigma</i>	σ
<i>emag</i>	$\int_V \frac{1}{2\mu_0} \mathbf{B}^2 dV$
<i>bmax</i>	$\max(\mathbf{B})$
<i>brms</i>	$\langle \mathbf{B}^2 \rangle^{1/2}$
<i>arms</i>	$\langle A^2 \rangle^{1/2}$
<i>erms</i>	$\langle E^2 \rangle^{1/2}$
<i>bfrms</i>	$\langle B'^2 \rangle^{1/2}$

Module 'meanfield.f90'

<i>qsm</i>	$\langle q_p(\overline{B}) \rangle$
<i>qpm</i>	$\langle q_p(\overline{B}) \rangle$
<i>qem</i>	$\langle q_e(\overline{B}) \rangle$, in the paper referred to as $\langle q_g(\overline{B}) \rangle$
<i>qam</i>	$\langle q_a(\overline{B}) \rangle$

<i>alpm</i>	$\langle \alpha \rangle$!(where is this implemented?)
<i>etatm</i>	$\langle \eta_t \rangle$
<i>EMFmz1</i>	$\langle \mathcal{E} \rangle_{xy} x$
<i>EMFmz2</i>	$\langle \mathcal{E} \rangle_{xy} y$
<i>EMFmz3</i>	$\langle \mathcal{E} \rangle_{xy} z$
<i>EMFdotBm</i>	$\langle \mathcal{E} \cdot \mathbf{B} \rangle$
<i>EMFdotB_int</i>	$\int \mathcal{E} \cdot \mathbf{B} dV$
<i>alpKjbm</i>	$\langle \alpha_K \overline{\mathbf{B}} \cdot \overline{\mathbf{J}} \rangle$
<i>alpKm</i>	$\langle \alpha_K \rangle$
Module ‘meanfield_demfdt.f90’	
<i>EMFrms</i>	$(\langle \mathcal{E} \rangle)_{\text{rms}}$
<i>EMFmax</i>	$\max(\langle \mathcal{E} \rangle)$
<i>EMFmin</i>	$\min(\langle \mathcal{E} \rangle)$
Module ‘neutralvelocity.f90’	
<i>epsKn</i>	$\langle 2\nu_n \varrho_n \mathbf{S}_n^2 \rangle$
Module ‘noentropy.f90’	
<i>dte</i>	$\delta t / [c_{\delta t} \delta_x / \max c_s]$ (time step relative to acoustic time step; see § 5.16)
<i>ethm</i>	$\langle \varrho e \rangle$ (mean thermal [=internal] energy)
<i>pdivum</i>	$\langle p \nabla \mathbf{u} \rangle$
<i>csm</i>	$\langle c_s \rangle$
Module ‘particles_caustics.f90’	
<i>TrSigmapm</i>	$\langle \text{Tr} [\sigma] \rangle$
<i>blowupm</i>	Mean no. of times σ falls below cutoff
<i>lnVpm</i>	Mean of (logarithm of) Volume around an inertial particle
Module ‘particles_chemistry.f90’	
<i>Shchm</i>	meanparticleSherwoodnumber
Module ‘particles_dust.f90’	
<i>xpm</i>	x_{part}
<i>xpmin</i>	x_{part}
<i>xpmax</i>	x_{part}
<i>xp2m</i>	x_{part}^2
<i>vrelpabsm</i>	Absolutevalueofmeanrelativevelocity
<i>vpxm</i>	u_{part}
<i>vpx2m</i>	u_{part}^2
<i>vpcoalx2m</i>	$v_{\text{coal,part}}^2$ (thermal/coalescence velocity entering the Brownian coagulation kernel)
<i>ekinp</i>	$E_{\text{kin,part}}$
<i>vpxmax</i>	$MAX(u_{\text{part}})$
<i>vpxmin</i>	$MIN(u_{\text{part}})$
<i>npm</i>	meanparticlenumberdensity
Module ‘particles_dust_brdeplete.f90’	
<i>xpm</i>	x_{part}
<i>xp2m</i>	x_{part}^2

<i>vrelpabsm</i>	Absolutevalueofmeanrelativevelocity
<i>vpxm</i>	u_{part}
<i>vpx2m</i>	u_{part}^2
<i>ekinp</i>	$E_{kin,part}$
<i>vpxmax</i>	$MAX(u_{part})$
<i>vpxmin</i>	$MIN(u_{part})$
<i>npm</i>	meanparticlenumberdensity
Module ‘particles_lagrangian.f90’	
<i>xpm</i>	x_{part}
<i>xp2m</i>	x_{part}^2
<i>vrelpabsm</i>	Absolutevalueofmeanrelativevelocity
<i>vpxm</i>	u_{part}
<i>vpx2m</i>	u_{part}^2
<i>ekinp</i>	$E_{kin,part}$
<i>vpxmax</i>	$MAX(u_{part})$
<i>vpxmin</i>	$MIN(u_{part})$
<i>npm</i>	meanparticlenumberdensity
Module ‘particles_mass_swarm.f90’	
<i>mpm</i>	$\overline{m_p}$
<i>mpmin</i>	$\min_j m_{p,j}$
<i>mpmax</i>	$\max_j m_{p,j}$
Module ‘particles_surfspec.f90’	
<i>dtpchm</i>	$dt_{particle,chemistry}$
Module ‘particles_tetrad.f90’	
<i>TVolm</i>	Mean absolute volume of the tetrads
<i>TVolpm</i>	Mean of positive volume of the tetrads
<i>TVolnm</i>	Mean of negative volume of the tetrads
<i>VelVolm</i>	Mean absolute volume of the tetrads in velocity space
<i>VelVolpm</i>	Mean of positive volume of the tetrads in velocity space
<i>VelVolnm</i>	Mean of negative volume of the tetrads in velocity space.
Module ‘polymer.f90’	
<i>polytrm</i>	$\langle Tr[C_{ij}] \rangle$
<i>frmax</i>	$\max(f(r))$
Module ‘pscalar_nolog.f90’	
<i>rhoccm</i>	$\langle \rho c_1 \rangle$
<i>ccmax</i>	$\max[c_1]$
<i>ccmin</i>	$\min[c_1]$
<i>mrclncm</i>	$\langle \rho c_1 \log(c_1) \rangle$
<i>rhoccmax</i>	$\max[\rho c_1]$
<i>rhoc2m</i>	$\langle \rho c_2 \rangle$
<i>rhoc3m</i>	$\langle \rho c_3 \rangle$
<i>ucm</i>	$\langle u_z c_1 \rangle$ (with an extra factor of $2 \cos(z)$ if <code>lgradC_profile=T</code>)
<i>uudcm</i>	$\langle u_z \mathbf{u} \cdot \nabla c_1 \rangle$
<i>Cz2m</i>	$\langle \rho c_1 z^2 \rangle$
<i>Cz4m</i>	$\langle \rho c_1 z^4 \rangle$

<i>Crmsm</i>	$\sqrt{\langle \rho c_1^2 \rangle}$
<i>ccrms</i>	$\sqrt{\langle c_1^2 \rangle}$
<i>uxcm</i>	$\langle u_x c_1 \rangle$
<i>uycm</i>	$\langle u_y c_1 \rangle$
<i>uzcm</i>	$\langle u_z c_1 \rangle$
<i>cc1m</i>	$\langle c_1^{-1} \rangle$
<i>cc2m</i>	$\langle c_1^{-2} \rangle$
<i>cc3m</i>	$\langle c_1^{-3} \rangle$
<i>cc4m</i>	$\langle c_1^{-4} \rangle$
<i>cc5m</i>	$\langle c_1^{-5} \rangle$
<i>cc6m</i>	$\langle c_1^{-6} \rangle$
<i>cc7m</i>	$\langle c_1^{-7} \rangle$
<i>cc8m</i>	$\langle c_1^{-8} \rangle$
<i>cc9m</i>	$\langle c_1^{-9} \rangle$
<i>cc10m</i>	$\langle c_1^{-10} \rangle$
<i>gcc1m</i>	$\langle \nabla c_1 \rangle$
<i>gcc2m</i>	$\langle \nabla c_1 ^2 \rangle$
<i>gcc3m</i>	$\langle \nabla c_1 ^3 \rangle$
<i>gcc4m</i>	$\langle \nabla c_1 ^4 \rangle$
<i>gcc5m</i>	$\langle \nabla c_1 ^5 \rangle$
<i>gcc6m</i>	$\langle \nabla c_1 ^6 \rangle$
<i>gcc7m</i>	$\langle \nabla c_1 ^7 \rangle$
<i>gcc8m</i>	$\langle \nabla c_1 ^8 \rangle$
<i>gcc9m</i>	$\langle \nabla c_1 ^9 \rangle$
<i>gcc10m</i>	$\langle \nabla c_1 ^{10} \rangle$
<i>cugccm</i>	$\langle c_1 \mathbf{u} \cdot \nabla c_1 \rangle$
<i>ccugum</i>	$\langle c_1 \mathbf{u} \cdot \nabla u_z \rangle$
<i>ccglhnm</i>	$\langle c_1 \partial_z \log \rho \rangle$

Module ‘radial_dist_func.f90’

<i>rad</i>	r/r_*
<i>tauk</i>	τ_k
<i>tt1m</i>	$\langle T \rangle$
<i>qq1m</i>	$\langle \ln T \rangle$
<i>qq2m</i>	$\langle \ln T^2 \rangle$
<i>qq3m</i>	$\langle \ln T^3 \rangle$
<i>qq4m</i>	$\langle \ln T^4 \rangle$

Module ‘reaction_0D.f90’

<i>eem</i>	$\langle \eta \rangle$
<i>ee1m</i>	$\langle \eta \rangle$
<i>ee2m</i>	$\langle \eta^2 \rangle$
<i>ee3m</i>	$\langle \eta^3 \rangle$
<i>ee4m</i>	$\langle \eta^4 \rangle$
<i>ee10</i>	$\langle \eta_{10\%} \rangle$
<i>ee50</i>	$\langle \eta_{50\%} \rangle$
<i>ee90</i>	$\langle \eta_{90\%} \rangle$
<i>ee99</i>	$\langle \eta_{99\%} \rangle$
<i>AAm</i>	$\langle [A] \rangle$
<i>DDm</i>	$\langle [D] \rangle$

<i>LLm</i>	$\langle [L] \rangle$
<i>DLm</i>	$\langle [D] + [L] \rangle$
<i>kC</i>	k_C
<i>A1</i>	A_1
<i>A2</i>	A_2
<i>A3</i>	A_3
<i>A4</i>	A_4
<i>A5</i>	A_5
<i>D1</i>	D_1
<i>D2</i>	D_2
<i>D3</i>	D_3
<i>D4</i>	D_4
<i>D5</i>	D_5
<i>L1</i>	L_1
<i>L2</i>	L_2
<i>L3</i>	L_3
<i>L4</i>	L_4
<i>L5</i>	L_5
Module ‘rel_1d.f90’	
<i>betm</i>	$\langle \beta \rangle$
<i>betmax</i>	$\beta_{\max}$
Module ‘schur_triple_decomp.f90’	
<i>uSH2</i>	$\langle u^{\text{SH}2} \rangle$
<i>uRR2</i>	$\langle u^{\text{RR}2} \rangle$
<i>uEL2</i>	$\langle u^{\text{EL}2} \rangle$
<i>uRRm</i>	$\langle 2u^{\text{SH}}u^{\text{RR}} \rangle$
<i>bSH2</i>	$\langle b^{\text{SH}2} \rangle$
<i>bRR2</i>	$\langle b^{\text{RR}2} \rangle$
<i>bEL2</i>	$\langle b^{\text{EL}2} \rangle$
<i>bRRm</i>	$\langle 2b^{\text{SH}}b^{\text{RR}} \rangle$
Module ‘selfgravity.f90’	
<i>rugpotselfm</i>	$\langle \rho \mathbf{u} \cdot \nabla \Phi \rangle$
<i>gpotself2m</i>	$\langle (\nabla \Phi)^2 \rangle$
Module ‘shear.f90’	
<i>dtshear</i>	<code>advec_shear/cdt</code>
<i>deltay</i>	<code>deltay</code>
Module ‘shock.f90’	
<i>shockmax</i>	Max shock number
Module ‘shock_highorder.f90’	
<i>gshockmax</i>	$\max \nabla \nu_{\text{shock}} $
Module ‘solar_corona.f90’	

<i>dtvel</i>	Velocity driver time step
<i>dtnewt</i>	Radiative cooling time step
<i>dtradloss</i>	Radiative losses time step
<i>dtchi2</i>	$\delta t / [c_{\delta t, v} \delta x^2 / \chi_{\max}]$ (time step relative to time step based on heat conductivity; see § 5.16)
<i>dtspitzer</i>	Spitzer heat conduction time step
<i>mag_flux</i>	Total vertical magnetic flux at
Module ‘solid_cells_CGE0.f90’	
Module ‘solid_cells_ogrid_chemistry.f90’	
<i>dtchem</i>	dt_{chem}
Module ‘solid_cells_reactive.f90’	
Module ‘temperature_idealgas.f90’	
<i>TTmax</i>	$\max(T)$
<i>gTmax</i>	$\max(\nabla T)$
<i>TTmin</i>	$\min(T)$
<i>TTm</i>	$\langle T \rangle$
<i>TTzmask</i>	$\langle T \rangle$ for the temp_zaver_range
<i>TT2m</i>	$\langle T^2 \rangle$
<i>TugTm</i>	$\langle T \mathbf{u} \cdot \nabla T \rangle$
<i>Trms</i>	$\sqrt{\langle T^2 \rangle}$
<i>uxTm</i>	$\langle u_x T \rangle$
<i>uyTm</i>	$\langle u_y T \rangle$
<i>uzTm</i>	$\langle u_z T \rangle$
<i>gT2m</i>	$\langle (\nabla T)^2 \rangle$
<i>guxgTm</i>	$\langle \nabla u_x \cdot \nabla T \rangle$
<i>guygTm</i>	$\langle \nabla u_y \cdot \nabla T \rangle$
<i>guzgTm</i>	$\langle \nabla u_z \cdot \nabla T \rangle$
<i>Tugux_uxugTm</i>	$\langle T \mathbf{u} \cdot \nabla u_x + u_x \mathbf{u} \cdot \nabla T \rangle = \langle \mathbf{u} \cdot \nabla (u_x T) \rangle$
<i>Tuguy_uyugTm</i>	$\langle T \mathbf{u} \cdot \nabla u_y + u_y \mathbf{u} \cdot \nabla T \rangle = \langle \mathbf{u} \cdot \nabla (u_y T) \rangle$
<i>Tuguz_uzugTm</i>	$\langle T \mathbf{u} \cdot \nabla u_z + u_z \mathbf{u} \cdot \nabla T \rangle = \langle \mathbf{u} \cdot \nabla (u_z T) \rangle$
<i>Tdxpm</i>	$\langle T dp/dx \rangle$
<i>Tdypm</i>	$\langle T dp/dy \rangle$
<i>Tdzpm</i>	$\langle T dp/dz \rangle$
<i>fradtop</i>	$< -K \frac{dT}{dz} >_{\text{top}}$ (top radiative flux)
<i>fradbot</i>	$< -K \frac{dT}{dz} >_{\text{bot}}$ (bottom radiative flux)
<i>yHmax</i>	DOCUMENT ME
<i>yHmin</i>	DOCUMENT ME
<i>yHm</i>	DOCUMENT ME
<i>ethm</i>	$\langle e_{\text{th}} \rangle = \langle c_v \rho T \rangle$ (mean thermal energy)
<i>eem</i>	$\langle e \rangle = \langle c_v T \rangle$ (mean internal energy)
<i>ethtot</i>	$\int_V \rho e dV$ (total thermal energy)
<i>ssm</i>	$\bar{S}$
<i>thcool</i>	τ_{cool}
<i>ppm</i>	$\bar{P}$
<i>csm</i>	$\bar{c}_s$
<i>csmmax</i>	$\max(c_s)$

<i>dtc</i>	$\delta t / [c_{\delta t} \delta x / \max c_s]$ (time step relative to acoustic time step; see § 5.16)
<i>dtchi</i>	$\delta t / [c_{\delta t, v} \delta x^2 / \chi_{\max}]$ (time step relative to time step based on heat conductivity; see § 5.16)

Module ‘temperature_ionization.f90’

<i>TTmax</i>	$\max(T)$
<i>TTmin</i>	$\min(T)$
<i>TTm</i>	$\langle T \rangle$
<i>ethm</i>	$\langle e_{\text{th}} \rangle = \langle c_v \rho T \rangle$ (mean thermal energy)
<i>eem</i>	$\langle e \rangle$ (mean internal energy)
<i>ppm</i>	$\langle p \rangle$
<i>Tppm</i>	$\langle \max(p_{\text{thresh}} - p, 0)_{\text{norm}} \rangle$
<i>heatThm</i>	α_{Th}
<i>TTref</i>	T_{ref}

Module ‘test_chemistry.f90’

<i>dtchem</i>	dt_{chem}
---------------	--------------------

Module ‘testfield_axisym.f90’

<i>alpPERP</i>	$\alpha_{\perp}$
<i>alpPARA</i>	$\alpha_{\perp}$
<i>gam</i>	γ
<i>betPERP</i>	$\beta_{\perp}$
<i>betPARA</i>	$\beta_{\perp}$
<i>del</i>	δ
<i>kapPERP</i>	$\kappa_{\perp}$
<i>kapPARA</i>	$\kappa_{\perp}$
<i>mu</i>	μ
<i>alpPERPz</i>	$\alpha_{\perp}(z)$
<i>alpPARAz</i>	$\alpha_{\perp}(z)$
<i>gamz</i>	$\gamma(z)$
<i>betPERPz</i>	$\beta_{\perp}(z)$
<i>betPARAz</i>	$\beta_{\perp}(z)$
<i>delz</i>	$\delta(z)$
<i>kapPERPz</i>	$\kappa_{\perp}(z)$
<i>kapPARAz</i>	$\kappa_{\perp}(z)$
<i>muz</i>	$\mu(z)$
<i>bx1pt</i>	b_x^1
<i>bx2pt</i>	b_x^2
<i>bx3pt</i>	b_x^3
<i>b1rms</i>	$\langle b_1^2 \rangle^{1/2}$
<i>b2rms</i>	$\langle b_2^2 \rangle^{1/2}$
<i>b3rms</i>	$\langle b_3^2 \rangle^{1/2}$

Module ‘testfield_axisym2.f90’

<i>alpPERP</i>	$\alpha_{\perp}$
<i>alpPARA</i>	$\alpha_{\perp}$
<i>gam</i>	γ
<i>betPERP</i>	$\beta_{\perp}$

<i>bet</i> PARA	$\beta_{\perp}$
<i>del</i>	δ
<i>kap</i> PERP	$\kappa_{\perp}$
<i>kap</i> PARA	$\kappa_{\perp}$
<i>mu</i>	μ
<i>bx</i> 1pt	b_x^1
<i>bx</i> 2pt	b_x^2
<i>bx</i> 3pt	b_x^3
<i>b</i> 1rms	$\langle b_1^2 \rangle^{1/2}$
<i>b</i> 2rms	$\langle b_2^2 \rangle^{1/2}$
<i>b</i> 3rms	$\langle b_3^2 \rangle^{1/2}$

Module ‘testfield_axisym4.f90’

<i>alp</i> PERP	$\alpha_{\perp}$
<i>alp</i> PARA	$\alpha_{\perp}$
<i>gam</i>	γ
<i>bet</i> PERP	$\beta_{\perp}$
<i>bet</i> PERP2	$\beta_{\perp}^{(2)}$
<i>bet</i> PARA	$\beta_{\perp}$
<i>del</i>	δ
<i>del</i> 2	$\delta^{(2)}$
<i>kap</i> PERP	$\kappa_{\perp}$
<i>kap</i> PERP2	$\kappa_{\perp}^{(2)}$
<i>kap</i> PARA	$\kappa_{\perp}$
<i>mu</i>	μ
<i>mu</i> 2	$\mu^{(2)}$
<i>alp</i> PERPz	$\alpha_{\perp}(z)$
<i>alp</i> PARAz	$\alpha_{\perp}(z)$
<i>gam</i> z	$\gamma(z)$
<i>bet</i> PERPz	$\beta_{\perp}(z)$
<i>bet</i> PARAz	$\beta_{\perp}(z)$
<i>del</i> z	$\delta(z)$
<i>kap</i> PERPz	$\kappa_{\perp}(z)$
<i>kap</i> PARAz	$\kappa_{\perp}(z)$
<i>muz</i>	$\mu(z)$
<i>bx</i> 1pt	b_x^1
<i>bx</i> 2pt	b_x^2
<i>bx</i> 3pt	b_x^3
<i>b</i> 1rms	$\langle b_1^2 \rangle^{1/2}$
<i>b</i> 2rms	$\langle b_2^2 \rangle^{1/2}$
<i>b</i> 3rms	$\langle b_3^2 \rangle^{1/2}$

Module ‘testfield_compress_z.f90’

<i>alp</i> 11	α_{11}
<i>alp</i> 21	α_{21}
<i>alp</i> 31	α_{31}
<i>alp</i> 12	α_{12}
<i>alp</i> 22	α_{22}
<i>alp</i> 32	α_{32}

<i>eta11</i>	$\eta_{11}k$
<i>eta21</i>	$\eta_{21}k$
<i>eta12</i>	$\eta_{12}k$
<i>eta22</i>	$\eta_{22}k$
<i>alpK</i>	α^K
<i>alpM</i>	α^M
<i>alpMK</i>	α^{MK}
<i>phi11</i>	ϕ_{11}
<i>phi21</i>	ϕ_{21}
<i>phi12</i>	ϕ_{12}
<i>phi22</i>	ϕ_{22}
<i>phi32</i>	ϕ_{32}
<i>psi11</i>	$\psi_{11}k$
<i>psi21</i>	$\psi_{21}k$
<i>psi12</i>	$\psi_{12}k$
<i>psi22</i>	$\psi_{22}k$
<i>sig1</i>	σ_1
<i>sig2</i>	σ_2
<i>sig3</i>	σ_3
<i>tau1</i>	τ_1
<i>tau2</i>	τ_2
<i>phiK</i>	ϕ^K
<i>phiM</i>	ϕ^M
<i>phiMK</i>	ϕ^{MK}
<i>alp11cc</i>	$\alpha_{11} \cos^2 kz$
<i>alp21sc</i>	$\alpha_{21} \sin kz \cos kz$
<i>alp12cs</i>	$\alpha_{12} \cos kz \sin kz$
<i>alp22ss</i>	$\alpha_{22} \sin^2 kz$
<i>eta11cc</i>	$\eta_{11} \cos^2 kz$
<i>eta21sc</i>	$\eta_{21} \sin kz \cos kz$
<i>eta12cs</i>	$\eta_{12} \cos kz \sin kz$
<i>eta22ss</i>	$\eta_{22} \sin^2 kz$
<i>s2kzDFm</i>	$\langle \sin 2kz \nabla \cdot F \rangle$
<i>M11</i>	$\mathcal{M}_{11}$
<i>M22</i>	$\mathcal{M}_{22}$
<i>M33</i>	$\mathcal{M}_{33}$
<i>M11cc</i>	$\mathcal{M}_{11} \cos^2 kz$
<i>M11ss</i>	$\mathcal{M}_{11} \sin^2 kz$
<i>M22cc</i>	$\mathcal{M}_{22} \cos^2 kz$
<i>M22ss</i>	$\mathcal{M}_{22} \sin^2 kz$
<i>M12cs</i>	$\mathcal{M}_{12} \cos kz \sin kz$
<i>bx11pt</i>	b_x^{11}
<i>bx21pt</i>	b_x^{21}
<i>bx12pt</i>	b_x^{12}
<i>bx22pt</i>	b_x^{22}
<i>bx0pt</i>	b_x^0
<i>by11pt</i>	b_y^{11}
<i>by21pt</i>	b_y^{21}
<i>by12pt</i>	b_y^{12}
<i>by22pt</i>	b_y^{22}

<i>by0pt</i>	b_y^0
<i>u11rms</i>	$\langle u_{11}^2 \rangle^{1/2}$
<i>u21rms</i>	$\langle u_{21}^2 \rangle^{1/2}$
<i>u12rms</i>	$\langle u_{12}^2 \rangle^{1/2}$
<i>u22rms</i>	$\langle u_{22}^2 \rangle^{1/2}$
<i>h11rms</i>	$\langle h_{11}^2 \rangle^{1/2}$
<i>h21rms</i>	$\langle h_{21}^2 \rangle^{1/2}$
<i>h12rms</i>	$\langle h_{12}^2 \rangle^{1/2}$
<i>h22rms</i>	$\langle h_{22}^2 \rangle^{1/2}$
<i>j11rms</i>	$\langle j_{11}^2 \rangle^{1/2}$
<i>b11rms</i>	$\langle b_{11}^2 \rangle^{1/2}$
<i>b21rms</i>	$\langle b_{21}^2 \rangle^{1/2}$
<i>b12rms</i>	$\langle b_{12}^2 \rangle^{1/2}$
<i>b22rms</i>	$\langle b_{22}^2 \rangle^{1/2}$
<i>ux0m</i>	$\langle u_{0x} \rangle$
<i>uy0m</i>	$\langle u_{0y} \rangle$
<i>ux11m</i>	$\langle u_{11x} \rangle$
<i>uy11m</i>	$\langle u_{11y} \rangle$
<i>u0rms</i>	$\langle u_0^2 \rangle^{1/2}$
<i>b0rms</i>	$\langle b_0^2 \rangle^{1/2}$
<i>h0rms</i>	$\langle h_0^2 \rangle^{1/2}$
<i>rho0m</i>	$\langle \exp h_0 \rangle$
<i>u0max</i>	$\max \mathbf{u}_0 $
<i>b0max</i>	$\max \mathbf{b}_0 $
<i>h0max</i>	$\max h_0$
<i>bhrms</i>	$\langle b_h^2 \rangle^{1/2}$
<i>jb0m</i>	$\langle j b_0 \rangle$
<i>E11rms</i>	$\langle \mathcal{E}_{11}^2 \rangle^{1/2}$
<i>E21rms</i>	$\langle \mathcal{E}_{21}^2 \rangle^{1/2}$
<i>E12rms</i>	$\langle \mathcal{E}_{12}^2 \rangle^{1/2}$
<i>E22rms</i>	$\langle \mathcal{E}_{22}^2 \rangle^{1/2}$
<i>E0rms</i>	$\langle \mathcal{E}_0^2 \rangle^{1/2}$
<i>E0mrms</i>	$\langle \mathcal{E}_0^2 \rangle^{1/2}$
<i>E0xrms</i>	$\langle \mathcal{E}_{0,x}^2 \rangle^{1/2}$
<i>E0yrms</i>	$\langle \mathcal{E}_{0,y}^2 \rangle^{1/2}$
<i>Ex11pt</i>	$\mathcal{E}_x^{11}$
<i>Ex21pt</i>	$\mathcal{E}_x^{21}$
<i>Ex12pt</i>	$\mathcal{E}_x^{12}$
<i>Ex22pt</i>	$\mathcal{E}_x^{22}$
<i>Ex0pt</i>	$\mathcal{E}_x^0$
<i>Ey11pt</i>	$\mathcal{E}_y^{11}$
<i>Ey21pt</i>	$\mathcal{E}_y^{21}$
<i>Ey12pt</i>	$\mathcal{E}_y^{12}$
<i>Ey22pt</i>	$\mathcal{E}_y^{22}$
<i>Ey0pt</i>	$\mathcal{E}_y^0$
<i>bamp</i>	bamp

<i>E111z</i>	$\mathcal{E}_1^{11}$
<i>E211z</i>	$\mathcal{E}_2^{11}$
<i>E311z</i>	$\mathcal{E}_3^{11}$
<i>E121z</i>	$\mathcal{E}_1^{21}$
<i>E221z</i>	$\mathcal{E}_2^{21}$
<i>E321z</i>	$\mathcal{E}_3^{21}$
<i>E112z</i>	$\mathcal{E}_1^{12}$
<i>E212z</i>	$\mathcal{E}_2^{12}$
<i>E312z</i>	$\mathcal{E}_3^{12}$
<i>E122z</i>	$\mathcal{E}_1^{22}$
<i>E222z</i>	$\mathcal{E}_2^{22}$
<i>E322z</i>	$\mathcal{E}_3^{22}$
<i>alp11z</i>	$\alpha_{11}(z, t)$
<i>alp21z</i>	$\alpha_{21}(z, t)$
<i>alp12z</i>	$\alpha_{12}(z, t)$
<i>alp22z</i>	$\alpha_{22}(z, t)$
<i>eta11z</i>	$\eta_{11}(z, t)$
<i>eta21z</i>	$\eta_{21}(z, t)$
<i>eta12z</i>	$\eta_{12}(z, t)$
<i>eta22z</i>	$\eta_{22}(z, t)$
<i>E10z</i>	$\mathcal{E}_1^0$
<i>E20z</i>	$\mathcal{E}_2^0$
<i>E30z</i>	$\mathcal{E}_3^0$
<i>EBpq</i>	$\mathcal{E} \cdot \mathbf{B}^{pq}$
<i>E0Um</i>	$\mathcal{E}^0 \cdot \mathbf{U}$
<i>E0Wm</i>	$\mathcal{E}^0 \cdot \mathbf{W}$
<i>bx0mz</i>	$\langle b_x \rangle_{xy}$
<i>by0mz</i>	$\langle b_y \rangle_{xy}$
<i>bz0mz</i>	$\langle b_z \rangle_{xy}$
<i>M11z</i>	$\langle \mathcal{M}_{11} \rangle_{xy}$
<i>M22z</i>	$\langle \mathcal{M}_{22} \rangle_{xy}$
<i>M33z</i>	$\langle \mathcal{M}_{33} \rangle_{xy}$

Module 'testfield_meri.f90'

<i>E11xy</i>	E_{11xy}
<i>E12xy</i>	E_{12xy}
<i>E13xy</i>	E_{13xy}
<i>E21xy</i>	E_{21xy}
<i>E22xy</i>	E_{22xy}
<i>E23xy</i>	E_{23xy}
<i>E31xy</i>	E_{31xy}
<i>E32xy</i>	E_{32xy}
<i>E33xy</i>	E_{33xy}
<i>E41xy</i>	E_{41xy}
<i>E42xy</i>	E_{42xy}
<i>E43xy</i>	E_{43xy}
<i>E51xy</i>	E_{51xy}
<i>E52xy</i>	E_{52xy}
<i>E53xy</i>	E_{53xy}
<i>E61xy</i>	E_{61xy}

<i>E62xy</i>	E_{62xy}
<i>E63xy</i>	E_{63xy}
<i>E71xy</i>	E_{71xy}
<i>E72xy</i>	E_{72xy}
<i>E73xy</i>	E_{73xy}
<i>E81xy</i>	E_{81}
<i>E82xy</i>	E_{82}
<i>E83xy</i>	E_{83}
<i>E91xy</i>	E_{91}
<i>E92xy</i>	E_{92}
<i>E93xy</i>	E_{93}
<i>a11xy</i>	α_{11}
<i>a12xy</i>	α_{12}
<i>a13xy</i>	α_{13}
<i>a21xy</i>	α_{21}
<i>a22xy</i>	α_{22}
<i>a23xy</i>	α_{23}
<i>a31xy</i>	α_{31}
<i>a32xy</i>	α_{32}
<i>a33xy</i>	α_{33}
<i>b111xy</i>	$_{111}$
<i>b121xy</i>	$_{121}$
<i>b131xy</i>	$_{131}$
<i>b211xy</i>	$_{211}$
<i>b221xy</i>	$_{221}$
<i>b231xy</i>	$_{231}$
<i>b311xy</i>	$_{311}$
<i>b321xy</i>	$_{321}$
<i>b331xy</i>	$_{331}$
<i>b112xy</i>	$_{112}$
<i>b122xy</i>	$_{122}$
<i>b132xy</i>	$_{132}$
<i>b212xy</i>	$_{212}$
<i>b222xy</i>	$_{222}$
<i>b232xy</i>	$_{232}$
<i>b312xy</i>	$_{312}$
<i>b322xy</i>	$_{322}$
<i>b332xy</i>	$_{332}$

Module ‘testfield_nonlin_z.f90’

<i>alp11</i>	α_{11}
<i>alp21</i>	α_{21}
<i>alp31</i>	α_{31}
<i>alp12</i>	α_{12}
<i>alp22</i>	α_{22}
<i>alp32</i>	α_{32}
<i>eta11</i>	$\eta_{11}k$
<i>eta21</i>	$\eta_{21}k$
<i>eta12</i>	$\eta_{12}k$
<i>eta22</i>	$\eta_{22}k$

<i>alpK</i>	α^K
<i>alpM</i>	α^M
<i>alpMK</i>	α^{MK}
<i>phi11</i>	ϕ_{11}
<i>phi21</i>	ϕ_{21}
<i>phi12</i>	ϕ_{12}
<i>phi22</i>	ϕ_{22}
<i>phi32</i>	ϕ_{32}
<i>psi11</i>	$\psi_{11}k$
<i>psi21</i>	$\psi_{21}k$
<i>psi12</i>	$\psi_{12}k$
<i>psi22</i>	$\psi_{22}k$
<i>phiK</i>	ϕ^K
<i>phiM</i>	ϕ^M
<i>phiMK</i>	ϕ^{MK}
<i>alp11cc</i>	$\alpha_{11} \cos^2 kz$
<i>alp21sc</i>	$\alpha_{21} \sin kz \cos kz$
<i>alp12cs</i>	$\alpha_{12} \cos kz \sin kz$
<i>alp22ss</i>	$\alpha_{22} \sin^2 kz$
<i>eta11cc</i>	$\eta_{11} \cos^2 kz$
<i>eta21sc</i>	$\eta_{21} \sin kz \cos kz$
<i>eta12cs</i>	$\eta_{12} \cos kz \sin kz$
<i>eta22ss</i>	$\eta_{22} \sin^2 kz$
<i>s2kzDFm</i>	$\langle \sin 2kz \nabla \cdot F \rangle$
<i>M11</i>	$\mathcal{M}_{11}$
<i>M22</i>	$\mathcal{M}_{22}$
<i>M33</i>	$\mathcal{M}_{33}$
<i>M11cc</i>	$\mathcal{M}_{11} \cos^2 kz$
<i>M11ss</i>	$\mathcal{M}_{11} \sin^2 kz$
<i>M22cc</i>	$\mathcal{M}_{22} \cos^2 kz$
<i>M22ss</i>	$\mathcal{M}_{22} \sin^2 kz$
<i>M12cs</i>	$\mathcal{M}_{12} \cos kz \sin kz$
<i>bx11pt</i>	b_x^{11}
<i>bx21pt</i>	b_x^{21}
<i>bx12pt</i>	b_x^{12}
<i>bx22pt</i>	b_x^{22}
<i>bx0pt</i>	b_x^0
<i>by11pt</i>	b_y^{11}
<i>by21pt</i>	b_y^{21}
<i>by12pt</i>	b_y^{12}
<i>by22pt</i>	b_y^{22}
<i>by0pt</i>	b_y^0
<i>u11rms</i>	$\langle u_{11}^2 \rangle^{1/2}$
<i>u21rms</i>	$\langle u_{21}^2 \rangle^{1/2}$
<i>u12rms</i>	$\langle u_{12}^2 \rangle^{1/2}$
<i>u22rms</i>	$\langle u_{22}^2 \rangle^{1/2}$
<i>j11rms</i>	$\langle j_{11}^2 \rangle^{1/2}$
<i>b11rms</i>	$\langle b_{11}^2 \rangle^{1/2}$
<i>b21rms</i>	$\langle b_{21}^2 \rangle^{1/2}$

<i>b12rms</i>	$\langle b_{12}^2 \rangle^{1/2}$
<i>b22rms</i>	$\langle b_{22}^2 \rangle^{1/2}$
<i>ux0m</i>	$\langle u_{0x} \rangle$
<i>uy0m</i>	$\langle u_{0y} \rangle$
<i>ux11m</i>	$\langle u_{11x} \rangle$
<i>uy11m</i>	$\langle u_{11y} \rangle$
<i>u0rms</i>	$\langle u_0^2 \rangle^{1/2}$
<i>b0rms</i>	$\langle b_0^2 \rangle^{1/2}$
<i>u0max</i>	$\max \mathbf{u}_0 $
<i>b0max</i>	$\max \mathbf{b}_0 $
<i>jb0m</i>	$\langle j_0 \cdot b_0 \rangle$
<i>ub0m</i>	$\langle u_0 \cdot b_0 \rangle$
<i>uj0m</i>	$\langle u_0 \cdot j_0 \rangle$
<i>E11rms</i>	$\langle \mathcal{E}_{11}^2 \rangle^{1/2}$
<i>E21rms</i>	$\langle \mathcal{E}_{21}^2 \rangle^{1/2}$
<i>E12rms</i>	$\langle \mathcal{E}_{12}^2 \rangle^{1/2}$
<i>E22rms</i>	$\langle \mathcal{E}_{22}^2 \rangle^{1/2}$
<i>E0rms</i>	$\langle \mathcal{E}_0^2 \rangle^{1/2}$
<i>Ex11pt</i>	$\mathcal{E}_x^{11}$
<i>Ex21pt</i>	$\mathcal{E}_x^{21}$
<i>Ex12pt</i>	$\mathcal{E}_x^{12}$
<i>Ex22pt</i>	$\mathcal{E}_x^{22}$
<i>Ex0pt</i>	$\mathcal{E}_x^0$
<i>Ey11pt</i>	$\mathcal{E}_y^{11}$
<i>Ey21pt</i>	$\mathcal{E}_y^{21}$
<i>Ey12pt</i>	$\mathcal{E}_y^{12}$
<i>Ey22pt</i>	$\mathcal{E}_y^{22}$
<i>Ey0pt</i>	$\mathcal{E}_y^0$
<i>bamp</i>	bamp
<i>E111z</i>	$\mathcal{E}_1^{11}$
<i>E211z</i>	$\mathcal{E}_2^{11}$
<i>E311z</i>	$\mathcal{E}_3^{11}$
<i>E121z</i>	$\mathcal{E}_1^{21}$
<i>E221z</i>	$\mathcal{E}_2^{21}$
<i>E321z</i>	$\mathcal{E}_3^{21}$
<i>E112z</i>	$\mathcal{E}_1^{12}$
<i>E212z</i>	$\mathcal{E}_2^{12}$
<i>E312z</i>	$\mathcal{E}_3^{12}$
<i>E122z</i>	$\mathcal{E}_1^{22}$
<i>E222z</i>	$\mathcal{E}_2^{22}$
<i>E322z</i>	$\mathcal{E}_3^{22}$
<i>E10z</i>	$\mathcal{E}_1^0$
<i>E20z</i>	$\mathcal{E}_2^0$
<i>E30z</i>	$\mathcal{E}_3^0$
<i>EBpq</i>	$\mathcal{E} \cdot \mathbf{B}^{pq}$
<i>E0Um</i>	$\mathcal{E}^0 \cdot \mathbf{U}$
<i>E0Wm</i>	$\mathcal{E}^0 \cdot \mathbf{W}$
<i>bx0mz</i>	$\langle b_x \rangle_{xy}$
<i>by0mz</i>	$\langle b_y \rangle_{xy}$

<i>bz0mz</i>	$\langle b_z \rangle_{xy}$
<i>M11z</i>	$\langle \mathcal{M}_{11} \rangle_{xy}$
<i>M22z</i>	$\langle \mathcal{M}_{22} \rangle_{xy}$
<i>M33z</i>	$\langle \mathcal{M}_{33} \rangle_{xy}$

Module ‘testfield_x.f90’

<i>alp11</i>	α_{11}
<i>alp21</i>	α_{21}
<i>alp31</i>	α_{31}
<i>alp12</i>	α_{12}
<i>alp22</i>	α_{22}
<i>alp32</i>	α_{32}
<i>eta11</i>	$\eta_{11}k$
<i>eta21</i>	$\eta_{21}k$
<i>eta12</i>	$\eta_{12}k$
<i>eta22</i>	$\eta_{22}k$
<i>alp11cc</i>	$\alpha_{11} \cos^2 kx$
<i>alp21sc</i>	$\alpha_{21} \sin kx \cos kx$
<i>alp12cs</i>	$\alpha_{12} \cos kx \sin kx$
<i>alp22ss</i>	$\alpha_{22} \sin^2 kx$
<i>eta11cc</i>	$\eta_{11} \cos^2 kx$
<i>eta21sc</i>	$\eta_{21} \sin kx \cos kx$
<i>eta12cs</i>	$\eta_{12} \cos kx \sin kx$
<i>eta22ss</i>	$\eta_{22} \sin^2 kx$
<i>alp11_x</i>	$\alpha_{11}x$
<i>alp21_x</i>	$\alpha_{21}x$
<i>alp12_x</i>	$\alpha_{12}x$
<i>alp22_x</i>	$\alpha_{22}x$
<i>eta11_x</i>	$\eta_{11}kx$
<i>eta21_x</i>	$\eta_{21}kx$
<i>eta12_x</i>	$\eta_{12}kx$
<i>eta22_x</i>	$\eta_{22}kx$
<i>alp11_x2</i>	$\alpha_{11}x^2$
<i>alp21_x2</i>	$\alpha_{21}x^2$
<i>alp12_x2</i>	$\alpha_{12}x^2$
<i>alp22_x2</i>	$\alpha_{22}x^2$
<i>eta11_x2</i>	$\eta_{11}kx^2$
<i>eta21_x2</i>	$\eta_{21}kx^2$
<i>eta12_x2</i>	$\eta_{12}kx^2$
<i>eta22_x2</i>	$\eta_{22}kx^2$
<i>b11rms</i>	$\langle b_{11}^2 \rangle^{1/2}$
<i>b21rms</i>	$\langle b_{21}^2 \rangle^{1/2}$
<i>b12rms</i>	$\langle b_{12}^2 \rangle^{1/2}$
<i>b22rms</i>	$\langle b_{22}^2 \rangle^{1/2}$
<i>b0rms</i>	$\langle b_0^2 \rangle^{1/2}$
<i>E11rms</i>	$\langle \mathcal{E}_{11}^2 \rangle^{1/2}$
<i>E21rms</i>	$\langle \mathcal{E}_{21}^2 \rangle^{1/2}$
<i>E12rms</i>	$\langle \mathcal{E}_{12}^2 \rangle^{1/2}$
<i>E22rms</i>	$\langle \mathcal{E}_{22}^2 \rangle^{1/2}$

<i>E0rms</i>	$\langle \mathcal{E}_0^2 \rangle^{1/2}$
<i>E111z</i>	$\mathcal{E}_1^{11}$
<i>E211z</i>	$\mathcal{E}_2^{11}$
<i>E311z</i>	$\mathcal{E}_3^{11}$
<i>E121z</i>	$\mathcal{E}_1^{21}$
<i>E221z</i>	$\mathcal{E}_2^{21}$
<i>E321z</i>	$\mathcal{E}_3^{21}$
<i>E112z</i>	$\mathcal{E}_1^{12}$
<i>E212z</i>	$\mathcal{E}_2^{12}$
<i>E312z</i>	$\mathcal{E}_3^{12}$
<i>E122z</i>	$\mathcal{E}_1^{22}$
<i>E222z</i>	$\mathcal{E}_2^{22}$
<i>E322z</i>	$\mathcal{E}_3^{22}$
<i>E10z</i>	$\mathcal{E}_1^0$
<i>E20z</i>	$\mathcal{E}_2^0$
<i>E30z</i>	$\mathcal{E}_3^0$
<i>EBpq</i>	$\mathcal{E} \cdot \mathbf{B}^{pq}$
<i>bx0mz</i>	$\langle b_x \rangle_{xy}$
<i>by0mz</i>	$\langle b_y \rangle_{xy}$
<i>bz0mz</i>	$\langle b_z \rangle_{xy}$
<i>alp11x</i>	$\alpha_{11}(x, t)$
<i>alp21x</i>	$\alpha_{21}(x, t)$
<i>alp12x</i>	$\alpha_{12}(x, t)$
<i>alp22x</i>	$\alpha_{22}(x, t)$
<i>eta11x</i>	$\eta_{11}(x, t)$
<i>eta21x</i>	$\eta_{21}(x, t)$
<i>eta12x</i>	$\eta_{12}(x, t)$
<i>eta22x</i>	$\eta_{22}(x, t)$

Module ‘testfield_xz.f90’

<i>E111z</i>	$\mathcal{E}_1^{11}$
<i>E211z</i>	$\mathcal{E}_2^{11}$
<i>E311z</i>	$\mathcal{E}_3^{11}$
<i>E121z</i>	$\mathcal{E}_1^{21}$
<i>E221z</i>	$\mathcal{E}_2^{21}$
<i>E321z</i>	$\mathcal{E}_3^{21}$
<i>alp11</i>	α_{11}
<i>alp21</i>	α_{21}
<i>eta11</i>	$\eta_{113}k$
<i>eta21</i>	$\eta_{213}k$
<i>b11rms</i>	$\langle b_{11}^2 \rangle$
<i>b21rms</i>	$\langle b_{21}^2 \rangle$

Module ‘testfield_z.f90’

<i>alp11</i>	α_{11}
<i>alp21</i>	α_{21}
<i>alp31</i>	α_{31}
<i>alp12</i>	α_{12}
<i>alp22</i>	α_{22}
<i>alp32</i>	α_{32}

<i>alp13</i>	α_{13}
<i>alp23</i>	α_{23}
<i>eta11</i>	$\eta_{113}k$ or $\eta_{11}k$ if <i>leta_rank2</i> =T
<i>eta21</i>	$\eta_{213}k$ or $\eta_{21}k$ if <i>leta_rank2</i> =T
<i>eta31</i>	$\eta_{313}k$
<i>eta12</i>	$\eta_{123}k$ or $\eta_{12}k$ if <i>leta_rank2</i> =T
<i>eta22</i>	$\eta_{223}k$ or $\eta_{22}k$ if <i>leta_rank2</i> =T
<i>eta32</i>	$\eta_{323}k$
<i>alp11cc</i>	$\alpha_{11} \cos^2 kz$
<i>alp21sc</i>	$\alpha_{21} \sin kz \cos kz$
<i>alp12cs</i>	$\alpha_{12} \cos kz \sin kz$
<i>alp22ss</i>	$\alpha_{22} \sin^2 kz$
<i>eta11cc</i>	$\eta_{11} \cos^2 kz$
<i>eta21sc</i>	$\eta_{21} \sin kz \cos kz$
<i>eta12cs</i>	$\eta_{12} \cos kz \sin kz$
<i>eta22ss</i>	$\eta_{22} \sin^2 kz$
<i>s2kzDFm</i>	$\langle \sin 2kz \nabla \cdot F \rangle$
<i>M11</i>	$\mathcal{M}_{11}$
<i>M22</i>	$\mathcal{M}_{22}$
<i>M33</i>	$\mathcal{M}_{33}$
<i>M11cc</i>	$\mathcal{M}_{11} \cos^2 kz$
<i>M11ss</i>	$\mathcal{M}_{11} \sin^2 kz$
<i>M22cc</i>	$\mathcal{M}_{22} \cos^2 kz$
<i>M22ss</i>	$\mathcal{M}_{22} \sin^2 kz$
<i>M12cs</i>	$\mathcal{M}_{12} \cos kz \sin kz$
<i>bx11pt</i>	b_x^{11}
<i>bx21pt</i>	b_x^{21}
<i>bx12pt</i>	b_x^{12}
<i>bx22pt</i>	b_x^{22}
<i>bx0pt</i>	b_x^0
<i>by11pt</i>	b_y^{11}
<i>by21pt</i>	b_y^{21}
<i>by12pt</i>	b_y^{12}
<i>by22pt</i>	b_y^{22}
<i>by0pt</i>	b_y^0
<i>b11rms</i>	$\langle b_{11}^2 \rangle^{1/2}$
<i>b21rms</i>	$\langle b_{21}^2 \rangle^{1/2}$
<i>b12rms</i>	$\langle b_{12}^2 \rangle^{1/2}$
<i>b22rms</i>	$\langle b_{22}^2 \rangle^{1/2}$
<i>b0rms</i>	$\langle b_0^2 \rangle^{1/2}$
<i>jb0m</i>	$\langle j b_0 \rangle$
<i>E11rms</i>	$\langle \mathcal{E}_{11}^2 \rangle^{1/2}$
<i>E21rms</i>	$\langle \mathcal{E}_{21}^2 \rangle^{1/2}$
<i>E12rms</i>	$\langle \mathcal{E}_{12}^2 \rangle^{1/2}$
<i>E22rms</i>	$\langle \mathcal{E}_{22}^2 \rangle^{1/2}$
<i>E0rms</i>	$\langle \mathcal{E}_0^2 \rangle^{1/2}$
<i>Ex11pt</i>	$\mathcal{E}_x^{11}$
<i>Ex21pt</i>	$\mathcal{E}_x^{21}$
<i>Ex12pt</i>	$\mathcal{E}_x^{12}$

<i>Ex22pt</i>	$\mathcal{E}_x^{22}$
<i>Ex0pt</i>	$\mathcal{E}_x^0$
<i>Ey11pt</i>	$\mathcal{E}_y^{11}$
<i>Ey21pt</i>	$\mathcal{E}_y^{21}$
<i>Ey12pt</i>	$\mathcal{E}_y^{12}$
<i>Ey22pt</i>	$\mathcal{E}_y^{22}$
<i>Ey0pt</i>	$\mathcal{E}_y^0$
<i>bamp</i>	bamp
<i>alp11z</i>	$\alpha_{11}(z, t)$
<i>alp21z</i>	$\alpha_{21}(z, t)$
<i>alp12z</i>	$\alpha_{12}(z, t)$
<i>alp22z</i>	$\alpha_{22}(z, t)$
<i>alp13z</i>	$\alpha_{13}(z, t)$
<i>alp23z</i>	$\alpha_{23}(z, t)$
<i>eta11z</i>	$\eta_{11}(z, t)$
<i>eta21z</i>	$\eta_{21}(z, t)$
<i>eta12z</i>	$\eta_{12}(z, t)$
<i>eta22z</i>	$\eta_{22}(z, t)$
<i>uzjx1z</i>	$u_z j_x^{11}$
<i>uzjy1z</i>	$u_z j_y^{11}$
<i>uzjz1z</i>	$u_z j_z^{11}$
<i>uzjx2z</i>	$u_z j_x^{21}$
<i>uzjy2z</i>	$u_z j_y^{21}$
<i>uzjz2z</i>	$u_z j_z^{21}$
<i>uzjx3z</i>	$u_z j_x^{12}$
<i>uzjy3z</i>	$u_z j_y^{12}$
<i>uzjz3z</i>	$u_z j_z^{12}$
<i>uzjx4z</i>	$u_z j_x^{22}$
<i>uzjy4z</i>	$u_z j_y^{22}$
<i>uzjz4z</i>	$u_z j_z^{22}$
<i>E111z</i>	$\mathcal{E}_1^{11}$
<i>E211z</i>	$\mathcal{E}_2^{11}$
<i>E311z</i>	$\mathcal{E}_3^{11}$
<i>E121z</i>	$\mathcal{E}_1^{21}$
<i>E221z</i>	$\mathcal{E}_2^{21}$
<i>E321z</i>	$\mathcal{E}_3^{21}$
<i>E112z</i>	$\mathcal{E}_1^{12}$
<i>E212z</i>	$\mathcal{E}_2^{12}$
<i>E312z</i>	$\mathcal{E}_3^{12}$
<i>E122z</i>	$\mathcal{E}_1^{22}$
<i>E222z</i>	$\mathcal{E}_2^{22}$
<i>E322z</i>	$\mathcal{E}_3^{22}$
<i>E10z</i>	$\mathcal{E}_1^0$
<i>E20z</i>	$\mathcal{E}_2^0$
<i>E30z</i>	$\mathcal{E}_3^0$
<i>EBpq</i>	$\mathcal{E} \cdot B^{pq}$
<i>E0Um</i>	$\mathcal{E}^0 \cdot U$
<i>E0Wm</i>	$\mathcal{E}^0 \cdot W$
<i>bx0mz</i>	$\langle b_x \rangle_{xy}$
<i>by0mz</i>	$\langle b_y \rangle_{xy}$

<i>bz0mz</i>	$\langle b_z \rangle_{xy}$
<i>M11z</i>	$\langle \mathcal{M}_{11} \rangle_{xy}$
<i>M22z</i>	$\langle \mathcal{M}_{22} \rangle_{xy}$
<i>M33z</i>	$\langle \mathcal{M}_{33} \rangle_{xy}$
Module ‘testflow_z.f90’	
<i>gal</i>	GAL-coefficients, couple $\bar{F}$ and $\bar{U}$
<i>aklam</i>	AKA- λ -tensor, couples $\bar{F}$ and $\bar{W} = \nabla \times \bar{U}$
<i>gamma</i>	γ -vector, couples $\bar{F}$ and $\nabla \cdot \bar{U}$
<i>nu</i>	ν -tensor, couples $\bar{F}$ and $\partial^2 \bar{U} / \partial z^2$
<i>zeta</i>	ζ -vector, couples $\bar{F}$ and $\bar{G}_z = \nabla_z \bar{H}$
<i>xi</i>	ξ -vector, couples $\bar{F}$ and $\partial^2 \bar{H} / \partial z^2$
<i>aklamQ</i>	$aklam^Q$ -vector, couples $\bar{Q}$ and $\bar{W}$
<i>gammaQ</i>	γ^Q -scalar, couples $\bar{Q}$ and $\nabla \cdot \bar{U} = dU_z/dz$
<i>nuQ</i>	ν^Q -vector, couples $\bar{Q}$ and $\partial^2 \bar{U} / \partial z^2$
<i>zetaQ</i>	ζ^Q -scalar, couples $\bar{Q}$ and $\bar{G}_z$
<i>xiQ</i>	ξ^Q -scalar, couples $\bar{Q}$ and $\partial^2 \bar{H} / \partial z^2$
<i>ux0mz</i>	$\alpha_{K,ij} \gamma_i \nu_{ij} \zeta_i \xi_i \nu_i^Q aklam_i^Q \mathcal{F}_i^{pq} \mathcal{Q}^{pq} \langle u^{pq2} \rangle \langle h^{pq2} \rangle$
<i>uy0mz</i>	$\langle u_x \rangle_{xy}$
<i>uz0mz</i>	$\langle u_y \rangle_{xy}$
	$\langle u_z \rangle_{xy}$
Module ‘testperturb.f90’	
<i>alp11</i>	α_{11}
<i>alp21</i>	α_{21}
<i>alp31</i>	α_{31}
<i>alp12</i>	α_{12}
<i>alp22</i>	α_{22}
<i>alp32</i>	α_{32}
<i>eta11</i>	$\eta_{113}k$
<i>eta21</i>	$\eta_{213}k$
<i>eta31</i>	$\eta_{313}k$
<i>eta12</i>	$\eta_{123}k$
<i>eta22</i>	$\eta_{223}k$
<i>eta32</i>	$\eta_{323}k$
Module ‘testscalar.f90’	
<i>gam11</i>	$\gamma_1^{(1)}$
<i>gam12</i>	$\gamma_2^{(1)}$
<i>gam13</i>	$\gamma_3^{(1)}$
<i>gam21</i>	$\gamma_1^{(2)}$
<i>gam22</i>	$\gamma_2^{(2)}$
<i>gam23</i>	$\gamma_3^{(2)}$
<i>gam31</i>	$\gamma_1^{(3)}$
<i>gam32</i>	$\gamma_2^{(3)}$
<i>gam33</i>	$\gamma_3^{(3)}$
<i>kap11</i>	κ_{11}
<i>kap21</i>	κ_{21}
<i>kap31</i>	κ_{31}

<i>kap12</i>	κ_{12}
<i>kap22</i>	κ_{22}
<i>kap32</i>	κ_{32}
<i>kap13</i>	κ_{13}
<i>kap23</i>	κ_{23}
<i>kap33</i>	κ_{33}
<i>gam11z</i>	$\gamma_1^{(1)}(z, t)$
<i>gam12z</i>	$\gamma_2^{(1)}(z, t)$
<i>gam13z</i>	$\gamma_3^{(1)}(z, t)$
<i>gam21z</i>	$\gamma_1^{(2)}(z, t)$
<i>gam22z</i>	$\gamma_2^{(2)}(z, t)$
<i>gam23z</i>	$\gamma_3^{(2)}(z, t)$
<i>gam31z</i>	$\gamma_1^{(3)}(z, t)$
<i>gam32z</i>	$\gamma_2^{(3)}(z, t)$
<i>gam33z</i>	$\gamma_3^{(3)}(z, t)$
<i>kap11z</i>	$\kappa_{11}(z, t)$
<i>kap21z</i>	$\kappa_{21}(z, t)$
<i>kap31z</i>	$\kappa_{31}(z, t)$
<i>kap12z</i>	$\kappa_{12}(z, t)$
<i>kap22z</i>	$\kappa_{22}(z, t)$
<i>kap32z</i>	$\kappa_{32}(z, t)$
<i>kap13z</i>	$\kappa_{13}(z, t)$
<i>kap23z</i>	$\kappa_{23}(z, t)$
<i>kap33z</i>	$\kappa_{33}(z, t)$
<i>mgam33</i>	$\tilde{\gamma}_{33}$
<i>mkap33</i>	$\tilde{\kappa}_{33}$
<i>ngam33</i>	$\hat{\gamma}_{33}$
<i>nkap33</i>	$\hat{\kappa}_{33}$
<i>c1rms</i>	$\langle c_1^2 \rangle^{1/2}$
<i>c2rms</i>	$\langle c_2^2 \rangle^{1/2}$
<i>c3rms</i>	$\langle c_3^2 \rangle^{1/2}$
<i>c4rms</i>	$\langle c_4^2 \rangle^{1/2}$
<i>c5rms</i>	$\langle c_5^2 \rangle^{1/2}$
<i>c6rms</i>	$\langle c_6^2 \rangle^{1/2}$
<i>c1pt</i>	c^1
<i>c2pt</i>	c^2
<i>c3pt</i>	c^3
<i>c4pt</i>	c^4
<i>c5pt</i>	c^5
<i>c6pt</i>	c^6
<i>F11z</i>	$\mathcal{F}_1^1$
<i>F21z</i>	$\mathcal{F}_2^1$
<i>F31z</i>	$\mathcal{F}_3^1$
<i>F12z</i>	$\mathcal{F}_1^2$
<i>F22z</i>	$\mathcal{F}_2^2$
<i>F32z</i>	$\mathcal{F}_3^2$

<i>muc1</i>	$\mu^{(c1)}$
<i>muc2</i>	$\mu^{(c2)}$
<i>gamc</i>	$\gamma^{(c)}$
<i>kapcPERP1</i>	$\kappa_{\perp}^{(1)}$
<i>kapcPERP2</i>	$\kappa_{\perp}^{(2)}$
<i>kapcPARA</i>	$\kappa_{\parallel}$
<i>mucz</i>	$\mu^{(c)}(z, t)$
<i>gamcz</i>	$\gamma^{(c)}(z, t)$
<i>kapcPERPz</i>	$\kappa_{\perp}(z, t)$
<i>kapcPARAz</i>	$\kappa_{\parallel}(z, t)$
<i>gam11</i>	$\gamma_1^{(1)}$
<i>gam12</i>	$\gamma_2^{(1)}$
<i>gam13</i>	$\gamma_3^{(1)}$
<i>gam21</i>	$\gamma_1^{(2)}$
<i>gam22</i>	$\gamma_2^{(2)}$
<i>gam23</i>	$\gamma_3^{(2)}$
<i>gam31</i>	$\gamma_1^{(3)}$
<i>gam32</i>	$\gamma_2^{(3)}$
<i>gam33</i>	$\gamma_3^{(3)}$
<i>kap11</i>	κ_{11}
<i>kap21</i>	κ_{21}
<i>kap31</i>	κ_{31}
<i>kap12</i>	κ_{12}
<i>kap22</i>	κ_{22}
<i>kap32</i>	κ_{32}
<i>kap13</i>	κ_{13}
<i>kap23</i>	κ_{23}
<i>kap33</i>	κ_{33}
<i>gam11z</i>	$\gamma_1^{(1)}(z, t)$
<i>gam12z</i>	$\gamma_2^{(1)}(z, t)$
<i>gam13z</i>	$\gamma_3^{(1)}(z, t)$
<i>gam21z</i>	$\gamma_1^{(2)}(z, t)$
<i>gam22z</i>	$\gamma_2^{(2)}(z, t)$
<i>gam23z</i>	$\gamma_3^{(2)}(z, t)$
<i>gam31z</i>	$\gamma_1^{(3)}(z, t)$
<i>gam32z</i>	$\gamma_2^{(3)}(z, t)$
<i>gam33z</i>	$\gamma_3^{(3)}(z, t)$
<i>gam3z</i>	$\gamma^{(c)}(z, t)$
<i>kap11z</i>	$\kappa_{11}(z, t)$
<i>kap21z</i>	$\kappa_{21}(z, t)$
<i>kap31z</i>	$\kappa_{31}(z, t)$
<i>kap12z</i>	$\kappa_{12}(z, t)$
<i>kap22z</i>	$\kappa_{22}(z, t)$
<i>kap32z</i>	$\kappa_{32}(z, t)$
<i>kap13z</i>	$\kappa_{13}(z, t)$
<i>kap23z</i>	$\kappa_{23}(z, t)$
<i>kap33z</i>	$\kappa_{33}(z, t)$

<i>mgam33</i>	$\tilde{\gamma}_{33}$
<i>mkap33</i>	$\tilde{\kappa}_{33}$
<i>ngam33</i>	$\hat{\gamma}_{33}$
<i>nkap33</i>	$\hat{\kappa}_{33}$
<i>c1rms</i>	$\langle c_1^2 \rangle^{1/2}$
<i>c2rms</i>	$\langle c_2^2 \rangle^{1/2}$
<i>c3rms</i>	$\langle c_3^2 \rangle^{1/2}$
<i>c4rms</i>	$\langle c_4^2 \rangle^{1/2}$
<i>c5rms</i>	$\langle c_5^2 \rangle^{1/2}$
<i>c6rms</i>	$\langle c_6^2 \rangle^{1/2}$
<i>c1pt</i>	c^1
<i>c2pt</i>	c^2
<i>c3pt</i>	c^3
<i>c4pt</i>	c^4
<i>c5pt</i>	c^5
<i>c6pt</i>	c^6
<i>F11z</i>	$\mathcal{F}_1^1$
<i>F21z</i>	$\mathcal{F}_2^1$
<i>F31z</i>	$\mathcal{F}_3^1$
<i>F12z</i>	$\mathcal{F}_1^2$
<i>F22z</i>	$\mathcal{F}_2^2$
<i>F32z</i>	$\mathcal{F}_3^2$

Module ‘testscalar_simple.f90’

<i>gam11</i>	$\gamma_1^{(1)}$
<i>gam12</i>	$\gamma_2^{(1)}$
<i>gam13</i>	$\gamma_3^{(1)}$
<i>gam21</i>	$\gamma_1^{(2)}$
<i>gam22</i>	$\gamma_2^{(2)}$
<i>gam23</i>	$\gamma_3^{(2)}$
<i>gam31</i>	$\gamma_1^{(3)}$
<i>gam32</i>	$\gamma_2^{(3)}$
<i>gam33</i>	$\gamma_3^{(3)}$
<i>kap11</i>	κ_{11}
<i>kap21</i>	κ_{21}
<i>kap31</i>	κ_{31}
<i>kap12</i>	κ_{12}
<i>kap22</i>	κ_{22}
<i>kap32</i>	κ_{32}
<i>kap13</i>	κ_{13}
<i>kap23</i>	κ_{23}
<i>kap33</i>	κ_{33}
<i>gam11z</i>	$\gamma_1^{(1)}(z, t)$
<i>gam12z</i>	$\gamma_2^{(1)}(z, t)$
<i>gam13z</i>	$\gamma_3^{(1)}(z, t)$
<i>gam21z</i>	$\gamma_1^{(2)}(z, t)$
<i>gam22z</i>	$\gamma_2^{(2)}(z, t)$

<i>gam23z</i>	$\gamma_3^{(2)}(z, t)$
<i>gam31z</i>	$\gamma_1^{(3)}(z, t)$
<i>gam32z</i>	$\gamma_2^{(3)}(z, t)$
<i>gam33z</i>	$\gamma_3^{(3)}(z, t)$
<i>kap11z</i>	$\kappa_{11}(z, t)$
<i>kap21z</i>	$\kappa_{21}(z, t)$
<i>kap31z</i>	$\kappa_{31}(z, t)$
<i>kap12z</i>	$\kappa_{12}(z, t)$
<i>kap22z</i>	$\kappa_{22}(z, t)$
<i>kap32z</i>	$\kappa_{32}(z, t)$
<i>kap13z</i>	$\kappa_{13}(z, t)$
<i>kap23z</i>	$\kappa_{23}(z, t)$
<i>kap33z</i>	$\kappa_{33}(z, t)$
<i>mgam33</i>	$\tilde{\gamma}_{33}$
<i>mkap33</i>	$\tilde{\kappa}_{33}$
<i>ngam33</i>	$\hat{\gamma}_{33}$
<i>nkap33</i>	$\hat{\kappa}_{33}$
<i>c1rms</i>	$\langle c_1^2 \rangle^{1/2}$
<i>c2rms</i>	$\langle c_2^2 \rangle^{1/2}$
<i>c3rms</i>	$\langle c_3^2 \rangle^{1/2}$
<i>c4rms</i>	$\langle c_4^2 \rangle^{1/2}$
<i>c5rms</i>	$\langle c_5^2 \rangle^{1/2}$
<i>c6rms</i>	$\langle c_6^2 \rangle^{1/2}$
<i>c1pt</i>	c^1
<i>c2pt</i>	c^2
<i>c3pt</i>	c^3
<i>c4pt</i>	c^4
<i>c5pt</i>	c^5
<i>c6pt</i>	c^6
<i>F11z</i>	$\mathcal{F}_1^1$
<i>F21z</i>	$\mathcal{F}_2^1$
<i>F31z</i>	$\mathcal{F}_3^1$
<i>F12z</i>	$\mathcal{F}_1^2$
<i>F22z</i>	$\mathcal{F}_2^2$
<i>F32z</i>	$\mathcal{F}_3^2$

Module ‘thermal_energy.f90’

<i>TTmax</i>	$\max(T)$
<i>TTmin</i>	$\min(T)$
<i>ppm</i>	$\langle p \rangle$
<i>TTm</i>	$\langle T \rangle$
<i>ethm</i>	$\langle e_{\text{th}} \rangle = \langle c_v \rho T \rangle$ (mean thermal energy)
<i>ethtot</i>	$\int_V e_{\text{th}} dV$ (total thermal energy)
<i>ethmin</i>	$\min e_{\text{th}}$
<i>ethmax</i>	$\max e_{\text{th}}$
<i>eem</i>	$\langle e \rangle = \langle c_v T \rangle$ (mean internal energy)
<i>etot</i>	$\langle e_{\text{th}} + \rho u^2/2 \rangle$

Module ‘training_torchfort.f90’

<i>loss</i>	torchfort training loss
<i>tauerror</i>	$\sqrt{\langle (\sum_{i,j} u_i * u_j - \tau u_{ij})^2 \rangle}$
Module ‘viscosity.f90’	
<i>nu_tdep</i>	time-dependent viscosity
<i>ell_gam</i>	time-dependent photon mean free path
<i>fviscm</i>	Mean value of viscous acceleration
<i>fviscmmin</i>	Min value of viscous acceleration (redundant)
<i>fviscmmax</i>	Max absolute viscous acceleration
<i>fviscrmsx</i>	Rms value of viscous acceleration for the vis_xaver_range
<i>num</i>	Mean value of viscosity
<i>numax</i>	Max value of viscosity
<i>numin</i>	Min value of viscosity
<i>nusmagm</i>	Mean value of Smagorinsky viscosity
<i>nusmagmin</i>	Min value of Smagorinsky viscosity
<i>nusmagmax</i>	Max value of Smagorinsky viscosity
<i>nu_LES</i>	Mean value of Smagorinsky viscosity
<i>visc_heatm</i>	Mean value of viscous heating
<i>qfviscm</i>	$\langle \mathbf{q} \cdot \mathbf{f}_{\text{visc}} \rangle$
<i>ufviscm</i>	$\langle \mathbf{u} \cdot \mathbf{f}_{\text{visc}} \rangle$
<i>Sij2m</i>	$\langle \mathbf{S}^2 \rangle$
<i>epsK</i>	$\langle 2\nu\rho\mathbf{S}^2 \rangle$
<i>epsK2</i>	$\langle (2\nu\rho\mathbf{S}^2)^2 \rangle$
<i>epsK3</i>	$\langle (2\nu\rho\mathbf{S}^2)^3 \rangle$
<i>epsK4</i>	$\langle (2\nu\rho\mathbf{S}^2)^4 \rangle$
<i>epsKint</i>	$\int (2\nu\rho\mathbf{S}^2) dV$
<i>sijoiojm</i>	$\langle S_{i,j}\omega_i\omega_j \rangle$
<i>dtnu</i>	$\delta t / [c_{\delta t,v} \delta x^2 / \nu_{\text{max}}]$ (time step relative to viscous time step; see § 5.16)
<i>meshRemax</i>	Max mesh Reynolds number
<i>mesh3Remax</i>	Max hyper3 mesh Reynolds number
<i>Reshock</i>	Mesh Reynolds number at shock

K.5 List of parameters for ‘video.in’

The following table lists all (at the time of writing, July 23, 2026) possible inputs to the file ‘video.in’.

Variable	Meaning
Module ‘hydro.f90’	
<i>uu</i>	velocity vector $\mathbf{u}$; writes all three components separately to files ‘u[xyz].{xz,yz,xy,xy2}’
<i>u2</i>	kinetic energy density $\mathbf{u}^2$; writes ‘u2.{xz,yz,xy,xy2}’
<i>oo</i>	vorticity vector $\boldsymbol{\omega} = \nabla \times \mathbf{u}$; writes all three components separately to files ‘oo[xyz].{xz,yz,xy,xy2}’
<i>o2</i>	enstrophy $\omega^2 = \nabla \times \mathbf{u} ^2$; writes ‘o2.{xz,yz,xy,xy2}’
<i>divu</i>	$\nabla \cdot \mathbf{u}$; writes ‘divu.{xz,yz,xy,xy2}’

<i>mach</i>	Mach number squared Ma^2 ; writes ‘mach.{xz,yz,xy,xy2}’
Module ‘density.f90’	
<i>lnrho</i>	logarithmic density $\ln \rho$; writes ‘lnrho.{xz,yz,xy,xy2}’
<i>rho</i>	density ρ ; writes ‘rho.{xz,yz,xy,xy2}’
Module ‘entropy.f90’	
<i>ss</i>	entropy s ; writes ‘ss.{xz,yz,xy,xy2}’
<i>pp</i>	pressure p ; writes ‘pp.{xz,yz,xy,xy2}’
Module ‘temperature_idealgas.f90’	
<i>lnTT</i>	logarithmic temperature $\ln T$; writes ‘lnTT.{xz,yz,xy,xy2}’
<i>TT</i>	temperature T ; writes ‘TT.{xz,yz,xy,xy2}’
Module ‘shock.f90’	
<i>shock</i>	shock viscosity ν_{shock} ; writes ‘shock.{xz,yz,xy,xy2}’
Module ‘eos_ionization.f90’	
<i>yH</i>	ionization fraction y_{H} ; writes ‘yH.{xz,yz,xy,xy2}’
Module ‘radiation_ray.f90’	
<i>Qrad</i>	radiative heating rate Q_{rad} ; writes ‘Qrad.{xz,yz,xy,xy2}’
<i>Isurf</i>	surface intensity I_{surf} (?); writes ‘Isurf.xz’
Module ‘magnetic.f90’	
<i>aa</i>	magnetic vector potential $\mathbf{A}$; writes ‘aa[xyz].{xz,yz,xy,xy2}’
<i>bb</i>	magnetic flux density $\mathbf{B}$; writes ‘bb[xyz].{xz,yz,xy,xy2}’
<i>b2</i>	magnetic energy density B^2 ; writes ‘b2.{xz,yz,xy,xy2}’
<i>jj</i>	current density $\mathbf{j}$; writes ‘jj[xyz].{xz,yz,xy,xy2}’
<i>j2</i>	current density squared j^2 ; writes ‘j2.{xz,yz,xy,xy2}’
<i>jb</i>	$\mathbf{jB}$; writes ‘jb.{xz,yz,xy,xy2}’
<i>beta1</i>	inverse plasma beta $B^2/(2\mu_0 p)$; writes ‘beta1.{xz,yz,xy,xy2}’
<i>Poynting</i>	Poynting vector $\eta \mathbf{j} \times \mathbf{B} - (\mathbf{u} \times \mathbf{B}) \times \mathbf{B}/\mu_0$; writes ‘Poynting[xyz].{xz,yz,xy,xy2}’
<i>ab</i>	magnetic helicity density $\mathbf{A} \cdot \mathbf{B}$; writes ‘ab[xyz].{xz,yz,xy,xy2}’
Module ‘pscalar.f90’	
<i>lncc</i>	logarithmic density of passive scalar $\ln c$; writes ‘lncc.{xz,yz,xy,xy2}’
Module ‘cosmicray.f90’	
<i>ecr</i>	energy e_{cr} of cosmic rays (?); writes ‘ec.{xz,yz,xy,xy2}’

K.6 List of parameters for ‘phiaver.in’

The following table lists all (at the time of writing, November 2003) possible inputs to the file ‘phiaver.in’.

<i>Variable</i>	<i>Meaning</i>
Module ‘cdata.f90’	
<i>rcylmphi</i>	cylindrical radius $\varpi = \sqrt{x^2 + y^2}$ (useful for debugging azimuthal averages)
<i>phimphi</i>	azimuthal angle $\varphi = \arctan \frac{y}{x}$ (useful for debugging)
<i>zmphi</i>	<i>z</i> -coordinate (useful for debugging)
<i>rmphi</i>	spherical radius $r = \sqrt{\varpi^2 + z^2}$ (useful for debugging)
Module ‘hydro.f90’	
<i>urmphi</i>	$\langle u_{\varpi} \rangle_{\varphi}$ [cyl. polar coords (ϖ, φ, z)]
<i>upmphi</i>	$\langle u_{\varphi} \rangle_{\varphi}$
<i>uzmphi</i>	$\langle u_z \rangle_{\varphi}$
<i>rurmphi</i>	$\langle \rho u_{\varpi} \rangle_{\varphi}$
<i>rupmphi</i>	$\langle \rho u_{\varphi} \rangle_{\varphi}$
<i>ruzmphi</i>	$\langle \rho u_z \rangle_{\varphi}$
<i>ur2mphi</i>	$\langle u_{\varpi}^2 \rangle_{\varphi}$
<i>up2mphi</i>	$\langle u_{\varphi}^2 \rangle_{\varphi}$
<i>uz2mphi</i>	$\langle u_z^2 \rangle_{\varphi}$
<i>urupmphi</i>	$\langle u_{\varpi} u_{\varphi} \rangle_{\varphi}$
<i>uruzmphi</i>	$\langle u_{\varpi} u_z \rangle_{\varphi}$
<i>upuzmphi</i>	$\langle u_{\varphi} u_z \rangle_{\varphi}$
<i>rurupmphi</i>	$\langle \rho u_{\varpi} u_{\varphi} \rangle_{\varphi}$
<i>ruruzmphi</i>	$\langle \rho u_{\varpi} u_z \rangle_{\varphi}$
<i>rupuzmphi</i>	$\langle \rho u_{\varphi} u_z \rangle_{\varphi}$
<i>ursphmphi</i>	$\langle u_r \rangle_{\varphi}$
<i>uthmphi</i>	$\langle u_{\vartheta} \rangle_{\varphi}$
<i>rursphmphi</i>	$\langle \rho u_r \rangle_{\varphi}$
<i>ruthmphi</i>	$\langle \rho u_{\vartheta} \rangle_{\varphi}$
<i>uumphi</i>	shorthand for <i>urmphi</i> , <i>upmphi</i> and <i>uzmphi</i> together
<i>uusphmphi</i>	shorthand for <i>ursphmphi</i> , <i>uthmphi</i> and <i>upmphi</i> together
<i>u2mphi</i>	$\langle \mathbf{u}^2 \rangle_{\varphi}$
<i>o2mphi</i>	$\langle \omega^2 \rangle_{\varphi}$
<i>fkinrsphmphi</i>	$\langle \frac{1}{2} \rho \mathbf{u}^2 u_r \rangle_{\varphi}$
Module ‘density.f90’	
<i>lnrhomphi</i>	$\langle \ln \varrho \rangle_{\varphi}$
<i>rhomphi</i>	$\langle \varrho \rangle_{\varphi}$
Module ‘entropy.f90’	
<i>ssmphi</i>	$\langle s \rangle_{\varphi}$
<i>ss2mphi</i>	$\langle s^2 \rangle_{\varphi}$
<i>cs2mphi</i>	$\langle c_s^2 \rangle_{\varphi}$
<i>TTmphi</i>	$\langle T \rangle_{\varphi}$
<i>ppmphi</i>	$\langle p \rangle_{\varphi}$
<i>dcoolmphi</i>	divergence of combined heating and cooling fluxes
<i>divcoolmphi</i>	divergence of cooling flux
<i>divheatmphi</i>	divergence of heating flux
<i>fradrsphmphi kramers</i>	F_{rad} (φ -averaged, from Kramers’ opacity)

<i>fradrsphmphi_Kconst</i>	F_{rad} (φ -averaged, for K-const)
<i>fradrsphmphi_Kprof</i>	F_{rad} (φ -averaged, for K-profile)
<i>fconvrsphmphi</i>	$\langle c_p \varrho u_r T \rangle_\varphi$
<i>fconvthsphmphi</i>	$\langle c_p \varrho u_\theta T \rangle_\varphi$
<i>fconvpsphmphi</i>	$\langle c_p \varrho u_\phi T \rangle_\varphi$
<i>ursphTTmphi</i>	$\langle u_r T \rangle_\varphi$
<i>fturbrrsphmphi</i>	F_{SGS} (φ -averaged SGS diffusion for star-in-a-box simulations)

Module ‘magnetic.f90’

<i>jbmphi</i>	$\langle \mathbf{J} \cdot \mathbf{B} \rangle_\varphi$
<i>brmphi</i>	$\langle B_\varpi \rangle_\varphi$ [cyl. polar coords (ϖ, φ, z)]
<i>bpmphi</i>	$\langle B_\varphi \rangle_\varphi$
<i>bzmphi</i>	$\langle B_z \rangle_\varphi$
<i>br2mphi</i>	$\langle B_\varpi^2 \rangle_\varphi$
<i>bp2mphi</i>	$\langle B_\varphi^2 \rangle_\varphi$
<i>bz2mphi</i>	$\langle B_z^2 \rangle_\varphi$
<i>bbmphi</i>	shorthand for <i>brmphi</i> , <i>bpmphi</i> and <i>bzmphi</i> together
<i>bbosphmphi</i>	shorthand for <i>brosphmphi</i> , <i>bthmphi</i> and <i>bpmphi</i> together
<i>b2mphi</i>	$\langle \mathbf{B}^2 \rangle_\varphi$
<i>brosphmphi</i>	$\langle B_r \rangle_\varphi$
<i>bthmphi</i>	$\langle B_\theta \rangle_\varphi$
<i>brsmphi</i>	$\langle \sin \theta B_\varpi \rangle_\varphi$
<i>brcmphi</i>	$\langle \cos \theta B_\varpi \rangle_\varphi$
<i>bpsmphi</i>	$\langle \sin \theta B_\varphi \rangle_\varphi$
<i>bpcmphi</i>	$\langle \cos \theta B_\varphi \rangle_\varphi$
<i>bzsmphi</i>	$\langle \sin \theta B_z \rangle_\varphi$
<i>bzcmphi</i>	$\langle \cos \theta B_z \rangle_\varphi$
<i>brbpmphi</i>	$\langle B_\varpi B_\varphi \rangle_\varphi$
<i>brbzmphi</i>	$\langle B_\varpi B_z \rangle_\varphi$
<i>bpbzmphi</i>	$\langle B_\varphi B_z \rangle_\varphi$

Module ‘anelastic.f90’

<i>lnrhomphi</i>	$\langle \ln \varrho \rangle_\varphi$
<i>rhomphi</i>	$\langle \varrho \rangle_\varphi$

Module ‘entropy_anelastic.f90’

<i>ssmphi</i>	$\langle s \rangle_\varphi$
<i>cs2mphi</i>	$\langle c_s^2 \rangle_\varphi$

Module ‘hydro_potential.f90’

<i>urmphi</i>	$\langle u_\varpi \rangle_\varphi$ [cyl. polar coords (ϖ, φ, z)]
<i>upmphi</i>	$\langle u_\varphi \rangle_\varphi$
<i>uzmphi</i>	$\langle u_z \rangle_\varphi$
<i>ursphmphi</i>	$\langle u_r \rangle_\varphi$
<i>uthmphi</i>	$\langle u_\theta \rangle_\varphi$
<i>uumphi</i>	shorthand for <i>urmphi</i> , <i>upmphi</i> and <i>uzmphi</i> together
<i>uusphmphi</i>	shorthand for <i>ursphmphi</i> , <i>uthmphi</i> and <i>upmphi</i> together
<i>u2mphi</i>	$\langle \mathbf{u}^2 \rangle_\varphi$

Module ‘magnetic_shearboxJ.f90’	
<i>jbmphi</i>	$\langle \mathbf{J} \cdot \mathbf{B} \rangle_\varphi$
<i>brmphi</i>	$\langle B_\varpi \rangle_\varphi$ [cyl. polar coords (ϖ, φ, z)]
<i>bpmphi</i>	$\langle B_\varphi \rangle_\varphi$
<i>bzmphi</i>	$\langle B_z \rangle_\varphi$
<i>bbmphi</i>	shorthand for <i>brmphi</i> , <i>bpmphi</i> and <i>bzmphi</i> together
<i>bbsphmphi</i>	shorthand for <i>brsphmphi</i> , <i>bthmphi</i> and <i>bpmphi</i> together
<i>b2mphi</i>	$\langle B^2 \rangle_\varphi$
<i>brsphmphi</i>	$\langle B_r \rangle_\varphi$
<i>bthmphi</i>	$\langle B_\theta \rangle_\varphi$
Module ‘viscosity.f90’	
<i>fviscrsphmphi</i>	$\langle 2\nu \varrho u_i \mathcal{S}_{ir} \rangle_\varphi$ (<i>r</i> -xomponent of viscous flux)

K.7 List of parameters for ‘xyaver.in’

The following table lists possible inputs to the file ‘xyaver.in’. This list is not complete and maybe outdated.

Variable	Meaning
Module ‘cdata.f90’	
<i>dtvmaxz</i>	z-dependent version of <i>dtv</i>
Module ‘hydro.f90’	
<i>u2mz</i>	$\langle \mathbf{u}^2 \rangle_{xy}$
<i>o2mz</i>	$\langle \mathbf{W}^2 \rangle_{xy}$
<i>divu2mz</i>	$\langle (\nabla \cdot \mathbf{u})^2 \rangle_{xy}$
<i>curlru2mz</i>	$\langle (\nabla \times \varrho \mathbf{U})^2 \rangle_{xy}$
<i>divru2mz</i>	$\langle (\nabla \cdot \varrho \mathbf{u})^2 \rangle_{xy}$
<i>fmasszmz</i>	$\langle \varrho u_z \rangle_{xy}$
<i>fkinzmz</i>	$\langle \frac{1}{2} \varrho \mathbf{u}^2 u_z \rangle_{xy}$
<i>fkinzupmz</i>	$\langle \frac{1}{2} \varrho \mathbf{u}^2 u_{z\uparrow} \rangle_{xy}$
<i>fkinzdownmz</i>	$\langle \frac{1}{2} \varrho \mathbf{u}^2 u_{z\downarrow} \rangle_{xy}$
<i>uxmz</i>	$\langle u_x \rangle_{xy}$ (horiz. averaged <i>x</i> velocity)
<i>uymz</i>	$\langle u_y \rangle_{xy}$
<i>uzmz</i>	$\langle u_z \rangle_{xy}$
<i>uxph1mz</i>	$\langle u_x \rangle_{xy} _{\text{phase1}}$
<i>uxph2mz</i>	$\langle u_y \rangle_{xy} _{\text{phase2}}$
<i>uxph3mz</i>	$\langle u_x \rangle_{xy} _{\text{phase3}}$
<i>uyph1mz</i>	$\langle u_y \rangle_{xy} _{\text{phase1}}$
<i>uyph2mz</i>	$\langle u_y \rangle_{xy} _{\text{phase2}}$
<i>uyph3mz</i>	$\langle u_y \rangle_{xy} _{\text{phase3}}$
<i>uzph1mz</i>	$\langle u_z \rangle_{xy} _{\text{phase1}}$
<i>uzph2mz</i>	$\langle u_z \rangle_{xy} _{\text{phase2}}$
<i>uzph3mz</i>	$\langle u_z \rangle_{xy} _{\text{phase3}}$
<i>u2ph1mz</i>	$\langle u^2 \rangle_{xy} _{\text{phase1}}$

<i>u2ph2mz</i>	$\langle u^2 \rangle_{xy} \text{phase2}$
<i>u2ph3mz</i>	$\langle u^2 \rangle_{xy} \text{phase3}$
<i>ux2ph1mz</i>	$\langle u_x^2 \rangle_{xy} \text{phase1}$
<i>ux2ph2mz</i>	$\langle u_x^2 \rangle_{xy} \text{phase2}$
<i>ux2ph3mz</i>	$\langle u_x^2 \rangle_{xy} \text{phase3}$
<i>uy2ph1mz</i>	$\langle u_y^2 \rangle_{xy} \text{phase1}$
<i>uy2ph2mz</i>	$\langle u_y^2 \rangle_{xy} \text{phase2}$
<i>uy2ph3mz</i>	$\langle u_y^2 \rangle_{xy} \text{phase3}$
<i>uz2ph1mz</i>	$\langle u_z^2 \rangle_{xy} \text{phase1}$
<i>uz2ph2mz</i>	$\langle u_z^2 \rangle_{xy} \text{phase2}$
<i>uz2ph3mz</i>	$\langle u_z^2 \rangle_{xy} \text{phase3}$
<i>ffdownmz</i>	Filling factor of downflows
<i>uzupmz</i>	$\langle u_{z\uparrow} \rangle_{xy}$
<i>uzdownmz</i>	$\langle u_{z\downarrow} \rangle_{xy}$
<i>ruzupmz</i>	$\langle \rho u_{z\uparrow} \rangle_{xy}$
<i>ruzdownmz</i>	$\langle \rho u_{z\downarrow} \rangle_{xy}$
<i>divumz</i>	$\langle \text{div} \mathbf{u} \rangle_{xy}$
<i>uzdivumz</i>	$\langle u_z \text{div} \mathbf{u} \rangle_{xy}$
<i>oxmz</i>	$\langle \omega_x \rangle_{xy}$
<i>oymz</i>	$\langle \omega_y \rangle_{xy}$
<i>ozmz</i>	$\langle \omega_z \rangle_{xy}$
<i>ux2mz</i>	$\langle u_x^2 \rangle_{xy}$
<i>uy2mz</i>	$\langle u_y^2 \rangle_{xy}$
<i>uz2mz</i>	$\langle u_z^2 \rangle_{xy}$
<i>ux3mz</i>	$\langle u_x^3 \rangle_{xy}$
<i>uy3mz</i>	$\langle u_y^3 \rangle_{xy}$
<i>uz3mz</i>	$\langle u_z^3 \rangle_{xy}$
<i>ux4mz</i>	$\langle u_x^4 \rangle_{xy}$
<i>uy4mz</i>	$\langle u_y^4 \rangle_{xy}$
<i>uz4mz</i>	$\langle u_z^4 \rangle_{xy}$
<i>uz2upmz</i>	$\langle u_{z\uparrow}^2 \rangle_{xy}$
<i>uz2downmz</i>	$\langle u_{z\downarrow}^2 \rangle_{xy}$
<i>ox2mz</i>	$\langle \omega_x^2 \rangle_{xy}$
<i>oy2mz</i>	$\langle \omega_y^2 \rangle_{xy}$
<i>oz2mz</i>	$\langle \omega_z^2 \rangle_{xy}$
<i>ruxmz</i>	$\langle \rho u_x \rangle_{xy}$
<i>ruymz</i>	$\langle \rho u_y \rangle_{xy}$
<i>ruzmxz</i>	$\langle \rho u_z \rangle_{xy}$
<i>ruxph1mz</i>	$\langle \rho u_x \rangle_{xy} \text{phase1}$
<i>ruxph2mz</i>	$\langle \rho u_x \rangle_{xy} \text{phase2}$
<i>ruxph3mz</i>	$\langle \rho u_x \rangle_{xy} \text{phase3}$
<i>ruyph1mz</i>	$\langle \rho u_y \rangle_{xy} \text{phase1}$
<i>ruyph2mz</i>	$\langle \rho u_y \rangle_{xy} \text{phase2}$
<i>ruyph3mz</i>	$\langle \rho u_y \rangle_{xy} \text{phase3}$
<i>ruzph1mz</i>	$\langle \rho u_z \rangle_{xy} \text{phase1}$
<i>ruzph2mz</i>	$\langle \rho u_z \rangle_{xy} \text{phase2}$
<i>ruzph3mz</i>	$\langle \rho u_z \rangle_{xy} \text{phase3}$

<i>rux2ph1mz</i>	$\langle \rho u_x^2 \rangle_{xy} \text{phase1}$
<i>rux2ph2mz</i>	$\langle \rho u_x^2 \rangle_{xy} \text{phase2}$
<i>rux2ph3mz</i>	$\langle \rho u_x^2 \rangle_{xy} \text{phase3}$
<i>ruy2ph1mz</i>	$\langle \rho u_y^2 \rangle_{xy} \text{phase1}$
<i>ruy2ph2mz</i>	$\langle \rho u_y^2 \rangle_{xy} \text{phase2}$
<i>ruy2ph3mz</i>	$\langle \rho u_y^2 \rangle_{xy} \text{phase3}$
<i>ruz2ph1mz</i>	$\langle \rho u_z^2 \rangle_{xy} \text{phase1}$
<i>ruz2ph2mz</i>	$\langle \rho u_z^2 \rangle_{xy} \text{phase2}$
<i>ruz2ph3mz</i>	$\langle \rho u_z^2 \rangle_{xy} \text{phase3}$
<i>ekinph1mz</i>	$\langle \frac{1}{2} \rho \mathbf{u}^2 \rangle_{xy} \text{phase1}$
<i>ekinph2mz</i>	$\langle \frac{1}{2} \rho \mathbf{u}^2 \rangle_{xy} \text{phase2}$
<i>ekinph3mz</i>	$\langle \frac{1}{2} \rho \mathbf{u}^2 \rangle_{xy} \text{phase3}$
<i>oxph1mz</i>	$\langle \omega_x \rangle_{xy} \text{phase1}$
<i>oxph2mz</i>	$\langle \omega_x \rangle_{xy} \text{phase2}$
<i>oxph3mz</i>	$\langle \omega_x \rangle_{xy} \text{phase3}$
<i>oyph1mz</i>	$\langle \omega_y \rangle_{xy} \text{phase1}$
<i>oyph2mz</i>	$\langle \omega_y \rangle_{xy} \text{phase2}$
<i>oyph3mz</i>	$\langle \omega_y \rangle_{xy} \text{phase3}$
<i>ozph1mz</i>	$\langle \omega_z \rangle_{xy} \text{phase1}$
<i>ozph2mz</i>	$\langle \omega_z \rangle_{xy} \text{phase2}$
<i>ozph3mz</i>	$\langle \omega_z \rangle_{xy} \text{phase3}$
<i>ouph1mz</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_{xy} \text{phase1}$
<i>ouph2mz</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_{xy} \text{phase2}$
<i>ouph3mz</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_{xy} \text{phase3}$
<i>rux2mz</i>	$\langle \rho u_x^2 \rangle_{xy}$
<i>ruy2mz</i>	$\langle \rho u_y^2 \rangle_{xy}$
<i>ruz2mz</i>	$\langle \rho u_z^2 \rangle_{xy}$
<i>uxuymz</i>	$\langle u_x u_y \rangle_{xy}$
<i>uxuzmz</i>	$\langle u_x u_z \rangle_{xy}$
<i>uyuzmz</i>	$\langle u_y u_z \rangle_{xy}$
<i>Rxymz</i>	$\langle u'_x u'_y \rangle_{xy}$
<i>Rxyupmz</i>	$\langle u'_{x\downarrow} u'_{y\downarrow} \rangle_{xy}$
<i>Rxydownmz</i>	$\langle u'_{x\uparrow} u'_{y\uparrow} \rangle_{xy}$
<i>Rxzmz</i>	$\langle u'_x u'_z \rangle_{xy}$
<i>Rxzupmz</i>	$\langle u'_{x\downarrow} u'_{z\downarrow} \rangle_{xy}$
<i>Rxzdownmz</i>	$\langle u'_{x\uparrow} u'_{z\uparrow} \rangle_{xy}$
<i>Ryzmz</i>	$\langle u'_y u'_z \rangle_{xy}$
<i>Ryzupmz</i>	$\langle u'_{y\downarrow} u'_{z\downarrow} \rangle_{xy}$
<i>Ryzdownmz</i>	$\langle u'_{y\uparrow} u'_{z\uparrow} \rangle_{xy}$
<i>ruxuymz</i>	$\langle \rho u_x u_y \rangle_{xy}$
<i>ruxuzmz</i>	$\langle \rho u_x u_z \rangle_{xy}$
<i>ruyuzmz</i>	$\langle \rho u_y u_z \rangle_{xy}$
<i>ruxuy2mz</i>	$\langle (\rho u_x u_y)^2 \rangle_{xy}$
<i>ruxuz2mz</i>	$\langle (\rho u_x u_z)^2 \rangle_{xy}$
<i>ruyuz2mz</i>	$\langle (\rho u_y u_z)^2 \rangle_{xy}$

<i>rufmz</i>	$\langle \rho \mathbf{u} \cdot \mathbf{F} \rangle_{xy}$, where $\mathbf{F}$ is the continuous forcing (lforcing_cont_uu=T)
<i>oxuxxmz</i>	$\langle \omega_x u_{x,x} \rangle_{xy}$
<i>oyuxymz</i>	$\langle \omega_y u_{x,y} \rangle_{xy}$
<i>oxuyxmz</i>	$\langle \omega_x u_{y,x} \rangle_{xy}$
<i>oyuyymz</i>	$\langle \omega_y u_{y,y} \rangle_{xy}$
<i>oxuzxmz</i>	$\langle \omega_x u_{z,x} \rangle_{xy}$
<i>oyuzymz</i>	$\langle \omega_y u_{z,y} \rangle_{xy}$
<i>uyxuzxmz</i>	$\langle u_{y,x} u_{z,x} \rangle_{xy}$
<i>uyyuzymz</i>	$\langle u_{y,y} u_{z,y} \rangle_{xy}$
<i>uyzuzzmz</i>	$\langle u_{y,z} u_{z,z} \rangle_{xy}$
<i>ekinmz</i>	$\langle \frac{1}{2} \rho \mathbf{u}^2 \rangle_{xy}$
<i>oumz</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_{xy}$
<i>Remz</i>	$\langle \frac{ \mathbf{u} \cdot \mathbf{u} }{\left \frac{\partial}{\partial x_j} (\nu S_{ij}) \right } \rangle_{xy}$
<i>oguxmz</i>	$\langle (\boldsymbol{\omega} \cdot \nabla \mathbf{u})_x \rangle_{xy}$
<i>oguymz</i>	$\langle (\boldsymbol{\omega} \cdot \nabla \mathbf{u})_y \rangle_{xy}$
<i>oguzmz</i>	$\langle (\boldsymbol{\omega} \cdot \nabla \mathbf{u})_z \rangle_{xy}$
<i>ogux2mz</i>	$\langle (\boldsymbol{\omega} \cdot \nabla \mathbf{u})_x^2 \rangle_{xy}$
<i>oguy2mz</i>	$\langle (\boldsymbol{\omega} \cdot \nabla \mathbf{u})_y^2 \rangle_{xy}$
<i>oguz2mz</i>	$\langle (\boldsymbol{\omega} \cdot \nabla \mathbf{u})_z^2 \rangle_{xy}$
<i>oxdivumz</i>	$\langle \omega_x \nabla \cdot \mathbf{u} \rangle_{xy}$
<i>oydivumz</i>	$\langle \omega_y \nabla \cdot \mathbf{u} \rangle_{xy}$
<i>ozdivumz</i>	$\langle \omega_z \nabla \cdot \mathbf{u} \rangle_{xy}$
<i>oxdivu2mz</i>	$\langle (\omega_x \nabla \cdot \mathbf{u})^2 \rangle_{xy}$
<i>oydivu2mz</i>	$\langle (\omega_y \nabla \cdot \mathbf{u})^2 \rangle_{xy}$
<i>ozdivu2mz</i>	$\langle (\omega_z \nabla \cdot \mathbf{u})^2 \rangle_{xy}$
<i>acczmz</i>	$\langle Du_z / Dt \rangle_{xy}$
<i>acczupmz</i>	$\langle Du_z / Dt \rangle_{xy+}$
<i>acczdownmz</i>	$\langle Du_z / Dt \rangle_{xy-}$
<i>accpowzmz</i>	$\langle (u_z Du_z / Dt)^2 \rangle_{xy}$
<i>accpowzupmz</i>	$\langle (u_z Du_z / Dt)^2 \rangle_{xy+}$
<i>accpowzdownmz</i>	$\langle (u_z Du_z / Dt)^2 \rangle_{xy-}$
<i>totalforcezmz</i>	$\langle \rho Du_z / Dt \rangle_{xy}$
<i>totalforcezupmz</i>	$\langle \rho Du_z / Dt \rangle_{xy+}$
<i>totalforcezdownmz</i>	$\langle \rho Du_z / Dt \rangle_{xy-}$

Module 'density.f90'

<i>rhomz</i>	$\langle \rho \rangle_{xy}$
<i>rhoupmz</i>	$\langle \rho_{\uparrow} \rangle_{xy}$
<i>rhodownmz</i>	$\langle \rho_{\downarrow} \rangle_{xy}$
<i>rho2mz</i>	$\langle \rho^2 \rangle_{xy}$
<i>rho2upmz</i>	$\langle \rho_{\uparrow}^2 \rangle_{xy}$
<i>rho2downmz</i>	$\langle \rho_{\downarrow}^2 \rangle_{xy}$
<i>rhof2mz</i>	$\langle \rho'^2 \rangle_{xy}$
<i>rhof2upmz</i>	$\langle \rho'_{\uparrow}{}^2 \rangle_{xy}$
<i>rhof2downmz</i>	$\langle \rho'_{\downarrow}{}^2 \rangle_{xy}$

<i>gzlnrhomz</i>	$\langle \nabla_z \ln \varrho \rangle_{xy}$
<i>uglnrhomz</i>	$\langle \mathbf{u} \cdot \nabla \ln \varrho \rangle_{xy}$
<i>ugrhomz</i>	$\langle \mathbf{u} \cdot \nabla \varrho \rangle_{xy}$
<i>uygzlnrhomz</i>	$\langle u_y \nabla_z \ln \varrho \rangle_{xy}$
<i>uzgylnrhomz</i>	$\langle u_z \nabla_y \ln \varrho \rangle_{xy}$
<i>rhoph1mz</i>	$\langle \bar{n}^2 \rangle_{xy} _{\text{phase1}}$
<i>rhoph2mz</i>	$\langle \bar{n}^2 \rangle_{xy} _{\text{phase2}}$
<i>rhoph3mz</i>	$\langle \bar{n}^2 \rangle_{xy} _{\text{phase3}}$
<i>rho2ph1mz</i>	$\langle \bar{n}^2 \rangle_{xy} _{\text{phase1}}$
<i>rho2ph2mz</i>	$\langle \bar{n}^2 \rangle_{xy} _{\text{phase2}}$
<i>rho2ph3mz</i>	$\langle \bar{n}^2 \rangle_{xy} _{\text{phase3}}$
<i>rho2mx</i>	$\langle \varrho^2 \rangle_{yz}$

Module ‘entropy.f90’

<i>fradz</i>	$\langle F_{\text{rad}} \rangle_{xy}$
<i>fconvz</i>	$\langle c_p \varrho u_z T \rangle_{xy}$
<i>Fenthz</i>	$\langle c_p (\varrho u_z)' T' \rangle_{xy}$
<i>Fenthupz</i>	$\langle (c_p (\varrho u_z)' T')_{\uparrow} \rangle_{xy}$
<i>Fenthdownz</i>	$\langle (c_p (\varrho u_z)' T')_{\downarrow} \rangle_{xy}$
<i>ssmz</i>	$\langle s \rangle_{xy}$
<i>ssupmz</i>	$\langle s_{\uparrow} \rangle_{xy}$
<i>ssdownmz</i>	$\langle s_{\downarrow} \rangle_{xy}$
<i>ss2mz</i>	$\langle s^2 \rangle_{xy}$
<i>ss2upmz</i>	$\langle s_{\uparrow}^2 \rangle_{xy}$
<i>ss2downmz</i>	$\langle s_{\downarrow}^2 \rangle_{xy}$
<i>ssf2mz</i>	$\langle s'^2 \rangle_{xy}$
<i>ssf2upmz</i>	$\langle s'_{\uparrow}{}^2 \rangle_{xy}$
<i>ssf2downmz</i>	$\langle s'_{\downarrow}{}^2 \rangle_{xy}$
<i>ppmz</i>	$\langle p \rangle_{xy}$
<i>TTmz</i>	$\langle T \rangle_{xy}$
<i>TTdownmz</i>	$\langle T_{\downarrow} \rangle_{xy}$
<i>TTupmz</i>	$\langle T_{\uparrow} \rangle_{xy}$
<i>TT2mz</i>	$\langle T^2 \rangle_{xy}$
<i>TT2upmz</i>	$\langle T_{\uparrow}^2 \rangle_{xy}$
<i>TT2downmz</i>	$\langle T_{\downarrow}^2 \rangle_{xy}$
<i>TTf2mz</i>	$\langle T'^2 \rangle_{xy}$
<i>TTf2upmz</i>	$\langle T'_{\uparrow}{}^2 \rangle_{xy}$
<i>TTf2downmz</i>	$\langle T'_{\downarrow}{}^2 \rangle_{xy}$
<i>ugradpmz</i>	$\langle \mathbf{u} \cdot \nabla p \rangle_{xy}$
<i>gradpxmz</i>	$\langle \nabla p_x \rangle_{xy}$
<i>gradpymz</i>	$\langle \nabla p_y \rangle_{xy}$
<i>gradpzmz</i>	$\langle \nabla p_z \rangle_{xy}$
<i>pdivumz</i>	$\langle p \nabla \cdot \mathbf{u} \rangle_{xy}$
<i>uxTTmz</i>	$\langle u_x T \rangle_{xy}$
<i>uyTTmz</i>	$\langle u_y T \rangle_{xy}$

<code>uzTTmz</code>	$\langle u_z T \rangle_{xy}$
<code>uzTTupmz</code>	$\langle (u_z T)_{\uparrow} \rangle_{xy}$
<code>uzTTdownmz</code>	$\langle (u_z T)_{\downarrow} \rangle_{xy}$
<code>gTxgsxmz</code>	$\langle (\nabla T \times \nabla s)_x \rangle_{xy}$
<code>gTxgsymz</code>	$\langle (\nabla T \times \nabla s)_y \rangle_{xy}$
<code>gTxgszmz</code>	$\langle (\nabla T \times \nabla s)_z \rangle_{xy}$
<code>gTxgsx2mz</code>	$\langle (\nabla T \times \nabla s)_x^2 \rangle_{xy}$
<code>gTxgsy2mz</code>	$\langle (\nabla T \times \nabla s)_y^2 \rangle_{xy}$
<code>gTxgsz2mz</code>	$\langle (\nabla T \times \nabla s)_z^2 \rangle_{xy}$
<code>fradz_kramers</code>	F_{rad} (from Kramers' opacity)
<code>fradz_Kprof</code>	F_{rad} (from Kprof)
<code>fradz_constchi</code>	F_{rad} (from chi_const)
<code>fturbz</code>	$\langle \rho T \chi_t \nabla_z s \rangle_{xy}$ (turbulent heat flux)
<code>fturbtz</code>	$\langle \rho T \chi_t \nabla_z s \rangle_{xy}$ (turbulent heat flux)
<code>fturbmz</code>	$\langle \rho T \chi_t \nabla_z \bar{s} \rangle_{xy}$ (turbulent heat flux)
<code>fturbfz</code>	$\langle \rho T \chi_t \nabla_z s' \rangle_{xy}$ (turbulent heat flux)
<code>dcoolz</code>	divergence of heating term (only for get_heat_cool_gravz)
<code>heatmz</code>	divergence of heating term (all terms except tau_relax_ss and tau_cool_ss)
<code>Kkramersmz</code>	$\langle K_0 T^{(3-b)} / \rho^{(a+1)} \rangle_{xy}$
<code>ethmz</code>	$\langle \rho e \rangle_{xy}$
<code>fpreszmz</code>	$-\langle \frac{\nabla p}{\rho} \rangle_{xy}$
<code>gTT2mz</code>	$\langle \nabla T^2 \rangle_{xy}$
<code>gss2mz</code>	$\langle \nabla s^2 \rangle_{xy}$
<code>fracvph1mz</code>	$\langle f_v \rangle_{xy} _{\text{phase1}}$ (phase 1 fractional volume)
<code>fracvph2mz</code>	$\langle f_v \rangle_{xy} _{\text{phase2}}$ (phase 2 fractional volume)
<code>fracvph3mz</code>	$\langle f_v \rangle_{xy} _{\text{phase3}}$ (phase 3 fractional volume)

Module 'magnetic.f90'

<code>axmz</code>	$\langle \mathcal{A}_x \rangle_{xy}$
<code>aymz</code>	$\langle \mathcal{A}_y \rangle_{xy}$
<code>azmz</code>	$\langle \mathcal{A}_z \rangle_{xy}$
<code>abuxmz</code>	$\langle (\mathbf{A} \cdot \mathbf{B}) u_x \rangle_{xy}$
<code>abuymz</code>	$\langle (\mathbf{A} \cdot \mathbf{B}) u_y \rangle_{xy}$
<code>abuzmz</code>	$\langle (\mathbf{A} \cdot \mathbf{B}) u_z \rangle_{xy}$
<code>uabxmz</code>	$\langle (\mathbf{u} \cdot \mathbf{A}) B_x \rangle_{xy}$
<code>uabymz</code>	$\langle (\mathbf{u} \cdot \mathbf{A}) B_y \rangle_{xy}$
<code>uabzmz</code>	$\langle (\mathbf{u} \cdot \mathbf{A}) B_z \rangle_{xy}$
<code>bbxmz</code>	$\langle \mathcal{B}'_x \rangle_{xy}$
<code>bbymz</code>	$\langle \mathcal{B}'_y \rangle_{xy}$
<code>bbzmz</code>	$\langle \mathcal{B}'_z \rangle_{xy}$
<code>bxmz</code>	$\langle \mathcal{B}_x \rangle_{xy}$
<code>bymz</code>	$\langle \mathcal{B}_y \rangle_{xy}$
<code>bzmz</code>	$\langle \mathcal{B}_z \rangle_{xy}$
<code>jxmz</code>	$\langle \mathcal{J}_x \rangle_{xy}$
<code>jymz</code>	$\langle \mathcal{J}_y \rangle_{xy}$
<code>jzmz</code>	$\langle \mathcal{J}_z \rangle_{xy}$

<i>Exmz</i>	$\langle \mathcal{E}_x \rangle_{xy}$
<i>Eymz</i>	$\langle \mathcal{E}_y \rangle_{xy}$
<i>Ezmz</i>	$\langle \mathcal{E}_z \rangle_{xy}$
<i>bx2mz</i>	$\langle B_x^2 \rangle_{xy}$
<i>by2mz</i>	$\langle B_y^2 \rangle_{xy}$
<i>bz2mz</i>	$\langle B_z^2 \rangle_{xy}$
<i>bx2rmz</i>	$\langle B_x^2 / \varrho \rangle_{xy}$
<i>by2rmz</i>	$\langle B_y^2 / \varrho \rangle_{xy}$
<i>bz2rmz</i>	$\langle B_z^2 / \varrho \rangle_{xy}$
<i>beta1mz</i>	$\langle (B^2 / 2\mu_0) / p \rangle_{xy}$
<i>betamz</i>	$\langle \beta \rangle_{xy}$
<i>beta2mz</i>	$\langle \beta^2 \rangle_{xy}$
<i>jbmz</i>	$\langle \mathbf{J} \cdot \mathbf{B} \rangle_{ xy}$
<i>bdel2amz</i>	$\langle \mathbf{B} \cdot \nabla^2 \mathbf{A} \rangle_{ xy}$
<i>jdel2amz</i>	$\langle \mathbf{J} \cdot \nabla^2 \mathbf{A} \rangle_{ xy}$
<i>d6abmz</i>	$\langle \nabla^6 \mathbf{A} \cdot \mathbf{B} \rangle_{ xy}$
<i>d6amz1</i>	$\langle \nabla^6 \mathbf{A} \rangle_{xy} x$
<i>d6amz2</i>	$\langle \nabla^6 \mathbf{A} \rangle_{xy} y$
<i>d6amz3</i>	$\langle \nabla^6 \mathbf{A} \rangle_{xy} z$
<i>abmz</i>	$\langle \mathbf{A} \cdot \mathbf{B} \rangle_{ xy}$
<i>ubmz</i>	$\langle \mathbf{u} \cdot \mathbf{B} \rangle_{ xy}$
<i>ujmz</i>	$\langle \mathbf{u} \cdot \mathbf{J} \rangle_{ xy}$
<i>obmz</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{B} \rangle_{ xy}$
<i>uamz</i>	$\langle \mathbf{u} \cdot \mathbf{A} \rangle_{ xy}$
<i>bzuamz</i>	$\langle B_z \mathbf{u} \cdot \mathbf{A} \rangle_{ xy}$
<i>bzaymz</i>	$\langle B_z A_y \rangle_{ xy}$
<i>bzdivamz</i>	$\langle B_z \nabla \cdot \mathbf{A} \rangle_{ xy}$
<i>bzLammz</i>	$\langle B_z \Lambda \rangle_{ xy}$
<i>divamz</i>	$\langle \nabla \cdot \mathbf{A} \rangle_{ xy}$
<i>uxbxmz</i>	$\langle u_x b_x \rangle_{ xy}$
<i>uybxmz</i>	$\langle u_y b_x \rangle_{ xy}$
<i>uzbxmz</i>	$\langle u_z b_x \rangle_{ xy}$
<i>uxbymz</i>	$\langle u_x b_y \rangle_{ xy}$
<i>uybymz</i>	$\langle u_y b_y \rangle_{ xy}$
<i>uzbymz</i>	$\langle u_z b_y \rangle_{ xy}$
<i>uxbzmz</i>	$\langle u_x b_z \rangle_{ xy}$
<i>uybzmz</i>	$\langle u_y b_z \rangle_{ xy}$
<i>uzbzmz</i>	$\langle u_z b_z \rangle_{ xy}$
<i>ujxbmz</i>	$\langle \mathbf{u} \cdot (\mathbf{J} \times \mathbf{B}) \rangle_{xy}$
<i>examz1</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_{xy} x$
<i>examz2</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_{xy} y$
<i>examz3</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_{xy} z$
<i>exatotalmz1</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_{xy} x$
<i>exatotalmz2</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_{xy} y$
<i>exatotalmz3</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_{xy} z$
<i>e3xamz1</i>	$\langle \mathbf{E}_{hyper3} \times \mathbf{A} \rangle_{xy} x$
<i>e3xamz2</i>	$\langle \mathbf{E}_{hyper3} \times \mathbf{A} \rangle_{xy} y$
<i>e3xamz3</i>	$\langle \mathbf{E}_{hyper3} \times \mathbf{A} \rangle_{xy} z$
<i>etatotalmz</i>	$\langle \eta \rangle_{xy}$

ay2mz	$\langle A_y^2 \rangle_{xy}$
aybxmz	$\langle A_y B_x \rangle_{xy}$
bxbymz	$\langle B_x B_y \rangle_{xy}$
bxbzmmz	$\langle B_x B_z \rangle_{xy}$
bybzmmz	$\langle B_y B_z \rangle_{xy}$
a2mz	$\langle \mathbf{A}^2 \rangle_{xy}$
b2mz	$\langle \mathbf{B}^2 \rangle_{xy}$
bf2mz	$\langle \mathbf{B}'^2 \rangle_{xy}$
j2mz	$\langle \mathbf{j}^2 \rangle_{xy}$
poynzmmz	Averaged poynting flux in z direction
bcurlfmz	$\langle \mathbf{B} \cdot \nabla \times \mathbf{F} / \mu_0 \rangle_{xy}$, where $\mathbf{F}$ is the additional EMF imposed through continuous forcing (lforcing_cont_aa=T)
epsMmz	$\langle \eta \mu_0 \mathbf{j}^2 \rangle_{xy}$
vmagfricmmz	$\langle 1/\nu_{\text{mag}} \mathbf{j} \times \mathbf{B} / B^2 \rangle_{xy}$
bxph1mz	$\langle \mathcal{B}_x \rangle_{xy} _{\text{phase1}}$
bxph2mz	$\langle \mathcal{B}_x \rangle_{xy} _{\text{phase2}}$
bxph3mz	$\langle \mathcal{B}_x \rangle_{xy} _{\text{phase3}}$
byph1mz	$\langle \mathcal{B}_y \rangle_{xy} _{\text{phase1}}$
byph2mz	$\langle \mathcal{B}_y \rangle_{xy} _{\text{phase2}}$
byph3mz	$\langle \mathcal{B}_y \rangle_{xy} _{\text{phase3}}$
bzph1mz	$\langle \mathcal{B}_z \rangle_{xy} _{\text{phase1}}$
bzph2mz	$\langle \mathcal{B}_z \rangle_{xy} _{\text{phase2}}$
bzph3mz	$\langle \mathcal{B}_z \rangle_{xy} _{\text{phase3}}$
bx2ph1mz	$\langle \mathcal{B}_x^2 \rangle_{xy} _{\text{phase1}}$
bx2ph2mz	$\langle \mathcal{B}_x^2 \rangle_{xy} _{\text{phase2}}$
bx2ph3mz	$\langle \mathcal{B}_x^2 \rangle_{xy} _{\text{phase3}}$
by2ph1mz	$\langle \mathcal{B}_y^2 \rangle_{xy} _{\text{phase1}}$
by2ph2mz	$\langle \mathcal{B}_y^2 \rangle_{xy} _{\text{phase2}}$
by2ph3mz	$\langle \mathcal{B}_y^2 \rangle_{xy} _{\text{phase3}}$
bz2ph1mz	$\langle \mathcal{B}_z^2 \rangle_{xy} _{\text{phase1}}$
bz2ph2mz	$\langle \mathcal{B}_z^2 \rangle_{xy} _{\text{phase2}}$
bz2ph3mz	$\langle \mathcal{B}_z^2 \rangle_{xy} _{\text{phase3}}$
bx2rph1mz	$\langle B_x^2 / \varrho \rangle_{xy} _{\text{phase1}}$
bx2rph2mz	$\langle B_x^2 / \varrho \rangle_{xy} _{\text{phase2}}$
bx2rph3mz	$\langle B_x^2 / \varrho \rangle_{xy} _{\text{phase3}}$
by2rph1mz	$\langle B_y^2 / \varrho \rangle_{xy} _{\text{phase1}}$
by2rph2mz	$\langle B_y^2 / \varrho \rangle_{xy} _{\text{phase2}}$
by2rph3mz	$\langle B_y^2 / \varrho \rangle_{xy} _{\text{phase3}}$
bz2rph1mz	$\langle B_z^2 / \varrho \rangle_{xy} _{\text{phase1}}$
bz2rph2mz	$\langle B_z^2 / \varrho \rangle_{xy} _{\text{phase2}}$
bz2rph3mz	$\langle B_z^2 / \varrho \rangle_{xy} _{\text{phase3}}$
abph1mz	$\langle \mathbf{A} \cdot \mathbf{B} \rangle_{xy} _{\text{phase1}}$
abph2mz	$\langle \mathbf{A} \cdot \mathbf{B} \rangle_{xy} _{\text{phase2}}$
abph3mz	$\langle \mathbf{A} \cdot \mathbf{B} \rangle_{xy} _{\text{phase3}}$
jbph1mz	$\langle \mathbf{J} \cdot \mathbf{B} \rangle_{xy} _{\text{phase1}}$
jbph2mz	$\langle \mathbf{J} \cdot \mathbf{B} \rangle_{xy} _{\text{phase2}}$
jbph3mz	$\langle \mathbf{J} \cdot \mathbf{B} \rangle_{xy} _{\text{phase3}}$

<i>poynzph1mz</i>	Averaged poynting flux in z direction for phase 1
<i>poynzph2mz</i>	Averaged poynting flux in z direction for phase 2
<i>poynzph3mz</i>	Averaged poynting flux in z direction for phase 3
<i>jxph1mz</i>	$\langle \mathcal{J}_x \rangle_{xy} _{\text{phase1}}$
<i>jxph2mz</i>	$\langle \mathcal{J}_x \rangle_{xy} _{\text{phase2}}$
<i>jxph3mz</i>	$\langle \mathcal{J}_x \rangle_{xy} _{\text{phase3}}$
<i>jyph1mz</i>	$\langle \mathcal{J}_y \rangle_{xy} _{\text{phase1}}$
<i>jyph2mz</i>	$\langle \mathcal{J}_y \rangle_{xy} _{\text{phase2}}$
<i>jyph3mz</i>	$\langle \mathcal{J}_y \rangle_{xy} _{\text{phase3}}$
<i>jzph1mz</i>	$\langle \mathcal{J}_z \rangle_{xy} _{\text{phase1}}$
<i>jzph2mz</i>	$\langle \mathcal{J}_z \rangle_{xy} _{\text{phase2}}$
<i>jzph3mz</i>	$\langle \mathcal{J}_z \rangle_{xy} _{\text{phase3}}$
Module ‘ascalar.f90’	
<i>accmz</i>	$\langle c \rangle_{xy}$
Module ‘bfield.f90’	
<i>bmz</i>	$\langle B \rangle_{xy}$
<i>b2mz</i>	$\langle B^2 \rangle_{xy}$
<i>bxmz</i>	$\langle B_x \rangle_{xy}$
<i>bymz</i>	$\langle B_y \rangle_{xy}$
<i>bzmz</i>	$\langle B_z \rangle_{xy}$
<i>bx2mz</i>	$\langle B_x^2 \rangle_{xy}$
<i>by2mz</i>	$\langle B_y^2 \rangle_{xy}$
<i>bz2mz</i>	$\langle B_z^2 \rangle_{xy}$
<i>bxbymz</i>	$\langle B_x B_y \rangle_{xy}$
<i>bxzbzmz</i>	$\langle B_x B_z \rangle_{xy}$
<i>byzbzmz</i>	$\langle B_y B_z \rangle_{xy}$
<i>betamz</i>	$\langle \beta \rangle_{xy}$
<i>beta2mz</i>	$\langle \beta^2 \rangle_{xy}$
Module ‘cosmicray_nolog.f90’	
<i>ecrmz</i>	$\langle \epsilon_{cr} \rangle_{xy}$
<i>ecrph1mz</i>	$\langle \epsilon_{cr} \rangle_{xy} _{\text{phase1}}$
<i>ecrph2mz</i>	$\langle \epsilon_{cr} \rangle_{xy} _{\text{phase2}}$
<i>ecrph3mz</i>	$\langle \epsilon_{cr} \rangle_{xy} _{\text{phase3}}$
Module ‘density_stratified.f90’	
<i>drhomz</i>	$\langle \Delta \rho / \rho_0 \rangle_{xy}$
<i>drho2mz</i>	$\langle (\Delta \rho / \rho_0)^2 \rangle_{xy}$
Module ‘disp_current.f90’	
<i>exmz</i>	$\langle \mathcal{E}_x \rangle_{xy}$
<i>eymz</i>	$\langle \mathcal{E}_y \rangle_{xy}$
<i>ezmz</i>	$\langle \mathcal{E}_z \rangle_{xy}$
<i>e2mz</i>	$\langle \mathbf{E}^2 \rangle_{xy}$
Module ‘electroweaksu2.f90’	
<i>W1xmz</i>	$\langle \mathcal{W}_x^1 \rangle_{xy}$
<i>W1ymz</i>	$\langle \mathcal{W}_y^1 \rangle_{xy}$

$W1zmz$	$\langle \mathcal{W}_z^1 \rangle_{xy}$
$W2xmz$	$\langle \mathcal{W}_x^2 \rangle_{xy}$
$W2ymz$	$\langle \mathcal{W}_y^2 \rangle_{xy}$
$W2zmz$	$\langle \mathcal{W}_z^2 \rangle_{xy}$
$W3xmz$	$\langle \mathcal{W}_x^3 \rangle_{xy}$
$W3ymz$	$\langle \mathcal{W}_y^3 \rangle_{xy}$
$W3zmz$	$\langle \mathcal{W}_z^3 \rangle_{xy}$
$dW1xmz$	$\langle \dot{\mathcal{W}}_x^1 \rangle_{xy}$
$dW1ymz$	$\langle \dot{\mathcal{W}}_y^1 \rangle_{xy}$
$dW1zmz$	$\langle \dot{\mathcal{W}}_z^1 \rangle_{xy}$
$dW2xmz$	$\langle \dot{\mathcal{W}}_x^2 \rangle_{xy}$
$dW2ymz$	$\langle \dot{\mathcal{W}}_y^2 \rangle_{xy}$
$dW2zmz$	$\langle \dot{\mathcal{W}}_z^2 \rangle_{xy}$
$dW3xmz$	$\langle \dot{\mathcal{W}}_x^3 \rangle_{xy}$
$dW3ymz$	$\langle \dot{\mathcal{W}}_y^3 \rangle_{xy}$
$dW3zmz$	$\langle \dot{\mathcal{W}}_z^3 \rangle_{xy}$

Module 'gravity_simple.f90'

$epotmz$	$\langle \varrho \Phi_{\text{grav}} \rangle_{xy}$
$epotuzmz$	$\langle \varrho \Phi_{\text{grav}} u_z \rangle_{xy}$ (potential energy flux)

Module 'hydro_potential.f90'

$u2mz$	$\langle \mathbf{u}^2 \rangle_{xy}$
$o2mz$	$\langle \mathbf{W}^2 \rangle_{xy}$
$divu2mz$	$\langle (\nabla \cdot \mathbf{u})^2 \rangle_{xy}$
$curlru2mz$	$\langle (\nabla \times \varrho \mathbf{U})^2 \rangle_{xy}$
$divru2mz$	$\langle (\nabla \cdot \varrho \mathbf{u})^2 \rangle_{xy}$
$fmasszmz$	$\langle \varrho u_z \rangle_{xy}$
$fkinzmz$	$\langle \frac{1}{2} \varrho \mathbf{u}^2 u_z \rangle_{xy}$
$uxmz$	$\langle u_x \rangle_{xy}$ (horiz. averaged x velocity)
$uymz$	$\langle u_y \rangle_{xy}$
$uzmz$	$\langle u_z \rangle_{xy}$
$uzupmz$	$\langle u_{z\uparrow} \rangle_{xy}$
$uzdownmz$	$\langle u_{z\downarrow} \rangle_{xy}$
$ruzupmz$	$\langle \varrho u_{z\uparrow} \rangle_{xy}$
$ruzdownmz$	$\langle \varrho u_{z\downarrow} \rangle_{xy}$
$divumz$	$\langle \text{div} \mathbf{u} \rangle_{xy}$
$uzdivumz$	$\langle u_z \text{div} \mathbf{u} \rangle_{xy}$
$oxmz$	$\langle \omega_x \rangle_{xy}$
$oymz$	$\langle \omega_y \rangle_{xy}$
$ozmz$	$\langle \omega_z \rangle_{xy}$
$ux2mz$	$\langle u_x^2 \rangle_{xy}$
$uy2mz$	$\langle u_y^2 \rangle_{xy}$

uz2mz	$\langle u_z^2 \rangle_{xy}$
ox2mz	$\langle \omega_x^2 \rangle_{xy}$
oy2mz	$\langle \omega_y^2 \rangle_{xy}$
oz2mz	$\langle \omega_z^2 \rangle_{xy}$
ruxmz	$\langle \rho u_x \rangle_{xy}$
ruymz	$\langle \rho u_y \rangle_{xy}$
ruzmmz	$\langle \rho u_z \rangle_{xy}$
rux2mz	$\langle \rho u_x^2 \rangle_{xy}$
ruy2mz	$\langle \rho u_y^2 \rangle_{xy}$
ruz2mz	$\langle \rho u_z^2 \rangle_{xy}$
uxuymz	$\langle u_x u_y \rangle_{xy}$
uxuzmmz	$\langle u_x u_z \rangle_{xy}$
uyuzmmz	$\langle u_y u_z \rangle_{xy}$
ruxuymz	$\langle \rho u_x u_y \rangle_{xy}$
ruxuzmmz	$\langle \rho u_x u_z \rangle_{xy}$
ruyuzmmz	$\langle \rho u_y u_z \rangle_{xy}$
ruxuy2mz	$\langle \rho u_x u_y \rangle_{xy}$
ruxuz2mz	$\langle \rho u_x u_z \rangle_{xy}$
ruyuz2mz	$\langle \rho u_y u_z \rangle_{xy}$
oxuxxmz	$\langle \omega_x u_{x,x} \rangle_{xy}$
oyuxymz	$\langle \omega_y u_{x,y} \rangle_{xy}$
oxuyxmz	$\langle \omega_x u_{y,x} \rangle_{xy}$
oyuyymz	$\langle \omega_y u_{y,y} \rangle_{xy}$
oxuzxmz	$\langle \omega_x u_{z,x} \rangle_{xy}$
oyuzymz	$\langle \omega_y u_{z,y} \rangle_{xy}$
uyxuzxmz	$\langle u_{y,x} u_{z,x} \rangle_{xy}$
uyyuzymz	$\langle u_{y,y} u_{z,y} \rangle_{xy}$
uyzuzzmmz	$\langle u_{y,z} u_{z,z} \rangle_{xy}$
ekinmz	$\langle \frac{1}{2} \rho \mathbf{u}^2 \rangle_{xy}$
oummz	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_{xy}$
Remz	$\langle \frac{ \mathbf{u} \cdot \mathbf{u} }{\left \frac{\partial}{\partial x_j} (\nu S_{ij}) \right } \rangle_{xy}$
oguxmmz	$\langle (\boldsymbol{\omega} \cdot \nabla \mathbf{u})_x \rangle_{xy}$
oguymmz	$\langle (\boldsymbol{\omega} \cdot \nabla \mathbf{u})_y \rangle_{xy}$
oguzmmz	$\langle (\boldsymbol{\omega} \cdot \nabla \mathbf{u})_z \rangle_{xy}$
ogux2mmz	$\langle (\boldsymbol{\omega} \cdot \nabla \mathbf{u})_x^2 \rangle_{xy}$
oguy2mmz	$\langle (\boldsymbol{\omega} \cdot \nabla \mathbf{u})_y^2 \rangle_{xy}$
oguz2mmz	$\langle (\boldsymbol{\omega} \cdot \nabla \mathbf{u})_z^2 \rangle_{xy}$
oxdivumz	$\langle \omega_x \nabla \cdot \mathbf{u} \rangle_{xy}$
oydivumz	$\langle \omega_y \nabla \cdot \mathbf{u} \rangle_{xy}$
ozdivumz	$\langle \omega_z \nabla \cdot \mathbf{u} \rangle_{xy}$
oxdivu2mmz	$\langle (\omega_x \text{nabla} \cdot \mathbf{u})^2 \rangle_{xy}$
oydivu2mmz	$\langle (\omega_y \nabla \cdot \mathbf{u})^2 \rangle_{xy}$
ozdivu2mmz	$\langle (\omega_z \nabla \cdot \mathbf{u})^2 \rangle_{xy}$
accpowzmmz	$\langle (u_z Du_z / Dt)^2 \rangle_{xy}$
accpowzupmmz	$\langle (u_z Du_z / Dt)^2 \rangle_{xy+}$
accpowzdownmmz	$\langle (u_z Du_z / Dt)^2 \rangle_{xy-}$

Module 'interstellar.f90'	
<i>rhoHCmz</i>	$\langle \rho \Gamma - \rho^2 \Lambda \rangle_{xy}$
Module 'magnetic_shearboxJ.f90'	
<i>axmz</i>	$\langle \mathcal{A}_x \rangle_{xy}$
<i>aymz</i>	$\langle \mathcal{A}_y \rangle_{xy}$
<i>azmz</i>	$\langle \mathcal{A}_z \rangle_{xy}$
<i>abuxmz</i>	$\langle (\mathbf{A} \cdot \mathbf{B}) u_x \rangle_{xy}$
<i>abuymz</i>	$\langle (\mathbf{A} \cdot \mathbf{B}) u_y \rangle_{xy}$
<i>abuzmz</i>	$\langle (\mathbf{A} \cdot \mathbf{B}) u_z \rangle_{xy}$
<i>uabxmz</i>	$\langle (\mathbf{u} \cdot \mathbf{A}) B_x \rangle_{xy}$
<i>uabymz</i>	$\langle (\mathbf{u} \cdot \mathbf{A}) B_y \rangle_{xy}$
<i>uabzmz</i>	$\langle (\mathbf{u} \cdot \mathbf{A}) B_z \rangle_{xy}$
<i>bbxmz</i>	$\langle \mathcal{B}'_x \rangle_{xy}$
<i>bbymz</i>	$\langle \mathcal{B}'_y \rangle_{xy}$
<i>bbzmz</i>	$\langle \mathcal{B}'_z \rangle_{xy}$
<i>bxmz</i>	$\langle \mathcal{B}_x \rangle_{xy}$
<i>bymz</i>	$\langle \mathcal{B}_y \rangle_{xy}$
<i>bzmz</i>	$\langle \mathcal{B}_z \rangle_{xy}$
<i>jxmz</i>	$\langle \mathcal{J}_x \rangle_{xy}$
<i>jymz</i>	$\langle \mathcal{J}_y \rangle_{xy}$
<i>jzmz</i>	$\langle \mathcal{J}_z \rangle_{xy}$
<i>Exmz</i>	$\langle \mathcal{E}_x \rangle_{xy}$
<i>Eymz</i>	$\langle \mathcal{E}_y \rangle_{xy}$
<i>Ezmz</i>	$\langle \mathcal{E}_z \rangle_{xy}$
<i>bx2mz</i>	$\langle B_x^2 \rangle_{xy}$
<i>by2mz</i>	$\langle B_y^2 \rangle_{xy}$
<i>bz2mz</i>	$\langle B_z^2 \rangle_{xy}$
<i>bx2rmz</i>	$\langle B_x^2 / \varrho \rangle_{xy}$
<i>by2rmz</i>	$\langle B_y^2 / \varrho \rangle_{xy}$
<i>bz2rmz</i>	$\langle B_z^2 / \varrho \rangle_{xy}$
<i>beta1mz</i>	$\langle (B^2 / 2\mu_0) / p \rangle_{xy}$
<i>betamz</i>	$\langle \beta \rangle_{xy}$
<i>beta2mz</i>	$\langle \beta^2 \rangle_{xy}$
<i>jbmz</i>	$\langle \mathbf{J} \cdot \mathbf{B} \rangle _{xy}$
<i>d6abmz</i>	$\langle \nabla^6 \mathbf{A} \cdot \mathbf{B} \rangle _{xy}$
<i>d6amz1</i>	$\langle \nabla^6 \mathbf{A} \rangle_{xy} _x$
<i>d6amz2</i>	$\langle \nabla^6 \mathbf{A} \rangle_{xy} _y$
<i>d6amz3</i>	$\langle \nabla^6 \mathbf{A} \rangle_{xy} _z$
<i>abmz</i>	$\langle \mathbf{A} \cdot \mathbf{B} \rangle _{xy}$
<i>ubmz</i>	$\langle \mathbf{u} \cdot \mathbf{B} \rangle _{xy}$
<i>uamz</i>	$\langle \mathbf{u} \cdot \mathbf{A} \rangle _{xy}$
<i>uxbxmz</i>	$\langle u_x b_x \rangle _{xy}$
<i>uybxmz</i>	$\langle u_y b_x \rangle _{xy}$
<i>uzbxmz</i>	$\langle u_z b_x \rangle _{xy}$
<i>uxbymz</i>	$\langle u_x b_y \rangle _{xy}$
<i>uybymz</i>	$\langle u_y b_y \rangle _{xy}$
<i>uzbymz</i>	$\langle u_z b_y \rangle _{xy}$

<i>uxbzmz</i>	$\langle u_x b_z \rangle_{xy}$
<i>uybzmz</i>	$\langle u_y b_z \rangle_{xy}$
<i>uzbzmz</i>	$\langle u_z b_z \rangle_{xy}$
<i>examz1</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_{xy} x$
<i>examz2</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_{xy} y$
<i>examz3</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_{xy} z$
<i>e3xamz1</i>	$\langle \mathbf{E}_{hyper3} \times \mathbf{A} \rangle_{xy} x$
<i>e3xamz2</i>	$\langle \mathbf{E}_{hyper3} \times \mathbf{A} \rangle_{xy} y$
<i>e3xamz3</i>	$\langle \mathbf{E}_{hyper3} \times \mathbf{A} \rangle_{xy} z$
<i>etatotalmz</i>	$\langle \eta \rangle_{xy}$
<i>bxbymz</i>	$\langle B_x B_y \rangle_{xy}$
<i>bxbzmz</i>	$\langle B_x B_z \rangle_{xy}$
<i>bybzmz</i>	$\langle B_y B_z \rangle_{xy}$
<i>b2mz</i>	$\langle \mathbf{B}^2 \rangle_{xy}$
<i>bf2mz</i>	$\langle \mathbf{B}'^2 \rangle_{xy}$
<i>j2mz</i>	$\langle \mathbf{j}^2 \rangle_{xy}$
<i>poynzmz</i>	Averaged poynting flux in z direction
<i>epsMmz</i>	$\langle \eta \mu_0 \mathbf{j}^2 \rangle_{xy}$

Module ‘meanfield.f90’

<i>qpmz</i>	$\langle q_p \rangle_{xy}$
-------------	----------------------------

Module ‘shock_highorder.f90’

Module ‘temperature_idealgas.f90’

<i>ppmz</i>	$\langle p \rangle_{xy}$
<i>TTmz</i>	$\langle T \rangle_{xy}$
<i>ethmz</i>	$\langle e_{th} \rangle_{xy}$
<i>fpresxmz</i>	$\langle (\nabla p)_x \rangle_{xy}$
<i>fpresymz</i>	$\langle (\nabla p)_y \rangle_{xy}$
<i>fpreszmz</i>	$\langle (\nabla p)_z \rangle_{xy}$
<i>TT2mz</i>	$\langle T^2 \rangle_{xy}$
<i>uxTmz</i>	$\langle u_x T \rangle_{xy}$
<i>uyTmz</i>	$\langle u_y T \rangle_{xy}$
<i>uzTmz</i>	$\langle u_z T \rangle_{xy}$
<i>fradmz</i>	$\langle F_{rad} \rangle_{xy}$
<i>fconvmz</i>	$\langle c_p \rho u_z T \rangle_{xy}$

Module ‘temperature_ionization.f90’

<i>puzmz</i>	$\langle p u_z \rangle_{xy}$
<i>pr1mz</i>	$\langle p / \rho \rangle_{xy}$
<i>eruzmz</i>	$\langle e \rho u_z \rangle_{xy}$
<i>ffakez</i>	$\langle \rho u_z c_p T \rangle_{xy}$
<i>mumz</i>	$\langle \mu \rangle_{xy}$
<i>TTmz</i>	$\langle T \rangle_{xy}$
<i>ssmz</i>	$\langle s \rangle_{xy}$
<i>eemz</i>	$\langle e \rangle_{xy}$
<i>ppmz</i>	$\langle p \rangle_{xy}$

Module ‘thermal_energy.f90’

<i>ppmz</i>	$\langle p \rangle_{xy}$
<i>TTmz</i>	$\langle T \rangle_{xy}$

Module ‘viscosity.f90’

<i>fviscmz</i>	$\langle 2\nu \varrho u_i \mathcal{S}_{iz} \rangle_{xy}$ (<i>z</i> -component of viscous flux)
<i>fviscsmmz</i>	$\langle 2\nu_{\text{Smag}} \varrho u_i \mathcal{S}_{iz} \rangle_{xy}$ (<i>z</i> -component of viscous flux)
<i>epsKmz</i>	$\langle 2\nu \varrho \mathbf{S}^2 \rangle_{xy}$
<i>sijxxmz</i>	$\langle \mathbf{S}_{xx} \rangle_{xy}$
<i>sijxymz</i>	$\langle \mathbf{S}_{xy} \rangle_{xy}$
<i>sijxzmz</i>	$\langle \mathbf{S}_{xz} \rangle_{xy}$
<i>sijyymz</i>	$\langle \mathbf{S}_{yy} \rangle_{xy}$
<i>sijyzmz</i>	$\langle \mathbf{S}_{yz} \rangle_{xy}$
<i>sijzzmz</i>	$\langle \mathbf{S}_{zz} \rangle_{xy}$
<i>viscforcezmz</i>	$\langle (\varrho \mathbf{f}_{\text{visc}})_z \rangle_{xy}$
<i>viscforcezupmz</i>	$\langle (\varrho \mathbf{f}_{\text{visc}})_z \rangle_{xy+}$
<i>viscforcezdownmz</i>	$\langle (\varrho \mathbf{f}_{\text{visc}})_z \rangle_{xy-}$

K.8 List of parameters for ‘xzaver.in’

The following table lists possible inputs to the file ‘xzaver.in’. This list is not complete and maybe outdated.

<i>Variable</i>	<i>Meaning</i>
Module ‘hydro.f90’	
<i>uxmy</i>	$\langle u_x \rangle_{xz}$
<i>uymy</i>	$\langle u_y \rangle_{xz}$
<i>uzmy</i>	$\langle u_z \rangle_{xz}$
<i>oumy</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_{xz}$
Module ‘density.f90’	
<i>rhomy</i>	$\langle \varrho \rangle_{xz}$
Module ‘entropy.f90’	
<i>ssmy</i>	$\langle s \rangle_{xz}$
<i>ppmy</i>	$\langle p \rangle_{xz}$
<i>TTmy</i>	$\langle T \rangle_{xz}$
Module ‘magnetic.f90’	
<i>bxmy</i>	$\langle B_x \rangle_{xz}$
<i>bymy</i>	$\langle B_y \rangle_{xz}$
<i>bzmy</i>	$\langle B_z \rangle_{xz}$
<i>bx2my</i>	$\langle B_x^2 \rangle_{xz}$
<i>by2my</i>	$\langle B_y^2 \rangle_{xz}$
<i>bz2my</i>	$\langle B_z^2 \rangle_{xz}$
<i>bxbmy</i>	$\langle B_x B_y \rangle_{xz}$
<i>bxbzmy</i>	$\langle B_x B_z \rangle_{xz}$

<i>bybzm</i>	$\langle B_y B_z \rangle_{xz}$
Module ‘density_stratified.f90’	
<i>drhom</i>	$\langle \Delta \rho / \rho_0 \rangle_{xz}$
<i>drho2m</i>	$\langle (\Delta \rho / \rho_0)^2 \rangle_{xz}$
Module ‘gravity_simple.f90’	
<i>epotm</i>	$\langle \rho \Phi_{\text{grav}} \rangle_{xz}$
Module ‘hydro_potential.f90’	
<i>uxm</i>	$\langle u_x \rangle_{xz}$
<i>uym</i>	$\langle u_y \rangle_{xz}$
<i>uzm</i>	$\langle u_z \rangle_{xz}$
<i>oum</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_{xz}$
Module ‘magnetic_shearboxJ.f90’	
<i>bxm</i>	$\langle B_x \rangle_{xz}$
<i>bym</i>	$\langle B_y \rangle_{xz}$
<i>bzm</i>	$\langle B_z \rangle_{xz}$
<i>bx2m</i>	$\langle B_x^2 \rangle_{xz}$
<i>by2m</i>	$\langle B_y^2 \rangle_{xz}$
<i>bz2m</i>	$\langle B_z^2 \rangle_{xz}$
<i>bxbym</i>	$\langle B_x B_y \rangle_{xz}$
<i>bxbz</i>	$\langle B_x B_z \rangle_{xz}$
<i>bybz</i>	$\langle B_y B_z \rangle_{xz}$
Module ‘shock_highorder.f90’	
Module ‘temperature_idealgas.f90’	
<i>ppm</i>	$\langle p \rangle_{xz}$
<i>TTm</i>	$\langle T \rangle_{xz}$
Module ‘thermal_energy.f90’	
<i>ppm</i>	$\langle p \rangle_{xz}$
<i>TTm</i>	$\langle T \rangle_{xz}$

K.9 List of parameters for ‘yzaver.in’

The following table lists possible inputs to the file ‘yzaver.in’. This list is not complete and maybe outdated.

<i>Variable</i>	<i>Meaning</i>
Module ‘hydro.f90’	
<i>u2mx</i>	$\langle u^2 \rangle_{yz}$
<i>uxmx</i>	$\langle u_x \rangle_{yz}$
<i>uymx</i>	$\langle u_y \rangle_{yz}$
<i>uzmx</i>	$\langle u_z \rangle_{yz}$
<i>ruvmx</i>	$\langle \rho u_x \rangle_{yz}$
<i>ruymx</i>	$\langle \rho u_y \rangle_{yz}$

<i>ruzmx</i>	$\langle \varrho u_z \rangle_{yz}$
<i>rux2mx</i>	$\langle \rho u_x^2 \rangle_{yz}$
<i>ruy2mx</i>	$\langle \rho u_y^2 \rangle_{yz}$
<i>ruz2mx</i>	$\langle \rho u_z^2 \rangle_{yz}$
<i>ruxuymx</i>	$\langle \rho u_x u_y \rangle_{yz}$
<i>ruxuzmx</i>	$\langle \rho u_x u_z \rangle_{yz}$
<i>ruyuzmx</i>	$\langle \rho u_y u_z \rangle_{yz}$
<i>ux2mx</i>	$\langle u_x^2 \rangle_{yz}$
<i>uy2mx</i>	$\langle u_y^2 \rangle_{yz}$
<i>uz2mx</i>	$\langle u_z^2 \rangle_{yz}$
<i>ox2mx</i>	$\langle \omega_x^2 \rangle_{yz}$
<i>oy2mx</i>	$\langle \omega_y^2 \rangle_{yz}$
<i>oz2mx</i>	$\langle \omega_z^2 \rangle_{yz}$
<i>uxuymx</i>	$\langle u_x u_y \rangle_{yz}$
<i>uxuzmx</i>	$\langle u_x u_z \rangle_{yz}$
<i>uyuzmx</i>	$\langle u_y u_z \rangle_{yz}$
<i>oumx</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_{yz}$
<i>ekinmx</i>	$\langle \frac{1}{2} \rho u^2 \rangle_{yz}$
<i>fkinxmx</i>	$\langle \frac{1}{2} \varrho \mathbf{u}^2 u_x \rangle_{yz}$

Module ‘density.f90’

<i>rhomx</i>	$\langle \varrho \rangle_{yz}$
--------------	--------------------------------

Module ‘entropy.f90’

<i>ssmx</i>	$\langle s \rangle_{yz}$
<i>ss2mx</i>	$\langle s^2 \rangle_{yz}$
<i>ppmx</i>	$\langle p \rangle_{yz}$
<i>TTmx</i>	$\langle T \rangle_{yz}$
<i>TT2mx</i>	$\langle T^2 \rangle_{yz}$
<i>uxTTmx</i>	$\langle u_x T \rangle_{yz}$
<i>uyTTmx</i>	$\langle u_y T \rangle_{yz}$
<i>uzTTmx</i>	$\langle u_z T \rangle_{yz}$
<i>fconvxmx</i>	$\langle c_p \varrho u_x T \rangle_{yz}$
<i>fradm</i>	$\langle F_{\text{rad}} \rangle_{yz}$ (for K-profile or constant K)
<i>fturbmx</i>	$\langle \varrho T \chi_t \nabla_x s \rangle_{yz}$ (turbulent heat flux)
<i>Kkramersmx</i>	$\langle K_0 T^{(3-b)/\text{rho}(a+1)} \rangle_{yz}$
<i>dcoolx</i>	surface cooling flux
<i>fradx_kramers</i>	F_{rad} (from Kramers’ opacity)
<i>fradx_constchi</i>	$\langle F_{\text{rad}} \rangle_{yz}$ (for chi-const)

Module ‘magnetic.f90’

<i>b2mx</i>	$\langle B^2 \rangle_{yz}$
<i>j2mx</i>	$\langle J^2 \rangle_{yz}$
<i>jbmx</i>	$\langle \mathbf{J} \cdot \mathbf{B} \rangle_{yz}$
<i>b2mmx</i>	$\langle B^2 \rangle_{yz, \text{mask}}$
<i>bxmx</i>	$\langle B_x \rangle_{yz}$
<i>bymx</i>	$\langle B_y \rangle_{yz}$
<i>bzmx</i>	$\langle B_z \rangle_{yz}$
<i>bx2mx</i>	$\langle B_x^2 \rangle_{yz}$

<i>by2mx</i>	$\langle B_y^2 \rangle_{yz}$
<i>bz2mx</i>	$\langle B_z^2 \rangle_{yz}$
<i>bxbymx</i>	$\langle B_x B_y \rangle_{yz}$
<i>bxbzmx</i>	$\langle B_x B_z \rangle_{yz}$
<i>bybzm</i>	$\langle B_y B_z \rangle_{yz}$
<i>betamx</i>	$\langle \beta \rangle_{yz}$
<i>beta2mx</i>	$\langle \beta^2 \rangle_{yz}$
<i>etatotalmx</i>	$\langle \eta \rangle_{yz}$
Module ‘bfield.f90’	
<i>bmx</i>	$\langle B \rangle_{yz}$
<i>b2mx</i>	$\langle B^2 \rangle_{yz}$
<i>bxmx</i>	$\langle B_x \rangle_{yz}$
<i>bymx</i>	$\langle B_y \rangle_{yz}$
<i>bzmx</i>	$\langle B_z \rangle_{yz}$
<i>bx2mx</i>	$\langle B_x^2 \rangle_{yz}$
<i>by2mx</i>	$\langle B_y^2 \rangle_{yz}$
<i>bz2mx</i>	$\langle B_z^2 \rangle_{yz}$
<i>bxbymx</i>	$\langle B_x B_y \rangle_{yz}$
<i>bxbzmx</i>	$\langle B_x B_z \rangle_{yz}$
<i>bybzm</i>	$\langle B_y B_z \rangle_{yz}$
<i>betamx</i>	$\langle \beta \rangle_{yz}$
<i>beta2mx</i>	$\langle \beta^2 \rangle_{yz}$
Module ‘density_stratified.f90’	
<i>drhomx</i>	$\langle \Delta \rho / \rho_0 \rangle_{yz}$
<i>drho2mx</i>	$\langle (\Delta \rho / \rho_0)^2 \rangle_{yz}$
Module ‘disp_current.f90’	
<i>e2mx</i>	$\langle E^2 \rangle_{yz}$
Module ‘gravity_simple.f90’	
<i>epotmx</i>	$\langle \rho \Phi_{\text{grav}} \rangle_{yz}$
<i>epotuxmx</i>	$\langle \rho \Phi_{\text{grav}} u_x \rangle_{yz}$ (potential energy flux)
Module ‘hydro_potential.f90’	
<i>uxmx</i>	$\langle u_x \rangle_{yz}$
<i>uymx</i>	$\langle u_y \rangle_{yz}$
<i>uzmx</i>	$\langle u_z \rangle_{yz}$
<i>ruymx</i>	$\langle \rho u_x \rangle_{yz}$
<i>ruymx</i>	$\langle \rho u_y \rangle_{yz}$
<i>ruzmx</i>	$\langle \rho u_z \rangle_{yz}$
<i>ruy2mx</i>	$\langle \rho u_x^2 \rangle_{yz}$
<i>ruy2mx</i>	$\langle \rho u_y^2 \rangle_{yz}$
<i>ruz2mx</i>	$\langle \rho u_z^2 \rangle_{yz}$
<i>ruyuymx</i>	$\langle \rho u_x u_y \rangle_{yz}$
<i>ruyuzmx</i>	$\langle \rho u_x u_z \rangle_{yz}$
<i>ruyuzmx</i>	$\langle \rho u_y u_z \rangle_{yz}$
<i>ux2mx</i>	$\langle u_x^2 \rangle_{yz}$
<i>uy2mx</i>	$\langle u_y^2 \rangle_{yz}$

<i>uz2mx</i>	$\langle u_z^2 \rangle_{yz}$
<i>ox2mx</i>	$\langle \omega_x^2 \rangle_{yz}$
<i>oy2mx</i>	$\langle \omega_y^2 \rangle_{yz}$
<i>oz2mx</i>	$\langle \omega_z^2 \rangle_{yz}$
<i>uxuymx</i>	$\langle u_x u_y \rangle_{yz}$
<i>uxuzmx</i>	$\langle u_x u_z \rangle_{yz}$
<i>uyuzmx</i>	$\langle u_y u_z \rangle_{yz}$
<i>oumx</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_{yz}$
<i>ekinmx</i>	$\langle \frac{1}{2} \rho u^2 \rangle_{yz}$
<i>fkinxmx</i>	$\langle \frac{1}{2} \rho \mathbf{u}^2 u_x \rangle_{yz}$
Module ‘magnetic_shearboxJ.f90’	
<i>b2mx</i>	$\langle B^2 \rangle_{yz}$
<i>bxmx</i>	$\langle B_x \rangle_{yz}$
<i>bymx</i>	$\langle B_y \rangle_{yz}$
<i>bzmx</i>	$\langle B_z \rangle_{yz}$
<i>bx2mx</i>	$\langle B_x^2 \rangle_{yz}$
<i>by2mx</i>	$\langle B_y^2 \rangle_{yz}$
<i>bz2mx</i>	$\langle B_z^2 \rangle_{yz}$
<i>bxbymx</i>	$\langle B_x B_y \rangle_{yz}$
<i>bxbzmx</i>	$\langle B_x B_z \rangle_{yz}$
<i>bybzmx</i>	$\langle B_y B_z \rangle_{yz}$
<i>betamx</i>	$\langle \beta \rangle_{yz}$
<i>beta2mx</i>	$\langle \beta^2 \rangle_{yz}$
<i>etatotalmx</i>	$\langle \eta \rangle_{yz}$
Module ‘shock_highorder.f90’	
Module ‘temperature_idealgas.f90’	
<i>ppmx</i>	$\langle p \rangle_{yz}$
<i>TTmx</i>	$\langle T \rangle_{yz}$
Module ‘thermal_energy.f90’	
<i>ppmx</i>	$\langle p \rangle_{yz}$
<i>TTmx</i>	$\langle T \rangle_{yz}$
Module ‘viscosity.f90’	
<i>fviscmx</i>	$\langle 2\nu \varrho u_i \mathcal{S}_{ix} \rangle_{yz}$ (<i>x</i> -component of viscous flux)
<i>numx</i>	$\langle \nu \rangle_{yz}$ (<i>yz</i> -averaged viscosity)

K.10 List of parameters for ‘yaver.in’

The following table lists possible inputs to the file ‘yaver.in’. This list is not complete and maybe outdated.

<i>Variable</i>	<i>Meaning</i>
Module ‘hydro.f90’	

<i>uxmxz</i>	$\langle u_x \rangle_y$
<i>uymxz</i>	$\langle u_y \rangle_y$
<i>uzmxz</i>	$\langle u_z \rangle_y$
<i>ux2mxz</i>	$\langle u_x^2 \rangle_y$
<i>uy2mxz</i>	$\langle u_y^2 \rangle_y$
<i>uz2mxz</i>	$\langle u_z^2 \rangle_y$
<i>uxuymxz</i>	$\langle u_x u_y \rangle_y$
<i>uxuzmxz</i>	$\langle u_x u_z \rangle_y$
<i>uyuzmxz</i>	$\langle u_y u_z \rangle_y$
<i>oumxz</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_y$
<i>ox2mxz</i>	$\langle \omega_x^2 \rangle_y$
<i>oy2mxz</i>	$\langle \omega_y^2 \rangle_y$
<i>oz2mxz</i>	$\langle \omega_z^2 \rangle_y$
<i>oymxz</i>	$\langle \omega_y \rangle_y$
Module ‘density.f90’	
<i>rhomxz</i>	$\langle \rho \rangle_y$
Module ‘entropy.f90’	
<i>TTmxz</i>	$\langle T \rangle_y$
<i>ssmxz</i>	$\langle s \rangle_y$
Module ‘magnetic.f90’	
<i>b2mxz</i>	$\langle \mathbf{B}^2 \rangle_y$
<i>axmxz</i>	$\langle A_x \rangle_y$
<i>aymxz</i>	$\langle A_y \rangle_y$
<i>azmxz</i>	$\langle A_z \rangle_y$
<i>bx1mxz</i>	$\langle B_x \rangle_y$
<i>by1mxz</i>	$\langle B_y \rangle_y$
<i>bz1mxz</i>	$\langle B_z \rangle_y$
<i>bxmxz</i>	$\langle B_x \rangle_y$
<i>bymxz</i>	$\langle B_y \rangle_y$
<i>bzmxz</i>	$\langle B_z \rangle_y$
<i>jxmxz</i>	$\langle J_x \rangle_y$
<i>jymxz</i>	$\langle J_y \rangle_y$
<i>jzmxz</i>	$\langle J_z \rangle_y$
<i>bx2mxz</i>	$\langle B_x^2 \rangle_y$
<i>by2mxz</i>	$\langle B_y^2 \rangle_y$
<i>bz2mxz</i>	$\langle B_z^2 \rangle_y$
<i>bxbymxz</i>	$\langle B_x B_y \rangle_y$
<i>bxbxmxz</i>	$\langle B_x B_z \rangle_y$
<i>bybxmxz</i>	$\langle B_y B_z \rangle_y$
<i>uybxmxz</i>	$\langle U_y B_x \rangle_y$
<i>uybxmxz</i>	$\langle U_y B_z \rangle_y$
<i>Exmxz</i>	$\langle \mathcal{E}_x \rangle_y$
<i>Eymxz</i>	$\langle \mathcal{E}_y \rangle_y$
<i>Ezmxz</i>	$\langle \mathcal{E}_z \rangle_y$

<i>vAmxz</i>	$\langle v_A^2 \rangle_y$
Module ‘density_stratified.f90’	
<i>drhomxz</i>	$\langle \Delta \rho / \rho_0 \rangle_y$
<i>drho2mxz</i>	$\langle (\Delta \rho / \rho_0)^2 \rangle_y$
Module ‘hydro_potential.f90’	
<i>uxmxz</i>	$\langle u_x \rangle_y$
<i>uymxz</i>	$\langle u_y \rangle_y$
<i>uzmxz</i>	$\langle u_z \rangle_y$
<i>ux2mxz</i>	$\langle u_x^2 \rangle_y$
<i>uy2mxz</i>	$\langle u_y^2 \rangle_y$
<i>uz2mxz</i>	$\langle u_z^2 \rangle_y$
<i>uxuymxz</i>	$\langle u_x u_y \rangle_y$
<i>uxuzmxz</i>	$\langle u_x u_z \rangle_y$
<i>uyuzmxz</i>	$\langle u_y u_z \rangle_y$
<i>oumxz</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_y$
Module ‘magnetic_shearboxJ.f90’	
<i>b2mxz</i>	$\langle B^2 \rangle_y$
<i>axmxz</i>	$\langle A_x \rangle_y$
<i>aymxz</i>	$\langle A_y \rangle_y$
<i>azmxz</i>	$\langle A_z \rangle_y$
<i>bx1mxz</i>	$\langle B_x \rangle_y$
<i>by1mxz</i>	$\langle B_y \rangle_y$
<i>bz1mxz</i>	$\langle B_z \rangle_y$
<i>bxmxz</i>	$\langle B_x \rangle_y$
<i>bymxz</i>	$\langle B_y \rangle_y$
<i>bzmxz</i>	$\langle B_z \rangle_y$
<i>jxmxz</i>	$\langle J_x \rangle_y$
<i>jymxz</i>	$\langle J_y \rangle_y$
<i>jzmxz</i>	$\langle J_z \rangle_y$
<i>bx2mxz</i>	$\langle B_x^2 \rangle_y$
<i>by2mxz</i>	$\langle B_y^2 \rangle_y$
<i>bz2mxz</i>	$\langle B_z^2 \rangle_y$
<i>bxbymxz</i>	$\langle B_x B_y \rangle_y$
<i>bxbzmxz</i>	$\langle B_x B_z \rangle_y$
<i>bybzmzx</i>	$\langle B_y B_z \rangle_y$
<i>uybxmxz</i>	$\langle U_y B_x \rangle_y$
<i>uybzmzx</i>	$\langle U_y B_z \rangle_y$
<i>Exmxz</i>	$\langle \mathcal{E}_x \rangle_y$
<i>Eymxz</i>	$\langle \mathcal{E}_y \rangle_y$
<i>Ezmxz</i>	$\langle \mathcal{E}_z \rangle_y$
<i>vAmxz</i>	$\langle v_A^2 \rangle_y$
Module ‘meanfield.f90’	
<i>peffmxz</i>	$\langle \mathcal{P}_{\text{eff}} \rangle_y$
<i>alpmxz</i>	$\langle \alpha \rangle_y$

Module ‘temperature_idealgas.f90’

<i>TTmxz</i>	$\langle T \rangle_y$
<i>EmAIA94mxz</i>	Emission off AIA 94 channel integrated over y direction
<i>EmAIA131mxz</i>	Emission off AIA 131 channel integrated over y direction
<i>EmAIA171mxz</i>	Emission off AIA 171 channel integrated over y direction
<i>EmAIA193mxz</i>	Emission off AIA 193 channel integrated over y direction
<i>EmAIA211mxz</i>	Emission off AIA 211 channel integrated over y direction
<i>EmAIA304mxz</i>	Emission off AIA 304 channel integrated over y direction
<i>EmAIA335mxz</i>	Emission off AIA 335 channel integrated over y direction
<i>EmXRTmxz</i>	Emission off XRT Al-poly channel integrated over y direction

Module ‘thermal_energy.f90’

<i>TTmxz</i>	$\langle T \rangle_y$
--------------	-----------------------

K.11 List of parameters for ‘zaver.in’

The following table lists possible inputs to the file ‘zaver.in’. This list is not complete and maybe outdated.

<i>Variable</i>	<i>Meaning</i>
Module ‘hydro.f90’	
<i>uxmxy</i>	$\langle u_x \rangle_z$
<i>uymxy</i>	$\langle u_y \rangle_z$
<i>uzmxy</i>	$\langle u_z \rangle_z$
<i>uxupmxy</i>	$\langle u_{x\uparrow} \rangle_z$
<i>uxdownmxy</i>	$\langle u_{x\downarrow} \rangle_z$
<i>ruxupmxy</i>	$\langle \rho u_{x\uparrow} \rangle_z$
<i>ruxdownmxy</i>	$\langle \rho u_{x\downarrow} \rangle_z$
<i>ux2upmxy</i>	$\langle u_{x\uparrow}^2 \rangle_z$
<i>ux2downmxy</i>	$\langle u_{x\downarrow}^2 \rangle_z$
<i>ffdownmxy</i>	Filling factor of downflows
<i>uxuymxy</i>	$\langle u_x u_y \rangle_z$
<i>uxuzmxy</i>	$\langle u_x u_z \rangle_z$
<i>uyuzmxy</i>	$\langle u_y u_z \rangle_z$
<i>Rxymxy</i>	$\langle u'_x u'_y \rangle_z$
<i>Rxyupmxy</i>	$\langle (u'_x u'_y)_{\uparrow} \rangle_z$
<i>Rxydownmxy</i>	$\langle (u'_x u'_y)_{\downarrow} \rangle_z$
<i>Rxzmxy</i>	$\langle u'_x u'_z \rangle_z$
<i>Rxzupmxy</i>	$\langle (u'_x u'_z)_{\uparrow} \rangle_z$
<i>Rxzdownmxy</i>	$\langle (u'_x u'_z)_{\downarrow} \rangle_z$
<i>Ryzmxy</i>	$\langle u'_y u'_z \rangle_z$
<i>Ryzupmxy</i>	$\langle (u'_y u'_z)_{\uparrow} \rangle_z$
<i>Ryzdownmxy</i>	$\langle (u'_y u'_z)_{\downarrow} \rangle_z$
<i>oxmxy</i>	$\langle \omega_x \rangle_z$
<i>oymxy</i>	$\langle \omega_y \rangle_z$
<i>ozmxy</i>	$\langle \omega_z \rangle_z$
<i>oumxy</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_z$

<i>pvzmxxy</i>	$\langle (\omega_z + 2\Omega)/\varrho \rangle_z$ (z component of potential vorticity)
<i>uguxmxy</i>	$\langle (\mathbf{u} \cdot \nabla \mathbf{u})_x \rangle_z$
<i>uguymxy</i>	$\langle (\mathbf{u} \cdot \nabla \mathbf{u})_y \rangle_z$
<i>uguzmxy</i>	$\langle (\mathbf{u} \cdot \nabla \mathbf{u})_z \rangle_z$
<i>ruxmxy</i>	$\langle \rho u_x \rangle_z$
<i>ruymxy</i>	$\langle \rho u_y \rangle_z$
<i>ruzmxxy</i>	$\langle \rho u_z \rangle_z$
<i>ux2mxy</i>	$\langle u_x^2 \rangle_z$
<i>uy2mxy</i>	$\langle u_y^2 \rangle_z$
<i>uz2mxy</i>	$\langle u_z^2 \rangle_z$
<i>ox2mxy</i>	$\langle \omega_x^2 \rangle_z$
<i>oy2mxy</i>	$\langle \omega_y^2 \rangle_z$
<i>oz2mxy</i>	$\langle \omega_z^2 \rangle_z$
<i>rux2mxy</i>	$\langle \rho u_x^2 \rangle_z$
<i>ruy2mxy</i>	$\langle \rho u_y^2 \rangle_z$
<i>ruz2mxy</i>	$\langle \rho u_z^2 \rangle_z$
<i>ruxuymxy</i>	$\langle \rho u_x u_y \rangle_z$
<i>ruxuzmxy</i>	$\langle \rho u_x u_z \rangle_z$
<i>ruyuzmxy</i>	$\langle \rho u_y u_z \rangle_z$
<i>fkinxmxy</i>	$\langle \frac{1}{2} \varrho \mathbf{u}^2 u_x \rangle_z$
<i>fkinymxy</i>	$\langle \frac{1}{2} \varrho \mathbf{u}^2 u_y \rangle_z$
<i>fkinxupmxy</i>	$\langle \frac{1}{2} \varrho \mathbf{u}^2 u_{x\uparrow} \rangle_z$
<i>fkinxdowmxy</i>	$\langle \frac{1}{2} \varrho \mathbf{u}^2 u_{x\downarrow} \rangle_z$

Module ‘density.f90’

<i>rhomxy</i>	$\langle \varrho \rangle_z$
<i>rho2mxy</i>	$\langle \varrho^2 \rangle_z$

Module ‘entropy.f90’

<i>TTmxy</i>	$\langle T \rangle_z$
<i>ssmxy</i>	$\langle s \rangle_z$
<i>uxTTmxy</i>	$\langle u_x T \rangle_z$
<i>uyTTmxy</i>	$\langle u_y T \rangle_z$
<i>uzTTmxy</i>	$\langle u_z T \rangle_z$
<i>gTxmxy</i>	$\langle \nabla_x T \rangle_z$
<i>gTymxy</i>	$\langle \nabla_y T \rangle_z$
<i>gTzmxy</i>	$\langle \nabla_z T \rangle_z$
<i>gsxmxy</i>	$\langle \nabla_x s \rangle_z$
<i>gsymxy</i>	$\langle \nabla_y s \rangle_z$
<i>gszmxy</i>	$\langle \nabla_z s \rangle_z$
<i>gTxgsxmxy</i>	$\langle (\nabla T \times \nabla s)_x \rangle_z$
<i>gTxgsymxy</i>	$\langle (\nabla T \times \nabla s)_y \rangle_z$
<i>gTxgszmxy</i>	$\langle (\nabla T \times \nabla s)_z \rangle_z$
<i>gTxgsx2mxy</i>	$\langle (\nabla T \times \nabla s)_x^2 \rangle_z$
<i>gTxgsy2mxy</i>	$\langle (\nabla T \times \nabla s)_y^2 \rangle_z$
<i>gTxgsz2mxy</i>	$\langle (\nabla T \times \nabla s)_z^2 \rangle_z$
<i>fconvxy</i>	$\langle c_p \varrho u_x T \rangle_z$
<i>fconvyxy</i>	$\langle c_p \varrho u_y T \rangle_z$
<i>fconvzxy</i>	$\langle c_p \varrho u_z T \rangle_z$

<i>fradxy_Kprof</i>	F_x^{rad} (x -component of radiative flux, z -averaged, from Kprof)
<i>fradymxy_Kprof</i>	F_y^{rad} (y -component of radiative flux, z -averaged, from Kprof)
<i>fradxy_kramers</i>	F_{rad} (z -averaged, from Kramers’ opacity)
<i>fradr_constchixy</i>	F_{rad} (from chi.const)
<i>fturbxy</i>	$\langle \varrho T \chi_t \nabla_x s \rangle_z$
<i>fturbymxy</i>	$\langle \varrho T \chi_t \nabla_y s \rangle_z$
<i>fturbxxy</i>	$\langle \varrho T \chi_{ri} \nabla_i s \rangle_z$ (radial part of anisotropic turbulent heat flux)
<i>fturbthxy</i>	$\langle \varrho T \chi_{\theta i} \nabla_i s \rangle_z$ (latitudinal part of anisotropic turbulent heat flux)
<i>dcoolxy</i>	surface cooling flux

Module ‘magnetic.f90’

<i>bxmxy</i>	$\langle B_x \rangle_z$
<i>bymxy</i>	$\langle B_y \rangle_z$
<i>bzmxy</i>	$\langle B_z \rangle_z$
<i>jxmxy</i>	$\langle J_x \rangle_z$
<i>jymxy</i>	$\langle J_y \rangle_z$
<i>jzmxy</i>	$\langle J_z \rangle_z$
<i>axmxy</i>	$\langle A_x \rangle_z$
<i>aymxy</i>	$\langle A_y \rangle_z$
<i>azmxy</i>	$\langle A_z \rangle_z$
<i>bx2mxy</i>	$\langle B_x^2 \rangle_z$
<i>by2mxy</i>	$\langle B_y^2 \rangle_z$
<i>bz2mxy</i>	$\langle B_z^2 \rangle_z$
<i>bxbymxy</i>	$\langle B_x B_y \rangle_z$
<i>bxbzmxxy</i>	$\langle B_x B_z \rangle_z$
<i>bybzmxxy</i>	$\langle B_y B_z \rangle_z$
<i>poynxmxy</i>	$\langle \mathbf{E} \times \mathbf{B} \rangle_z x$
<i>poynymxy</i>	$\langle \mathbf{E} \times \mathbf{B} \rangle_z y$
<i>poynzmxy</i>	$\langle \mathbf{E} \times \mathbf{B} \rangle_z z$
<i>etatotalmxy</i>	$\langle \eta \rangle_z$
<i>jbmxy</i>	$\langle \mathbf{J} \cdot \mathbf{B} \rangle_z$
<i>abmxy</i>	$\langle \mathbf{A} \cdot \mathbf{B} \rangle_z$
<i>ubmxy</i>	$\langle \mathbf{U} \cdot \mathbf{B} \rangle_z$
<i>examxy1</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_z x$
<i>examxy2</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_z y$
<i>examxy3</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_z z$
<i>StokesImxy</i>	$\langle \epsilon_{B\perp} \rangle_z z$
<i>StokesQmxy</i>	$-\langle \epsilon_{B\perp} \cos 2\chi \rangle_z z$
<i>StokesUmxy</i>	$-\langle \epsilon_{B\perp} \sin 2\chi \rangle_z z$
<i>StokesQ1mxy</i>	$+\langle F \epsilon_{B\perp} \sin 2\chi \rangle_z z$
<i>StokesU1mxy</i>	$-\langle F \epsilon_{B\perp} \cos 2\chi \rangle_z z$
<i>beta1mxy</i>	$\langle \mathbf{B}^2 / (2\mu_0 p) \rangle_z z$

Module ‘axionSU2back.f90’

<i>grandxy</i>	$\langle \mathcal{T}^Q \rangle_z$
<i>grantxy</i>	$\langle \mathcal{T}^\chi \rangle_z$

Module ‘density_stratified.f90’

<i>drhomxy</i>	$\langle \Delta\rho/\rho_0 \rangle_z$
----------------	---------------------------------------

<i>drho2mxy</i>	$\langle (\Delta\rho/\rho_0)^2 \rangle_z$
Module ‘gravity_simple.f90’	
<i>epotmxy</i>	$\langle \varrho \Phi_{\text{grav}} \rangle_z$
<i>epotuxmxy</i>	$\langle \varrho \Phi_{\text{grav}} u_x \rangle_z$ (potential energy flux)
Module ‘hydro_potential.f90’	
<i>uxmxy</i>	$\langle u_x \rangle_z$
<i>uymxy</i>	$\langle u_y \rangle_z$
<i>uzmxy</i>	$\langle u_z \rangle_z$
<i>uxuymxy</i>	$\langle u_x u_y \rangle_z$
<i>uxuzmxy</i>	$\langle u_x u_z \rangle_z$
<i>uyuzmxy</i>	$\langle u_y u_z \rangle_z$
<i>oxmxy</i>	$\langle \omega_x \rangle_z$
<i>oymxy</i>	$\langle \omega_y \rangle_z$
<i>ozmxy</i>	$\langle \omega_z \rangle_z$
<i>oumxy</i>	$\langle \boldsymbol{\omega} \cdot \mathbf{u} \rangle_z$
<i>ruxmxy</i>	$\langle \rho u_x \rangle_z$
<i>ruymxy</i>	$\langle \rho u_y \rangle_z$
<i>ruzmxy</i>	$\langle \rho u_z \rangle_z$
<i>ux2mxy</i>	$\langle u_x^2 \rangle_z$
<i>uy2mxy</i>	$\langle u_y^2 \rangle_z$
<i>uz2mxy</i>	$\langle u_z^2 \rangle_z$
<i>rux2mxy</i>	$\langle \rho u_x^2 \rangle_z$
<i>ruy2mxy</i>	$\langle \rho u_y^2 \rangle_z$
<i>ruz2mxy</i>	$\langle \rho u_z^2 \rangle_z$
<i>ruxuymxy</i>	$\langle \rho u_x u_y \rangle_z$
<i>ruxuzmxy</i>	$\langle \rho u_x u_z \rangle_z$
<i>ruyuzmxy</i>	$\langle \rho u_y u_z \rangle_z$
<i>fkinxmxy</i>	$\langle \frac{1}{2} \varrho \mathbf{u}^2 u_x \rangle_z$
<i>fkinymxy</i>	$\langle \frac{1}{2} \varrho \mathbf{u}^2 u_y \rangle_z$
Module ‘magnetic_shearboxJ.f90’	
<i>bxmxy</i>	$\langle B_x \rangle_z$
<i>bymxy</i>	$\langle B_y \rangle_z$
<i>bzmxy</i>	$\langle B_z \rangle_z$
<i>jxmxy</i>	$\langle J_x \rangle_z$
<i>jymxy</i>	$\langle J_y \rangle_z$
<i>jzmxy</i>	$\langle J_z \rangle_z$
<i>axmxy</i>	$\langle A_x \rangle_z$
<i>aymxy</i>	$\langle A_y \rangle_z$
<i>azmxy</i>	$\langle A_z \rangle_z$
<i>bx2mxy</i>	$\langle B_x^2 \rangle_z$
<i>by2mxy</i>	$\langle B_y^2 \rangle_z$
<i>bz2mxy</i>	$\langle B_z^2 \rangle_z$
<i>bxbymxy</i>	$\langle B_x B_y \rangle_z$
<i>bxbzmxy</i>	$\langle B_x B_z \rangle_z$
<i>bybzmxy</i>	$\langle B_y B_z \rangle_z$
<i>poynxmxy</i>	$\langle \mathbf{E} \times \mathbf{B} \rangle_x$
<i>poynymxy</i>	$\langle \mathbf{E} \times \mathbf{B} \rangle_y$

<i>poynzmxxy</i>	$\langle \mathbf{E} \times \mathbf{B} \rangle_z$
<i>jbmxy</i>	$\langle \mathbf{J} \cdot \mathbf{B} \rangle_z$
<i>abmxy</i>	$\langle \mathbf{A} \cdot \mathbf{B} \rangle_z$
<i>examxy1</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_z _x$
<i>examxy2</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_z _y$
<i>examxy3</i>	$\langle \mathbf{E} \times \mathbf{A} \rangle_z _z$
<i>StokesImxy</i>	$\langle \epsilon_{B\perp} \rangle_z _z$
<i>StokesQmxy</i>	$-\langle \epsilon_{B\perp} \cos 2\chi \rangle_z _z$
<i>StokesUmxy</i>	$-\langle \epsilon_{B\perp} \sin 2\chi \rangle_z _z$
<i>StokesQ1mxy</i>	$+\langle F\epsilon_{B\perp} \sin 2\chi \rangle_z _z$
<i>StokesU1mxy</i>	$-\langle F\epsilon_{B\perp} \cos 2\chi \rangle_z _z$
<i>beta1mxy</i>	$\langle \mathbf{B}^2/(2\mu_0 p) \rangle_z _z$
Module ‘temperature_idealgas.f90’	
<i>TTmxy</i>	$\langle T \rangle_z$
<i>EmAIA94mxy</i>	Emission off AIA 94 channel integrated over z direction
<i>EmAIA131mxy</i>	Emission off AIA 131 channel integrated over z direction
<i>EmAIA171mxy</i>	Emission off AIA 171 channel integrated over z direction
<i>EmAIA193mxy</i>	Emission off AIA 193 channel integrated over z direction
<i>EmAIA211mxy</i>	Emission off AIA 211 channel integrated over z direction
<i>EmAIA304mxy</i>	Emission off AIA 304 channel integrated over z direction
<i>EmAIA335mxy</i>	Emission off AIA 335 channel integrated over z direction
<i>EmXRTmxy</i>	Emission off XRT Al-poly channel integrated over z direction
Module ‘thermal_energy.f90’	
<i>TTmxy</i>	$\langle T \rangle_z$
Module ‘viscosity.f90’	
<i>fviscmxy</i>	$\langle 2\nu \varrho u_i \mathcal{S}_{ix} \rangle_z$ (x-xomponent of viscous flux)
<i>fviscsmmxy</i>	$\langle 2\nu_{\text{Smag}} \varrho u_i \mathcal{S}_{ix} \rangle_z$ (x-xomponent of viscous flux)
<i>fviscymxy</i>	$\langle 2\nu \varrho u_i \mathcal{S}_{iy} \rangle_z$ (y-xomponent of viscous flux)

K.12 Boundary conditions

The following tables list all possible boundary condition labels that are documented.

K.12.1 Boundary condition bcx

Variable	Meaning
Module ‘boundcond.f90’	
<i>0</i>	zero value in ghost zones, free value on boundary
<i>p</i>	periodic
<i>s</i>	symmetry, $f_{N+i} = f_{N-i}$; implies $f'(x_N) = f'''(x_0) = 0$
<i>sf</i>	symmetry with respect to interface
<i>ss</i>	symmetry, plus function value given
<i>sds</i>	symmetric-derivative-set
<i>s0d</i>	symmetry, function value such that $df/dx=0$
<i>a</i>	antisymmetry, $f_{N+i} = -f_{N-i}$; implies $f(x_N) = f''(x_0) = 0$
<i>af</i>	antisymmetry with respect to interface

<i>a2</i>	antisymmetry relative to boundary value, $f_{N+i} = 2f_N - f_{N-i}$; implies $f''(x_0) = 0$
<i>a2v</i>	set boundary value and antisymmetry relative to it $f_{N+i} = 2f_N - f_{N-i}$; implies $f''(x_0) = 0$
<i>a2r</i>	sets $d^2f/dr^2 + 2df/dr - 2f/r^2 = 0$ This is the replacement of zero second derivative in spherical coordinates, in radial direction.
<i>cpc</i>	cylindrical perfect conductor implies $f'' + f'/R = 0$
<i>cpp</i>	cylindrical perfect conductor for Aphi implies $RA'' + A' = 0$
<i>cpz</i>	cylindrical perfect conductor for Az implies $R(RA)'' - (RA)' = 0$
<i>spr</i>	spherical perfect conductor implies $f'' + 2f'/R = 0$ and $f(x_N) = 0$
<i>v</i>	vanishing third derivative
<i>cop</i>	copy value of last physical point to all ghost cells
<i>1s</i>	onesided
<i>d1s</i>	onesided for 1st/2nd derivative in two first inner points, Dirichlet in boundary point
<i>n1s</i>	onesided for 1st/2nd derivative in two first inner points, Neumann in boundary point
<i>1so</i>	onesided
<i>cT</i>	constant temperature (implemented as condition for entropy s or temperature T)
<i>c1</i>	constant conductive flux
<i>Fgs</i>	black body: $-\chi_t \rho T \text{grad}(s) - K \text{grad}(T) = \sigma_{\text{SB}} T^4$
<i>Fct</i>	$F_{\text{bot}} = -K \text{grad}(T) - \chi_t \rho T \text{grad}(s)$
<i>Fcm</i>	$F_{\text{bot}} = -K * \text{grad}(\overline{T}) - \chi_t * \overline{\rho} * \overline{T} * \text{grad}(\overline{s})$
<i>sT</i>	symmetric temperature, $T_{N-i} = T_{N+i}$; implies $T'(x_N) = T'''(x_0) = 0$
<i>asT</i>	select entropy for uniform ghost temperature matching fluctuating boundary value, $T_{N-i} = T_N$; implies $T'(x_N) = T'(x_0) = 0$
<i>db</i>	low-order one-sided derivatives (“no boundary condition”) for density
<i>f</i>	“freeze” value, i.e. maintain initial value; antisymm wrt boundary
<i>fg</i>	“freeze” value, i.e. maintain initial value at boundary, also maintaining the ghost zones at the initial coded value, i.e., keep the gradient frozen as well
<i>1</i>	$f = 1$ (for debugging)
<i>set</i>	set boundary value to <i>fbcx</i>
<i>st</i>	set boundary value to value generated by function <i>bc.st</i> . Special time-dependent boundary condition to model temporal changes.
<i>st2</i>	set boundary value to value generated by function <i>bc.st</i> . Special time-dependent boundary condition to model temporal changes.
<i>der</i>	set derivative on boundary to <i>fbcx</i>
<i>slo</i>	set slope at the boundary = <i>fbcx</i>
<i>slp</i>	set slope at the boundary and in ghost cells = <i>fbcx</i>
<i>shx</i>	set shearing boundary proportional to x with slope= <i>fbcx</i> and abscissa= <i>fbcx</i> ²
<i>shy</i>	set shearing boundary proportional to y with slope= <i>fbcx</i> and abscissa= <i>fbcx</i> ²
<i>shz</i>	set shearing boundary proportional to z with slope= <i>fbcx</i> and abscissa= <i>fbcx</i> ²
<i>dr0</i>	set boundary value [really??]
<i>ovr</i>	overshoot boundary condition ie $(d/dx - 1/\text{dist})f = 0$.

<i>out</i>	allow outflow, but no inflow forces ghost cells and boundary to not point inwards
<i>e1o</i>	allow outflow, but no inflow uses the e1 extrapolation scheme
<i>ant</i>	stops and prompts for adding documentation
<i>e1</i>	extrapolation [describe]
<i>e2</i>	extrapolation [describe]
<i>e2h</i>	extrapolation [describe]
<i>e3</i>	extrapolation in log [maintain a power law]
<i>el</i>	linear extrapolation from last two active cells
<i>pl</i>	extrapolate using power law with the power index specified by <i>fbcx</i>
<i>hat</i>	top hat jet profile in spherical coordinate.
<i>jet</i>	top hat jet profile in cartesian coordinate.
<i>spd</i>	sets $d(rA_\alpha)/dr = \text{fbcx}(j)$
<i>sfr</i>	stress-free boundary condition for spherical coordinate system.
<i>sr1</i>	Stress-free bc for spherical coordinate system. Implementation with one-sided derivative.
<i>nfr</i>	Normal-field bc for spherical coordinate system. Some people call this the “(angry) hedgehog bc”.
<i>nr1</i>	Normal-field bc for spherical coordinate system. Some people call this the “(angry) hedgehog bc”. Implementation with one-sided derivative.
<i>sa2</i>	$(d/dr)(rB_\phi) = 0$ imposes boundary condition on 2nd derivative of rA_ϕ . Same applies to θ component.
<i>pf</i>	perfect-conductor in spherical coordinate: $d/dr(A_r) + 2/r = 0$.
<i>fix</i>	set boundary value [really??]
<i>fil</i>	set boundary value from a file
<i>cfb</i>	radial centrifugal balance
<i>g</i>	set to given value(s) or function
<i>ioc</i>	inlet/outlet on western/eastern hemisphere in cylindrical coordinates
<i>exp</i>	exponentiate x ghost zone of other variable
<i>slc</i>	set x ghost zones from slice.
<i>density_wind</i>	‘wind’ bc for <i>lnrho</i>
<i>nil’,’,no</i>	do nothing; assume that everything is set

Module ‘boundcond_alt.f90’

<i>0</i>	zero value in ghost zones, free value on boundary
<i>p</i>	periodic
<i>s</i>	symmetry, $f_{N+i} = f_{N-i}$; implies $f'(x_N) = f'''(x_0) = 0$
<i>ss</i>	symmetry, plus function value given
<i>s0d</i>	symmetry, function value such that $df/dx=0$
<i>a</i>	antisymmetry, $f_{N+i} = -f_{N-i}$; implies $f(x_N) = f''(x_0) = 0$
<i>a2</i>	antisymmetry relative to boundary value, $f_{N+i} = 2f_N - f_{N-i}$; implies $f''(x_0) = 0$
<i>a2r</i>	sets $d^2f/dr^2 + 2df/dr - 2f/r^2 = 0$ This is the replacement of zero second derivative in spherical coordinates, in radial direction.
<i>cpc</i>	cylindrical perfect conductor implies $f'' + f'/R = 0$
<i>cpp</i>	cylindrical perfect conductor implies $f'' + f'/R = 0$
<i>cpz</i>	cylindrical perfect conductor implies $f'' + f'/R = 0$
<i>spr</i>	spherical perfect conductor implies $f'' + 2f'/R = 0$ and $f(x_N) = 0$
<i>v</i>	vanishing third derivative
<i>cop</i>	copy value of last physical point to all ghost cells

<i>ls</i>	onesided
<i>lso</i>	onesided
<i>cT</i>	constant temperature (implemented as condition for entropy <i>s</i> or temperature <i>T</i>)
<i>c1</i>	constant temperature (or maybe rather constant conductive flux??)
<i>Fgs</i>	$F_{\text{conv}} = -\chi_t \cdot \rho \cdot T \cdot \text{grad}(s)$
<i>Fct</i>	$F_{\text{bot}} = -K \cdot \text{grad}(T) - \chi_t \cdot \rho \cdot T \cdot \text{grad}(s)$
<i>Fcm</i>	$F_{\text{bot}} = -K \cdot \text{grad}(\bar{T}) - \chi_t \cdot \bar{\rho} \cdot \bar{T} \cdot \text{grad}(\bar{s})$
<i>sT</i>	symmetric temperature, $T_{N-i} = T_{N+i}$; implies $T'(x_N) = T'''(x_0) = 0$
<i>asT</i>	select entropy for uniform ghost temperature matching fluctuating boundary value, $T_{N-i} = T_N =$; implies $T'(x_N) = T'(x_0) = 0$
<i>f</i>	“freeze” value, i.e. maintain initial
<i>fg</i>	“freeze” value, i.e. maintain initial
<i>1</i>	$f = 1$ (for debugging)
<i>set</i>	set boundary value to <i>fbcx12</i>
<i>der</i>	set derivative on boundary to <i>fbcx12</i>
<i>slo</i>	set slope at the boundary = <i>fbcx12</i>
<i>dr0</i>	set boundary value [really??]
<i>ovr</i>	overshoot boundary condition ie $(d/dx - 1/\text{dist})f = 0$.
<i>out</i>	allow outflow, but no inflow forces ghost cells and boundary to not point inwards
<i>e1o</i>	allow outflow, but no inflow uses the e1 extrapolation scheme
<i>ant</i>	stops and prompts for adding documentation
<i>e1</i>	extrapolation [describe]
<i>e2</i>	extrapolation [describe]
<i>e3</i>	extrapolation in log [maintain a power law]
<i>hat</i>	top hat jet profile in spherical coordinate.
<i>jet</i>	top hat jet profile in cartesian coordinate.
<i>spd</i>	sets $d(rA_\alpha)/dr = \text{fbcx12}(j)$
<i>sfr</i>	stress-free boundary condition for spherical coordinate system.
<i>nfr</i>	Normal-field bc for spherical coordinate system. Some people call this the “(angry) hedgehog bc”.
<i>sa2</i>	$(d/dr)(rB_\phi) = 0$ imposes boundary condition on 2nd derivative of rA_ϕ . Same applies to θ component.
<i>pfc</i>	perfect-conductor in spherical coordinate: $d/dr(A_r) + 2/r = 0$.
<i>fix</i>	set boundary value [really??]
<i>fil</i>	set boundary value from a file
<i>g</i>	set to given value(s) or function
<i>nil</i>	do nothing; assume that everything is set
<i>ioc</i>	inlet/outlet on western/eastern hemisphere in cylindrical coordinates
	do nothing; assume that everything is set
<i>s</i>	implies $f'(y_N) = f'''(y_0) = 0$

K.12.2 Boundary condition bcy

Variable	Meaning
Module ‘boundcond.f90’	
<i>0</i>	zero value in ghost zones, free value on boundary
<i>p</i>	periodic

<i>pp</i>	periodic across the pole
<i>yy</i>	Yin-Yang grid
<i>ap</i>	anti-periodic across the pole
<i>s</i>	symmetry, $f_{N+i} = f_{N-i}$; implies $f'(y_N) = f'''(y_0) = 0$
<i>sf</i>	symmetry with respect to interface
<i>ss</i>	symmetry, plus function value given
<i>sds</i>	symmetric-derivative-set
<i>cds</i>	complex symmetric-derivative-set
<i>s0d</i>	symmetry, function value such that $df/dy=0$
<i>a</i>	antisymmetry
<i>af</i>	antisymmetry with respect to interface
<i>a2</i>	antisymmetry relative to boundary value
<i>v</i>	vanishing third derivative
<i>v3</i>	vanishing third derivative
<i>out</i>	allow outflow, but no inflow forces ghost cells and boundary to not point inwards
<i>1s</i>	onesided
<i>d1s</i>	onesided for 1st and 2nd derivative in two first inner points, Dirichlet in boundary point
<i>n1s</i>	onesided for 1st and 2nd derivative in two first inner points, Neumann in boundary point
<i>cT</i>	constant temp.
<i>sT</i>	symmetric temp.
<i>asT</i>	select entropy for uniform ghost temperature matching fluctuating boundary value, $T_{N-i} = T_N$; implies $T'(x_N) = T'(x_0) = 0$
<i>f</i>	freeze value
<i>s+f</i>	freeze value
<i>fg</i>	“freeze” value, i.e. maintain initial
<i>fBs</i>	frozen-in B-field (s)
<i>fB</i>	frozen-in B-field (a2)
<i>1</i>	f=1 (for debugging)
<i>set</i>	set boundary value
<i>sse</i>	symmetry, set boundary value
<i>sep</i>	set boundary value
<i>e1</i>	extrapolation
<i>e2</i>	extrapolation
<i>e3</i>	extrapolation in log [maintain a power law]
<i>der</i>	set derivative on the boundary
<i>cop</i>	outflow: copy value of last physical point to all ghost cells
<i>c+k</i>	no-inflow: copy value of last physical point to all ghost cells, but suppressing any inflow
<i>sfr</i>	stress-free boundary condition for spherical coordinate system.
<i>nfr</i>	Normal-field bc for spherical coordinate system. Some people call this the “(angry) hedgehog bc”.
<i>spt</i>	spherical perfect conducting boundary condition along θ boundary $f'' + \cot \theta f' = 0$ and $f(x_N) = 0$
<i>pfC</i>	perfect conducting boundary condition along θ boundary
<i>exp</i>	exponentiate y ghost zone of other variable
<i>slc</i>	set x ghost zones from slice.
<i>nil',no</i>	do nothing; assume that everything is set

Module 'boundcond_alt.f90'

<i>sds</i>	symmetric-derivative-set
<i>0</i>	zero value in ghost zones, free value on boundary
<i>p</i>	periodic
<i>pp</i>	periodic across the pole
<i>ap</i>	anti-periodic across the pole
<i>s</i>	symmetry symmetry, $f_{N+i} = f_{N-i}$;
<i>ss</i>	symmetry, plus function value given
<i>sds</i>	symmetric-derivative-set
<i>cds</i>	complex symmetric-derivative-set
<i>s0d</i>	symmetry, function value such that $df/dy=0$
<i>a</i>	antisymmetry
<i>a2</i>	antisymmetry relative to boundary value
<i>v</i>	vanishing third derivative
<i>v3</i>	vanishing third derivative
<i>out</i>	allow outflow, but no inflow forces ghost cells and boundary to not point inwards
<i>1s</i>	onesided
<i>cT</i>	constant temp.
<i>sT</i>	symmetric temp.
<i>asT</i>	select entropy for uniform ghost temperature matching fluctuating boundary value, $T_{N-i} = T_N =$; implies $T'(x_N) = T'(x_0) = 0$
<i>f</i>	freeze value
<i>s+f</i>	freeze value
<i>fg</i>	"freeze" value, i.e. maintain initial
<i>1</i>	$f=1$ (for debugging)
<i>set</i>	set boundary value
<i>sse</i>	symmetry, set boundary value
<i>sep</i>	set boundary value
<i>e1</i>	extrapolation
<i>e2</i>	extrapolation
<i>e3</i>	extrapolation in log [maintain a power law]
<i>der</i>	set derivative on the boundary
<i>cop</i>	outflow: copy value of last physical point to all ghost cells
<i>c+k</i>	no-inflow: copy value of last physical point to all ghost cells, but suppressing any inflow
<i>sfr</i>	stress-free boundary condition for spherical coordinate system.
<i>nfr</i>	Normal-field bc for spherical coordinate system. Some people call this the "(angry) hedgehog bc".
<i>spt</i>	spherical perfect conducting boundary condition along θ boundary $f'' + \cot \theta f' = 0$ and $f(x_N) = 0$
<i>pfc</i>	perfect conducting boundary condition along θ boundary
<i>nil',</i>	do nothing; assume that everything is set
<i>sep</i>	set boundary value

K.12.3 Boundary condition bez

Variable	Meaning
----------	---------

Module 'boundcond.f90'

<i>in</i>	allow inflow, but no outflow forces ghost cells and boundary to not point outwards
<i>0</i>	zero value in ghost zones, free value on boundary
<i>p</i>	periodic
<i>yy</i>	Yin-Yang grid
<i>s</i>	symmetry
<i>sf</i>	symmetry with respect to interface
<i>s0d</i>	symmetry, function value such that $df/dz=0$
<i>0ds</i>	symmetry, function value such that $df/dz=0$
<i>a</i>	antisymmetry
<i>a2</i>	antisymmetry relative to boundary value
<i>a2v</i>	set boundary value and antisymmetry relative to it
<i>af</i>	antisymmetry with respect to interface
<i>a0d</i>	antisymmetry with zero derivative
<i>v</i>	vanishing third derivative
<i>v3</i>	vanishing third derivative
<i>1s</i>	one-sided
<i>d1s</i>	onesided for 1st and 2nd derivative in two first inner points, Dirichlet in boundary point
<i>n1s</i>	onesided for 1st and 2nd derivative in two first inner points, Neumann in boundary point
<i>a1s</i>	special for perfect conductor with const alpha and etaT when A considered as B; one-sided for 1st and 2nd derivative in two first inner points
<i>fg</i>	"freeze" value, i.e. maintain initial value at boundary, also mantaining the ghost zones at the initial coded value, i.e., keep the gradient frozen as well
<i>c1</i>	special boundary condition for $\ln \rho$ and s : constant heat flux through the boundary
<i>c1s</i>	complex
<i>Fgs</i>	black body: $-\chi_t \rho T \text{grad}(s) - K \text{grad}(T) = \sigma_{\text{SB}} T^4$
<i>Fct</i>	$F_{\text{bot}} = -K \text{grad}(T) - \chi_t \rho T \text{grad}(s)$
<i>c3</i>	constant flux at the bottom with a variable hcond
<i>pfe</i>	potential field extrapolation
<i>p1D</i>	potential field extrapolation in 1D
<i>pot</i>	potential magnetic field
<i>pwd</i>	a variant of 'pot' for nprocx=1
<i>hds</i>	hydrostatic equilibrium with a high-frequency filter
<i>cT</i>	constant temperature. If used for lnrho, sets both lnrho and ss (in which case the BC for ss should be set to 'nil') If used for ss, sets only ss.
<i>cT1</i>	constant temperature using one-sided derivatives
<i>cT2</i>	constant temp. (keep lnrho)
<i>cT3</i>	constant temp. (keep lnrho)
<i>hs</i>	hydrostatic equilibrium
<i>hse</i>	hydrostatic extrapolation rho or lnrho is extrapolated linearly and the temperature is calculated in hydrostatic equilibrium.
<i>cp</i>	constant pressure
<i>sT</i>	symmetric temp.

<i>ctz</i>	for interstellar runs copy T
<i>cdz</i>	for interstellar runs limit rho
<i>ism</i>	exponential decay/growth in rho/T by scale height
<i>asT</i>	select entropy for uniform ghost temperature matching fluctuating boundary value, $T_{N-i} = T_N =$; implies $T'(x_N) = T'(x_0) = 0$
<i>c2</i>	special boundary condition for s: constant temperature at the boundary — requires boundary condition 'a2' for $\ln \rho$
<i>db</i>	low-order one-sided derivatives ("no boundary condition") for density
<i>ce</i>	complex
<i>e1</i>	extrapolation
<i>e2</i>	extrapolation
<i>ex</i>	simple linear extrapolation in first order
<i>exf</i>	simple linear extrapolation in first order
<i>exd</i>	simple linear extrapolation in first order
<i>exm</i>	simple linear extrapolation in first order
<i>b1</i>	extrapolation with zero value (improved 'a')
<i>b2</i>	extrapolation with zero value (improved 'a')
<i>b3</i>	extrapolation with zero value (improved 'a')
<i>f',fa</i>	freeze value + antisymmetry
<i>fs</i>	freeze value + symmetry
<i>fBs</i>	frozen-in B-field (s)
<i>fB</i>	frozen-in B-field (a2)
<i>g</i>	set to given value(s) or function
<i>1</i>	f=1 (for debugging)
<i>StS</i>	solar surface boundary conditions
<i>set</i>	set boundary value
<i>sep</i>	set boundary value
<i>der</i>	set derivative on the boundary
<i>div</i>	set the divergence of u to a given value use bc = 'div' for iuz
<i>ovr</i>	set boundary value
<i>inf</i>	allow inflow, but no outflow
<i>ouf</i>	allow outflow, but no inflow
<i>in</i>	allow inflow, but no outflow forces ghost cells and boundary to not point outwards
<i>out</i>	allow outflow, but no inflow forces ghost cells and boundary to not point inwards
<i>crk</i>	no-inflow: copy value of last physical point to all ghost cells, but suppressing any inflow
<i>in0</i>	allow inflow, but no outflow forces ghost cells and boundary to not point outwards relaxes to vanishing 1st derivative at boundary
<i>ou0</i>	allow outflow, but no inflow forces ghost cells and boundary to not point inwards relaxes to vanishing 1st derivative at boundary
<i>ind</i>	allow inflow, but no outflow forces ghost cells and boundary to not point outwards creates inwards pointing or zero 1st derivative at boundary
<i>oud</i>	allow outflow, but no inflow forces ghost cells and boundary to not point inwards creates outwards pointing or zero 1st derivative at boundary
<i>ubs</i>	copy boundary outflow, reduce inflow speed outside the boundary
<i>win</i>	forces massflux given as $\Sigma \rho_i (u_i + u_0) = \text{fbcz}1/2(\rho)$
<i>cop</i>	copy value of last physical point to all ghost cells
<i>exp</i>	exponentiate z ghost zone of other variable

<i>slc</i>	set z ghost zones from slice.
<i>nil',',no</i>	do nothing; assume that everything is set
Module 'boundcond_alt.f90'	
<i>cfb</i>	radial centrifugal balance
<i>fBs</i>	frozen-in B-field (s)
<i>fB</i>	frozen-in B-field (a2)
<i>0</i>	zero value in ghost zones, free value on boundary
<i>p</i>	periodic
<i>s</i>	symmetry
<i>sf</i>	symmetry with respect to interface
<i>s0d</i>	symmetry, function value such that $df/dz=0$
<i>0ds</i>	symmetry, function value such that $df/dz=0$
<i>a</i>	antisymmetry
<i>a2</i>	antisymmetry relative to boundary value
<i>af</i>	antisymmetry with respect to interface
<i>a0d</i>	antisymmetry with zero derivative
<i>v</i>	vanishing third derivative
<i>v3</i>	vanishing third derivative
<i>1s</i>	one-sided
<i>fg</i>	"freeze" value, i.e. maintain initial
<i>c1</i>	complex
<i>Fgs</i>	$F_{\text{conv}} = -\chi_t \rho T \text{grad}(s)$
<i>Fct</i>	$F_{\text{bot}} = -K \text{grad}(T) - \chi_t \rho T \text{grad}(s)$
<i>c3</i>	constant flux at the bottom with a variable hcond
<i>pfe</i>	potential field extrapolation
<i>p1D</i>	potential field extrapolation in 1D
<i>pot</i>	potential magnetic field
<i>pwd</i>	a variant of 'pot' for nprocx=1
<i>hds</i>	hydrostatic equilibrium with a high-frequency filter
<i>cT</i>	constant temp.
<i>cT2</i>	constant temp. (keep lnrho)
<i>cT3</i>	constant temp. (keep lnrho)
<i>hs</i>	hydrostatic equilibrium
<i>hse</i>	hydrostatic extrapolation rho or lnrho is extrapolated linearly and the temperature is calculated in hydrostatic equilibrium.
<i>cp</i>	constant pressure
<i>sT</i>	symmetric temp.
<i>ctz</i>	for interstellar runs copy T
<i>cdz</i>	for interstellar runs limit rho
<i>asT</i>	select entropy for uniform ghost temperature matching fluctuating boundary value, $T_{N-i} = T_N$; implies $T'(x_N) = T'(x_0) = 0$
<i>c2</i>	complex
<i>db</i>	complex
<i>ce</i>	complex
<i>e1</i>	extrapolation
<i>e2</i>	extrapolation
<i>ex</i>	simple linear extrapolation in first order
<i>exf</i>	simple linear extrapolation in first order
<i>exd</i>	simple linear extrapolation in first order

<i>exm</i>	simple linear extrapolation in first order
<i>b1</i>	extrapolation with zero value (improved 'a')
<i>b2</i>	extrapolation with zero value (improved 'a')
<i>b3</i>	extrapolation with zero value (improved 'a')
<i>f',fa</i>	freeze value + antisymmetry
<i>fs</i>	freeze value + symmetry
<i>fBs</i>	frozen-in B-field (s)
<i>fB</i>	frozen-in B-field (a2)
<i>g</i>	set to given value(s) or function
<i>1</i>	f=1 (for debugging)
<i>StS</i>	solar surface boundary conditions
<i>set</i>	set boundary value
<i>der</i>	set derivative on the boundary
<i>div</i>	set the divergence of u to a given value use bc = 'div' for iuz
<i>ovr</i>	set boundary value
<i>inf</i>	allow inflow, but no outflow
<i>ouf</i>	allow outflow, but no inflow
<i>in</i>	allow inflow, but no outflow forces ghost cells and boundary to not point outwards
<i>out</i>	allow outflow, but no inflow forces ghost cells and boundary to not point inwards
<i>in0</i>	allow inflow, but no outflow forces ghost cells and boundary to not point outwards relaxes to vanishing 1st derivative at boundary
<i>ou0</i>	allow outflow, but no inflow forces ghost cells and boundary to not point inwards relaxes to vanishing 1st derivative at boundary
<i>ind</i>	allow inflow, but no outflow forces ghost cells and boundary to not point outwards creates inwards pointing or zero 1st derivative at boundary
<i>oud</i>	allow outflow, but no inflow forces ghost cells and boundary to not point inwards creates outwards pointing or zero 1st derivative at boundary
<i>ubs</i>	copy boundary outflow,
<i>win</i>	forces massflux given as $\Sigma \rho_i (u_i + u_0) = \text{fbcz}1/2(\rho)$
<i>cop</i>	copy value of last physical point to all ghost cells
<i>nil</i>	do nothing; assume that everything is set

K.13 Initial condition parameter dependence

The following tables list which parameters from each Namelist are required (●), optional (◇) or irrelevant (blank). The distinction is made between required and optional where by a parameter requires a setting if the default value would give an invalid or degenerate case for the initial condition.

L bin scripts

Brief description of the scripts included in pencil-code/bin

<i>Script</i>	<i>Meaning</i>
run	
<i>pc_build</i>	Compile the executables in the running directory.
<i>pc_start</i>	Check the start.in file and initialize the simulation.
<i>pc_run</i>	Check the run.in file and run the simulation
CVS	
<i>cvsci_run</i>	Add run directory to CVS. csh version.
<i>cvsci_run_bash</i>	Add run directory to CVS. bash version.
git	
<i>pc_git</i>	Update and merge the local git repository with the master branch.

References

- [1] Abramowitz, A., Stegun, I. A., *Pocketbook of Mathematical Functions*, Harri Deutsch, Frankfurt (1984)
- [2] Babkovskaia, N., Haugen, N. E. L., Brandenburg, A.: 2011, “A high-order public domain code for direct numerical simulations of turbulent combustion,” *J. Comp. Phys.* **230**, 1-12 (arXiv/1005.5301)
- [3] Banerjee, R., & Jedamzik, K., *Phys. Rev. D* **70**, 123003 (2004) “Evolution of cosmic magnetic fields: From the very early Universe, to recombination, to the present”
- [4] Barekat, A., & Brandenburg, A., *Astron. Astrophys.* **571**, A68 (2014) “Near-polytropic stellar simulations with a radiative surface”
- [5] Bhat, P., & Brandenburg, A., *Astron. Astrophys.* **587**, A90 (2016) “Hydraulic effects in a radiative atmosphere with ionization”
- [6] Brandenburg, A., *Astrophys. J.* **550**, 824–840 (2001) “The inverse cascade and non-linear alpha-effect in simulations of isotropic helical hydromagnetic turbulence”
- [7] Brandenburg, A., in *Advances in non-linear dynamos*, ed. A. Ferriz-Mas & M. Núñez Jiménez, (The Fluid Mechanics of Astrophysics and Geophysics, Vol. **9**) Taylor & Francis, London and New York, pp. 269–344 (2003); <http://arXiv.org/abs/astro-ph/0109497>
- [8] Brandenburg, A., Dobler, W., *Astron. Astrophys.* **369**, 329–338 (2001) “Large scale dynamos with helicity loss through boundaries”
- [9] Brandenburg, A., & Hazlehurst, J., *Astron. Astrophys.* **370**, 1092–1102 (2001) “Evolution of highly buoyant thermals in a stratified layer”
- [10] Brandenburg, A., & Kahniashvili, T. *Phys. Rev. Lett.* **118**, 055102 (2017) “Classes of hydrodynamic and magnetohydrodynamic turbulent decay”
- [11] Brandenburg, A., He, Y., Kahniashvili, T., Rheinhardt, M., & Schober, J. *Astrophys. J.* **911**, 110 (2021) “Gravitational waves from the chiral magnetic effect”
- [12] Brandenburg, A., & Sarson, G. R., *Phys. Rev. Lett.* **88**, 055003 (2002) “The effect of hyperdiffusivity on turbulent dynamos with helicity”
- [13] Brandenburg, A., Dobler, W., & Subramanian, K., *Astron. Nachr.* **323**, 99–122 (2002) “Magnetic helicity in stellar dynamos: new numerical experiments”
- [14] Brandenburg, A., Enqvist, K., & Olesen, P., *Phys. Rev. D* **54**, 1291–1300 (1996) “Large-scale magnetic fields from hydromagnetic turbulence in the very early universe”
- [15] Brandenburg, A., Jennings, R. L., Nordlund, Å., Rieutord, M., Stein, R. F., & Tuominen, I., *J. Fluid Mech.* **306**, 325–352 (1996) “Magnetic structures in a dynamo simulation”
- [16] A. Brandenburg, T. Kahniashvili, S. Mandal, A. Roper Pol, A. G. Tevzadze, and T. Vachaspati, *Phys. Rev. D* **96**, 123528 (2017) “Evolution of hydromagnetic turbulence from the electroweak phase transition”
- [17] Brandenburg, A., Moss, D., & Shukurov, A., *MNRAS* **276**, 651–662 (1995) “Galactic fountains as magnetic pumps”

-
- [18] Brandenburg, A., Nordlund, Å., Stein, R. F., & Torkelsson, U., *Astrophys. J.* **446**, 741–754 (1995) “Dynamo-generated turbulence and large scale magnetic fields in a Keplerian shear flow”
 - [19] Brandenburg, A., & Protiti, U., *Entropy* **25**, 1270 (2023) “Electromagnetic conversion into kinetic and thermal energies”
 - [20] Collatz, L., *The numerical treatment of differential equations*, Springer-Verlag, New York, p. 164 (1966)
 - [21] Dobler, W., Stix, M., & Brandenburg, A.: 2006, “Convection and magnetic field generation in fully convective spheres,” *Astrophys. J.* **638**, 336-347
 - [22] Durrer, R., “The Cosmic Microwave Background,” Cambridge University Press (2008)
 - [23] Gammie, C. F., *Astrophys. J.* **553**, 174–183 (2001) “Nonlinear outcome of gravitational instability in cooling, gaseous disks”
 - [24] Goodman, J., Narayan, R. & Goldreich, P., *Month. Not. Roy. Soc.* **225**, 695–711 (1987) “The stability of accretion tori – II. Nonlinear evolution to discrete planets”
 - [25] Haugen, N. E. L., & Brandenburg, A. *Phys. Rev. E* **70**, 026405 (2004) “Inertial range scaling in numerical turbulence with hyperviscosity”
 - [26] He, Y., Roper Pol, A., & Brandenburg, A. *J. Cosmol. Astropart. Phys.* **06**, 025 (2023) “Modified propagation of gravitational waves from the early radiation era”
 - [27] Hockney, R. W., & Eastwood, J. W., *Computer Simulation Using Particles*, McGraw-Hill, New York (1981)
 - [28] Hurlburt, N. E., Toomre, J., & Massaguer, J. M., *Astrophys. J.* **282**, 557–573 (1984) “Two-dimensional compressible convection extending over multiple scale heights”
 - [29] Kim, J., Moin, P. & Moser, R *J. of Fluid Mech.* **177**, 133 (1987) “Turbulence statistics in fully developed channel flow at low Reynolds number”
 - [30] Kippenhahn, R. & Weigert, A. *Stellar structure and evolution*, Springer: Berlin (1990)
 - [31] Krause, F., Rädler, K.-H., *Mean-Field Magnetohydrodynamics and Dynamo Theory*, Akademie-Verlag, Berlin; also Pergamon Press, Oxford (1980)
 - [32] Lele, S. K., *J. Comp. Phys.* **103**, 16–42 (1992) “Compact finite difference schemes with spectral-like resolution”
 - [33] Martel, H., & Shapiro, P. R. *Month. Not. Roy. Soc.* **297**, 467–485 (1998) “A convenient set of comoving cosmological variables and their application”
 - [34] Misner, C. W., Thorne, K. S. & Wheeler, J. A. *Gravitation*, San Francisco: W.H. Freeman and Co. (1973), p. 213.
 - [35] Mitra, D., Tavakol, R., Brandenburg, A., & Moss, D.: 2009, “Turbulent dynamos in spherical shell segments of varying geometrical extent,” *Astrophys. J.* **697**, 923-933 (arXiv/0812.3106)
 - [36] Nordlund, Å., & Galsgaard, K., *A 3D MHD code for Parallel Computers*, <http://www.astro.ku.dk/~aake/NumericalAstro/papers/kg/mhd.ps.gz> (1995)

- [37] Nordlund, Å., Stein, R. F., *Comput. Phys. Commun.* **59**, 119 (1990) “3-D simulations of solar and stellar convection and magnetoconvection”
- [38] Olesen, P., *Phys. Lett. B* **398**, 321 (1997) “Inverse cascades and primordial magnetic fields”
- [39] Press, W., Teukolsky, S., Vetterling, W., & Flannery, B., *Numerical Recipes in Fortran 90*, 2nd ed., Cambridge (1996)
- [40] Stanescu, D., Habashi, W. G., *J. Comp. Phys.* **143**, 674 (1988) “ $2N$ -storage low dissipation and dispersion Runge–Kutta schemes for computational acoustics”
- [41] Williamson, J. H., *J. Comp. Phys.* **35**, 48 (1980) “Low-storage Runge–Kutta schemes”
- [42] Pencil Code Collaboration, *J. Open Source Software* **6**, 2807 (2021) “The Pencil Code, a modular MPI code for partial differential equations and particles: multi-purpose and multiuser-maintained”
- [43] Porter, T. A., Jóhannesson, G., & Moskalenko, I. V., *Astrophys. J. Supp.* **262**, 30 (2022) “The GALPROP Cosmic-ray Propagation and Nonthermal Emissions Framework: Release v57”
- [44] Intel <https://software.intel.com/en-us/fortran-compiler-developer-guide-and-reference>

Part IV

Indexes

File Index

*.c	11	bb[xyz].{xz,yz,xy,xy2}	268
*.f90	11	beta1.{xz,yz,xy,xy2}	268
*.in	32, 33, 202	bfield.f90	226, 279, 287
*.local	33	bin/	5, 6, 9, 11, 13, 32, 34, 85
../	155	bin/pc_autotest	22
../32c	157	boundcond.f90	295, 298, 301
.adapt-mkfile.inc	84	boundcond_alt.f90	297, 300, 303
.bashrc	4	bugs/	11
.conf	18		
.cshrc	4	cdata.f90 ...	96, 97, 117, 210, 269, 271
.emacs	135	chem.inp	69
.svn/	11	chemistry.f90	227
/scratch/	6	chemistry_simple.f90	227
<ID>	16	chiral_mhd.f90	227
\$PENCIL_HOME/python/pencil/files/remesh.py	155	collapse.f90	228
\$PENCIL_HOME/python/pencil/files/sim/remesh.py	155	compilers	16
\$PENCIL_HOME	94	config	17, 18
\$PENCIL_HOME/python	47	config-local	17
~/config/systemd/user/	115	config/	17
~/idl_history	45	ConfigFinder	16, 17
~/pencil-auto-test	114	configure	96
1D_loop.f90	222	conv-slab/	5, 8, 50, 161, 162
1d-test/ambipolar-diffusion	68	conv-slab/src/	10
2d-tests/baroclinic	113	coronae.f90	228
2d-tests/battery_term	92	cosmicray.f90	268
2d-tests/spherical_viscous_ring	113	cosmicray_current.f90	228
3level-atom.f90	222	cosmicray_nolog.f90	279
		cparam.inc	13, 33, 85
aa[xyz].{xz,yz,xy,xy2}	268	cparam.local	6, 11, 13, 21, 41, 50, 55, 108, 157
ab[xyz].{xz,yz,xy,xy2}	268	cparam_pencils.inc	110
adapt-mkfile	20	ctimeavg.local	11, 31, 32
advective_gauge.f90	223		
alive.info	26, 204	data/	6, 13, 28, 34, 51, 93
anelastic.f90	223, 270	data/	13
ascalar.f90	279	data/dim.dat	186, 196
ascale_collapse.f90	223	data/param.nml	93, 158, 186, 196
auto-test	2, 9	data/procN/	26
axionSU2back.f90	224, 293	data/proc*/	27–29
axionU1back.f90	225	datadir.in	6, 13
		debug_c.c	118
b2.{xz,yz,xy,xy2}	268	density.f90	32, 68, 103, 189, 214, 268, 269, 274, 284, 286, 289, 292
backreact_infl.f90	190, 225	density_stratified.f90	228, 279, 285, 287, 290, 293
backreact_infl_before.f90	226	detonate.f90	228
bb.net	42		

- developers.txt 3, 99
- dim.dat 13, 14, 32
- disp_current.f90 ... 192, 228, 279, 287
- divu.{xz,yz,xy,xy2} 267
- doc/ 11
- dust-vortex/ 74
- dustdensity.f90 230
- dx/ 11
- dx/basic 42
- dx/macros/ 33
- ec.{xz,yz,xy,xy2} 268
- electroweaksu2.f90 230, 279
- entropy.f90 32, 118, 189, 215, 268, 269,
275, 284, 286, 289, 292
- entropy_anelastic.f90 231, 270
- eos_idealgas.f90 193
- eos_ionization.f90 268
- Equ 103
- equ.f90 106
- examples/pro/ 43
- experimental 136
- forced/ 11
- forced/idl/ 56
- forcing.f90 232
- fourier_fftpack.f90 54
- fourier_transform_y 54
- generate_kvectors.pro 161
- getconf.csh 9, 11, 13, 34
- gravitational_waves.f90 232
- gravitational_waves_hij6.f90 ... 234
- gravitational_waves_hTXk.f90 ... 232
- gravitational_waves_hTXk_no_
xpara.f90 233
- gravity_simple.f90 193, 234, 280, 285,
287, 294
- gravz/ 11
- grid.dat 14, 24
- H2_flamespeed/ 39
- heatflux.f90 234
- helical-MHDturb 161
- helical-MHDturb/ 36
- host-ID 17
- host_ID.conf 16
- hosts 16, 114
- hosts/ 16
- hsections.pro 162
- hydro.f90 32, 68, 83, 103, 108, 188, 210,
267, 269, 271, 284, 285, 288, 291
- hydro_kinematic.f90 235
- hydro_potential.f90 235, 270, 280, 285,
287, 290, 294
- hyperresi_strict_2nd 147
- hypervisc_strict_2nd 147
- idl/ 11, 44
- index.pro 14, 33
- inlinedoc-modules.tex 183
- Intel.conf 16
- Intel_MPI.conf 16
- interlocked-fluxrings 113
- interstellar.f90 237, 282
- inverse_cascade.f90 237
- Isurf.xz 268
- j2.{xz,yz,xy,xy2} 268
- jb.{xz,yz,xy,xy2} 268
- jj[xyz].{xz,yz,xy,xy2} 268
- k.dat 33, 161
- K_VECTORS/ 161
- kinematic/ 11
- klein_gordon.f90 238
- klein_gordon_philippe.f90 238
- klein_gordon_tmp.f90 239
- Lambda_CDM.f90 223
- legend.dat 14
- lncc.{xz,yz,xy,xy2} 268
- lnrho.{xz,yz,xy,xy2} 268
- lnrho.net 42
- lnTT.{xz,yz,xy,xy2} 268
- local_remesh.py 155
- lorenz_gauge.f90 240
- lucky_droplet.f90 240
- mach.{xz,yz,xy,xy2} 268
- magnetic.f90 viii, 32, 104, 106, 118, 189,
216, 268, 270, 276, 284, 286, 289,
293
- magnetic_shearboxJ.f90 . 240, 271, 282,
285, 288, 290, 294
- make_read_videofiles 28
- Makefile . 12, 20, 21, 32, 33, 85, 86, 132
- Makefile.local viii, 6, 11–13, 33, 50, 52,
53, 55, 64, 66, 84, 85, 96
- Makefile.src 6, 11, 13, 20, 85
- manual.tex 98, 112
- maxwell.f90 244
- meanfield.f90 193, 244, 283, 290
- meanfield_demfdt.f90 245

mkcparam.....	13, 85, 110	PHIAVG N	31
mpicomm.f90	86	phiavg.pro	31
neutraldensity.f90	68	pointmasses.f90	75
neutralvelocity.f90	68, 245	polymer.f90	246
NEWDIR	35, 90	power	53
nochiral.....	85	power.pro	54
noentropy.f90	245	power_kin.dat	53
NOERASE	93	power_krms.dat	53
noinitial_condition.f90	113	power_spectrum.f90	194
noionization.f90	206	powerbx_x.dat	53
nomagnetic.f90	viii, 106, 107	powerhel_mag.dat	53
nompicomm.f90	86, 119	powerux_x.dat	53
nospecial.f90	111	poweruz_xy.dat	204
o2.{xz,yz,xy,xy2}	267	Poynting[xyz].{xz,yz,xy,xy2}....	268
oldvar=\$VAR,	155	pp.{xz,yz,xy,xy2}	268
oo[xyz].{xz,yz,xy,xy2}	267	print.in..	8, 12, 26, 106, 107, 158, 210
os/Unix.conf	18	proc N	14, 33, 34
param.nml	14, 33, 157	proc0	14
param2.nml	14, 33	proc1	14
param_io.f90	185	proc N /	117
params.log	14, 34	pscalar.f90	222, 268
particles_breakup.f90	194	pscalar_nolog.f90	246
particles_caustics.f90	245	pt_positions.dat	108
particles_chemistry.f90	245	Qrad.{xz,yz,xy,xy2}	268
particles_dust.f90	245	r.pro	45, 51
particles_dust_brdeplete.f90	245	radial_dist_func.f90	247
particles_lagrangian.f90	246	radiation_ray.f90	268
particles_mass_swarm.f90	246	rall.pro	45, 51
particles_stalker.f90	194	reaction_OD.f90	247
particles_surfspec.f90	246	README	33, 115, 161
particles_tetrad.f90	246	reference.out	8, 13
pc_read_phiavg.pro	31	rel_1d.f90	248
pc_read_video	29	RELOAD	14, 34
pc_read_xyaver	30	remesh.csh	185, 196
pc_setupsrc	5	RERUN	35, 90
pencil-code/	10, 94	rho.{xz,yz,xy,xy2}	268
pencil-code/idl/read	7	rings/	11
pencil-code/utils	34	run.csh	9, 11, 13, 32, 34, 35, 90
pencil-runs/	10, 11	run.in	viii, 7, 12, 27–29, 31–35, 37, 39–41, 53, 55, 89, 108, 155, 157, 164, 168, 185, 202
Pencil::ConfigFinder	16	run.pro	33
Pencil::ConfigParser	16	run.x	7, 14, 32, 40, 115
pencil_check.f90	89	runA_32a.....	21
PENCIL_HOME/python/tests/README.md	116	runs/	11
pencil_test.service	115	rvid_box.....	28
pencil_test.timer	116	rvid_box.pro	28
phiaver.in	30, 268	rvid_line.pro	28
phiaverages.dat	30	rvid_plane.pro	28

samples/ 2–4, 9, 13, 127
 samples/parameter_scan 33
 samples/README 11
 SAVE 35
 schur_triple_decomp.f90 194, 248
 SCRATCH_DIR 90
 sedimentation/ 57
 seed.dat 14, 41
 selfgravity.f90 194, 248
 sfb-1.dat 55
 sfu-1.dat 55
 sfz1-1.dat 55
 shear.f90 248
 shock.{xz,yz,xy,xy2} 268
 shock.f90 248, 268
 shock_highorder.f90 248, 283, 285, 288
 slice 93
 slice_uu1.yz 27
 slize* 28
 sn_series.dat 13, 14
 sn_series.in 13, 14
 solar_corona.f90 248
 solid_cells_CGEO.f90 249
 solid_cells_ogrid_chemistry.f90 . 249
 solid_cells_reactive.f90 249
 sound-spherical-noequi/ 25
 sourceme 4, 82
 sourceme.csh 4, 11, 82, 93
 sourceme.sh 4, 11, 82, 93
 SPEED 33
 spher/ 11
 src/ viii, 3, 6, 10, 11, 13, 34, 85
 src/ 13
 src/*.local 32
 src/.config-files 19
 src/cparam.local 69, 71, 155, 157
 src/Makefile.inc 52
 src/Makefile.local .. 22, 39, 113, 155,
 157, 162
 src/read_all_videofiles.x 28
 src/read_videofiles.x 28, 29
 ss.{xz,yz,xy,xy2} 268
 ss.net 42
 start.csh .. 9, 11, 13, 32, 34, 40, 69, 89,
 155
 start.in .. viii, 7, 12, 25, 29, 33, 35–37,
 40, 41, 60, 89, 108, 113, 155, 158,
 164, 185, 195, 202
 start.pro 33, 45, 93
 start.x . 7, 14, 40, 42, 84, 89, 115, 207
 STOP 34, 35
 structure.pro 56
 sub.f90 104
 TAVGN 32
 temperature_idealgas.f90 ... 249, 268,
 283, 285, 288, 291, 295
 temperature_ionization.f90 . 250, 283
 test_chemistry.f90 250
 testfield_axisym.f90 250
 testfield_axisym2.f90 250
 testfield_axisym4.f90 251
 testfield_compress_z.f90 251
 testfield_meri.f90 254
 testfield_nonlin_z.f90 255
 testfield_x.f90 258
 testfield_xz.f90 259
 testfield_z.f90 259
 testflow_z.f90 262
 testperturb.f90 262
 testscalar.f90 262
 testscalar_axisym.f90 263
 testscalar_simple.f90 265
 thermal_energy.f90 266, 284, 285, 288,
 291, 295
 time.dat 14, 41
 time_series.dat 8, 13, 14, 26, 32, 33, 42,
 45
 timeavg.dat 32
 top.log 34
 training_torchfort.f90 266
 tran.dat 69
 ts.pro 43–45, 162
 tsnap.dat 14, 203
 TT.{xz,yz,xy,xy2} 268
 tvid.dat 14
 u2.{xz,yz,xy,xy2} 267
 u[xyz].{xz,yz,xy,xy2} 267
 uu.net 42
 VAR 41, 93, 155, 204
 VAR*N* 14, 27, 32, 34, 37, 203
 var.dat 7, 14, 26, 32, 35, 37,
 41, 43, 45, 93, 154–156, 185, 187,
 196, 203, 204
 var.general 42
 VAR0 187, 195
 VAR1 156
 VAR# 35
 video.in 27–29, 158, 267

viscosity.f90 . 195, 267, 271, 284, 288,
295
vsections.pro 163
vsections2.pro 163

X.xy 15
X.xz 15
X.yz 15
xyaver.in 30, 158, 271
xyaverages.dat 30
xzaver.in 30, 284
xzaverages.dat 30

yaver.in 30, 288
yaverages.dat 30
yH.{xz,yz,xy,xy2} 268
yzaver.in 30, 285
yzaverages.dat 30

zaver.in 30, 291
zaverages.dat 30

Variable Index

<i>.true.</i>	30	<i>ab_int</i>	216, 240
<i>0ds</i>	301, 303	<i>ab_spec</i>	204
<i>1s</i>	296, 298–301, 303	<i>ABCA</i>	207
<i>1so</i>	296, 298	<i>ABC_B</i>	207
<i>0</i>	295, 297, 298, 300, 301, 303	<i>ABC_C</i>	207
<i>1</i>	296, 298–300, 302, 304	<i>abm</i>	216, 240
<i>a</i>	224, 225, 238, 295, 297, 299–301, 303	<i>abmh</i>	216, 240
<i>a0d</i>	301, 303	<i>abmn</i>	216, 240
<i>a0rms</i>	229	<i>abms</i>	216, 240
<i>A1</i>	248	<i>abmxy</i>	293, 295
<i>a11xy</i>	255	<i>abmz</i>	277, 282
<i>a12xy</i>	255	<i>abph1mz</i>	278
<i>a13xy</i>	255	<i>abph2mz</i>	278
<i>a1s</i>	301	<i>abph3mz</i>	278
<i>A2</i>	248	<i>abrms</i>	216, 240
<i>a2</i>	296, 297, 299–301, 303	<i>abumx</i>	216, 240
<i>a21xy</i>	255	<i>abumy</i>	216, 240
<i>a22xy</i>	255	<i>abumz</i>	216, 240
<i>a23xy</i>	255	<i>abuxmz</i>	276, 282
<i>a2b2m</i>	217	<i>abuymz</i>	276, 282
<i>a2m</i>	219, 242	<i>abuzmz</i>	276, 282
<i>a2mz</i>	278	<i>accmz</i>	279
<i>a2r</i>	296, 297	<i>accpowzdownmz</i>	274, 281
<i>a2rhogphim</i>	225, 226, 238, 239	<i>accpowzmz</i>	274, 281
<i>a2rhogpsim</i>	238, 239	<i>accpowzupmz</i>	274, 281
<i>a2rhom</i>	225, 226, 238, 239	<i>acczdownmz</i>	274
<i>a2rhophim</i>	225, 226, 238, 239	<i>acczmz</i>	274
<i>a2rhopm</i>	225, 226, 238, 239	<i>acczupmz</i>	274
<i>a2rhopsim</i>	238, 239	<i>adphiB21m</i>	229
<i>a2v</i>	296, 301	<i>adphiB2m</i>	229
<i>A3</i>	248	<i>adphiBm</i>	229
<i>a31xy</i>	255	<i>adphiBrms</i>	229
<i>a32xy</i>	255	<i>adphiJBm</i>	229
<i>a33xy</i>	255	<i>aem</i>	221
<i>A4</i>	248	<i>af</i>	295, 299, 301, 303
<i>a4rhophim</i>	225	<i>afact</i>	229
<i>a4rhophim_crit</i>	190	<i>ajm</i>	217, 240
<i>A5</i>	248	<i>aklam</i>	262
<i>aa</i>	161, 268	<i>aklamQ</i>	262
<i>aa0</i>	161	<i>akxpt</i>	238, 244
<i>aa2m</i>	237, 244	<i>AL</i>	225
<i>aabm</i>	223	<i>ALdot</i>	225
<i>AAm</i>	247	<i>alp11</i>	251, 255, 258, 259, 262
<i>ab</i>	268	<i>alp11_x</i>	258
		<i>alp11_x2</i>	258
		<i>alp11cc</i>	252, 256, 258, 260

<i>alp11x</i>	259	<i>ampllncc2</i>	201
<i>alp11z</i>	254, 261	<i>ampllnrho</i>	189, 198
<i>alp12</i>	251, 255, 258, 259, 262	<i>ampluu</i>	138, 188, 197
<i>alp12_x</i>	258	<i>ant</i>	297, 298
<i>alp12_x2</i>	258	<i>ap</i>	299, 300
<i>alp12cs</i>	252, 256, 258, 260	<i>apbrms</i>	223
<i>alp12x</i>	259	<i>aphimax</i>	190
<i>alp12z</i>	254, 261	<i>AR</i>	225
<i>alp13</i>	260	<i>ARdot</i>	225
<i>alp13z</i>	261	<i>arms</i>	219, 238, 242, 244
<i>alp21</i>	251, 255, 258, 259, 262	<i>ascale</i>	223–225
<i>alp21_x</i>	258	<i>ascale_visc</i>	195
<i>alp21_x2</i>	258	<i>asT</i>	296, 298–300, 302, 303
<i>alp21sc</i>	252, 256, 258, 260	<i>axmxy</i>	293, 294
<i>alp21x</i>	259	<i>axmxz</i>	289, 290
<i>alp21z</i>	254, 261	<i>axmz</i>	276, 282
<i>alp22</i>	251, 255, 258, 259, 262	<i>axp2</i>	218, 241
<i>alp22_x</i>	258	<i>axpt</i>	218, 241
<i>alp22_x2</i>	258	<i>ay2mz</i>	278
<i>alp22ss</i>	252, 256, 258, 260	<i>aybxmz</i>	278
<i>alp22x</i>	259	<i>aymxy</i>	293, 294
<i>alp22z</i>	254, 261	<i>aymxz</i>	289, 290
<i>alp23</i>	260	<i>aymz</i>	276, 282
<i>alp23z</i>	261	<i>ayp2</i>	218, 241
<i>alp31</i>	251, 255, 258, 259, 262	<i>aypt</i>	218, 241
<i>alp32</i>	251, 255, 258, 259, 262	<i>Azmid_max</i>	219
<i>alpha_tdep_t1</i>	193	<i>Azmid_min</i>	219
<i>alpha_tdep_type</i>	194	<i>azmxy</i>	293, 294
<i>alpK</i>	252, 256	<i>azmxz</i>	289, 290
<i>alpKjbm</i>	245	<i>azmz</i>	276, 282
<i>alpKm</i>	245	<i>azp2</i>	218, 242
<i>alpM</i>	252, 256	<i>azpt</i>	218, 241
<i>alpm</i>	245	<i>b0max</i>	253, 257
<i>alpMK</i>	252, 256	<i>b0rms</i>	253, 257, 258, 260
<i>alpmxz</i>	290	<i>b1</i>	302, 304
<i>alpPARA</i>	250, 251	<i>b111xy</i>	255
<i>alpPARAz</i>	250, 251	<i>b112xy</i>	255
<i>alpPERP</i>	250, 251	<i>b11rms</i>	253, 256, 258–260
<i>alpPERPz</i>	250, 251	<i>b121xy</i>	255
<i>amax</i>	219, 242	<i>b122xy</i>	255
<i>ambmz</i>	220, 243	<i>b12m</i>	216
<i>ambmzh</i>	220, 243	<i>b12rms</i>	253, 257, 258, 260
<i>ambmzn</i>	220, 243	<i>b131xy</i>	255
<i>ambmzs</i>	220, 243	<i>b132xy</i>	255
<i>ampl_ff</i>	208	<i>b1b23m</i>	221, 243
<i>ampl_forc</i>	205	<i>b1b32m</i>	221, 243
<i>ampl_ss</i>	199	<i>b1m</i>	216, 240
<i>amplaa</i>	189, 201	<i>b1rms</i>	250, 251
<i>amplaa2</i>	189, 201	<i>b2</i>	268, 302, 304
<i>ampllncc</i>	201	<i>b211xy</i>	255

<i>b212xy</i>	255	<i>bcx</i>	40, 41, 90, 197, 204, 295
<i>b21rms</i>	253, 256, 258–260	<i>bcy</i>	40, 197, 204, 298
<i>b221xy</i>	255	<i>bcz</i>	40, 41, 90, 197, 204, 300
<i>b222xy</i>	255	<i>BdEdtm</i>	229
<i>b22rms</i>	253, 257, 258, 260	<i>bdel2amz</i>	277
<i>b231xy</i>	255	<i>bEL2</i>	248
<i>b232xy</i>	255	<i>beta1</i>	268
<i>b2b13m</i>	221, 243	<i>beta1m</i>	219, 242
<i>b2b31m</i>	221, 243	<i>beta1max</i>	219, 242
<i>b2divum</i>	221, 244	<i>beta1mxy</i>	293, 295
<i>b2m</i>	216, 226, 240	<i>beta1mz</i>	277, 282
<i>b2mmx</i>	286	<i>beta2mx</i>	287, 288
<i>b2mphi</i>	270, 271	<i>beta2mz</i>	277, 279, 282
<i>b2mx</i>	286–288	<i>betam</i>	219, 227, 242
<i>b2mxz</i>	289, 290	<i>betamax</i>	219, 227, 242
<i>b2mz</i>	278, 279, 283	<i>betamin</i>	219, 227, 242
<i>b2rms</i>	250, 251	<i>betamx</i>	287, 288
<i>b2ruzum</i>	216, 240	<i>betamz</i>	277, 279, 282
<i>b2sphm</i>	222	<i>betm</i>	228, 248
<i>b2tm</i>	216, 240	<i>betmax</i>	248
<i>b2uzm</i>	216, 240	<i>betPARA</i>	250, 251
<i>b3</i>	302, 304	<i>betPARAz</i>	250, 251
<i>b311xy</i>	255	<i>betPERP</i>	250, 251
<i>b312xy</i>	255	<i>betPERP2</i>	251
<i>b321xy</i>	255	<i>betPERPz</i>	250, 251
<i>b322xy</i>	255	<i>bf2m</i>	218
<i>b331xy</i>	255	<i>bf2mz</i>	278, 283
<i>b332xy</i>	255	<i>bf4m</i>	218
<i>b3b12m</i>	221, 243	<i>bfm</i>	232
<i>b3b21m</i>	220, 243	<i>bfrms</i>	218, 238, 242, 244
<i>b3rms</i>	250, 251	<i>bgmu5rms</i>	222, 228
<i>b4m</i>	216	<i>bgmuSrms</i>	222, 228
<i>b6m</i>	216	<i>bhrms</i>	253
<i>b8m</i>	216	<i>bij2m</i>	221
<i>B_ext</i>	207	<i>bij_cov_diffmax</i>	219
<i>bamp</i>	253, 257, 261	<i>bjtm</i>	216, 240
<i>bb</i>	118, 268	<i>blowupm</i>	245
<i>bbmphi</i>	270, 271	<i>bm</i>	226
<i>bbsphmphi</i>	270, 271	<i>bm2</i>	216, 240
<i>bbxmax</i>	218, 242	<i>bmax</i>	218, 226, 238, 242, 244
<i>bbxmz</i>	276, 282	<i>bmin</i>	226
<i>bbymax</i>	218, 242	<i>bmz</i>	30, 219, 242, 287
<i>bbymz</i>	276, 282	<i>bmxy_rms</i>	221, 244
<i>bbzmax</i>	218, 242	<i>bmy</i>	30, 219, 242
<i>bbzmz</i>	276, 282	<i>bmz</i>	30, 219, 242, 279
<i>bc{x,y,z}</i>	40	<i>bmzA2</i>	219, 242
<i>bcosphz</i>	220, 243	<i>bmzph</i>	219, 243
<i>BcurlBm</i>	229	<i>bmzphe</i>	219, 243
<i>BcurlEm</i>	229	<i>bmzS2</i>	219, 242
<i>bcurlfmz</i>	278	<i>boostprms</i>	229

<i>bp2mph</i>	270	<i>bxbymxz</i>	289, 290
<i>bpbzmp</i>	270	<i>bxbymy</i>	284, 285
<i>bpcmp</i>	270	<i>bxbymz</i>	278, 279, 283
<i>bpmph</i>	270, 271	<i>bxbypt</i>	218
<i>bprimerms</i>	229	<i>bxbz</i>	219, 226
<i>bpsmp</i>	270	<i>bxbzmx</i>	287, 288
<i>br2mp</i>	270	<i>bxbzmxxy</i>	293, 294
<i>brbpmp</i>	270	<i>bxbzmxz</i>	289, 290
<i>brbzmp</i>	270	<i>bxbzmy</i>	284, 285
<i>brcmp</i>	270	<i>bxbz</i>	278, 279, 283
<i>brmp</i>	270, 271	<i>bxm</i>	219, 226, 242
<i>brms</i>	138, 218, 226, 238, 242, 244	<i>bxmax</i>	218, 226, 242
<i>brmsx</i>	222, 244	<i>bxmin</i>	218, 242
<i>brmsz</i>	222, 244	<i>bxmx</i>	286–288
<i>bRR2</i>	248	<i>bxmxy</i>	293, 294
<i>bRRm</i>	248	<i>bxmxz</i>	289, 290
<i>brsmph</i>	270	<i>bxmy</i>	284, 285
<i>brsphmp</i>	270, 271	<i>bxmz</i>	276, 279, 282
<i>bSH2</i>	248	<i>bxp2</i>	218, 241
<i>bsinphz</i>	220, 243	<i>bxph1mz</i>	278
<i>bthmp</i>	270, 271	<i>bxph2mz</i>	278
<i>butm</i>	216	<i>bxph3mz</i>	278
<i>bx0mz</i>	254, 257, 259, 261	<i>bxpt</i>	218, 241
<i>bx0pt</i>	252, 256, 260	<i>by0mz</i>	254, 257, 259, 261
<i>bx11pt</i>	252, 256, 260	<i>by0pt</i>	253, 256, 260
<i>bx12pt</i>	252, 256, 260	<i>by11pt</i>	252, 256, 260
<i>bx1mxz</i>	289, 290	<i>by12pt</i>	252, 256, 260
<i>bx1pt</i>	250, 251	<i>by1mxz</i>	289, 290
<i>bx21pt</i>	252, 256, 260	<i>by21pt</i>	252, 256, 260
<i>bx22pt</i>	252, 256, 260	<i>by22pt</i>	252, 256, 260
<i>bx2m</i>	220, 226, 243	<i>by2m</i>	220, 226, 243
<i>bx2mx</i>	286–288	<i>by2mx</i>	287, 288
<i>bx2mxy</i>	293, 294	<i>by2mxy</i>	293, 294
<i>bx2mxz</i>	289, 290	<i>by2mxz</i>	289, 290
<i>bx2my</i>	284, 285	<i>by2my</i>	284, 285
<i>bx2mz</i>	277, 279, 282	<i>by2mz</i>	277, 279, 282
<i>bx2ph1mz</i>	278	<i>by2ph1mz</i>	278
<i>bx2ph2mz</i>	278	<i>by2ph2mz</i>	278
<i>bx2ph3mz</i>	278	<i>by2ph3mz</i>	278
<i>bx2pt</i>	250, 251	<i>by2rmz</i>	277, 282
<i>bx2rmz</i>	277, 282	<i>by2rph1mz</i>	278
<i>bx2rph1mz</i>	278	<i>by2rph2mz</i>	278
<i>bx2rph2mz</i>	278	<i>by2rph3mz</i>	278
<i>bx2rph3mz</i>	278	<i>by3m</i>	220
<i>bx3m</i>	220	<i>by4m</i>	220
<i>bx3pt</i>	250, 251	<i>bybzm</i>	219, 226
<i>bx4m</i>	220	<i>bybzm</i>	287, 288
<i>bxbym</i>	219, 226, 242	<i>bybzmxy</i>	293, 294
<i>bxbymx</i>	287, 288	<i>bybzm</i>	289, 290
<i>bxbymxy</i>	293, 294	<i>bybzm</i>	285

<i>bybzmz</i>	278, 279, 283	<i>bzph3mz</i>	278
<i>bybzpt</i>	218	<i>bzpt</i>	218, 241
<i>bym</i>	219, 226, 242	<i>bzsmphi</i>	270
<i>bymax</i>	218, 226, 242	<i>bzuamz</i>	277
<i>bymin</i>	218, 242	<i>c+k</i>	299, 300
<i>bymx</i>	286–288	<i>c1</i>	296, 298, 301, 303
<i>bymxy</i>	293, 294	<i>c1pt</i>	263, 265, 266
<i>bymxz</i>	289, 290	<i>c1rms</i>	263, 265, 266
<i>bymy</i>	284, 285	<i>c1s</i>	301
<i>bymz</i>	276, 279, 282	<i>c2</i>	302, 303
<i>byp2</i>	218, 241	<i>c2pt</i>	263, 265, 266
<i>byph1mz</i>	278	<i>c2rms</i>	263, 265, 266
<i>byph2mz</i>	278	<i>c3</i>	301, 303
<i>byph3mz</i>	278	<i>c3pt</i>	263, 265, 266
<i>bypt</i>	218, 241	<i>c3rms</i>	263, 265, 266
<i>bz0mz</i>	254, 258, 259, 262	<i>c4pt</i>	263, 265, 266
<i>bz1mxz</i>	289, 290	<i>c4rms</i>	263, 265, 266
<i>bz2m</i>	220, 226, 243	<i>c5pt</i>	263, 265, 266
<i>bz2mphi</i>	270	<i>c5rms</i>	263, 265, 266
<i>bz2mx</i>	287, 288	<i>c6pt</i>	263, 265, 266
<i>bz2mxy</i>	293, 294	<i>c6rms</i>	263, 265, 266
<i>bz2mxz</i>	289, 290	<i>c_light2</i>	73
<i>bz2my</i>	284, 285	<i>cc10m</i>	247
<i>bz2mz</i>	277, 279, 282	<i>cc1m</i>	247
<i>bz2ph1mz</i>	278	<i>cc2m</i>	247
<i>bz2ph2mz</i>	278	<i>cc3m</i>	247
<i>bz2ph3mz</i>	278	<i>cc4m</i>	247
<i>bz2rmz</i>	277, 282	<i>cc5m</i>	247
<i>bz2rph1mz</i>	278	<i>cc6m</i>	247
<i>bz2rph2mz</i>	278	<i>cc7m</i>	247
<i>bz2rph3mz</i>	278	<i>cc8m</i>	247
<i>bz3m</i>	220	<i>cc9m</i>	247
<i>bz4m</i>	220	<i>ccglnm</i>	222, 247
<i>bzaymz</i>	277	<i>ccmax</i>	222, 246
<i>bzbxpt</i>	218	<i>ccmin</i>	246
<i>bzcmphi</i>	270	<i>ccrms</i>	247
<i>bzdivamz</i>	277	<i>ccugum</i>	247
<i>bzLammz</i>	277	<i>cdiffrho</i>	205
<i>bzm</i>	219, 226, 242	<i>cds</i>	299, 300
<i>bzmax</i>	218, 226, 242	<i>cdt</i>	38, 39, 41, 203
<i>bzmin</i>	218, 242	<i>cdt_sigE</i>	192
<i>bzmphi</i>	270, 271	<i>cdts</i>	39
<i>bzmx</i>	286–288	<i>cdtsrc</i>	185
<i>bzmxy</i>	293, 294	<i>cdtv</i>	39, 203
<i>bzmxz</i>	289, 290	<i>cdz</i>	302, 303
<i>bzmy</i>	284, 285	<i>ce</i>	302, 303
<i>bzmz</i>	276, 279, 282	<i>cfb</i>	297, 303
<i>bzp2</i>	218, 241	<i>cgam</i>	215
<i>bzph1mz</i>	278	<i>chemistry.f90</i>	183
<i>bzph2mz</i>	278	<i>chi</i>	206, 224

<i>chi_t</i>	207	<i>d1s</i>	296, 299, 301
<i>chiddot</i>	224	<i>D2</i>	248
<i>chidot</i>	224	<i>d2davg</i>	31, 185, 204
<i>chikrammax</i>	216	<i>d2LamRAbrms</i>	223
<i>chikrammin</i>	216	<i>d2LamRArms</i>	223
<i>CHIRAL</i>	85	<i>D3</i>	248
<i>ckxrange</i>	194, 209	<i>D4</i>	248
<i>ckyrange</i>	209	<i>D5</i>	248
<i>coeff_grid</i>	24, 26	<i>d6abmz</i>	277, 282
<i>constrainteqn</i>	229	<i>d6amz1</i>	277, 282
<i>constrainteqnW</i>	231	<i>d6amz2</i>	277, 282
<i>cool</i>	206	<i>d6amz3</i>	277, 282
<i>cooltype</i>	206	<i>da0rms</i>	229
<i>cop</i>	296, 297, 299, 300, 302, 304	<i>damp</i>	205
<i>coriolis_number</i>	214	<i>dampu</i>	204, 205
<i>cosjbm</i>	222, 244	<i>dampuext</i>	205
<i>cosubm</i>	217, 241	<i>dampuint</i>	205
<i>count_eb0</i>	229	<i>datadir</i>	45
<i>count_eb0a</i>	225, 226, 238–240	<i>db</i>	296, 302, 303
<i>cp</i>	301, 303	<i>dbx2m</i>	226
<i>cpc</i>	296, 297	<i>dbxm</i>	226
<i>cpp</i>	296, 297	<i>dbxmax</i>	226
<i>cpz</i>	296, 297	<i>dbx2m</i>	226
<i>crk</i>	302	<i>dbym</i>	226
<i>Crmsm</i>	247	<i>dbymax</i>	226
<i>cs0</i>	193, 198, 205	<i>dbz2m</i>	227
<i>cs2</i>	118	<i>dbzm</i>	226
<i>cs2bot</i>	198, 205	<i>dbzmax</i>	226
<i>cs2cool</i>	206	<i>dcoolmphi</i>	269
<i>cs2mphi</i>	269, 270	<i>dcoolx</i>	286
<i>cs2top</i>	198, 205	<i>dcoolxy</i>	293
<i>csm</i>	215, 231, 245, 249	<i>dcoolz</i>	276
<i>csmx</i>	215, 249	<i>DDm</i>	247
<i>cT</i>	296, 298–301, 303	<i>ddotam</i>	225, 226, 238, 239
<i>cT1</i>	301	<i>del</i>	250, 251
<i>cT2</i>	301, 303	<i>del2</i>	251
<i>cT3</i>	301, 303	<i>deltay</i>	248
<i>ctz</i>	302, 303	<i>delz</i>	250, 251
<i>cugccm</i>	247	<i>density_wind</i>	297
<i>curlBrms</i>	229	<i>der</i>	296, 298–300, 302, 304
<i>curlru2mz</i>	271, 280	<i>detn</i>	228
<i>cutoff</i>	138	<i>dettot</i>	228
<i>cvsci_run</i>	307	<i>dexbmx</i>	221, 244
<i>cvsci_run_bash</i>	307	<i>dexbmy</i>	221, 244
<i>cvsid</i>	185, 195, 202	<i>dexbmz</i>	221, 244
<i>Cz2m</i>	246	<i>dgrant</i>	224
<i>Cz4m</i>	246	<i>dgrant_up</i>	224
<i>czrange</i>	209	<i>dheat_buffer1</i>	207
<i>D1</i>	248	<i>diffrho_hyper3_mesh</i>	151
<i>d1davg</i>	185	<i>div</i>	302, 304

<i>divabrms</i>	223	<i>drho2my</i>	285
<i>divamz</i>	277	<i>drho2mz</i>	279
<i>divapbrms</i>	223	<i>drhom</i>	214, 228
<i>divarms</i>	219, 242	<i>drhomax</i>	228
<i>divbmax</i>	227	<i>drhomx</i>	287
<i>divbrms</i>	227	<i>drhomxy</i>	293
<i>divcoolmphi</i>	269	<i>drhomxz</i>	290
<i>divdotW1m</i>	230	<i>drhomy</i>	285
<i>divdotW1rms</i>	230	<i>drhomz</i>	279
<i>divdotW2m</i>	231	<i>drhorms</i>	228
<i>divdotW2rms</i>	230	<i>dsnap</i>	27, 37, 185, 203
<i>divdotW3m</i>	231	<i>dsnap_down</i>	185
<i>divdotW3rms</i>	230	<i>dspec</i>	185, 204
<i>divEm</i>	229	<i>dstalk</i>	194, 202
<i>divErms</i>	229	<i>dt</i>	8, 39, 41, 203, 210
<i>divheatmphi</i>	269	<i>dt_Cflow1</i>	228
<i>divJm</i>	229	<i>dt_Cflow2</i>	228
<i>divJrms</i>	229	<i>dt_chiral</i>	223, 228
<i>divrhom</i>	223	<i>dt_CMW</i>	223, 228
<i>divrhomax</i>	212, 223, 236	<i>dt_D5</i>	223, 228
<i>divrhorms</i>	212, 223, 236	<i>dt_Dmu</i>	223, 228
<i>divru2mz</i>	271, 280	<i>dt_gammaf5</i>	223, 228
<i>divu</i>	267	<i>dt_lambda5</i>	223, 228
<i>divu2m</i>	212, 236	<i>dt_vmu</i>	223, 228
<i>divu2mz</i>	271, 280	<i>dtb</i>	219, 242
<i>divuHrms</i>	214, 237	<i>dtc</i>	8, 215, 231, 245, 250
<i>divum</i>	212, 236	<i>dtchem</i>	227, 249, 250
<i>divumz</i>	272, 280	<i>dtchi</i>	8, 215, 231, 250
<i>divW1m</i>	230	<i>dtchi2</i>	222, 228, 249
<i>divW1rms</i>	230	<i>dt diffus</i>	210
<i>divW2m</i>	230	<i>dt diffus2</i>	210
<i>divW2rms</i>	230	<i>dt diffus3</i>	210
<i>divW3m</i>	230	<i>dteta</i>	219, 229, 242
<i>divW3rms</i>	230	<i>dteta3</i>	219
<i>dk</i>	225	<i>dtF</i>	214
<i>DLm</i>	248	<i>dtGphirho</i>	226
<i>dlnascale_reheating</i>	190	<i>dtH</i>	215
<i>Dmu5_tdep</i>	223, 228	<i>dtmin</i>	203
<i>dobrms</i>	217, 241	<i>dtnewt</i>	249
<i>dphi2m</i>	225, 226, 238, 239	<i>dt nu</i>	8, 267
<i>dphim</i>	225, 226, 238, 239	<i>dt pchem</i>	246
<i>dphirms</i>	225, 226, 238, 239	<i>dt phi</i>	225
<i>dpsi2m</i>	238, 239	<i>dtq</i>	234
<i>dpsim</i>	238, 239	<i>dtq2</i>	234
<i>dpsirms</i>	238, 239	<i>dtrad</i>	222, 228
<i>dr0</i>	296, 298	<i>dtradloss</i>	249
<i>drho2m</i>	214, 228	<i>dt shear</i>	248
<i>drho2mx</i>	287	<i>dt sigE</i>	229
<i>drho2mxy</i>	294	<i>dt spitzer</i>	222, 228, 234, 249
<i>drho2mxz</i>	290	<i>dt src</i>	210

<i>dtu</i>	8, 213, 236	<i>E13xy</i>	254
<i>dtv</i>	210	<i>e1o</i>	297, 298
<i>dtvel</i>	249	<i>e2</i>	297–300, 302, 303
<i>dtvmaxz</i>	271	<i>E20z</i>	254, 257, 259, 261
<i>dubrms</i>	217, 241	<i>E211z</i>	254, 257, 259, 261
<i>dudx</i>	214, 237	<i>E212z</i>	254, 257, 259, 261
<i>durms</i>	211, 235	<i>E21rms</i>	253, 257, 258, 260
<i>dvid</i>	27, 28, 37, 185, 203	<i>E21xy</i>	254
<i>dW1max</i>	230	<i>E221z</i>	254, 257, 259, 261
<i>dW1rms</i>	230	<i>E222z</i>	254, 257, 259, 261
<i>dW1xm</i>	231	<i>E22rms</i>	253, 257, 258, 260
<i>dW1xmz</i>	280	<i>E22xy</i>	254
<i>dW1ym</i>	231	<i>E23xy</i>	254
<i>dW1ymz</i>	280	<i>e2h</i>	297
<i>dW1zm</i>	231	<i>e2mx</i>	287
<i>dW1zmz</i>	280	<i>e2mz</i>	279
<i>dW2max</i>	230	<i>e3</i>	297–300
<i>dW2rms</i>	230	<i>E30z</i>	254, 257, 259, 261
<i>dW2xm</i>	231	<i>E311z</i>	254, 257, 259, 261
<i>dW2xmz</i>	280	<i>E312z</i>	254, 257, 259, 261
<i>dW2ym</i>	231	<i>E31xy</i>	254
<i>dW2ymz</i>	280	<i>E321z</i>	254, 257, 259, 261
<i>dW2zm</i>	231	<i>E322z</i>	254, 257, 259, 261
<i>dW2zmz</i>	280	<i>E32xy</i>	254
<i>dW3max</i>	230	<i>E33xy</i>	254
<i>dW3rms</i>	230	<i>e3xamz1</i>	277, 283
<i>dW3xm</i>	231	<i>e3xamz2</i>	277, 283
<i>dW3xmz</i>	280	<i>e3xamz3</i>	277, 283
<i>dW3ym</i>	231	<i>E41xy</i>	254
<i>dW3ymz</i>	280	<i>E42xy</i>	254
<i>dW3zm</i>	231	<i>E43xy</i>	254
<i>dW3zmz</i>	280	<i>E51xy</i>	254
<i>dz_1</i>	24	<i>E52xy</i>	254
<i>dz_tilde</i>	24	<i>E53xy</i>	254
<i>E0mrms</i>	253	<i>E61xy</i>	254
<i>E0rms</i>	253, 257, 259, 260	<i>E62xy</i>	255
<i>E0Um</i>	254, 257, 261	<i>E63xy</i>	255
<i>E0Wm</i>	254, 257, 261	<i>E71xy</i>	255
<i>E0xrms</i>	253	<i>E72xy</i>	255
<i>E0yrms</i>	253	<i>E73xy</i>	255
<i>e1</i>	297–300, 302, 303	<i>E81xy</i>	255
<i>E10z</i>	254, 257, 259, 261	<i>E82xy</i>	255
<i>E111z</i>	254, 257, 259, 261	<i>E83xy</i>	255
<i>E112z</i>	254, 257, 259, 261	<i>E91xy</i>	255
<i>E11rms</i>	253, 257, 258, 260	<i>E92xy</i>	255
<i>E11xy</i>	254	<i>E93xy</i>	255
<i>E121z</i>	254, 257, 259, 261	<i>ebm</i>	230
<i>E122z</i>	254, 257, 259, 261	<i>EBpq</i>	254, 257, 259, 261
<i>E12rms</i>	253, 257, 258, 260	<i>echarge</i>	229
<i>E12xy</i>	254	<i>ecr</i>	268

<i>ecrmz</i>	279	<i>EmAIA335mxz</i>	291
<i>ecrph1mz</i>	279	<i>EmAIA94mxy</i>	295
<i>ecrph2mz</i>	279	<i>EmAIA94mxz</i>	291
<i>ecrph3mz</i>	279	<i>emax</i>	229
<i>edotb</i>	225	<i>embmz</i>	220, 243
<i>edotrms</i>	229	<i>EMFdotB_{int}</i>	245
<i>ee10</i>	247	<i>EMFdotB_m</i>	245
<i>ee1m</i>	247	<i>EMFmax</i>	245
<i>ee2m</i>	237, 244, 247	<i>EMFmin</i>	245
<i>ee3m</i>	247	<i>EMFmz1</i>	245
<i>ee4m</i>	247	<i>EMFmz2</i>	245
<i>ee50</i>	247	<i>EMFmz3</i>	245
<i>ee90</i>	247	<i>EMFrms</i>	245
<i>ee99</i>	247	<i>emxamz3</i>	220, 243
<i>EEEM</i>	228, 238, 244	<i>EmXRTmxy</i>	295
<i>EEGW</i>	233, 234	<i>EmXRTmxz</i>	291
<i>EEK</i>	214, 235, 237	<i>eos_merger</i>	97
<i>EEK2</i>	214	<i>epot</i>	234
<i>EEK3</i>	214	<i>epotmx</i>	287
<i>EEK4</i>	214	<i>epotmxy</i>	294
<i>EEM</i>	216	<i>epotmy</i>	285
<i>eem</i>	215, 231, 247, 249, 250, 266	<i>epotmz</i>	280
<i>EEM2</i>	216	<i>epottot</i>	234
<i>EEM3</i>	216	<i>epotuxmx</i>	287
<i>EEM4</i>	216	<i>epotuxmxy</i>	294
<i>eemz</i>	283	<i>epotuzmz</i>	280
<i>ekin</i>	214, 237	<i>eprimerms</i>	229
<i>ekin_{cr}</i>	228	<i>eps_{rkf}</i>	210
<i>ekinmx</i>	286, 288	<i>epsAD</i>	217, 241
<i>ekinmz</i>	274, 281	<i>epsilonaa</i>	189, 201
<i>ekinp</i>	245, 246	<i>epsK</i>	267
<i>ekinph1mz</i>	273	<i>epsK2</i>	267
<i>ekinph2mz</i>	273	<i>epsK3</i>	267
<i>ekinph3mz</i>	273	<i>epsK4</i>	267
<i>ekintot</i>	214, 237	<i>epsK_{int}</i>	267
<i>ekxpt</i>	238, 244	<i>epsK_{mz}</i>	284
<i>el</i>	297	<i>epsKn</i>	245
<i>ell_{gam}</i>	267	<i>epsM</i>	217, 241
<i>emag</i>	218, 238, 242, 244	<i>epsM2</i>	217
<i>EmAIA131mxy</i>	295	<i>epsM3</i>	217
<i>EmAIA131mxz</i>	291	<i>epsM4</i>	217
<i>EmAIA171mxy</i>	295	<i>epsM_{mz}</i>	278, 283
<i>EmAIA171mxz</i>	291	<i>erms</i>	229, 238, 244
<i>EmAIA193mxy</i>	295	<i>eruzmz</i>	283
<i>EmAIA193mxz</i>	291	<i>eta</i>	207
<i>EmAIA211mxy</i>	295	<i>eta11</i>	252, 255, 258–260, 262
<i>EmAIA211mxz</i>	291	<i>eta11_x</i>	258
<i>EmAIA304mxy</i>	295	<i>eta11_{x2}</i>	258
<i>EmAIA304mxz</i>	291	<i>eta11_{cc}</i>	252, 256, 258, 260
<i>EmAIA335mxy</i>	295	<i>eta11_x</i>	259

<i>eta11z</i>	254, 261	<i>Ex0pt</i>	253, 257, 261
<i>eta12</i>	252, 255, 258, 260, 262	<i>Ex11pt</i>	253, 257, 260
<i>eta12_x</i>	258	<i>Ex12pt</i>	253, 257, 260
<i>eta12_x2</i>	258	<i>Ex21pt</i>	253, 257, 260
<i>eta12cs</i>	252, 256, 258, 260	<i>Ex22pt</i>	253, 257, 261
<i>eta12x</i>	259	<i>exabot</i>	218, 242
<i>eta12z</i>	254, 261	<i>examx</i>	221, 243
<i>eta21</i>	252, 255, 258–260, 262	<i>examxy1</i>	293, 295
<i>eta21_x</i>	258	<i>examxy2</i>	293, 295
<i>eta21_x2</i>	258	<i>examxy3</i>	293, 295
<i>eta21sc</i>	252, 256, 258, 260	<i>examy</i>	221, 243
<i>eta21x</i>	259	<i>examz</i>	221, 243
<i>eta21z</i>	254, 261	<i>examz1</i>	277, 283
<i>eta22</i>	252, 255, 258, 260, 262	<i>examz2</i>	277, 283
<i>eta22_x</i>	258	<i>examz3</i>	277, 283
<i>eta22_x2</i>	258	<i>exatop</i>	218, 242
<i>eta22ss</i>	252, 256, 258, 260	<i>exatotalmx</i>	221
<i>eta22x</i>	259	<i>exatotalmy</i>	221
<i>eta22z</i>	254, 261	<i>exatotalmz</i>	221
<i>eta31</i>	260, 262	<i>exatotalmz1</i>	277
<i>eta32</i>	260, 262	<i>exatotalmz2</i>	277
<i>eta_ext</i>	207	<i>exatotalmz3</i>	277
<i>eta_int</i>	207	<i>exd</i>	302, 303
<i>eta_out</i>	207	<i>exf</i>	302, 303
<i>eta_tdep</i>	216	<i>exjmx</i>	221, 243
<i>etaaniso</i>	222	<i>exjmy</i>	221, 243
<i>etaanisoBB</i>	222	<i>exjnz</i>	221, 243
<i>etaj2max</i>	221, 244	<i>exm</i>	229, 302, 304
<i>etajmax</i>	221, 244	<i>Exmxy</i>	222, 244
<i>etajrhomax</i>	222, 244	<i>Exmxz</i>	289, 290
<i>etaSchw</i>	229	<i>Exmz</i>	277, 282
<i>etaSchw_max</i>	192	<i>exmz</i>	279
<i>etasmagm</i>	221, 244	<i>exp</i>	297, 299, 302
<i>etasmagmax</i>	221, 244	<i>Exp2</i>	218, 241
<i>etasmagmin</i>	221, 244	<i>Expt</i>	218, 241
<i>etat_tdep_t1</i>	194	<i>Ey0pt</i>	253, 257, 261
<i>etat_tdep_type</i>	194	<i>Ey11pt</i>	253, 257, 261
<i>etatm</i>	245	<i>Ey12pt</i>	253, 257, 261
<i>etatotalmx</i>	287, 288	<i>Ey21pt</i>	253, 257, 261
<i>etatotalmxy</i>	293	<i>Ey22pt</i>	253, 257, 261
<i>etatotalmz</i>	277, 283	<i>eym</i>	229
<i>etavamax</i>	221, 244	<i>Eymxy</i>	222, 244
<i>ethm</i>	215, 231, 245, 249, 250, 266	<i>Eymxz</i>	289, 290
<i>ethmax</i>	266	<i>Eymz</i>	277, 282
<i>ethmcr</i>	228	<i>eymz</i>	279
<i>ethmin</i>	266	<i>Eyp2</i>	218, 241
<i>ethmz</i>	276, 283	<i>Eypt</i>	218, 241
<i>ethtot</i>	215, 231, 249, 266	<i>ezm</i>	229
<i>etot</i>	266	<i>Ezmxy</i>	222, 244
<i>ex</i>	302, 303	<i>Ezmxz</i>	289, 290

<i>Ez</i> <i>mz</i>	277, 282	<i>fkinymxy</i>	292, 294
<i>ezmz</i>	279	<i>fkinzdownmz</i>	271
<i>Ezp2</i>	218, 241	<i>fkinzm</i>	210, 235
<i>Ezpt</i>	218, 241	<i>fkinzmz</i>	271, 280
<i>f</i>	296, 298–300	<i>fkinzupmz</i>	271
<i>f',fa</i>	302, 304	<i>fmasszmz</i>	271, 280
<i>F11z</i>	263, 265, 266	<i>fmax</i>	106
<i>F12z</i>	263, 265, 266	<i>force</i>	208
<i>F21z</i>	263, 265, 266	<i>fountain</i>	208
<i>F22z</i>	263, 265, 266	<i>fppf</i>	229
<i>F31z</i>	263, 265, 266	<i>fpresxmz</i>	283
<i>F32z</i>	263, 265, 266	<i>fpresymz</i>	283
<i>fact</i>	225	<i>fpreszmz</i>	276, 283
<i>fB</i>	299, 302–304	<i>fracvph1mz</i>	276
<i>fbcx</i>	296	<i>fracvph2mz</i>	276
<i>fbcx12</i>	298	<i>fracvph3mz</i>	276
<i>fbcx2</i>	296	<i>fradbot</i>	215, 231, 249
<i>fbm</i>	217, 241	<i>fradmx</i>	286
<i>Fbot</i>	207	<i>fradmz</i>	283
<i>fBs</i>	299, 302–304	<i>fradr_constchixy</i>	293
<i>FC</i>	88	<i>fradrsphmphi_Kconst</i>	270
<i>Fcm</i>	296, 298	<i>fradrsphmphi_Kprof</i>	270
<i>fconvm</i>	216	<i>fradrsphmphi_kramers</i>	269
<i>fconvmz</i>	283	<i>fradtop</i>	215, 231, 249
<i>fconvsphmphi</i>	270	<i>fradx_constchi</i>	286
<i>fconvsphmphi</i>	270	<i>fradx_kramers</i>	286
<i>fconvthsphmphi</i>	270	<i>fradxy_Kprof</i>	293
<i>fconvxmz</i>	286	<i>fradxy_kramers</i>	293
<i>fconvxy</i>	292	<i>fradymxy_Kprof</i>	293
<i>fconvyxy</i>	292	<i>fradz</i>	275
<i>fconvz</i>	275	<i>fradz_constchi</i>	276
<i>fconvzxy</i>	292	<i>fradz_Kprof</i>	276
<i>Fct</i>	296, 298, 301, 303	<i>fradz_kramers</i>	276
<i>Fenthdownz</i>	275	<i>fring1,fring2</i>	201
<i>Fenthupz</i>	275	<i>frmax</i>	246
<i>Fenthz</i>	275	<i>fs</i>	302, 304
<i>ffakez</i>	283	<i>fsum</i>	106
<i>ffdownmxy</i>	291	<i>fturbfz</i>	276
<i>ffdownmz</i>	272	<i>fturbmx</i>	286
<i>FFLAGS_DOUBLE</i>	52	<i>fturbmz</i>	276
<i>fg</i>	296, 298–301, 303	<i>fturbbrsphmphi</i>	270
<i>Fgravx</i>	234	<i>fturbbrxy</i>	293
<i>Fgs</i>	296, 298, 301, 303	<i>fturbthxy</i>	293
<i>fil</i>	297, 298	<i>fturbtz</i>	276
<i>fix</i>	297, 298	<i>fturbxy</i>	293
<i>fkinrsphmphi</i>	269	<i>fturbymxy</i>	293
<i>fkinxdownmxy</i>	292	<i>fturbz</i>	276
<i>fkinxmz</i>	286, 288	<i>fum</i>	213, 237
<i>fkinxmxy</i>	292, 294	<i>fviscm</i>	267
<i>fkinxupmxy</i>	292	<i>fviscmx</i>	267

<i>fviscmin</i>	267	<i>gammax</i>	210
<i>fviscmx</i>	288	<i>gamrms</i>	210
<i>fviscmxy</i>	295	<i>gamz</i>	250, 251
<i>fviscmz</i>	284	<i>gb2m</i>	221
<i>fviscrmsx</i>	267	<i>gcc10m</i>	247
<i>fviscrsphmphi</i>	271	<i>gcc1m</i>	247
<i>fviscsmmxy</i>	295	<i>gcc2m</i>	247
<i>fviscsmmz</i>	284	<i>gcc3m</i>	247
<i>fviscymxy</i>	295	<i>gcc4m</i>	247
<i>fxbxm</i>	217, 241	<i>gcc5m</i>	247
<i>g</i>	297, 298, 302, 304	<i>gcc6m</i>	247
<i>g11pt</i>	232–234	<i>gcc7m</i>	247
<i>g12pt</i>	232–234	<i>gcc8m</i>	247
<i>g22pt</i>	232–234	<i>gcc9m</i>	247
<i>g23pt</i>	232–234	<i>gdivu2m</i>	212, 236
<i>g31pt</i>	232–234	<i>geometrical_types.f90</i>	183
<i>g33pt</i>	232–234	<i>gg2m</i>	233, 234
<i>gal</i>	262	<i>ggT2m</i>	233, 234
<i>gam</i>	250, 251	<i>ggTm</i>	234
<i>gam11</i>	262, 264, 265	<i>ggTp2</i>	233, 234
<i>gam11z</i>	263–265	<i>ggTpt</i>	232–234
<i>gam12</i>	262, 264, 265	<i>ggTXm</i>	233, 234
<i>gam12z</i>	263–265	<i>ggX2m</i>	233, 234
<i>gam13</i>	262, 264, 265	<i>ggXm</i>	234
<i>gam13z</i>	263–265	<i>ggXp2</i>	233, 234
<i>gam21</i>	262, 264, 265	<i>ggXpt</i>	233, 234
<i>gam21z</i>	263–265	<i>gijij2m</i>	234
<i>gam22</i>	262, 264, 265	<i>gLamam</i>	216
<i>gam22z</i>	263–265	<i>gLambm</i>	216
<i>gam23</i>	262, 264, 265	<i>gLamRAbm</i>	223
<i>gam23z</i>	263, 264, 266	<i>gLamRARms</i>	223
<i>gam31</i>	262, 264, 265	<i>gmu5mx</i>	222, 227
<i>gam31z</i>	263, 264, 266	<i>gmu5my</i>	222, 227
<i>gam32</i>	262, 264, 265	<i>gmu5mz</i>	222, 228
<i>gam32z</i>	263, 264, 266	<i>gmu5rms</i>	222, 227
<i>gam33</i>	262, 264, 265	<i>gmuSrms</i>	222, 227
<i>gam33z</i>	263, 264, 266	<i>gphirms</i>	238
<i>gam3z</i>	264	<i>gpotsself2m</i>	248
<i>gam_EBrms</i>	229	<i>gpu_astaroth.f90</i>	183
<i>Gam_phi</i>	226	<i>gradpxmz</i>	275
<i>gamc</i>	264	<i>gradpymz</i>	275
<i>gamcz</i>	264	<i>gradpzmz</i>	275
<i>gamf5m</i>	222, 227	<i>grads0</i>	199
<i>Gamm</i>	237	<i>grand2</i>	224
<i>gamm</i>	210	<i>grandxy</i>	293
<i>gamma</i>	193, 198, 205, 262	<i>grantxy</i>	293
<i>Gamma_phi</i>	226	<i>grav_amp</i>	193, 199
<i>Gamma_phi0</i>	190	<i>grav_tilt</i>	193, 199
<i>Gamma_phi_exp</i>	190	<i>gravz</i>	139, 193, 199, 208
<i>gammaQ</i>	262	<i>gravz_profile</i>	139, 140, 193, 199, 208

<i>grhomax</i>	215	<i>h23rms</i>	233, 234
<i>grid_func</i>	24	<i>h31rms</i>	233
<i>grms</i>	229	<i>h33rms</i>	233, 234
<i>gshockmax</i>	248	<i>hat</i>	297, 298
<i>gsrms</i>	215	<i>hcond0</i>	205–207
<i>gss2mz</i>	276	<i>hcond1</i>	205, 206
<i>gsxmxy</i>	292	<i>hcond2</i>	206
<i>gsymxy</i>	292	<i>hds</i>	301, 303
<i>gszmxy</i>	292	<i>headt</i>	117
<i>gT2m</i>	249	<i>headtt</i>	117
<i>gTimp2</i>	232	<i>heating</i>	225
<i>gTimp</i>	232	<i>heatmz</i>	276
<i>gTmax</i>	215, 249	<i>heatThm</i>	250
<i>gTrep2</i>	232	<i>height_eta</i>	207
<i>gTrept</i>	232	<i>height_ff</i>	208
<i>gTrms</i>	215	<i>hhT2m</i>	232–234
<i>gTT2mz</i>	276	<i>hhThhXm</i>	232
<i>gTxgsom</i>	216	<i>hhTp2</i>	233, 234
<i>gTxgsrms</i>	216	<i>hhTpt</i>	232–234
<i>gTxgsx2mxy</i>	292	<i>hhTXm</i>	233, 234
<i>gTxgsx2mz</i>	276	<i>hhX2m</i>	232–234
<i>gTxgsxmxy</i>	292	<i>hhXp2</i>	233, 234
<i>gTxgsxmz</i>	276	<i>hhXpt</i>	232–234
<i>gTxgsy2mxy</i>	292	<i>hijij2m</i>	234
<i>gTxgsy2mz</i>	276	<i>hjparallelm</i>	222, 244
<i>gTxgsymxy</i>	292	<i>hjperpm</i>	222, 244
<i>gTxgsymz</i>	276	<i>hjrms</i>	219, 242
<i>gTxgsz2mxy</i>	292	<i>Hmax</i>	215
<i>gTxgsz2mz</i>	276	<i>Hmax_ism</i>	237
<i>gTxgszmxy</i>	292	<i>hrms</i>	233, 234
<i>gTxgszmz</i>	276	<i>hs</i>	301, 303
<i>gTxmxy</i>	292	<i>Hscriptm</i>	225, 226, 238, 239
<i>gTymxy</i>	292	<i>hse</i>	301, 303
<i>gTzmxy</i>	292	<i>hTimp2</i>	232
<i>gu2m</i>	213	<i>hTimp</i>	232
<i>guxgTm</i>	249	<i>hTrep2</i>	232
<i>guygTm</i>	249	<i>hTrept</i>	232
<i>guzgTm</i>	249	<i>Hubble</i>	223, 224
<i>gXimp2</i>	232	<i>hXimp2</i>	232
<i>gXimpt</i>	232	<i>hXimpt</i>	232
<i>gXrep2</i>	232	<i>hXrep2</i>	232
<i>gXrept</i>	232	<i>hXrept</i>	232
<i>gzlnrhomz</i>	275	<i>hydro.f90</i>	183
<i>H</i>	224, 225	<i>hydro.potential.f90</i>	183
<i>h0max</i>	253	<i>ialive</i>	26, 204
<i>h0rms</i>	253	<i>idiag_jbm</i>	106
<i>h11rms</i>	233, 253	<i>idiff</i>	151
<i>h12rms</i>	233, 253	<i>IDL_PATH</i>	4, 56
<i>h21rms</i>	253	<i>idx_tavg</i>	31, 32, 204
<i>h22rms</i>	233, 234, 253	<i>iforce</i>	208

<i>iforce2</i>	208	<i>iz2</i>	27, 203
<i>iheatcond</i>	206, 207	<i>j11rms</i>	253, 256
<i>imAL</i>	225	<i>j2</i>	268
<i>imAR</i>	225	<i>j2b2m</i>	217
<i>imARdot</i>	225	<i>j2m</i>	216, 227, 240
<i>imax</i>	55, 56	<i>j2mx</i>	286
<i>imTR</i>	224	<i>j2mz</i>	278, 283
<i>in</i>	301, 302, 304	<i>J2sigEm</i>	229
<i>in0</i>	302, 304	<i>jb</i>	106, 268
<i>ind</i>	302, 304	<i>jb0m</i>	253, 257, 260
<i>inertiaxx</i>	215	<i>jb_int</i>	216, 240
<i>inertiaxx_car</i>	215	<i>jbm</i>	106, 217, 240
<i>inertiayy</i>	215	<i>jbmh</i>	217, 240
<i>inertiayy_car</i>	215	<i>jbmh</i>	217, 240
<i>inertiazz</i>	215	<i>jbmphi</i>	270, 271
<i>inertiazz_car</i>	215	<i>jbmz</i>	217, 240
<i>inf</i>	302, 304	<i>jbmz</i>	286
<i>init_ads_mol_frac</i>	202	<i>jbmxy</i>	293, 295
<i>init_surf_mol_frac</i>	202	<i>jbmz</i>	277, 282
<i>initaa</i>	200	<i>jbph1mz</i>	278
<i>initaa2</i>	201	<i>jbph2mz</i>	278
<i>initlncc</i>	201	<i>jbph3mz</i>	278
<i>initlncc2</i>	201	<i>jbrms</i>	217, 240
<i>initlnrho</i>	189, 198	<i>jbtm</i>	216, 240
<i>initpower</i>	137, 138	<i>jdel2am</i>	221
<i>initpower2</i>	137	<i>jdel2amz</i>	277
<i>initss</i>	199	<i>jem</i>	221
<i>inituu</i>	188, 197	<i>jet</i>	297, 298
<i>inz</i>	209	<i>jfm</i>	232
<i>ioc</i>	297, 298	<i>jh2m1</i>	220
<i>ip</i>	36, 185, 195, 202	<i>JJ</i>	225
<i>Iring1,Iring2</i>	201	<i>jj</i>	268
<i>isav</i>	26	<i>JJ L</i>	225
<i>isave</i>	203	<i>JJ R</i>	225
<i>ism</i>	302	<i>jm</i>	227
<i>isohtop</i>	189, 200, 206	<i>jm2</i>	216, 240
<i>Isurf</i>	268	<i>jmax</i>	219, 227, 242
<i>it</i>	8, 34, 210	<i>jmbmz</i>	220, 243
<i>it1</i>	26, 30, 203	<i>jmin</i>	227
<i>it1_ldt_report</i>	185	<i>jmx</i>	219, 243
<i>it1_reset_value</i>	190	<i>jmy</i>	219, 243
<i>it1d</i>	30, 203	<i>jmz</i>	219, 243
<i>itorder</i>	37, 39, 203	<i>Johmrms</i>	229
<i>iuut</i>	108	<i>jparallelm</i>	222, 244
<i>ivar</i>	117	<i>jperpm</i>	222, 244
<i>ivisc</i>	209	<i>jprimerms</i>	229
<i>iwig</i>	203	<i>jrms</i>	219, 227, 242
<i>ix</i>	27, 203	<i>jutm</i>	216
<i>iy</i>	27, 203	<i>jx2m</i>	220, 227
<i>iz</i>	27, 203	<i>jx2m1</i>	220

<i>jx2m2</i>	220	<i>jzm</i>	219, 227
<i>jx2m3</i>	220	<i>jzmax</i>	219, 227, 242
<i>jx4m</i>	220	<i>jzmxy</i>	293, 294
<i>jxaprms</i>	223	<i>jzmxz</i>	289, 290
<i>jxarms</i>	223	<i>jzmz</i>	276, 282
<i>jxbm</i>	220, 243	<i>jzp2</i>	218, 241
<i>jxbmx</i>	221, 243	<i>jzph1mz</i>	279
<i>jxbmy</i>	221, 243	<i>jzph2mz</i>	279
<i>jxbmz</i>	221, 243	<i>jzph3mz</i>	279
<i>jxbr2m</i>	221, 244	<i>jzpt</i>	218, 241
<i>jxbrmax</i>	221, 244	<i>k0</i>	225
<i>jxbrms</i>	217	<i>k_forc</i>	205
<i>jxbrqm</i>	213, 221	<i>kap11</i>	262, 264, 265
<i>jbxm</i>	217, 241	<i>kap11z</i>	263, 264, 266
<i>jxbym</i>	217, 241	<i>kap12</i>	263–265
<i>jxbzm</i>	217, 241	<i>kap12z</i>	263, 264, 266
<i>jxgLamRArms</i>	223	<i>kap13</i>	263–265
<i>jxm</i>	219, 223, 227, 228	<i>kap13z</i>	263, 264, 266
<i>jxmax</i>	218, 227, 242	<i>kap21</i>	262, 264, 265
<i>jxmxy</i>	293, 294	<i>kap21z</i>	263, 264, 266
<i>jxmxz</i>	289, 290	<i>kap22</i>	263–265
<i>jxmz</i>	276, 282	<i>kap22z</i>	263, 264, 266
<i>jxp2</i>	218, 241	<i>kap23</i>	263–265
<i>jxph1mz</i>	279	<i>kap23z</i>	263, 264, 266
<i>jxph2mz</i>	279	<i>kap31</i>	262, 264, 265
<i>jxph3mz</i>	279	<i>kap31z</i>	263, 264, 266
<i>jxpt</i>	218, 241	<i>kap32</i>	263–265
<i>jy2m</i>	220, 227	<i>kap32z</i>	263, 264, 266
<i>jy2m1</i>	220	<i>kap33</i>	263–265
<i>jy2m2</i>	220	<i>kap33z</i>	263, 264, 266
<i>jy2m3</i>	220	<i>kap_tdep</i>	215
<i>jy4m</i>	220	<i>kapcPARA</i>	264
<i>jybxm</i>	217, 241	<i>kapcPARAz</i>	264
<i>jybym</i>	217, 241	<i>kapcPERP1</i>	264
<i>jybz</i>	217, 241	<i>kapcPERP2</i>	264
<i>jym</i>	219, 227	<i>kapcPERPz</i>	264
<i>jymax</i>	218, 227, 242	<i>kapPARA</i>	250, 251
<i>jymxy</i>	293, 294	<i>kapPARAz</i>	250, 251
<i>jymxz</i>	289, 290	<i>kapPERP</i>	250, 251
<i>jymz</i>	276, 282	<i>kapPERP2</i>	251
<i>jyp2</i>	218, 241	<i>kapPERPz</i>	250, 251
<i>jyph1mz</i>	279	<i>kC</i>	248
<i>jyph2mz</i>	279	<i>kfountain</i>	208
<i>jyph3mz</i>	279	<i>khorrss</i>	200
<i>jypt</i>	218, 241	<i>KK2m</i>	230
<i>jz2m</i>	220, 227	<i>KKm</i>	230
<i>jz4m</i>	220	<i>Kkramersm</i>	216
<i>jzbxm</i>	217, 241	<i>Kkramersmx</i>	286
<i>jzby</i>	217, 241	<i>Kkramersmz</i>	276
<i>jzbzm</i>	217, 241	<i>km0EM</i>	218

<i>km1EM</i>	218	<i>ldraglaw_steadystate</i>	209
<i>kmz</i>	220, 243	<i>ldt_disp_current</i>	192
<i>kpeak</i>	138	<i>ldt_Gamma_phi</i>	190
<i>kx</i>	207	<i>legendre_lmax</i>	210
<i>kx_aa</i>	201, 220, 243	<i>leigvec</i>	194
<i>kx_lnc</i>	201	<i>lequidist</i>	24
<i>ky</i>	207	<i>letat_tdep</i>	194
<i>ky_aa</i>	201	<i>lfirst</i>	117
<i>ky_lnc</i>	201	<i>lfirstpoint</i>	117
<i>kz</i>	207	<i>lforcing_cont_uu_diff</i>	188
<i>kz_aa</i>	201	<i>lg_Gamma_phi_fraction_firststep</i> ..	190
<i>kz_lnc</i>	201	<i>lhalf_factor_in_GW</i>	209
<i>L1</i>	248	<i>lhcond_global</i>	207
<i>l1</i>	117	<i>lheating_keep_on</i>	190
<i>L2</i>	248	<i>lignore_adphiB_term_in_MHD_current</i>	
<i>l2</i>	117	192	
<i>L3</i>	248	<i>lignore_Bext_in_b2</i>	207
<i>L4</i>	248	<i>linclude_dphiB_in_MHD</i>	192
<i>L5</i>	248	<i>linclude_gphixE_in_MHD</i>	192
<i>la2rhop_wrong_factor</i>	190	<i>linclude_rho_EB_in_wstate</i>	191
<i>ladvance_ee</i>	192	<i>linclude_rho_EBK_in_wstate</i>	191
<i>lalpha_tdep</i>	194	<i>linclude_rhokin_in_a2rho</i>	191
<i>Lamm</i>	237	<i>lintegrate_shell</i>	209
<i>lampluu_adjust_ascale</i>	188	<i>lintegrate_z</i>	209
<i>LamRAbzm</i>	223	<i>linv_BD</i>	191
<i>LamRAbzmz</i>	223	<i>lit1_reset_if_solve_for_phi</i>	191
<i>LamRAm</i>	223	<i>llate_reset_el_pencil</i>	192
<i>LamRAp2</i>	223	<i>LLm</i>	248
<i>LamRApt</i>	223	<i>lna</i>	223, 224
<i>LamRARms</i>	223	<i>lna_switch_toMHD</i>	191
<i>LANG</i>	91	<i>lnam</i>	225, 226, 238, 239
<i>lappend_pc_constants</i>	185	<i>lnc</i>	268
<i>lapply_BD_kNy_factor</i>	190	<i>lnowrite</i>	185, 196
<i>lapply_BD_k1D_factor</i>	190	<i>lnrho</i>	268
<i>lb_nxgrid</i>	55	<i>lnrhomax</i>	214
<i>lbaryons</i>	185	<i>lnrhomin</i>	214
<i>lBD_scaling_wHubble</i>	190	<i>lnrhomphi</i>	269, 270
<i>lBryan_etal14</i>	195	<i>lnrhorms</i>	214
<i>lbubble</i>	209	<i>lnTT</i>	268
<i>lcalc_heatcond_constchi</i>	206	<i>lnVpm</i>	245
<i>lcomplex</i>	209	<i>logbm</i>	216
<i>lcorrect_integer_kcalc</i>	194	<i>lohmic_heating_ee</i>	192
<i>lcorrect_sign_adphiB_term</i>	192	<i>lohmic_heating_justee</i>	192
<i>lcylindrical_spectra</i>	209	<i>lold_ldt_phi</i>	191
<i>ldamp_fade</i>	205	<i>lold_lrho_chi_dtconstraint</i>	191
<i>ldefine_a2rhophi_with_Vpotential</i> ..	190	<i>loss</i>	267
<i>ldefine_a2rhopm_without_Vpotential</i> ..	190	<i>lout</i>	117
<i>ldensity_var</i>	97	<i>lperi</i>	186, 195
<i>ldragforce_dust_par</i>	209	<i>lpress_equil</i>	190, 201
<i>ldragforce_gas_par</i>	209	<i>lpress_equil_uu</i>	189

<i>lprocz_slowest</i>	51, 186, 195	<i>lwrite_ic</i>	187, 195
<i>lpscalar_sink</i>	209	<i>lwrite_phiaverages</i>	30
<i>lread_aux</i>	186, 196	<i>lwrite_yaverages</i>	30
<i>lread_gauss_quadrature</i>	210	<i>lwrite_zaverages</i>	30
<i>lread_hcond</i>	207	<i>lwstate_crit</i>	192
<i>lread_oldsnap</i>	186, 196	<i>lwstate_crit_old</i>	192
<i>lread_oldsnap_nomag</i>	186, 196	<i>Lxyz</i>	24, 26, 36, 187, 195
<i>lread_oldsnap_nopscalar</i>	186, 196	<i>m</i>	117
<i>lresistive_gauge_disp</i>	192	<i>m1</i>	117
<i>lresistive_gauge_ee</i>	192	<i>M11</i>	252, 256, 260
<i>lrho_chi_corrected</i>	191	<i>M11cc</i>	252, 256, 260
<i>lrho_chi_inhom</i>	191	<i>M11ss</i>	252, 256, 260
<i>lrho_rad</i>	191	<i>M11z</i>	254, 258, 262
<i>lrho_rad_apply</i>	191	<i>M12cs</i>	252, 256, 260
<i>lrho_rad_apply2</i>	191	<i>m2</i>	117
<i>lroot</i>	117	<i>M22</i>	252, 256, 260
<i>lshift_origin</i>	186, 196	<i>M22cc</i>	252, 256, 260
<i>lshift_origin_lower</i>	186, 196	<i>M22ss</i>	252, 256, 260
<i>lsigE_const</i>	191	<i>M22z</i>	254, 258, 262
<i>lsigE_const_if_solve_for_phi</i>	191	<i>M33</i>	252, 256, 260
<i>lsigE_const_ifnot_solve_for_phi</i>	191	<i>M33z</i>	254, 258, 262
<i>lsmooth_Gamma_phi</i>	191	<i>mach</i>	268
<i>lsnap_tcrit</i>	186	<i>mag_flux</i>	249
<i>lsolve_for_phi_always</i>	191	<i>magfricmax</i>	243
<i>lsolve_for_phi_switch</i>	192	<i>MAGNETIC_INIT_PARS</i>	186, 196
<i>lspec_tcrit</i>	186	<i>Mamax</i>	214, 237
<i>lspecies_transfer [T]</i>	209	<i>Marms</i>	214, 237
<i>lstalk_ap</i>	194, 202	<i>mass</i>	214, 223, 228
<i>lstalk_bb</i>	194, 202	<i>massm</i>	228
<i>lstalk_grho</i>	194, 202	<i>maux</i>	33, 108
<i>lstalk_guu</i>	194, 202	<i>maxadvec</i>	151, 210
<i>lstalk_relvel</i>	194, 202	<i>mesh3Remax</i>	267
<i>lstalk_rho</i>	194, 202	<i>meshRemax</i>	267
<i>lstalk_uu</i>	194, 202	<i>mfpf</i>	229
<i>lstalk_vv</i>	194, 202	<i>mgam33</i>	263, 265, 266
<i>lstalk_xx</i>	194, 202	<i>mkap33</i>	263, 265, 266
<i>lswitch_toMHD_at_lna</i>	192	<i>MMxm</i>	230
<i>lswitch_toMHD_when_heating</i>	192	<i>MMym</i>	230
<i>lswitch_toMHD_when_nophi</i>	192	<i>MMzm</i>	230
<i>lthiele [T]</i>	209	<i>mpm</i>	246
<i>luminosity</i>	206	<i>mpmax</i>	246
<i>lupw_lnrho</i>	205	<i>mpmin</i>	246
<i>lupw_ss</i>	207	<i>mpoly0</i>	199, 200
<i>luse_Bext_in_b2</i>	207	<i>mpoly1</i>	200
<i>luse_bgb_as_jxb</i>	190	<i>mpoly2</i>	200
<i>luse_G_Newton</i>	195	<i>mrclncm</i>	246
<i>lvid_tcrit</i>	187	<i>mu</i>	250, 251
<i>lvisc_const_below_ascale</i>	195	<i>mu0</i>	73
<i>lwrite_2d</i>	187, 196	<i>mu2</i>	251
<i>lwrite_aux</i>	69, 108, 187, 196, 204	<i>mu51m</i>	222, 227

<i>mu53m</i>	222, 227	<i>nt</i>	7, 37, 202
<i>mu54m</i>	222, 227	<i>nu</i>	208, 209, 262
<i>mu5abs</i>	222, 227	<i>nu_epicycle</i>	193, 199, 208
<i>mu5b2m</i>	223, 228	<i>nu_hyper2</i>	208
<i>mu5bjm</i>	222, 228	<i>nu_hyper3</i>	208
<i>mu5bjrms</i>	223, 228	<i>nu_LES</i>	267
<i>mu5bxm</i>	223, 228	<i>nu_liq</i>	194
<i>mu5jbm</i>	223, 228	<i>nu_tdep</i>	267
<i>mu5m</i>	222, 227	<i>nucrate</i>	227
<i>mu5max</i>	222, 227	<i>nuclrmin</i>	227
<i>mu5min</i>	222, 227	<i>num</i>	267
<i>mu5rms</i>	222, 227	<i>numax</i>	267
<i>muc1</i>	264	<i>numin</i>	267
<i>muc2</i>	264	<i>numx</i>	288
<i>mucz</i>	264	<i>nuQ</i>	262
<i>mumz</i>	283	<i>nusmagm</i>	267
<i>muSm</i>	222, 227	<i>nusmagmax</i>	267
<i>muSmax</i>	222, 227	<i>nusmagmin</i>	267
<i>muSrms</i>	222, 227	<i>nv</i>	117
<i>muz</i>	250, 251	<i>nvar</i>	27, 90
<i>mvar</i>	33, 85, 186, 196	<i>nx</i>	30, 110, 117, 118
<i>mx</i>	27, 117, 118	<i>nxgrid</i>	53, 55, 117
<i>my</i>	27, 117, 118	<i>ny</i>	30, 117, 156
<i>mz</i>	27, 117, 118	<i>nygrid</i>	41
<i>n</i>	117	<i>nz</i>	30, 117, 156
<i>n1</i>	117	<i>nzgrid</i>	41
<i>n1s</i>	296, 299, 301	<i>o2</i>	267
<i>n2</i>	117	<i>o2m</i>	213, 237
<i>n_segment_x</i>	209	<i>o2mphi</i>	269
<i>nfr</i>	297–300	<i>o2mz</i>	271, 280
<i>ngam33</i>	263, 265, 266	<i>o2sphm</i>	213
<i>nil</i>	298, 304	<i>o2u2m</i>	213
<i>nil,'</i>	300	<i>obm</i>	217
<i>nil','no</i>	297, 299, 303	<i>obmz</i>	277
<i>nkcap33</i>	263, 265, 266	<i>odel2um</i>	213, 237
<i>nlin0</i>	233	<i>offset_min_calc</i>	187
<i>nlin1</i>	233	<i>ofm</i>	232
<i>nlin2</i>	233	<i>oglnrxgdum</i>	213
<i>noentropy.f90</i>	183	<i>ogux2mz</i>	274, 281
<i>nogpu.f90</i>	183	<i>oguxmz</i>	274, 281
<i>nohydro.f90</i>	183	<i>oguy2mz</i>	274, 281
<i>nopower_spectrum.f90</i>	183	<i>oguymz</i>	274, 281
<i>noyinyang.f90</i>	183	<i>oguz2mz</i>	274, 281
<i>noyinyang_mpi.f90</i>	183	<i>oguzmz</i>	274, 281
<i>npm</i>	245, 246	<i>omax</i>	213, 237
<i>nprocy</i>	51, 186, 195	<i>Omega</i>	204
<i>nprocz</i>	51	<i>omega_ff</i>	208
<i>nr1</i>	297	<i>omumz</i>	212, 236
<i>nr_directions</i>	56	<i>oned</i>	204
<i>nrhom</i>	237	<i>onedall</i>	204

<i>oo</i>	267	<i>oy2mxz</i>	289
<i>opmphi</i>	213	<i>oy2mz</i>	272, 281
<i>ormphi</i>	213	<i>oy3m</i>	213
<i>orms</i>	213, 237	<i>oy4m</i>	213
<i>ou0</i>	302, 304	<i>oydivu2mz</i>	274, 281
<i>ou_int</i>	213, 237	<i>oydivumz</i>	274, 281
<i>ou_spec</i>	204	<i>oymxy</i>	291, 294
<i>oud</i>	302, 304	<i>oymxz</i>	289
<i>ouf</i>	302, 304	<i>oymz</i>	272, 280
<i>oum</i>	26, 213, 236	<i>oyozm</i>	213, 237
<i>oumph</i>	213, 237	<i>oyph1mz</i>	273
<i>oumx</i>	286, 288	<i>oyph2mz</i>	273
<i>oumxy</i>	291, 294	<i>oyph3mz</i>	273
<i>oumxz</i>	289, 290	<i>oyuxymz</i>	274, 281
<i>oumy</i>	284, 285	<i>oyuyymz</i>	274, 281
<i>oumz</i>	274, 281	<i>oyuzym</i>	213, 237
<i>ouph1mz</i>	273	<i>oyuzymz</i>	274, 281
<i>ouph2mz</i>	273	<i>oz2m</i>	213, 237
<i>ouph3mz</i>	273	<i>oz2mx</i>	286, 288
<i>ourms</i>	213, 235	<i>oz2mxy</i>	292
<i>out</i>	297–300, 302, 304	<i>oz2mxz</i>	289
<i>out1</i>	203	<i>oz2mz</i>	272, 281
<i>out2</i>	203	<i>oz3m</i>	213
<i>outm</i>	210	<i>oz4m</i>	213
<i>ovr</i>	296, 298, 302, 304	<i>ozdivu2mz</i>	274, 281
<i>ox2m</i>	213, 237	<i>ozdivumz</i>	274, 281
<i>ox2mx</i>	286, 288	<i>ozmphi</i>	213
<i>ox2mxy</i>	292	<i>ozmxy</i>	291, 294
<i>ox2mxz</i>	289	<i>ozmz</i>	272, 280
<i>ox2mz</i>	272, 281	<i>ozph1mz</i>	273
<i>ox3m</i>	213	<i>ozph2mz</i>	273
<i>ox4m</i>	213	<i>ozph3mz</i>	273
<i>oxdivu2mz</i>	274, 281	<i>p</i>	295, 297, 298, 300, 301, 303
<i>oxdivumz</i>	274, 281	<i>p1D</i>	301, 303
<i>oxmxy</i>	291, 294	<i>p%curlb</i>	73
<i>oxmz</i>	272, 280	<i>p%jj_ohm</i>	73
<i>oxoym</i>	213, 237	<i>particles_adsorbed.f90</i>	183
<i>oxozm</i>	213, 237	<i>particles_chemistry.f90</i>	183
<i>oxph1mz</i>	273	<i>particles_surfspec.f90</i>	183
<i>oxph2mz</i>	273	<i>PATH</i>	4, 11
<i>oxph3mz</i>	273	<i>pc_build</i>	307
<i>oxum</i>	213	<i>pc_run</i>	307
<i>oxurms</i>	213, 235	<i>pc_start</i>	307
<i>oxuxxmz</i>	274, 281	<i>pc_git</i>	307
<i>oxuyxmz</i>	274, 281	<i>pdf_max</i>	209
<i>oxuzxm</i>	213, 237	<i>pdf_max_logscale</i>	209
<i>oxuzxmz</i>	274, 281	<i>pdf_min</i>	209
<i>oy2m</i>	213, 237	<i>pdf_min_logscale</i>	209
<i>oy2mx</i>	286, 288	<i>pdivum</i>	215, 231, 245
<i>oy2mxy</i>	292	<i>pdivumz</i>	275

<i>peffmxz</i>	290	<i>pretend lnTT</i>	187, 197
<i>PENCIL_HOME</i>	4, 82	<i>PrM</i>	195
<i>PENCIL_HOST_ID</i>	17	<i>pscalar_diff</i>	208
<i>pertss</i>	199	<i>PSCALAR_INIT_PARS</i>	186, 196
<i>pfc</i>	297–300	<i>pscalar_sink_rate</i>	209
<i>pfe</i>	301, 303	<i>psi</i>	224
<i>phi</i>	224, 225	<i>psi11</i>	252, 256
<i>phi11</i>	252, 256	<i>psi12</i>	252, 256
<i>phi12</i>	252, 256	<i>psi21</i>	252, 256
<i>phi21</i>	252, 256	<i>psi22</i>	252, 256
<i>phi22</i>	252, 256	<i>psi2m</i>	238, 239
<i>phi2m</i>	225, 226, 238, 239	<i>psi_anal</i>	224
<i>phi32</i>	252, 256	<i>psiddot</i>	224
<i>phibmx</i>	221, 244	<i>psidot</i>	224
<i>phibmy</i>	221, 244	<i>psiL</i>	224
<i>phibmz</i>	221, 244	<i>psim</i>	238, 239
<i>phibzm</i>	240	<i>psirms</i>	238, 239
<i>phibzmz</i>	240	<i>puzmz</i>	283
<i>phidot</i>	224, 225	<i>puzm</i>	213
<i>phiK</i>	252, 256	<i>puzmxy</i>	292
<i>phiM</i>	252, 256	<i>pwd</i>	301, 303
<i>phim</i>	225, 226, 238–240	<i>PYTHONPATH</i>	47
<i>phiMK</i>	252, 256		
<i>phimphi</i>	269	<i>Q</i>	224
<i>phip2</i>	240	<i>q2m</i>	213, 237
<i>phipt</i>	240	<i>qam</i>	244
<i>phirms</i>	225, 226, 238, 239	<i>Qddot</i>	224
<i>pl</i>	297	<i>Qdot</i>	224
<i>placeholder</i>	209	<i>qem</i>	244
<i>plasma_frictm</i>	238	<i>gezsum</i>	213
<i>polytrm</i>	246	<i>qfm</i>	213, 237
<i>pot</i>	301, 303	<i>qfviscm</i>	267
<i>power_spectrum.f90</i>	183	<i>qmax</i>	55, 56, 213, 222, 234, 237
<i>Poynting</i>	268	<i>qom</i>	213, 237
<i>poynxmxy</i>	293, 294	<i>qpm</i>	244
<i>poynymxy</i>	293, 294	<i>qpmz</i>	283
<i>poynzmxy</i>	293, 295	<i>qq1m</i>	240, 247
<i>poynzmz</i>	278, 283	<i>qq2m</i>	240, 247
<i>poynzph1mz</i>	279	<i>qq3m</i>	240, 247
<i>poynzph2mz</i>	279	<i>qq4m</i>	240, 247
<i>poynzph3mz</i>	279	<i>Qrad</i>	268
<i>pp</i>	268, 299, 300	<i>qrms</i>	213, 222, 235, 237
<i>ppm</i>	215, 231, 249, 250, 266	<i>qsatmin</i>	235
<i>ppmax</i>	215	<i>qsatrms</i>	235
<i>ppmin</i>	215	<i>qsglnrhom</i>	213
<i>ppmphi</i>	269	<i>qshear</i>	202, 209
<i>ppmx</i>	286, 288	<i>qsm</i>	244
<i>ppmy</i>	284, 285	<i>quxom</i>	213, 237
<i>ppmz</i>	275, 283, 284	<i>quysm</i>	213
<i>pr1mz</i>	283	<i>qxmax</i>	235
		<i>qxmin</i>	234

<i>qymax</i>	235	<i>rhoccm</i>	222, 246
<i>qymin</i>	235	<i>rhoccmx</i>	246
<i>qzmax</i>	235	<i>rhodownmz</i>	274
<i>qzmin</i>	235	<i>rhoe</i>	225
<i>r_ext</i>	207	<i>rhoem</i>	229
<i>r_ff</i>	208	<i>rhoerms</i>	229
<i>r_int</i>	207	<i>rhof2downmz</i>	274
<i>rad</i>	240, 247	<i>rhof2m</i>	214
<i>radius</i>	190, 201	<i>rhof2mz</i>	274
<i>radius_ss</i>	199	<i>rhof2upmz</i>	274
<i>random_gen</i>	187, 197, 204	<i>rhoHCmz</i>	282
<i>rcool</i>	207	<i>rhoLm</i>	237
<i>rcylmphi</i>	269	<i>rhom</i>	8, 26, 214, 223
<i>rdamp</i>	205	<i>rhomax</i>	214, 228
<i>rdampext</i>	205	<i>rhomaxloc</i>	214
<i>rdampint</i>	205	<i>rhomin</i>	214, 228
<i>rdivum</i>	212, 236	<i>rhominloc</i>	214
REAL PRECISION	52, 154	<i>rhomphi</i>	269, 270
<i>redshift</i>	223, 224	<i>rhomx</i>	286
<i>reinitialize_lnc</i>	208	<i>rhomxmask</i>	214
<i>relhel</i>	161, 208	<i>rhomxy</i>	292
<i>relhel_uu</i>	138	<i>rhomxz</i>	289
<i>Remz</i>	274, 281	<i>rhomy</i>	284
<i>Reshock</i>	267	<i>rhomz</i>	274
<i>rgxm</i>	234	<i>rhomzmask</i>	214
<i>rho</i>	268	<i>rhoph1mz</i>	275
<i>rho0</i>	193, 198, 205, 209	<i>rhoph2mz</i>	275
<i>rho0m</i>	253	<i>rhoph3mz</i>	275
<i>rho12m</i>	214	<i>rhorms</i>	214
<i>rho2downmz</i>	274	<i>rhoT</i>	224
<i>rho2m</i>	214	<i>rhoupmz</i>	274
<i>rho2mx</i>	275	<i>rhoW1m</i>	231
<i>rho2mxy</i>	292	<i>rhoW1rms</i>	230
<i>rho2mz</i>	274	<i>rhoW2m</i>	231
<i>rho2ph1mz</i>	275	<i>rhoW2rms</i>	230
<i>rho2ph2mz</i>	275	<i>rhoW3m</i>	231
<i>rho2ph3mz</i>	275	<i>rhoW3rms</i>	230
<i>rho2upmz</i>	274	<i>rlx2m</i>	212, 236
<i>rho4m</i>	214	<i>rlxm</i>	212, 236
<i>rho6m</i>	214	<i>rly2m</i>	212, 236
<i>rho8m</i>	214	<i>rlym</i>	212, 236
<i>rho_chi</i>	225, 226, 238, 239	<i>rlz2m</i>	212, 236
<i>rho_left</i>	189, 198	<i>rlzm</i>	212, 236
<i>rho_phi</i>	238	<i>Rmesh</i>	210
<i>rho_rad</i>	225	<i>Rmesh3</i>	210
<i>rho_right</i>	189, 198	<i>Rmmz</i>	220
<i>rhob</i>	225	<i>rmphi</i>	269
<i>rhoc2m</i>	246	<i>Rring1,Rring2</i>	201
<i>rhoc3m</i>	246	<i>rufm</i>	232
<i>rhoccm</i>	222, 246	<i>rufmz</i>	274
		<i>rugm</i>	234

<i>rugpotselfm</i>	248	<i>ruyuz2mz</i>	273, 281
<i>rumax</i>	212, 236	<i>ruyuzm</i>	212, 236
<i>rupmphi</i>	269	<i>ruyuzmx</i>	286, 287
<i>rupuzmphi</i>	269	<i>ruyuzmxy</i>	292, 294
<i>rurmphi</i>	269	<i>ruyuzmz</i>	273, 281
<i>rursphmphi</i>	269	<i>ruz2m</i>	212, 236
<i>rurupmphi</i>	269	<i>ruz2mx</i>	286, 287
<i>ruruzmphi</i>	269	<i>ruz2mxy</i>	292, 294
<i>ruthmphi</i>	269	<i>ruz2mz</i>	273, 281
<i>rux2m</i>	212, 236	<i>ruz2ph1mz</i>	273
<i>rux2mx</i>	286, 287	<i>ruz2ph2mz</i>	273
<i>rux2mxy</i>	292, 294	<i>ruz2ph3mz</i>	273
<i>rux2mz</i>	273, 281	<i>ruzdownmz</i>	272, 280
<i>rux2ph1mz</i>	273	<i>ruzm</i>	212, 236
<i>rux2ph2mz</i>	273	<i>ruzmphi</i>	269
<i>rux2ph3mz</i>	273	<i>ruzmx</i>	286, 287
<i>ruxdownmxy</i>	291	<i>ruzmxy</i>	292, 294
<i>ruxm</i>	212, 236	<i>ruzmz</i>	272, 281
<i>ruxmx</i>	285, 287	<i>ruzph1mz</i>	272
<i>ruxmxy</i>	292, 294	<i>ruzph2mz</i>	272
<i>ruxmz</i>	272, 281	<i>ruzph3mz</i>	272
<i>ruxph1mz</i>	272	<i>ruzupmz</i>	272, 280
<i>ruxph2mz</i>	272	<i>Rxydownmxy</i>	291
<i>ruxph3mz</i>	272	<i>Rxydownmz</i>	273
<i>ruxt看</i>	212, 236	<i>Rxymxy</i>	291
<i>ruxupmxy</i>	291	<i>Rxymz</i>	273
<i>ruxuy2mz</i>	273, 281	<i>Rxyupmxy</i>	291
<i>ruxuym</i>	212, 236	<i>Rxyupmz</i>	273
<i>ruxuymx</i>	286, 287	<i>Rxzdownmxy</i>	291
<i>ruxuymxy</i>	292, 294	<i>Rxzdownmz</i>	273
<i>ruxuymz</i>	273, 281	<i>Rxzmxy</i>	291
<i>ruxuz2mz</i>	273, 281	<i>Rxzmz</i>	273
<i>ruxuzm</i>	212, 236	<i>Rxzupmxy</i>	291
<i>ruxuzmx</i>	286, 287	<i>Rxzupmz</i>	273
<i>ruxuzmxy</i>	292, 294	<i>Ryzdownmxy</i>	291
<i>ruxuzmz</i>	273, 281	<i>Ryzdownmz</i>	273
<i>ruy2m</i>	212, 236	<i>Ryzmxy</i>	291
<i>ruy2mx</i>	286, 287	<i>Ryzmz</i>	273
<i>ruy2mxy</i>	292, 294	<i>Ryzupmxy</i>	291
<i>ruy2mz</i>	273, 281	<i>Ryzupmz</i>	273
<i>ruy2ph1mz</i>	273	<i>s</i>	295, 297–301, 303
<i>ruy2ph2mz</i>	273	<i>s+f</i>	299, 300
<i>ruy2ph3mz</i>	273	<i>s0d</i>	295, 297, 299–301, 303
<i>ruym</i>	212, 236	<i>s2kzDFm</i>	252, 256, 260
<i>ruymx</i>	285, 287	<i>sa2</i>	297, 298
<i>ruymxy</i>	292, 294	<i>Sc</i>	189
<i>ruymz</i>	272, 281	<i>sds</i>	295, 299, 300
<i>ruyph1mz</i>	272	<i>sep</i>	299, 300, 302
<i>ruyph2mz</i>	272	<i>set</i>	296, 298–300, 302, 304
<i>ruyph3mz</i>	272	<i>sf</i>	295, 299, 301, 303

<i>sfr</i>	297–300	<i>ssdownmz</i>	275
<i>Shchm</i>	245	<i>sse</i>	299, 300
<i>shock</i>	268	<i>ssf2downmz</i>	275
<i>shockmax</i>	248	<i>ssf2mz</i>	275
<i>shx</i>	296	<i>ssf2upmz</i>	275
<i>shy</i>	296	<i>ssm</i>	8, 215, 231, 249
<i>shz</i>	296	<i>ssmax</i>	215
<i>sig1</i>	252	<i>ssmin</i>	215
<i>sig2</i>	252	<i>ssmphl</i>	269, 270
<i>sig3</i>	252	<i>ssmx</i>	286
<i>sigBBEm</i>	229	<i>ssmxy</i>	232, 292
<i>sigBm</i>	229	<i>ssmxz</i>	232, 289
<i>sigBma</i>	225, 226, 238–240	<i>ssmy</i>	284
<i>sigBrms</i>	229	<i>ssmz</i>	275, 283
<i>sigE_ceiling</i>	193	<i>ssruzmm</i>	215
<i>sigE_const_value</i>	193	<i>ssupmz</i>	275
<i>sigEE2m</i>	229	<i>ssuzm</i>	215
<i>sigEm</i>	229	<i>sT</i>	296, 298–301, 303
<i>sigEma</i>	225, 226, 238–240	<i>st</i>	296
<i>sigEmax</i>	193	<i>st2</i>	296
<i>sigErms</i>	229	<i>Stgm</i>	233
<i>sigma</i>	238, 244	<i>STimp2</i>	232
<i>Sij2m</i>	267	<i>STimpt</i>	232
<i>sijbibjm</i>	221	<i>StokesImxy</i>	293, 295
<i>sijoiojm</i>	267	<i>StokesQ1mxy</i>	293, 295
<i>sijxxmz</i>	284	<i>StokesQmxy</i>	293, 295
<i>sijxymz</i>	284	<i>StokesU1mxy</i>	293, 295
<i>sijxzmz</i>	284	<i>StokesUmxy</i>	293, 295
<i>sijyymz</i>	284	<i>STrep2</i>	232
<i>sijyzmz</i>	284	<i>STrept</i>	232
<i>sijzzmz</i>	284	<i>strTpt</i>	232
<i>slc</i>	297, 299, 303	<i>strXpt</i>	232
<i>slice_position</i>	27, 28, 203	<i>StS</i>	302, 304
<i>slo</i>	296, 298	<i>SXimp2</i>	232
<i>slp</i>	296	<i>SXimpt</i>	232
<i>spd</i>	297, 298	<i>SXrep2</i>	232
<i>sphmass</i>	214	<i>SXrept</i>	232
<i>spr</i>	296, 297	<i>t</i>	8, 26, 117, 210
<i>spt</i>	299, 300	<i>T00m</i>	211
<i>sr1</i>	297	<i>T0irms</i>	212
<i>srce5m</i>	222, 227	<i>T0x2m</i>	211
<i>ss</i>	268, 295, 297, 299, 300	<i>T0y2m</i>	211
<i>ss2downmz</i>	275	<i>T0z2m</i>	212
<i>ss2m</i>	215, 231	<i>tau1</i>	252
<i>ss2mphl</i>	269	<i>tau2</i>	252
<i>ss2mx</i>	286	<i>taucmin</i>	237
<i>ss2mz</i>	275	<i>tauerror</i>	267
<i>ss2upmz</i>	275	<i>tauheat_buffer</i>	207
<i>ss_run_averm</i>	215	<i>tauhtmin</i>	215
<i>ssbycpm</i>	215	<i>tauk</i>	240, 247

<i>tauqmax</i>	234	<i>TT2m</i>	216, 249
<i>tavg</i>	31, 32, 204	<i>TT2mx</i>	286
<i>tdamp</i>	204, 205	<i>TT2mz</i>	275, 283
<i>Tdxpm</i>	249	<i>TT2upmz</i>	275
<i>Tdypm</i>	249	<i>TTdownmz</i>	275
<i>Tdzpm</i>	249	<i>TTf2downmz</i>	275
<i>tensor_pscalar_diff</i>	208	<i>TTf2mz</i>	275
<i>test_chemistry.f90</i>	183	<i>TTf2upmz</i>	275
<i>thcool</i>	249	<i>TTheat_buffer</i>	207
<i>theta</i>	204	<i>TTm</i>	215, 249, 250, 266
<i>timestep.f90</i>	183	<i>TTmax</i>	215, 249, 250, 266
<i>timestep_strang.f90</i>	184	<i>TTmaxloc</i>	215
<i>timestep_subcycle.f90</i>	184	<i>TTmin</i>	215, 249, 250, 266
<i>TL</i>	224	<i>TTminloc</i>	215
<i>TLdoteff2km</i>	224	<i>TTmphi</i>	269
<i>TLdoteff2m</i>	224	<i>TTmx</i>	286, 288
<i>TLeff2km</i>	224	<i>TTmxy</i>	292, 295
<i>TLeff2m</i>	224	<i>TTmxz</i>	289, 291
<i>tmax</i>	203	<i>TTmy</i>	284, 285
<i>tot_ang_mom</i>	213, 236	<i>TTmz</i>	275, 283, 284
<i>total_carbon_sites</i>	202	<i>ttransient</i>	204
<i>totalforcezdownmz</i>	274	<i>TTref</i>	250
<i>totalforcezmz</i>	274	<i>TTtop</i>	215, 231
<i>totalforcezupmz</i>	274	<i>TTupmz</i>	275
<i>totmass</i>	214	<i>TTzmask</i>	249
<i>tph</i>	223, 224, 226	<i>TugTm</i>	249
<i>Tppm</i>	250	<i>Tugux_uxugTm</i>	249
<i>TR</i>	224	<i>Tuguy_uyugTm</i>	249
<i>TR_anal</i>	224	<i>Tuguz_uzugTm</i>	249
<i>TRddot</i>	224	<i>tvid_crit_log_interval</i>	187
<i>TRdot</i>	224	<i>TVolm</i>	246
<i>TRdoteff2km</i>	224	<i>TVolnm</i>	246
<i>TRdoteff2m</i>	224	<i>TVolpm</i>	246
<i>TRdotpsim</i>	224	<i>Txxm</i>	211
<i>TReff2km</i>	224	<i>Txym</i>	211
<i>TReff2m</i>	224	<i>Tyym</i>	211
<i>trigger_snap</i>	187	<i>Tyzm</i>	211
<i>trigger_spec</i>	194	<i>Tzxm</i>	211
<i>trigger_vid</i>	187	<i>Tzzm</i>	211
<i>Trms</i>	249	<i>u0max</i>	253, 257
<i>TRpsidotm</i>	224	<i>u0rms</i>	253, 257
<i>TRpsikm</i>	224	<i>u11rms</i>	253, 256
<i>TRpsim</i>	224	<i>u12rms</i>	253, 256
<i>TrSigmapm</i>	245	<i>u1u23m</i>	212, 236
<i>ts</i>	45	<i>u1u32m</i>	212, 236
<i>tsnap_crit_log_interval</i>	187	<i>u2</i>	267
<i>tspec_crit_log_interval</i>	187	<i>u21rms</i>	253, 256
<i>TT</i>	268	<i>u22rms</i>	253, 256
<i>tt1m</i>	240, 247	<i>u2m</i>	210, 235
<i>TT2downmz</i>	275	<i>u2mphi</i>	269, 270

<i>u2mx</i>	285	<i>ujxb1m</i>	229
<i>u2mz</i>	271, 280	<i>ujxbm</i>	221, 244
<i>u2ph1mz</i>	271	<i>ujxbmz</i>	277
<i>u2ph2mz</i>	272	<i>uL</i>	224
<i>u2ph3mz</i>	272	<i>uLeff2km</i>	224
<i>u2sphm</i>	210	<i>uLeff2m</i>	224
<i>u2tm</i>	210, 235	<i>umamz</i>	212, 236
<i>u2u13m</i>	212, 236	<i>umax</i>	8, 211, 235
<i>u2u31m</i>	212, 236	<i>umbmz</i>	212, 236
<i>u3u12m</i>	212, 236	<i>umin</i>	211, 235
<i>u3u21m</i>	212, 236	<i>umx</i>	30, 212, 236
<i>u4m</i>	211	<i>umxbmz</i>	212, 236
<i>u6m</i>	211	<i>umy</i>	30, 212, 236
<i>u8m</i>	211	<i>umz</i>	30, 212, 236
<i>uabxmz</i>	276, 282	<i>unit_density</i>	37, 187, 197
<i>uabymz</i>	276, 282	<i>unit_length</i>	37, 187, 197
<i>uabzmz</i>	276, 282	<i>unit_system</i>	37, 188, 196
<i>uam</i>	217, 241	<i>unit_temperature</i>	37, 188, 197
<i>uamz</i>	277, 282	<i>unit_velocity</i>	37, 188, 197
<i>ub0m</i>	257	<i>uotm</i>	210
<i>ubbzm</i>	216, 240	<i>up2mphi</i>	269
<i>ubgbpm</i>	221	<i>upmphi</i>	269, 270
<i>ubm</i>	217, 240	<i>upuzmphi</i>	269
<i>ubmxy</i>	293	<i>uR</i>	224
<i>ubmz</i>	277, 282	<i>ur2mphi</i>	269
<i>ubs</i>	302, 304	<i>urand</i>	188, 198
<i>ubtm</i>	216	<i>uReff2km</i>	224
<i>ucm</i>	246	<i>uReff2m</i>	224
<i>udpxxm</i>	214, 237	<i>urmpphi</i>	269, 270
<i>uduum</i>	211	<i>urms</i>	8, 26, 138, 211, 235
<i>uEL2</i>	248	<i>urmsx</i>	211, 235
<i>ufm</i>	232	<i>urmsz</i>	211, 235
<i>ufpresm</i>	216	<i>uRR2</i>	248
<i>ufviscm</i>	267	<i>uRRm</i>	248
<i>ugb22m</i>	221	<i>ursphmphi</i>	269, 270
<i>uglnrhom</i>	214	<i>ursphTTmphi</i>	270
<i>uglnrhomz</i>	275	<i>urupmphi</i>	269
<i>ugm</i>	234	<i>uruzmphi</i>	269
<i>ugradpmz</i>	275	<i>uSH2</i>	248
<i>ugrhom</i>	214, 223	<i>uthmphi</i>	269, 270
<i>ugrhomz</i>	275	<i>uu</i>	267
<i>ugu2m</i>	213	<i>uu and</i>	161
<i>ugurmsx</i>	213	<i>uu0</i>	161
<i>uguxmxy</i>	292	<i>uu left</i>	188, 198
<i>uguymxy</i>	292	<i>uu right</i>	188, 198
<i>uguzmxy</i>	292	<i>uudcm</i>	246
<i>uj0m</i>	257	<i>uumphi</i>	269, 270
<i>ujm</i>	217, 241	<i>uusphmphi</i>	269, 270
<i>ujmz</i>	277	<i>uut</i>	108
<i>ujtm</i>	216	<i>ux0m</i>	253, 257

<i>ux0mz</i>	262	<i>uxTTmx</i>	286
<i>ux11m</i>	253, 257	<i>uxTTmxy</i>	292
<i>ux2ccm</i>	212, 235	<i>uxTTmz</i>	275
<i>ux2downmxy</i>	291	<i>uxupmxy</i>	291
<i>ux2m</i>	211, 235	<i>uxuy2m</i>	211
<i>ux2mx</i>	286, 287	<i>uxuyesm</i>	212, 236
<i>ux2mxy</i>	292, 294	<i>uxuydivum</i>	214, 237
<i>ux2mxz</i>	289, 290	<i>uxuym</i>	212, 236
<i>ux2mz</i>	272, 280	<i>uxuymx</i>	286, 288
<i>ux2ph1mz</i>	272	<i>uxuymxy</i>	291, 294
<i>ux2ph2mz</i>	272	<i>uxuymxz</i>	289, 290
<i>ux2ph3mz</i>	272	<i>uxuymz</i>	273, 281
<i>ux2ssm</i>	212, 235	<i>uxuypt</i>	210
<i>ux2upmxy</i>	291	<i>uxuzm</i>	212, 236
<i>ux3m</i>	211	<i>uxuzmx</i>	286, 288
<i>ux3mz</i>	272	<i>uxuzmxy</i>	291, 294
<i>ux4m</i>	211	<i>uxuzmxz</i>	289, 290
<i>ux4mz</i>	272	<i>uxuzmz</i>	273, 281
<i>uxbm</i>	220, 243	<i>uxxrms</i>	214, 237
<i>uxbmx</i>	221, 243	<i>uxzrms</i>	214, 237
<i>uxbmy</i>	221, 243	<i>uy0m</i>	253, 257
<i>uxbmz</i>	221, 243	<i>uy0mz</i>	262
<i>uxbxm</i>	217, 241	<i>uy11m</i>	253, 257
<i>uxbxmz</i>	277, 282	<i>uy2ccm</i>	212, 235
<i>uxbym</i>	217, 241	<i>uy2m</i>	211, 235
<i>uxbymz</i>	277, 282	<i>uy2mx</i>	286, 287
<i>uxbzm</i>	217, 241	<i>uy2mxy</i>	292, 294
<i>uxbzmz</i>	277, 283	<i>uy2mxz</i>	289, 290
<i>uxcm</i>	247	<i>uy2mz</i>	272, 280
<i>uxdownmxy</i>	291	<i>uy2ph1mz</i>	272
<i>uxglnrmy</i>	214, 237	<i>uy2ph2mz</i>	272
<i>uxjxm</i>	217	<i>uy2ph3mz</i>	272
<i>uxjym</i>	217	<i>uy2ssm</i>	212, 236
<i>uxjzm</i>	217	<i>uy3m</i>	211
<i>uxm</i>	211, 235	<i>uy3mz</i>	272
<i>uxmax</i>	211, 235	<i>uy4m</i>	211
<i>uxmin</i>	211, 235	<i>uy4mz</i>	272
<i>uxmx</i>	285, 287	<i>uybxm</i>	217, 241
<i>uxmxy</i>	291, 294	<i>uybxmxz</i>	289, 290
<i>uxmxz</i>	289, 290	<i>uybxmz</i>	277, 282
<i>uxmy</i>	284, 285	<i>uybym</i>	217, 241
<i>uxmz</i>	271, 280	<i>uybymz</i>	277, 282
<i>uxp2</i>	210, 235	<i>uybzm</i>	217, 241
<i>uxph1mz</i>	271	<i>uybzmz</i>	289, 290
<i>uxph2mz</i>	271	<i>uybzmz</i>	277, 283
<i>uxph3mz</i>	271	<i>uycm</i>	247
<i>uxpt</i>	210, 235	<i>uyglnrxm</i>	214, 237
<i>uxrms</i>	211, 235	<i>uygzlnrhomz</i>	275
<i>uxTm</i>	249	<i>uyjxm</i>	217
<i>uxTmz</i>	283	<i>uyjym</i>	217

<i>uyjzm</i>	217	<i>uzbym</i>	217, 241
<i>uym</i>	211, 235	<i>uzbymz</i>	277, 282
<i>uymax</i>	211, 235	<i>uzbzm</i>	217, 241
<i>uym_{in}</i>	211, 235	<i>uzbzmz</i>	277, 283
<i>uymx</i>	285, 287	<i>uzcm</i>	247
<i>uymxy</i>	291, 294	<i>uzcx10m</i>	211
<i>uymxz</i>	289, 290	<i>uzdivum</i>	214, 237
<i>uymy</i>	284, 285	<i>uzdivumz</i>	272, 280
<i>uymz</i>	271, 280	<i>uzdownmz</i>	272, 280
<i>uyp2</i>	210, 235	<i>uzgylnrhomz</i>	275
<i>uyph1mz</i>	271	<i>uzjx1z</i>	261
<i>uyph2mz</i>	271	<i>uzjx2z</i>	261
<i>uyph3mz</i>	271	<i>uzjx3z</i>	261
<i>uypt</i>	210, 235	<i>uzjx4z</i>	261
<i>uyrms</i>	211, 235	<i>uzjxm</i>	217
<i>uyTm</i>	249	<i>uzjy1z</i>	261
<i>uyTmz</i>	283	<i>uzjy2z</i>	261
<i>uyTTmx</i>	286	<i>uzjy3z</i>	261
<i>uyTTmxy</i>	292	<i>uzjy4z</i>	261
<i>uyTTmz</i>	275	<i>uzjym</i>	217
<i>uyuz2m</i>	211	<i>uzjz1z</i>	261
<i>uyuzm</i>	212, 236	<i>uzjz2z</i>	261
<i>uyuzmx</i>	286, 288	<i>uzjz3z</i>	261
<i>uyuzmxy</i>	291, 294	<i>uzjz4z</i>	261
<i>uyuzmxz</i>	289, 290	<i>uzjzm</i>	217
<i>uyuzmz</i>	273, 281	<i>uzm</i>	211, 235
<i>uyuzpt</i>	210	<i>uzmax</i>	211, 235
<i>uyxuzxmz</i>	274, 281	<i>uzmin</i>	211, 235
<i>uyyrms</i>	214, 237	<i>uzmphi</i>	269, 270
<i>uyyuzymz</i>	274, 281	<i>uzmx</i>	285, 287
<i>uyzrms</i>	214, 237	<i>uzmxy</i>	291, 294
<i>uyzuzzmz</i>	274, 281	<i>uzmxz</i>	289, 290
<i>uz0mz</i>	262	<i>uzmy</i>	284, 285
<i>uz2downmz</i>	272	<i>uzmz</i>	271, 280
<i>uz2m</i>	211, 235	<i>uzp2</i>	210, 235
<i>uz2mphi</i>	269	<i>uzph1mz</i>	271
<i>uz2mx</i>	286, 288	<i>uzph2mz</i>	271
<i>uz2mxy</i>	292, 294	<i>uzph3mz</i>	271
<i>uz2mxz</i>	289, 290	<i>uzpt</i>	210, 235
<i>uz2mz</i>	272, 281	<i>uzrms</i>	211, 235
<i>uz2ph1mz</i>	272	<i>uzsx10m</i>	211
<i>uz2ph2mz</i>	272	<i>uzTm</i>	249
<i>uz2ph3mz</i>	272	<i>uzTmz</i>	283
<i>uz2upmz</i>	272	<i>uzTTdownmz</i>	276
<i>uz3m</i>	211	<i>uzTTmx</i>	286
<i>uz3mz</i>	272	<i>uzTTmxy</i>	292
<i>uz4m</i>	211	<i>uzTTmz</i>	276
<i>uz4mz</i>	272	<i>uzTTupmz</i>	276
<i>uzbxm</i>	217, 241	<i>uzupmz</i>	272, 280
<i>uzbxmz</i>	277, 282	<i>uzux2m</i>	211

<i>uzuxpt</i>	210	<i>W2xm</i>	231
<i>uzyrms</i>	214, 237	<i>W2xmz</i>	280
<i>uzzrms</i>	214	<i>W2ym</i>	231
<i>v</i>	296, 297, 299–301, 303	<i>W2ymz</i>	280
<i>v3</i>	299–301, 303	<i>W2zm</i>	231
<i>vA23rms</i>	219	<i>W2zmz</i>	280
<i>vAm</i>	227	<i>W3ddotrms</i>	230
<i>vAmax</i>	219, 227, 242	<i>W3dotW3m</i>	231
<i>vAmin</i>	227	<i>W3max</i>	230
<i>vAmxz</i>	290	<i>W3rms</i>	230
<i>vArms</i>	219, 242	<i>W3xm</i>	231
<i>vel_spec</i>	204	<i>W3xmz</i>	280
<i>VelVolm</i>	246	<i>W3ym</i>	231
<i>VelVolnm</i>	246	<i>W3ymz</i>	280
<i>VelVolpm</i>	246	<i>W3zm</i>	231
<i>velxrms</i>	211	<i>W3zmz</i>	280
<i>velxx2m</i>	211	<i>w_forc</i>	205
<i>velxy2m</i>	211	<i>wall_lorentz</i>	238
<i>velxz2m</i>	211	<i>wall_pos</i>	238
<i>visc_heatm</i>	267	<i>wall_vel</i>	238
<i>viscforcezdownmz</i>	284	<i>walltime</i>	210
<i>viscforcezmz</i>	284	<i>wcool</i>	207
<i>viscforcezupmz</i>	284	<i>wdamp</i>	205
<i>vmagfricmax</i>	220	<i>Wgrav</i>	234
<i>vmagfricmz</i>	278	<i>wheat</i>	206
<i>vmagfricrms</i>	220	<i>width_ff</i>	208
<i>vol</i>	215	<i>widthaa</i>	190, 201
<i>upcoalx2m</i>	245	<i>widthlnrho</i>	189, 198
<i>Vprimem</i>	238	<i>widthss</i>	199, 206
<i>Vprimepsim</i>	238	<i>widthuu</i>	188, 197
<i>upx2m</i>	245, 246	<i>win</i>	302, 304
<i>upxm</i>	245, 246	<i>WL2D</i>	221
<i>upxmax</i>	245, 246	<i>WL3D</i>	221
<i>upxmin</i>	245, 246	<i>WL3D2</i>	221
<i>urelpabsm</i>	245, 246	<i>wr1,wr2</i>	201
<i>W1ddotrms</i>	230	<i>write_slices</i>	28
<i>W1dotW1m</i>	231	<i>wsnaps.f90</i>	203
<i>W1max</i>	230	<i>wstate</i>	225
<i>W1rms</i>	230	<i>wstate_aver</i>	225
<i>W1xm</i>	231	<i>wstate_crit</i>	192
<i>W1xmz</i>	279	<i>wstate_tolerance</i>	192
<i>W1ym</i>	231	<i>xi</i>	262
<i>W1ymz</i>	279	<i>xiQ</i>	262
<i>W1zm</i>	231	<i>xp2m</i>	245, 246
<i>W1zmz</i>	280	<i>xpm</i>	245, 246
<i>W2ddotrms</i>	230	<i>xpmax</i>	245
<i>W2dotW2m</i>	231	<i>xpmin</i>	245
<i>W2max</i>	230	<i>xy</i>	203
<i>W2rms</i>	230	<i>xy2</i>	203
		<i>xy_spec</i>	204

<i>xyz0</i>	24, 26, 188, 195
<i>xyz_star</i>	26
<i>yH</i>	268
<i>yHm</i>	215, 249
<i>yHmax</i>	215, 249
<i>yHmin</i>	249
<i>yinyang.f90</i>	184
<i>yinyang_mpi.f90</i>	184
<i>yy</i>	299, 301
<i>z0aa</i>	190, 201
<i>z1</i>	193, 199
<i>z2</i>	193, 199
<i>zbot_slice</i>	27, 29, 203
<i>zeta</i>	208, 262
<i>zetaQ</i>	262
<i>zheat_buffer</i>	207
<i>zmphi</i>	269
<i>zref</i>	193, 198, 199, 208
<i>ztop_slice</i>	27, 29, 203

Index

This index contains options, names, definitions and commands. Files and variables have their own indexes.

- 'VAR10'* 46
- script-tests* 116
- .r* 46
- .run* 45
- .svn* 11
- /trimall* 46
- _kin* 55
- _mag* 55
- _saffman* 55
- 2N-scheme 170
- 6th-order derivatives 166

- pc_mkproctree 16* 157

- ab_spec=T* 55
- adapt-mkfile* 84
- adapt-mkfile* 20, 96
- Anaconda 47
- anelastic 70
- Auto-tests 114
- autoconf 95
- Autoconf 95, 96
- Autoconf/automake 95
- Averages 30, 31
- Azimuthal averages 30

- bandwidth 51
- Bash 4, 82
- bash* 45
- bc* 45
- Beowulf clusters 51
- Bidiagonal scheme 169
- bin/pc_run* 90
- Boundary conditions 40
- Bourne shell 4

- C viii, 1, 118
- Canopy 47
- Cdata 117
- cgs units 37, 188, 196
- Changes 136
- Check-in details 99
- Chiral MHD 72
- Coding standards 105, 132
- Combustion 68
- Comments 133
- Coordinate systems 60
- copy-proc-to-proc* 32
- Coriolis force 204
- Cosmic rays 71
- Cosmological expansion 76
- Courant number 37
- Covariant derivatives 176
- Cron 114
- crontab -e* 114
- CSH viii
- Csh 1, 4, 11, 82
- csh* 82
- Curvilinear coordinates 60, 176
- CVS 2
- CVS vii
- Cvs 10
- cvs-add-rundir* 32

- Daainit 158
- Data directory 13
- Data explorer 1, 42
- datafiles 26
- Density_init_pars 198
- Density_run_pars 205
- Diffusivity, time-dependent 153
- double precision 52, 154
- Download 2, 44, 82
- Download forbidden 82
- DX viii, 1, 4, 11, 33, 42

- Electromagnetism 73
- Emacs settings 135
- Entropy 63
- Entropy 14, 41, 63
- Entropy.f90 28
- Entropy_init_pars 199
- Entropy_run_pars 205
- Equation of state 66
- Error, diffusive 171
- Etatest 158

- f-array 108
- f90* 20, 21
- F95 viii, 1

-
- f95* 20
 - FAQ 82
 - FBCX1 158
 - FBCX2.2 158
 - ff* 46
 - ff.varname* 46
 - FFT 12
 - Fftpack 53
 - Filters 147
 - Flag 195, 202
 - Flux rings 138
 - Forcing_run_pars 37, 160, 161, 208
 - Fortran record 27, 31
 - Fortran record 27
 - fp-array 109
 - Frequently Asked Questions 82
 - ftp* 45
 - Fully qualified host name 17
 - G77 84
 - G95 52, 83
 - GDL 42, 43
 - Gfortran viii, 52
 - Ghost points 40
 - Ghost zones 40, 51
 - Git 2
 - Git 2
 - git* 3
 - Glibc 84
 - Gnu Data Language 42
 - GNU gcc viii
 - Gnuplot 14, 42
 - gnuplot* 45
 - Grav_init_pars 198
 - Grav_r 12
 - Grav_run_pars 158, 208
 - Gravitational Waves 78
 - Gravity 12
 - Gravity_simple 12
 - grep* viii, 103
 - grid, nonuniform 23
 - h-index 99
 - Hydro.f90 28
 - Hydro_init_pars 197
 - Hydro_run_pars 186, 196, 204
 - hyperdiffusivity 147
 - Hyperviscosity 147, 149–151, 166, 168
 - Icc 83
 - IDL viii, 1, 4, 7, 8, 11, 14, 28, 29, 31, 33, 42, 44, 45, 54, 56
 - IDL* 161
 - idl* 44
 - Ifc 83
 - Ifort 83, 84
 - ifort* 52
 - incompressible 70
 - Init_pars 35, 36, 185, 195
 - Initial conditions 137
 - InitialCondition module 113
 - Intel 83
 - Interlocked flux rings 138
 - Interstellar 13, 14
 - IO 117, 118
 - Io_mpiodist.f90 14
 - Ionization 66, 172
 - Ionization.f90 37
 - ireset_tstart=0, lread_oldsnap=T, lwrite-var_anyway=T* 186, 196
 - itorder_GW=2* 81
 - Janus 20, 21
 - Lambda effect 179
 - ldensity* 97
 - Linux 1, 83
 - locate mpif.h* 88
 - lspec_start=T* 54
 - Magnetic 106
 - Magnetic 106
 - Magnetic diffusivity, time-dep 153
 - Magnetic helicity vii
 - Magnetic.f90 28
 - Magnetic_init_pars 200
 - Magnetic_run_pars 158, 207
 - Make 1, 12, 21, 85
 - make* 6, 12, 18
 - make clean* 83
 - Makefile 6, 13, 20, 87
 - Makefile* 19
 - Manual iv, 98, 112
 - Mesh Reynolds number 37, 38, 72, 151, 172, 210
 - mesh, nonuniform 23
 - Message passing interface 50
 - Module viii
 - Module.h 83
 - Modulefile 5
 - Modules 12
 - Modules viii, 12

MPEG	28	Option ‘a’	40
Mpeg_encode	28	Option ‘a2’	40
MPI	vii, 1, 9, 12–14, 20, 50, 84, 96	Option ‘c1’	40, 142, 207
<i>mpif90 -show</i>	88	Option ‘c2’	40, 205
<i>mpif90 -showme</i>	88	Option ‘ce’	40
<i>mpirun</i>	13	Option ‘cT’	40
multiple special modules	112	Option ‘db’	40
namelist	112	Option ‘g’	41
Namelist	33	Option ‘hs’	41
Namelist	34, 35	Option ‘p’	40
Networks	42	Option ‘pot’	143
NEWDIR file	35	Option ‘pwd’	143
Newphysics	113	Option ‘s’	40
Noentropy	14, 41, 64	Option ‘she’	40
NOERASE file	93	<i>ou_spec=T</i>	55
Nogravity	12	Particles	73, 158
Noionization.f90	37	Particles_ads_init_pars	202
Nomodule.f90	83	Particles_ads_run_pars	209
Nonewphysics	113	Particles_chem_init_pars	202
nonuniform grid	23	Particles_chem_run_pars	209
Nospecial.f90	112	Particles_run_pars	209
<i>octave</i>	45	Particles_stalker_init_pars	202
Onsager	21	Particles_surf_init_pars	202
OpenDX	1	Particles_surf_run_pars	209
Option ‘-host-id’	17	pc.jobtransfer	35
Option ‘-max-level’	115	<i>pc_auto-test</i>	19, 20, 99, 114, 116
Option ‘-use-pc’	115	<i>pc_auto-test -help</i>	20, 114
Option ‘-use-pc_auto-test’	20	Pc.build	83
Option ‘-b’	20	<i>pc_build</i>	16, 18, 19
Option ‘-D’	115	<i>pc_build -cleanall</i>	115
Option ‘-f’	114	<i>pc_build -help</i>	19
Option ‘-fast’	21	<i>pc_get_quantity</i>	44, 46
Option ‘-fno-second-underscore’	83, 84	<i>pc_jobtransfer</i>	35
Option ‘-H’	17, 115	<i>pc_mkdir</i>	6
Option ‘-l’	114	<i>pc_newrun</i>	6
Option ‘-lmpi’	20	<i>pc_read_const</i>	46
Option ‘-m’	115	<i>pc_read_param</i>	47
Option ‘-mcmodel=large’	83	<i>pc_read_pvar</i>	47
Option ‘-mcmodel=medium’	83	<i>pc_read_t</i>	46
Option ‘-N’	115	<i>pc_read_var</i>	44, 46
Option ‘-nothreads’	84	<i>pc_read_var_raw</i>	44, 46
Option ‘-O2 -u’	20	<i>pc_read_xyaver</i>	46
Option ‘-O3’	20, 86	<i>pc_read_xzaver</i>	46
Option ‘-qextname’	83	<i>pc_read_zyaver</i>	46
Option ‘-T’	115	<i>pc_run</i>	16, 18, 19, 90
Option ‘-t’	115	<i>pc_run -help</i>	19
Option ‘-Uc’	115	<i>pc_setupsr</i>	11, 33, 52, 82, 83
Option ‘-Wa,-max-level=2’	115	<i>pc_sunup</i>	3
Option ‘/png’	28	<i>pc_sunup -val</i>	3
		<i>pc_tsnap</i>	27

-
- Pencil case 110
 - Pencil check 111
 - Pencil Code 95
 - Pencil consistency check 111
 - Pencil design 11
 - pencil-test* 20, 114, 115
 - pencil-test -help* 20, 114
 - pencil_check_small* 69
 - Pencils vii, 110
 - Perl viii, 1, 11
 - perldoc Pencil::ConfigFinder* 16
 - perldoc PENCIL::ConfigParser* 16
 - Planet solution 144
 - PNG 28
 - Point masses 75
 - Polytropic atmosphere 140
 - Potential-field boundary condition . . 143
 - power* 54
 - power_mag.dat* 55
 - power_saffman_ub.dat* 55
 - Power_spectrum_run_pars 209
 - pretend lnTT 64, 187, 197
 - Programming style 105, 132
 - Pscalar 113
 - Pscalar_init_pars 201
 - Pscalar_run_pars 208
 - pt output 108
 - Python viii, 9
 - Python 47

 - Radiative transfer 69, 173
 - Readline 44, 45
 - Regridding 154
 - Reheating 77
 - Remeshing 154
 - RERUN file 35
 - Resistive gauge 63
 - restart-new-dir ../32c* 157
 - Restarting 41, 157, 158
 - Restarting with different I/O 156
 - rlwrap* 44
 - Run directory 5
 - run.x* 31
 - Run_pars 27–29, 35, 157, 185, 202, 204
 - Runge-Kutta 170
 - Runge-Kutta time step 38
 - Runge-Kutta-Fehlberg time step . . . 39

 - Scale factor 76
 - Scripts 32
 - Setup 4, 43, 82

 - Shear 40
 - Shear_init_pars 202
 - Shear_run_pars 158, 209
 - Shock viscosity 38, 65
 - SI units 37, 188, 196
 - single point output 108
 - Sixth-order derivatives 166
 - SLD 168
 - slice files 30
 - Slice files 27
 - slope-limited diffusion 168
 - sound speed 39, 62, 79, 92, 117, 142, 195
 - Special module 111
 - start.csh* 41
 - Stdout 26
 - Stratification 139
 - structure* 56
 - Style 3, 105, 132
 - Sub 118
 - summarize-history* 35
 - svn 2, 3, 13, 125
 - Svn . . 2, 3, 10, 13, 32, 86, 125, 185, 195, 202
 - svn* 3
 - svn annotate src/*f90* v
 - svn up source.csh* 94
 - svn up source.sh* 94
 - svn update* 115
 - svn update -r #####* 3
 - Svn/git 2
 - Syscalls 83

 - tab* 105
 - Tab characters 132
 - tag_names* 46
 - Tcsh 4
 - Testfield method 77, 157
 - Time averages 31
 - Time step 38, 170
 - Toroidal averages 30
 - touch NEWDIR* 35
 - touch NOERASE* 186, 196
 - touch RELOAD* 202
 - type ld* 88

 - uname* 20
 - Underscore problem 84
 - Units 37, 188, 196
 - Unix 1
 - Upwinding 38, 103, 167
 - use* 117

Vector potential	63
Video files	27
Viscosity	38, 65
Viscosity, time-dependent	153
Viscosity_run_pars	158, 208
Whitespace	132
Xlf	83
Xmgrace	14

